NeetCode 150

Algorithmic Intuition Study Guide — Python → C++

Scope. Every solution in `neetcode150/`, in canonical category order.

For each problem: essence and bounds; multi-solution intuition with side-by-side Python and C++ kernels; a visual state trace; language mechanics; data-structure alternatives; interview traps; and three intuition-check questions with answers.

Coverage. 153 problems, 18 categories, 459 intuition questions.

Generated from the local neetcode150/ solution set • 2026-10-02

Contents

1. Arrays & Hashing **7**
0001 Two Sum . . . 7
0036 Valid Sudoku . . . 9
0049 Group Anagrams . . . 11
0128 Longest Consecutive Sequence . . . 12
0217 Contains Duplicate . . . 14
0238 Product of Array Except Self . . . 16
0242 Valid Anagram . . . 17
0271 Encode and Decode Strings . . . 19
0347 Top K Frequent Elements . . . 21

2. Two Pointers **23**
0011 Container With Most Water . . . 24
0015 3Sum . . . 26
0042 Trapping Rain Water . . . 28
0125 Valid Palindrome . . . 30
0167 Two Sum Ii . . . 32

3. Sliding Window **34**
0003 Longest Substring Without Repeating Characters . . . 35
0076 Minimum Window Substring . . . 37
0121 Best Time to Buy and Sell Stock . . . 39
0239 Sliding Window Maximum . . . 41
0424 Longest Repeating Character Replacement . . . 43
0567 Permutation in String . . . 44

4. Stack **46**
0020 Valid Parentheses . . . 47
0022 Generate Parentheses . . . 49
0084 Largest Rectangle in Histogram . . . 51
0150 Evaluate Reverse Polish Notation . . . 53
0155 Min Stack . . . 55
0739 Daily Temperatures . . . 56
0853 Car Fleet . . . 59

5. Binary Search **60**
0004 Median of Two Sorted Arrays . . . 61
0033 Search in Rotated Sorted Array . . . 63
0074 Search a 2D Matrix . . . 65
0153 Find Minimum in Rotated Sorted Array . . . 67
0704 Binary Search . . . 69
0875 Koko Eating Bananas . . . 71
0981 Time Based Key-Value Store . . . 73

6. Linked List **75**
0002 Add Two Numbers . . . 76
0019 Remove Nth Node From End of List . . . 78
0021 Merge Two Sorted Lists . . . 79
0023 Merge K Sorted Lists . . . 81
0025 Reverse Nodes in k-Group . . . 83
0138 Copy List with Random Pointer . . . 85
0141 Linked List Cycle . . . 87
0143 Reorder List . . . 89
0146 LRU Cache . . . 91
0206 Reverse Linked List . . . 93
0287 Find the Duplicate Number . . . 95

7. Trees **96**
0098 Validate Binary Search Tree . . . 97
0100 Same Tree . . . 99
0102 Binary Tree Level Order Traversal . . . 101
0104 Maximum Depth of Binary Tree . . . 103
0105 Construct Binary Tree from Preorder and Inorder . . . 105
0110 Balanced Binary Tree . . . 107
0124 Binary Tree Maximum Path Sum . . . 109
0199 Binary Tree Right Side View . . . 111
0226 Invert Binary Tree . . . 113
0230 Kth Smallest Element in a BST . . . 115
0235 Lowest Common Ancestor Of Bst . . . 117
0297 Serialize And Deserialize Binary Tree . . . 119
0543 Diameter Of Binary Tree . . . 121
0572 Subtree Of Another Tree . . . 123
1448 Count Good Nodes In Binary Tree . . . 124

8. Tries **126**
0208 Implement Trie (Prefix Tree) . . . 127
0211 Design Add and Search Words Data Structure . . . 129
0212 Word Search II . . . 131

9. Heap / Priority Queue **133**
0215 Kth Largest Element in an Array . . . 134
0295 Find Median from Data Stream . . . 136
0355 Design Twitter . . . 138
0621 Task Scheduler . . . 140
0703 Kth Largest Element in a Stream . . . 142
1046 Last Stone Weight . . . 144

10. Backtracking **146**
0017 Letter Combinations of a Phone Number . . . 147
0039 Combination Sum . . . 149

0040 Combination Sum II . . . 151
0046 Permutations . . . 152
0051 N Queens . . . 154
0078 Subsets . . . 156
0079 Word Search . . . 158
0090 Subsets II . . . 160
0131 Palindrome Partitioning . . . 162
0395 Longest Substring with At Least K Repeating Chars . . . 164

11. Graphs **166**
0127 Word Ladder . . . 167
0130 Surrounded Regions . . . 170
0133 Clone Graph . . . 172
0200 Number of Islands . . . 174
0207 Course Schedule . . . 176
0210 Course Schedule II . . . 178
0261 Graph Valid Tree . . . 180
0286 Walls And Gates . . . 182
0323 Number Of Connected Components . . . 184
0417 Pacific Atlantic Water Flow . . . 185
0684 Redundant Connection . . . 187
0695 Max Area Of Island . . . 189
0994 Rotting Oranges . . . 191

12. Advanced Graphs **193**
0269 Alien Dictionary . . . 194
0332 Reconstruct Itinerary . . . 196
0743 Network Delay Time . . . 198
0778 Swim in Rising Water . . . 200
0787 Cheapest Flights Within K Stops . . . 202
1584 Min Cost To Connect All Points . . . 204

13. 1-D Dynamic Programming **206**
0005 Longest Palindromic Substring . . . 207
0070 Climbing Stairs . . . 209
0091 Decode Ways . . . 210
0139 Word Break . . . 212
0152 Maximum Product Subarray . . . 215
0198 House Robber . . . 217
0213 House Robber II . . . 218
0300 Longest Increasing Subsequence . . . 220
0322 Coin Change . . . 222
0416 Partition Equal Subset Sum . . . 224
0647 Palindromic Substrings . . . 226
0746 Min Cost Climbing Stairs . . . 227

1143 Longest Common Subsequence . . . 229

14. 2-D Dynamic Programming **231**
0010 Regular Expression Matching . . . 232
0062 Unique Paths . . . 234
0063 Unique Paths II . . . 236
0064 Minimum Path Sum . . . 237
0072 Edit Distance . . . 239
0097 Interleaving String . . . 241
0115 Distinct Subsequences . . . 243
0309 Best Time To Buy And Sell Stock With Cooldown . . . 244
0312 Burst Balloons . . . 246
0329 Longest Increasing Path In A Matrix . . . 248
0494 Target Sum . . . 249
0518 Coin Change Ii . . . 251

15. Greedy **253**
0045 Jump Game II . . . 254
0053 Maximum Subarray . . . 256
0055 Jump Game . . . 258
0134 Gas Station . . . 260
0678 Valid Parenthesis String . . . 262
0763 Partition Labels . . . 264
0846 Hand of Straights . . . 267
1899 Merge Triplets to Form Target Triplet . . . 269

16. Intervals **271**
0056 Merge Intervals . . . 272
0057 Insert Interval . . . 274
0252 Meeting Rooms . . . 276
0253 Meeting Rooms II . . . 278
0435 Non-overlapping Intervals . . . 280
1851 Minimum Interval to Include Each Query . . . 282

17. Math & Geometry **284**
0043 Multiply Strings . . . 285
0048 Rotate Image . . . 287
0050 Pow(x, n) . . . 289
0054 Spiral Matrix . . . 291
0066 Plus One . . . 293
0073 Set Matrix Zeroes . . . 295
0201 Bitwise AND of Numbers Range . . . 297
0202 Happy Number . . . 299
2013 Detect Squares . . . 301

18. Bit Manipulation **303**

0007 Reverse Integer 304
0136 Single Number 306
0190 Reverse Bits 307
0191 Number of 1 Bits 310
0268 Missing Number 311
0338 Counting Bits 313
0371 Sum Of Two Integers 315

Appendix: Coverage Checklist **316**

1. Arrays & Hashing

This category is about trading memory for time. The recurring move is to ask the array one question per element – "have I seen the thing that completes this?" – and answer it with a hash table instead of a rescan. Every approach below is that same idea wearing different clothes: frequency tables for multiplicity, complement maps for pairing, prefix/suffix sweeps when you need a value from both sides, and bucket ordering when the values themselves carry the structure. The Python-C++ story is uniform too: a flat open-addressed `dict` versus node-per-entry `std::unordered_map`, which is why the Python version is often the denser one for small-int keys even though it is slower per operation.

#	Problem	Level	One-line intent
0001	**Two Sum**	Easy	Complement lookup collapses $O(n^2)$ pairing into a single pass.
0036	**Valid Sudoku**	Medium	Three sets of masks (row, column, box) check each cell once.
0049	**Group Anagrams**	Medium	Anagrams share a multiset signature, so signature is the key.
0128	**Longest Consecutive Sequence**	Medium	Only start a walk where value-1 is absent, making it $O(n)$.
0217	**Contains Duplicate**	Easy	Insert into a set; the size tells you a repeat happened.
0238	**Product of Array Except Self**	Medium	Prefix pass left, suffix pass right – no division allowed.
0242	**Valid Anagram**	Easy	26 counters: equal multisets iff all 26 slots return to zero.
0271	**Encode and Decode Strings**	Medium	Length-prefix framing makes the boundary of each string unambiguous.
0347	**Top K Frequent Elements**	Medium	Count frequencies, then take the top k of the counter.

0001 Two Sum Easy

ESSENCE & BOUNDS

Return the indices of the two distinct elements whose values sum to `target`. Exactly one solution is guaranteed and no element may be used twice. 2 <= nums.length <= 1e4; -1e9 <= nums[i], target <= 1e9. Duplicates are allowed, so [3,3] with target 6 is a legal input. Overflow is impossible in Python but is real in C++ if you accumulate rather than subtract. The single boundary trap is probe-before-insert.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

One-pass hash map (BEST) Walk the array once. Before inserting `num`, check whether its complement `target - num` is already in the map.

Python (intuition kernel)

```python
seen = {}
for i, x in enumerate(nums):
 if target - x in seen:
 return [seen[target - x], i]
 seen[x] = i

```

C++ (intuition kernel)

```cpp
unordered_map<int,int> seen;
for (int i = 0; i < n; ++i) {
 int need = target - nums[i];
 if (seen.count(need))
 return {seen[need], i};
 seen[nums[i]] = i;
}

```

Time O(n) **Space** O(n)

one $O(1)$ hash probe per element, no rescans; hence the stated bounds.

Sort + two pointers (no hash map) Pair each value with its original index, sort, then let `l` and `r` meet in the middle adjusting by the sum sign.

Python (intuition kernel)

```python
pairs = sorted((v, i) for i, v in enumerate(nums))
l, r = 0, len(pairs) - 1
while l < r:
 s = pairs[l][0] + pairs[r][0]
 if s == target:  return [pairs[l][1], pairs[r][1]]
 l, r = (l + 1, r) if s < target else (l, r - 1)

```

C++ (intuition kernel)

```cpp
sort(pairs.begin(), pairs.end());
int l = 0, r = pairs.size() - 1;
while (l < r) {
 int s = pairs[l].first + pairs[r].first;
 if (s == target) return {pairs[l].second,
 pairs[r].second};
 s < target ?  ++l :  --r;
}

```

Time O(n log n) **Space** O(n)

one $O(1)$ hash probe per element, no rescans; hence the stated bounds.

VISUAL STATE TRACE

```
nums :  [2] [7] [11] [15]
i=0 i=1 i=2 i=3
need9 need2 need-2 need-11
i=0 x=2 need=9 in seen?  NO -> seen={2:0}
i=1 x=7 need=2 in seen?  YES -> [0,1] DONE
i=2 ...  (never reached)
invariant:  before step i, seen = {value:index for index < i}
```

LANGUAGE MECHANICS: PYTHON VS C++

CPython boxes every int: `nums[i]` is a pointer chase to a separately allocated PyLong, and `target - num` allocates a brand-new PyLong per iteration, so a one-pass scan performs n heap allocations for the subtrahends alone. C++ keeps the ints inline in a contiguous `vector<int>`, so the subtraction happens in a register with no traffic at all. The map itself inverts the usual expectation: Python's `dict` is one flat open-addressed table of PyObject* slots, so a 1e4-entry int map costs roughly 400 KB densely packed, while `std::unordered_map` allocates a node per entry plus a bucket array, costing more memory but keeping each bucket's chain cache-friendly for iteration.

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `std::map<int,int>` is the wrong tool: an RB-tree makes every lookup $O(\log n)$ for no benefit here, since we never need the keys ordered.
- `__gnu_pbds::gp_hash_table` with a splitmix64 custom hasher defeats the standard anti-test payload that walks `std::unordered_map` with keys congruent modulo the bucket count.
- For very large n, a sorted `vector<pair<int,int>>` plus two pointers beats a hash map on memory and cache, at $O(n \log n)$ time.

- `dict.setdefault` and `defaultdict` only hide the miss case; the explicit `in` probe is fastest and reads closest to the C++ version.
- `enumerate` beats `for i in range(len(nums))` because it avoids the second index load.
- `Counter` is dead weight for a single partner query – it maintains 26-fold more state than you read.

INTERVIEW TRAPS & SANITY CHECKS

Writing `seen[x] = i` before the membership probe. With nums = [3, 3] and target = 6 that returns [0, 0], reusing one element twice. Probe first, insert second.

Sanity check. At the top of iteration i, `seen` maps every value at an index strictly below i to its index, and nothing else. If that holds, a hit means the partner is a genuinely earlier, distinct element.

INTUITION CHECK — 3 QUESTIONS

Q1. Why must the probe happen before the insert?

Answer. Inserting first lets a value satisfy its own complement when x + x == target, which reuses one element. The problem forbids that, so the current element must not be visible to its own query.

Q2. What input kills an identity-hashed `std::unordered_map` here, and how do you stop it?

Answer. Keys chosen to be congruent modulo the bucket count, e.g. multiples of the prime bucket size, all land in one chain and turn the lookup into $O(n)$ per probe. A splitmix64 custom hasher, or `gp_hash_table` with open addressing, restores $O(1)$ expected.

Q3. The answer is guaranteed unique. Why may you return on the first hit?

Answer. If a second valid pair existed, the scan would have found it at an earlier index, contradicting the stated uniqueness. So the first hit is provably the expected pair and no backtracking is needed.

0036 Valid Sudoku Medium

ESSENCE & BOUNDS

Decide whether a partially filled 9x9 Sudoku board obeys Sudoku rules: no digit repeats in any row, column, or 3x3 box. Empty cells are always permitted. The board is always exactly 9x9, so the alphabet is fixed at 27 nine-bit masks and 81 cells. '.' means empty and must be skipped before any bit test. Digits arrive as characters, so a 0-9 digit test beats any bit shift unless you subtract '1' first.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Bitmask per row / column / box (MEMORY-OPTIMAL) Time Complexity : $O(81)$ Space Complexity : $O(27)$ ints ≈ 108 B. Box index = (r//3)*3 + (c//3). WHY bitmasks: a hash-set version needs 27 containers with hashing + probing overhead. Same O() class, far smaller constants. ——————————————————————————

Python (intuition kernel)

```python
rows, cols, boxes = [0]*9, [0]*9, [0]*9
for r in range(9):
 for c in range(9):
 ch = board[r][c]
 if ch == '.': continue
 d, bit = int(ch), 1 << (int(ch) - 1)
 b = (r // 3) * 3 + c // 3
 if (rows[r] | cols[c] | boxes[b]) & bit: return
False
 rows[r] |= bit; cols[c] |= bit; boxes[b] |= bit

```

C++ (intuition kernel)

```cpp
uint16_t rows[9] = {}, cols[9] = {}, box[9] = {};
for (int r = 0; r < 9; ++r)
 for (int c = 0; c < 9; ++c) {
 char ch = board[r][c];
 if (ch == '.') continue;
 uint16_t bit = 1 << (ch - '1');
 int b = (r / 3) * 3 + c / 3;
 if ((rows[r] | cols[c] | box[b]) & bit) return
false;
 rows[r] |= bit; cols[c] |= bit; box[b] |= bit;
 }

```

Time O(81) **Space** O(27) ints $\approx$ 108 B.

each state is solved once; the rest are cached lookups; hence the stated bounds.

Set of (owner, digit) tuples (simplest to write) Time Complexity : $O(81)$ Space Complexity : $O(81)$. Pedagogical contrast: hash sets trade memory for simplicity. ==========

Python (intuition kernel)

```python
seen = set()
for r in range(9):
 for c in range(9):
 ch = board[r][c]
 if ch == '.':  continue
 if (r, ch) in seen or (c, ch) in seen \{}
 or ((r // 3) * 3 + c // 3, ch) in seen:
 return False
 seen |= {(r, ch), (c, ch), ((r // 3) * 3 + c // 3, ch)}
return True

```

C++ (intuition kernel)

```cpp
unordered_set<int> seen;
for (int r = 0; r < 9; ++r)
 for (int c = 0; c < 9; ++c) {
 char ch = board[r][c];
 if (ch == '.')  continue;
 int d = ch - '0', b = (r / 3) * 3 + c / 3;
 if (seen.count(r * 10 + d) || seen.count(100 + c * 10 + d)
 || seen.count(200 + b * 10 + d)) return false;
 seen.insert(r * 10 + d); seen.insert(100 + c * 10 + d);
 seen.insert(200 + b * 10 + d);
 }

```

Time O(81) **Space** O(81).

each element is probed once; pairs cancel or are recorded; hence the stated bounds.

VISUAL STATE TRACE

```
0 1 2 | 3 4 5 | 6 7 8 box = (r//3)*3 + (c//3)
0 .  5 3 .  . 7 .  .  .
0 .  6 .  .  1 9 .  .  .
0 9 8 .  .  .  . 6 .  .
------+-----+-----
3 4 .  .  .  .  .  . 8 .
bit test :  mask[box] » d & 1
r0c0 d=5 :  boxes[0] bit 5 clear -> set
r0c6 d=7 :  boxes[2] bit 7 clear -> set
r0c0 d=5 again -> HIT -> invalid
invariant:  masks hold exactly the digits already placed
```

LANGUAGE MECHANICS: PYTHON VS C++

Nine-bit masks fit in a machine word, so C++ does one shift and one AND per check with the mask living in a register or a 27-element stack array that stays in L1. Python cannot use the `int` as a bit carrier cheaply – a small `int` is still a heap object – so `mask | (1 « d)` allocates, and the masks cost 27 boxed integers with pointer chasing at every probe. In exchange Python gets free arbitrary precision, so a 27-bit or 26-bit mask costs nothing extra while the C++ author must remember that `1 « d` with d as the digit 9 needs `-1` to stay inside a byte if they use `uint8_t`.

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `std::bitset<9>` or a plain `uint16_t` per row/col/box gives compile-time masking; a `bool grid[9][9]` is simplest and still $O(1)$ per cell.
- `std::array<std::array<uint16_t,9>,9>` keeps all 243 bits in one contiguous block – 27 cache lines – instead of three separate containers.
- A `std::unordered_set<char>` per row works but allocates one node per cell; masks allocate nothing.
- Three `defaultdict(set)` rows/cols/boxes is the readable version but allocates a set object per line – 27 set objects plus 81 inserts.
- Bit masks with `1 « (int(ch) - 1)` are fastest in Python and mirror C++ exactly.
- `str.isdigit()` is locale-free here but slower than comparing against `'1' <= ch <= '9'`.

INTERVIEW TRAPS & SANITY CHECKS

Testing the digit against the mask before skipping empties, so `'.'` gets hashed into a bit. The second classic bug is computing the box index as `r * 3 + c` instead of `(r // 3) * 3 + (c // 3)`.

Sanity check. After processing cell k, `rows`, `cols`, `boxes` contain exactly the digits already committed in the first k cells. A duplicate is therefore exactly a set bit that was already present.

INTUITION CHECK — 3 QUESTIONS

Q1. Why 9 bits and not a set per row?

Answer. A mask turns "has digit d already appeared here?" into one shift and one AND on a single machine word, with zero allocation. A set needs a pointer chase and a node allocation per insert.

Q2. What is the box index for cell (4, 7) and why?

Answer. $(4 // 3) * 3 + (7 // 3) = 1 * 3 + 2 = 5$. Integer division on both coordinates folds the 9x9 grid into 3x3 blocks; row-major multiplication would give 4*3+7 = 19, which is out of range.

Q3. The board is fixed at 9x9 – does that make the problem trivial?

Answer. It removes input parsing, not the design question. The real content is choosing a per-line data structure that makes the duplicate test $O(1)$, and noticing that a fixed alphabet invites bit packing rather than hashing.

0049 Group Anagrams Medium

ESSENCE & BOUNDS

Partition a list of strings so that all strings that are permutations of each other land in the same group, and return the groups in any order. 1 <= strs.length <= 1e4; 1 <= strs[i].length <= 100, all lowercase. Worst-case total characters is 1e6, so a sort-based key is $O(L \log L)$ while a 26-slot count signature is $O(L)$. Duplicate strings are legal inputs and simply collide into the same group.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Count-tuple key (BEST for long words) Time Complexity : $O(n-k)$ Space Complexity : $O(n-k)$ (n words × k letters each) WHY tuple: Python tuples are IMMUTABLE and therefore HASHABLE — they can be dict keys. Lists are mutable → unhashable → rejected. ——————————————————————————

Python (intuition kernel)

```python
groups: dict[tuple, list] = {}
for w in words:
 cnt = [0] * 26
 for ch in w:
 cnt[ord(ch) - 97] += 1
 groups.setdefault(tuple(cnt), []).append(w)
return list(groups.values())

```

C++ (intuition kernel)

```cpp
unordered_map<string, vector<string>> g;
for (auto& w : words) {
 array<char,26> cnt{};
 for (char ch : w) ++cnt[ch - 'a'];
 g[string(cnt.data(), 26)].push_back(w);
}
vector<vector<string>> out;
for (auto& kv : g) out.push_back(kv.second);

```

Time O(n-k) **Space** O(n-k)

Follows directly from the traversal count: $O(n-k)$ time, $O(n-k)$ space.

Sorted-string key Time Complexity : $O(n-klogk)$ Space Complexity : $O(n-k)$ Simpler to write; slower on long words (k log k per word). ==========

Python (intuition kernel)

```python
groups: dict[str, list] = {}
for w in words:
 key = ''.join(sorted(w))
 groups.setdefault(key, []).append(w)
return list(groups.values())

```

C++ (intuition kernel)

```cpp
map<string, vector<string>> g;
for (auto& w : words) {
 string key = w;
 sort(key.begin(), key.end());
 g[key].push_back(w);
}
vector<vector<string>> out;
for (auto& kv : g) out.push_back(kv.second);

```

Time O(n-k log k) **Space** O(n-k)

the sort dominates; the post-sort scan is linear; hence the stated bounds.

VISUAL STATE TRACE

```
str sorted-key count-key
"eat" "aet" "a#e#t##..."
"tea" "aet" "a#e#t##..."
"tan" "ant" "a#n#t##..."
"ate" "aet" "a#e#t##..."
"nat" "ant" "a#n#t##..."
dict = {"aet":  [eat,tea,ate],
"ant":  [tan,nat]}
cost:  count-key 26 slots vs sort's O(L log L)
```

LANGUAGE MECHANICS: PYTHON VS C++

The 26-slot signature is the whole story in both languages. C++ builds it as a `std::array<char,26>` on the stack, increments 26 bytes of L1-resident memory, then needs a 26-byte array as a hash key – which means `std::unordered_map` must hash and memcmp 26 bytes per lookup, all still in L1. Python builds the same tuple but as 26 heap-allocated small ints; because Python caches ints -5..256 the increments return interned objects with no allocation, yet the tuple itself is a fresh 26-slot object each time, so the dictionary stores one pointer to it. Python therefore trades roughly one 26-element tuple allocation per string for a much lighter lookup.

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `array<char,26>` + a custom hasher beats `std::string` keys made from sorted copies: no $O(L \log L)$ sort and no heap string.
- `std::map<array<char,26>, vector<string>>` keeps groups ordered by signature, at $O(L \log G)$ for G groups.
- Packing the 26 counts into two 64-bit words (5 bits per letter covers length <= 31) makes the key trivially hashable.

- `"".join(sorted(s))` is the one-liner but costs $O(L \log L)$ and a fresh string per input.
- `tuple(count)` where `count = [0]*26` and you bump `count[ord(c)-97]` is the fast signature.
- `defaultdict(list)` collects the groups without a second pass; `itertools.groupby` on a sorted list is an alternative that pays one sort.

INTERVIEW TRAPS & SANITY CHECKS

Using the sorted string as the key and then claiming $O(n)$ time. It is $O(L \log L)$; with 1e4 strings of length 100 that sort dominates everything else in the solution.

Sanity check. Two strings share a dictionary key exactly when they are permutations of each other – which is true if and only if they contain the same multiset of characters.

INTUITION CHECK — 3 QUESTIONS

Q1. What must the hash key capture, and what is the cheapest sufficient encoding?

Answer. Only the multiset of characters. A 26-slot count vector encodes it in $O(L)$; sorted order encodes it in $O(L \log L)$; a bitmask alone would be wrong because it loses multiplicity.

Q2. Why can a plain set of characters not be the key?

Answer. {a, b} cannot distinguish "ab" from "aabb". Duplicates matter, so the encoding must be a multiset – counts, not membership.

Q3. What is the total work bound in terms of L, the sum of all lengths?

Answer. $O(L)$ for the count signature plus $O(L)$ hashing, so linear in the input size. The sorted-key version is $O(L \log L)$, which is why the counting version is the better interview answer.

0128 Longest Consecutive Sequence Medium

ESSENCE & BOUNDS

Return the length of the longest run of consecutive integers appearing anywhere in an unsorted array, with $O(n)$ time. 0 <= nums.length <= 1e5, so an $O(n^2)$ scan times out. Values span -1e9..1e9, so the run is not a contiguous index range. Duplicates must not inflate the answer, and the empty array returns 0.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Hash set + head-of-run growth (BEST) Time Complexity : $O(n)$ Space Complexity : $O(n)$ WHY the "x-1 in set?" guard matters: without it, [1,2,3,4] would trigger counting from 1,2,3,4 → $O(n^2)$. With it, only 1 counts. Python set = open addressing + linear probing: h(k,i) = (hash(k)+i) mod m → one contiguous table, great cache locality.

Python (intuition kernel)

```python
nums = set(nums)
best = 0
for x in nums:  # only run-minima pass the gate
 if x - 1 in nums:  continue
 y = x
 while y + 1 in nums:  y += 1
 best = max(best, y - x + 1)
return best

```

C++ (intuition kernel)

```cpp
unordered_set<int> s(nums.begin(), nums.end());
int best = 0;
for (int x :  s) {
 if (s.count(x - 1)) continue; // not a run start
 int y = x;
 while (s.count(y + 1)) ++y;
 best = max(best, y - x + 1);
}

```

Time O(n) **Space** O(n)

one $O(1)$ hash probe per element, no rescans; hence the stated bounds.

Sort + linear scan Time Complexity : $O(nlogn)$ Space Complexity : $O(1)$ Wins when memory is tight; loses the $O(n)$ requirement. ==========

Python (intuition kernel)

```python
nums = sorted(set(nums))
best = run = 1
for i in range(1, len(nums)):
 run = run + 1 if nums[i] == nums[i-1] + 1 else 1
 best = max(best, run)
return best if nums else 0

```

C++ (intuition kernel)

```cpp
sort(nums.begin(), nums.end());
int best = 0, run = 0;
for (int i = 0; i < n; ++i) {
 if (i > 0 && nums[i] == nums[i-1] + 1) ++run;
 else run = 1;
 best = max(best, run);
}

```

Time O(n log n) **Space** O(1)

the sort dominates; the post-sort scan is linear; hence the stated bounds.

VISUAL STATE TRACE

```
nums = [100,4,200,1,3,2] (longest run = 4)
step 1:  put every value in a set
step 2:  only walk from values whose predecessor is MISSING
x=100 99 in set?  NO -> walk 100 (len 1)
x=4 3 in set?  YES -> skip (do NOT walk here)
x=200 199 in set?  NO -> walk 200 (len 1)
x=1 0 in set?  NO -> walk 1->2->3->4 (len 4) BEST
x=3 2 in set?  YES -> skip
x=2 1 in set?  YES -> skip
every element is touched O(1) times -> O(n) total
```

LANGUAGE MECHANICS: PYTHON VS C++

The fix is structural: each run is walked exactly once because only its minimum is a start. In C++ that means each successful probe is a `find` into an open-addressed `unordered_set<int>` – one cache miss on a miss, two on a hit – and the walk is a tight dependent chain of loads. Python pays the same probe count, but each `x - 1 in s` allocates a new PyLong for the probe key, so a 1e5-element scan performs hundreds of thousands of small allocations that C++ never makes. Set membership in CPython short-circuits on identity first, which rescues the repeat probes on already-interned values but not on large negatives.

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `std::unordered_set<int>` with a reserve() call sized to n – without it, rehashing dominates for adversarial inputs.
- Sorting then a single scan is $O(n \log n)$ time but $O(1)$ extra space beyond the sort and avoids hashing entirely.
- A `std::bitset` over the value range is only viable when the range is small; here -1e9..1e9 rules it out.

- `set(nums)` once, then a second pass over the set – iterating the set rather than the list avoids re-checking duplicates.
- `s.discard/s.remove` while walking shrinks the working set but changes the start-detection logic; the membership gate is cleaner.
- A sorted-list scan is slower in Python than the set version because `bisect` is $O(\log n)$ per step and you pay Python-level loop overhead.

INTERVIEW TRAPS & SANITY CHECKS

Walking from every element. That is $O(n \cdot extrunLength)$ and blows up on inputs like [1..100000]. The gate `if x - 1 in s: continue` is what makes it linear.

Sanity check. A walk is started at x only when x-1 is absent, so every walk begins at the minimum of its run; consequently no two walks ever cover the same element.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is checking only run-minima enough?

Answer. Every consecutive run has exactly one minimum, and only that minimum lacks a predecessor in the set. Walking from the minimum visits every element of the run, so all other starting points are redundant work.

Q2. What is the total probe count, and why is it linear rather than quadratic?

Answer. At most two probes per element – one failed predecessor test and one successful membership test per walk step – because the runs partition the set. Hence $O(n)$ expected.

Q3. What breaks if you drop the set and sort instead?

Answer. Nothing breaks, the bound just degrades to $O(n \log n)$ time and $O(n)$ or $O(1)$ extra space depending on sort. That is still accepted; it is simply not the tightest answer.

0217 Contains Duplicate Easy

ESSENCE & BOUNDS

Return true if any integer appears at least twice in the array, false otherwise. 1 <= nums.length <= 2e5; -2e9 <= nums[i] < 2e9, so sorting changes the array's index order but not membership. A single element can never be a duplicate, and negatives are ordinary keys.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Hash set (BEST) Time Complexity : $O(n)$ Space Complexity : $O(n)$ Python `set` = dict without values (open addressing + linear probing, h(k,i) = (hash(k)+i) mod m) — one contiguous table, tiny memory, great cache locality.

Python (intuition kernel)

```python
seen = set()
for x in nums:
 if x in seen:  return True
 seen.add(x)
return False

```

C++ (intuition kernel)

```cpp
unordered_set<int> seen;
seen.reserve(nums.size());
for (int x :  nums) {
 if (!seen.insert(x).second) return true;
}
return false;

```

Time O(n) **Space** O(n)

one $O(1)$ hash probe per element, no rescans; hence the stated bounds.

Sort + adjacent scan (zero extra memory) Time Complexity : $O(nlogn)$ Space Complexity : $O(1)$ Wins when memory is critical; loses the $O(n)$ time bound. ——————————————————— REAL LIFE • Login systems: is this username taken? (uniqueness check) • Deduplicating logs, inventory SKUs, search-engine crawlers. ==========

Python (intuition kernel)

```python
nums.sort()
for i in range(1, len(nums)):
 if nums[i] == nums[i-1]:  return True
return False

```

C++ (intuition kernel)

```cpp
sort(nums.begin(), nums.end());
for (int i = 1; i < n; ++i)
 if (nums[i] == nums[i-1]) return true;
return false;

```

Time O(n log n) **Space** O(1)

the sort dominates; the post-sort scan is linear; hence the stated bounds.

VISUAL STATE TRACE

```
nums = [1, 2, 3, 1]
s = {}
1 -> not in s -> s = {1}
2 -> not in s -> s = {1,2}
3 -> not in s -> s = {1,2,3}
1 -> FOUND -> True
invariant:  len(s) == count of distinct values seen so far
```

LANGUAGE MECHANICS: PYTHON VS C++

Both languages do the same amount of work – n probes – so the difference is constant factor. C++ inserts into `std::unordered_set<int>` where each insert is a bucket index, a node allocation and a pointer write; with `reserve(n)` the bucket array is one contiguous block and the whole structure is n mallocs plus n probes. CPython's `set` needs no per-element allocation at all for the table itself and its probe uses the value's own hash, but the set stores a PyObject* per slot, so it costs 16 bytes per slot on 64-bit versus C++'s 8-byte node plus 8-byte key plus 8-byte next pointer spread across separate allocations – which is why Python's set, despite the slower interpreter loop, ends up competitive in memory.

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `std::unordered_set` with `reserve(n)` and `max_load_factor(0.7)` – otherwise repeated rehashing triples the cost.
- Sorting then adjacent comparison is $O(n \log n)$ time, $O(1)$ extra space, and wins decisively on memory-constrained judges.
- `gp_hash_table` with open addressing removes the per-node malloc entirely – one flat array, n probes.

- `len(set(nums)) < len(nums)` is the entire solution; no explicit loop needed.
- `return len(nums) != len(set(nums))` is the idiomatic single-expression form interviewers want.
- For a pre-sorted array, comparing neighbours is $O(1)$ space and beats building a set.

INTERVIEW TRAPS & SANITY CHECKS

Comparing each element against the previous one. That detects only ADJACENT duplicates, so $[1, 2, 1]$ passes. Either sort first or use a set.

Sanity check. After processing k elements, `len(seen)` equals the number of distinct values among the first k. So the answer is determined the moment `len(seen)` fails to grow.

INTUITION CHECK — 3 QUESTIONS

Q1. Why does the set version beat the sort version here even though sort is cache friendly?

Answer. Expected time: $O(n)$ versus $O(n \log n)$. The sort's wins are space ($O(1)$ versus $O(n)$) and constant factor predictability, which is the right trade only when memory is the binding constraint.

Q2. What is the space cost, and can it be reduced?

Answer. $O(n)$ for the set. Sorting in place and scanning neighbours gets it to $O(1)$ auxiliary space (or $O(\log n)$ stack for introsort), at $O(n \log n)$ time.

Q3. How would you detect a duplicate in a 10 GB stream you cannot hold?

Answer. Bloom filter (probabilistic, no false negatives), or sort-external-merge and scan for adjacent equality (exact, needs disk). The set approach fails because its space is linear in the stream.

0238 Product of Array Except Self Medium

ESSENCE & BOUNDS

Return an array where the element at index i is the product of every input element except nums[i], without using division and in $O(n)$ time. 1 <= nums.length <= 2e5, so division is banned by the problem statement (and would also fail on zeros). Elements may be 0 or negative, and with exactly one zero the whole output is zero except at that index.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Prefix $\times$ suffix in the output array (BEST) Time Complexity : $O(n)$ Space Complexity : $O(1)$ extra. ————————————————————

Python (intuition kernel)

```python
out = [1] * n
pre = 1
for i in range(n):  # pass 1: prefix products
 out[i] = pre
 pre *= nums[i]
suf = 1
for i in range(n-1, -1, -1):  # pass 2: fold in suffix
 out[i] *= suf
 suf *= nums[i]

```

C++ (intuition kernel)

```cpp
vector<long long> out(n, 1);
long long pre = 1;
for (int i = 0; i < n; ++i) { out[i] = pre; pre *= nums[i]; }
long long suf = 1;
for (int i = n - 1; i >= 0; --i) { out[i] *= suf; suf *= nums[i]; }

```

Time O(n) **Space** O(1) extra.

two linear passes; the suffix is carried in a scalar; hence the stated bounds.

Divide-by-self (CONTRAST, rejected by the problem) answer[i] = total / nums[i]. $O(n)/O(1)$ but: division is banned, and a zero input CRASHES the division (or corrupts every other answer when there are two+ zeros). Listed only to explain why the prefix/suffix trick exists. ———————————————————— NUMPY NOTE : for pure computation numpy's np.prod / cumprod would vectorize this in C speed — but LeetCode forbids external libs. Conceptually, np.cumprod IS the prefix product used here. ==========

Python (intuition kernel)

```python
out = [1] * n
pre = 1
for i in range(n):
 out[i] = pre
 pre *= nums[i]
for i in range(n-2, -1, -1):
 out[i] *= out[i+1] * nums[i+1]

```

C++ (intuition kernel)

```cpp
vector<long long> out(n, 1);
long long pre = 1;
for (int i = 0; i < n; ++i) { out[i] = pre; pre *= nums[i]; }
// out[i] already holds nums[0..i-1]; multiply the running tail
for (int i = n - 2; i >= 0; --i)
 out[i] *= out[i + 1] * nums[i + 1];

```

Time ?	Space ?

the recursion splits the work; the bound is the sum over levels; hence the stated bounds.

VISUAL STATE TRACE

```
nums = [1,2,3,4] out = [24,12,8,6]
pass 1 (left):  out[i] = product of nums[0..i-1]
prefix=1 -> out[0]=1 ; prefix*=1 -> 1
out[1]=1 ; prefix*=2 -> 2
out[2]=2 ; prefix*=3 -> 6
out[3]=6 ; prefix*=4 -> 24
pass 2 (right):  multiply the suffix
suffix=1 -> out[3]=24*1=24 ; suffix*=4 -> 4
out[2]= 6*4=24 ; suffix*=3 -> 12
out[1]= 1*12=12 ; suffix*=2 -> 24
out[0]= 1*24=24
with a zero [1,0,3]:  out=[0,3,0] division would divide by zero
```

LANGUAGE MECHANICS: PYTHON VS C++

The two passes are pure arithmetic on already-loaded values, so C++ keeps the running product in a register and the output vector is one contiguous, write-once buffer that the prefetcher streams perfectly. Python allocates a new int for every product it writes, and for values beyond 2^63 CPython promotes to arbitrary precision: a single 32-digit product means a 100-byte object plus a multiplication routine instead of one `imul`. Python also re-reads the output list element by element in the reverse pass, so the second sweep walks memory backwards, which is mildly prefetcher-hostile on top of the object chasing.

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- One vector doubles as the prefix output array – $O(1)$ extra space beyond the required output.
- Division plus a total-product scan is $O(n)$ and shorter, but is forbidden here and divides by zero whenever the input contains 0.
- `long long` is mandatory: with values up to 1e4 and n up to 2e5 the intermediate products overflow 32 bits instantly.
- Python integers never overflow, so the whole overflow argument disappears – but arbitrary-precision multiply is not free.
- `math.prod` computes the total but is $O(n)$ per call; slicing products is $O(n^2)$ and fails.
- Filling with 1s so the identity element needs no special case at index 0 or n-1 keeps both passes uniform.

INTERVIEW TRAPS & SANITY CHECKS

Using the total product and dividing. It is banned, and it crashes on any zero. The prefix/suffix sweep is the only structure that survives zeros.

Sanity check. After the left pass, out[i] equals the product of nums[0]..nums[i-1]. After the right pass at index i, out[i] equals product(nums[0..i-1]) * product(nums[i+1..n-1]), which is exactly the answer.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is division forbidden even when no zeros are present?

Answer. The problem bans it, and more fundamentally it only works when every element is invertible. A single 0 in the input makes the total product 0 and the quotient undefined, so the algorithm would need a zero pass anyway – at which point the prefix/suffix version is simpler.

Q2. Why does out[0] need to be seeded with 1 rather than nums[0]?

Answer. out[i] must hold the product of everything strictly left of i, so at $i = 0$ that product is the empty product, which is 1. Seeding with nums[0] would double-count the element.

Q3. What is the space complexity, and can it be $O(1)$?

Answer. $O(1)$ auxiliary, because the output array itself is reused as the prefix store. Total space including the required output is $O(n)$.

0242 Valid Anagram Easy

ESSENCE & BOUNDS

Decide whether two strings are anagrams of each other, that is whether one is a permutation of the other. 1 <= s.length, t.length <= 5e4. Lengths must match first, which rejects half the cases in $O(1)$. Only lowercase English letters are guaranteed, so a 26-slot array is exact; a general solution needs a 26-entry array plus an ASCII guard or a dict.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Fixed-size frequency array (BEST) Only 26 lowercase letters → a list of 26 ints. Index = ord(c)-97.

Python (intuition kernel)

```python
if len(s) != len(t):  return False
need = [0] * 26
for ch in s:  need[ord(ch) - 97] += 1
for ch in t:
 i = ord(ch) - 97
 if need[i] == 0:  return False
 need[i] -= 1
return all(v == 0 for v in need)

```

C++ (intuition kernel)

```cpp
if (s.size() != t.size()) return false;
array<int,26> need{};
for (char ch :  s) ++need[ch - 'a'];
for (char ch :  t) {
 if (--need[ch - 'a'] < 0) return false;
}
return all_of(need.begin(), need.end(),
 [](int v) { return v == 0; });

```

Time O(n) **Space** O(1) (26 slots!)

Follows directly from the traversal count: $O(n)$ time, $O(1)$ (26 slots!) space.

Sort both strings Time Complexity : $O(nlogn)$ Space Complexity : $O(1)$. Canonical form of an anagram is its sorted self. ==========

Python (intuition kernel)

```python
return sorted(s) == sorted(t)

```

C++ (intuition kernel)

```cpp
string a = s, b = t;
sort(a.begin(), a.end());
sort(b.begin(), b.end());
return a == b;

```

Time O(n log n) **Space** O(1).

the sort dominates; the post-sort scan is linear; hence the stated bounds.

VISUAL STATE TRACE

```
s = "anagram", t = "nagaram"
need = [0]*26
for ch in s:  need[ord(ch)-97] += 1
a+1 n+1 g+1 r+1 m+2
for ch in t:
if need[ord(ch)-97] == 0:  return False (t longer/alien)
need[ord(ch)-97] -= 1
return all(v == 0 for v in need) (s fully consumed)
invariant:  need counts (s prefix) minus (t prefix); never negative
```

LANGUAGE MECHANICS: PYTHON VS C++

Twenty-six counters is 26 bytes, so C++ touches a single cache line and increments in place with a `++count[ch - 'a']` – no allocation, no branching beyond the bounds guard. Python's list of 26 small ints is heap-allocated and CPython caches the values 0..256, so `need[i] += 1` stores back an interned int and allocates nothing, but the list itself costs 264 bytes plus 26 pointers of indirection per access. Python pays about 3x in interpreter dispatch overhead per increment; the memory story is a wash.

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `std::array<int,26> need{}` is stack-allocated and zero-initialised in one instruction sequence.
- Sorting both strings is $O(L \log L)$ but $O(1)$ extra space and trivially correct for any character set.
- `std::string_view` avoids the by-value copies that `std::string` parameters would force.

- `Counter(s) == Counter(t)` is a one-liner but builds two 26-entry dicts.
- `sorted(s) == sorted(t)` is $O(L \log L)$ and allocates two lists of L elements.
- `collections.Counter(s) - Counter(t)` computes the multiset difference directly, which is the same idea with dict internals.

INTERVIEW TRAPS & SANITY CHECKS

Checking only that every character of s appears in t. Without the counters running back to zero, "aab" vs "abb" or "aab" vs "aba" pass. The final all-zero test is the load-bearing line.

Sanity check. After each t character, `need[c]` equals the count of c in s minus the count of c in the t prefix. It must never go negative, and it is all zeros at the end exactly when s and t are permutations.

INTUITION CHECK — 3 QUESTIONS

Q1. Why check the lengths first?

Answer. Different lengths cannot be permutations, and the check is $O(1)$. It also guarantees that if no counter ever goes negative, all counters must finish at zero – which lets you drop the final scan.

Q2. How do you adapt this to arbitrary characters including unicode?

Answer. Replace the 26-array with a dict keyed by code point, or map code points to ids first. The invariant is unchanged; only the index function changes.

Q3. What is the difference between the sorted and counted approaches in asymptotics and space?

Answer. Counted is $O(L)$ time and $O(1)$ space (26 slots). Sorted is $O(L \log L)$ time and $O(L)$ space for the two sorted copies. Counting wins on both once the alphabet is fixed.

0271 Encode and Decode Strings Medium

ESSENCE & BOUNDS

Implement `encode` turning a list of strings into one string and `decode` recovering the exact original list. The two must be perfect inverses on any input, including the delimiter character itself. 0 <= strs.length <= 1e5; 0 <= strs[i].length <= 100, so a list of 1e5 empty strings is legal and must round-trip. Total encoded length is bounded by roughly 7e6. The delimiter-collision case is the entire difficulty: any scheme that just joins on a character is broken by that character appearing inside a word.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Length-prefix framing (BEST, robust) Time Complexity : $O(T)$ Space Complexity : $O(T)$ (T = total characters). WHY length prefix beats a delimiter: if a word contains the delimiter itself, parsing breaks. Lengths are unambiguous — the same trick as HTTP Content-Length and Protobuf varints. ———————————————————————————

Python (intuition kernel)

```python
def encode(strs):
 return ''.join(f'{len(w)}#{w}' for w in strs)

def decode(s):
 out, i = [], 0
 while i < len(s):
 j = s.index('#', i)
 n = int(s[i:j])
 out.append(s[j+1:j+1+n])
 i = j + 1 + n
 return out

```

C++ (intuition kernel)

```cpp
string encode(vector<string>& strs) {
 string out;
 for (auto& w : strs) { out += to_string(w.size());
 out += '#'; out += w; }
 return out;
}
vector<string> decode(string& s) {
 vector<string> out; int i = 0;
 while (i < s.size()) {
 size_t j = s.find('#', i);
 int n = stoi(s.substr(i, j - i));
 out.push_back(s.substr(j + 1, n));
 i = j + 1 + n;
 }
 return out;
}

```

Time O(T) **Space** O(T)

two linear passes; the suffix is carried in a scalar; hence the stated bounds.

Escaped delimiter ('#' $\to$ "/#", '/' $\to$ "//") Also $O(T)/O(T)$. Works, but every escape costs CPU and the stream is harder to read by eye. ==========

Python (intuition kernel)

```python
def encode(strs):
 return '#'.join(w.replace('/', '//').replace('#', '/#')
 for w in strs)

def decode(s):
 out, cur, esc = [], '', False
 for ch in s:
 if esc: cur += ch; esc = False
 elif ch == '/': esc = True
 elif ch == '#': out.append(cur); cur = ''
 else: cur += ch
 return out

```

C++ (intuition kernel)

```cpp
string encode(vector<string>& strs) {
 string out;
 for (auto& w : strs)
 for (char ch : w) {
 if (ch == '/') out += "//";
 else if (ch == '#') out += "/#";
 else out += ch;
 }
 out += '#';
 return out;
}

```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

VISUAL STATE TRACE

```
input : ["a", "b#c", "d"] (delimiter '#')
BAD join : "a#b#c#d"
decode cannot tell if the token is
[a,b,c,d] or [a,b#c,d]
GOOD length-prefix :
"1#a#3#b#c#1#d"
^ each header states the exact length,
so the decoder never guesses a boundary
decode:
read digits until '#' -> len
take next len chars -> word
advance; repeat until exhausted
round trip: [a, b#c, d] -> same list, always
```

LANGUAGE MECHANICS: PYTHON VS C++

This is a serialisation problem, so the cost is dominated by per-byte append. In C++ `str.append(s)` writes into a `std::string` whose capacity doubles, so the total copy work is amortised $O(total)$ and the final buffer is one contiguous block – the decoder can then parse with a raw pointer and a `stoi`-like digit loop touching nothing but the string's own bytes. Python's `str += piece` on a str is also amortised $O(1)$ thanks to CPython's resize heuristic, but the string is UCS-2/UCS-4 internally, so each element costs 1-4 bytes; joining a list of pieces with `".join` is measurably faster because it does one pass with a precomputed total length instead of repeated resize checks.

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `std::string::append(s.data(), len)` with a manual length header beats `to_string` + + because it avoids temporary string objects.
- Reserve the output up front with `reserve(total)` to remove the doubling copies entirely.
- `std::string_view` lets `decode` parse without owning a copy; safe only if the encoded buffer outlives the views.

- `".join(parts)` in one pass beats repeated += for many small pieces.
- Building the length header with an f-string `f'{len(s)}#'` is faster than `str(len(s)) + '#'`.
- `io.StringIO` with `getvalue()` is the other way to batch appends without quadratic risk.

INTERVIEW TRAPS & SANITY CHECKS

Joining on a delimiter with no escaping. That round-trips fine on the examples and fails the moment an input string contains the delimiter, which is exactly the hidden test.

Sanity check. At every decode step, the cursor sits on the first digit of a length header whose declared length is exactly the number of characters up to the next cursor position. If that holds, the parse never needs to search for a separator.

INTUITION CHECK — 3 QUESTIONS

Q1. What property must the encoding have for `decode` to be correct?

Answer. It must be unambiguously decodable: the encoded string must determine the original list uniquely. Length-prefixing achieves that by making every boundary explicit rather than inferred from a separator.

Q2. Is the escaping scheme ('#' -> '/#', '/' -> '//') correct?

Answer. It is decodable provided no input string ever contains '/'. That is an unstated assumption about the alphabet, which is why the length-prefix design is the safer answer.

Q3. Why does length-prefixing still decode when the input list is empty or all elements are empty?

Answer. An empty list encodes to the empty string and decodes to nothing. A list of k empty strings encodes to '0#' repeated k times, and each header of 0 consumes zero characters – so the cursor advances by the header length only, and the count of headers carries the count of empty strings.

0347 Top K Frequent Elements Medium

ESSENCE & BOUNDS

Return the k most frequent elements in an array, in any order. 1 <= nums.length <= 2e4; 1 <= k <= nums.length, so k can equal n and the answer is then just every distinct value. All values are integers in a bounded range, so counting is exact rather than approximate.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Counter + bucket sort (BEST) buckets[f] = values appearing exactly f times. Walk buckets from high f down until k collected.

Python (intuition kernel)

```python
count = Counter(nums)
buckets = [[] for _ in range(len(nums) + 1)]
for value, freq in count.items():
 buckets[freq].append(value)
out = []
for freq in range(len(nums), 0, -1):
 for v in buckets[freq]:
 out.append(v)
 if len(out) == k:  return out

```

C++ (intuition kernel)

```cpp
unordered_map<int,int> count;
for (int x :  nums) ++count[x];
vector<vector<int>> bucket(n + 1);
for (auto& kv :  count)
bucket[kv.second].push_back(kv.first);
vector<int> out;
for (int f = n; f >= 1 && out.size() < k; --f)
 for (int v :  bucket[f]) out.push_back(v);

```

Time O(n) **Space** O(n)

one $O(1)$ counter update per element; hence the stated bounds.

Counter + min-heap of size k Time Complexity : $O(nlogk)$ Space Complexity : $O(n)$ Wins when k n and data arrives streaming (no full array). ==========

Python (intuition kernel)

```python
return heapq.nlargest(k, Counter(nums).items(),
 key=lambda kv:  kv[1])

```

C++ (intuition kernel)

```cpp
vector<pair<int,int>> f; // (freq, value)
for (auto& kv :  count) f.push_back({kv.second,
kv.first});
nth_element(f.begin(), f.begin() + k, f.end(),
 [](auto& a, auto& b) { return a.first > b.first; });
vector<int> out;
for (int i = 0; i < k; ++i)
out.push_back(f[i].second);

```

Time O(n log k) **Space** O(n)

one $O(1)$ counter update per element; hence the stated bounds.

VISUAL STATE TRACE

```
nums = [1,1,1,2,2,3] k = 2
counter:  {1:3, 2:2, 3:1}
heap.nlargest(2, counter, key=counter.get)
push (3,1) -> keep 1
push (2,2) -> keep 2,2
push (1,3) -> better than 2 -> drop 2
-> [1,2] correct
bucket sort O(n):
buckets[3]=[1] buckets[2]=[2] buckets[1]=[3]
walk freq 3..1, collect 1 then 2 -> [1,2]
```

LANGUAGE MECHANICS: PYTHON VS C++

Counting is the same integer-increment work in both languages, but the selection step diverges. C++ `nth_element` partitions the frequency vector in place with introselect: linear average time, no allocation, and it leaves the k largest in the first k slots unordered – exactly the freedom this problem allows. Python has no `nth_element`, so `heapq.nlargest(k, ...)` is a lazy min-heap of size k (or `sorted` for $O(n \log n)$); both allocate a real heap of k Python tuples and pay full interpreter overhead per comparison. Bucket sort in Python is $O(n)$ but with a large constant because `max(counts)` plus n bucket appends is n Python-level operations.

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `std::nth_element(begin, begin + k, end)` on a vector of pairs is $O(n)$ average with no extra memory.
- Sorting a vector of (frequency, value) pairs descending and taking k is simpler and $O(n \log n)$.
- For a bounded value range, counting into a fixed-size array and then a descending frequency sweep gives strict $O(n)$.

- `Counter(nums).most_common(k)` is the intended one-liner; internally it is `heapq.nlargest`.
- `heapq.nlargest(k, counter, key=counter.get)` avoids materialising the (count, value) pairs.
- `sorted(counter, key=counter.get, reverse=True)[:k]` is the readable fallback but $O(n \log n)$.

INTERVIEW TRAPS & SANITY CHECKS

Sorting the input array and taking a slice. Order carries no frequency information, so this returns the k largest VALUES, not the k most frequent ones – wrong for [1,1,1,2,2,3], k=2.

Sanity check. The counter's value set equals the distinct values of nums, and each count equals its exact multiplicity. Selecting the top k of that multiset of counts is therefore equivalent to selecting the k most frequent values.

INTUITION CHECK — 3 QUESTIONS

Q1. Why can you do better than sorting the array?

Answer. Only multiplicities matter, not positions, so you first compress n items into at most n distinct counts and then select the top k of those. Sorting nums spends its effort on a total order that is then discarded.

Q2. What does `nth_element` guarantee, and what does it not?

Answer. It guarantees the first k elements are the k smallest (or largest, per comparator) as a set, in linear average time. It does NOT sort them, which is fine here since any order is accepted.

Q3. If k equals n, which approach collapses?

Answer. Everything works, but heapq.nlargest special-cases k >= len and returns a sorted list directly. The bucket sweep also degenerates to a full scan of the frequency axis, so it stays $O(n)$.

2. Two Pointers

Two pointers is a search-space-reduction technique, not a data structure. You start with two extremes of the feasible region and every move permanently discards a subrange, so the algorithm is a depth-first walk of an $O(n^2)$ decision tree pruned to a single path. The two flavours are opposite pointers on a sorted-or-unsorted array where a monotone predicate justifies each move, and same-direction pointers where a window's validity is monotone. The invariant to hunt for is always the same: after each move, the discarded end provably cannot appear in any strictly better answer than the one already recorded.

#	Problem	Level	One-line intent
0011	**Container With Most Water**	Medium	Move the shorter wall; area is width times the shorter height.
0015	**3Sum**	Medium	Fix one value, two-pointer the rest, and skip duplicates.
0042	**Trapping Rain Water**	Hard	Always advance the side with the smaller running maximum.
0125	**Valid Palindrome**	Easy	Two pointers that skip non-alphanumeric and fold case.
0167	**Two Sum Ii**	?	Sorted input means one pointer move per step; watch the 1-based indices.

0011 Container With Most Water Medium

ESSENCE & BOUNDS

Given an array of non-negative heights, find the two lines that, with the x-axis, hold the most water. Return that maximum area. 1 <= height.length <= 2e5; 0 <= height[i] <= 1e4. Fewer than two lines returns 0. The area fits in 32 bits but the intermediate product is the widest arithmetic in the problem, so be careful in C++.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Brute force (baseline, O(n^2)) Try EVERY pair of walls (i, j), compute the enclosed area, keep the maximum. Correct but wasteful: n^2 pairs, most dominated by a taller wall elsewhere.

Python (intuition kernel)

```python
best = 0
l, r = 0, n - 1
while l < r:
 best = max(best, (r - l) * min(h[l], h[r]))
 if h[l] < h[r]:  l += 1
 else:  r -= 1

```

C++ (intuition kernel)

```cpp
int best = 0, l = 0, r = n - 1;
while (l < r) {
 best = max(best, (r - l) * min(h[l], h[r]));
 h[l] < h[r] ?  ++l :  -r;
}

```

Time O(n^2) **Space** O(1)

nested enumeration dominates: $O(n\hat{}k)$ pairs, no pruning; hence the stated bounds.

Two pointers (BEST, O(n)) Start l = 0, r = n-1 (maximal width). Compute the area. Then move ONLY the shorter of the two walls inward: • If h[l] <= h[r], advancing l can only make the area grow (the limiting height might rise) — advancing r can never beat this. • Else advance r. Both pointers meet after exactly n-1 moves → $O(n)$ total areas.

Python (intuition kernel)

```python
for i in range(n):
 for j in range(i + 1, n):
 best = max(best, (j - i) * min(h[i], h[j]))
return best

```

C++ (intuition kernel)

```cpp
int best = 0;
for (int i = 0; i < n; ++i)
 for (int j = i + 1; j < n; ++j)
 best = max(best, (j - i) * min(h[i], h[j]));

```

Time O(n) **Space** O(1)

Follows directly from the traversal count: $O(n)$ time, $O(1)$ space.

VISUAL STATE TRACE

```
h = [1, 8, 6, 2, 5, 4, 8, 3, 7]
0 1 2 3 4 5 6 7 8
l=0,r=8:  (8-0) * min(1,7) = 8 8 -> move l (h[l]=1 < h[r])
l=1,r=8:  (8-1) * min(8,7) = 7 7 -> move r (h[r]=7 < h[l])
l=1,r=7:  (7-1) * min(8,3) =18 18 -> move r
l=1,r=6:  (6-1) * min(8,8) =40 40 -> tie:  move either
l=2,r=6:  (6-2) * min(6,8) =24
l=3,r=6:  (6-3) * min(2,8) = 6 -> move l
...  best so far = 49 at l=4,r=8?  no:  l=2,r=7 -> 5*6=30
ANSWER 49 = (8-1) * min(h[1],h[7]) is wrong; true best is
l=1,r=6 area 40, or l=4,r=8:  4*min(5,7)=20
full sweep records every candidate; only the pruned walk is fast
```

LANGUAGE MECHANICS: PYTHON VS C++

The loop is a dependent chain: the next comparison needs the previous pointer, so the branch predictor sees a hard-to-predict pattern and mispredicts often, which is where the real cycles go. C++ keeps `height[l]` and `height[r]` in registers across the iteration and computes the area in two multiplies with no memory traffic beyond two L1 loads; the compiler can even hoist the comparison into a conditional move, eliminating the branch entirely. CPython re-dereferences the list pointer, boxes the indices for `height[l]`, calls `min()` as a Python-level function, and allocates a new int for every `max(...)` candidate – roughly 15 to 20 bytecodes per iteration for work C++ does in about six instructions.

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- Ternary `h[l] < h[r] ? ++l : --r` instead of an if/else lets the compiler emit a branchless cmov, which removes the misprediction entirely.
- Prefix and suffix max arrays give an $O(n)$ one-pass alternative but need $O(n)$ extra memory; the two-pointer version needs none.
- For 64-bit safety on huge inputs use `1LL * (r - l) * min(h[l], h[r])` even though the bound here fits in 32 bits.

- `min/max` are Python function calls; inlining them as conditional expressions is measurably faster in a hot loop.
- `while l < r` with `l, r = (l+1, r) if ... else (l, r-1)` avoids rebinding overhead in some interpreters – measure before trusting it.
- `zip(height, reversed(height))` is a cute $O(1)$-space form but it stops at the midpoint, so it is wrong unless combined with an explicit index.

INTERVIEW TRAPS & SANITY CHECKS

Moving the TALLER wall, or moving both pointers. Any answer containing the shorter wall is bounded by the area just computed, so that wall can never improve on the recorded best; discarding it is safe. Discarding the taller one is not, because the shorter wall could pair with something further out.

Sanity check. At the top of each iteration, `best` is at least the maximum area over every pair that uses an index already discarded. That is what makes the discard safe.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is it always safe to move the shorter wall?

Answer. Any pair (l, j) with j < r holds at most h[l] of water and a width smaller than r - l, so its area is at most what we just recorded. Same symmetric argument for r. Hence the discarded end cannot appear in a better answer.

Q2. What does the prefix/suffix max version buy you, and what does it cost?

Answer. It removes the dependency between the two pointer positions, letting you sweep in one direction. It costs $O(n)$ extra memory and two extra passes, so for this problem the two-pointer version strictly dominates.

Q3. How many pairs does the two-pointer version actually examine?

Answer. At most $n - 1$, because each iteration moves exactly one pointer toward the other. The brute force examines $n(n-1)/2$. The speedup comes from the proof above, not from the count alone.

0015 3Sum Medium

ESSENCE & BOUNDS

Return every unique triple of distinct indices whose values sum to zero. Duplicated triples must be reported once and the order within a triple is irrelevant. 3 <= nums.length <= 3000; -1e5 <= nums[i] <= 1e5. At least one triple is guaranteed. The $O(n^2)$ target is mandatory because the naive triple loop is $O(n^3)$ and times out at n = 3000.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Sort + two pointers (BEST, O(n^2)) Sort nums. For each pivot i (skipping duplicate pivots): • target = -nums[i]; l = i+1, r = n-1. • Sum nums[l] + nums[r]: < target → l += 1, > target → r -= 1, == target → record, then skip duplicates on BOTH sides. • nums[i] > 0 → early exit: three positives can never sum to 0.

Python (intuition kernel)

```python
nums.sort()
out = []
for i in range(n - 2):
 if nums[i] > 0:  break
 if i and nums[i] == nums[i-1]:  continue
 l, r = i + 1, n - 1
 while l < r:
 s = nums[i] + nums[l] + nums[r]
 if s < 0:  l += 1
 elif s > 0:  r -= 1
 else:
 out.append([nums[i], nums[l], nums[r]])
 while l < r and nums[l] == nums[l+1]:  l += 1
 while l < r and nums[r] == nums[r-1]:  r -= 1
 l += 1; r -= 1

```

C++ (intuition kernel)

```cpp
sort(nums.begin(), nums.end());
vector<vector<int>> out;
for (int i = 0; i + 2 < n; ++i) {
 if (nums[i] > 0) break;
 if (i && nums[i] == nums[i-1]) continue;
 int l = i + 1, r = n - 1;
 while (l < r) {
 int s = nums[i] + nums[l] + nums[r];
 if (s < 0) ++l; else if (s > 0) --r;
 else { out.push_back({nums[i], nums[l], nums[r]});
 while (l < r && nums[l] == nums[l+1]) ++l;
 while (l < r && nums[r] == nums[r-1]) --r;
 ++l; --r; }
 }
}

```

Time O(n^2) **Space** O(1) extra

the sort dominates; the post-sort scan is linear; hence the stated bounds.

Hash map with dedupe set (O(n^2) time, O(n) space) Same pivot loop, but the inner 2-sum uses a set of seen values: for each j, look up -(nums[i]+nums[j]). Insert triplets into a set of canonical "a,b,c" tuples to kill duplicates.

Python (intuition kernel)

```python
out, seen = [], set()
for i in range(n):
 for j in range(i + 1, n):
 need = -nums[i] - nums[j]
 if need in seen:
 out.append(sorted([nums[i], nums[j], need]))
 seen.add(nums[j])
return {tuple(t) for t in out}

```

C++ (intuition kernel)

```cpp
set<vector<int>> uniq;
for (int i = 0; i < n; ++i) {
 unordered_set<int> seen;
 for (int j = i + 1; j < n; ++j) {
 int need = -nums[i] - nums[j];
 if (seen.count(need)) {
 vector<int> t{nums[i], nums[j], need};
 sort(t.begin(), t.end()); uniq.insert(t);
 }
 seen.insert(nums[j]);
 }
}

```

Time O(n^2) **Space** O(n)

one $O(1)$ hash probe per element, no rescans; hence the stated bounds.

VISUAL STATE TRACE

```
nums = [-1, 0, 1, 2, -1, -4] -> [[-1,-1,2], [-1,0,1]]
sort:  [-4, -1, -1, 0, 1, 2]
i=0 x=-4 nums[i]>0?  no
l=1 r=5 sum=-3 <0 -> l++ (need bigger)
l=2 r=5 sum=-3 <0 -> l++
l=3 r=5 sum=-1 <0 -> l++
l=4 r=5 sum=-1 <0 -> l++ -> l>=r, next i
i=1 x=-1 nums[i-1]==-1?  YES -> SKIP (dup left)
l=2 r=5 sum=0 HIT -> [-1,-1,2]
skip dup l (nums[l]==nums[l+1]) -> l=4
skip dup r (nums[r]==nums[r-1]) -> r=4
l+=1 r-=1 -> l>r, exit
i=2 x=-1 nums[i-1]==-1?  YES -> SKIP (dup left)
i=3 x=0 l=4 r=5 sum=3 >0 -> r-
l=4 r=4 -> exit
i=4 x=1 nums[i]>0 -> BREAK (rest cannot sum to 0)
result [[-1,-1,2],[-1,0,1]]
```

LANGUAGE MECHANICS: PYTHON VS C++

Sorting dominates here. C++ `std::sort` is introsort – quicksort with a depth cap, then heapsort, then insertion sort for small ranges – so it is branch-heavy, cache-friendly on ints (a single 24 KB array for n = 3000, entirely in L1/L2) and never allocates. CPython's `list.sort()` is Timsort, which on an already-sorted or nearly-sorted run costs $O(n)$, and it uses a scratch buffer plus $O(\log n)$ comparisons per merge step. The real Python cost is not the sort but the triple loop: the inner while is pure bytecode with a `max` of two indices, and CPython's per-iteration overhead of roughly 200 ns dominates the three integer adds that do the actual work.

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `vector<long long>` for the sum if you want headroom; int is enough for the stated bounds but -1e5 * 3 fits comfortably anyway.
- Prune the inner loop with `if (nums[i] + nums[i+1] + nums[i+2] > 0) break;` and `if (nums[i] + nums[n-2] + nums[n-1] < 0) continue;` – standard three-sum pruning that shaves the constant noticeably.
- Sorting once into `vector<pair<int,int>>` keeps original indices if you need them; this problem does not.
- `while l < r and nums[l] == nums[l+1]: l += 1` after a hit is what makes the output unique; omitting it is the classic bug.
- Two-sum per fixed i with a dict is $O(n^2)$ too but allocates a dict n times; the two-pointer version allocates nothing.
- `itertools.combinations` is $O(n^3)$ and unusable here, but its output shape is the fastest way to express the brute force.

INTERVIEW TRAPS & SANITY CHECKS

Finding a valid triple and appending it without the duplicate-skipping sweeps afterwards. You then emit the same triple many times, which LeetCode rejects as a wrong answer even though the set of triples is right.

Sanity check. For fixed i, `l < r` and `nums[l..r]` is sorted. A sum below zero means no pair at or right of l can work with this i and the current r, so l advances; symmetrically for r. Every triple with first index i is found or provably excluded.

INTUITION CHECK — 3 QUESTIONS

Q1. Why must you skip duplicates after a hit, and not before?

Answer. Skipping duplicates on the left before the loop only handles duplicate values of the fixed `i`. After a hit, the same triple can still be re-found by advancing one pointer, so the inner duplicate sweeps on both `l` and `r` are what make the output set unique.

Q2. Why is `if nums[i] > 0: break` safe?

Answer. The array is sorted, so once the fixed value is positive every later value is too and no triple including it can sum to zero. Symmetrically, `nums[i] + nums[n-2] + nums[n-1] < 0` lets you skip the current i entirely.

Q3. What is the time complexity, and where does each factor come from?

Answer. $O(n \log n)$ for the sort plus $O(n^2)$ for the nested i/l walk – each i sweeps at most n pointers. The third factor is eliminated by the monotone two-pointer argument inside the inner loop.

0042 Trapping Rain Water Hard

ESSENCE & BOUNDS

Given bar heights, compute how much trapped rain water sits between them, assuming the ends are infinitely tall walls. 0 <= height.length <= 2e4; 0 <= height[i] <= 1e5. The answer can reach about 10^9, so it just fits in a signed 32-bit int but any intermediate sum wants `long long`. An empty array or fewer than three bars traps nothing.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Prefix/suffix max arrays (intuitive, O(n) space) Pass 1: leftMax[i] = max(height[0..i]) — build left to right. Pass 2: rightMax[i] = max(height[i..n-1]) — build right to left. Pass 3: water[i] = min(leftMax[i], rightMax[i]) - height[i] $\geq$ 0. Sum the non-negative values. Three linear passes, dead simple.

Python (intuition kernel)

```python
l, r = 0, n - 1
lo = hi = 0
water = 0
while l < r:
 lo, hi = max(lo, h[l]), max(hi, h[r])
 if lo < hi:
 water += lo - h[l]; l += 1
 else:
 water += hi - h[r]; r -= 1

```

C++ (intuition kernel)

```cpp
int l = 0, r = n - 1, lo = 0, hi = 0, water = 0;
while (l < r) {
 lo = max(lo, h[l]); hi = max(hi, h[r]);
 if (lo < hi) { water += lo - h[l]; ++l; }
 else { water += hi - h[r]; --r; }
}

```

Time O(n) **Space** O(n)

two linear passes; the suffix is carried in a scalar; hence the stated bounds.

Two pointers (BEST, O(1) space) Maintain l = 0, r = n-1 plus two running maxima: lMax (tallest seen on the left side) and rMax (tallest seen on the right side). Invariant: if lMax <= rMax, bar l's true cap equals lMax — we can compute water[l] RIGHT NOW and advance l. Symmetric for r. Each bar is finalized exactly once.

Python (intuition kernel)

```python
left = [0] * n; right = [0] * n
lo = 0
for i in range(n):
 left[i] = lo = max(lo, h[i])
hi = 0
for i in range(n - 1, -1, -1):
 right[i] = hi = max(hi, h[i])
return sum(left[i] - h[i] if left[i] < right[i] else right[i] - h[i]
 for i in range(n))

```

C++ (intuition kernel)

```cpp
vector<int> L(n), R(n); int lo = 0, hi = 0;
for (int i = 0; i < n; ++i) L[i] = lo = max(lo, h[i]);
for (int i = n-1; i >= 0; --i) R[i] = hi = max(hi, h[i]);
int w = 0;
for (int i = 0; i < n; ++i) w += min(L[i], R[i]) - h[i];

```

Time O(n) **Space** O(1)

Follows directly from the traversal count: $O(n)$ time, $O(1)$ space.

VISUAL STATE TRACE

```
h = [4,2,0,3,2,5]
 ^
 | <- trapped between 4 and 5
 | .  .  .
 | [4] [2] [0] [3] [2] [5]
l=0 r=5 lo=4 hi=5 lo<hi -> water += 4-2 = 2 (at l=1) -> l=1
l=1 r=5 lo=4 hi=5 lo<hi -> water += 4-2 = 2 (at l=2) -> l=2
l=2 r=5 lo=4 hi=5 lo<hi -> water += 4-0 = 4 -> l=3
l=3 r=5 lo=4 hi=5 lo<hi -> water += 4-3 = 1 -> l=4
l=4 r=5 lo=4 hi=5 lo<hi -> water += 4-2 = 2 -> l=5 loop ends
total = 2+2+4+1+2 = 11 (check:  0,1,3,1,2,0)
key:  when lo<hi the LEFT bound is already known, so height at l
is irrelevant to the level - only the right side can rise.
```

LANGUAGE MECHANICS: PYTHON VS C++

The $O(1)$-space version reads `h[l]` and `h[r]` once per iteration and keeps both running maxima in registers, so the inner loop is two loads, two compares and one subtract – about as tight as user code gets, and the two loads are cache-friendly because the two fronts sweep inward toward each other and stay resident. Python re-reads `h[l]`, calls `max` twice per iteration, allocates a fresh int for every `water +=` accumulation, and performs the accumulate into a boxed object, so the interpreter spends roughly ten bytecodes doing what the C++ loop does in six instructions. The $O(n)$ prefix/suffix version is worse in C++ for the same reason: it makes two full forward and backward passes over memory to hold data that two registers already carry.

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- The $O(1)$-space two-pointer version needs only `lo`, `hi`, `l`, `r` – four registers, no allocation, no vector.
- A prefix-max vector plus a running suffix max gives one forward pass and one backward pass, $O(n)$ space but better branch prediction.
- Bucket approach for huge inputs: clamp heights into min(h, maxH) buckets, accumulate with a running max and a pending-water counter, which is a single streaming pass.

- Inlining `max(lo, h[l])` as `lo = lo if lo > h[l] else h[l]` avoids two Python function calls per iteration.
- The generator-sum form of the prefix/suffix version is concise but allocates three lists of size n.
- For small n a stack-based solution mirroring the histogram problem is also $O(n)$ and reuses a familiar kernel.

INTERVIEW TRAPS & SANITY CHECKS

Advancing the taller side, or forgetting to clamp negative contributions to zero. If you move `r` while `h[r] > h[l]` you subtract a height that has not yet been bounded on the left, and the total comes out too large.

Sanity check. `lo` is the maximum of `h[0..l]` and `hi` the maximum of `h[r..n-1]`. When `lo <= hi`, the water level at l is exactly `lo`, so adding `lo - h[l]` is final and `l` can be retired. Symmetrically for r.

INTUITION CHECK — 3 QUESTIONS

Q1. Why does the smaller running maximum decide which side to move?

Answer. If `lo <= hi`, the highest bar seen from the left already bounds column l, and the right side cannot get lower than `hi`. So the final water surface at l is `lo` and adding `lo - h[l]` now is safe; symmetrically when `hi < lo`. The two cases are the only possibilities, so every step is correct.

Q2. Is the $O(n)$-space prefix/suffix solution ever preferable?

Answer. Yes, if you also need the per-column trapped amounts later, or if a single left-to-right pass suits a streaming source. For a single scalar answer the two-pointer version is strictly better: less memory, fewer passes, and a shorter dependency chain.

Q3. How do you prove the two-pointer version never skips a column's water?

Answer. Each iteration retires exactly one index and adds its exact final contribution, and the pointers move only inward from 0 and n-1 until they meet. So every column is counted exactly once, and its level was provably final at the time it was counted.

0125 `Valid Palindrome` Easy

ESSENCE & BOUNDS

Decide whether a string reads the same forwards and backwards after discarding every non-alphanumeric character and ignoring letter case. 1 <= s.length <= 1e5, no bound on the character set. An empty result after filtering is a palindrome. Digits count as alphanumeric, so '0' must be compared too, and non-ASCII letters need an explicit alnum predicate.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Two pointers with in-place normalization (BEST, O(1) space) l walks from the left, r from the right. Skip any non-alphanumeric character (punctuation, spaces) on either side, then compare s[l].lower() vs s[r].lower(). Any mismatch → false; pointers meet at the center → true.

Python (intuition kernel)

```python
l, r = 0, len(s) - 1
while l < r:
 while l < r and not s[l].isalnum():  l += 1
 while l < r and not s[r].isalnum():  r -= 1
 if s[l].lower() != s[r].lower():  return False
 l += 1; r -= 1
return True

```

C++ (intuition kernel)

```cpp
int l = 0, r = s.size() - 1;
while (l < r) {
 while (l < r && !isalnum(s[l])) ++l;
 while (l < r && !isalnum(s[r])) -r;
 if (tolower(s[l]) != tolower(s[r])) return false;
 ++l; -r;
}
return true;

```

Time O(n) **Space** O(1)

the output array doubles as the state store; hence the stated bounds.

Filter + reverse compare Build a cleaned string (keep alnum, lowercase), then check it against its reverse. Conceptually simplest — a palindrome equals its own reverse — but allocates a second copy of the string.

Python (intuition kernel)

```python
t = ''.join(c.lower() for c in s if c.isalnum())
return t == t[::-1]

```

C++ (intuition kernel)

```cpp
string t;
for (char ch : s) if (isalnum(ch)) t += tolower(ch);
string u(t.rbegin(), t.rend());
return t == u;

```

Time O(n) **Space** O(n)

Follows directly from the traversal count: $O(n)$ time, $O(n)$ space.

VISUAL STATE TRACE

```
s = "A man, a plan, a canal:  Panama"
l=0 r=len-1
step 1:  skip non-alnum from both ends, fold case
step 2:  compare, advance both
A ...  a -> 'a' == 'a' ok
m ...  m -> ok
a ...  n -> 'a' != 'n' ...  keep going, real middle
\{}n
filtered = "amanaplanacanalpanama"
reversed = "amanaplanacanalpanama"
\{}n memory:  two pointers, zero allocation (BEST)
vs filtered + reversed = 2 extra strings of size n
```

LANGUAGE MECHANICS: PYTHON VS C++

The $O(1)$-space version is the whole point, and it is where the languages diverge most. C++ `isalnum` and `tolower` from `<cctype>` are locale-aware C functions operating directly on `char` inside the string's own buffer, so the scan is a pure in-place read with perfect prefetch behaviour and zero allocation. Python's `str.isalnum()` is a method call on a one-character string, and critically CPython caches single-character Latin-1 strings, so `s[l].isalnum()` does not allocate – but `s[l].lower()` on a character with no uppercase form also returns a cached object. The remaining cost is the four method dispatches per iteration, which at n = 1e5 dominates everything else.

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- Use `std::isalnum(static_cast<unsigned char>(ch))` – passing a plain signed `char` is undefined behaviour for bytes above 127.
- `std::string_view` avoids a copy of the input if you must take it by value elsewhere.
- A manual in-place compaction into a buffer (write pointer, read pointer) beats the two-pointer version when you need to keep the filtered string, and is still one pass.
- Generator filtering plus a comparison against the reversed generator is awkward; the explicit two-pointer loop is clearest.
- `s[l].isalnum()` beats `re.match` or `str.translate` here – the per-call regex machinery dwarfs the check.
- `casefold()` is stricter than `lower()` for Unicode but slower; with the ASCII guarantee, `lower()` is correct and fast.

INTERVIEW TRAPS & SANITY CHECKS

Using the whole-string reverse comparison and calling it $O(1)$ space. It is $O(n)$ space and allocates two strings. Also, forgetting that digits count, or comparing without folding case so 'A' != 'a' fails every real test.

Sanity check. Throughout the loop, every character strictly left of `l` and strictly right of `r` has already been matched with its mirror (or skipped as non-alphanumeric). So the remaining work is entirely inside the window [l, r].

INTUITION CHECK — 3 QUESTIONS

Q1. Why is the two-pointer version $O(1)$ space and the reversed-copy version not?

Answer. The two-pointer version only holds two integer indices and reads the original buffer in place. Building `t = filtered` and then `t[::-1]` materialises two new strings of length n, so that version is $O(n)$ space.

Q2. What does `isalnum` return for unicode digits and accented letters in Python?

Answer. True for any unicode alphanumeric, which is broader than the ASCII definition. C++ `std::isalnum` with the default 'C' locale only accepts ASCII. That divergence is exactly why a naive port can behave differently on non-ASCII input.

Q3. How do you make the digits comparison safe from sign issues in C++?

Answer. Cast to `unsigned char` before calling `isalnum/tolower`. A `char` with the high bit set is negative, and passing that where an `unsigned char` is expected is undefined behaviour – a real source of crashes on non-ASCII input.

0167 Two Sum Ii ?

ESSENCE & BOUNDS

Given a 1-indexed array of integers sorted in ascending order, return the 1-indexed positions of the two numbers that add to target. 2 <= numbers.length <= 3e4; -1e7 <= numbers[i] < 1e7; exactly one solution exists and l < r always. The output must be 1-based – the single most common failure on this problem.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

v1 v1 – see the kernel below.

Python (intuition kernel)

```python
l, r = 0, n - 1
while l < r:
 s = nums[l] + nums[r]
 if s == target:  return [l + 1, r + 1] # 1-based!
 s < target and l or l.__class__
 if s < target:  l += 1
 else:  r -= 1

```

C++ (intuition kernel)

```cpp
int l = 0, r = n - 1;
while (l < r) {
 int s = nums[l] + nums[r];
 if (s == target) return {l + 1, r + 1}; // 1-based
 s < target ?  ++l :  --r;
}

```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

v2 v2 – see the kernel below.

Python (intuition kernel)

```python
for i, x in enumerate(nums):
 need = target - x
 if need in seen:  return [seen[need] + 1, i + 1]
 seen[x] = i

```

C++ (intuition kernel)

```cpp
unordered_map<int,int> seen;
for (int i = 0; i < n; ++i) {
 int need = target - nums[i];
 auto it = seen.find(need);
 if (it != seen.end()) return {it->second + 1, i + 1};
 seen[nums[i]] = i;
}

```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

v3 v3 – see the kernel below.

Python (intuition kernel)

```python
l, r = 0, len(numbers) - 1
while l < r:
 s = numbers[l] + numbers[r]
 if s == target:  return [l + 1, r + 1]
 elif s < target:  l += 1
 else:  r -= 1

```

C++ (intuition kernel)

```cpp
while (l < r) { int s = numbers[l] + numbers[r];
 if (s == target) return {l + 1, r + 1};
 else if (s < target) l++;
 else r--;

```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

VISUAL STATE TRACE

```
numbers = [2, 7, 11, 15] target = 9
idx1 idx2 idx3 idx4
l=1 r=4 2 + 15 = 17 > 9 -> r=3
l=1 r=3 2 + 11 = 13 > 9 -> r=2
l=1 r=2 2 + 7 = 9 = 9 -> return [1, 2]
why r moves on 'too big':  numbers[l] is the smallest remaining,
so with this l no smaller r can work either -> retire l?  no,
retire r because any smaller r only makes the sum smaller,
and we already know l is at its smallest useful value here.
1-BASED TRAP: returning [0, 1] fails.
Binary-search alternative is O(n log n); the log n is wasted.
```

LANGUAGE MECHANICS: PYTHON VS C++

Sorted input plus two indices is the ideal shape for C++: `numbers[l]` and `numbers[r]` are loads from opposite ends of a 240 KB array, the cache lines holding both are fetched once and reused on every subsequent iteration, and the pointer update is a single decrement that the compiler turns into a branchless cmov. The array is read-only, so there is no store traffic and no write-allocate misses at all. Python does the same two loads through `numbers.__getitem__`, which returns a borrowed reference and interns nothing here, then allocates a new int for the sum each iteration; the comparison chain is identical but executed in bytecode, so the constant factor is 20 to 50 times larger even though the asymptotics match.

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- A single `for` with two index variables and a ternary move compiles to cmov and avoids one predictable-branch misprediction per iteration.
- `std::lower_bound` + `std::upper_bound` gives $O(n \log n)$ with branch-predictable library code – a reasonable fallback when readability matters more than the constant.
- Because the array is sorted and you know the index, you can prune the right end using `target - numbers[0]` as an initial bound.

- The dict version is $O(n)$ time but $O(n)$ space – strictly worse here, listed only as the contrast that motivates the two pointers.
- `bisect_left` per element is $O(n \log n)$ in C-implemented code, which for large n can beat a Python-level two-pointer loop despite the worse asymptotics.
- Using `numbers[l] + numbers[r] < target` directly avoids materialising `s`, saving one int allocation per iteration.

INTERVIEW TRAPS & SANITY CHECKS

Returning 0-based indices. The input is explicitly 1-indexed and the expected answer is `[1, 2]` for the standard example, not `[0, 1]`.

Sanity check. At every step, if a valid pair exists at all, one exists with `l' >= l` and `r' <= r`. That is exactly what lets each move retire one index permanently.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is moving the pointer toward the middle correct given sorted input?

Answer. If the sum is too large, every pair using the current `r` with a smaller `l` is also too large, because `numbers[l]` only decreases as `l` grows. So `r` cannot be part of a solution and can be retired. The sum-too-small case is symmetric for `l`.

Q2. Is the hash-map solution acceptable here?

Answer. It is $O(n)$ and correct, but the array is already sorted, so the two-pointer version is $O(n)$ time with $O(1)$ space and no hashing. Mentioning why the sorted invariant makes the map unnecessary is the point of the comparison.

Q3. What is the tight bound on pointer moves?

Answer. At most $n - 1$ moves, since each iteration moves exactly one pointer and the gap shrinks by one. Combined with the $O(1)$ cost of each load and add, the whole scan is a linear pass over two cache lines' worth of active data.

3. Sliding Window

A sliding window is a promise that the answer's defining property is monotone under window extension: for fixed-size validity (no repeats, at most k replacements, anagram) growing a window can only add violations, so the left pointer never needs to retreat. Three variants recur: the last-seen index map, which jumps `left` directly past the conflict in $O(1)$; the shrink-by-one set, which costs $O(26)$ extra per left step; and the counter-of-required-frequency with a formed/match counter. The invariant is always about what the window currently contains, and the performance hinges on how aggressively you can advance `left` rather than how many characters the window holds.

#	Problem	Level	One-line intent
0003	**Longest Substring Without Repeating Characters**	Medium	Jumping the left pointer past the last duplicate is $O(n)$.
0076	**Minimum Window Substring**	Hard	Shrink from the left while valid; count formed letters, not characters.
0121	**Best Time to Buy and Sell Stock**	Easy	Track the running minimum price; the best profit is price minus it.
0239	**Sliding Window Maximum**	Hard	A deque of indices, monotone decreasing, answers every window.
0424	**Longest Repeating Character Replacement**	Medium	Window length minus the max frequency, allowed to exceed k.
0567	**Permutation in String**	Medium	Fixed-size window with a match counter kept in $O(1)$ per slide.

0003 Longest Substring Without Repeating Characters Medium

ESSENCE & BOUNDS

Return the length of the longest contiguous substring of s containing no repeated characters. 1 <= s.length <= 5e4; the character set is unspecified, so a 256-entry array is needed rather than 26. The answer is at most the number of distinct characters, which caps how far the window can ever grow.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Sliding window + set (shrink step-by-step) Keep the set of characters inside [l, r]. On duplicate, evict s[l] and l++ until the duplicate is gone, then insert s[r].

Python (intuition kernel)

```python
last, l, best = {}, 0, 0
for r, ch in enumerate(s):
 if ch in last:
 l = max(l, last[ch] + 1) # never move l back
 last[ch] = r
 best = max(best, r - l + 1)
return best

```

C++ (intuition kernel)

```cpp
int last[256]; fill(last, last + 256, -1);
int l = 0, best = 0;
for (int r = 0; r < s.size(); ++r) {
 l = max(l, last[s[r]] + 1);
 last[s[r]] = r;
 best = max(best, r - l + 1);
}

```

Time O(n) **Space** O(min(n, m))

each element is probed once; pairs cancel or are recorded; hence the stated bounds.

Sliding window + last-seen index map (JUMP past dupe) Store each char's most recent index; on duplicate l jumps to last_seen[s[r]] + 1 instead of evicting char-by-char.

Python (intuition kernel)

```python
best, l = 0, 0
win = set()
for r, ch in enumerate(s):
 while ch in win:
 win.remove(s[l]); l += 1
 win.add(ch)
 best = max(best, r - l + 1)
return best

```

C++ (intuition kernel)

```cpp
unordered_set<char> win;
int l = 0, best = 0;
for (int r = 0; r < s.size(); ++r) {
 while (win.count(s[r])) { win.erase(s[l]); ++l; }
 win.insert(s[r]);
 best = max(best, r - l + 1);
}

```

Time O(n) **Space** O(min(n, m))

Follows directly from the traversal count: $O(n)$ time, O(min(n, m)) space.

VISUAL STATE TRACE

```
s = "abcbbad" -> answer 3 ("abc" or "bba"->3)
r=0 'a' last={} l=0 win="a" len 1
r=1 'b' last={a:0} l=0 win="ab" len 2
r=2 'c' last={a:0,b:1} l=0 win="abc" len 3 best=3
r=3 'b' last[b]=1 l=max(0,1+1)=2 win="cb" len 2
r=4 'b' last[b]=3 l=max(2,3+1)=4 win="b" len 1
r=5 'a' last={a:0,b:3} l=max(4,0+1)=4 win="ba" len 2
r=6 'd' l=4 win="bad" len 3
\{}n max(len) = 3
\{}n each index enters and leaves the window exactly once -> O(n)
```

LANGUAGE MECHANICS: PYTHON VS C++

Both versions do n probes, but the second version does strictly more work in Python. The last-seen map in C++ is a 256-entry `int` array sitting in L1 – 1 KB, fully resident, and the update `last[ch] = r` is a single store with no allocation and no bounds check if the array is fixed-size. The Python dict, by contrast, hashes the one-character string (a cached Latin-1 object, so the hash is cached too) and stores an interned int value; the `max(l, last[ch] + 1)` allocates a fresh int for `last[ch] + 1` every single iteration. That is n small allocations that C++ never performs, which is the dominant cost in Python.

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- A fixed `int last[256]` beats `unordered_map<char,int>` for an ASCII byte alphabet: $O(1)$ array indexing, no hashing, no node allocation.
- Use `unsigned char` for the index to avoid sign extension and out-of-bounds on bytes above 127.
- A 26-entry array plus a lowercase-only precondition is 104 bytes – guaranteed single cache line.

- `last.get(ch, -1)` in one lookup beats `if ch in last: last[ch]` which is two hash lookups.
- Iterating with `enumerate` avoids the explicit index increment and the range/len round trip.
- For a lowercase-alphabet guarantee, a `list` of size 26 indexed by `ord(ch) - 97` beats a dict by a wide margin.

INTERVIEW TRAPS & SANITY CHECKS

`l = last[ch] + 1` instead of `l = max(l, last[ch] + 1)`. If the conflict lies behind the current window, that assignment moves `l` backwards, and for inputs like `"abba"` the window shrinks incorrectly and the reported length is too small.

Sanity check. Before processing `s[r]`, `s[l..r-1]` has no duplicate characters and `last[ch]` is the greatest index < r where `ch` occurred. Setting `l = max(l, last[s[r]] + 1)` therefore yields the smallest valid `l`, which maximises the window length for this `r`.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is `max` needed rather than a plain assignment?

Answer. The previous occurrence of `s[r]` might be at an index smaller than `l - 1`, i.e. outside the window. A plain assignment would drag `l` backwards and shrink a window that is already valid. `max` enforces monotonicity of `l`, which is what keeps the total work linear.

Q2. What does the last-seen approach buy over the shrinking set?

Answer. One probe instead of up to 26 erase operations per left move, and each index is touched a constant number of times rather than once per possible duplicate. Both are $O(n)$ in theory; the map version has the smaller constant because it never re-reads characters it has already discarded.

Q3. Can the window length exceed the number of distinct characters?

Answer. No. If the window has d distinct characters then its length is d, so len <= d and also len <= 26 for a lowercase alphabet. That bound is a useful sanity check: any answer above 26 on a lowercase-guaranteed input is a bug.

0076 Minimum Window Substring Hard

ESSENCE & BOUNDS

Return the shortest substring of s that contains every character of t, including multiplicities, or the empty string if none exists. 1 <= s.length, t.length <= 1e5 over the printable ASCII set, so a 128- or 256-slot array is valid and a dict is unnecessary. The empty string is a valid t. Duplicate letters in t must be satisfied: t = 'aab' needs two a's.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Sliding window + two dicts (formed/required) Grow r. When the window count of s[r] reaches t's requirement, formed += 1. While formed == required: record the candidate, then shrink l (decrement its count; below the requirement → 1).

Python (intuition kernel)

```python
need, window = Counter(t), Counter()
for ch in s:  window[ch] += 1
need_size, formed, l, best = len(need), 0, 0, (1e9, 0, 0)
for r, ch in enumerate(s):
 window[ch] += 1
 if ch in need and window[ch] == need[ch]:  formed += 1
 while formed == need_size:
 if r - l + 1 < best[0]:  best = (r-l+1, l, r)
 out = s[l]
 window[out] -= 1
 if out in need and window[out] < need[out]:  formed -= 1
 l += 1

```

C++ (intuition kernel)

```cpp
unordered_map<char,int> need, win;
for (char ch :  t) ++need[ch];
for (char ch :  s) ++win[ch];
int formed = 0, l = 0, best = INT_MAX, bl = 0, br = 0;
for (int r = 0; r < s.size(); ++r) {
 if (win[s[r]]++ == need[s[r]]) ++formed;
 while (formed == need.size()) {
 if (r - l + 1 < best) { best = r-l+1; bl = l; br = r; }
 if (-win[s[l]] < need[s[l]]) -formed;
 ++l;
 }
}

```

Time O(n + m) **Space** O(m)

Follows directly from the traversal count: $O(n+m)$ time, $O(m)$ space.

Sliding window + 128-slot count arrays (no hashing) Identical logic, but need[128] / window_count[128] indexed by ASCII code: $O(1)$ worst case, zero hashing, fixed memory. matched mirrors formed; required counts nonzero need slots.

Python (intuition kernel)

```python
need = Counter(t)
for start in range(len(s) - len(t) + 1):
 win, formed = Counter(), 0
 for r in range(start, start + len(t)):
 win[s[r]] += 1
 if win[s[r]] == need[s[r]]:  formed += 1
 if formed == len(need):  return
s[start:start+len(t)]
return ''

```

C++ (intuition kernel)

```cpp
unordered_map<char,int> need;
for (char ch :  t) ++need[ch];
for (int i = 0; i + t.size() <= s.size(); ++i) {
 unordered_map<char,int> win;
 int formed = 0;
 for (int j = i; j < i + t.size(); ++j) {
 if (++win[s[j]] == need[s[j]]) ++formed;
 }
 if (formed == (int)need.size()) return s.substr(i,
t.size());
}
return "";

```

Time O(n + m) **Space** O(1)

one $O(1)$ hash probe per element, no rescans; hence the stated bounds.

VISUAL STATE TRACE

```
s = "ADOBECODEBANC", t = "ABC" -> "BANC"
need = {A:1, B:1, C:1} need_size = 3
win = {A:1,B:2,C:2,E:2,D:1,O:1} formed = 0
r:  A win[A]=1 == need[A] -> formed=1
r:  D win[D]=1, not in need formed=1
r:  O ..  formed=1
r:  B win[B]=1 == need[B] -> formed=2
r:  E ..  formed=2
r:  C win[C]=1 == need[C] -> formed=3 window "ADOBEC" valid
record 6 ; shrink:  drop A -> win[A]=0 < need -> formed=2, l++
r:  O ..  (invalid, keep extending)
r:  D ..
r:  E ..
r:  B win[B]=1 == need[B] -> formed=3 window "CODEBANC" valid
record 7 (worse, skip)
drop C -> formed=2, l++
r:  A win[A]=1 -> formed=3 window "E BANC" len 5?  record
drop O (not needed) -> still valid
...  best = "BANC" (4) -> return s[bl:br+1]
\{}n critical:  'formed' counts DISTINCT letters satisfied, not letters seen
```

LANGUAGE MECHANICS: PYTHON VS C++

The inner shrink loop is the hot path and it is branch-heavy: every character removal tests membership in `need` and compares against a stored count. C++ with two `unordered_map<char,int>` keeps each map in one or two cache lines for the ASCII range, so the probes are L1 hits after the first few characters; the alternative is a fixed `int need[128]`, `win[128]` pair, 1 KB total, which eliminates hashing entirely and turns `need[ch]` into a pure load. Python's `Counter` and `dict` do the same logical work but each `window[out] -= 1` is a dict update with an int object, and `out in need` is a second hash lookup – so the shrink loop pays roughly three dict operations per removed character where C++ pays two array loads.

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `array<int,128> need{}, win{}` replaces both maps for the printable-ASCII guarantee, turning hashing into indexing.
- Track a `matched` count against `need.size()` rather than recounting – that is what keeps the inner loop $O(1)$ per removal.
- `std::string_view` lets you return a view of the original buffer instead of copying the substring.
- `Counter` pre-decrement is not supported; `need.get(ch, 0)` is the idiomatic equivalent and avoids the `in + []` double lookup.
- `collections.defaultdict(int)` removes the `get` boilerplate but is measurably slower than `dict.get`.
- For a strictly lowercase alphabet, two 26-element lists are faster than any dict in CPython.

INTERVIEW TRAPS & SANITY CHECKS

Testing `all(v >= need[k] for k in need)` on every left move. That is $O(26)$ per step and blows the constant. The `formed` counter makes the validity test a single integer comparison, which is what keeps the whole thing linear.

Sanity check. `formed` equals the number of distinct characters of t whose window count has reached its required count, and no window count ever exceeds its requirement for a needed character because you extend before you shrink. So `formed == len(need)` is exactly "the window covers t".

INTUITION CHECK — 3 QUESTIONS

Q1. Why track distinct letters satisfied rather than total letters matched?

Answer. Total letters matched is unbounded and does not capture coverage: t = 'AB' with window 'AABB' matches 4 letters but is invalid. `formed` counts how many of the required distinct characters are at or above their requirement, so equality with `len(need)` is precisely the validity predicate.

Q2. Why does extending before shrinking keep counts from overshooting?

Answer. You add `s[r]` first and only then run the shrink loop, and you shrink exactly one character per iteration. Because the invariant forbids a needed count from exceeding its requirement, the `formed == need_size` test flips exactly once per shrink, which is what makes the loop terminate predictably.

Q3. What makes this problem harder than the no-repeating variant?

Answer. The required multiset is input-dependent rather than "all distinct", so you need two data structures and a counter, and the window can become invalid again after shrinking (unlike the simple case). That extra state is the entire delta.

0121 Best Time to Buy and Sell Stock Easy

ESSENCE & BOUNDS

Return the maximum profit obtainable from one buy followed by one sell on later days, or 0 if no profit is possible. 1 <= prices.length <= 1e5; 0 <= prices[i] <= 1e4, so the array is non-decreasing only in the degenerate sense and prices are never negative. n = 1 must return 0, and a strictly decreasing series must also return 0 rather than a negative number.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Brute force (all buy/sell pairs) For every buy day i, try every later sell day j and take the best difference. Correct but explores every pair.

Python (intuition kernel)

```python
lo, best = prices[0], 0
for p in prices:
 lo = min(lo, p)
 best = max(best, p - lo)
return best

```

C++ (intuition kernel)

```cpp
int lo = prices[0], best = 0;
for (int p :  prices) {
 lo = min(lo, p);
 best = max(best, p - lo);
}

```

Time O(n^2) **Space** O(1)

nested enumeration dominates: $O(n^k)$ pairs, no pruning; hence the stated bounds.

One pass, track the minimum price (BEST) Walk the array once. Keep min_price = cheapest day seen so far (the optimal "buy" for any future "sell"). At each day, the best profit selling today is prices[i] min_price.

Python (intuition kernel)

```python
best = 0
for i in range(n):
 for j in range(i + 1, n):
 best = max(best, prices[j] - prices[i])
return best

```

C++ (intuition kernel)

```cpp
int best = 0;
for (int i = 0; i < n; ++i)
 for (int j = i + 1; j < n; ++j)
 best = max(best, prices[j] - prices[i]);

```

Time O(n) **Space** O(1)

Follows directly from the traversal count: $O(n)$ time, $O(1)$ space.

VISUAL STATE TRACE

```
prices = [7,1,5,3,6,4] -> 5 (buy 1 sell 6)
p=7 lo=7 best=max(0, 7-7)=0
p=1 lo=1 best=max(0, 1-1)=0
p=5 lo=1 best=max(0, 5-1)=4
p=3 lo=1 best=max(0, 3-1)=2 (no shrink:  2 < 4)
p=6 lo=1 best=max(0, 6-1)=5 BEST
p=4 lo=1 best=max(0, 4-1)=3
\{}n invariant:  at day r, lo = min(prices[0..r])
so p - lo is the best profit among all sells on day r.
\{}n the two-pointer loop is O(n^2) because l restarts per r;
the running-minimum sweep does the same work in one pass.
```

LANGUAGE MECHANICS: PYTHON VS C++

This loop is as tight as C++ gets: one load per iteration, two `min`/`max` which the compiler emits as `cmov`, and a loop-carried dependency of only two registers. There is no memory traffic beyond the streaming load of `prices`, so the loop runs at roughly one element per cycle and is entirely bound by the load port. Python executes roughly 8 bytecodes for the same step and, critically, allocates a new int for each `p - lo` candidate before `max` discards most of them – so for n = 1e5 you get 1e5 short-lived int allocations, which is the difference between microseconds and milliseconds.

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `lo = min(lo, p); best = max(best, p - lo);` compiles to branchless cmovs, removing the unpredictable branch a naive `if` would create.
- `std::min_element` over the prefix is unnecessary – the running minimum is already optimal.
- For a second transaction variant, remember the best two profits with two running minima instead of a DP table.

- Conditional expressions beat the builtin calls in a hot Python loop because each `min()` is a function dispatch.
- Iterating `for p in prices` rather than indexing avoids the range/len overhead entirely.
- `itertools.accumulate(prices, min)` then a max over `p - lo` is C-speed but allocates an n-element list; faster for very large n.

INTERVIEW TRAPS & SANITY CHECKS

Updating `lo` after computing the candidate, so day r's profit is measured against itself and can never be positive on a new minimum day. The order must be: lower the floor first, then measure.

Sanity check. After processing `prices[r]`, `lo` equals the minimum price over indices 0..r and `best` equals the maximum profit over all buys strictly before some sell at index <= r. Together they mean `best` is final for every prefix, so no later recomputation is needed.

INTUITION CHECK — 3 QUESTIONS

Q1. Why does a running minimum replace a sliding window here?

Answer. Validity here is not about window content but about a max over pairs (i, j) with i < j. For each sell j the best buy is the prefix minimum, which is a single scalar carried forward. A window would have to re-examine every earlier price for every j, giving $O(n^2)$.

Q2. What does the strictly-decreasing input return, and why is that correct?

Answer. 0. `lo` equals the current price on every step so `p - lo = 0`, and `best` stays at its initial 0. Returning a negative profit would be wrong since doing nothing is allowed.

Q3. How would you extend this to at most two transactions?

Answer. Carry four scalars: two buys seen so far, one sell, and the best profit. The extra state is why the simple running minimum stops working – once you may re-enter after selling, the floor is no longer a single prefix minimum.

0239 Sliding Window Maximum Hard

ESSENCE & BOUNDS

For every contiguous window of size k in nums, return the maximum value in that window. 1 <= nums.length <= 1e5; -1e4 <= nums[i] <= 1e4; 1 <= k <= n. The output length is n - k + 1. Windows near the end must not produce output before r >= k - 1.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Monotonic deque of indices (BEST) For each i: pop front while it falls outside the window, pop back while values ≤ nums[i] (dominated), push i, then once the window is full record nums[front].

Python (intuition kernel)

```python
dq = collections.deque()
out = []
for i, v in enumerate(nums):
 while dq and nums[dq[-1]] <= v:  dq.pop() # dominated
 dq.append(i)
 if dq[0] <= i - k:  dq.popleft() # expired
 if i >= k - 1:  out.append(nums[dq[0]])
return out

```

C++ (intuition kernel)

```cpp
deque<int> dq;
vector<int> out;
for (int i = 0; i < n; ++i) {
 while (!dq.empty() && nums[dq.back()] <= nums[i]) dq.pop_back();
 dq.push_back(i);
 if (dq.front() <= i - k) dq.pop_front();
 if (i >= k - 1) out.push_back(nums[dq.front()]);
}

```

Time O(n) **Space** O(k)

each index enters and leaves the stack once: amortised $O(1)$; hence the stated bounds.

Max-heap with lazy deletion Push (value, index) pairs (max-heap via negation). The root is the global max; before reading it, lazily pop roots that have slid out of the window (index ≤ i k). Stale roots are removed only when they reach the top.

Python (intuition kernel)

```python
hp, out = [], []
for i, v in enumerate(nums):
 heapq.heappush(hp, (-v, i))
 if hp[0][1] <= i - k:  heapq.heappushpop(hp, (-v, i))
 if i >= k - 1:  out.append(-hp[0][0])
return out

```

C++ (intuition kernel)

```cpp
priority_queue<pair<int,int>> hp; // (value, idx)
vector<int> out;
for (int i = 0; i < n; ++i) {
 hp.push({nums[i], i});
 if (hp.top().second <= i - k) hp.pop();
 if (i >= k - 1) out.push_back(hp.top().first);
}

```

Time O(n log n) **Space** O(n)

one push (and at most one pop) per element; hence the stated bounds.

VISUAL STATE TRACE

```
nums = [1, 3, -1, -3, 5, 3, 6, 7], k = 3
r=0 1 push 0 dq=[0]
r=1 3 pop 0 (<=3) push 1 dq=[1] (1 dominated by 3)
r=2 -1 push 2 dq=[1,2]
r=3 -3 push 3 dq=[1,2,3] out[0]=nums[1]=3 window 0..2
r=4 5 pop 3,2,1 push 4 dq=[4] out[1]=5
r=5 3 push 5 dq=[4,5] out[2]=5
r=6 6 pop 5,4 push 6 dq=[6] out[3]=6
r=7 7 pop 6 push 7 dq=[7] out[4]=7
\{}n out = [3, 5, 5, 6, 7]
\{}n two pops per element, max once:
FRONT pop = expired (index < r-k+1)
BACK pop = dominated (nums[back] <= nums[r], never
the max of any future window containing it)
```

LANGUAGE MECHANICS: PYTHON VS C++

The deque is a linked structure in the general case, but both languages let you implement this one as a flat array with head and tail indices, which turns every push and pop into an integer increment or decrement on two registers. That is the key optimisation: in C++ the deque elements are indices, so the comparison `nums[dq.back()] <= nums[r]` is two loads from the same cache-line-dense array as the input, and the amortised cost is one comparison plus at most two writes per element. CPython's `collections.deque` is a genuinely optimised block-based structure with $O(1)$ end operations and no per-element malloc, so the amortised bound is the same; the overhead is the interpreted `while` conditions and the tuple-free but still boxed index arithmetic. The heap alternative costs $O(n \log n)$ in both.

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- Replace `std::deque<int>` with `vector<int> dq(n); int head = 0, tail = 0;` – two register indices, no block reallocation, best locality.
- `std::multiset` or `priority_queue` with lazy deletion is the simpler alternative at $O(n \log k)$ time and $O(k)$ space.
- For very large n, a segmented/two-pass block-maximum method (the van Herk / Gil-Werman algorithm) gets $O(n)$ with $O(1)$ memory and is cache-friendlier than pointer-based deques.

- `deque` is the right structure; a Python list with a moving head leaks memory unless you periodically slice, which is $O(n)$.
- `heapq.heappushpop` fuses the push and the stale pop into one C call, which is why the heap variant is competitive in Python.
- For the two-block (prefix/suffix max) approach, list comprehensions run at C speed and often beat an interpreted deque loop.

INTERVIEW TRAPS & SANITY CHECKS

Popping from the back with < instead of <=. With <= the newer equal element replaces the older one, which keeps the deque smaller and is still correct. With < the deque fills with stale equals and the amortised argument weakens. Also: pop the front BEFORE reading `dq[0]`, otherwise you can report an expired index.

Sanity check. The deque holds indices strictly increasing, whose values are strictly decreasing, and all lie within the current window [r-k+1, r]. Therefore `dq[0]` is always the window maximum and any index removed from the back is provably never a future maximum.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is it safe to discard a smaller element when a bigger one arrives behind it?

Answer. If `nums[i] <= nums[r]` with $i < r$, then every future window containing i also contains r (because r is to its right and windows are contiguous), so i can never be a maximum again. That domination argument is the entire correctness proof of the back-pop.

Q2. What is the amortised cost of the whole algorithm?

Answer. Each index is pushed once and popped at most once from either end, so across n iterations there are at most 2n pops and n pushes: $O(n)$ total, $O(k)$ space. The per-iteration cost looks like a while loop but is amortised constant.

Q3. When would the heap be the better answer in an interview?

Answer. When k is large relative to n or when the same structure must serve several different window sizes, or when you want to expose the trade-off. It is $O(n \log k)$ with the same $O(k)$ space, so it is never asymptotically better here.

0424 Longest Repeating Character Replacement Medium

ESSENCE & BOUNDS

Return the length of the longest substring containing at most k distinct repeating characters, i.e. at most k characters differing from the substring's most common character. 1 <= s.length <= 1e5; 1 <= k <= s.length; s consists of uppercase English letters, so a 26-slot array is exact. When k >= 26 the answer is simply n, which is a useful early exit.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Sliding window + running max frequency (BEST) count[26] tracks letter frequencies inside [l, r]. max_freq is the max over ALL history — it never decreases, which is sound: when len max_freq > k, shrink l. max_len = max window seen.

Python (intuition kernel)

```python
need = [0] * 26
l = best = mx = 0
for r, ch in enumerate(s):
 i = ord(ch) - 65
 need[i] += 1
 mx = max(mx, need[i])
 while r - l + 1 - mx > k:
 need[ord(s[l]) - 65] -= 1; l += 1
 best = max(best, r - l + 1)
return best

```

C++ (intuition kernel)

```cpp
int need[26] = {}, l = 0, best = 0, mx = 0;
for (int r = 0; r < s.size(); ++r) {
 int i = s[r] - 'A';
 ++need[i]; mx = max(mx, need[i]);
 while (r - l + 1 - mx > k) { -need[s[l] - 'A']; ++l;
}
 best = max(best, r - l + 1);
}

```

Time O(n) **Space** O(26) = O(1)

Follows directly from the traversal count: $O(n)$ time, $O(26) = O(1)$ space.

Brute force (every start, expand while valid) For each start i, expand j while the $\leq$ k condition holds using a fresh per-window count. Correct, but restarts all state for every i.

Python (intuition kernel)

```python
best = 0
for i in range(len(s)):
 for j in range(i, len(s)):
 cnt = Counter(s[i:j+1])
 if j - i + 1 - max(cnt.values()) > k: break
 best = max(best, j - i + 1)
return best

```

C++ (intuition kernel)

```cpp
int best = 0;
for (int i = 0; i < s.size(); ++i) {
 int cnt[26] = {};
 for (int j = i; j < s.size(); ++j) {
 int mx = max(++cnt[s[j]-'A'], mx);
 if (j - i + 1 - mx > k) break;
 best = max(best, j - i + 1);
 }
}

```

Time O(n^2) **Space** O(26) = O(1)

nested enumeration dominates: $O(n \hat{} k)$ pairs, no pruning; hence the stated bounds.

VISUAL STATE TRACE

```
s = "AABABBA", k = 1 -> answer 4 ("ABBA")
r=0 A need[A]=1 mx=1 len 1-1=0 <=1 -> best 1
r=1 A need[A]=2 mx=2 len 2-2=0 <=1 -> best 2
r=2 B need[B]=1 mx=2 len 3-2=1 <=1 -> best 3
r=3 A need[A]=3 mx=3 len 4-3=1 <=1 -> best 4 BEST
r=4 B need[B]=2 mx=3 len 5-3=2 >1 -> shrink 1
len 4-3=1 <=1 -> best 4
r=5 B need[B]=3 mx=3 len 6-3=3 >1 -> shrink 1
...  shrinks until len-mx<=1 again
r=6 A ...
\{}n why mx may STALL: after a shrink the true max can only
drop, but keeping the stale (larger) mx keeps len-mx small,
so the window stays valid.  Only ever UPDATE mx upward.
```

LANGUAGE MECHANICS: PYTHON VS C++

The inner shrink loop decrements one array slot and increments one index, so the whole algorithm is a pair of sweeps over a 104-byte array that never leaves L1. C++ performs that in a handful of instructions per character, giving $O(n)$ with a very small constant. Python does the same arithmetic but each `need[s[l] - 'A'] -= 1` performs an index computation, a list lookup, an int subtraction and a list store – roughly 12 bytecodes for work C++ does in one `dec` instruction. The subtle correctness point is that `mx` is allowed to be stale, which is safe because `mx` never decreases; that monotonicity is trivially expressible in both languages and is the whole trick.

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `int need[26] = {}` on the stack beats any map; the working set is one and a half cache lines.
- Hoist `mx` out of the loop as a separate variable rather than recomputing `max(*max_element(...))` each iteration.
- For mixed case, `s[r] - 'A'` breaks; either normalise once or index with a 256-entry array.

- `Counter` is much slower than a 26-element list because dict operations are hash lookups.
- Tracking `mx` as a separate scalar rather than recomputing `max(count)` is essential – `max()` over 26 slots per iteration would make it $O(26n)$.
- `ord(ch) - 65` is needed for uppercase; using the character directly with a list fails.

INTERVIEW TRAPS & SANITY CHECKS

Recomputing `mx = max(count)` after each shrink, which is both slower and – if implemented as a stale-value reset – wrong. `mx` must be monotonically non-decreasing: you only ever set `mx = max(mx, need[ch])` on extension.

Sanity check. `mx` is the maximum frequency observed in the window at any point since the window reached its current minimum size. Because frequencies only decrease on a shrink, `len - mx` is a lower bound on the true number of replacements, so `len - mx <= k` is sufficient (never falsely invalid) for validity, and the window recorded is genuinely valid.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is it legal to keep a stale `mx` after the window shrinks?

Answer. If the true maximum drops, then `len - trueMax <= len - staleMax`, so using the larger stale value gives a smaller, still-safe replacement count. Using it can never make an invalid window look valid – it only makes the shrink test fire later. The recorded window is checked by the true condition implicitly.

Q2. What happens when `k >= 26`?

Answer. With a 26-letter alphabet, `len - mx <= len - 1 <= n - 1 <= k` whenever `k >= n`, and for `k >= 26` the shrink loop never fires after the first 26 characters. An early `if k >= 26: return n` short-circuits the whole sweep.

Q3. Why does the shrink loop need a while, not a single if?

Answer. Dropping one character can reduce the window length by one while `mx` stays the same, so the excess `len - mx - k` may exceed 1 by an arbitrary amount when k is small. The while loop keeps dropping until the condition holds, and total drops across the whole run is exactly n because `l` only moves forward.

0567 Permutation in String Medium

ESSENCE & BOUNDS

Return true if s2 contains a permutation of s1, that is a contiguous substring of s2 whose character multiset equals that of s1. 1 <= s1.length, s2.length <= 1e5; s1 and s2 contain lowercase and/or uppercase English letters, so a 52-slot or 128-slot ASCII array is required – a 26-slot array is wrong for mixed case. s1 may be longer than s2, which must return false immediately.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Fixed window + 26-array counters + matches (BEST) Count s1 into c1[26] and the first |s1| chars of s2 into c2[26]. matches = #slots where c1[i] == c2[i]. Sliding: adding s2[i] and removing s2[i |s1|] can each flip ONLY ONE slot's equality → matches updates in $O(1)$ per slide. matches == 26 identical multisets true.

Python (intuition kernel)

```python
need, win = [0]*26, [0]*26
for ch in s1:  need[ord(ch) - 97] += 1
for ch in s2[:len(s1)]:  win[ord(ch) - 97] += 1
hits = sum(1 for i in range(26) if win[i] == need[i])
for r in range(len(s1), len(s2)):
 a, b = ord(s2[r]) - 97, ord(s2[r-len(s1)]) - 97
 hits += (win[a] == need[a]) - (win[b] == need[b])
 win[a] += 1; win[b] -= 1
 if hits == 26:  return True
return hits == 26

```

C++ (intuition kernel)

```cpp
int need[26] = {}, win[26] = {};
for (char ch :  s1) ++need[ch - 'a'];
for (int i = 0; i < s1.size(); ++i) ++win[s2[i] - 'a'];
int hits = 0;
for (int i = 0; i < 26; ++i) hits += (win[i] == need[i]);
for (int r = s1.size(); r < s2.size(); ++r) {
 int a = s2[r] - 'a', b = s2[r - s1.size()] - 'a';
 hits += (win[a] == need[a]) - (win[b] == need[b]);
 ++win[a]; --win[b];
 if (hits == 26) return true;
}
return hits == 26;

```

Time O(n) **Space** O(1)

one $O(1)$ counter update per element; hence the stated bounds.

Fixed window + Counter (dict) comparison Same window, but counters live in a collections.Counter and the full comparison happens every slide — O() per slide.

Python (intuition kernel)

```python
if len(s1) > len(s2):
target:  Counter[str] = Counter(s1)
window_count:  Counter[str] = Counter(s2[:  len(s1)])
if window_count == target:
 return True # the very first window already matches
for i in range(len(s1), len(s2)):
 window_count[s2[i]] += 1 # add new char
 left = s2[i - len(s1)]

```

C++ (intuition kernel)

```cpp
if (s1.size() > s2.size()) { return false;

```

Time O(n-) **Space** O()

one $O(1)$ counter update per element; hence the stated bounds.

VISUAL STATE TRACE

```
s1 = "ab", s2 = "eidbaooo" -> true ("ba" at index 3)
need[26] vs win[26], window width W = len(s1) = 2
matches = #{ i :  need[i] == win[i] } (0 or 26)
\{}n slide by one:  OUT = s2[r-W], IN = s2[r]
matches += (win[OUT] == need[OUT])
win[OUT] -= 1
matches += (win[OUT] == need[OUT]) (after change)
win[IN] += 1
matches += (win[IN] == need[IN])
\{}n each slide:  3 compares + 2 increments = O(1)
the naive version recomputes matches = sum(...)  = O(26) per slide
\{}n when |s1| > |s2| -> no window of width W exists -> False
```

LANGUAGE MECHANICS: PYTHON VS C++

The whole algorithm exists to avoid an $O(26)$ recomputation per slide, and in C++ the compare-and-adjust pattern compiles to three conditional increments on a counter in a register, with the 26-element arrays permanently in L1. That makes a slide about six instructions. Python performs the same logic with list indexing and int boxing: each `(win[out] == need[out])` yields a bool object (interned, so no allocation) but the arithmetic `matches += True - False` still creates a new int, and the list index computation `ord(s2[r]) - 97` allocates nothing but costs several bytecodes. For n = 1e5 slides this is roughly a million bytecodes in Python against well under a million cycles in C++.

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- Use `int need[128], win[128]` indexed by `unsigned char` – handles mixed case without any case-folding branch.
- `std::array` instead of a raw array gives you `operator[]` with no behavioural difference but clearer intent.
- For the fixed 52-letter case, `memcmp(need, win, 52) == 0` per slide is 52 bytes of SIMD-comparable load, which can beat the counter for small alphabets – but it is $O(\sigma)$ again.
- `Counter(s2[i:i+len(s1)]) == Counter(s1)` is correct but $O(W \cdot \sigma)$ per window and $O(W)$ allocations – it times out at n = 1e5.
- The `matches` counter is the whole optimization; keep it as a plain int and avoid `sum()` per slide.
- For lowercase-only inputs, `[0]*26` lists beat `Counter` by a factor of several.

INTERVIEW TRAPS & SANITY CHECKS

Using a 26-slot array and assuming lowercase. The problem explicitly allows uppercase, so "Aa" and "aA" are both permutations and a mixed-case input will corrupt the counts.

Sanity check. `win[i]` is the count of character i in `s2[r-W+1 .. r]`, and `matches` is exactly the number of slots where `win[i] == need[i]`. A window is a permutation iff `matches == 26` (equivalently sigma), because equal counts at every slot means equal multisets.

INTUITION CHECK — 3 QUESTIONS

Q1. Why maintain a match counter instead of comparing arrays?

Answer. Comparing the two arrays each slide is $O(\sigma)$ per slide, giving $O(n\sigma)$ overall – 2.6 million comparisons at the limits. The counter makes each slide $O(1)$ because only two slots change, so the total drops to $O(n)$.

Q2. What is the order of the update when a character leaves and another enters?

Answer. Remove first, then add. If the incoming and outgoing characters are the same slot you would double-count; by decrementing before incrementing and adjusting `matches` after each individual mutation, the invariant holds at every intermediate point and no special case is needed.

Q3. How do you adapt this to the fixed-size-anagram-inside-s1 case (567) versus arbitrary-length?

Answer. Here the window width is pinned to `len(s1)`, which is what makes the check a pure frequency comparison. For arbitrary-length windows you would instead need the formed/required counter pattern of problem 76, where the requirement is a multiset rather than an exact match.

4. Stack

The stack is the right structure whenever the last unclosed thing must be resolved first – nested brackets, nested calls, infix evaluation, and monotonic scans where a later element can retroactively cancel an earlier one. Three flavours recur across this category: a stack of openers with a closing-to-opener map, a stack that carries a running summary (a min-stack, a running arrival time) so a query is $O(1)$ instead of $O(n)$, and a MONOTONIC stack where each element is pushed once and popped at most once, giving amortised $O(1)$ per item. The monotonic pattern is the hardest and most valuable: it converts an $O(n^2)$ nested scan into a single pass by keeping only the elements that are still capable of being an answer.

#	Problem	Level	One-line intent
0020	**Valid Parentheses**	Easy	Push opens, pop and match on every close.
0022	**Generate Parentheses**	Medium	Backtrack with two budgets: opens left, closes left.
0084	**Largest Rectangle in Histogram**	Hard	Increasing stack of indices; each pop reveals one maximal rectangle.
0150	**Evaluate Reverse Polish Notation**	Medium	Two operands plus an operator form a three-token reduction.
0155	**Min Stack**	Medium	Push the running minimum alongside every value.
0739	**Daily Temperatures**	Medium	Monotonic decreasing stack: a warmer day resolves the wait.
0853	**Car Fleet**	Medium	Sort by position; scan from the back comparing arrival times.

0020 `Valid Parentheses` Easy

ESSENCE & BOUNDS

Return true if every opening bracket in the string is closed by the same type of bracket in the correct order. 1 <= s.length <= 1e4; the only characters present are '(', ')', '{', '}', '[' and ']'. An empty string is valid. The invariant must be checked on close before popping, so an unmatched closer on an empty stack returns false immediately rather than underflowing.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Stack + dict (BEST) Single pass. Closers are looked up in a dict and compared to the stack top; any mismatch (or a closer over an empty stack) is an immediate False. A valid string leaves the stack EMPTY.

Python (intuition kernel)

```python
match: Dict[str, str] = {")": "(", "}": "{", "]": "["}
st: List[str] = []
for c in s:
 if c in match:
 if not st or st[-1] != match[c]:
 return False # wrong type or unmatched -- fail fast
 st.pop() # pair matched $\{}to$ remove the opener
 else: st.append(c)

```

C++ (intuition kernel)

```cpp
std::unordered_map<char, char> match = { {')', '('}, {'}', '{'}, {']', '['}
```

Time O(n) **Space** O(n)

each element is pushed once and popped at most once; hence the stated bounds.

Repeated pair erasure (no stack, no dict) Delete the substrings "()", "{}", "[]" from a working copy until it stops shrinking; valid the result is empty.

Python (intuition kernel)

```python
work = s
while True:
 before = work
 for pair in ("()", "{}", "[]"):
 work = work.replace(pair, "")
 if work == before:
 break # stable: fully balanced (empty) or invalid
return work == ""

```

C++ (intuition kernel)

```cpp
while (true) { bool changed = false;
 static const std::string pairs[] = {"()", "{}",
"[]"};
 for (const std::string& p : pairs) { std::size_t
pos = work.find(p);
 if (pos != std::string::npos) { work.erase(pos, 2);
 changed = true; // did remove a pair this pass
 }
 }
 if (!changed) break;

```

Time O(n^2) **Space** O(n)

each element is pushed once and popped at most once; hence the stated bounds.

VISUAL STATE TRACE

```
s = "([)]" -> false
s = "({[]})" -> true
ch ( stack: (
ch { stack: ( {
ch [ stack: ( { [
ch ] close ] -> pop [ match! stack: ( {
ch } close } -> pop { match! stack: (
ch ) close ) -> pop ( match! stack: (empty)
end: stack empty AND no mismatch -> true
s = "([)]" -> false
( stack: (
[ stack: ( [
) close ) -> TOP is [ not ( -> MISMATCH -> false
\{}n trap: popping before checking empties the stack
and lets ")]" through
```

LANGUAGE MECHANICS: PYTHON VS C++

The stack here is at most three elements deep for a well-formed string, so both languages live entirely in registers or L1 and the difference is pure interpreter overhead. C++ runs the loop as a load, an array index with a branch on the character class, and a compare – the `switch` compiles to a jump table, which is a single indirect branch the CPU predicts well for a repeating bracket pattern. CPython must execute roughly 20 bytecodes per character (unpack the char, index the dict for the closer, compare strings) and allocates nothing here only because CPython interns single-character Latin-1 strings, so `ch` is a pointer chase to a cached object rather than a fresh allocation.

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `switch (ch)` on the character compiles to a jump table, beating a `std::unordered_map<char,char>` lookup (no hashing, no node).
- `std::string::find_last_of` style scanning is $O(n^2)$ and wrong; a `std::stack<char>` is fine but a raw `std::array<char,64>` avoids allocation.

- A `dict` of closer-to-opener beats `if/elif` chains and keeps the intent data-driven.
- `while stack and stack[-1] != pairs[ch]` followed by `if not stack: return False` is the bug-free idiom – it checks the guard and the match in one pass.
- An f-string-free `stack.pop() != opener` comparison is the tightest check: one pop, one compare, no temp variable.

INTERVIEW TRAPS & SANITY CHECKS

Popping the top and comparing, without first verifying the stack is non-empty. `s = ')'` then indexes an empty list, or in C++ reads uninitialised memory.

Sanity check. After each processed character, the stack contains exactly the opening brackets that have been seen but not yet matched, in their original order. So a closer must match the top element, and if it does not, the string is invalid regardless of what comes later.

INTUITION CHECK — 3 QUESTIONS

Q1. Why does only the top of the stack need checking?

Answer. Nesting requires proper LIFO order, so a closer must immediately undo the most recent unclosed opener. Anything deeper in the stack was opened earlier and must be closed after the current one, so a mismatch at the top is already fatal.

Q2. What is the difference between counting and matching?

Answer. Counting openers and closers of each type ignores order, so `')('` and `'([)]'` would pass. Order is exactly what the stack enforces, which is why this problem is a stack exercise rather than a counting exercise.

Q3. Can this be solved with a counter?

Answer. Only for a single bracket type. With three types you need at minimum a stack of types, since `{[)]` has balanced counts but wrong nesting. The counter version works for `()` and nothing else.

0022 `Generate Parentheses` Medium

ESSENCE & BOUNDS

Generate all combinations of well-formed parentheses of length n, using exactly n pairs. $1 <= n <= 8$, so at most $2n = 16$ characters and $C(8,4) = 70$ valid strings – the answer set is small enough to enumerate exhaustively. Every generated string must have exactly n opens and n closes.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Backtracking DFS (BEST) Recursively grow a string, only taking the two legal moves above, and record the string when it reaches length 2n. Every generated string is valid by construction — no filtering needed.

Python (intuition kernel)

```python
result:  List[str] = []
if len(cur) == 2 * n:
result.append(cur)
return # no more moves possible from a full string
if open_count < n:
dfs(cur + "(", open_count + 1, close_count)
if close_count < open_count:
dfs(cur + ")", open_count, close_count + 1)

```

C++ (intuition kernel)

```cpp
[&](std::string cur, int open, int close) { if
(static_cast<int>(cur.size()) == 2 * n) {
result.push_back(cur);
 return; // no more moves possible from a full string
}
if (open < n) { dfs(cur + '(', open + 1, close);
}
if (close < open) { dfs(cur + ')', open, close + 1);
}

```

Time O(4^n/$\sqrt{\ }$n) **Space** O(n) recursion

every node is visited once and returns $O(1)$ state; hence the stated bounds.

Iterative BFS (explicit queue, no recursion) Same expansion, level by level: a deque of (string, opens, closes) grows every frontier simultaneously until length 2n.

Python (intuition kernel)

```python
result: List[str] = []
frontier: Deque[Tuple[str, int, int]] = deque([("", 0, 0)])
while frontier:
 cur, open_count, close_count = frontier.popleft()
 if len(cur) == 2 * n:
 result.append(cur)
 continue # leaf: nothing else to expand
 if open_count < n:

```

C++ (intuition kernel)

```cpp
while (!frontier.empty()) { auto [cur, open, close] = frontier.front();
 frontier.pop();
 if (static_cast<int>(cur.size()) == 2 * n) { result.push_back(cur);
 continue; // leaf: nothing else to expand
 }
 if (open < n) { frontier.push({cur + '(', open + 1, close});
 }
 if (close < open) { frontier.push({cur + ')', open, close + 1});

```

Time O(4^n/$\sqrt{\ }$n) **Space** O(4^n/$\sqrt{\ }$n)

every node is enqueued and dequeued once; hence the stated bounds.

VISUAL STATE TRACE

```
n = 3 -> ["((()))", "(()())", "(())()", "()(())", "()()()"]
dfs(path, open, close):
if len(path) == 2n: emit; return
if open < n: dfs(path + '(' , open+1, close) # can always open
if close < open: dfs(path + ')' , open, close+1) # can only close
'(' open 1 close 0
/ \{}
'(' open2 cl0 ')' open1 close1
/ \{}\{}| / \{}
'(' o3c0 ')' o2c1 '(' o2c1 ')' o1c2
| \{}\{} | \{}\{}
')' '(())' '(()())' '(())()' '()()()'
\{}n the close branch needs close < open: you may never
close more than you have opened (else ')' first).
```

LANGUAGE MECHANICS: PYTHON VS C++

This is a search tree of depth 2n with branching factor 2, so there are at most 2^{16} nodes here and about 1e5 in the worst allowed case – tiny. The interesting difference is memory: every node in Python holds a new `path` string built by `path + '('`, which allocates a fresh string of length up to 2n and copies the whole prefix. That is $O(n)$ per node, making the whole thing $O(n \cdot 2^n)$ in allocation traffic. C++ has exactly the same trap with `string path`, but passing `path` by value and appending, or mutating a single buffer with push/pop and one `push_back`, avoids the copy entirely and turns each node into $O(1)$ amortised work.

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- Mutate one `string path` with `push_back` / `pop_back` instead of copying per node – the single biggest win here.
- Use a fixed `std::array<char,16>` and a length counter for zero-allocation enumeration.
- `std::function` recursion has indirect-call overhead; a plain static recursive function or a template lambda is inlined.

- A list `path` with `append`/`pop` avoids the $O(n)$ string copy per node; join once at the leaves.
- Pruning with `if close < open` is what keeps the tree at Catalan size instead of 2^{2n}.
- An iterative BFS with a deque is the second approach; it trades recursion for explicit queue churn and is not faster here.

INTERVIEW TRAPS & SANITY CHECKS

Allowing a close when `close == open`. That produces strings like `()` with an unmatched closer and you end up emitting invalid output unless you also check `open == close == n` at the leaf.

Sanity check. `path` is always a prefix of a valid parenthesis string, `open` counts its '(', `close` counts its ')', and the guard `close <= open <= n` holds at every node. That is sufficient to guarantee every leaf is well formed.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is the condition `close < open` enough to prevent invalid prefixes?

Answer. A prefix is extendable to a valid string iff it never has more closers than openers, and we never exceed n of either. Checking `close < open` before appending a closer preserves `close <= open` at every node, so every leaf has balanced counts and proper nesting.

Q2. How many strings are produced?

Answer. The n-th Catalan number, $C_n = \frac{1}{n+1}\binom{2n}{n}$, which is 70 for n = 8 and 5 for n = 3. The output is exponential, so the algorithm is output-bound rather than work-bound.

Q3. What would be wrong with a greedy construction?

Answer. There is no fixed rule for where to place the next bracket – the choice depends on the prefix. That is precisely why this is a backtracking search rather than a greedy or DP problem.

0084 Largest Rectangle in Histogram Hard

ESSENCE & BOUNDS

Return the largest area of an axis-aligned rectangle fully contained in a histogram, where the rectangle's height is limited by the shortest bar it spans. 1 <= heights.length <= 1e5; 0 <= heights[i] <= 1e4. Areas reach about 10^9, so `long long` is the safe C++ type. Zero-height bars are legal and contribute area 0.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Monotonic increasing stack (BEST) Invariant: stack indices hold STRICTLY INCREASING heights. For each new bar h: pop every index whose height is > h. Each popped bar p has h as its first-shorter bar to the RIGHT; the new stack top is the first-shorter bar to its LEFT → width resolved, area computed once, never again. A sentinel 0 at the end forces every bar to be popped exactly once.

Python (intuition kernel)

```python
st: List[int] = [] # indices; heights[st] strictly increasing
best = 0
h = heights + [0]
for i, h_i in enumerate(h):
 while st and heights[st[-1]] > h_i:
 height = heights[st.pop()]
 width = i if not st else i - st[-1] - 1
 best = max(best, height * width)

```

C++ (intuition kernel)

```cpp
for (int i = 0; i < static_cast<int>(h.size()); ++i) { while (!st.empty() && h[st.top()] > h[i]) { int height = h[st.top()];
 st.pop();
 int width = st.empty() ? i : i - st.top() - 1;
 best = std::max(best, height * width);
 }
 st.push(i);

```

Time O(n) **Space** O(n)

each element is pushed once and popped at most once; hence the stated bounds.

Brute force (contrast) For every pair (i, j) of bars, the best rectangle spanning them has height min(heights[i..j]); take the max over all pairs. Correct but quadratic — no structure used.

Python (intuition kernel)

```python
n = len(heights)
best = 0
for i in range(n):
 min_h = heights[i]
 for j in range(i, n):
 min_h = min(min_h, heights[j]) # shrink bound
 best = max(best, min_h * (j - i + 1))
return best

```

C++ (intuition kernel)

```cpp
for (int i = 0; i < n; ++i) { int min_h = heights[i];
 for (int j = i; j < n; ++j) {
 min_h = std::min(min_h, heights[j]); // shrink bound
 best = std::max(best, min_h * (j - i + 1));
 }

```

Time O(n^2) **Space** O(1)

nested enumeration dominates: $O(n\hat{}k)$ pairs, no pruning; hence the stated bounds.

VISUAL STATE TRACE

```
h = [2,1,5,6,2,3] -> answer 10 (bars 2..4, height 5)
i=0 h=2 stack:[0]
i=1 h=1 pop 0:  w = 1-(empty?  -1:  )-1 = 1-0 = 1?  width=1 area=2*1=2
stack:[1]
i=2 h=5 stack:[1,2]
i=3 h=6 stack:[1,2,3]
i=4 h=2 pop 3:  width = 4-2-1 = 1 area=6*1=6
pop 2:  width = 4-1-1 = 2 area=5*2=10 BEST
stack:[1,4]
i=5 h=3 stack:[1,4,5]
i=6 SENTINEL h=0 pop 5:  width=6-4-1=1 area=3
pop 4:  width=6-1-1=4 area=2*4=8
pop 1:  width=6-(-1)-1=6 area=1*6=6
stack:[]
\{}n width rule:  the popped bar's height is valid all the way
from (new stack top + 1) to (i - 1).
```

LANGUAGE MECHANICS: PYTHON VS C++

This is where the monotonic-stack pattern pays off most. In C++ the stack is a preallocated `vector<int>` with an index pointer, so push and pop are a store and an integer decrement into a single cache line, and the inner `while` reads `heights[st.back()]` – a load from the histogram array at an index the CPU has just computed, so it is an L1 hit. Total work is a single pass with 2n stack operations. In Python the same loop runs against a `list` of ints, which is nearly as good for the stack itself, but `heights[st[-1]]` boxes nothing here (existing int objects are reused) while `max(area, height * (i - w - 1))` allocates a new int for every candidate, and the `while` condition re-evaluates `st` truthiness plus an indexing operation. The asymptotic bound is identical; the constant is roughly 30x.

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `vector<int> st; st.reserve(n + 1);` and an index pointer beat `std::stack<int>` – no adaptor indirection, one contiguous block.
- Push a sentinel height of 0 (or -1) as `heights[n]` to flush the stack in the same loop, avoiding a second pass.
- For very large n, the two-pass block-maximum approach (van Herk/Gil-Werman) gives $O(n)$ with better cache behaviour and no stack at all.

- `while st and heights[st[-1]] > h:` is the canonical loop; using >= breaks ties differently but is also correct as long as the width formula stays consistent.
- Append a 0 sentinel to the heights list rather than special-casing the loop exit – removes a duplicated code path.
- Python lists support amortised $O(1)$ `append`/`pop`, so a list is a fine stack; no `deque` needed.

INTERVIEW TRAPS & SANITY CHECKS

Computing the width before popping, or using `i` instead of the new stack top in the width formula. The width is `i - st.back() - 1` evaluated AFTER the pop, because the popped bar is no longer a boundary.

Sanity check. The stack holds indices with strictly increasing heights, and every index between consecutive stack entries is shorter than the entry above it. When index i is popped, the stack top is the nearest bar to the left that is shorter, so bars from top+1 to i-1 all have height at least `heights[i]`.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is the width `i - st.back() - 1` after popping, not before?

Answer. After popping index i, `st.back()` is the nearest bar strictly to the left with a smaller height, which is exactly the left boundary of the maximal span for `heights[i]`. The right boundary is i-1 because the current bar is shorter. Using the pre-pop top would include the boundary bar and overcount.

Q2. What does the sentinel flush accomplish?

Answer. It forces every remaining index to be popped as if a bar of height 0 had appeared at position n. Without it you need a separate loop after the main one, and forgetting that loop loses the rectangles that extend to the right edge.

Q3. Why is the whole algorithm linear despite the nested while?

Answer. Each index is pushed once and popped at most once, so the total number of inner-loop iterations across the entire run is at most n. The nested loop is amortised $O(1)$ per outer iteration.

0150 Evaluate Reverse Polish Notation Medium

ESSENCE & BOUNDS

Evaluate an expression given in reverse Polish notation, where every operator follows its operands and / truncates toward zero. 1 <= tokens.length <= 1e5, with integer operands in [-100, 100] and division by zero guaranteed not to occur. Operand count is always valid, so the stack never underflows. Intermediate results can exceed 32 bits, which is the classic C++ trap.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Stack + operator set (BEST) Operands are pushed; operators pop a and b (b popped SECOND is the LEFT operand — order matters for - and /), then push the result. A set answers "is this token an operator?" in $O(1)$.

Python (intuition kernel)

```python
ops: Set[str] = {"+", "-", "*", "/"}
st: List[int] = []
for tok in tokens:
 if tok not in ops:
 st.append(int(tok))
 continue
 b = st.pop()
 a = st.pop()

```

C++ (intuition kernel)

```cpp
for (const std::string& tok : tokens) { if (ops.find(tok) == ops.end()) { st.push(std::stoi(tok));
 continue;
 }
 int b = st.top(); st.pop();
 int a = st.top(); st.pop();
 if (tok == "+") st.push(a + b);
 else if (tok == "-") st.push(a - b);
 else if (tok == "*") st.push(a * b);

```

Time O(n) **Space** O(n)

each element is probed once; pairs cancel or are recorded; hence the stated bounds.

Stack + char switch (no hash set) Same algorithm, but operators are recognised by shape: every operator token is a SINGLE character and not a digit. That makes a 4-case if/elif chain possible and avoids allocating a set.

Python (intuition kernel)

```python
st: List[int] = []
for tok in tokens:
 is_op = len(tok) == 1 and not tok.isdigit()
 if not is_op:
 st.append(int(tok))
 continue
 b = st.pop()
 a = st.pop()

```

C++ (intuition kernel)

```cpp
for (const std::string& tok : tokens) { bool is_op = tok.size() == 1 &&
 !std::isdigit(static_cast<unsigned char>(tok[0]));
 if (!is_op) { st.push(std::stoi(tok));
 continue;
 }
 int b = st.top(); st.pop();
 int a = st.top(); st.pop();
 switch (tok[0]) { case '+': st.push(a + b); break;

```

Time O(n) **Space** O(n)

one $O(1)$ hash probe per element, no rescans; hence the stated bounds.

VISUAL STATE TRACE

```
tokens = ["2","1","+","3","*"] -> 9
is an operand? push 2 [2]
is an operand? push 1 [2,1]
'+' : b=pop()=1, a=pop()=2 -> 2+1=3 [3]
'3' : push [3,3]
'*' : b=3, a=3 -> 3*3=9 [9]
\{}n operand test: token is a digit, or starts with '-'/'+' followed
by a digit. The classic bug is testing isdigit() alone,
which rejects negative operands like "-3".
\{}n DIVISION TRAP: C++ int division truncates toward zero, so
-7 / 2 == -3 (not -4). Python's // floors to -4, so Python
needs int(a / b) to match.
```

LANGUAGE MECHANICS: PYTHON VS C++

The stack depth is bounded by the number of operands, which for a balanced RPN expression is modest, so both languages keep it in cache. C++ does one `isdigit` scan per token and one division per operator, with the token comparison against `'-'` handled by a branch. CPython's `str.isdigit()` allocates nothing (one-character cached strings) but the repeated `int(token)` conversion goes through the full string-to-integer parser, and the operator dispatch via a dict lookup returns a function object that is then called – a full Python-level call per operator. That call overhead is the dominant cost, not the arithmetic.

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- Dispatch operators with a `switch` on `tok[0]` rather than an `unordered_map<string,function>` – no string hashing, no indirect call.
- `stoll` on the operand after the digit check, or `std::from_chars` which is allocation-free and locale-independent.
- Reserve the stack with `stack.reserve(tokens.size())` to avoid repeated reallocation.

- A dict of lambdas for the five operators is concise but each call is a Python-level function call.
- `tok.isdigit() or (tok[0] in '+-' and tok[1:].isdigit())` correctly handles negative operands.
- Use `int(a / b)` not `a // b` so the truncation matches LeetCode's C++ semantics.

INTERVIEW TRAPS & SANITY CHECKS

Popping in the wrong order. For a non-commutative operator like - or /, the FIRST pop is the RIGHT operand and the second pop is the LEFT one. Swapping them gives correct-looking output on symmetric inputs and wrong answers otherwise.

Sanity check. After processing each token, the stack holds, in order, the values of all complete subexpressions that have not yet been consumed by an operator. An operator therefore always finds its two operands on top.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is the first pop the right operand?

Answer. In reverse Polish notation the operands appear before the operator, left operand first. Popping takes the most recently pushed, which is the right operand, so `a = pop()` must be the SECOND pop. Subtraction and division are not commutative, so this matters immediately.

Q2. How do you distinguish an operand from an operator robustly?

Answer. Check the first character: digits or a leading sign followed by a digit mean operand. `isdigit()` alone fails on `-3`, and a set of operator characters alone is fragile if the token set grows.

Q3. What is the C++-versus-Python semantic trap here?

Answer. Integer division. C++ truncates toward zero, so -7/2 is -3; Python's `//` floors, giving -4. A Python port must use `int(a / b)` to reproduce the judge's expected output.

0155 Min Stack Medium

ESSENCE & BOUNDS

Support push, pop, top, and getMin in constant worst-case time each. Operations arrive in any order with at most 3e6 calls, so worst-case $O(1)$ per operation is required. The main stack holds up to 1e5 elements. Duplicates are legal, so the auxiliary structure must handle equal minima correctly.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Two parallel stacks (BEST) Main stack holds values; min stack holds the running minimum. push(v): push v; push min(min_top, v). pop(): pop BOTH stacks together (lockstep by design). getMin(): peek the min stack's top.

Python (intuition kernel)

```python
self.data:  List[Tuple[int, int]] = []
new_min = val if not self.data else
min(self.data[-1][1], val)
self.data.append((val, new_min))
self.data.pop()
return self.data[-1][0]
return self.data[-1][1]

```

C++ (intuition kernel)

```cpp
stack<int> main_s, mins;
void push(int v) {
 main_s.push(v);
 mins.push(mins.empty() || v < mins.top() ?  v
 :  mins.top());
}
void pop() { main_s.pop(); mins.pop(); }
int top() { return main_s.top(); }
int getMin() { return mins.top(); }

```

Time O(1) for EVERY operation — nothing is scanned.

Space O(n) — two stacks, n entries each.

each element is pushed once and popped at most once; hence the stated bounds.

Single stack of (value, runningMin) pairs One stack; every element is a tuple (v, currentMin). Same logic, one physical stack, but each entry carries a second word.

Python (intuition kernel)

```python
self.stack:  List[int] = [] # the real stack of
values
self.min_stack:  List[int] = [] # running minimum at
each depth
self.stack.append(val)
new_min = val if not self.min_stack else
min(self.min_stack[-1], val)
self.min_stack.append(new_min)
self.stack.pop()
self.min_stack.pop()
return self.stack[-1]

```

C++ (intuition kernel)

```cpp
vector<pair<int,int>> st; // (value, runningMin)
void push(int v) {
 int mn = st.empty() ?  v :  min(v,
st.back().second);
 st.emplace_back(v, mn);
}
void pop() { st.pop_back(); }
int top() { return st.back().first; }
int getMin() { return st.back().second; }

```

Time O(1)

Space O(n)

each element is pushed once and popped at most once; hence the stated bounds.

VISUAL STATE TRACE

```
ops:  push(-2) push(0) push(-3) getMin() pop() top() getMin()
main :  [-2] [-2,0] [-2,0,-3]
mins :  [-2] [-2,-2] [-2,-2,-3]
getMin -> mins.back() = -3
pop() :  main.pop() -> -3 ; mins.pop() -> -2 (restored!)
top() :  main.back() -> 0
getMin -> mins.back() = -2 CORRECT (0 was popped?  no, min was -2)
\{}n invariant:  |mins| == |main| always, and
mins[i] == min(main[0..i]) at every depth i.
Popping is symmetric, so the two stay in lockstep.
alternative:  one stack of (value, runningMin) pairs -
half the push/pop calls, twice the per-node memory.
```

LANGUAGE MECHANICS: PYTHON VS C++

This is the rare problem where Python's memory model is a genuine liability. Two parallel stacks mean two `deques`, and each CPython deque block holds pointers, so a 1e5-element stack costs roughly 800 KB of pointers plus the int objects. C++ `std::stack<int>` over a `std::vector<int>` stores the values inline in one 400 KB block with perfect locality, and the running-min vector is likewise 400 KB – 1.6x denser with no per-element object header and no indirection on read. Pushing a Python int does not allocate when the value is interned or already referenced, but a large computed minimum is a fresh PyLong, so the auxiliary stack allocates on every push where the min changes.

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `vector<pair<int,int>>` holding (value, runningMin) halves the number of push/pop calls at the cost of doubling per-node bytes.
- A single `vector<int>` for values plus a `vector<int>` for mins, with `reserve`, is faster than `std::stack` because there is no adaptor indirection.
- An `std::multiset` with a matching iterator per element gives $O(logn)$ getMin with $O(1)$ push/pop and erase – only worth it if you need ordering too.
- `deque` is the natural stack; a `list` with negative indexing works too and is faster to push/pop but its `pop()` from the end is the same operation.
- Storing `(value, running_min)` tuples halves the calls but allocates a tuple per push – usually a net loss in CPython.
- `min_stack[-1]` is $O(1)$; do not use `min(min_stack)` anywhere in the hot path.

INTERVIEW TRAPS & SANITY CHECKS

Pushing to the auxiliary stack only when the new value is smaller. Then the auxiliary stack no longer has the same depth as the main one, and a pop cannot know how many minima to discard – you must push unconditionally and pop unconditionally.

Sanity check. `|mins| == |main|` and `mins[i] == min(main[0..i])` for every depth i. Push appends `min(v, mins.top())`, pop discards one entry, and the invariant then holds for the shorter stack by construction.

INTUITION CHECK — 3 QUESTIONS

Q1. Why must you push the running min unconditionally?

Answer. The auxiliary stack's depth must mirror the main stack's so that popping once from each stays in sync. Pushing only on decrease leaves fewer entries than there are values, and there is no way to recover how many to discard on pop.

Q2. What is the space cost of the two-stack design versus the pair design?

Answer. Two stacks of n ints is 2n words. A stack of n pairs is also 2n words, so the memory is identical. The difference is call count: two stacks do two push and two pop calls per operation, the pair design does one each but allocates a pair per node.

Q3. Can getMin be $O(1)$ without extra space?

Answer. Not without extra state – you would have to rescan, which is $O(n)$. The whole point of the problem is that the extra structure buys $O(1)$; the space cost is the price.

0739 Daily Temperatures Medium

ESSENCE & BOUNDS

For each day, return how many days you must wait for a strictly warmer temperature; 0 if none exists. 1 <= temperatures.length <= 1e5; 30 <= temperatures[i] <= 100, so the value range is tiny and a 71-slot array could replace the stack, but the stack generalises. A day with no warmer future returns 0, not -1.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Monotonic decreasing stack (BEST) Invariant: stack holds indices whose temperatures are STRICTLY DECREASING from bottom to top. For each day i: while the stack is non-empty and temperatures[i] > temperatures[stack[-1]], the top index has found its first warmer day → answer = i top. Then push i. Every index is pushed and popped exactly once.

Python (intuition kernel)

```python
n = len(temperatures)
answer = [0] * n
st:  List[int] = []
for i, temp in enumerate(temperatures):
 while st and temperatures[st[-1]] < temp:
 answer[st[-1]] = i - st[-1]
 st.pop() # resolved -- remove from the pending stack
 st.append(i)

```

C++ (intuition kernel)

```cpp
for (int i = 0; i < n; ++i) {
 while (!st.empty() && temperatures[i] > temperatures[st.top()]) { answer[st.top()] = i - st.top();
 st.pop(); // resolved -- remove from the pending stack
 }
 st.push(i);

```

Time O(n) **Space** O(n)

each element is pushed once and popped at most once; hence the stated bounds.

Brute force (contrast) For each day scan forward until a warmer day is found.

Python (intuition kernel)

```python
n = len(temperatures)
answer = [0] * n
for i in range(n):
 for j in range(i + 1, n):
 if temperatures[j] > temperatures[i]:
 answer[i] = j - i
 break # stop scanning this i
return answer

```

C++ (intuition kernel)

```cpp
for (int i = 0; i < n; ++i) { for (int j = i + 1; j < n; ++j) { if (temperatures[j] > temperatures[i]) { answer[i] = j - i;
 break; // stop scanning this i
 }
 }

```

Time O(n^2) **Space** O(1)

nested enumeration dominates: $O(n\hat{}k)$ pairs, no pruning; hence the stated bounds.

VISUAL STATE TRACE

```
T = [73, 74, 75, 71, 69, 72, 76, 73]
idx 0 1 2 3 4 5 6 7
i=0 73 st=[] -> push 0 st=[0]
i=1 74 T[0]=73 < 74 -> ans[0]=1-0=1 ; push 1 st=[1]
i=2 75 T[1]=74 < 75 -> ans[1]=1 ; push 2 st=[2]
i=3 71 T[2]=75 > 71 -> not warmer; push 3 st=[2,3]
i=4 69 push 4 st=[2,3,4]
i=5 72 pop 4 (69<72) ans[4]=1
pop 3 (71<72) ans[3]=2
T[2]=75 > 72 stop; push 5 st=[2,5]
i=6 76 pop 5 (72) ans[5]=1
pop 2 (75) ans[2]=4
push 6 st=[6]
i=7 73 push 7 st=[6,7] (no one left to warm them)
\{}n ans = [1,1,4,2,1,1,0,0] (index 6,7 stay 0)
```

LANGUAGE MECHANICS: PYTHON VS C++

Each index is pushed once and popped at most once, so the stack operations total 2n and the amortised cost is $O(1)$. In C++ the stack is a `vector<int>` with a pointer, the `while (T[st.back()] <= T[i])` test is two L1 loads, and popping is one decrement – all register work with the histogram array streaming through L1 once. Python's list-based stack behaves nearly identically for the stack itself, but each iteration allocates an int for `i - st[-1]` before storing it into the answer list, and CPython cannot keep the loop-carried comparison in a register. Realistically the C++ version runs about 25 times faster at n = 1e5, purely from interpreter dispatch.

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- Use <= in the pop condition so equal temperatures also pop; < leaves equal values in the stack, which is still correct but does more work.
- Because temperatures are 30..100, a `vector<int> last(101, -1)` forward pass is also $O(n)$ with no stack.
- `std::vector<int>` plus an index counter beats `std::stack<int>` (which defaults to `std::deque`) on both speed and locality.

- `while st and T[st[-1]] < t:` must be re-tested after each pop; the loop condition alone handles it.
- `st.append(i)` / `st.pop()` on a list is amortised $O(1)$ and faster than `deque` here because there is no block splitting.
- Leaving the answer list pre-filled with zeros avoids a second pass to handle unresolved stack entries.

INTERVIEW TRAPS & SANITY CHECKS

Using < instead of <= in the pop condition, or popping on <=. With <= equal temperatures resolve each other (and the later one correctly reports 0 for a day whose equal-temperature successor is not strictly warmer – check this against the expected output). More commonly the bug is forgetting to initialise unresolved entries to 0.

Sanity check. The stack holds indices in strictly increasing order of position and strictly decreasing order of temperature, and every one of them is still waiting for a warmer day. When a warmer day arrives it resolves all stack entries with a smaller temperature, and each is resolved exactly once with the nearest qualifying day.

INTUITION CHECK — 3 QUESTIONS

Q1. Why does each entry get the NEAREST warmer day rather than just any?

Answer. Because entries are popped the moment a warmer day is seen, and days are processed left to right. The first warmer day encountered after index i is by definition the nearest one, so the answer is correct the instant the pop happens.

Q2. Why must the stack be decreasing rather than increasing?

Answer. An increasing stack would pop on a colder day, but the question asks for warmer. Decreasing order means the stack top is the most recent day that nothing warmer has resolved, so a warmer arrival pops exactly the entries it can answer.

Q3. What happens to indices left in the stack at the end?

Answer. Their answers stay 0, meaning no warmer day ever appeared. That is the correct answer and the reason the result array is initialised to 0 rather than assigned in a final flush.

0853 `Car Fleet` Medium

ESSENCE & BOUNDS

Return the minimum number of cars that arrive at the target. A faster car that catches a slower one ahead merges into that fleet and arrives with it. 1 <= n <= 1e4; 1 <= target <= 1e9; 1 <= speed[i], position[i] <= 1e9. `target - position[i]` can be zero or negative and must be handled as an immediate arrival. The product speed * time can reach about 1e18, so C++ needs `long long`.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Sort + stack of arrival times (BEST) Sort (position, speed) pairs by position DESCENDING (nearest to the target first). Maintain a stack of arrival times that is STRICTLY INCREASING bottom→top. Push t only if t is greater than the current top — otherwise the car merges into the fleet ahead. Answer = stack size.

Python (intuition kernel)

```python
speed: List[int]) -> int:
cars = sorted(zip(position, speed), reverse=True)
times: List[float] = []
for pos, spd in cars:
 t = (target - pos) / spd
 if not times or t > times[-1]:
 times.append(t)
return len(times)

```

C++ (intuition kernel)

```cpp
sort(cars.begin(), cars.end());
vector<double> t;
for (auto& c : cars)
 t.push_back(1.0 * (target - c.first) / c.second);
int fleets = 0;
double slowest = 0;
for (int i = (int)t.size() - 1; i >= 0; --i) {
 if (t[i] > slowest) { ++fleets; slowest = t[i]; }
}
return fleets;

```

Time O(n log n) **Space** O(n)

the sort dominates; the post-sort scan is linear; hence the stated bounds.

Sort + running maximum (no stack) Same sort and walk, but the "stack" shrinks to one variable: the arrival time of the slowest leader so far (the fleet's time). Count a new fleet every time a car's time exceeds it.

Python (intuition kernel)

```python
speed: List[int]) -> int:
cars = sorted(zip(position, speed), reverse=True)
slowest: float = 0.0
fleets = 0
for pos, spd in cars:
 t = (target - pos) / spd
 if t > slowest:
 fleets += 1

```

C++ (intuition kernel)

```cpp
sort(cars.begin(), cars.end());
vector<double> t;
for (auto& c : cars)
 t.push_back(1.0 * (target - c.first) / c.second);
double slow = 0;
for (int i = (int)t.size() - 1; i >= 0; --i)
 if (t[i] > slow) slow = t[i];
// count suffix maxima
int fleets = 0; slow = -1;
for (int i = (int)t.size() - 1; i >= 0; --i)
 if (t[i] > slow) { ++fleets; slow = t[i]; }

```

Time O(n log n)	**Space** O(1) extra

the sort dominates; the post-sort scan is linear; hence the stated bounds.

VISUAL STATE TRACE

```
target = 12, position = [10,8,0,5,3], speed = [2,4,1,1,3]
sorted by position desc:  (10,2) (8,4) (5,1) (3,3) (0,1)
scan from the BACK (closest to target first):
(3,3) time = 9/3 = 3 -> slowest=3, fleets=1
(5,1) time = 7/1 = 7 -> 7 > 3 :  SLOWER, new fleet fleets=2
(8,4) time = 4/4 = 1 -> 1 < 3 :  catches up, joins fleet
(10,2) time = 2/2 = 1 -> 1 < 3 :  catches up, joins fleet
(0,1) time = 12/1 = 12 -> 12 > 3:  SLOWER, new fleet fleets=3
\{}n ANSWER 3
invariant:  'slowest' is the arrival time of the fleet nearest the
target seen so far.  A car arriving LATER must be its own fleet;
a car arriving EARLIER or equal merges behind it.
```

LANGUAGE MECHANICS: PYTHON VS C++

The dominant cost here is the sort of up to 1e4 pairs. C++ sorts `vector<pair<int,int>>` with introsort – the second element makes it lexicographic, which is a comparison on two registers per element, so roughly 1e5 comparisons in well-predicted branches on a 80 KB array that fits in L2. CPython's `list.sort()` uses Timsort with tuple comparison, and tuple `__lt__` goes through a generic rich-comparison slot, so each of the ~1e5 comparisons is an indirect C call. Then the scan divides by `speed[i]`, and Python's `/` on ints produces a float – allocating a float object per car and risking precision issues near 1e18, which is why the integer ceil-division form is mandatory in both languages here.

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `sort` by position descending, or sort ascending and iterate from `n-1` down.
- Integer ceil division `(d + s - 1) / s` avoids the float round-trip and the precision hazard entirely.
- `long long` throughout: `(target - position) * time` intermediate products overflow 32 bits immediately.
- `(d + s - 1) // s` is exact integer ceiling division and much faster than `math.ceil(d / s)`.
- Sort with `key=lambda p: -p[0]` or sort ascending and reverse-index; both avoid a custom comparator, which in Python is far more expensive than a key function.
- `max(slowest, t)` inline instead of `max()` calls keeps the scan free of function dispatch.

INTERVIEW TRAPS & SANITY CHECKS

Scanning from the front (largest position gap first) instead of the back. The nearest car sets the pace for everyone behind it, so you must process from the target backwards. Scanning forward gives the wrong fleet count.

Sanity check. After processing the i closest cars, `slowest` is the arrival time of the rearmost fleet among them. A car with arrival time <= slowest catches that fleet; a car with arrival time > slowest cannot catch anything and forms its own fleet, updating slowest.

INTUITION CHECK — 3 QUESTIONS

Q1. Why scan from the back?

Answer. Fleet membership is determined by arrival time ordering, and the car closest to the target has no car ahead of it, so it always forms its own fleet. Working backwards, each subsequent car either catches that fleet (arrives no later) or overtakes it and becomes the new pace-setter. Scanning forwards has no such anchor.

Q2. What does the running `slowest` value represent?

Answer. The arrival time at the target of the rearmost fleet encountered so far. It is non-decreasing as you scan backwards, which is the invariant that makes the single comparison per car sufficient.

Q3. Why must you use integer ceiling division?

Answer. Two reasons. Exactness: floating point near 1e18 has no room for an integer, so `(d + s - 1) / s` is the only safe form. And correctness of the comparison: a car that arrives 0.0000001 later is a separate fleet, and float rounding would merge them incorrectly.

5. Binary Search

Binary search is not just "halve the array" – it is "find the boundary where a monotone predicate flips". Every problem in this category has a hidden monotone predicate: is `nums[mid] <= target` (704), is the window feasible (875), is the timestamp at most t (981), is the half sorted (153). The transferable skill is naming that predicate and its monotonicity, because once the predicate is monotone the proof is mechanical. Three flavours recur: closed lower-bound search on values, boundary search on a rotated array where one half is always sorted, and binary search over a search SPACE of size up to 1e9 – where the array being searched has nothing to do with the quantity being found.

#	Problem	Level	One-line intent
0004	**Median of Two Sorted Arrays**	Hard	Binary search the partition of the shorter array, not the value.
0033	**Search in Rotated Sorted Array**	Medium	One half is always sorted – use it to discard the other.
0074	**Search a 2D Matrix**	Medium	Treat the matrix as one flat sorted array of m*n elements.
0153	**Find Minimum in Rotated Sorted Array**	Medium	Compare against the right edge; equals means shrink one side.
0704	**Binary Search**	Easy	The loop bounds decide leftmost versus rightmost insertion.
0875	**Koko Eating Bananas**	Medium	Binary search the SPEED; feasibility is monotone.
0981	**Time Based Key-Value Store**	Medium	Per key, a sorted timestamp list; upper-bound then step back.

0004 Median of Two Sorted Arrays Hard

ESSENCE & BOUNDS

Given two sorted arrays of different sizes, return the median of their combined elements in O(log(min(m,n))) time. 1 <= m, n <= 1000; both arrays are sorted ascending and contain only non-negative ints. m + n can be odd (then there is a true middle) or even (then it is the average of the two middles). One array may be a single element.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Merge, then pick the middle (baseline, O(m+n)) Walk both arrays with two pointers, always appending the smaller head — the merge step of merge sort. Then: • odd total → the middle element. • even total → average of the two middle elements. Correct and simple; too slow for the O(log(m+n)) requirement but a perfect ground-truth referee.

Python (intuition kernel)

```python
merged: List[int] = []
i, j = 0, 0
while i < len(nums1) and j < len(nums2):
 if nums1[i] <= nums2[j]:
 merged.append(nums1[i]) # take from nums1.
 i += 1
 else: merged.append(nums2[j]) # take from nums2.
 j += 1

```

C++ (intuition kernel)

```cpp
vector<int> merged;
merged.reserve(m + n);
size_t i = 0, j = 0;
while (i < m && j < n)
 merged.push_back(A[i] <= B[j] ? A[i++] : B[j++]);
while (i < m) merged.push_back(A[i++]);
while (j < n) merged.push_back(B[j++]);
int half = (m + n) / 2;
if ((m + n) % 2) return merged[half];
return (merged[half - 1] + merged[half]) / 2;

```

Time O(m+n) **Space** O(m+n)

Follows directly from the traversal count: $O(m+n)$ time, $O(m+n)$ space.

Binary search on the partition (BEST, O(log min(m,n))) Let A be the SMALLER array (minimizes probes). The median cut is: i elements from A's left, j = (m+n+1)//2 - i from B's left. Probe i with binary search; the cut is valid when A[i-1] <= B[j] AND B[j-1] <= A[i]. If A[i-1] > B[j] the cut is too far right in A → search left; otherwise search right. When balanced, read the median off the boundary elements.

Python (intuition kernel)

```python
A, B = (nums1, nums2) if len(nums1) <= len(nums2) else (nums2, nums1)
total = len(A) + len(B)
left_count = (total + 1) // 2
lo, hi = 0, len(A)
while lo <= hi:
 i = lo + (hi - lo) // 2 # probe the cut in A.
 j = left_count - i
 a_left = A[i - 1] if i >= 1 else float("-inf")

```

C++ (intuition kernel)

```cpp
if (m > n) return median(A, B, n, m, k);
int half = (m + n + 1) / 2;
int lo = max(0, half - n), hi = min(m, half);
while (lo <= hi) {
 int j = (lo + hi) / 2;
 int i = half - j;
 int lA = i > 0 ? A[i-1] : INT_MIN;
 int rA = i < m ? A[i] : INT_MAX;
 int lB = j > 0 ? B[j-1] : INT_MIN;
 int rB = j < n ? B[j] : INT_MAX;
 if (lA <= rB && lB <= rA)
 return (m + n) % 2 ? max(lA, lB)
 : (max(lA, lB) + (long)min(rA, rB)) / 2;
 if (lA > rB) hi = j - 1; else lo = j + 1;
}
return 0;

```

Time O(log(min(m, n))) **Space** O(1)

the interval halves each iteration: $O(logn)$ probes; hence the stated bounds.

VISUAL STATE TRACE

```
A = [1,3] B = [2] combined = [1,2,3] median = 2

search on the SHORTER array (A, m=2). Partition A into

left A[0..j-1] and right A[j..m-1], B into left B[0..k-1]

and right B[k..n-1], with j + k = half = (m+n+1)/2 = 2

j=1 k=1: leftA=[1] rightA=[3] | leftB=[2] rightB=[]

check: leftA <= rightB (1 <= inf) ok

leftB <= rightA (2 <= 3) ok

left count = 1+1 = 2 = half

median = max(leftA, leftB) = 2 CORRECT

j=0 k=2: leftA=[] rightA=[1,3]

leftB=[2] rightB=[]

leftB(2) <= rightA(1)? NO -> j too small, move right

conditions to satisfy:

leftA_max <= rightB_min and leftB_max <= rightA_min

plus left_count == half (or half+1 for even)
```

LANGUAGE MECHANICS: PYTHON VS C++

Binary search over a partition position has at most $\log2(1000) = 10$ iterations, so both languages finish in microseconds and interpreter overhead is invisible. The interesting difference is in the guards: C++ needs explicit `INT_MIN/INT_MAX` sentinels because indexing `A[j-1]` with $j = 0$ is out of bounds, and getting that wrong is undefined behaviour rather than an exception. Python's negative indexing silently wraps around, so `A[j-1]` at $j = 0$ returns the LAST element instead of raising – which is a silent wrong answer rather than a crash. That single asymmetry is the most valuable thing to know when porting this problem.

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- Guard the boundaries explicitly with `j == 0 ? INT_MIN : A[j-1]` and `j == m ? INT_MAX : A[j]` – the alternative is index-0/1 padding with sentinels, which removes the branches entirely.
- `std::max_element` on the two left partitions gives the median side in $O(1)$; no need to merge.
- Always search the shorter array: log(min(m,n)) <= log(max(m,n)), so swapping when m > n is free and halves the worst case.
- Pad conceptually with `float('-inf')` and `float('inf')` instead of branching – Python's arbitrary comparison makes it clean.
- Beware that Python's `A[-1]` is the last element, so the j = 0 case must be special-cased explicitly.
- `while lo <= hi:` with the standard partition search is clearer here than a half-open loop, because the answer needs both neighbours.

INTERVIEW TRAPS & SANITY CHECKS

Searching for the median VALUE with a standard binary search. The median is not necessarily an element of either array, so there is nothing to search for; you must search for the partition position, then take the max of the two left partitions.

Sanity check. The partition is valid iff `max(leftA) <= min(rightB)` and `max(leftB) <= min(rightA)` and the left side holds `ceil((m+n)/2)` elements. Monotonicity says: if `leftB_max > rightA_min`, then j must increase; if `leftA_max > rightB_min`, then j must decrease. That is what makes the search valid.

INTUITION CHECK — 3 QUESTIONS

Q1. Why binary search on the shorter array?

Answer. The partition count is bounded by the shorter length, so the search space is min(m,n)+1 and the depth is log(min(m,n)). Searching the longer array works but is strictly worse, and swapping the pointers first is free.

Q2. What are the two validity conditions and how do they drive the search?

Answer. (1) every element on the left is <= every element on the right, split into the two cross pairs leftA vs rightB and leftB vs rightA. (2) the left side has exactly half the elements. If leftB is too large, j must grow; if leftA is too large, j must shrink. Both conditions are monotone in j, which is what licenses binary search.

Q3. How do you compute the median once the partition is valid?

Answer. If m+n is odd, it is max(leftA, leftB). If even, it is the average of max(leftA, leftB) and min(rightA, rightB). Getting the odd/even indexing right matters: use half = (m+n+1)//2 so the left side absorbs the extra element.

0033 Search in Rotated Sorted Array Medium

ESSENCE & BOUNDS

Return the index of target in a strictly increasing array that was rotated at an unknown pivot, or -1 if absent. 1 <= nums.length <= 5e4; values are distinct integers, so there are no duplicates and exactly one sorted half per step. n = 1 must work, and a single-element array rotated is itself.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Direct BS: find the sorted half, decide, discard Invariant: target, if present, is in [lo, hi]. Per iteration: 1. mid = lo + (hi-lo)//2; hit → return. 2. If nums[lo] <= nums[mid], the LEFT half is sorted: target in [nums[lo], nums[mid]) → hi = mid-1, else lo = mid+1. 3. Else the RIGHT half is sorted: target in (nums[mid], nums[hi]] → lo = mid+1, else hi = mid-1. The <= in step 2 is the off-by-one: with a 2-element window [x, y], mid == lo and the left half [x] IS sorted — test it.

Python (intuition kernel)

```python
lo, hi = 0, len(nums) - 1
while lo <= hi:
 mid = lo + (hi - lo) // 2
 if nums[mid] == target:
 return mid # probe hit -- done.
 if nums[lo] <= nums[mid]:
 if nums[lo] <= target < nums[mid]:
 hi = mid - 1 # target lives in sorted left.

```

C++ (intuition kernel)

```cpp
while (lo <= hi) {
 int mid = lo + (hi - lo) / 2; // overflow-safe
midpoint.
 if (nums[mid] == target) {
 return mid; // probe hit -- done.
 }
 if (nums[lo] <= nums[mid]) {
 if (nums[lo] <= target && target < nums[mid]) {
 hi = mid - 1; // target lives in sorted left.

```

Time O(log n) **Space** O(1)

the sort dominates; the post-sort scan is linear; hence the stated bounds.

Find the pivot, then plain BS in the right segment The pivot is the minimum element — the only place the array drops. Locate it with the 0153 trick: while nums[mid] > nums[hi] the pivot is to the right (lo = mid+1), else hi = mid. After the loop lo IS the pivot. Compare target with nums[0] to pick the segment, then run plain 704-style BS inside it.

Python (intuition kernel)

```python
n = len(nums)
lo, hi = 0, n - 1
while lo < hi:
 mid = lo + (hi - lo) // 2
 if nums[mid] > nums[hi]:
 lo = mid + 1 # pivot is to the right.
 else: hi = mid # mid may BE the pivot.
pivot = lo # index of the minimum.

```

C++ (intuition kernel)

```cpp
while (lo < hi) { int mid = lo + (hi - lo) / 2;
 if (nums[mid] > nums[hi]) {
 lo = mid + 1; // pivot is to the right.
 } else {
 hi = mid; // mid may BE the pivot.
 }

```

Time O(log n) **Space** O(1)

Follows directly from the traversal count: $O(logn)$ time, $O(1)$ space.

VISUAL STATE TRACE

```
nums = [4,5,6,7,0,1,2], target = 0 -> index 4
mid = 3 (value 7). nums[3]=7 > nums[6]=2 -> RIGHT half sorted.
target 0 <= 7 and 0 not in [7,2] (which spans 2..7) -> search LEFT
lo=0 hi=2
mid=1 (value 5). nums[1]=5 > nums[2]=6? no -> LEFT half [4,5] sorted
target 0 < 5 -> search RIGHT: lo=2 hi=2
mid=2 (value 6). 0 < 6 -> search RIGHT: lo=3 > hi=2 -> return -1?
WRONG STEP - corrected: at lo=2 hi=2 mid=2 val=6, 0<6 so
lo=3, loop ends. But target IS at index 4.
the correct trace (target=0):
lo=0 hi=6 mid=3 val=7, right sorted [7,0,1,2] spans 2..7
target 0 < nums[3]=7 -> lo=0 hi=2
mid=1 val=5, left sorted [4,5] ; target 0 < 5 -> lo=2 hi=2
mid=2 val=6 ; nums[2]=6 > nums[0]=4 so right 'sorted'...
with lo==hi==2 just compare: nums[2]==0? no -> -1
KEY: at each step only ONE half is guaranteed sorted, and
the target must lie in it (or exactly at its boundary) unless
the sorted half excludes it - then go the other way.
```

LANGUAGE MECHANICS: PYTHON VS C++

Both do at most log2(50000) = 16 iterations, so this is entirely overhead-dominated. In C++ the two comparisons `(nums[mid] >= nums[lo])` and `(target >= nums[lo])` are register comparisons and the loop is a handful of instructions with one hard-to-predict branch. CPython executes roughly 25 bytecodes per iteration for the same four comparisons and subscripts, and each `nums[mid]` returns a borrowed reference to an existing int with no allocation – so this is pure dispatch overhead. At 16 iterations the absolute difference is a few microseconds, which is precisely why this problem is a correctness problem rather than a performance one.

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- Always compute `mid = lo + (hi - lo) / 2` rather than `(lo + hi) / 2` to avoid overflow, even though the bounds here make it safe.
- Decide the sorted half with `nums[mid] >= nums[lo]`; the mirrored form `nums[mid] <= nums[hi]` is equally valid and equally fast.
- An alternative is to find the pivot with one binary search, then binary search in the guaranteed-sorted segment – two searches but conceptually simpler.

- A `while lo <= hi` loop with `mid = (lo + hi) // 2` is idiomatic; `/` would produce a float and break the indices.
- Python's negative indices do not help here, so `lo` and `hi` must stay genuine in-range bounds.
- For a rotating search, `bisect` is unusable – the array is not globally sorted.

INTERVIEW TRAPS & SANITY CHECKS

Assuming BOTH halves are sorted, or assuming the whole array is sorted. Also, testing the sorted half's range with `nums[lo] <= target <= nums[mid]` on the wrong half. The reliable formulation: identify which half is sorted, then check whether the target lies in its value range; if it does, recurse there, otherwise recurse into the other half.

Sanity check. If `nums[lo..mid]` is the sorted half then `nums[mid] >= nums[lo]`, and the target is in `[nums[lo], nums[mid]]` exactly when it is present in that half; otherwise the answer lies in `[mid+1, hi]`. Exactly one half is sorted because the rotation has a single pivot and all values are distinct.

INTUITION CHECK — 3 QUESTIONS

Q1. How do you know which half is sorted?

Answer. Compare `nums[mid]` with `nums[lo]`. If `nums[mid] >= nums[lo]` the left half is sorted (a rotation only breaks monotonicity across the pivot); otherwise the right half is. With distinct values exactly one of the two holds at every step.

Q2. How do you decide which half to keep?

Answer. Once the sorted half is identified, test whether the target's value falls inside that half's range. If yes, discard the other half; if no, the answer must be in the other half. Because all values are distinct the target cannot be in both, so this decision never loses the answer.

Q3. What changes when duplicates are allowed (problem 153)?

Answer. Both halves can appear unsorted when `nums[mid] == nums[hi]`. Then you cannot tell which half is sorted, so you must shrink by `hi-` and rely on the fact that the element you dropped equals nums[hi], so it cannot be the unique answer. That branch is what makes 153 harder.

0074 Search a 2D Matrix Medium

ESSENCE & BOUNDS

Return true if target appears in the m-by-n integer matrix, whose rows and columns are both sorted ascending, or false otherwise. m, n >= 1 and m*n <= 1e6, so at most log2(1e6) = 20 iterations. Integer overflow in the mid computation is the classic C++ bug: `rows * cols` up to 1e6 is fine, but `(low + high) / 2` computed as an index into a flattened array needs 64-bit arithmetic if the matrix is large.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Flattened-index binary search (BEST, O(log(m-n))) Binary search over [0, m*n). For each probe `mid`, the matrix coordinates are row = mid // n, col = mid % n — pure integer arithmetic, no copy of the matrix. Compare, then discard half of the flattened array exactly like 0704.

Python (intuition kernel)

```python
m, n = len(matrix), len(matrix[0])
lo, hi = 0, m * n - 1
while lo <= hi:
 mid = lo + (hi - lo) // 2 # probe in the FLAT array.
 r, c = divmod(mid, n) # r = mid // n, c = mid % n.
 v = matrix[r][c] # virtual[mid] = matrix[r][c].
 if v == target:
 return True # exact hit.

```

C++ (intuition kernel)

```cpp
while (lo <= hi) {
 int mid = lo + (hi - lo) / 2; // probe in the FLAT array.
 int r = mid / n; // how many full rows fit.
 int c = mid % n; // leftover position in row.
 int v = matrix[r][c]; // virtual[mid] = matrix[r][c].
 if (v == target) { return true; // exact hit.
 }
 if (v < target) {

```

Time O(log(m-n)) **Space** O(1)

the interval halves each iteration: $O(logn)$ probes; hence the stated bounds.

Two-level binary search (O(log m + log n)) Phase 1: binary search the FIRST COLUMN to find the unique row that could hold target (last row with first-value <= target). Phase 2: plain 0704 binary search inside that row. Conceptually cleaner (two 1D searches), one extra boundary check; log m + log n steps vs log(m-n) — asymptotically identical.

Python (intuition kernel)

```python
m, n = len(matrix), len(matrix[0])
lo, hi = 0, m - 1
row = 0
while lo <= hi:
 mid = lo + (hi - lo) // 2
 if matrix[mid][0] <= target:
 row = mid # feasible row -- keep going down.
 lo = mid + 1

```

C++ (intuition kernel)

```cpp
while (lo <= hi) { int mid = lo + (hi - lo) / 2;
 if (matrix[mid][0] <= target) {
 row = mid; // feasible row -- keep going down.
 lo = mid + 1;
 } else {
 hi = mid - 1; // first value too big -- go up.
 }

```

Time O(log m + log n) **Space** O(1)

the interval halves each iteration: $O(logn)$ probes; hence the stated bounds.

VISUAL STATE TRACE

```
m=3 n=4 matrix = [[1, 3, 5, 7], [10,11,16,20], [23,30,34,60]]
flatten:  idx 0..11 -> 1 3 5 7 10 11 16 20 23 30 34 60
lo=0 hi=11 mid=5 -> row 5//4=1 col 5%4=1 -> 11
11 > 23?  target 13 < 11?  no:  13 < 11 false
13 > 11 -> lo = 6
lo=6 hi=11 mid=8 -> row 2 col 0 -> 23 ; 23 > 13 -> hi = 7
lo=6 hi=7 mid=6 -> row 1 col 2 -> 16 ; 16 > 13 -> hi = 6
lo=6 hi=6 mid=6 -> 16 != 13 -> lo = 7 > hi -> false
\{}n the mapping mid -> (mid // n, mid % n) is the whole trick:
no need for a staircase search at all.
```

LANGUAGE MECHANICS: PYTHON VS C++

Twenty iterations means both languages are effectively instant, so the difference is only in the index arithmetic. C++ computes `rows * cols` in a register and uses a 64-bit mid to be safe; the div and mod by `cols` compile to a single hardware division instruction (about 20 cycles on x86) or are strength-reduced by the compiler into a multiply-shift pair. CPython does `mid // cols` and `mid % cols` as two separate boxed operations on Python ints, each of which is a full integer-division routine – so you pay two divisions per iteration instead of one fused divide-and-remainder.

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `long long mid = lo + (hi - lo) / 2` and `int r = mid / cols, c = mid % cols` – the compiler fuses the div/mod pair.
- The staircase search (start at top-right, move left or down) is $O(m+n)$ and uses $O(1)$ space; it is the pedagogically important alternative even though it is asymptotically worse.
- Flatten into a single `vector<long long>` once if you search many times – converts the matrix to a plain binary search.

- `mid // cols, mid % cols` is two separate operations; use `divmod(mid, cols)` if you need both and it is measurably no slower.
- The staircase version is arguably more Pythonic and avoids arithmetic entirely: `while 0 <= r < m and 0 <= c < n`.
- `bisect` on a flattened list is the fastest if you are allowed to allocate the flat list.

INTERVIEW TRAPS & SANITY CHECKS

Using the staircase search's $O(m+n)$ bound when n*m is 1e6 – that is up to 4000 steps versus 20. The flattened binary search is asymptotically strictly better here and is the intended solution.

Sanity check. The matrix sorted row-major is a sorted 1-D array, so the standard binary search invariant applies to the flattened index space: if target exists, its flat index lies in [lo, hi]. The div/mod mapping is a bijection between flat indices and cells, so no candidate is skipped or duplicated.

INTUITION CHECK — 3 QUESTIONS

Q1. Why does the flattened array count as sorted?

Answer. Row-major order visits every row left to right, and the row-max of row i is <= the row-min of row i+1 because the columns are sorted too. So the concatenation of the rows is globally ascending, which is exactly what binary search needs.

Q2. How do you recover (row, col) from a flat index?

Answer. `row = mid // cols`, `col = mid % cols`. Integer division and remainder by the column count decompose the flat index; no cell is visited twice and none is skipped, which is the bijection argument.

Q3. When would the staircase search be preferable?

Answer. If you needed $O(m+n)$ and could not afford to assume both rows and columns are sorted (for example only rows sorted), or if the matrix is streamed and cannot be flattened. Here it is dominated.

0153 Find Minimum in Rotated Sorted Array Medium

ESSENCE & BOUNDS

Return the minimum element of an ascending array that may have been rotated, allowing duplicates. 1 <= nums.length <= 5e4; -1e4 <= nums[i] <= 1e4. Duplicates ARE allowed, which is the entire difference from problem 33. n = 1 returns that element; an all-equal array must terminate.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Binary search: compare mid against the RIGHT edge Invariant: the minimum is inside [lo, hi]. Per iteration: • nums[mid] > nums[hi] → the drop is strictly to the right of mid → lo = mid + 1. • nums[mid] <= nums[hi] → mid lies on the right run; the minimum is mid or to its left → hi = mid. Loop is `lo < hi` because hi = mid keeps the window size >= 1; after the loop lo == hi == the pivot.

Python (intuition kernel)

```python
lo, hi = 0, len(nums) - 1
while lo < hi:
 mid = lo + (hi - lo) // 2 # overflow-safe midpoint.
 if nums[mid] > nums[hi]:
 lo = mid + 1
 else:  hi = mid
return nums[lo] # lo == hi == pivot index.

```

C++ (intuition kernel)

```cpp
while (lo < hi) {
 int mid = lo + (hi - lo) / 2; // overflow-safe midpoint.
 if (nums[mid] > nums[hi]) { lo = mid + 1;
 } else { hi = mid;
 }

```

Time O(log n) **Space** O(1)

the interval halves each iteration: $O(logn)$ probes; hence the stated bounds.

Sort, then take the first (cheating contrast) Sort a copy and read the front. Correct, trivial, $O(nlogn)$ — a reminder of WHY the binary-search version matters: at n = 1e6 the BS does ~20 probes; sorting touches millions of elements.

Python (intuition kernel)

```python
sorted_copy = sorted(nums)
return sorted_copy[0]

```

C++ (intuition kernel)

```cpp
 for (size_t i = 1; i < nums.size(); ++i) { if (nums[i] < expect) { expect = nums[i]; }

```

Time O(n log n) **Space** O(n)

the sort dominates; the post-sort scan is linear; hence the stated bounds.

VISUAL STATE TRACE

```
nums = [1,3,5,1,1,1] hmm use [3,1,3,1,1,3]
lo=0 hi=5 mid=2 (val 3)
nums[hi]=3 == nums[mid]=3 -> AMBIGUOUS: hi- (drop a 3)
lo=0 hi=4
mid=2 (3) vs nums[hi]=1 :  3 > 1 -> RIGHT half holds the min
lo = 3
lo=3 hi=4 mid=3 (1) vs nums[hi]=1 :  equal -> hi-
lo=3 hi=3 -> return nums[3] = 1 CORRECT
the three-way decision at each step:
nums[mid] < nums[hi] -> min is in [lo, mid], hi = mid
nums[mid] > nums[hi] -> min is in [mid+1, hi], lo = mid+1
nums[mid] == nums[hi] -> cannot tell, discard hi
\{}n worst case degrades to O(n) with all-equal input:
every step is the equal branch, hi decrements n times.
```

LANGUAGE MECHANICS: PYTHON VS C++

This is the one binary-search problem in the set whose worst case is genuinely linear, because the equal branch only shrinks by one element per iteration. On an all-equal array of length 5e4 the C++ loop runs 5e4 times with two loads and two compares per iteration – still only about 200 microseconds. Python runs the same 5e4 iterations in bytecode, roughly 20x that, which is the difference between 0.2 ms and 4 ms. Worth noting that the algorithm's *asymptotic* worst case is $O(n)$ while its *typical* case is $O(logn)$, and interviews often probe exactly that distinction.

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `lo = mid + 1` in the greater branch is mandatory; `lo = mid` would loop forever when lo == mid.
- Handle n == 1 up front so the loop is never entered with lo == hi ambiguously.
- For a variant without duplicates you can drop the equal branch entirely and get a strict $O(logn)$ guarantee.
- `while lo < hi:` with `mid = (lo + hi) // 2` and `hi = mid` in the smaller branch – this is the half-open formulation that avoids the final check.
- The equal branch is the only one that costs an extra comparison; ordering the branches so equality is tested last keeps the common case cheap.
- `min()` calls per iteration are pure overhead; use explicit comparisons.

INTERVIEW TRAPS & SANITY CHECKS

Writing `lo = mid` in the `nums[mid] > nums[hi]` branch. If `lo == mid` the loop never progresses, and if the minimum happens to sit at mid the algorithm returns it by luck rather than by proof – either way it is wrong on some inputs.

Sanity check. The minimum lies in [lo, hi] at every step. When `nums[mid] < nums[hi]`, the segment [lo, mid] is sorted ascending so its minimum is `nums[mid]` itself, and the true minimum cannot be to the right of mid because nums[mid] < everything in [mid, hi]. The other two branches follow symmetrically, and the equal branch only discards a duplicate of `nums[hi]`, which cannot be the unique minimum unless every element equals it – in which case any remaining element is the answer.

INTUITION CHECK — 3 QUESTIONS

Q1. Why does the duplicate case make the worst case linear?

Answer. When `nums[mid] == nums[hi]` there is no information to narrow the search: the minimum could be on either side. You can only shrink one position per step, so an array of all equal values forces n iterations. Without duplicates this branch never fires and the bound is strict $O(logn)$.

Q2. Is discarding `hi` always safe?

Answer. Yes: `nums[hi] == nums[mid]`, so `nums[hi]` has an equal twin at or left of mid. If nums[hi] were the unique minimum then so would nums[mid] be, and the answer would be reproducible from the remaining window. Nothing is lost.

Q3. What does the half-open formulation change?

Answer. With `while lo < hi` and `hi = mid` in the smaller branch, the loop exits when lo == hi and `nums[lo]` is the answer with no post-loop comparison. It is one fewer line and removes an entire class of off-by-one, at the cost of a slightly subtler mental model.

0704 Binary Search Easy

ESSENCE & BOUNDS

Return the index of target in a sorted ascending array of distinct integers, or -1 if absent. 1 <= nums.length <= 1e4; -1e4 <= nums[i] <= 1e4; values are distinct, so any hit is unique. An empty array returns -1. The same code with `nums = [1,2,2,2,3]` is the lower-bound variant, so the boundary conditions are the real content.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Classic lo <= hi loop (find ANY hit) Invariant: target (if present) lives in [lo, hi]. Probe mid = lo + (hi-lo)//2. On a hit return immediately; otherwise discard the half that cannot contain target. Loop exits with lo > hi — the window inverted ⇒ element absent.

Python (intuition kernel)

```python
lo, hi = 0, len(nums) - 1
while lo <= hi:
 mid = lo + (hi - lo) // 2
 if nums[mid] == target:
 return mid # exact hit -- done in one probe.
 if nums[mid] < target:
 lo = mid + 1 # discard left half entirely.
 else:

```

C++ (intuition kernel)

```cpp
while (lo <= hi) { int mid = lo + (hi - lo) / 2;
 if (nums[mid] == target) {
 return mid; // exact hit -- done in one probe.
 }
 if (nums[mid] < target) {
 lo = mid + 1; // discard left half entirely.
 } else {
 hi = mid - 1; // discard right half entirely.

```

Time O(log n) **Space** O(1)

Follows directly from the traversal count: $O(logn)$ time, $O(1)$ space.

lo < hi loop (lower bound, then verify) Half-open window [lo, hi) shrinking toward the FIRST index whose value >= target (this is std::lower_bound semantics): nums[mid] >= target → hi = mid (mid may be the answer) nums[mid] < target → lo = mid+1 (mid is definitely too small) Loop ends with lo == hi; one final check distinguishes an exact hit from "absent". This predicate pattern (false\ldots{}false → true\ldots{}true) is the invariant that generalizes to 0033, 0153, 0875, 0981.

Python (intuition kernel)

```python
lo, hi = 0, len(nums)
while lo < hi:
 mid = lo + (hi - lo) // 2
 if nums[mid] >= target:
 hi = mid
 else:  lo = mid + 1
if lo < len(nums) and nums[lo] == target:
 return lo

```

C++ (intuition kernel)

```cpp
while (lo < hi) { int mid = lo + (hi - lo) / 2;
 if (nums[mid] >= target) { hi = mid;
 } else { lo = mid + 1;
 }

```

Time O(log n) **Space** O(1)

the interval halves each iteration: $O(logn)$ probes; hence the stated bounds.

VISUAL STATE TRACE

```
nums = [-1,0,3,5,9,12], target = 9
classic lo<=hi:  lo<hi lower-bound:
lo=0 hi=5 mid=2 (3) lo=0 hi=5 mid=2 (3)
3 < 9 -> lo=3 3 < 9 -> lo=3
lo=3 hi=5 mid=4 (9) lo=3 hi=5 mid=4 (9)
9 == 9 -> return 4 9 <= 9 -> hi=4
DONE lo=3 hi=4 mid=3 (5)
5 < 9 -> lo=4
lo==hi=4; check nums[4]==9 -> return 4
\{}n WHY: lo<=hi allows hi to keep shrinking, so an exact hit
terminates the search at the found index.
lo<hi keeps nums[hi] as a LIVE candidate, so the loop
exits with hi = lower_bound(target).
For RIGHTMOST insertion use upper_bound:  nums[hi] < t
-> lo=hi+1, then verify.
```

LANGUAGE MECHANICS: PYTHON VS C++

Both perform about 14 iterations and finish in microseconds. The genuine difference is in the loop-bound idiom: C++ prefers `lo < hi` with the invariant that `nums[lo..hi]` always contains the answer position as a candidate, which the compiler can use to eliminate bounds checks on `nums[mid]` when the array size is known. Python has no such reasoning, but the `(lo + hi) // 2` computation allocates a new int per iteration for values beyond 256 – 14 small allocations, which is measurable at this scale only because everything else is so cheap. Python also risks `-1` on an empty array if you do not guard `if not nums:  return -1` first, since `hi = len(nums) - 1` becomes -1 and the loop still terminates by luck.

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `std::lower_bound(nums.begin(), nums.end(), t)` is the library lower-bound, $O(logn)$ with better branch prediction.
- `std::binary_search` is the one-liner for the pure existence question, but it gives no index.
- `std::upper_bound` gives the rightmost insertion point – the only change needed between the two variants.
- `bisect.bisect_left(nums, t)` and `bisect_right(nums, t)` are the C-implemented lower and upper bounds – faster than any Python loop.

- `target in nums` first, then `nums.index(target)`, is $O(n)$ and only wins for tiny arrays where the constant dominates.
- Use `lo < hi` rather than `lo <= hi` for the lower-bound form to avoid the final verification step.

INTERVIEW TRAPS & SANITY CHECKS

Mixing the two formulations: using `lo <= hi` with `hi = mid` (instead of `mid - 1`) which can loop forever when `lo == mid`. If you use the inclusive loop, the update must be `hi = mid - 1`.

Sanity check. For the `lo <= hi` form: if target exists, its index lies in [lo, hi]. At each step, comparing `nums[mid]` with target either finds it or eliminates `mid` and exactly one side, since the array is sorted. Termination is guaranteed because either lo increases or hi decreases on every non-terminating iteration.

INTUITION CHECK — 3 QUESTIONS

Q1. Why does `lo < hi` compute a lower bound instead of just finding the element?

Answer. Because `hi` is never decremented to `mid` when the condition holds, `nums[hi]` stays a live candidate. The loop exits when `lo == hi`, and the invariant then forces that single position to be the first index where `nums[i] >= target` – the lower bound. With duplicates that is the leftmost insertion point.

Q2. What single change turns this into upper_bound?

Answer. Change the continue condition from `nums[mid] < target` to `nums[mid] <= target`. Then equal elements are consumed and the loop exits at the first index where `nums[i] > target`, which is the rightmost insertion point. In C++ that is literally `upper_bound` instead of `lower_bound`.

Q3. How do you avoid integer overflow in the midpoint?

Answer. Use `lo + (hi - lo) / 2`. In Python this is irrelevant (arbitrary precision), but in C++ `(lo + hi) / 2` overflows once the values exceed about 2^31, and for a negative-midpoint adjustment loop it can also round the wrong way. The safe form costs nothing.

0875 Koko Eating Bananas Medium

ESSENCE & BOUNDS

Find the minimum eating speed at which Koko finishes all piles within h hours, where each hour she eats one banana from a single pile. 1 <= piles.length <= 1e4; 1 <= piles[i] <= 1e9; 1 <= h <= 1e9. Both search-space endpoints are up to 1e9, which is exactly why a linear sweep over speeds is infeasible. The answer always exists because h can be large enough to eat the largest pile in one hour.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Binary search on speed (BEST, O(n log M)) lo = 1, hi = max(piles). Probe mid and evaluate canFinish(mid) in $O(n)$ by summing ceil(piles[i]/mid): • hours <= h → feasible → optimum is mid or less (hi = mid). • hours > h → too slow → lo = mid + 1. The `lo < hi` loop collapses onto the smallest feasible speed.

Python (intuition kernel)

```python
lo, hi = 1, max(piles)
while lo < hi:
 mid = lo + (hi - lo) // 2 # probe a speed.
 hours = sum((p + mid - 1) // mid for p in piles)
 if hours <= h:
 hi = mid # feasible ${}to$ try slower.
 else: lo = mid + 1 # too slow ${}to$ speed up.
return lo # smallest feasible speed.

```

C++ (intuition kernel)

```cpp
for (size_t i = 0; i < piles.size(); ++i) {
 hi = std::max(hi, piles[i]); // upper bound: 1
pile/hour worst case.

```

Time O(n log M) M = max(piles); log M probes, **Space** O(1)

the interval halves each iteration: $O(logn)$ probes; hence the stated bounds.

Linear sweep of speeds (contrast, O(n·M)) Try every k from 1 to max(piles); return the first feasible one. Identical result, but with max(piles) = 1e9 that is a billion sweeps — the log M probe count is what scales.

Python (intuition kernel)

```python
for k in range(1, max(piles) + 1):
 hours = sum((p + k - 1) // k for p in piles)
 if hours <= h:
 return k # smallest feasible speed.
return max(piles) # unreachable per constraints.

```

C++ (intuition kernel)

```cpp
 for (size_t i = 0; i < piles.size(); ++i) { hi =
std::max(hi, piles[i]);
 2
```

Time O(n-M) **Space** O(1)

each interval enters and leaves the frontier once; hence the stated bounds.

VISUAL STATE TRACE

```
piles = [3,6,7,11], h = 8 -> speed 4
feasibility is MONOTONE in speed:
speed 1 -> ceil sums 27 > 8 FAIL
speed 2 -> 2+3+4+6=15 > 8 FAIL
speed 3 -> 1+2+3+4=10 > 8 FAIL
speed 4 -> 1+2+2+3= 8 <= 8 OK <- smallest feasible
\{}n binary search on [1, max(piles)] = [1, 11]:
lo=1 hi=11 mid=6 feasible(6)=1+1+2+2=6<=8 -> hi=6
lo=1 hi=6 mid=3 feasible(3)=1+2+3+4=10>8 -> lo=4
lo=4 hi=6 mid=5 feasible(5)=1+2+2+3=8<=8 -> hi=5
lo=4 hi=5 mid=4 feasible(4)=8<=8 -> hi=4
lo==hi==4 -> return 4
\{}n the search space is 1e9 wide but the predicate is monotone,
so only log2(1e9) ~ 30 probes are needed.
```

LANGUAGE MECHANICS: PYTHON VS C++

The outer loop runs at most 30 iterations, but the inner feasibility check runs once per pile, so the real work is $O(P \log V)$ with P up to 1e4 and V up to 1e9 – about 3e5 divisions. That is where the languages diverge. C++ computes `(pile + speed - 1) / speed` as a single 64-bit integer division, roughly 20 to 30 cycles, pipelined across independent piles. Python's `//` on ints is also a C-level division but the surrounding `sum(...)` with a generator expression builds an iterator protocol chain and boxes every quotient – roughly 8 times slower at this operation count. The C++ version runs in about 1 ms, the Python version in about 10 ms.

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `(pile + speed - 1) / speed` is exact integer ceiling division; using floating point and `ceil` risks an off-by-one at the exact boundary.
- `long long` is required: `pile + speed - 1` can approach 2e9 and the sum of hours can exceed 32 bits.
- Early exit from the feasibility loop as soon as `hours > h` – turns the worst case into the average case and is the single biggest constant-factor win.
- `(pile + speed - 1) // speed` avoids `math.ceil` and the float round-trip entirely.
- `sum(...)` with a generator is fine; for large piles a `for` loop with an early break is measurably faster because it can bail out.
- `lo, hi = 1, max(piles)` is the tightest possible range; starting `hi` at `sum(piles)` is correct but 1e9 probes wide initially, costing more iterations.

INTERVIEW TRAPS & SANITY CHECKS

Returning the first speed that passes rather than the smallest. With `hi = mid` on success you converge to the minimum, but with `lo = mid` you converge to the maximum feasible speed. Also, using floating-point division and `ceil` makes the exact-boundary case (`pile` an exact multiple of `speed`) fragile.

Sanity check. `lo` is always infeasible-or-the-answer and `hi` is always feasible, so the answer stays inside [lo, hi]. Feasibility is monotone because a faster Koko never takes more hours on any pile, which makes the sum non-increasing in speed – a monotone predicate is exactly what binary search requires.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is feasibility monotone?

Answer. Each pile contributes ceil(pile / speed) hours, which is non-increasing as speed grows, so their sum is non-increasing too. Once it drops to at most h it stays there. A non-monotone predicate would make binary search unsound, which is why the monotonicity argument is the whole proof.

Q2. What is the correct search range?

Answer. [1, max(piles)]. The upper bound works because at speed = max(piles) every pile takes exactly one hour, so the total is P, and P <= 1e4 <= h only sometimes – if h < P you need a higher bound. sum(piles) is always safe. Both are valid; max(piles) is tighter when h is large.

Q3. Why does the early exit matter so much?

Answer. Feasibility usually fails partway through the pile list, so returning early on the first `hours > h` skips most of the work. Without it you pay P divisions per probe always; with it the average is far fewer. It changes nothing about correctness, only the constant.

0981 Time Based Key-Value Store Medium

ESSENCE & BOUNDS

Implement a set(key, value, timestamp) store plus get(key, timestamp) returning the value stored at the largest timestamp not exceeding the query, defaulting to the empty string. Each call has key/value/time of length 3, at most 1e5 calls per test set, so the total timestamp count is bounded and the log factor is small. Timestamps are strictly increasing for a given key, which is what lets the list stay sorted without insertion sort.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

dict[str, list[(ts,val)]] + bisect (BEST) Python : dict maps the key to a list of (timestamp, value) pairs appended in order. get() uses bisect_right on a parallel list of timestamps — returns the INSERTION POINT after any equal entries, i.e. the first index with ts > query; the element before it is the answer. If the insertion point is 0, every record is too new. (C++ equivalent: std::map<string, vector<pair<int,string»> + std::upper_bound — identical semantics.)

Python (intuition kernel)

```python
store = defaultdict(list)

def set(k, v, ts):
 store[k].append((ts, v))

def get(k, ts):
 lst = store.get(k)
 if not lst:  return ''
 i = bisect.bisect_right(lst, (ts, chr(0x10FFFF)))
 return lst[i - 1][1] if i else ''

```

C++ (intuition kernel)

```cpp
if (it == store.end()) { return "";

```

Time set O(1) amortized, get O(log n) **Space** O(records) total across all keys

the interval halves each iteration: $O(logn)$ probes; hence the stated bounds.

Linear scan of the per-key list (contrast) Same storage, but get() walks the list from the END backward and returns the first record with ts <= query. Correct, zero binary search machinery — but $O(n)$ per get; a hot key queried a million times becomes a billion comparisons.

Python (intuition kernel)

```python
store = defaultdict(list)

def set(k, v, ts):
 store[k].append((ts, v))

def get(k, ts):
 ans = ''
 for t, v in store.get(k, []):
 if t <= ts:  ans = v
 return ans

```

C++ (intuition kernel)

```cpp
if (it == store.end()) { return ""; 

```

Time set O(1), get O(n) **Space** O(records)

Follows directly from the traversal count: set $O(1)$, get $O(n)$ time, $O(records)$ space.

VISUAL STATE TRACE

```
ops:  set(a,v1,1) set(a,v2,5) set(b,v3,3)
get(a,6) get(a,2) get(b,4) get(c,9)
store = { a:  [(1,'v1'), (5,'v2')], b:  [(3,'v3')] }
get(a,6):  bisect_right(times, 6) = 2 -> index 1 -> 'v2' OK
get(a,2):  bisect_right(times, 2) = 1 -> index 0 -> 'v1' OK
get(b,4):  bisect_right([3], 4) = 1 -> index 0 -> 'v3' OK
get(c,9):  key missing -> ''
\{}n why bisect_right and not bisect_left:
the answer must include the EXACT timestamp, so we want
the first index strictly greater than t, then step back.
Using bisect_left would miss an exact match at the last slot.
\{}n invariant:  each list is strictly increasing in timestamp,
so index bisect_right(t)-1 is the unique latest <= t.
```

LANGUAGE MECHANICS: PYTHON VS C++

This is a case where the standard library wins outright. `std::map<string, vector<pair<int,string>>>` plus `lower_bound` is a tree walk of about 17 pointer dereferences per query, and those are cache misses, so the query cost is dominated by memory latency rather than computation. CPython's `dict` lookup hashes the key with SipHash (a strong, slow hash) but then gets a single $O(1)$ array probe, and the per-key list search can be `bisect.bisect_right`, which runs entirely inside CPython's C code with no bytecode at all. So for short lists and few keys, Python is genuinely competitive or faster despite its interpreter, because it avoids the pointer chasing that dominates the C++ tree.

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `std::map` for ordered keys, `std::unordered_map` for $O(1)$ – unordered wins unless you need ordered iteration.
- `std::vector<std::pair<int,std::string>>` per key with `lower_bound` and a `prev()` call; `upper_bound` then `-` is the same thing.
- `std::string_view` for the key avoids copying the key into the map on every `set`.

- `collections.defaultdict(list)` removes the `if key not in store` branch on `set`.
- `bisect_right(times, t) - 1` is the entire query; guard `idx < 0` to return the empty string.
- Store timestamps and values as parallel lists only if you care about tuple allocation; a list of tuples is clearer and the difference is negligible at this scale.

INTERVIEW TRAPS & SANITY CHECKS

Using `bisect_left` instead of `bisect_right`. If a value was set at exactly the query timestamp, `bisect_left` returns that index but stepping back once lands on the previous element and returns a stale value. `bisect_right` then decrement is the correct pairing.

Sanity check. For each key, timestamps are strictly increasing, so `bisect_right(times, t)` returns the count of timestamps <= t; subtracting one gives the index of the largest such timestamp, which is the answer's timestamp, or -1 if none exists.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is `bisect_right` the right primitive?

Answer. We want the LAST timestamp <= t, which is at index `bisect_right(t) - 1`. `bisect_right` returns the first index of a value greater than t, so subtracting one lands exactly on the greatest value not exceeding t. `bisect_left` would skip a value equal to t.

Q2. What if two sets use the same timestamp for one key?

Answer. The problem guarantees strictly increasing timestamps per key, so this cannot happen. If it could, you would need to decide whether the later set wins, which would change `set` from append to a replace-if-newer.

Q3. What is the space complexity across all calls?

Answer. Space is proportional to the total number of sets, since each stores one (timestamp, value) pair. Time per set is $O(logm)$ for the append (amortised $O(1)$ since timestamps only increase) and per get is $O(logm)$ for the binary search plus $O(1)$ for the hash lookup on the key.

6. Linked List

#	Problem	Level	One-line intent
0002	**Add Two Numbers**	Medium	Digit-by-digit addition from the tail, carrying forward.
0019	**Remove Nth Node From End of List**	Medium	Two pointers at distance n make the gap land on the target.
0021	**Merge Two Sorted Lists**	Easy	Dummy head plus a tail pointer makes the splice trivial.
0023	**Merge K Sorted Lists**	Hard	Either divide and conquer, or a k-way heap of the heads.
0025	**Reverse Nodes in k-Group**	Hard	Reverse each group of k; the final short group is left alone.
0138	**Copy List with Random Pointer**	Medium	Interleave new nodes so random pointers resolve in one pass.
0141	**Linked List Cycle**	Easy	Floyd: one fast pointer eventually overtakes the slow one.
0143	**Reorder List**	Medium	Split at the middle, reverse the back half, then interleave.
0146	**LRU Cache**	Medium	Hash map for $O(1)$ lookup, doubly linked list for $O(1)$ reordering.
0206	**Reverse Linked List**	Easy	Three pointers: prev, current, next – and never lose next.
0287	**Find the Duplicate Number**	Medium	Floyd again: the entry of the functional cycle is the duplicate.

0002 Add Two Numbers Medium

ESSENCE & BOUNDS

Add two numbers given as reversed digit lists and return the sum as a reversed list. Each list has at least one digit, up to about 100 digits, so the result may exceed any fixed integer type – this is the problem where C++ must avoid converting to int. Carries can chain past the leading digit, requiring a final append.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Iterative with dummy head + carry (BEST) Walk both lists while either lives OR carry is non-zero; append digit, keep carry.

Python (intuition kernel)

```python
dummy = ListNode() # sentinel: no first-append case
tail = dummy
carry = 0
while l1 is not None or l2 is not None or carry != 0:
 total = carry
 if l1 is not None:
 total += l1.val
 l1 = l1.next

```

C++ (intuition kernel)

```cpp
while (l1 != nullptr || l2 != nullptr || carry != 0)
{ int sum = carry;
 if (l1 != nullptr) { sum += l1->val; l1 = l1->next;
}
 if (l2 != nullptr) { sum += l2->val; l2 = l2->next;
}
 carry = sum / 10; // overflow moves to the next
column
 tail->next = new ListNode(sum % 10); // one result
digit
 tail = tail->next;

```

Time O(max(n, m)) **Space** O(1)

Follows directly from the traversal count: O(max(n, m)) time, $O(1)$ space.

Recursive Same column arithmetic on the call stack. Base case: both lists empty and carry == 0 → None.

Python (intuition kernel)

```python
return self._add(l1, l2, 0)
if a is None and b is None and carry == 0:
total = carry # null nodes contribute 0
if a is not None:
 total += a.val
if b is not None:
 total += b.val
node = ListNode(total % 10)

```

C++ (intuition kernel)

```cpp
 if (a == nullptr && b == nullptr && carry == 0) {
return nullptr;

```

Time O(max(n, m)) **Space** O(max(n, m))

Follows directly from the traversal count: O(max(n, m)) time, O(max(n, m)) space.

VISUAL STATE TRACE

```
l1 = [2,4,3] (342) l2 = [5,6,4] (465) -> [7,0,8] (807)
l1:  2 -> 4 -> 3 -> NULL
l2:  5 -> 6 -> 4 -> NULL
step a=2 b=5 carry=0 -> sum 7 emit 7 carry 0
step a=4 b=6 carry=0 -> sum 10 emit 0 carry 1
step a=3 b=4 carry=1 -> sum 8 emit 8 carry 0
step a=NULL b=NULL c=0 -> emit nothing
(a NULL, b=7, c=1 -> emit 8, then tail b)
invariant:  carry in {0,1}; sum <= 9+9+1 = 19 < 20
```

LANGUAGE MECHANICS: PYTHON VS C++

Human arithmetic is done right-to-left (least significant first), which is EXACTLY the layout of these lists — the carry never forces you to walk back, and the result is produced in final order without reversing. If the lists were stored most-significant first, you would need a stack or a reverse step. This problem is the canonical "linked list = digit stream" pattern used in arbitrary-precision arithmetic: Python ints, Java BigInteger and C++ boost::multiprecision all store digits in chunks and add them with exactly this carry loop, on arrays instead of nodes.
==========
/ #include <vector> // std::vector — test data #include <chrono> // std::chrono — timing #include <iostream> // std::cout — output #include <sys/resource.h> // getrusage — peak RAM #include "list_node.h" // our header: ListNode, buildList

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- Reusing the input nodes saves an allocation per digit; a fresh vector is simpler and avoids aliasing bugs.
- `long long` is wrong here – 100 digits overflows it. Keep `int` per digit and accumulate the carry manually.
- `0` as the terminal sentinel avoids the None-check in the loop condition entirely.
- `divmod(a + b + carry, 10)` returns (digit, carry) in one call.

INTERVIEW TRAPS & SANITY CHECKS

Stopping when either pointer is null instead of when both are null. With l1 = [5] and l2 = [5,4] the correct answer is [9,4], and stopping at the first null loses the 4.

Sanity check. After k iterations, `carry` is exactly the carry out of the lowest k digit positions, so each emitted digit is the correct digit of the true sum.

INTUITION CHECK — 3 QUESTIONS

Q1. Why are the digits reversed in the input?

Answer. Because then the least significant digit is at the head, which lets a single forward traversal perform the carry propagation in the correct order with no reversing.

Q2. When is the final `while (carry)` loop needed?

Answer. When one input is exhausted while a nonzero carry remains – e.g. [9] plus [1] gives digit 0 and carry 1, so the result is [0,1]. Omitting that tail is the classic bug.

Q3. What is the space complexity, and can it be $O(1)$?

Answer. $O(1)$ auxiliary if you reuse the input nodes, O(max(m,n)) if you build a new list. The in-place version is the more interesting answer because it also avoids the allocator.

0019 Remove Nth Node From End of List Medium

ESSENCE & BOUNDS

Remove the n-th node from the end of a linked list, counting the last node as n = 1, in one pass. n >= 1 and n <= the list length, so the target always exists. n = 1 means removing the tail. A one-node list with n = 1 must become empty, which is the boundary that forces the dummy-head trick.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

One pass: dummy + fixed-gap two pointers (BEST) right walks n steps ahead; both slide until right hits None; left.next is the victim; left.next = left.next.next deletes.

Python (intuition kernel)

```python
dummy = ListNode(0, head) # absorbs the delete-head case
left:  Optional[ListNode] = dummy
right:  Optional[ListNode] = dummy
for _ in range(n):
 right = right.next if right else None
while right is not None and right.next is not None:
 left = left.next if left else None
 right = right.next

```

C++ (intuition kernel)

```cpp
for (int i = 0; i < n; ++i) { right = right->next;

```

Time O(n) **Space** O(1)

Follows directly from the traversal count: $O(n)$ time, $O(1)$ space.

Two passes: count, then delete Pass 1 counts L nodes; victim is node (L n + 1) from start; walk to its predecessor and skip it.

Python (intuition kernel)

```python
length = 0
cur = head
while cur is not None:
 length += 1
 cur = cur.next
dummy = ListNode(0, head)
cur = dummy
for _ in range(length - n):

```

C++ (intuition kernel)

```cpp
for (ListNode* cur = head; cur != nullptr; cur = cur->next) { ++length;

```

Time O(n) **Space** O(1)

Follows directly from the traversal count: $O(n)$ time, $O(1)$ space.

VISUAL STATE TRACE

```
list = [1,2,3,4,5], n = 2 -> [1,2,3,5]
dummy -> 1 -> 2 -> 3 -> 4 -> 5 -> NULL
^f ^s (gap = 2)
advance s by n -> s at 4; advance f to 2 (one before target)
advance both:  f=3, s=5
unlink:  f.next = f.next.next (removes 4)
gap arithmetic:  after the first loop the distance fast-slow
is exactly n nodes.  After n+1 more single steps, slow is on
the n-th from the end and fast is one past it.
n = 1:  dummy protects the case where slow would be head
```

LANGUAGE MECHANICS: PYTHON VS C++

A singly linked list can only delete a node if you can SEE its predecessor — that is why we stop at left->next, never at the victim itself. The one tricky case is deleting the head (no predecessor exists): the dummy node makes the head a normal node with a normal predecessor. This "dummy absorbs the edge case" idea is the same trick that powers 0021 (merge) and 0025 (reverse k-group). Memory hygiene note: in a real runtime you also free the skipped node (we do, then rebuild from test data). Two-pointer gap windows appear in real systems: network ring buffers keep a read pointer exactly K bytes behind a write pointer, i.e. "the n-from-the-end element".
==========
/ #include <vector> // std::vector — test data #include <chrono> // std::chrono — timing #include <iostream> // std::cout — output #include <sys/resource.h> // getrusage — peak RAM #include "list_node.h" // our header: ListNode, buildList

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `ListNode dummy(0); dummy.next = head;` avoids the head-is-null and head-is-target special cases.
- An iterative two-pass (count length, then walk) is simpler to reason about but touches every node twice.

- A sentinel object with a `.next` attribute is the standard trick; it removes both boundary branches.
- `fast.next is not None` in the guard means the guarantee `n <= length` is doing real work.

INTERVIEW TRAPS & SANITY CHECKS

Using two separate loops (one to compute length, one to find the target) without a dummy head, then failing when the head itself must be removed – `head = head.next` must be paired with returning the new head rather than modifying through a local pointer.

Sanity check. Throughout, `fast` is exactly n nodes ahead of `slow`. After the two advance phases, `slow` is the predecessor of the n-th-from-last node and `fast` is the n-th-from-last node.

INTUITION CHECK — 3 QUESTIONS

Q1. Why a dummy head?

Answer. Because the node to remove can be the head itself. With a dummy in front, the target always has a predecessor, so the unlink is always `prev.next = prev.next.next` with no special case.

Q2. What exactly does the distance-n trick buy?

Answer. One pass instead of two. Because the problem guarantees n <= length, you never need to measure the length first – you simply lag the fast pointer by n nodes.

Q3. How would you solve it without the guarantee n <= length?

Answer. You cannot: the slow pointer would run off the end. The standard follow-up asks for a return value indicating whether the node existed, which requires a length pass first.

0021 Merge Two Sorted Lists Easy

ESSENCE & BOUNDS

Merge two ascending sorted lists into one ascending sorted list, reusing existing nodes. 0 <= list1.length, list2.length <= 50, and either list may be null, so the empty cases must return the other list rather than crash. Both lists are already sorted, which is what allows the $O(1)$-per-step decision.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Iterative with a dummy head (BEST) dummy = sentinel before the result head → no "first append?" branch. While both lists live, attach the smaller head; then splice the leftover chain with one pointer assignment.

Python (intuition kernel)

```python
dummy = ListNode() # sentinel: no first-append branch
tail = dummy # tail of the result chain
while l1 is not None and l2 is not None:
 if l1.val <= l2.val:
 tail.next = l1 # l1's head wins this round
 l1 = l1.next
 else: tail.next = l2 # l2's head wins
 l2 = l2.next

```

C++ (intuition kernel)

```cpp
while (l1 != nullptr && l2 != nullptr) { if (l1->val <= l2->val) {
 tail->next = l1; // l1's head wins this round
 l1 = l1->next; // advance only l1
 } else { tail->next = l2; // l2's head wins
 l2 = l2->next;
 }
 tail = tail->next; // result chain grows by one

```

Time O(n + m) **Space** O(1)

Follows directly from the traversal count: $O(n+m)$ time, $O(1)$ space.

Recursive Empty list → the other list IS the answer. Otherwise the smaller head leads and its tail merges with the other list recursively.

Python (intuition kernel)

```python
if l1 is None:
 return l2 # empty $\{}to$ the other list IS the result
if l2 is None:
 return l1
if l1.val <= l2.val:
 l1.next = self.mergeTwoLists(l1.next, l2) # l1 leads
 return l1
l2.next = self.mergeTwoLists(l1, l2.next) # l2 leads

```

C++ (intuition kernel)

```cpp
if (l1->val <= l2->val) { l1->next = mergeTwoLists(l1->next, l2);
 return l1;

```

Time O(n + m) **Space** O(n + m) stack

Follows directly from the traversal count: $O(n+m)$ time, $O(n+m)$ stack space.

VISUAL STATE TRACE

```
l1 = [1,3,5] l2 = [2,4,6]
dummy -> tail = dummy
a=1 b=2 :  1 <= 2 -> tail.next = a ; tail = 1 ; a = 3
a=3 b=2 :  3 > 2 -> tail.next = b ; tail = 2 ; b = 4
a=3 b=4 :  3 <= 4 -> tail.next = a ; tail = 3 ; a = 5
a=5 b=4 :  5 > 4 -> tail.next = b ; tail = 4 ; b = 6
a=5 b=6 :  5 <= 6 -> tail.next = a ; tail = 5 ; a = NULL
a=NULL : tail.next = b (6) -> done
tail.next = NULL (must set!)
invariant:  tail is always the LAST node of the merged prefix,
so tail.next = x is a single pointer store, not a traversal.
```

LANGUAGE MECHANICS: PYTHON VS C++

Building a result chain with `tail = tail->next` demands an initial tail. Without a dummy you must write `if (head==nullptr) head = node; else tail->next = node;` on EVERY append — a branch paid $O(n)$ times. The dummy removes the branch: dummy = new Node; tail = dummy; append always via tail->next list_node.h's buildList, every insert-at-front solution, and problem 0025 (reverse k-group) all rely on this trick. The dummy is scaffolding: discard it at the end (delete) and return dummy->next. Real-world: kernel linked lists (struct list_head) embed sentinel heads exactly like this. ========== / #include <vector> // std::vector — test data #include <chrono> // std::chrono — timing #include <iostream> // std::cout — output #include <sys/resource.h> // getrusage — peak RAM #include "list_node.h" // our header: ListNode, buildList, printList

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `ListNode* tail = &dummy;` with a stack-allocated dummy avoids a heap allocation for the sentinel.
- The recursive version is 4 lines and is arguably clearer, at $O(m+n)$ stack depth.

- A sentinel `ListNode(0)` object removes the `if not list1` branches.
- Chasing `tail = tail.next` after each append is the $O(1)$ trick that separates a good answer from the $O(n^2)$ one.

INTERVIEW TRAPS & SANITY CHECKS

Appending by walking to the end on every insertion, which is $O(n^2)$. Maintain a tail pointer instead – that single change is the whole difference between the two complexities.

Sanity check. After each iteration, the nodes up to `tail` form the sorted merge of the prefixes already consumed, and `tail` is the last of them. Appending the smaller head preserves sortedness.

INTUITION CHECK — 3 QUESTIONS

Q1. Why the <= versus < choice at the head comparison?

Answer. Either is correct for correctness, but <= and < differ in which list's nodes are preferred when heads are equal. Since all nodes must be kept, both are fine; only stability across a later dedupe pass would care.

Q2. How does the dummy head simplify the termination?

Answer. You splice the entire non-empty remainder with a single `tail.next = a ? a : b`, then set `tail.next = NULL`. Without a sentinel you'd need to special-case an initially empty list.

Q3. What if the lists are not sorted?

Answer. The $O(1)$ decision breaks – you cannot know which head is smaller without scanning ahead. Merging then costs $O(m+n)$ only because sortedness makes the comparison a single node read.

0023 Merge K Sorted Lists Hard

ESSENCE & BOUNDS

Merge k ascending sorted lists into one ascending sorted list, reusing nodes where possible. 1 <= k <= 1e4 with a combined total of 1e4 nodes, so heap operations number 1e4 and log k is about 14. An empty input array must return null.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Min-heap of k heads (BEST) Push all k heads (value, list index). Pop the smallest, append, push its successor back. Heap never exceeds k entries.

Python (intuition kernel)

```python
heap: list[tuple[int, int, ListNode]] = []
for i, head in enumerate(lists):
 if head is not None:
 heapq.heappush(heap, (head.val, i, head))
dummy = ListNode() # result chain sentinel
tail = dummy
while heap:
 _, i, node = heapq.heappop(heap) # smallest head wins

```

C++ (intuition kernel)

```cpp
for (int i = 0; i < static_cast<int>(lists.size()); ++i) { if (lists[i] != nullptr) { heap.push({lists[i]->val, i});
 }

```

Time O(n-log k) **Space** O(k)

one push (and at most one pop) per element; hence the stated bounds.

Sequential two-list merges result = lists[0]; fold each later list in with the 0021 merge. A node from list i is re-merged i times.

Python (intuition kernel)

```python
result: Optional[ListNode] = None
for head in lists:
 result = self._merge_two(result, head) # fold one in
return result

```

C++ (intuition kernel)

```cpp
while (a != nullptr && b != nullptr) { if (a->val <= b->val) { tail->next = a;
 a = a->next;
 } else { tail->next = b;
 b = b->next;
 }
 tail = tail->next;

```

Time O(n-k) **Space** O(1)

Follows directly from the traversal count: $O(n-k)$ time, $O(1)$ space.

VISUAL STATE TRACE

```
lists = [[1,4,5],[1,3,4],[2,6]] -> [1,1,2,3,4,4,5,6]
HEAP approach: heap of (value, listIndex, node)
init: 1(L0) 1(L1) 2(L2)
pop 1(L0) -> emit; push L0.next = 4
pop 1(L1) -> emit; push L1.next = 3
pop 2(L2) -> emit; push L2.next = 6
pop 3(L1) -> emit; push L1.next = 4
pop 4(L0) -> emit; L0 exhausted
pop 4(L1) -> emit; push 1... L1 exhausted
pop 5(L0) -> emit; L0 exhausted
pop 6(L2) -> emit
total: 8 pops, each O(log k)
DIVIDE AND CONQUER:
merge(list[0..k/2]) with merge(list[k/2..k])
T(k) = 2T(k/2) + O(N) -> O(N log k)
```

LANGUAGE MECHANICS: PYTHON VS C++

Sequential merging pays a hidden tax: a node from list k1 is touched k1 times (re-merged once per earlier list). The heap touches every node exactly once. The heap here is a PRIORITY QUEUE keyed by node value — the standard tool whenever the "best next candidate" must be found among many streams: k-way merge is the internal engine of external sorts, database merge joins, and log shard aggregation. In C++, std::priority_queue (heap) with a custom comparator; entries are (value, listId) so equal values don't compare pointers.
==========
/ #include <vector> // std::vector — the array of lists #include <queue> // std::priority_queue — min-heap for heads #include <functional> // std::greater — ordering for the heap #include <chrono> // std::chrono — timing #include <iostream> // std::cout — output #include <sys/resource.h> // getrusage — peak RAM #include "list_node.h" // our header: ListNode, buildList

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `priority_queue<int, vector<int>, cmp>` with an index array beats storing node pointers for cache locality.
- Divide and conquer with an explicit vector of heads is often simpler to implement correctly than a comparator object.

- `heapq` is min-only; push `(value, listIndex, node)` tuples and pop all three together.
- `heapq.merge(*lists)` is a lazy iterator that does exactly this and is a strong interview answer.

INTERVIEW TRAPS & SANITY CHECKS

Rebuilding the result list from scratch inside the loop instead of relinking heads, which loses the $O(1)$-per-node property and allocates a node per element.

Sanity check. The heap always contains exactly the smallest unconsumed node of each non-exhausted list. Its minimum is therefore the smallest unconsumed node overall, which is the next output element.

INTUITION CHECK — 3 QUESTIONS

Q1. Which is asymptotically better, the heap or divide and conquer?

Answer. Both are $O(N \log k)$. The heap makes one pass with a priority queue; divide and conquer makes log k passes of pairwise merges. The heap uses $O(k)$ space, D&C uses $O(\log k)$ stack.

Q2. Why store (value, listIndex, node) rather than just values?

Answer. Because you need to know which list's next node to advance after a pop. Encoding all three in the tuple means the heap comparison uses value first and the tuple supplies the bookkeeping.

Q3. What does k = 1 degenerate to?

Answer. The loop pops the single head, pushes nothing (the list is exhausted), and the heap empties – returning that one list unchanged, which is correct.

0025 `Reverse Nodes in k-Group` Hard

ESSENCE & BOUNDS

Reverse the order of nodes in groups of k, leaving the last group untouched if it has fewer than k nodes. 1 <= k <= n <= 5000. The final group being short is not an edge case but the normal case, and getting it wrong is the single most common failure.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Iterative, dummy + anchored segment flips (BEST) Peek k nodes ahead (fewer → stop, tail untouched); flip exactly k nodes after cursor; sew tail→next-group and cursor→new-head; cursor jumps to the block's old head (now its tail).

Python (intuition kernel)

```python
dummy = ListNode(0, head) # uniform sewing for group #1
cursor: Optional[ListNode] = dummy # node BEFORE current group
while True:
 probe = cursor
 for _ in range(k):
 probe = probe.next if probe else None
 if probe is None:
 return dummy.next # leftover tail stays as-is

```

C++ (intuition kernel)

```cpp
while (true) { ListNode* probe = cursor;
 for (int i = 0; i < k; ++i) { probe = probe->next;
 if (probe == nullptr) { ListNode* result = dummy->next;
 delete dummy;
 return result;
 }
 }
 ListNode* oldHead = cursor->next;

```

Time O(n) **Space** O(1)

Follows directly from the traversal count: $O(n)$ time, $O(1)$ space.

Recursive per group Reverse first k nodes; recurse on the remainder; sew this group's tail to the next group's reversed head.

Python (intuition kernel)

```python
probe = head
for _ in range(k):
 if probe is None:
 return head # fewer than k $\{}to$ keep as-is
 probe = probe.next
prev: Optional[ListNode] = None
cur = head
for _ in range(k):

```

C++ (intuition kernel)

```cpp
for (int i = 0; i < k; ++i) { if (probe == nullptr) {
 return head; // fewer than k $\{}to$ this tail stays as-is
 }
 probe = probe->next;

```

Time O(n) **Space** O(n/k) stack

Follows directly from the traversal count: $O(n)$ time, $O(n/k)$ stack space.

VISUAL STATE TRACE

```
list [1,2,3,4,5], k = 2 -> [2,1,4,3,5]
group [1,2] -> [2,1] FULL, reversed
group [3,4] -> [4,3] FULL, reversed
group [5] -> [5] SHORT, kept
iteration guard:
for i in range(0, n - k + 1, k):  <- stops early
so a short tail never enters the loop
reverse-in-place [a,b,c] -> [c,b,a]
prev = NULL, cur = groupFirst
while cur:  nxt = cur.next; cur.next = prev
prev = cur; cur = nxt
the bug:  using range(0, n, k) includes the short tail and
reverses it, giving [2,1,4,3] -> and for k=2,n=5 -> [2,1,4,3,5]
is right but [5] alone reversed is still [5]...  the real failure
is when the short tail has 2 nodes and k = 3.
```

LANGUAGE MECHANICS: PYTHON VS C++

Iterative k-group reversal is the "between markers" version of the flip: instead of flipping until null, flip exactly k nodes. The three anchors are: cursor — node before the segment (never moves while reversing inside); oldHead = cursor->next (first node of the segment, becomes the segment's TAIL); prev/cur — the standard 0206 sliders doing the flips. After k flips, cur points at the next segment's first node, and oldHead's ->next must be set to it (sewing the chain back). This "reverse exactly K between anchors" pattern is the same one used by doubly-linked-page list manipulation and by block-wise transforms in databases. Get the anchor invariants right here and 0206/0143/0025 become one skill.
==========
/ #include <vector> // std::vector — test data #include <chrono> // std::chrono — timing #include <iostream> // std::cout — output #include <sys/resource.h> // getrusage — peak RAM #include "list_node.h" // our header: ListNode, buildList

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- Precompute the node vector once, then reverse slices in place – $O(n)$ space but far simpler to get right.
- `reverse(begin + i, min(begin + i + k, end))` per group needs the vector form.
- Slice-and-assign on a node array (`arr[i:i+k] = arr[i:i+k][::-1]`) is the shortest correct answer.
- Three pointers on the array form: `arr[i:j] = reversed(arr[i:j])`.

INTERVIEW TRAPS & SANITY CHECKS

Reversing the final incomplete group. The loop bound must be `n - k + 1`, or equivalently the group end must be clamped to `min(i + k, n)` and skipped if shorter than k.

Sanity check. Before group i starts, the first i nodes are already correctly reversed and wired; the group being processed is exactly nodes [i, i+k) when that range is full.

INTUITION CHECK — 3 QUESTIONS

Q1. How do you detect a short final group without counting?

Answer. Either bound the loop by `n - k + 1`, or check whether the group's predecessor list still has k nodes. The loop bound is cleaner and makes the condition impossible to violate.

Q2. Why is a dummy head needed here but not in a plain reverse?

Answer. Because you must reconnect the reversed groups to their predecessors. The dummy gives every group a stable predecessor to write into, and it also makes returning an empty list trivial.

Q3. What does the array-based approach cost?

Answer. $O(n)$ extra space for the node array, in exchange for turning pointer surgery into index arithmetic. That trade is usually worth taking in an interview and worth naming explicitly.

0138 Copy List with Random Pointer Medium

ESSENCE & BOUNDS

Deep-copy a linked list where each node also has a random pointer to any node or null, with no node in the copy pointing into the original. The list is non-empty and each node has `next` and `random`. The aliasing is the whole difficulty: a naive DFS with a visited map handles it, but costs $O(n)$ extra space and recursion depth.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Interleaving / weave — O(1) space (BEST) Pass 1: insert a clone after every node (old→new→old→new\ldots{}). Pass 2: clone.random = old.random.next (the clone beside it). Pass 3: unweave: restore old.next, chain the clones.

Python (intuition kernel)

```python
if head is None:
cur = head
while cur is not None:
 cur.next = Node(cur.val, cur.next) # clone born beside it
 cur = cur.next.next # jump to next old node
cur = head
while cur is not None:
 if cur.next is not None:

```

C++ (intuition kernel)

```cpp
if (head == nullptr) { return nullptr;

```

Time O(n) **Space** O(1) extra

Follows directly from the traversal count: $O(n)$ time, $O(1)$ extra space.

Hash map old $\to$ new Pass 1: create all clones, record old→new. Pass 2: wire next and random through the map.

Python (intuition kernel)

```python
if head is None:
old_to_new: dict[Node, Node] = {}
cur = head
while cur is not None:
 old_to_new[cur] = Node(cur.val)
 cur = cur.next
cur = head
while cur is not None:

```

C++ (intuition kernel)

```cpp
if (head == nullptr) { return nullptr;

```

Time O(n) **Space** O(n)

one $O(1)$ hash probe per element, no rescans; hence the stated bounds.

VISUAL STATE TRACE

```
original:  A -> B -> C -> NULL
A.random = C, B.random = A
PASS 1 weave:  A' -> B' -> C'
A -> A' -> B -> B' -> C -> C'
(copies inserted right after originals)
PASS 2 random:  for each original X with copy X' = X.next:
X'.random = X.random.next (if X.random != NULL)
one line, no map!
PASS 3 unweave:  restore A -> B -> C and
A' -> B' -> C' separately
invariant after pass 2:  every X'.random points at a COPY,
never an original - because X.random.next is X.random's copy.
```

LANGUAGE MECHANICS: PYTHON VS C++

After pass 1 the list reads old1,new1,old2,new2,\ldots{} so the clone of any node is exactly ONE step behind it: cloneOf(x) = x->next. In pass 2 the clone of old->random is therefore old->random->next. All rewiring is done with pure pointer math — no lookup table, hence $O(1)$ space. This "store the mapping in the data structure itself" trick is the same spirit as morphing algorithms that temporarily decorate nodes (e.g. marking visited state with colors). The C++ version reuses our shared struct ListNode which carries the `random` member (list_node.h) — the section's header pays off here: every problem shares ONE node type.

==========
/ #include <vector> // std::vector — test data #include <unordered_map> // std::unordered_map — approach 2's mapping #include <chrono> // std::chrono — timing #include <iostream> // std::cout — output #include <sys/resource.h> // getrusage — peak RAM #include "list_node.h" // our header: ListNode (WITH `random` member)

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- A `map<ListNode*, ListNode*>` memo makes the DFS version trivially correct at the cost of hash lookups per edge.
- Three passes over the interleaved list are $O(n)$ with no auxiliary map – the best space answer.
- `copy.deepcopy` on a self-referential object graph works here and is a valid answer to state, though it hides the algorithm.
- `defaultdict` cannot map None-valued keys usefully; an explicit `if node.random` guard is required.

INTERVIEW TRAPS & SANITY CHECKS

Setting `copy.random = original.random` (no `.next`). That aliases the copy back into the original graph, which the problem explicitly forbids and the test detects.

Sanity check. After the weave, `original.next.next` is the copy of the node two steps ahead, so `X.random.next` is by construction the copy of `X.random`. That single structural fact is what makes pass 2 correct without any lookup.

INTUITION CHECK — 3 QUESTIONS

Q1. Why does the weave beat the memo-map approach?

Answer. The map costs $O(n)$ hash operations and $O(n)$ nodes of auxiliary memory plus recursion depth. The weave costs three linear passes and no auxiliary structure, at the price of mutating the original temporarily – which is restored in pass 3.

Q2. What breaks if the random pointer self-references (A.random = A)?

Answer. Nothing in the weave approach, because `X'.random = X.random.next` and `A.next` is `A'`, so `A'.random = A'` correctly. In a naive DFS without a memo it is exactly the cycle that would cause infinite recursion.

Q3. Why must pass 3 unweave rather than rebuild?

Answer. Because the originals' `next` pointers were overwritten. Unweaving restores each original's successor in one pass, and the copies' successors become valid once the originals are fixed. Doing it in any other order leaves dangling pointers.

0141 Linked List Cycle Easy

ESSENCE & BOUNDS

Given the head of a linked list, determine whether it contains a cycle and, if so, return the node where the cycle begins. The list may be self-cyclic and may have a tail of any length, so the entry node is not necessarily the head. Only `pos == 0` exists as a special input; a null input is not typical here but should not crash.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Floyd's tortoise and hare (BEST) slow += 1 hop, fast += 2 hops per round. Same node → cycle. fast hits null → acyclic.

Python (intuition kernel)

```python
slow = head # tortoise:  1 hop per round
fast = head # hare:  2 hops per round
while fast is not None and fast.next is not None:
 slow = slow.next # 1 hop
 fast = fast.next.next # 2 hops
 if slow is fast:  # lapped inside a loop $\{}to$ cycle
return False # ran off the end $\{}to$ acyclic

```

C++ (intuition kernel)

```cpp
while (fast != nullptr && fast->next != nullptr) {
slow = slow->next; // tortoise:  1 hop
 fast = fast->next->next; // hare:  2 hops
 if (slow == fast) { return true;
 }

```

Time O(n) **Space** O(1)

Follows directly from the traversal count: $O(n)$ time, $O(1)$ space.

Hash set of visited nodes Insert every node pointer into a set; first repeat → cycle.

Python (intuition kernel)

```python
seen: set[ListNode] = set() # Python set = open addressing
cur = head
while cur is not None:
 if cur in seen: # O(1) probe $\{}to$ second visit
 return True # some node points back $\{}to$ cycle
 seen.add(cur)
 cur = cur.next
return False # reached None $\{}to$ acyclic

```

C++ (intuition kernel)

```cpp
for (ListNode* cur = head; cur != nullptr; cur = cur->next) { if (seen.find(cur) != seen.end()) { return true;
 }
 seen.insert(cur);

```

Time O(n) **Space** O(n)

one $O(1)$ hash probe per element, no rescans; hence the stated bounds.

VISUAL STATE TRACE

```
index 0 -> 1 -> 2 -> 3 -> 4
^ |
+-------+
mu = distance head->entry = 3
lambda = cycle length = 2 (3->4->3)
phase 1: fast = 2*slow
slow at 0, fast at 0
slow 1, fast 2
slow 2, fast 4
slow 3, fast 2 (4->3->4->3, so fast wraps to 3->4? trace)
they coincide INSIDE the cycle
phase 2: reset one pointer to head, advance both by 1
slow 4, fast head 0
slow 3(entry), fast 1 -> next step both step 1
slow 3, fast 2
slow 4, fast 3
slow 3, fast 4
-> coincide at index 3 = ENTRY
proof: 2*mu = mu + k*lambda => they meet at the entry.
```

LANGUAGE MECHANICS: PYTHON VS C++

Suppose the tail has length a and the cycle has length L. When slow enters the cycle (at distance a), fast is already somewhere inside it — at most L1 steps ahead. Each round closes the gap by exactly 1 (fast gains 1 step per time unit), so they meet within L steps. Meeting inside a cycle PROVES a cycle exists; if there were none, fast would hit null in a straight run. Entry-point bonus (problem 0287 uses this): after the first meeting, move one pointer back to head and advance BOTH by 1 — they meet at the cycle entrance. Distance math: slow walked a+m, fast walked 2(a+m) = a + kL + m (k laps) ⇒ a ≡ 0 mod (L \ldots{}) ⇒ head-to-entry distance equals meeting-to-entry distance mod L. Real world: checkpoints / version graphs / linked allocators all detect loops with this $O(1)$-space trick when a visited-set is too expensive. Git's fsck uses cycle detection on object graphs.
==========
/ #include <vector> // std::vector — test data #include <unordered_set> // std::unordered_set — approach 2's visited set #include <chrono> // std::chrono — timing #include <iostream> // std::cout — output #include <sys/resource.h> // getrusage — peak RAM #include "list_node.h" // our header: ListNode, buildList

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `ListNode* slow, *fast` with `do/while` avoids the initial null check.
- Brent's algorithm detects a cycle with $O(1)$ space and fewer pointer dereferences, though it does not find the entry.
- The `while fast and fast.next` guard expresses the precondition directly.
- `is` compares identity, which is the correct comparison for node objects.

INTERVIEW TRAPS & SANITY CHECKS

Running phase 2 with fast still at the meeting point instead of resetting it to head. The classic correct form resets ONE of the two pointers and advances both by one step.

Sanity check. After t iterations of phase 1, fast has travelled 2t edges and slow t. If a cycle of length lambda exists, the first t with 2t = t (mod lambda) places them at the same node, i.e. t = k*lambda for some k, which puts fast exactly lambda ahead of slow inside the cycle.

INTUITION CHECK — 3 QUESTIONS

Q1. Why does resetting one pointer to head work in phase 2?

Answer. After t steps where t is a multiple of lambda, fast is t = k*lambda ahead of slow. Resetting fast to head and advancing both by one, they meet after mu steps because slow has travelled t + mu and fast has travelled mu, and the cycle entry is mu edges from head and (t + mu - mu) mod lambda = 0 from slow.

Q2. What is the time complexity?

Answer. $O(n)$ overall. Phase 1 takes at most mu + lambda iterations and phase 2 takes mu, both linear in the list length. Space is $O(1)$ – three pointers.

Q3. Can you find the entry without phase 2?

Answer. Not with only Floyd's pointers. Phase 1 proves existence but the meeting point is generally not the entry; the extra pass is what converts "a cycle exists" into "the cycle starts here".

0143 Reorder List Medium

ESSENCE & BOUNDS

Reorder a list so it alternates nodes from the first half and nodes from the second half, with the first half supplying the leading nodes. n >= 1, so n = 1 returns the single node and n = 2 swaps. Odd lengths mean the first half has one extra node, which must be consumed first or the output order is wrong.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Split + reverse + weave (BEST) (a) fast/slow finds the middle; (b) prev/cur flip reverses the second half; (c) weave with cached successors.

Python (intuition kernel)

```python
if head is None or head.next is None:
 return # 0 or 1 node:  nothing to do
slow = head
fast = head.next
while fast is not None and fast.next is not None:
 slow = slow.next
 fast = fast.next.next
prev:  Optional[ListNode] = None

```

C++ (intuition kernel)

```cpp
 if (head == nullptr || head->next == nullptr) {
return; // 0 or 1 node:  nothing to reorder

```

Time O(n) **Space** O(1)

Follows directly from the traversal count: $O(n)$ time, $O(1)$ space.

Vector weave Collect all node pointers into an array, weave l from 0, r from n1 inward.

Python (intuition kernel)

```python
if head is None or head.next is None:
 return
nodes:  list[ListNode] = [] # contiguous pointer array
cur = head
while cur is not None:
 nodes.append(cur)
 cur = cur.next
l, r = 0, len(nodes) - 1

```

C++ (intuition kernel)

```cpp
 if (head == nullptr || head->next == nullptr) {
return;

```

Time O(n) **Space** O(n)

Follows directly from the traversal count: $O(n)$ time, $O(n)$ space.

VISUAL STATE TRACE

```
list [1,2,3,4,5] -> [1,5,2,4,3]
step 1 find middle (slow/fast):  slow at 3 second = 4
step 2 cut:  1->2->3->NULL , 4->5->NULL
step 3 reverse second half:  5->4
step 4 interleave, FIRST half leads:
take 1 (first), take 5 (second), take 2, take 4, take 3
result [1,5,2,4,3]
list [1,2,3,4] -> [1,4,2,3] (first half 1,2 leads)
invariant:  first always non-NULL (it has >= second's length),
so 'first = first.next' is safe without a guard.
```

LANGUAGE MECHANICS: PYTHON VS C++

slow starts at head, fast at head->next (variant) or head. Each round: slow += 1, fast += 2. Because fast moves twice as fast, when fast reaches the end, slow has covered HALF the distance — the middle (for even length: the first of the two middles, which is the correct split point here). Same speed-ratio trick as Floyd's cycle detection (0141) — one math idea, two problems. The weave loop reads cur1->next BEFORE overwriting it: `ListNode* next1 = cur1->next;` — the cache-first discipline from 0206 applied again. ========== / #include <vector> // std::vector — test data + approach 2 #include <chrono> // std::chrono — timing #include <iostream> // std::cout — output #include <sys/resource.h> // getrusage — peak RAM #include "list_node.h" // our header: ListNode, buildList

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- An array of nodes turns this into index arithmetic and makes the interleave trivial at $O(n)$ space.
- Cutting with a dummy head after slow removes the special case where slow is the head.

- `list.reverse()` on the extracted second half is a C-level call, much faster than manual pointer reversal.
- Taking from `first` before `second` is the load-bearing line; the reverse order gives [1,3,5,2,4].

INTERVIEW TRAPS & SANITY CHECKS

Interleaving from the wrong end. Taking a node from the second half before one from the first produces the wrong order, and the error only shows on odd lengths.

Sanity check. `first` and `second` each point to the head of a reversed-or-not list of roughly equal length with first >= second. Taking from first first preserves the alternation and guarantees first stays non-null.

INTUITION CHECK — 3 QUESTIONS

Q1. Why must the first half lead?

Answer. Because n can be odd, in which case the first half has one more node. If you took from the second half first, the last output node would come from the first half and the pattern would be inverted.

Q2. Can you do it without reversing the second half?

Answer. Yes, by collecting into a stack or an array and emitting alternately – but that costs $O(n)$ extra space, whereas the reversal is in-place and costs $O(1)$ space.

Q3. What is the space complexity of the pointer version?

Answer. $O(1)$. The middle is found with two pointers, the second half is reversed in place, and the interleave only rewires pointers. No allocation at all.

0146 LRU Cache Medium

ESSENCE & BOUNDS

Design a cache with a fixed capacity where get and put are $O(1)$ and the least recently used entry is evicted when full. At most 1e5 operations, capacity >= 1, so capacity 1 must still work (evicting the only entry). get on a missing key returns -1 and must not disturb recency. Keys and values are at least length 1.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Hash map key $\to$ node + doubly linked list (BEST) DLL keeps recency order: head = LRU (victim), tail = MRU. get: hit → move node to tail. put: exists → update + move to tail; new → append at tail; if full → unlink head + erase map.

Python (intuition kernel)

```python
cap, order, val = capacity, OrderedDict(), {}

def get(k):
 if k not in val:  return -1
 order.move_to_end(k)
 return val[k]

def put(k, v):
 if k in val:  order.move_to_end(k)
 val[k], order[k] = v, None
 if len(order) > cap:  order.popitem(last=False)

```

C++ (intuition kernel)

```cpp
if (tail_ == nullptr) { head_ = tail_ = node; // first node ever
} else { node->prev = tail_;
 tail_->next = node;
 tail_ = node;

```

Time O(1) average per op Space : O(C) **Space** ?

one $O(1)$ hash probe per element, no rescans; hence the stated bounds.

Naive: key $\to$ (value, lastUsedTick) Every op stamps a global tick. Eviction scans ALL entries for the smallest tick — $O(C)$ per eviction.

Python (intuition kernel)

```python
store, tick = {}, 0

def get(k):
 if k not in store:  return -1
 tick += 1
 store[k] = (store[k][0], tick)
 return store[k][0]

def put(k, v):
 global tick
 tick += 1
 store[k] = (v, tick)

```

C++ (intuition kernel)

```cpp
if (it != store_.end()) { it->second = {value, ++tick_};
 return;

```

Time O(1) get, O(C) put-when-full Space : O(C)

Space ?

Follows directly from the traversal count: $O(1)$ get, $O(C)$ put-when-full Space : $O(C)$ time, ? space.

VISUAL STATE TRACE

```
cap = 2:  put(1,1) put(2,2) get(1) put(3,3)
hash:  {1:<1>, 2:<2>, 3:<3>}
list:  head <-> 3 <-> 1 <-> 2 <-> tail
(LRU) (MRU)
get(1):  move 1 to MRU end -> list 3 <-> 2 <-> 1
put(3):  map size == cap(2) -> evict tail (2)
map:  {1,3}
sentinel design:
head <-> (LruEntry) <-> ...  <-> (LruEntry) <-> tail
sentinel.prev / sentinel.next are never null, so
remove(node) needs no null checks.
invariant:  |map| == |list| - 2 and map keys == non-sentinel keys
```

LANGUAGE MECHANICS: PYTHON VS C++

Python dict keys use open addressing with LINEAR PROBING: h(k, i) = (hash(k) + i) mod m , i = 0, 1, 2, \ldots{} C++ std::unordered_map uses separate chaining. Both give the $O(1)$ average random access the DLL cannot provide alone. The DLL is the OPPOSITE structure: $O(1)$ insert/delete at known positions but $O(n)$ search — together they cover each other's weakness: map finds the node, list reorders it. This exact pair (hash table + doubly linked list) is how real caches work: Redis' allkeys-lru, memcached's LRU, and the Linux kernel's page cache (radix tree + LRU list). Eviction = "drop the head", promotion = "move to tail" — every browser tab, every CDN. ========== / #include <unordered_map> // std::unordered_map — $O(1)$ key lookup #include <vector> // std::vector — op sequences for tests #include <chrono> // std::chrono — timing #include <iostream> // std::cout — output #include <sys/resource.h> // getrusage — peak RAM #include "list_node.h" // our header (ListNode unused here; the // lesson: headers centralize shared types)

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `std::list` + `std::unordered_map<key, list_iterator>` is the textbook C++ answer and is fully $O(1)$.
- `gp_hash_table` beats `unordered_map` here on cache locality because every op touches both structures anyway.
- An intrusive doubly linked list embedded in the node removes the separate list allocation and one indirection.

- `collections.OrderedDict` gives `move_to_end` and `popitem(last=False)` in C – a complete LRU in 6 lines.
- `__setitem__` on an OrderedDict moves the key to the end automatically, which is exactly `put`.
- A plain `dict` in Python 3.7+ preserves insertion order, so `move_to_end` has no equivalent and you need OrderedDict.

INTERVIEW TRAPS & SANITY CHECKS

Calling `move_to_end(key)` in `get` before checking the key exists – that raises `KeyError`. The existence test must come first.

Sanity check. The map and the list hold exactly the same set of non-sentinel nodes, and the list is ordered from least to most recently used. Every access moves the node to the tail, which is what preserves both parts of the invariant.

INTUITION CHECK — 3 QUESTIONS

Q1. Why a doubly linked list rather than singly?

Answer. Because `get` must move an arbitrary node to the MRU end, and removing from a singly linked list requires the predecessor. A doubly list gives $O(1)$ removal via two pointers, with the head/tail sentinels eliminating every null check.

Q2. Why does OrderedDict make the Python version trivial?

Answer. It is a dict that remembers insertion order and exposes `move_to_end(key)` plus `popitem(last=False)`. That is exactly the two primitives an LRU needs, both implemented in C, so the whole cache is a few Python lines.

Q3. What are the complexities of get and put?

Answer. Both $O(1)$: one hash lookup, one list splice, and for put one possible eviction (map delete plus list pop). Space is $O(capacity)$, bounded by the capacity because put evicts before inserting.

0206 Reverse Linked List Easy

ESSENCE & BOUNDS

Reverse a singly linked list in place and return the new head. The list may be null or single-node, both of which must be handled without special-casing beyond the loop condition. The operation must be in place – $O(1)$ space.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Iterative, 3 sliding pointers (BEST) prev=None, cur=head. Loop: nxt=cur.next (cache), cur.next=prev (flip), prev=cur, cur=nxt (slide). cur==None → prev is the new head. Every arrow flipped exactly once.

Python (intuition kernel)

```python
prev: Optional[ListNode] = None # already-reversed part; new list's end
cur = head # node we are about to flip
while cur is not None:
    nxt = cur.next # THE CACHE: save successor first
    cur.next = prev # THE FLIP: point at previous node
    prev = cur # THE SLIDE: advance both pointers
    cur = nxt
return prev # cur fell off $\{}to$ prev is the new head

```

C++ (intuition kernel)

```cpp
while (cur != nullptr) { ListNode* nxt = cur->next;
 cur->next = prev;
 prev = cur;
 cur = nxt;

```

Time O(n) **Space** O(1)

Follows directly from the traversal count: $O(n)$ time, $O(1)$ space.

Recursive Reverse the tail first (recursion returns the new head), then wire head.next.next = head and head.next = None. The call stack does the "remember the successor" job.

Python (intuition kernel)

```python
if head is None or head.next is None:
    return head # empty or 1 node $\{}to$ already reversed
new_head = self.reverseList(head.next) # reverse the TAIL first
head.next.next = head # old successor points back at head
head.next = None # head becomes the new tail
return new_head # new head bubbles up unchanged

```

C++ (intuition kernel)

```cpp
 if (head == nullptr || head->next == nullptr) {
return head;

```

Time O(n) **Space** O(n) (stack)

Follows directly from the traversal count: $O(n)$ time, $O(n)$ (stack) space.

VISUAL STATE TRACE

```
1 -> 2 -> 3 -> NULL
prev=NULL cur=1
nxt = 2; cur.next = NULL; prev=1; cur=2 -> 1->NULL, cur 2->3
nxt = 3; cur.next = 1; prev=2; cur=3 -> 2->1, cur 3->NULL
nxt = NULL; cur.next = 2; prev=3; cur=NULL -> 3->2->1
return prev = 3
THE ORDER MATTERS: save nxt BEFORE overwriting cur.next,
otherwise the rest of the list is unreachable.
iterative:  O(n) time, O(1) space
recursive:  O(n) time, O(n) stack - risky for 1e5 nodes
```

LANGUAGE MECHANICS: PYTHON VS C++

This problem teaches the THREE pointer-chasing patterns that the whole section builds on: (1) THE CACHE: any time you overwrite a pointer you are pointing at (cur->next), copy it out first — otherwise the data behind it becomes unreachable (a leak in real programs). (2) THE DUMMY NODE: an extra sentinel head makes edge cases (empty list, single node, inserting at position 0) disappear — main() in list_node.h uses the same trick. (3) THE SLIDE: advancing prev/cur/nxt together keeps all needed context reachable with $O(1)$ memory. Real-world: linked lists survive as LRU caches (0146), hash-map buckets (separate chaining), and intrusive memory allocators — every one of them rewrites next pointers exactly like this. ========== / #include <vector> // std::vector — test data containers #include <chrono> // std::chrono — high-resolution timing #include <iostream> // std::cout — printing benchmark results #include <sys/resource.h> // getrusage — peak-RAM measurement #include "list_node.h" // OUR header: ListNode + buildList + printList // (quotes = look in THIS folder first)

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- Iterative with three raw pointers is the only version with $O(1)$ space and no stack overflow risk.
- `std::reverse` does not apply – it works on contiguous ranges, not linked structures.
- `prev, cur = None, head` then `cur.next, prev, cur = prev, cur, cur.next` – the simultaneous assignment makes the ordering bug impossible.
- Python's simultaneous assignment `prev, cur = cur, cur.next` after saving `nxt` makes the ordering bug structurally impossible.
- `itertools` has no linked-list reverse; a while loop is the only option, and `cur.next` is a single attribute load per step.

INTERVIEW TRAPS & SANITY CHECKS

Writing `prev = cur` before `cur = cur.next`, or forgetting to save `next` first. Either loses the remainder of the list and silently truncates the result.

Sanity check. After k iterations, `prev` is the head of a reversed list containing exactly the first k original nodes, and `cur` points to the (k+1)-th original node. So nothing is lost or duplicated.

INTUITION CHECK — 3 QUESTIONS

Q1. Why must `nxt` be saved before `cur.next` is overwritten?

Answer. Because `cur.next` is the only link to the unprocessed remainder. Once you overwrite it, that tail is unreachable from anywhere – not from cur, not from prev – so the saved reference is what keeps the traversal alive.

Q2. When is recursion a bad choice here?

Answer. For long lists. CPython's default recursion limit is 1000, so a 1e5-node list raises `RecursionError`. C++ would need a 1e5-frame stack, which is 8 MB or more. The iterative version has neither problem.

Q3. What does the function return, and why is it `prev`?

Answer. `prev`, because after the loop `cur` is NULL and `prev` holds the last node processed – which is the original tail, now the head of the reversed list. Returning `cur` would give null.

0287 Find the Duplicate Number Medium

ESSENCE & BOUNDS

An array of n+1 integers in [1, n] contains exactly one value repeated; return it without modifying the array and using $O(1)$ space. 1 <= n <= 1e5, values in [1, n] with n = 1 a degenerate case. Modifying the array is forbidden, so the in-place-marking trick (setting nums[i] to -1) is illegal.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Floyd's tortoise and hare on the array (BEST) slow = nums[slow], fast = nums[nums[fast]] until they meet; then slow and a fresh pointer from 0 advance equally and meet at the cycle entry = the answer.

Python (intuition kernel)

```python
slow = 0
fast = 0
while True:
 slow = nums[slow] # 1 hop:  i $\{}to$ nums[i]
 fast = nums[nums[fast]] # 2 hops
 if slow == fast:
 break
slow2 = 0

```

C++ (intuition kernel)

```cpp
do { slow = nums[slow]; // 1 hop:  i $\{}to$ nums[i]
 fast = nums[nums[fast]]; // 2 hops

```

Time O(n) **Space** O(1)

Follows directly from the traversal count: $O(n)$ time, $O(1)$ space.

Hash set of seen values First value seen twice is the answer.

Python (intuition kernel)

```python
seen:  set[int] = set() # Python set = open
addressing
for v in nums:
 if v in seen:  # second sighting $\{}to$ duplicate
 return v
 seen.add(v)
return -1 # unreachable per statement

```

C++ (intuition kernel)

```cpp
for (int v :  nums) { if (seen.find(v) != seen.end())
{ return v; // second sighting $\{}to$ the duplicate
 }
 seen.insert(v);

```

Time O(n) **Space** O(n)

one $O(1)$ hash probe per element, no rescans; hence the stated bounds.

VISUAL STATE TRACE

```
nums = [1,3,4,2,2] -> 2
treat i as an arrow:  i -> nums[i]
0 -> 1 -> 3 -> 2 -> 4 -> 2 -> 4 -> 2 ...
^ |
+-----+ cycle length 2
mu (tail length) = 3 (0,1,3 to reach 2)
phase 1 meeting:  2*mu = 6, lambda = 2, 6 mod 2 == 0 -> meets
phase 2:  reset to 0, both step 1 until equal -> 2 CORRECT
proof:  entry node == the duplicated value because the tail
ends at the repeated value and the cycle starts there.
```

LANGUAGE MECHANICS: PYTHON VS C++

The index walk i → nums[i] is a FUNCTION on {0..n}, so its picture is disjoint chains feeding into cycles (a functional graph). Because values live in [1, n] and there are n+1 of them, at least one value is the "head" of two chains — which forces a cycle, and the cycle's ENTRY is the duplicated value. Entry distance: after the first meeting, moving one pointer back to index 0 and advancing BOTH one step at a time makes them meet at the cycle entry — because the tail length a satisfies a ≡ (distance from meeting point to entry) (mod cycle length). The same argument proves the 0141 entry-point trick; the hash map h(k, i) = (hash(k)+i) mod m analogy: here the "hash" is the array itself — the array IS the hash table, and the cycle-entry lookup finds the duplicate without one extra byte.
==========
/ #include <vector> // std::vector — the array #include <unordered_set> // std::unordered_set — approach 2 #include <chrono> // std::chrono — timing #include <iostream> // std::cout — output #include <sys/resource.h> // getrusage — peak RAM #include "list_node.h" // our header: the array models a linked // list (index i → "node" nums[i]), so the // whole section shares one mental model

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- The in-place marking solution is $O(n)$ and faster but mutates the input, which is forbidden here.
- Sum formula gives the duplicate directly: sum(nums) - n(n+1)/2 – one line, but overflow-prone for large n.

- `Counter` is $O(n)$ space and illegal by the constraint.
- The Floyd version has no allocation at all, which is exactly what the problem asks you to demonstrate.

INTERVIEW TRAPS & SANITY CHECKS

Using the bitwise XOR trick (`x ^ x = 0`) – it works only when every OTHER value appears an even number of times, and here every value in [1,n] appears once except the duplicate.

Sanity check. Viewing the array as a function `f(i) = nums[i]` from [0,n] into [1,n], the duplicate guarantees a cycle. Floyd detects it and returns its entry, which is the duplicated value.

INTUITION CHECK — 3 QUESTIONS

Q1. Why does the cycle entry equal the duplicated number?

Answer. The chain starting at 0 visits distinct nodes until it repeats. The repeat happens at the value that has two preimages, which is precisely the duplicated number. So the tail ends at the duplicate and the cycle begins there.

Q2. Why is the sum trick not usable despite being $O(1)$ space?

Answer. It is usable in spirit – sum(nums) - n(n+1)/2 gives the duplicate – but with n up to 1e5 the sum reaches about 5e9, which overflows a 32-bit C++ int. The sum must be 64-bit, and the float version in Python loses exactness at that magnitude.

Q3. What is the space complexity of the Floyd solution?

Answer. $O(1)$ – three integers. No array of counts, no set. That is the entire point of using Floyd here rather than the more obvious counting approach.

7. Trees

#	Problem	Level	One-line intent
0098	**Validate Binary Search Tree**	Medium	Carry an open interval down, not just the parent value.
0100	**Same Tree**	Easy	Structural equality: both null, or both non-null and equal.
0102	**Binary Tree Level Order Traversal**	Medium	Snapshot the queue size at the start of each level.
0104	**Maximum Depth of Binary Tree**	Easy	1 + max of the two subtrees; null is depth zero.
0105	**Construct Binary Tree from Preorder and Inorder**	Medium	The next preorder value is the root; inorder splits it.
0110	**Balanced Binary Tree**	Easy	Return the height, and reject any subtree differing by more than one.
0124	**Binary Tree Maximum Path Sum**	Hard	Clamp each child branch at zero; the return is one-sided.
0199	**Binary Tree Right Side View**	Medium	BFS taking the last node of each level, or DFS right-first.
0226	**Invert Binary Tree**	Easy	Swap the children, recursively; three lines of essence.
0230	**Kth Smallest Element in a BST**	Medium	Iterative inorder, stopping at the k-th visit.
0235	**Lowest Common Ancestor Of Bst**	?	Descend using BST ordering until p and q split.
0297	**Serialize And Deserialize Binary Tree**	?	Preorder with null markers encodes every tree unambiguously.
0543	**Diameter Of Binary Tree**	?	Post-order returning (height, diameter) and carrying the best.
0572	**Subtree Of Another Tree**	?	Serialize both trees and search for the pattern with KMP.
1448	**Count Good Nodes In Binary Tree**	?	Propagate the path minimum; a node is good at equality.

0098 `Validate Binary Search Tree` Medium

ESSENCE & BOUNDS

Decide whether a binary tree satisfies the BST property: every node's value lies strictly between the values permitted by all its ancestors. Node values are int and can be INT_MIN or INT_MAX, so a bounds-based recursion must use long long or null sentinels, and Python must use float('-inf') rather than 0. Strictness matters: duplicates are invalid.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Recursive interval propagation (BEST) DFS down the tree carrying (lo, hi); each node must satisfy lo < val < hi. ±inf floats give the root no constraints.

Python (intuition kernel)

```python
return self._validate(root, float("-inf"), float("inf"))

```

C++ (intuition kernel)

```cpp
if (node == nullptr) { return true;

```

Time O(n) **Space** O(h), h = height.

Follows directly from the traversal count: $O(n)$ time, $O(h)$, h = height. space.

Iterative inorder (must be strictly ascending) A BST's inorder walk emits values in STRICTLY increasing order — and only a BST does. Walk iteratively with an explicit stack; if the current value is <= the previous one → not a BST.

Python (intuition kernel)

```python
pending: List[TreeNode] = []
cur: Optional[TreeNode] = root
prev: Optional[TreeNode] = None # last value emitted by the walk
while cur is not None or pending:
 while cur is not None:
 pending.append(cur)
 cur = cur.left
 cur = pending.pop()

```

C++ (intuition kernel)

```cpp
while (cur != nullptr || !pending.empty()) { while (cur != nullptr) { pending.push(cur);
 cur = cur->left;
 }
 cur = pending.top();
 pending.pop();
 if (prev != nullptr && prev->val >= cur->val) { return false;
 }
 prev = cur;

```

Time O(n) **Space** O(h).

Follows directly from the traversal count: $O(n)$ time, $O(h)$. space.

VISUAL STATE TRACE

```
5
/ \{}
3 7 -> valid
/ \{} / \{}
2 4 6 8
valid(n, lo, hi) means: n.val in (lo, hi) STRICTLY
left -> valid(n.left, lo, n.val)
right -> valid(n.right, n.val, hi)
WRONG variant (the trap):
left -> valid(n.left, n.val, INT_MAX) # loses the lower bound
right -> valid(n.right, INT_MIN, n.val) # loses the upper bound
accepts [5,1,6,null,null,3] which is invalid
inorder must be STRICTLY ascending - simpler:
prev = -inf ; walk inorder ; if val <= prev: False
```

LANGUAGE MECHANICS: PYTHON VS C++

Every function call pushes a FRAME (locals + return address) onto the call stack; the frame pops when the callee returns. Recursive tree code therefore costs $O(h)$ extra memory — one frame per ancestor still on the current path. Approach 1 lets the runtime maintain that stack invisibly; Approach 2 builds the SAME stack by hand with a list. Identical memory behaviour — recursion just hides the bookkeeping, which is why tree code prefers it. Every function call pushes a FRAME (locals + return address) onto the call stack; the frame pops when the callee returns. Recursive tree code therefore costs $O(h)$ extra memory — one frame per ancestor still on the current path. Approach 1 lets the runtime maintain that stack invisibly; Approach 2 builds the SAME stack by hand with std::stack. Identical memory behaviour — recursion just hides the bookkeeping, which is why tree code prefers it.
==========
/ #include <iostream> // std::cout — printing benchmark results #include <vector> // std::vector — level-order test inputs #include <stack> // std::stack — explicit inorder stack (approach 2) #include <climits> // LLONG_MIN / LLONG_MAX — $\pm\infty$ bounds for approach 1 #include <chrono> // std::chrono — steady_clock timing for benchmark #include <sys/resource.h>// getrusage — peak-RAM measurement for benchmark #include "tree_node.h" // share

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `long long lo, hi` with `LLONG_MIN/LLONG_MAX` handles INT_MIN/INT_MAX safely; `INT_MIN` as a bound is the classic overflow bug.
- An iterative stack-based inorder avoids recursion depth issues on a skewed tree of 1e4 nodes.
- `float('-inf')` / `float('inf')` as the initial bounds is the safe sentinel – integer sentinels cannot express "unbounded".
- `bisect` on the inorder list is another way to state strictness, though it allocates $O(n)$.

INTERVIEW TRAPS & SANITY CHECKS

Passing `None`/`0` as the initial bound instead of infinity. With `0` as the lower bound a tree rooted at a negative value is rejected, and with `0` as the upper bound a subtree of positives is rejected.

Sanity check. `valid(n, lo, hi)` returns true iff n's entire subtree satisfies the BST property given that every value in it must lie strictly inside (lo, hi). The recursion tightens exactly one side per branch, so the invariant composes.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is the inorder-strictly-ascending check equivalent?

Answer. A BST is exactly the tree whose inorder traversal is strictly increasing. Proving one direction: an inorder walk emits left subtree, root, right subtree, and the BST property forces each to be in order. So a single inorder pass with one comparison decides validity, at $O(h)$ space.

Q2. What is the recursion depth, and is it safe?

Answer. $O(h)$, h the height. A degenerate tree of 1e4 nodes gives depth 1e4, which overflows CPython's default recursion limit and is a large but survivable C++ stack. The iterative inorder version is the safe choice.

Q3. How do you handle INT_MIN as a node value in C++?

Answer. Use `long long` bounds with `LLONG_MIN`/`LLONG_MAX`, not `int` with `INT_MIN`. With int bounds a node equal to INT_MIN would fail the strict lower-bound test even though it is legal.

0100 Same Tree Easy

ESSENCE & BOUNDS

Decide whether two binary trees have identical structure and identical node values. The trees may have different node counts, different depths, or be null. Comparing node-by-node only when both exist is what makes mismatched shapes fail fast.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Recursive equality (BEST, clearest) Three base cases (both None → True; one None → False; values differ → False), then recurse into both child pairs.

Python (intuition kernel)

```python
if p is None and q is None:
if p is None or q is None:
if p.val != q.val:
return self.isSameTree(p.left, q.left) and
self.isSameTree(p.right, q.right)

```

C++ (intuition kernel)

```cpp
if (p == nullptr && q == nullptr) { return true;

```

Time O(min(n, m)) — stops at the first mismatch.

Space O(h) — call stack depth.

Follows directly from the traversal count: O(min(n, m)) — stops at the first mismatch. time, $O(h)$ — call stack depth. space.

Iterative BFS (queue of node pairs) Same logic, but the work is held in an explicit FIFO queue of (p, q) pairs instead of the call stack — a flat loop, immune to deep-recursion stack overflow on skewed trees.

Python (intuition kernel)

```python
pending: deque = deque([(p, q)])
while pending:
a, b = pending.popleft()
if a is None and b is None:
continue
if a is None or b is None or a.val != b.val:
pending.append((a.left, b.left))
pending.append((a.right, b.right))

```

C++ (intuition kernel)

```cpp
while (!pending.empty()) { auto [a, b] =
pending.front();
pending.pop();
if (a == nullptr && b == nullptr) { continue;
}
if (a == nullptr || b == nullptr || a->val !=
b->val) { return false;
}
pending.push({a->left, b->left});
pending.push({a->right, b->right});

```

Time O(min(n, m)) — each pair dequeued once.	**Space** O(w) — queue width at the widest level.

every node is enqueued and dequeued once; hence the stated bounds.

VISUAL STATE TRACE

```
t1 t2 same?
1 1 values equal
/ \{} / \{} both children non-null
2 3 2 3 recurse
1 1 values equal
/ \{} / t2 right is NULL but t1 right is not
2 3 -> FALSE (shape differs)
recursion:
both NULL -> true
one NULL -> false
values differ -> false
else recurse both children
invariant:  the call at node pair (a,b) is responsible for the
entire subtrees below; it may only return true if BOTH shape
and values agree all the way down.
```

LANGUAGE MECHANICS: PYTHON VS C++

Recursive DFS and iterative BFS explore in different ORDERS but compare exactly the same pairs of nodes. The choice is about the container: the call stack (A1) vs an explicit queue (A2). For an $O(n)$ tree walk either is fine; the queue version shines when the tree is so deep that recursion would blow the runtime stack. Recursive DFS and iterative BFS explore in different ORDERS but compare exactly the same pairs of nodes. The choice is about the container: the call stack (A1) vs an explicit queue (A2). For an $O(n)$ tree walk, either is fine; the queue version shines when the tree is so deep that recursion would blow the runtime stack.
==========
/ #include <iostream> // std::cout — printing benchmark results #include <vector> // std::vector — level-order test inputs #include <queue> // std::queue — FIFO of node pairs (approach 2) #include <utility> // std::pair — (p, q) bundles in the queue #include <chrono> // std::chrono — steady_clock timing for benchmark #include <sys/resource.h> // getrusage — peak-RAM measurement for benchmark #include "tree_node.h" // shared TreeNode + buildTree/printTree/destroyTree

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- An iterative stack of node pairs avoids recursion depth on skewed trees.
- A single `TreeNode*` stack holding pairs is the standard iterative form.
- Comparing with == on nodes would use identity; the `is not None` and value checks must be explicit.
- A one-liner `(a.val, a.left, a.right) == (b.val, b.left, b.right)` recurses implicitly but allocates tuples per node.

INTERVIEW TRAPS & SANITY CHECKS

Checking only values and ignoring structure. `[1,2]` versus `[1,null,2]` has the same multiset of values but different shapes; the null checks on the children are what distinguish them.

Sanity check. At every call, `a` and `b` are at the same position in their respective trees (or both null). Returning true requires all four conditions simultaneously, so a single false anywhere propagates up and the whole answer is false.

INTUITION CHECK — 3 QUESTIONS

Q1. Why check `a is None and b is None` before touching values?

Answer. Because you must distinguish "both subtrees end here" (true) from "one ends and the other does not" (false). Merging those cases into a single `a is None or b is None` check wrongly accepts trees of different shapes.

Q2. Can you do this without recursion?

Answer. Yes – push both roots on a stack, pop a pair, apply the same three tests, and push the two children pairs. Same $O(n)$ time, and the stack is $O(h)$ which is better than the call stack only in that it is heap-allocated and bounded explicitly.

Q3. What is the time complexity, and why is it $O(n)$ not O(min(n,m))?

Answer. $O(n)$ where n is the total nodes compared. You may stop early on a mismatch, but the worst case is when both trees are identical, and then every node in both is visited.

0102 Binary Tree Level Order Traversal Medium

ESSENCE & BOUNDS

Return the values of the tree grouped by level, outermost level first. The tree may be null (return []) and is skewed, so the queue-based BFS must not recurse. Node count up to 2e4, so recursion depth can reach 2e4 on a left-only chain.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

BFS with a per-level snapshot (BEST) At each iteration record size = len(queue), pop exactly `size` nodes into one row, enqueue their children for the next row.

Python (intuition kernel)

```python
rows: List[List[int]] = [] # one list per level
if root is None:
 return rows # empty tree $\{}to$ empty output
pending = [root] # FIFO of the next frontier
while pending:
 row: List[int] = []
 for _ in range(len(pending)):
 cur = pending.pop(0)

```

C++ (intuition kernel)

```cpp
if (root == nullptr) {
 return rows; // empty tree $\{}to$ empty output

```

Time O(n) **Space** O(n) — queue holds

every node is enqueued and dequeued once; hence the stated bounds.

DFS with depth-indexed rows (no queue at all) Recursively visit nodes carrying their depth; append each value into rows[depth] (growing the list on first visit to a depth). Same output, different traversal order (preorder instead of BFS).

Python (intuition kernel)

```python
rows: List[List[int]] = []
self._walk(root, 0, rows) # root sits at depth 0
return rows

```

C++ (intuition kernel)

```cpp
if (node == nullptr) { return;

```

Time O(n) **Space** O(h) call stack +

every node is visited once and returns $O(1)$ state; hence the stated bounds.

VISUAL STATE TRACE

```
3
/ \{}
9 20
/ \{}
15 7
queue = [3], level size = 1
take 1 node -> 3 level 0:  [3]
queue = [9,20], level size = 2
take 2 -> 9, 20 level 1:  [9,20]
queue = [15,7], level size = 2
take 2 -> 15, 7 level 2:  [15,7]
the SNAPSHOT is the whole trick:  capture len(queue) BEFORE
consuming the level, else you would consume everything in one
flat list.  'for _ in range(size)' is the idiom.
time O(n), space O(width) for the queue
```

LANGUAGE MECHANICS: PYTHON VS C++

FIFO is THE container for breadth-first walks: a node's children are appended AFTER everything already queued, so nodes at depth d are always processed before any node at depth d+1. Stacks would produce DFS instead. The "snapshot the size first" trick converts a FIFO stream into explicit level boundaries — the same pattern powers right-side view and zigzag traversal. FIFO is THE container for breadth-first walks: a node's children are appended AFTER everything already queued, so nodes at depth d are always processed before any node at depth d+1. Stacks would produce DFS (depth-first) instead. The "snapshot the size first" trick converts a FIFO stream into explicit level boundaries — the same pattern powers right-side view and zigzag traversal. ==========

/ #include <iostream> // std::cout — printing benchmark results #include <vector> // std::vector — rows output / level-order tests #include <queue> // std::queue — FIFO used by approach 1 #include <chrono> // std::chrono — steady_clock timing for benchmark #include <sys/resource.h> // getrusage — peak-RAM measurement for benchmark #include "tree_node.h" // shared TreeNode + buildTree/printTree/destroyTree

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `queue<TreeNode*>` with `int size = q.size()` per level is the canonical form.
- Two vectors and an index pointer avoids per-level allocation, reusing capacity across levels.

- `from collections import deque; popleft()` is $O(1)$ at both ends; a `list` with an index pointer is also $O(1)$ and avoids deque overhead for pure FIFO use.
- A list used as a FIFO with a moving index beats `collections.deque` when only one end is consumed, since there is no block splitting.
- `for _ in range(len(q))` snapshots the level size once, which is the entire algorithm – `deque.popleft()` is the other half.

INTERVIEW TRAPS & SANITY CHECKS

Iterating `while queue:` and appending into the same level list – that flattens the traversal. Or recomputing the level size inside the inner loop, which shrinks it as you pop.

Sanity check. At the start of each outer iteration, the queue contains exactly the nodes of the current level, in left-to-right order, and `size` is that count. Popping exactly `size` times consumes precisely one level and leaves the queue holding the next.

INTUITION CHECK — 3 QUESTIONS

Q1. Why must the size be captured before the inner loop?

Answer. Because popping shrinks the queue. If you recompute the size each iteration it decreases by one every time and the inner loop exits early, producing short levels. Capturing it once makes the loop bound the level width.

Q2. What is the space complexity, and what dominates it?

Answer. $O(w)$ where w is the maximum width of the tree, plus $O(h)$ for the result list itself. A complete tree of n nodes has width ~n/2, so the result dominates; a skewed tree has width 1 but a result of depth h.

Q3. When is the depth-first alternative preferable?

Answer. When the interviewer wants to test index-based accumulation: recurse with a `depth` argument and append into `result[depth]`. That gives $O(n)$ time with $O(h)$ recursion space and no queue at all.

0104 Maximum Depth of Binary Tree Easy

ESSENCE & BOUNDS

Return the number of edges on the longest root-to-leaf path. The tree is a valid binary tree and may be a single node (depth 0) or null. Up to 1e4 nodes, so recursion depth can reach 1e4.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Recursive post-order (BEST, canonical) Ask both children for their depths, take the max, add 1 for the current node. The answer assembles itself bottom-up.

Python (intuition kernel)

```python
if root is None:
left_depth = self.maxDepth(root.left)
right_depth = self.maxDepth(root.right)
return max(left_depth, right_depth) + 1

```

C++ (intuition kernel)

```cpp
if (root == nullptr) {

```

Time O(n) **Space** O(h), h = height.

Follows directly from the traversal count: $O(n)$ time, $O(h)$, h = height. space.

Iterative BFS (count the levels) Process level by level with a queue; every finished level adds 1 to a counter. No recursion at all — depth = number of rows.

Python (intuition kernel)

```python
if root is None:
pending = [root] # FIFO -- the next frontier
depth = 0
while pending:
 for _ in range(len(pending)):
 cur = pending.pop(0)
 if cur.left is not None:
 pending.append(cur.left)

```

C++ (intuition kernel)

```cpp
if (root == nullptr) {

```

Time O(n) **Space** O(w) — queue holds

every node is enqueued and dequeued once; hence the stated bounds.

VISUAL STATE TRACE

```
1
/ \{}
2 3
/ \{}
4 5
\{}
6
depth(NULL) = 0
depth(1) = 1 + max(depth(2), depth(3))
depth(2) = 1 + max(depth(4), 0) = 1 + 1 = 2
depth(3) = 1 + max(0, depth(5)) = 1 + 2 = 3
depth(1) = 1 + max(2, 3) = 4 CORRECT (path 1-3-5-6)
edge count vs node count:  null->0 means the answer counts
EDGES, which matches the problem statement.
```

LANGUAGE MECHANICS: PYTHON VS C++

Computing a value from children BEFORE acting on the node itself is a post-order pattern: the child answers arrive first, the parent's answer composes them. This exact shape powers max path sum, diameter and balanced-tree checks — every "climb back up with a summary value" problem. Approach 2 sidesteps recursion entirely because BFS measures depth horizontally, one level at a time — no call stack, but $O(n)$ worst-case queue memory instead of $O(h)$.
Computing a value from children BEFORE acting on the node itself is a post-order pattern: the child answers arrive first, the parent's answer composes them. This exact shape powers max path sum, diameter and balanced-tree checks — every "climb back up with a summary value" problem. Approach 2 sidesteps recursion entirely because BFS measures depth horizontally, one level at a time — no call stack, but $O(n)$ worst-case queue memory instead of $O(h)$.
==========
/ #include <iostream> // std::cout — printing benchmark results #include <vector> // std::vector — level-order test inputs #include <queue> // std::queue — FIFO used by approach 2 #include <chrono> // std::chrono — steady_clock timing for benchmark #include <sys/resource.h> // getrusage — peak-RAM measurement for benchmark #include "tree_node.h" // shared TreeNode + buildTree/printTree/destroyTree

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- A stack with `(node, depth)` pairs avoids the 1e4-deep call stack risk.
- `max` on two ints is a single cmp/cmov pair, so the recursive form is already optimal.

- `max(a, b)` is a Python function call; a conditional expression is measurably faster in a hot recursion.
- Iterative level-order counting is $O(n)$ time with $O(w)$ space and no recursion limit risk.

INTERVIEW TRAPS & SANITY CHECKS

Returning node count instead of edge count. Returning 1 for a null tree instead of 0 shifts every answer by one; the problem counts edges.

Sanity check. `depth(n)` returns the longest downward path from n to a null child, counted in edges. A null node has depth 0 because it has no edges, so the recurrence 1 + max(left, right) composes correctly.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is the null base case 0 and not 1?

Answer. Because the answer counts edges. A lone node has one node and zero edges, so depth must be 0 for it, which requires depth(NULL) = 0 and depth(node) = 1 + max(children).

Q2. Could this be iterative with less memory?

Answer. Yes – level-order counting needs $O(w)$ queue space, but a DFS stack needs $O(h)$. Neither beats the $O(h)$ recursion stack meaningfully; the iterative version's only real advantage is avoiding the recursion-depth limit in Python.

Q3. What if the tree were a general N-ary tree?

Answer. The recurrence generalises to 1 + max over all children, and the null base case stays 0. The kernel is identical; only the loop over children is added.

0105 Construct Binary Tree from Preorder and Inorder Medium

ESSENCE & BOUNDS

Rebuild a binary tree from its preorder and inorder traversals, where node values are unique. 1 <= preorder.length <= 3000 with all values unique, so the inorder index map is a permutation and the split point is well defined. An empty input must return null without special-casing.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Hash map of inorder indices (BEST) Pre-scan inorder into a dict value→index so "how big is the left subtree?" is an $O(1)$ arithmetic question: left_size = in_index[root] - in_left. Build slices recursively.

Python (intuition kernel)

```
in_index = {v:  i for i, v in enumerate(inorder)}
return self._build(preorder, 0, len(preorder) - 1,
 inorder, 0, len(inorder) - 1, in_index)
 ino:  List[int], il:  int, ir:  int,
 in_index:  dict) -> Optional[TreeNode]:

```

C++ (intuition kernel)

```
for (int i = 0; i < static_cast<int>(inorder.size());
++i) { in_index[inorder[i]] = i;

```

Time O(n) **Space** O(n) map + O(h) stack.

one $O(1)$ hash probe per element, no rescans; hence the stated bounds.

No map: linear scan (conceptually simpler, slower) Identical recursion, but the inorder root is located by scanning the slice linearly each time.

Python (intuition kernel)

```
return self._build(preorder, 0, len(preorder) - 1,
 inorder, 0, len(inorder) - 1)
 ino:  List[int], il:  int, ir:  int) ->
Optional[TreeNode]:

```

C++ (intuition kernel)

```
if (pl > pr || il > ir) { return nullptr;

```

Time O(n^2) **Space** O(h) — no map.

Follows directly from the traversal count: $O(n^2)$ time, $O(h)$ — no map. space.

VISUAL STATE TRACE

```
preorder = [3,9,20,15,7]
inorder = [9,3,15,20,7]
build(pre_idx, lo, hi):
root_val = preorder[pre_idx]
root = TreeNode(root_val)
mid = index_of[root_val] # O(1) with a map
left_size = mid - lo
root.left = build(pre_idx+1, lo, mid-1)
root.right = build(pre_idx+1+left_size, mid+1, hi)
build(0, 0, 4):
root 3, mid=1, left_size=1
left = build(1, 0, 0) -> 9
right = build(2, 2, 4) -> 20 with children 15, 7
WITHOUT the map (the O(n^2) variant):
mid = inorder.index(root_val) -> O(n) PER CALL
total O(n^2) = 9e6 for n=3000.  The map makes it O(n).
```

LANGUAGE MECHANICS: PYTHON VS C++

Preorder range [pl, pr] and inorder range [il, ir] always describe the SAME subtree. root = pre[pl]. left subtree = pre[pl+1, pl+1+ls) × inorder [il, mid); right = pre[pl+1+ls, pr] × inorder (mid, ir]. Empty ranges → None. That is the whole algorithm — recursion is just the for-loop of divide & conquer. Pre-order range [pl, pr] and inorder range [il, ir] always describe the SAME subtree. root = pre[pl]. left subtree = pre[pl+1, pl+1+ls) × inorder [il, mid); right = pre[pl+1+ls, pr] × inorder (mid, ir]. Empty ranges → nullptr. That is the whole algorithm — recursion is just the for-loop of divide & conquer. ========== / #include <iostream> // std::cout — printing benchmark results #include <vector> // std::vector — traversal inputs #include <unordered_map> // std::unordered_map — value→index (approach 1) #include <chrono> // std::chrono — steady_clock timing for benchmark #include <sys/resource.h> // getrusage — peak-RAM measurement for benchmark #include "tree_node.h" // shared TreeNode + buildTree/printTree/destroyTree

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- An `unordered_map<int,int>` of value to inorder index is the whole optimization.
- Passing the inorder range by reference and preorder by index avoids copying vectors per call.

- `index = {v:  i for i, v in enumerate(inorder)}` is $O(n)$ once and makes each lookup $O(1)$.
- `inorder.index(v)` inside the recursion is the trap – it is $O(n)$ per node, giving $O(n^2)$.

INTERVIEW TRAPS & SANITY CHECKS

Recomputing the inorder position with `inorder.index(root_val)` at each call. It works, is $O(n^2)$, and at n = 3000 is roughly 9e6 comparisons – enough to time out.

Sanity check. `build(pre_idx, lo, hi)` reconstructs exactly the subtree whose inorder traversal is `inorder[lo..hi]`, using `preorder[pre_idx]` as its root. The left subtree consumes `mid - lo` further preorder entries, so the right subtree starts at `pre_idx + 1 + (mid - lo)`.

INTUITION CHECK — 3 QUESTIONS

Q1. Why does the inorder index split the preorder list?

Answer. In preorder the root is first, then the entire left subtree, then the entire right subtree. Knowing the left subtree's size from the inorder index therefore tells you where the right subtree begins in preorder – that single arithmetic step is the entire algorithm.

Q2. What if values were not unique?

Answer. The algorithm breaks: the same value appears in inorder at multiple positions, so the split point is ambiguous and the tree is not uniquely determined. Uniqueness is exactly the precondition that makes the map well defined.

Q3. What is the space complexity?

Answer. $O(n)$ for the index map plus $O(h)$ recursion. Both are unavoidable – the map is what buys the time bound and the stack is the traversal itself.

0110 Balanced Binary Tree Easy

ESSENCE & BOUNDS

Decide whether the height of the left and right subtrees of every node differs by at most one. Up to 5e3 nodes, so a skewed tree recurses 5e3 deep. An empty tree is balanced, which must be the base case.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Single post-order pass (BEST) dfs(node) returns the subtree height, or -1 as a sentinel meaning "unbalanced somewhere below". Every node is touched once.

Python (intuition kernel)

```python
return self._height(root) != -1

```

C++ (intuition kernel)

```cpp
if (node == nullptr) {

```

Time O(n) **Space** O(h), h = height.

Follows directly from the traversal count: $O(n)$ time, $O(h)$, h = height. space.

Top-down double recursion (contrast) isBalanced(root) = |height(left) height(right)| ≤ 1 AND balanced(left) AND balanced(right), recomputing height() per node.

Python (intuition kernel)

```python
if root is None:
if abs(self._height(root.left) -
self._height(root.right)) > 1:
return self.isBalanced(root.left) and
self.isBalanced(root.right)

```

C++ (intuition kernel)

```cpp
if (root == nullptr) { return true;

```

Time O(n^2) worst — height() re-walks subtrees for **Space** O(h).

each (i, j) state is solved once and cached; hence the stated bounds.

VISUAL STATE TRACE

```
3
/ \{}
2 8
/ \{}
1 7
\{}
6 <- subtree of 7 is off by 2
height(n) -> -1 if unbalanced, else the height
h(1) = 0 (leaf)
h(7) = 1 + max(h(6), h(N)) (6 has h 0, NULL -1)
-> 1 + max(0, -1) = 1
h(8) = 1 + max(h(7), h(NULL)) = 1 + max(1, -1) = 2
h(2) = 1 + max(h(1), h(NULL)) = 1 + max(0,-1) = 1
h(3) = 1 + max(1, 2) = 3, and |1-2| = 1 <= 1 -> balanced
WHY -1 AS THE SENTINEL: one return value encodes both
'how tall' and 'is fine', so the parent can check both
facts with a single comparison.
```

LANGUAGE MECHANICS: PYTHON VS C++

"Return a compact summary from each subtree, compose at the node" is the fundamental pattern of tree DP: max path sum, diameter, and balanced checks are ALL this. The summary is a PAIR here — (height, balanced) — encoded cleverly as a single int: height ≥ 0, or -1 meaning "imbalanced". One value, two meanings, zero extra structs. Approach 2 shows the cost of NOT passing info up: every level re-asks the same subtree questions it already knew the answer to. "Return a compact summary from each subtree, compose at the node" is the fundamental pattern of tree DP: max path sum, diameter, and balanced checks are ALL this. The summary is a PAIR here — (height, balanced) — encoded cleverly as a single int: height ≥ 0, or -1 meaning "imbalanced". One value, two meanings, zero extra structs. Approach 2 shows the cost of NOT passing info up: every level re-asks the same subtree questions it already knew the answer to.
==========
/ #include <iostream> // std::cout — printing benchmark results #include <vector> // std::vector — level-order test inputs #include <chrono> // std::chrono — steady_clock timing for benchmark #include <sys/resource.h> // getrusage — peak-RAM measurement for benchmark #include "tree_node.h" // shared TreeNode + buildTree/printTree/destroyTree

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- Returning -1 for failure is the single-pass standard; returning a bool with an out-parameter height costs a second traversal.
- An iterative post-order with a `(node, state)` stack avoids the recursion limit at 5e3 depth.
- `return -1 if ... else 1 + h` written as one expression avoids a Python-level branch and a temp variable.
- A generator-free explicit stack is safer than recursion past 1000 in CPython, and 5e3 exceeds that limit.

INTERVIEW TRAPS & SANITY CHECKS

Recomputing both subtree heights with separate calls inside the parent, which visits subtrees repeatedly and can become $O(n^2)$ on skewed inputs. Returning height from a single pass avoids that entirely.

Sanity check. `height(n)` returns the height of n's subtree if that subtree is balanced at every node, or -1 if any node in the subtree is unbalanced. Because -1 propagates up, once a violation is found every ancestor also returns -1 and the whole answer is false.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is a single return value better than a bool plus a height?

Answer. Because it halves the work. The parent needs both the height (to compute its own) and the validity (to fail fast). Returning -1 on failure gives one int that answers both, so there is no second traversal and no out-parameter.

Q2. What is the worst-case time complexity?

Answer. $O(n)$ – each node is visited exactly once in post-order, whether the tree is balanced or not. The naive version that recurses for left height, right height, and then again to check, visits each node once per ancestor and is $O(n^2)$ on a chain.

Q3. How do you avoid the Python recursion limit here?

Answer. Convert to an explicit stack of nodes with a visited flag, or push `(node, False)` then `(node, True)` and children to emulate post-order. That replaces the C-level call stack with a heap list you control.

0124 Binary Tree Maximum Path Sum Hard

ESSENCE & BOUNDS

Return the largest sum of values along any path in a binary tree, where node values may be negative and the path may start and end anywhere. -1e3 <= node.val <= 1e3 and up to 3e4 nodes, so the answer can be as low as -1e3 (a single negative node) and clamping at zero must not make the answer 0 incorrectly. A path cannot revisit a node, so it bends at most once.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Post-order with best-so-far (BEST) dfs(node) returns the best single-branch down-sum through node; the caller updates a shared `best` with the full path through it. Negative branches are dropped (max(0, child)) — a path may simply not extend into a subtree that would shrink it.

Python (intuition kernel)

```python
self.best = float("-inf") # -$\{}infty$:  every node re-arms the max
self._down_sum(root)
return int(self.best)
if node is None:
left = max(0, self._down_sum(node.left))
right = max(0, self._down_sum(node.right))
self.best = max(self.best, left + node.val + right)
return node.val + max(left, right)

```

C++ (intuition kernel)

```cpp
if (node == nullptr) {

```

Time O(n) **Space** O(h), h = height.

Follows directly from the traversal count: $O(n)$ time, $O(h)$, h = height. space.

Brute force, path-through-every-node (contrast) For every node compute the max down-sum through each child with a separate helper, then combine. Same idea, but each child's down- sum is recomputed for every ancestor.

Python (intuition kernel)

```python
if root is None:
self.best = float("-inf")
self._consider(root) # every node as the path's peak
return int(self.best)

```

C++ (intuition kernel)

```cpp
if (root == nullptr) {

```

Time O(n^2) worst **Space** O(h).

nested enumeration dominates: $O(n\hat{}k)$ pairs, no pruning; hence the stated bounds.

VISUAL STATE TRACE

```
-10 best through -10:
/ \{}
9 20 9 + (-10) + 20 = 19
/ \{}
15 7
dfs(n) = best DOWNWARD path starting at n (one branch only)
dfs(15) = 15 best so far stays 0
dfs(9) = 9 + max(0, 15) = 24
dfs(7) = 7 + max(0, 0) = 7
dfs(20) = 20 + max(0, 7) = 27
at -10:  candidate = 9-path(24) + (-10) + 20-path(27)
= 24 - 10 + 27 = 41 <- the answer, a BENT path
dfs(-10) = -10 + max(0, 24) = 14 (one-sided, for the parent)
TWO different quantities:
return value = may extend upward -> ONE branch -> clamp
best tracker = any shape -> TWO branches -> no clamp
Conflating them is THE classic bug.
```

LANGUAGE MECHANICS: PYTHON VS C++

dfs returning ONE value while a shared accumulator tracks the answer is the canonical Hard-tree pattern: the return value is the "legal to extend" summary (one branch only), the global is the "any shape" answer (both branches, no parent). Confusing the two is THE classic bug on this problem — a path can bend through the node (left + right) but can only EXTEND upward along one branch. Clamping child contributions at 0 is safe because an empty path is always allowed. dfs returning ONE value while a shared accumulator tracks the answer is the canonical Hard-tree pattern: the return value is the "legal to extend" summary (one branch only), the global is the "any shape" answer (both branches, no parent). Confusing the two is THE classic bug on this problem — a path can bend through the node (left + right) but can only EXTEND upward along one branch. Clamping child contributions at 0 is safe because an empty path is always allowed.
==========
/ #include <iostream> // std::cout — printing benchmark results #include <vector> // std::vector — level-order test inputs #include <algorithm> // std::max — clamping child contributions #include <climits> // INT_MIN — the "∞" starter for best-so-far #include <chrono> // std::chrono — steady_clock timing for benchmark #include <sys/resource.h> // getrusage — peak-RAM measurement for b

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- Passing `int& best` avoids a member variable and makes the function reentrant.
- `max(0, dfs(left))` is the clamp; without it a negative subtree poisons every ancestor's path.
- `self.best` as an instance attribute is the Python idiom; a nested function with a mutable cell is faster.
- Returning 0 from a null child makes the clamp unnecessary: `max(left, right)` already excludes negatives.

INTERVIEW TRAPS & SANITY CHECKS

Returning `node.val + left + right` (both branches) instead of the one-sided sum. That returns the best bent path, which the parent cannot extend upward, so ancestors compute wrong values.

Sanity check. `dfs(n)` returns the maximum sum of a path that starts at n and goes down into at most one subtree. Clamping children at 0 is safe because a path is not required to extend into a subtree at all, and the empty extension contributes 0.

INTUITION CHECK — 3 QUESTIONS

Q1. Why clamp at zero when the values may be negative?

Answer. Because a path is not obliged to visit any child. Choosing not to descend contributes 0, which is always at least as good as descending into a negative subtree. Clamping therefore models a legal choice rather than distorting the optimum.

Q2. Why are the return value and the global answer different quantities?

Answer. A path that bends through n uses both children and cannot be extended upward (the parent would revisit n). A path that starts at n and goes down uses one child and can be extended. So the return value is one-sided while the tracked answer is two-sided.

Q3. What happens on an all-negative tree?

Answer. Every clamp zeroes out, so each return is the node's own value, and the best tracker records the maximum single node value – which is the correct answer, since a path must contain at least one node.

0199 Binary Tree Right Side View Medium

ESSENCE & BOUNDS

Return the values seen from the right side, top to bottom – one value per level. The tree may be a single node, so there is exactly one level. A right-skewed tree gives all nodes in order.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

BFS level snapshots (BEST, intuitive) Same per-level snapshot loop as level-order traversal; instead of storing the whole row, keep only its last element.

Python (intuition kernel)

```python
view: List[int] = []
if root is None:
 return view
pending = [root] # FIFO -- next frontier
while pending:
 last_val = 0 # will hold the row's tail
 for _ in range(len(pending)):
 cur = pending.pop(0)

```

C++ (intuition kernel)

```cpp
if (root == nullptr) { return view;

```

Time O(n) **Space** O(n) — queue width.

every node is enqueued and dequeued once; hence the stated bounds.

DFS, right-first, first-visit-per-depth dfs(node, depth): if depth equals the count of levels recorded so far, this is the FIRST node seen at that depth → append it. Then recurse RIGHT first, LEFT second — rightmost wins the race.

Python (intuition kernel)

```python
view: List[int] = []
self._walk(root, 0, view)
return view

```

C++ (intuition kernel)

```cpp
if (node == nullptr) { return;

```

Time O(n) **Space** O(h), h = height.

every node is visited once and returns $O(1)$ state; hence the stated bounds.

VISUAL STATE TRACE

```
1
/ \{}
2 3
/ \{} / \{}
4 5 6 7
right side view = [1, 3, 7]
BFS: per level take the LAST enqueued node
level 0:  queue=[1] -> take 1
level 1:  queue=[2,3] -> take 3
level 2:  queue=[4,5,6,7] -> take 7
DFS right-first with OVERWRITE per depth:
depth 0:  visit(1) -> res[0] = 1
depth 1:  visit(3) -> res[1] = 3 (2 then overwrites?  no:
visiting right child first means 3 is set FIRST)
depth 2:  visit(7) -> res[2] = 7 (5 would overwrite - so
either visit right-first and overwrite, or
left-first and skip-if-set)
invariant BFS: nodes at each level enter the queue in
left-to-right order, so index size-1 is the rightmost.
```

LANGUAGE MECHANICS: PYTHON VS C++

BFS sees levels HORIZONTALLY (who is last in the row) — $O(w)$ queue memory. DFS sees levels VERTICALLY (who is first to arrive at a depth) — $O(h)$ stack memory. On a wide-but-shallow tree BFS costs more memory; on a narrow-but-deep tree DFS does. Both are $O(n)$ time; the choice is which axis you want to pay for. BFS sees levels HORIZONTALLY (who is last in the row) — $O(w)$ queue memory. DFS sees levels VERTICALLY (who is first to arrive at a depth) — $O(h)$ stack memory. On a wide-but-shallow tree BFS costs more memory; on a narrow-but-deep tree DFS does. Both are $O(n)$ time; the choice is which axis you want to pay for.
==========
/ #include <iostream> // std::cout — printing benchmark results #include <vector> // std::vector — output + level-order tests #include <queue> // std::queue — FIFO used by approach 1 #include <chrono> // std::chrono — steady_clock timing for benchmark #include <sys/resource.h> // getrusage — peak-RAM measurement for benchmark #include "tree_node.h" // shared TreeNode + buildTree/printTree/destroyTree

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `queue` + per-level size is the standard; taking `q.back()` before popping the level is the one-liner.
- A DFS with an explicit stack must push LEFT first so RIGHT is popped first.

- A deque plus `for _ in range(len(q))` and reading the last popped value is the idiomatic version.
- Storing `(node, depth)` and overwriting `res[depth]` is a clean DFS alternative.

INTERVIEW TRAPS & SANITY CHECKS

In the DFS variant, overwriting `res[depth]` while visiting LEFT first – then the leftmost node wins. The correct pairing is left-first traversal WITH overwrite, or right-first traversal WITH first-write-only.

Sanity check. In BFS, before each level's loop the queue holds exactly that level in left-to-right order, so the last element popped is the rightmost node of the level. One per level therefore gives the right view.

INTUITION CHECK — 3 QUESTIONS

Q1. Which node is the right view of a given level?

Answer. The rightmost, which is the last node enqueued at that depth because BFS enqueues children left to right. Reading the queue's back before draining the level gives it in $O(1)$.

Q2. How do the two DFS variants differ, and which is right?

Answer. Traversing right-first and overwriting works, because the first node written at each depth IS the rightmost and later left-side visits at the same depth overwrite it with a worse value. Traversing left-first requires first-write-only. Mixing them up is the bug.

Q3. Why is BFS conceptually safer than DFS here?

Answer. Because BFS's level structure is explicit – the queue length tells you the level size, so "one per level" is enforced structurally rather than by tracking a depth argument through recursion.

0226 Invert Binary Tree Easy

ESSENCE & BOUNDS

Invert a binary tree by swapping every node's left and right subtrees. Up to 1e4 nodes, so a skewed tree recurses 1e4 deep and blows CPython's default limit. Modifying in place is expected and avoids a second tree.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Recursive swap (BEST, 3 lines of essence) swap(node.left, node.right); invert(left); invert(right).

Python (intuition kernel)

```python
if root is None:
root.left, root.right = root.right, root.left
self.invertTree(root.left)
self.invertTree(root.right)
return root

```

C++ (intuition kernel)

```cpp
if (root == nullptr) { return nullptr;

```

Time O(n) **Space** O(h), h = height.

Follows directly from the traversal count: $O(n)$ time, $O(h)$, h = height. space.

Iterative BFS/DFS with an explicit queue Same swaps, but a queue (or stack) holds the nodes still awaiting their swap — no recursion, safe on pathologically deep trees where the runtime stack would overflow.

Python (intuition kernel)

```python
if root is None:
pending = [root]
while pending:
 cur = pending.pop(0)
 cur.left, cur.right = cur.right, cur.left
 if cur.left is not None:
 pending.append(cur.left)
 if cur.right is not None:

```

C++ (intuition kernel)

```cpp
if (root == nullptr) { return nullptr;

```

Time O(n) **Space** O(w), queue width.

every node is visited once and returns $O(1)$ state; hence the stated bounds.

VISUAL STATE TRACE

```
4 4
/ \{} / \{}
2 7 -> 7 2
/ \{} / \{} / \{} / \{}
1 3 6 9 9 6 3 1
invert(n):
if not n:  return
n.left, n.right = n.right, n.left
invert(n.left)
invert(n.right)
invariant:  on entry, n's subtree is un-inverted; on return it
is fully inverted.  The swap must come BEFORE recursing,
otherwise the recursion visits the wrong subtrees.
3 lines of essence, O(n) time, O(h) space.
Recursion depth = h; a 1e4 chain exceeds Python's limit.
```

LANGUAGE MECHANICS: PYTHON VS C++

The tree is a set of heap nodes linked by pointers. "Inverting" changes NO node values and allocates NOTHING — it rewrites two pointers per node in place. That is why the operation is $O(n)$ time and $O(1)$ extra space per node; the recursion/queue memory is the only overhead. Watch the addresses stay fixed; only the arrows flip. The tree is a set of heap nodes linked by pointers. "Inverting" changes NO node values and allocates NOTHING — it rewrites two pointers per node in place. That is why the operation is $O(n)$ time and $O(1)$ extra space per node; the recursion/queue memory is the only overhead. Watch the animation: the addresses stay fixed, the arrows flip.
==========
/ #include <iostream> // std::cout — printing benchmark results #include <vector> // std::vector — level-order test inputs #include <queue> // std::queue — FIFO work list (approach 2) #include <utility> // std::swap — flipping child pointers #include <chrono> // std::chrono — steady_clock timing for benchmark #include <sys/resource.h> // getrusage — peak-RAM measurement for benchmark #include "tree_node.h" // shared TreeNode + buildTree/printTree/destroyTree

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `swap(node->left, node->right)` from `<algorithm>` is a single inline exchange.
- An explicit stack of nodes avoids 1e4-deep recursion and is barely more code.

- Tuple assignment `node.left, node.right = node.right, node.left` is one bytecode pair and reads correctly.
- An explicit stack `[root]` with `node.left, node.right = node.right, node.left` then pushing both children is $O(1)$ extra space beyond the stack.

INTERVIEW TRAPS & SANITY CHECKS

Recursing before swapping. Then `invert(node.left)` operates on the node's original left subtree, and after the swap the right subtree is never visited – silently leaving half the tree uninverted.

Sanity check. `invert(n)` returns with n's subtree fully inverted and all other parts of the tree untouched. The pre-order placement of the swap is what preserves this: the swap happens on entry, so both recursive calls see subtrees that have already been repositioned.

INTUITION CHECK — 3 QUESTIONS

Q1. Why must the swap precede the recursion?

Answer. Because the recursive calls should operate on the node's children as they will be visited in the inverted tree. Swapping first and then recursing on `node.left` and `node.right` visits both original subtrees exactly once.

Q2. Can this be done without recursion?

Answer. Yes – push the root, then repeatedly pop a node, swap its children, and push both. That is $O(n)$ time and $O(h)$ stack, with no recursion limit concern. Worth mentioning given the 1e4-node bound.

Q3. What is the minimum information a node needs?

Answer. Just its two child pointers. The values are irrelevant, which is a useful observation: the problem is purely structural, so any node with two mutable child slots can be inverted.

0230 Kth Smallest Element in a BST Medium

ESSENCE & BOUNDS

Return the k-th smallest value in a binary search tree with distinct values, counting from 1. 1 <= k <= n and n <= 1e4, so k is always valid and the early exit always triggers. A skewed tree recurses 1e4 deep, which is why the iterative version is preferred.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Iterative inorder with early stop (BEST) Explicit stack descends the left spine, pops the smallest unseen value, counts it, then steps right. Stop at count == k.

Python (intuition kernel)

```python
pending: List[TreeNode] = [] # explicit call-stack stand-in
cur: Optional[TreeNode] = root
visited = 0 # values already emitted in order
while cur is not None or pending:
 while cur is not None:
 pending.append(cur)
 cur = cur.left
 cur = pending.pop()

```

C++ (intuition kernel)

```cpp
while (cur != nullptr || !pending.empty()) { while (cur != nullptr) { pending.push(cur);
 cur = cur->left;
 }
 cur = pending.top();
 pending.pop();
 if (++visited == k) { return cur->val;
 }
 cur = cur->right; // continue with the right subtree

```

Time O(h + k) — spine descent + k pops.

Space O(h) — explicit stack.

Follows directly from the traversal count: $O(h + k)$ — spine descent + k pops. time, $O(h)$ — explicit stack. space.

Recursive inorder with early stop Identical ordering, implicit call stack; the recursion returns a boolean "already found?" to unwind early.

Python (intuition kernel)

```python
self.visited = 0 # reset per call (member state)
return self._walk(root, k)

```

C++ (intuition kernel)

```cpp
if (node == nullptr) {
 return -1; // empty subtree holds nothing

```

Time O(h + k) — same walk, same stop condition.

Space O(h) — call stack.

Follows directly from the traversal count: $O(h + k)$ — same walk, same stop condition. time, $O(h)$ — call stack. space.

VISUAL STATE TRACE

```
5
/ \{}
3 6
/ \{} / \{}
2 4 7 8
inorder:  2 3 4 5 6 7 8
k=3 -> the 3rd value = 4
stack, cur = [], root
push 5 ; cur = 3
push 3 ; cur = 2
push 2 ; cur = None -> pop 2 visit 1
cur = None -> pop 3 visit 2
cur = 4 -> (4 unvisited, not None)
^ need pop-after-LEFT-visited, not pop-on-null-first
push 4 ; cur = None -> pop 4 visit 3 <- RETURN 4
invariant:  nodes on the stack are the unvisited ancestors in
descending order, so the top is always the smallest available.
```

LANGUAGE MECHANICS: PYTHON VS C++

A BST stores every node's left subtree BELOW it and right subtree ABOVE it. Inorder emits left → node → right, so a value arrives only after everything smaller (left subtree) and before everything larger (right subtree) — induction over the whole tree gives a fully sorted sequence. Any "k-th" question about a BST is really an inorder question; the explicit-stack version is the classic interview answer because it makes the early stop visible. A BST stores every node's left subtree BELOW it and right subtree ABOVE it. Inorder emits left → node → right, so a value arrives only after everything smaller (left subtree) and before everything larger (right subtree) — induction over the whole tree gives a fully sorted sequence. Any "k-th" question about a BST is really an inorder question; the explicit-stack version is the classic interview answer because it makes the early stop visible.
==========
/ #include <iostream> // std::cout — printing benchmark results #include <vector> // std::vector — level-order test inputs #include <stack> // std::stack — explicit inorder stack #include <chrono> // std::chrono — steady_clock timing for benchmark #include <sys/resource.h> // getrusage — peak-RAM measurement for benchmark #include "tree_node.h" // shared TreeNode + buildTree/printTree/destroyTree

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- A `stack<TreeNode*>` with `while (cur || !st.empty())` is the canonical iterative form.
- Using the size of the left subtree to compute the rank is a different $O(h)$ strategy: `k -= leftSize + 1` on each step.
- The two-phase loop `while cur or stack` with `stack.append/pop` is the standard Python form.
- `while stack and k > 0:  node = stack.pop(); k -= 1` is the clean early-exit shape.

INTERVIEW TRAPS & SANITY CHECKS

Decrementing k when pushing rather than when popping. The visit happens on pop, after the left subtree is exhausted, so that is the only place the rank advances.

Sanity check. The stack holds, from top to bottom, the ancestors whose left subtrees have been fully visited but whose own values have not yet been emitted. Popping therefore always yields the next smallest value in sorted order.

INTUITION CHECK — 3 QUESTIONS

Q1. Why does inorder give sorted order here?

Answer. The BST invariant places every value in the left subtree below the node and every value in the right subtree above it, so a left-node-right traversal emits values in strictly increasing order.

Q2. What is the complexity, and why can it be better than $O(n)$?

Answer. $O(h+k)$ for the iterative version – you descend at most h levels and pop at most k nodes, then stop. Worst case that is $O(n)$, but for small k on a balanced tree it is much closer to $O(\log n + k)$.

Q3. When would a size-based rank computation beat inorder?

Answer. If you augmented each node with subtree size, you could descend directly to the k-th element by comparing k against the left subtree's size – $O(h)$ regardless of k, and no stack. That needs extra state the problem does not provide.

0235 Lowest Common Ancestor Of Bst ?

ESSENCE & BOUNDS

Return the lowest node in a binary search tree that is an ancestor of (or equal to) both given nodes. p and q are distinct nodes in the tree, so a solution always exists. Values are unique, which is what makes the ordering comparisons unambiguous.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

v1 v1 – see the kernel below.

Python (intuition kernel)

```python
while root:
 if p.val < root.val and q.val < root.val:  root = root.left
 elif p.val > root.val and q.val > root.val:  root = root.right
 else:  return root

```

C++ (intuition kernel)

```cpp
while (cur != nullptr) { if (p->val < cur->val && q->val < cur->val) { cur = cur->left;
 }
 else if (p->val > cur->val && q->val > cur->val) { cur = cur->right;
 }
 else { return cur;
 }

```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

v2 v2 – see the kernel below.

Python (intuition kernel)

```python
if not root:  return None
if p.val < root.val and q.val < root.val:  return self.lowestCommonAncestor(root.left, p, q)
if p.val > root.val and q.val > root.val:  return self.lowestCommonAncestor(root.right, p, q)
return root

```

C++ (intuition kernel)

```cpp
if (root == nullptr) { return nullptr;

```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

VISUAL STATE TRACE

```
6
/ \{}
2 8
/ \{} / \{}
0 4 7 9
/ \{}
3 5
p = 2, q = 8 -> LCA = 6
p = 2, q = 4 -> LCA = 2
p = 3, q = 5 -> LCA = 4
descend(root=6):
both < 6 ?  (0,0) yes -> go LEFT
both > 6 ?  yes -> go RIGHT
otherwise root is the split point -> ANSWER
p=2 q=4:  at 6, 2<6 but 4<6 -> LEFT to 2.
2 is p itself -> return 2 (ancestor of itself)
invariant:  at every node, p and q are both <= node or
both >= node, unless the node is the answer.
```

LANGUAGE MECHANICS: PYTHON VS C++

For a general binary tree, LCA needs a parent-pointer/ancestor-set or a post-order search ($O(n)$). Here the ORDERING prunes one side of the tree at every step — the search space halves per level, giving $O(h)$ instead of $O(n)$. The splitting test (p < cur < q or q < cur < p) is the entire algorithm in one condition; the LCA is the node where that test first succeeds on the descent. ========== / #include <iostream> // std::cout — printing benchmark results #include <vector> // std::vector — level-order test inputs #include <chrono> // std::chrono — steady_clock timing for benchmark #include <sys/resource.h> // getrusage — peak-RAM measurement for benchmark #include "tree_node.h" // shared TreeNode + buildTree/printTree/destroyTree

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- A `TreeNode*` descent is $O(h)$ time and $O(1)$ space – strictly better than the general-tree ancestor-path method.
- `std::numeric_limits` is irrelevant here since no sentinel values are needed.

- The while loop mutates `root` in place, which is fine since the caller does not need it.
- For a general binary tree the same problem needs the parent-map approach; the BST ordering is what removes that requirement.

INTERVIEW TRAPS & SANITY CHECKS

Returning when `node == p or node == q` without first checking whether the other node is in that subtree. The correct order is: check the ordering split first, and only match a node when the two targets lie on opposite sides (or one equals it).

Sanity check. At the current node, both p and q are at most the node's value or both at least it, and neither is below the node on one side while the other is on the other. The first node where they split – or where one equals the node – is their lowest common ancestor.

INTUITION CHECK — 3 QUESTIONS

Q1. Why does the split point have to be the LCA?

Answer. If p and q are both below the current node's value they are both in the left subtree, so no ancestor at or above this node can be their LCA – it must be deeper. Symmetrically when both are above. So the search descends until the targets separate, and the node where they separate is the answer.

Q2. How does the general binary tree version differ?

Answer. Without ordering you cannot tell which subtree to descend, so you must record ancestor sets (or a parent map) for p, then walk up from q until you hit one of p's ancestors. That is $O(h)$ time but $O(h)$ space instead of $O(1)$.

Q3. What if p is an ancestor of q?

Answer. The descent walks down toward p, and when it reaches p the equality branch fires and p is returned. Treating a node as an ancestor of itself is the standard convention and is what makes that case work without special handling.

0297 Serialize And Deserialize Binary Tree ?

ESSENCE & BOUNDS

Encode a binary tree to a single string and recover the exact original tree from it. Up to 1e4 nodes, so the encoded string can be 2n+1 tokens. The encoding must be unambiguous – any tree must decode to itself and only itself.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Solution 1 Solution 1 – see the kernel below.

Python (intuition kernel)

```python
def serialize(root):
 out = []
 def walk(n):
 if n is None:  out.append('#'); return
 out.append(str(n.val)); walk(n.left); walk(n.right)
 walk(root)
 return ','.join(out)

def deserialize(s):
 toks = iter(s.split(','))
 def build():
 t = next(toks)
 if t == '#':  return None
 n = TreeNode(int(t))
 n.left, n.right = build(), build()
 return n
 return build()

```

C++ (intuition kernel)

```cpp
if (node == nullptr) { if (!out.empty()) { out += ","; }
 out += "null";
 return;

```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

Solution 2 Solution 2 – see the kernel below.

Python (intuition kernel)

```python
def serialize(root):
 return repr(root) # __repr__ writes the preorder +
 # null-marker token stream

def deserialize(s):
 return eval(s) # only for a controlled input

```

C++ (intuition kernel)

```cpp
if (root == nullptr) {
 return ""; // empty tree $\{}to$ empty payload

```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

VISUAL STATE TRACE

```
tree:  9
/ \{}
3 20
/ \{} / \{}
2 1 15 7
preorder with NULL markers:
9 3 2 # 1 # # 20 15 # # 7 #
^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^
| | | | | | | +- null for 7's right
| | | | | +- null for 20's left
| | | | +- null for 1's right
| | +- null for 1's left
| +- null for 3's right
+- root
invariant:  exactly 2n+1 tokens for n nodes, since every node
contributes one value and every null child contributes a marker.
The marker count equals n+1, which is a self-check.
decode:  consume a token; '#' -> null; else build node and
recurse twice (left then right).  No index bookkeeping.
```

LANGUAGE MECHANICS: PYTHON VS C++

n-node binary trees come in Catalan-many shapes but a traversal without markers has only 2n1 slots to describe 2n pointers — the null entries ARE the missing pointers. Preorder+nulls is exactly the classic "1 2 X X 3 X X" format; level-order+nulls is exactly the "[3,9,20,null,null,15,7]" format Leet-Code's own test harness uses. Both round-trip perfectly — the marker discipline is the entire trick. ==========
/ #include <iostream> // std::cout — printing benchmark results #include <vector> // std::vector — token storage / level-order #include <string> // std::string — the serialized payload #include <sstream> // std::istringstream — token splitting #include <queue> // std::queue — BFS (approach 2) + deserialize #include <chrono> // std::chrono — steady_clock timing for benchmark #include <sys/resource.h> // getrusage — peak-RAM measurement for benchmark #include "tree_node.h" // shared TreeNode + buildTree/printTree/destroyTree

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `stringstream` with `string` tokens avoids manual delimiter parsing and is fast enough at 1e4 nodes.
- Appending the delimiter as a character (`','`) plus a separate null marker is cheaper than a string per token.
- `','.join(tokens)` in one pass avoids repeated string concatenation, which would be $O(n^2)$ in CPython.
- `tokens.split(',')` in C beats any manual character loop.

INTERVIEW TRAPS & SANITY CHECKS

Omitting the null markers. Preorder values alone are ambiguous – [1,2] and [1,null,2] both encode to `1,2`. Every null child must contribute a marker or decoding is not injective.

Sanity check. Decoding consumes tokens left to right; each node token is followed by exactly two subtrees (left then right). Because every subtree is itself a valid encoding, the recursive consumption always lines up and the string is consumed exactly.

INTUITION CHECK — 3 QUESTIONS

Q1. Why are null markers necessary for injectivity?

Answer. Without them, a preorder value sequence cannot distinguish a node with a left child from one without. [1,2] encodes to `1,2` whether 2 is a left or a right child, and also whether 1 has an empty right subtree. Marking nulls fixes both ambiguities.

Q2. How many tokens does a tree of n nodes produce?

Answer. Exactly 2n+1: n values plus n+1 null markers (every node has two child slots, but the root has no parent, so there are 2n slots and n-1 are filled, leaving n+1 null). That identity is a useful integrity check on any implementation.

Q3. What is the complexity?

Answer. $O(n)$ to serialize and $O(n)$ to deserialize, with the token list itself being $O(n)$ space. The recursion adds $O(h)$, and a skewed tree has h = n, so the stack is the dominant term for pathological inputs.

0543 Diameter Of Binary Tree ?

ESSENCE & BOUNDS

Return the length in edges of the longest path between any two nodes, which need not pass through the root. Up to 1e4 nodes, so the diameter can reach 1e4 - 1 and must be computed in edges. A single-node tree has diameter 0.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

v1 v1 – see the kernel below.

Python (intuition kernel)

```python
self.res = 0
if not node:  return 0
l, r = dfs(node.left), dfs(node.right)
self.res = max(self.res, l + r); return 1 + max(l, r)
dfs(root); return self.res

```

C++ (intuition kernel)

```cpp
if (node == nullptr) {

```

Time ?	Space ?

Follows directly from the traversal count: ? time, ? space.

VISUAL STATE TRACE

```
1
/ \{}
2 3
/ \{} / \{}
4 5 6 7
longest paths:
4-2-5 = 2 edges
4-2-1-3-6 = 4 edges <- THROUGH THE ROOT
6-3-7 = 2 edges
ANSWER 4 (not found by a naive root-only computation)
post-order:  dfs(n) -> (height, best)
h4 = (0,0) h5 = (0,0)
h2:  left=(0,0) right=(0,0)
through = 0 + 1 + 0 = 1
height = 1 + max(0,0) = 1
return (1, max(0, 1, 0)) = (1, 1)
h6 = (0,0) h7 = (0,0)
h3:  through = 0+1+0 = 1 ; height = 1
return (1, 1)
h1:  left=(1,1) right=(1,1)
through = 1 + 1 + 1 = 3 (edges through root)
height = 1 + max(1,1) = 2
best = max(1, 1, 3) = 3
return (2, 3) ANSWER 3 (edges)
invariant:  dfs(n).height is the height of n's subtree;
dfs(n).best is the diameter WITHIN n's subtree.
```

LANGUAGE MECHANICS: PYTHON VS C++

This is the same post-order DP as max path sum and balanced tree: the node's own candidate combines BOTH children (leftH + rightH), but the value passed UP is a single-branch summary (height). Encoded as an int return + an out-param/global, no struct needed: best = max(best, lh + rh) ← "bent" path, stays local return max(lh, rh) + 1 ← "straight" chain, goes up Watch it in the animation: every node reports its height while the running best silently updates. ========== / #include <iostream> // std::cout — printing benchmark results #include <vector> // std::vector — level-order test inputs #include <algorithm> // std::max — folding the best candidate #include <chrono> // std::chrono — steady_clock timing for benchmark #include <sys/resource.h> // getrusage — peak-RAM measurement for benchmark #include "tree_node.h" // shared TreeNode + buildTree/printTree/destroyTree

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- Returning a `pair<int,int>` (height, best) is the canonical shape and needs no member state.
- `std::max` with an initializer list of three values is a single call but may not inline as well as chained maxes.
- Returning a tuple per node allocates one tuple per node; returning just the height and using a closure-captured `best` avoids that.
- The closure approach is both faster and easier to read in Python.

INTERVIEW TRAPS & SANITY CHECKS

Returning the height as the answer, or forgetting that the two deepest branches contribute `leftH + 1 + rightH` edges – the two +1s for the edges from the node to its children are the off-by-one that flips the whole answer.

Sanity check. `dfs(n)` returns the height of n's subtree and the diameter of n's subtree. The diameter is the max of the two children's diameters and the path through n, and the height is 1 plus the taller child. Both facts are computable from the children's return values because the recursion is post-order.

INTUITION CHECK — 3 QUESTIONS

Q1. Why must the diameter consider paths not through the root?

Answer. The longest path may lie entirely inside one subtree. The post-order `best` value carries each child's internal diameter upward precisely so the parent can compare it against the through-node path and its own other child.

Q2. Is the diameter in nodes or edges?

Answer. Edges. That is why the through-node candidate is `leftH + 1 + rightH` – two edges from n to its children – and why a leaf returns height 0 and best 0.

Q3. Why is post-order the natural traversal here?

Answer. Because both facts about a node depend only on facts about its children. Post-order guarantees the children's return values are available when the node composes them, which is exactly the dependency this recurrence has.

0572 Subtree Of Another Tree ?

ESSENCE & BOUNDS

Decide whether one binary tree is a descendant of another, matching structure and values exactly. 0 <= subRoot.val, root.val <= 1000 and up to 1000 nodes each, so an $O(n * m)$ recursive comparison is technically acceptable at 1e6 operations. Nodes need not be unique.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

v1 v1 – see the kernel below.

Python (intuition kernel)

```python
if not root:  return False
if self._same(root, subRoot):  return True
return self.isSubtree(root.left, subRoot) or self.isSubtree(root.right, subRoot)
if not p and not q:  return True
if not p or not q or p.val != q.val:  return False
return self._same(p.left, q.left) and self._same(p.right, q.right)

```

C++ (intuition kernel)

```cpp
if (root == nullptr) { return false;

```

Time ?	Space ?

Follows directly from the traversal count: ? time, ? space.

VISUAL STATE TRACE

```
root:  3
/ \{}
4 5
\{}
1
subRoot:  4
\{}
1
descend(root=3):  val 3 != 4 -> try root.left (4)
val 4 == 4 ; subRoot.right is None, so must match
(null-safe check!)  -> root[4].right = 1 matches 1, and
1's children are both null -> MATCH
serialise with nulls (the safe encoding):
root:  "3 4 1 # # 5 # #"
subRoot:  "4 1 # #"
-> the subRoot pattern occurs in the root string
why nulls matter:  without them, subRoot "1" would falsely match
the leaf 1 of a larger subtree.
```

LANGUAGE MECHANICS: PYTHON VS C++

Serializing WITHOUT null markers is ambiguous ([1,2] could be several shapes) — a naive find would give false positives. The marker discipline from 0297 makes the encoding injective, which is precisely what turns a tree problem into a string problem. ========== / #include <iostream> // std::cout — printing benchmark results #include <vector> // std::vector — level-order test inputs #include <string> // std::string — serialized canonical forms #include <chrono> // std::chrono — steady_clock timing for benchmark #include <sys/resource.h> // getrusage — peak-RAM measurement for benchmark #include "tree_node.h" // shared TreeNode + buildTree/printTree/destroyTree

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `stringstream` plus `std::search` is a clean implementation; KMP avoids the quadratic worst case of naive search.
- Comparing the recursive shapes directly is $O(n * m)$ worst case but uses no extra space and is often fast in practice.
- `' '.join` on a serialized list plus Python's `in` operator is the shortest correct answer, and `in` on strings uses a fast two-way search.
- `' '.join(list)` in `haystack.split(' ')` style substring search sidesteps delimiter-boundary false matches.

INTERVIEW TRAPS & SANITY CHECKS

Forgetting to compare the NULL children. If you only compare values and recurse while both are non-null, `[1]` matches a leaf that is actually the left child of a two-node subtree, giving a false positive.

Sanity check. `isSubtree(a, b)` is true iff a's value equals b's value AND (b is a leaf, OR a's corresponding child subtree matches b's). Writing the base cases explicitly – both null is true, exactly one null is false – is what makes it correct.

INTUITION CHECK — 3 QUESTIONS

Q1. Why include nulls in the serialization?

Answer. Because a subtree match must agree on structure, and value sequences alone cannot distinguish a leaf from a node with one non-null child. Emitting a marker for every empty slot makes the pattern unambiguous, so a plain substring search becomes exactly a subtree search.

Q2. What is the time complexity of the serialization approach?

Answer. $O(n + m)$ to serialize, plus a substring search. Python's `in` uses a linear-time algorithm on strings, so the whole thing is $O(n + m)$ in practice; a naive manual scan would be $O(n \cdot m)$.

Q3. Why can a naive recursive search be $O(n \cdot m)$?

Answer. There are up to n candidate root positions in the big tree, and each attempt may compare up to m nodes before failing. With identical shapes the first attempt succeeds quickly, but adversarial inputs with matching prefixes force the full product.

1448 Count Good Nodes In Binary Tree ?

ESSENCE & BOUNDS

Count nodes whose value is greater than or equal to every ancestor along the path from the root to that node. 0 <= node.val <= 1e4 and up to 1e4 nodes. The root is always good by definition, and equality with the minimum counts as good.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

v1 v1 – see the kernel below.

Python (intuition kernel)

```python
if not node:  return 0
res = 1 if node.val >= max_val else 0
max_val = max(max_val, node.val)
return res + dfs(node.left, max_val) +
dfs(node.right, max_val)
return dfs(root, root.val)

```

C++ (intuition kernel)

```cpp
if (node == nullptr) {

```

Time ?	Space ?

Follows directly from the traversal count: ? time, ? space.

VISUAL STATE TRACE

```
7 path min 7 -> GOOD
/ \{}
7 7 path min 7 -> both GOOD (equal counts)
/ \{}
2 6 path min 7 -> 2 < 7 NO, 6 < 7 NO
count = 3 (root + two 7s)
dfs(node, pathMin):
if not node:  return 0
pathMin = min(pathMin, node.val)
good = 1 if node.val >= pathMin else 0
return good + dfs(left, pathMin) + dfs(right, pathMin)
invariant:  pathMin is the minimum value among the node and
ALL its ancestors.  So 'node.val >= pathMin' is exactly the
definition of a good node.
NOTE: comparing only against the PARENT is the bug - node 7
under parent 7 passes, but a node 5 under parent 6 under a
grandparent 3 would pass the parent test yet fail against 3?  no:
5 >= 6 fails.  Correct example:  5 under 6 under 2 -> parent test
passes (5 < 6?  no, 5 < 6 so fails).  Use 9 under 8 under 3:
parent test 9>=8 passes, grandparent 3 gives min 3, 9>=3 passes.
The failing case is 5 under 6 under 1:  parent 6, grandparent 1;
a node 2 under 6 -> 2 >= 6 fails anyway.  The real bug case is
a node whose value beats its parent but loses to a HIGHER
ancestor that is smaller - impossible.  Hence tracking the
running MINIMUM (not the parent) is the correct formulation,
and parent-only comparison is merely incomplete reasoning.
```

LANGUAGE MECHANICS: PYTHON VS C++

Some tree questions need context from the ROOT (this max), others need summaries from the LEAVES (diameter). This problem is a "top-down luggage" problem: the extra argument is what makes each subtree question well-posed, and it travels DOWN only. Contrast with 0543/0124 where the info travels UP — recognizing which direction the state flows is half of solving tree problems.

```
================================================================================
/ #include <iostream> // std::cout — printing benchmark results #include <vector> // std::vector — level-order test inputs #include <stack> // std::stack — explicit DFS stack (approach 2) #include <utility> // std::pair — (node, maxSoFar) frames #include <chrono> // std::chrono — steady_clock timing for benchmark #include <sys/resource.h> // getrusage — peak-RAM measurement for benchmark #include "tree_node.h" // shared TreeNode + buildTree/printTree/destroyTree
```

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `int pathMin` passed by value is the whole state; no member variable needed.
- Passing `INT_MAX` as the initial minimum is correct because all values are non-negative here; `INT_MIN` is the general choice.
- A closure with a mutable counter is faster than returning counts and summing.
- `float('inf')` as the initial minimum is the safe general sentinel.

INTERVIEW TRAPS & SANITY CHECKS

Testing against the immediate parent only. The definition is against ALL ancestors, and while a node that beats its parent may still lose to a smaller ancestor further up, the correct and cheap formulation is to carry the running minimum.

Sanity check. At entry to `dfs(node, m)`, `m` is the minimum value among all strict ancestors of node. After `m = min(m, node.val)`, `node.val >= m` is exactly the good-node predicate, and children receive that updated minimum.

INTUITION CHECK — 3 QUESTIONS

Q1. Why carry a minimum rather than the parent value?

Answer. Because the predicate quantifies over all ancestors. Carrying the running minimum reduces that quantification to one comparison: `val >= runningMin`. Keeping just the parent would require walking ancestors per node, turning $O(n)$ into $O(nh)$.

Q2. Is the root always good?

Answer. Yes – by convention a node with no ancestors vacuously satisfies "greater than every ancestor". Initialising the minimum to infinity makes the root's test `val >= min(val, inf)` trivially true, so no special case is needed.

Q3. What is the space complexity, and can it be reduced?

Answer. $O(h)$ for the recursion stack. It can be reduced to $O(1)$ with an explicit Morris-style threaded traversal that temporarily rewires pointers to carry the path minimum without a stack, at the cost of mutating the tree.

8. Tries

#	Problem	Level	One-line intent
0208	**Implement Trie (Prefix Tree)**	Medium	Each node holds a child map plus an end-of-word flag.
0211	**Design Add and Search Words Data Structure**	Medium	Treat '.' as a branch point during the descent.
0212	**Word Search II**	Hard	Each trie node walks the grid instead of matching its word.

0208 `Implement Trie (Prefix Tree)` Medium

ESSENCE & BOUNDS

Implement a prefix tree supporting insert, search, and startsWith. lowercase English letters only, so at most 26 children per node. With 1e4 words of length 100 the node count can reach 1e6, which makes per-node memory the real constraint in Python.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

26-array trie (BEST for a fixed lowercase alphabet) Node = { children[26], isEnd }. children[i] is the child for letter 'a'+i; isEnd marks where an inserted word terminates. insert : walk letters, allocate missing children, set isEnd. search : walk letters; missing child ⇒ false; return isEnd. startsWith: walk letters; missing child ⇒ false; else true — NO isEnd check, any descendant counts.

Python (intuition kernel)

```python
self.children: List[Optional["SolutionOne._Node"]] = [None] * 26
self.isEnd: bool = False # True = a word terminates here
self.root: SolutionOne._Node = SolutionOne._Node()
node = self.root
for ch in word:
idx = ord(ch) - 97 # 'a'$\{}to$0 \{}ldots{} 'z'$\{}to$25 -- one multiply to find child
if node.children[idx] is None:
node.children[idx] = SolutionOne._Node() # grow the path

```

C++ (intuition kernel)

```cpp
for (char c : word) {
int i = c - 'a'; // letter $\{}to$ slot 0..25
if (!n->children[i]) n->children[i] = new TrieNode(); // grow path
n = n->children[i]; // descend one level

```

Time O(L) per op — L = word length. **Space** O(26-n-L) worst case — n-L nodes, each holding

Follows directly from the traversal count: $O(L)$ per op — L = word length. time, $O(26 - n - L)$ worst case — n-L nodes, each holding space.

Hash set + linear prefix scan (contrast) Store every word verbatim in a Python set (open addressing with LINEAR PROBING: h(k, i) = (hash(k) + i) mod m, i = 0, 1, 2, \ldots{}). search : $O(1)$ average — one probe sequence. startsWith: scan ALL stored words and prefix-compare each — $O(n - L)$, because a hash set cannot answer prefix queries.

Python (intuition kernel)

```python
self.words: set[str] = set()
self.words.add(word)
return word in self.words
return any(w.startswith(prefix) for w in self.words)

```

C++ (intuition kernel)

```cpp
for (const std::string& w : words) { if (w.size() >= prefix.size() &&
w.compare(0, prefix.size(), prefix) == 0) { return true;
}

```

Time search O(1) avg; startsWith O(n-L). **Space** O(n-L).

one $O(1)$ hash probe per element, no rescans; hence the stated bounds.

VISUAL STATE TRACE

```
insert('apple') then insert('app')
root
|-a
| \{}-p
| \{}-p <- endOfWord (word 'app' ends here)
| \{}-l
| \{}-e <- endOfWord (word 'apple' ends here)
insert('apple','app','apply'):
root -> a -> p -> p (flag=True)
-> l -> e (flag=True)
-> y (flag=True)
search('app') -> follow a,p,p and return flag -> True
search('apx') -> no child 'x' -> False EARLY
startsWith('ap') -> follow a,p, end of pattern, no flag -> True
invariant:  a node is reached by exactly one string prefix, so
the flag at a node is the sole record of "a word ends here".
```

LANGUAGE MECHANICS: PYTHON VS C++

The trie is a 26-way tree: every node has 26 child slots (one branch per letter position) + an isEnd flag. Words sharing a prefix share EVERY node of that prefix — "app" then "apple" costs only 2 extra nodes, not 5. Hash sets (dict/set probing h(k,i) = (hash(k)+i) mod m) give $O(1)$ EXACT matches but cannot do prefix queries — a prefix is not a key. The trie converts them into a single $O(L)$ walk at the price of memory: each node pre-allocates 26 slots (26×8 B ≈ 208 B) even when 1–2 children exist. Fixed small alphabet → list children (index = ord(c)-97); unbounded alphabets (Unicode) → dict children allocate only what is used.
==========
A trie is a 26-way tree: every node carries 26 child pointers (one branch per letter position) + an isEnd flag. Two words that share a prefix share EVERY node of that prefix — inserting "app" then "apple" costs only 2 extra nodes, not 5. Hash maps (C++ unordered_set = separate chaining; Python set = open addressing with linear probing h(k,i) = (hash(k) + i) mod m) give $O(1)$ EXACT matches, but a prefix is not a key — "all words starting with 'app'" forces a full scan. The trie turns that query into one $O(L)$ walk. The price is memory: each node pre-allocates 26 pointers (26×8 B ≈ 208 B) even when only 1–2 children exist. Fixed small alphabet → array children (child lookup = 1 multiply, cac

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `vector<TrieNode>` with `children[26]` as a fixed array is cache-ideal but costs 104 bytes per node; a `map` or `vector<pair>` child list is denser for sparse nodes.
- `gp_hash_table<char,int>` per node is a poor fit: one hash table per node is heavy, and 26 edges do not justify hashing.
- A plain `dict` per node is the standard; a `list` of 26 is faster for lookup but wastes memory on sparse tries.
- `defaultdict(dict)` for children removes the membership test on insert.
- A set of all words supports startsWith by scanning, but search becomes $O(\sum |w|)$ – worse, though trivially correct.

INTERVIEW TRAPS & SANITY CHECKS

Confusing search with startsWith: `search('app')` must require the end-of-word flag, `startsWith('app')` must not. Returning the node without checking the flag makes every prefix a word.

Sanity check. Following the character sequence of a query reaches a node iff that sequence is a prefix of some inserted word. The flag at that node distinguishes "a word ends here" from "a path continues", which is the only difference between the two query methods.

INTUITION CHECK — 3 QUESTIONS

Q1. Why does a trie beat a set for startsWith?

Answer. A set requires scanning all words or building a separate prefix structure, and a scan is $O(n)$ regardless of query length. A trie answers startsWith in $O(L)$ where L is the query length, independent of the number of words stored.

Q2. What is the memory cost per node?

Answer. A Python dict costs roughly 64 bytes empty and grows with entries, so a million-node trie can exceed 200 MB. C++ with `children[26]` costs 104 bytes plus the flag, but compresses into a flat array with child indices – predictable and cache-friendly.

Q3. How would you support arbitrary characters?

Answer. Replace the fixed 26-array or lowercase assumption with a map, and index by code point rather than `ord(c) - ord('a')`. Unicode tries need care with surrogate pairs in narrow builds.

0211 Design Add and Search Words Data Structure Medium

ESSENCE & BOUNDS

Implement a trie supporting insert and a search that allows '.' to match any single character. 1 <= word.length <= 25 with lowercase letters and '.', at most 100 words stored. A query of all dots must not blow up – without pruning it explores 26^d paths.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Trie + recursive DFS (best) Walk the trie with an index i into the pattern. pattern[i] == '.' → try EVERY child; true if any matches. pattern[i] == 'c' → follow only child 'c'; missing ⇒ false. i == len(pattern) → answer = isEnd of the current node. The '.' branch is $O(26)$ per level worst case; letters stay $O(1)$.

Python (intuition kernel)

```python
self.root: _TrieNode = _TrieNode()
node = self.root
for ch in word:
 node = node.children.setdefault(ch, _TrieNode()) # grow path
node.isEnd = True # mark word completion
 if i == len(word):
 return node.isEnd
 ch = word[i]

```

C++ (intuition kernel)

```cpp
if (c == '.') { for (int k = 0; k < 26; ++k) {
 if (n->children[k] && dfs(n->children[k], w, i + 1)) return true;
 }
 return false;

```

Time O(26^d) worst case, d = number of '.'; O(L)

Space O(26-n-L) trie nodes worst case + O(L) call stack.

every node is visited once and returns $O(1)$ state; hence the stated bounds.

Trie + iterative BFS (level-by-level frontier) Keep a FRONTIER of candidate nodes. For each pattern char: '.' → frontier = ALL children of every candidate. 'c' → frontier = only the 'c'-children of candidates. Empty frontier ⇒ no match. Finally, true iff some surviving node has isEnd. Same complexity as DFS, no recursion stack, and every candidate advances in lockstep — the natural "wavefront" view.

Python (intuition kernel)

```python
self.root: _TrieNode = _TrieNode()
node = self.root
for ch in word:
 node = node.children.setdefault(ch, _TrieNode())
node.isEnd = True
frontier: deque[_TrieNode] = deque([self.root]) # nodes alive so far
for ch in word:
 next_frontier: deque[_TrieNode] = deque()

```

C++ (intuition kernel)

```cpp
for (char c : word) {
 std::vector<TrieNode*> next; // candidates for the NEXT level
 if (c == '.') { for (const TrieNode* n : frontier) { for (int k = 0; k < 26; ++k) {
 if (n->children[k]) next.push_back(n->children[k]);
 }
 }
 } else { for (const TrieNode* n : frontier) { TrieNode* child = n->children[c - 'a'];
 if (child) next.push_back(child);

```

Time O(26^d) worst case, O(L) without wildcards. **Space** O(26^d) frontier worst case.

every node is enqueued and dequeued once; hence the stated bounds.

VISUAL STATE TRACE

```
insert('bad'), insert('dad'), insert('mad')
search('pad'):
root -> a exists; then '.'  branches over b,d,m
'a'+'d' exists ; third char 'd' exists -> flag?  yes -> True
-> 3 branches at ONE level only (the whole query has
a single dot), so this is cheap
the blow-up case:  search('.....')  with 25 dots
26 branches at each of 25 levels = 26^25 paths.
PRUNING: if a dot is the LAST character of the pattern,
accept immediately if the current node has any children.
That single check kills the exponential case:
"\{}." at the end -> return bool(children)
invariant:  at level i, node sits on exactly the trie prefix
matching pattern[0..i-1] under the '.'  rule.
```

LANGUAGE MECHANICS: PYTHON VS C++

Without '.' this is exactly LeetCode 208's trie: node = 26 child slots + isEnd. With '.', search becomes a tree WALK with branching: every '.' multiplies the live paths by up to 26, so d wildcards cost up to 26^d node visits. DFS visits one path to its end before backtracking; BFS visits all paths one level at a time — identical work, different order. Python dict children (open addressing, linear probing h(k,i) = (hash(k)+i) mod m) are lean for '.'-scanning: iterating only existing children beats scanning 26 fixed slots.
==========
Without '.' this is exactly LeetCode 208's trie: each node = 26 children + isEnd. With '.', the search becomes a tree WALK with branching: every '.' multiplies the number of live paths by (up to) 26, so d wildcards cost up to 26^d node visits. DFS visits one path to its end before backtracking; BFS visits all paths one level at a time — identical work, different order. Python dict children (probing h(k,i) = (hash(k)+i) mod m) are the lean choice for '.'-scanning here: iterating only the children that EXIST beats scanning 26 fixed slots per branch.
==========
/ #include <iostream> // std::cout — printing benchmark results #include <vector> // std::vector — BFS frontier, test ops #include <string> // std::string — words and patterns #

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- Recursive DFS with `const vector<TrieNode*>& children` avoids copying the child vector per call.
- Iterative DFS with an explicit stack bounds the stack at $O(L)$ regardless of the branching factor.

- `for c in children[node]:` iterates values directly, saving a dict lookup per child.
- Recursion depth equals word length (25), so the default Python limit is never a concern here.

INTERVIEW TRAPS & SANITY CHECKS

Not pruning when the '.' is the final character of the pattern. `26^L` is astronomically slow, and the fix is a one-line check that a node with any child satisfies a trailing wildcard.

Sanity check. `search(node, i)` is true iff there is an inserted word matching `pattern[i:]` within node's subtree. The split into "pattern[i] == '.'" (try every child) and "concrete character" (take one child) covers exactly the wildcard semantics, so the recursion is exhaustive and sound.

INTUITION CHECK — 3 QUESTIONS

Q1. What is the worst-case time of the wildcard search?

Answer. Exponential in the number of dots, up to 26^d for d dots. The trailing-dot optimisation caps it, and in practice LeetCode inputs keep dot counts low – but stating the exponential bound and the pruning is the interview answer.

Q2. Why does a trailing dot return immediately?

Answer. Because any descendant of the current node completes the word. If the node has at least one child, some continuation exists and matches the single remaining dot. If it has no children, there is nothing to match. That test is exact, not a heuristic.

Q3. How would you add prefix search with dots?

Answer. Return true if the recursion reaches i == len(pattern) without failing – whether or not a flag is set – which is exactly `startsWith` generalised to wildcards.

0212 Word Search II Hard

ESSENCE & BOUNDS

Find all words in a dictionary that exist in a grid using adjacent horizontal or vertical moves, each cell at most once per word. up to 3e4 cells, 1e5 words of length up to 10e3. Pruning is essential: without removing found words and without early termination the search is hopeless.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Trie + grid DFS backtracking (foundation) Build the trie (each terminal node stores its full word, which doubles as a "found" marker). For every cell, DFS: prune when the letter has no trie child; mark cells '#' while visiting to stop re-use, restore on backtrack. When a node's stored word is found, record it and CLEAR the stored word so it is never reported again.

Python (intuition kernel)

```python
root = _TrieNode()
for w in words:
 _insert(root, w)
rows, cols = len(board), len(board[0])
found:  List[str] = []
 if r < 0 or r >= rows or c < 0 or c >= cols:
 return
 if board[r][c] == "#":

```

C++ (intuition kernel)

```cpp
if (!child->word.empty()) { // this path spells a full word
 found.push_back(child->word); // report it\{}ldots{}
 child->word.clear(); // \{}ldots{}and never report it again

```

Time O(m-n-4^L) worst case — L = max word length, **Space** O(26-n-L) trie + O(L) recursion depth.

every node is visited once and returns $O(1)$ state; hence the stated bounds.

Trie + grid DFS with branch pruning (BEST) Same walk, plus a refs counter on every node = number of words still reachable through it. When a node's last remaining word is found, refs drops to 0 and the whole dead branch is DELETED from the trie — future DFS never re-explores it. This removes the classic "found word still costs DFS time on every later cell" waste.

Python (intuition kernel)

```python
root = _TrieNode()
for w in words:
 _insert(root, w)
rows, cols = len(board), len(board[0])
found:  List[str] = []
 if r < 0 or r >= rows or c < 0 or c >= cols:
 return
 if board[r][c] == "#":

```

C++ (intuition kernel)

```cpp
if (!child->word.empty()) { // word completed here
 found.push_back(child->word);
 child->word.clear();
 --child->refs; // one fewer word in this branch

```

Time O(m-n-4^L) worst case, faster in practice. **Space** O(26-n-L) trie + O(L) recursion depth.

every node is visited once and returns $O(1)$ state; hence the stated bounds.

VISUAL STATE TRACE

```
grid:  oaan
etaen
iok
le words:  [oath, pea, eat, rain]
THE SHIFT: each trie node stores the LIST OF GRID CELLS
where that prefix occurs.  So one node does O(4) work per cell
rather than rescanning its whole word.
root:  all (0,0) (0,1) (0,2) (0,3)
(1,0)...  (2,3)
node 'o':  (0,0) (1,0) (2,1)
node 'oa':  (0,0)
node 'oat':  (0,0) -> neighbour 'h' at (1,0)?  no -> dead
node 'e':  (0,1) (1,1) (1,3) (2,2)
node 'ea':  (1,1)
node 'eat':  (1,1) -> 't' at (1,2)?  yes -> flag -> 'eat' FOUND
remove 'eat' from the trie (prune)
per node work:  4 neighbours * (rows*cols) cells = O(4RC)
marking is by mutating the CELL to '#' then restoring,
so a cell is never reused within one path.
```

LANGUAGE MECHANICS: PYTHON VS C++

A plain hash set of words cannot guide the walk: from cell (r,c) you would have to ask "which words start with the letters spelled so far?" — a PREFIX query, exactly what hash maps cannot answer (probing h(k,i) = (hash(k)+i) mod m finds only exact keys). The trie answers it in $O(1)$ per letter: one child lookup decides whether to descend or prune. Deleting used words (A2) turns the trie into a live accumulator — each word is discovered exactly once and its branch dies afterwards, so later cells stop paying for it. Memory math: n words of avg length L share prefix nodes; worst case n-L nodes × 26 pointers each.
==========
A plain hash set of words cannot guide the walk: from cell (r,c) you would have to ask "which words start with the letters I have spelled so far?" — a PREFIX query, exactly what hash maps cannot answer (probe h(k,i) = (hash(k)+i) mod m only finds exact keys). The trie answers it in $O(1)$ per letter: one child lookup decides whether to descend or prune. Deleting used words (A2) turns the trie into a live accumulator — each word is discovered exactly once and its branch dies afterwards, so later cells stop paying for it. Memory math: n words of avg length L share prefix nodes, worst case n-L nodes × 26 pointers.
==========
/ #include <iostream> //

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- Store `(row, col)` pairs per node in a `vector`; that is the algorithm's essential data, not an optimisation.
- Deleting found words from the trie is the main pruning win – without it every grid cell keeps searching for words already reported.
- Encode positions as `int` instead of a pair to halve the node payload.

- A list of ints per node with `for p in node.positions:` avoids tuple unpacking.
- Deleting from the trie: set the node's children to an empty list so the DFS stops descending there.

INTERVIEW TRAPS & SANITY CHECKS

Forgetting to unmark the grid cell after the recursive call. The cell stays as '#' and every other word that needs it fails, so the answer is missing valid words.

Sanity check. The list at trie node u holds exactly the grid cells where u's prefix ends, for the current search. DFS from u visits each such cell, marks it, and recurses into the child matching the next character. Marking makes the path self-avoiding; restoring after the loop restores the invariant for sibling branches.

INTUITION CHECK — 3 QUESTIONS

Q1. What does storing grid positions in the trie buy?

Answer. It inverts the loops. Instead of every cell trying every dictionary word ($O(RC * W)$), each trie node does $O(4)$ work per cell it occupies, so the total is proportional to the number of (prefix, cell) pairs that actually exist. Shared prefixes are explored once instead of once per word.

Q2. Why is removing found words necessary?

Answer. Without it, a cell visited late still descends the full remaining trie looking for words already reported, wasting the dominant share of the work. Removing a found word prunes that subtree for every later cell, and since the answer is a set, reporting twice is wrong anyway.

Q3. How does marking prevent revisiting a cell?

Answer. Temporarily writing '#' into the cell makes any check of its character fail for the remainder of that path. Because the recursion restores the original character on the way out, sibling branches still see the true letter – so marking gives path-local, not global, exclusion.

9. Heap / Priority Queue

#	Problem	Level	One-line intent
0215	**Kth Largest Element in an Array**	Medium	A size-k min-heap, or quickselect, both avoid a full sort.
0295	**Find Median from Data Stream**	Hard	Two heaps with a size invariant; the answer is the max-heap top.
0355	**Design Twitter**	Medium	Follow graph plus a timestamp-ordered feed per user.
0621	**Task Scheduler**	Medium	The formula (maxFreq-1)*(n-maxFreqCount) + maxFreq.
0703	**Kth Largest Element in a Stream**	Easy	Keep the top k in a min-heap; its root is the answer.
1046	**Last Stone Weight**	Easy	Pop the two heaviest stones; a tie annihilates both.

0215 `Kth Largest Element in an Array` Medium

ESSENCE & BOUNDS

Return the k-th largest element of an array, counting the largest as first. 1 <= k <= nums.length <= 2e4, so k is always valid. Values are ints, so `heapq.nlargest` and `sorted` both work, but the $O(nlogn)$ sort is avoidable.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

- Min-heap of size k (best) 1. Push each value; if len(heap) > k, heappop (removes the smallest — the value that just fell out of the top-k). 2. After the sweep, heap[0] is the k-th largest.

Python (intuition kernel)

```python
heap = []
for x in nums:
 heapq.heappush(heap, x)
 if len(heap) > k:
 heapq.heappop(heap) # eject the smallest of top-k
return heap[0]

```

C++ (intuition kernel)

```cpp
for (int x : nums) {
 heap.push(x); // candidate for top-k
 if (heap.size() > static_cast<size_t>(k)) heap.pop();
// drop smallest

```

Time ? **Space ?**

one push (and at most one pop) per element; hence the stated bounds.

- Quickselect (Hoare's partition) Partition around a pivot; if the pivot lands at index len-k it IS the answer; otherwise recurse into the correct half. $O(n)$ average. ———————————————————— DEEP DIVE - Why a MIN-heap, not a max-heap A max-heap lets you pop the largest repeatedly, but the heap "loses" the k-th largest once you pop it. The min-heap of size k tracks the boundary of the top-k: new elements smaller than the root are ignored implicitly by the pop. Python's heapq is a min-heap natively — zero configuration. ———————————————————— TIME COMPLEXITY : $O(nlogk)$ — n pushes, each $O(logk)$. MEMORY COMPLEXITY: $O(k)$ — the heap holds at most k values. ==========

Python (intuition kernel)

```python
pivot = nums[random.randint(lo, hi)]
left, mid, right = lo, lo, hi
while mid <= right: # 3-way partition
 if nums[mid] < pivot:
 nums[left], nums[mid] = nums[mid], nums[left]
 left += 1; mid += 1
 elif nums[mid] > pivot:
 nums[mid], nums[right] = nums[right], nums[mid]

```

C++ (intuition kernel)

```cpp
int lo = 0, hi = n - 1, target = n - k;
while (lo < hi) {
 int p = partition(nums, lo, hi);
 if (p == target) break;
 p < target ? lo = p + 1 : hi = p - 1;
}
return nums[target];

```

Time O(n log k) — n pushes, each O(log k).

Space O(k) — the heap holds at most k values.

Follows directly from the traversal count: $O(nlogk)$ — n pushes, each $O(logk)$. time, $O(k)$ — the heap holds at most k values. space.

VISUAL STATE TRACE

```
nums = [3,2,1,5,6,4], k = 2 -> 5
heap of size k (a MIN-heap keeps the k LARGEST):
push 3 -> [3]
push 2 -> [2,3]
push 1 -> [1,3,2]
push 5 -> [1,5,2,3] size 4 > k, pop MIN 1 -> [2,5,3]
push 6 -> [2,5,3,6] size 4 > k, pop MIN 2 -> [3,5,6]
push 4 -> [3,4,6,5] size 4 > k, pop MIN 3 -> [4,5,6]
heap top = 4 = SMALLEST of the 4 largest {4,5,6}?  there
are only 4 kept -> top 4, we want 2nd largest of 6
numbers:  largest=6, 2nd=5.
-> heapify(heap); return heap[0] -> need the MIN of the
k largest, i.e.  pop the min once then the top is the 2nd.
Correct form:  heapq.nlargest(k, nums)[-1]
quickselect:  partition around a pivot, recurse into the
side that contains index len-k, average O(n).
```

LANGUAGE MECHANICS: PYTHON VS C++

A max-heap lets you pop the largest repeatedly, but the heap "loses" the k-th largest once you pop it. The min-heap of size k tracks the boundary of the top-k: new elements smaller than the root are ignored implicitly by the pop. Python's heapq is a min-heap natively — zero configuration. ——— ——————————————————————— TIME COMPLEXITY : $O(nlogk)$ — n pushes, each $O(logk)$. ==========

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `std::nth_element(nums.begin(), nums.begin() + k - 1, nums.end(), greater<int>())` then `nums[k-1]` is the answer: $O(n)$ average, no extra memory.
- `std::partial_sort` is the deterministic cousin of nth_element: $O(n \log k)$, and a better worst case.
- A heap of size k is $O(n \log k)$ and $O(k)$ space – the right answer when memory is constrained.

- `heapq.nlargest(k, nums)[-1]` is the idiomatic one-liner and is C-implemented.
- `sorted(nums, reverse=True)[k-1]` is $O(n \log n)$ – correct, and worth naming as the baseline.
- For small k and small n, `max` in a loop is actually competitive because the interpreter overhead is amortised over few calls.

INTERVIEW TRAPS & SANITY CHECKS

Off-by-one on the index: the k-th largest is at sorted index k-1, not k. Also, using a max-heap of size k and returning its top gives the k-th SMALLEST, not largest – the heap must be a MIN-heap of the k largest.

Sanity check. After processing any prefix of the array, the heap contains the k largest elements of that prefix (or all of them if the prefix is shorter than k). Each push-then-pop-min preserves this, so at the end the heap's minimum is the k-th largest of the whole array.

INTUITION CHECK — 3 QUESTIONS

Q1. Why a min-heap of size k rather than a max-heap?

Answer. Because you want to keep the k largest, so you need to know which retained element is the weakest – the minimum. A max-heap's top would be the largest retained, which tells you nothing about whether a new element belongs in the set.

Q2. What does `nth_element` actually guarantee?

Answer. That the first k positions hold the k largest elements as an unordered set, in linear average time. It does not sort them, which is exactly what this problem allows since the answer is a single value.

Q3. How do the complexities compare?

Answer. Heap: $O(n \log k)$ time, $O(k)$ space. nth_element: $O(n)$ average but $O(n^2)$ worst case on adversarial pivots, with $O(1)$ space. Sorting: $O(n \log n)$ time, and C++'s in-place sort is $O(\log n)$ stack.

0295 Find Median from Data Stream Hard

ESSENCE & BOUNDS

Support adding an integer and retrieving the median of all values added so far in amortized $O(1)$ each. Up to 1e5 operations and values up to 1e5, so the median is an integer and needs no rounding. Odd and even counts alternate, so the balancing logic must handle both.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

- Two heaps (best) 1. addNum: push to the lower max-heap; rebalance by moving the boundary element if lower's top exceeds upper's top. 2. findMedian: equal sizes -> mean of both tops; else top of the taller heap.

Python (intuition kernel)

```python
self.lower = [] # max-heap via negation
self.upper = [] # min-heap
heapq.heappush(self.lower, -num)
if self.lower and self.upper and -self.lower[0] > self.upper[0]:
    heapq.heappush(self.upper, -heapq.heappop(self.lower))
if len(self.lower) > len(self.upper) + 1:
    heapq.heappush(self.upper, -heapq.heappop(self.lower))
if len(self.upper) > len(self.lower):

```

C++ (intuition kernel)

```cpp
if (upper_.size() > lower_.size()) { // rebalance heights
    lower_.push(upper_.top());
    upper_.pop();

```

Time ? **Space** ?

one push (and at most one pop) per element; hence the stated bounds.

- Bisect-insert into a sorted list (baseline) data.insert(bisect_left(data, v), v) is $O(n)$ per add due to the memmove — fine for small n, hopeless at scale. ——— DEEP DIVE - The balance invariant Invariants after every add: (a) len(lower) == len(upper) or len(lower) == len(upper) + 1; (b) max(lower) <= min(upper). Fixing (b): when lower[-1] (its max, via -lower[0]) > upper[0], swap the two tops. Fixing (a): if upper ever gets taller, migrate its minimum to the lower heap. Median always reads off the seam. ——— TIME COMPLEXITY : $O(logn)$ per addNum ; $O(1)$ findMedian. MEMORY COMPLEXITY: $O(n)$. ==========

Python (intuition kernel)

```python
self.data = []
self.data.insert(bisect.bisect_left(self.data, num), num)
n = len(self.data)
if n % 2:
 return self.data[n // 2]
return (self.data[n // 2 - 1] + self.data[n // 2]) / 2.0

```

C++ (intuition kernel)

```cpp
vector<double> s; // kept sorted
void addNum(int v) {
 s.insert(lower_bound(s.begin(), s.end(), v), v);
}
double findMedian() {
 int n = s.size();
 return n % 2 ?  s[n / 2]
 :  (s[n/2 - 1] + s[n/2]) / 2.0;
}

```

Time O(log n) per addNum ; O(1) findMedian. **Space** O(n).

the sort dominates; the post-sort scan is linear; hence the stated bounds.

VISUAL STATE TRACE

```
add 1, 2, 3, 4, 5

lower (MAX-heap) upper (MIN-heap) median
-- ------ -------- ----

1 [1] [] 1 (1 = 1)

2 [1] [2] 1 (even -> low)

3 [2,1] [3] 2 (3 = 2)

4 [2,1] [3,4] 2 (even -> low)

5 [3,1,2] [4,5] 3 (5 = 3)

INVARIANT: len(lower) == len(upper) or

len(lower) == len(upper) + 1

EVERY element of lower <= EVERY element of upper

median:

odd count -> lower[0] (the max-heap top)

even count -> (-lower[0] + upper[0]) / 2

python's heapq is MIN-only, so lower stores NEGATED

values and lower[0] is the true maximum.
```

LANGUAGE MECHANICS: PYTHON VS C++

Invariants after every add: (a) len(lower) == len(upper) or len(lower) == len(upper) + 1; (b) max(lower) <= min(upper). Fixing (b): when lower[-1] (its max, via -lower[0]) > upper[0], swap the two tops. Fixing (a): if upper ever gets taller, migrate its minimum to the lower heap. Median always reads off the seam. ——————————————————————————— TIME COMPLEXITY : $O(logn)$ per addNum ; $O(1)$ findMedian. ==========

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `std::priority_queue<int>` (max) and `std::priority_queue<int, vector<int>, greater<int>>` (min) are the natural pair – no negation needed.
- `__gnu_pbds::rb_tree` with two order-statistics iterators would also work but is heavier than heaps.
- heapq is min-only: push negated values into `lower` and read `-lower[0]` as the maximum.
- `heapq.heappush` / `heappushpop` are C-level; keeping the balance check outside the heap calls matters more than the choice of push variant.

INTERVIEW TRAPS & SANITY CHECKS

Getting the parity convention backwards – returning `upper[0]` for an odd count. The whole design hinges on `lower` always holding the extra element when the count is odd, so that the median is always accessible at `lower[0]`.

Sanity check. `len(lower) == len(upper)` or `len(lower) == len(upper) + 1`, and `max(lower) <= min(upper)`. Together these make the median a read of one or two heap tops, and they are preserved by inserting into `lower` then moving its top to `upper` when `lower` grows too large.

INTUITION CHECK — 3 QUESTIONS

Q1. Why does `lower` hold the extra element when the count is odd?

Answer. So the median is always `lower`'s maximum. If the smaller half holds one more element, its top is exactly the middle value of the sorted sequence – for both parities, with a two-term average only when even.

Q2. Why is inserting into `lower` then rebalancing correct?

Answer. Adding to `lower` can only violate the ordering invariant if `lower` was empty or `upper` was non-empty and the new value exceeds `upper`'s minimum. In both cases moving `lower`'s maximum to `upper` restores the ordering, and fixing the sizes then restores the count invariant.

Q3. What is the space complexity?

Answer. $O(n)$ – every value ever added is retained, split across the two heaps. That is unavoidable: the median depends on the whole multiset, so nothing can be discarded.

0355 Design Twitter Medium

ESSENCE & BOUNDS

Design a Twitter-like API supporting follow, unfollow, post, and a user's recent posts including those of followed users. At most 1e5 operations and 5e9 user IDs, so IDs must be long and follow sets must be hash-based. The feed is capped at 10 tweets, which is the key constraint that shapes the data structures.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

- Merge-k with a max-heap (best) Push the newest tweet of every followed user; pop the max; then push that user's next-newest; stop at 10. Heap stays $\leq$ #follows.

Python (intuition kernel)

```python
self.tweets = collections.defaultdict(list) # user -> [(clock, id)]
self.follows = collections.defaultdict(set)
self.clock = 0
self.tweets[userId].append((self.clock, tweetId))
self.clock += 1
if followerId != followeeId:
 self.follows[followerId].add(followeeId)
self.follows[followerId].discard(followeeId)

```

C++ (intuition kernel)

```cpp
auto pushUser = [&](int author) { auto it = tweets_.find(author);
 if (it != tweets_.end() && !it->second.empty()) { size_t last = it->second.size() - 1;
 heap.emplace(it->second[last].time, it->second[last].id, author,
 static_cast<int>(last));
 }

```

Time ? **Space** ?

one push (and at most one pop) per element; hence the stated bounds.

- Gather-all + sort (baseline) Collect every tweet of every followed user, sort by time desc. ———————————————— DEEP DIVE - k-way merge vs flat sort Flat: O(F-K log(F-K)) with F follows and K tweets each. Merge-k: $O(FlogF)$ per popped tweet — the classic external-sort pattern. Ties: we never pop more than one tweet of a user before its successor enters the heap, so ordering stays strictly chronological. The clock counts up per post, so "most recent" == "largest clock". ———————————————————— TIME COMPLEXITY : getNewsFeed $O(FlogF)$; post/follow $O(1)$. MEMORY COMPLEXITY: $O(T)$ for T tweets + follow edges. ==========

Python (intuition kernel)

```python
self.all = collections.defaultdict(list)
self.follows = collections.defaultdict(set)
self.clock = 0
self.all[userId].append((self.clock, tweetId))
self.clock += 1
if followerId != followeeId:
 self.follows[followerId].add(followeeId)
self.follows[followerId].discard(followeeId)

```

C++ (intuition kernel)

```cpp
unordered_map<int, vector<int>> posts;
unordered_map<int, unordered_set<int>> follow;
int clock = 0;

vector<int> getRecent(int u, int k = 10) {
 vector<int> ids = posts[u];
 for (int f :  follow[u])
 for (int id :  posts[f]) ids.push_back(id);
 sort(ids.begin(), ids.end());
 if (ids.size() <= k) return ids;
 return vector<int>(ids.end() - k, ids.end());
}

```

Time getNewsFeed O(F log F); post/follow O(1).

Space O(T) for T tweets + follow edges.

the sort dominates; the post-sort scan is linear; hence the stated bounds.

VISUAL STATE TRACE

```
post(1, [2,3])
followers:  { 1:  {2,3} }
tweets[tweetId] = { id:1, uid:1, ts:...  }
userTweets[1] = heap of tweet ids by ts
getRecent(3):
candidates = {2,3} U self(3)
for each u in candidates:
merge the top of userTweets[u] into the result heap
pop the largest ts until 10 collected or all heaps empty
k = 10 CAP: you never need the whole timeline, so
per-user heaps are only drained lazily, at most 10 times.
That is what makes O(k log k)-ish work per query instead of
O(total tweets).
follow(a,b):  followers[a].add(b)
unfollow(a,b):  followers[a].discard(b) (a==b is a no-op)
post:  push to that user's heap, incrementing the global clock
```

LANGUAGE MECHANICS: PYTHON VS C++

Flat: O(F-K log(F-K)) with F follows and K tweets each. Merge-k: $O(FlogF)$ per popped tweet — the classic external-sort pattern. Ties: we never pop more than one tweet of a user before its successor enters the heap, so ordering stays strictly chronological. The clock counts up per post, so "most recent" == "largest clock".

———————————————————— TIME COMPLEXITY : getNewsFeed $O(FlogF)$; post/follow $O(1)$.

==========

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `unordered_map<int, unordered_set<int>>` for the follow graph; a set is required because unfollow must be $O(1)$ and duplicates must be idempotent.
- `multiset<pair<ts, tweetId>>` per user gives ordered access with erase-by-key, which a heap cannot do.
- `map<int, ...>` instead of unordered only if you need ordered user iteration, which the API does not require.

- `defaultdict(set)` for the follow graph; `set` gives $O(1)$ follow/unfollow and automatic dedupe.
- `OrderedDict` per user keyed by timestamp keeps insertion order without a heap, at $O(1)$ `move_to_end` and popitem.
- `heapq.merge(*[userTweets[u] for u in candidates])` is a lazy k-way merge – the cleanest expression of the query.

INTERVIEW TRAPS & SANITY CHECKS

Returning the 10 most recent tweets of ALL followed users rather than per-user merges, and ignoring the self-inclusion rule. A user always sees their own tweets, and duplicates must not appear if they follow someone who also follows them.

Sanity check. Each user's tweet container is ordered newest-first, so the newest tweet overall among a candidate set is the maximum of the per-user heads. Merging those heads and popping 10 times therefore yields exactly the 10 newest, and the merge touches at most 10 nodes per container.

INTUITION CHECK — 3 QUESTIONS

Q1. Why does the k=10 cap make this tractable?

Answer. Without it you would need to sort every tweet by every followed user – potentially unbounded work. With the cap, a k-way merge over per-user sorted streams needs at most k pops, each $O(\log |F|)$ where F is the follow count. The cap is what turns an unbounded query into a bounded one.

Q2. What structure per user, and why?

Answer. A heap, multiset, or ordered dict keyed by timestamp, all of which support newest-first access in $O(1)$. A plain list would need insertion in the middle ($O(n)$) unless you keep it sorted, which is worse. The requirement is ordered access to the newest element, not arbitrary access.

Q3. How do you make unfollow $O(1)$?

Answer. Store each user's following set as a hash set, not a list. Deletion from a set is expected $O(1)$, and re-following is idempotent. A list would need a scan, which is why the set is load-bearing rather than incidental.

0621 Task Scheduler Medium

ESSENCE & BOUNDS

Return the minimum number of time units to run all tasks given a non-negative cooldown between identical tasks. 1 <= tasks.length <= 1e4 and 1 <= n <= tasks.length, so the answer fits comfortably in int. Cooldown n can exceed the alphabet size, in which case the answer is just the task count.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

- Math formula (best) m = max frequency, c = count of tasks with frequency m. answer = max((m - 1) * (n + 1) + c, len(tasks)).

Python (intuition kernel)

```python
cnt = collections.Counter(tasks)
m = max(cnt.values())
c = sum(1 for v in cnt.values() if v == m)
return max((m - 1) * (n + 1) + c, len(tasks))

```

C++ (intuition kernel)

```cpp
while (!heap.empty() || !cooldown.empty()) { ++time;
 if (!heap.empty()) {
 int c = heap.top() - 1; heap.pop(); // run one instance
 if (c > 0) cooldown.push({c, time + n});
 }
 if (!cooldown.empty() && cooldown.front().second == time) {
 heap.push(cooldown.front().first); // cooled doum $\{}to$ ready
 cooldown.pop();

```

Time ? **Space** ?

constant time after one counting pass; hence the stated bounds.

- Heap simulation Max-heap of remaining counts + FIFO cooldown queue; every second run the most frequent available task, park it for n seconds. ——— DEEP DIVE - Why the formula is exact The busiest task T appears m times; between consecutive copies we need n other/ idle slots — that core costs (m-1)(n+1) units, and the LAST row fits all c tasks that tie with T. Filling the core gaps with the remaining tasks is always possible greedily, so the lower bound is tight. If tasks alone already fill every slot (len(tasks) larger), no idle is needed. $O(1)$ beats $O(26log26)$. ———

———————— TIME COMPLEXITY : $O(1)$ — one pass over 26 counters. MEMORY COMPLEXITY: $O(1)$. ==========

Python (intuition kernel)

```python
cnt = collections.Counter(tasks)
heap = [-v for v in cnt.values()]
heapq.heapify(heap)
cooldown = collections.deque() # (remaining, readyAt)
t = 0
while heap or cooldown:
 t += 1
 if heap:

```

C++ (intuition kernel)

```cpp
while (!heap.empty() || !cooldown.empty()) { ++time;
 if (!heap.empty()) {
 int c = heap.top() - 1; heap.pop(); // run one instance
 if (c > 0) cooldown.push({c, time + n});
 }
 if (!cooldown.empty() && cooldown.front().second == time) {
 heap.push(cooldown.front().first); // cooled down $\{}to$ ready
 cooldown.pop();

```

Time O(1) — one pass over 26 counters.

Space O(1).

one push (and at most one pop) per element; hence the stated bounds.

VISUAL STATE TRACE

```
tasks = [A,A,A,B,B,C,C,C], n = 2 -> answer 7
frequencies:  A:3 B:2 C:3
maxFreq = 3, maxFreqCount = 2 (A and C)
formula = (3-1) * (7-2) + 3 = 10 + 3 = 13 ?  NO
correct:  (maxFreq-1)*(n - maxFreqCount) + maxFreq
= (3-1)*(7-2) + 3 = 13 ...  but expected 7?
Recompute:  n = 2 cooldown.
A . .  A . .  A frames of length 3, 2 frames
B . .  B
C . .  C
total = 2*3 + 1 = 7
formula with maxFreq=3, maxFreqCount=2, len=7:
(3-1)*(7-2) + 3 = 13 is wrong; the correct form is
(maxFreq-1)*(n - maxFreqCount) + maxFreqCount
(3-1)*(7-2) + 2 = 12 also wrong for this example because
the OTHER tasks fill the gaps.
The derivation:
maxFreq-1 frames are needed, each frame has length n.
The most frequent tasks occupy maxFreqCount slots per frame.
Everything else (len - maxFreq*maxFreqCount tasks) fills the
remaining slack.  Result =
max( len, (maxFreq-1)*n + maxFreqCount )
```

LANGUAGE MECHANICS: PYTHON VS C++

The busiest task T appears m times; between consecutive copies we need n other/ idle slots — that core costs (m-1)(n+1) units, and the LAST row fits all c tasks that tie with T. Filling the core gaps with the remaining tasks is always possible greedily, so the lower bound is tight. If tasks alone already fill every slot (len(tasks) larger), no idle is needed. $O(1)$ beats $O(26log26)$. ———————— TIME COMPLEXITY : $O(1)$ — one pass over 26 counters. ==========

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- A 26-element frequency array plus two running maxima is $O(T + 26)$ with no map.
- The heap simulation is $O(T \log 26)$ and is more obviously correct, which is why it is the fallback.
- `Counter(tasks).values()` plus `max/count` gives the two quantities in one pass.
- The heap simulation with `Counter` and `list.index` on a 26-slot cooldown array is readable and fast enough.

INTERVIEW TRAPS & SANITY CHECKS

Using the widely-copied formula `(maxFreq-1)*(n-maxFreqCount) + maxFreq` without checking it against a worked example. It is not correct in general; the frame-based derivation gives `max(len, (maxFreq-1)*n + maxFreqCount)`.

Sanity check. No schedule can be shorter than `len` tasks, and the cooldown forces at least `maxFreq - 1` separate frames of at least `n` units with `maxFreqCount` tasks pinned per frame. The answer is the max of those two lower bounds, and both are achievable.

INTUITION CHECK — 3 QUESTIONS

Q1. Why do the most frequent tasks define the frame count?

Answer. A task occurring f times needs f-1 cooldowns between them, so it spans at least f slots. That forces at least maxFreq-1 frame boundaries, each of which must be at least n long or the cooldown would be violated.

Q2. When is the answer simply `len`?

Answer. When `(maxFreq-1)*n + maxFreqCount <= len`, i.e. the other tasks fill every frame completely. Then no idle slot is needed and the schedule runs back to back.

Q3. Why is a heap simulation not necessary?

Answer. Because the frame structure gives a closed-form lower bound that is achievable. The simulation is $O(T \log 26)$ and useful as a correctness check or when the cooldown structure is more complex, but it is strictly more work for no extra information.

0703 `Kth Largest Element in a Stream` Easy

ESSENCE & BOUNDS

Return the k-th largest value seen so far in a stream, for every add. 1 <= k <= 500 <= n <= 1e4, so the heap never exceeds 500 and every operation is comfortably bounded. Values are non-negative ints.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

- Min-heap of size k (best) Constructor: heapify nums, trim to size k by popping the root. add(v): heappush, pop if oversized, return heap[0].

Python (intuition kernel)

```python
self.k = k
self.heap = nums[:]
heapq.heapify(self.heap)
while len(self.heap) > k:
    heapq.heappop(self.heap)
def add(self, val):
    heapq.heappush(self.heap, val)
    if len(self.heap) > self.k:
        heapq.heappop(self.heap)
    return self.heap[0]
```

C++ (intuition kernel)

```cpp
for (const auto& c : cases) { int r1 = s1.add(c.val);
    int r2 = s2.add(c.val);
    bool ok = r1 == c.expect && r2 == c.expect;
    allOk = allOk && ok;
    std::cout << "add(" << c.val << ") $\{}to$ A1=" << r1 << " A2=" << r2
    << " (want " << c.expect << ") " << (ok ? "PASS" : "FAIL") << "\{}n";
}
```

Time ? **Space** ?

one push (and at most one pop) per element; hence the stated bounds.

- Naive sorted list (baseline) Append and re-sort descending on every add; answer is index k-1. —— DEEP DIVE - Why we can reject values forever Invariant: heap holds the k largest values of the stream. A new value $\leq$ heap[0] is $\leq$ every element in the top-k, so no matter how many future values arrive, this one can never rise into the top-k — discarding it

loses nothing. Values > heap[0] replace the root ($O(logk)$) and keep the invariant. This "small values are dead on arrival" insight is the whole trick of streaming top-k. ———————————————— TIME COMPLEXITY : $O(logk)$ per add ; $O(k)$ construction. MEMORY COMPLEXITY: $O(k)$. ==========

Python (intuition kernel)

```python
self.k = k
self.data = sorted(nums, reverse=True)[:k]
self.data.append(val)
self.data.sort(reverse=True)
self.data = self.data[:self.k]
return self.data[self.k - 1]

```

C++ (intuition kernel)

```cpp
for (const auto& c :  cases) { int r1 =
s1.add(c.val);
 int r2 = s2.add(c.val);
 bool ok = r1 == c.expect && r2 == c.expect;
 allOk = allOk && ok;
 std::cout « "add(" « c.val « ") $\{}to$ A1=" « r1 «
" A2=" « r2
 « " (want " « c.expect « ") " « (ok ?  "PASS" :
"FAIL") « "\{}n";

```

Time O(log k) per add ; O(k) construction.

Space O(k).

the sort dominates; the post-sort scan is linear; hence the stated bounds.

VISUAL STATE TRACE

```
k = 3, adds:  4, 5, 8, 2, 3

heap (size <= 3, MIN-heap of the k largest):

add 4 -> {4} kth = 4

add 5 -> {4,5} kth = 4

add 8 -> {4,5,8} kth = 4

add 2 -> {2,4,5,8} pop 2 -> {4,5,8} kth = 4

add 3 -> {3,4,5,8} pop 3 -> {4,5,8} kth = 4

heap root = MIN of the retained k = the k-th largest overall.

invariant:  |heap| == min(k, n) and heap contains the k largest

values seen so far.  Each add is one push plus at most one pop,

so amortized O(log k).

contrast with 215 (static array):  there you may sort or

quickselect; here the input is a stream, so nothing but the

retained top k may be stored.
```

LANGUAGE MECHANICS: PYTHON VS C++

Invariant: heap holds the k largest values of the stream. A new value ≤ heap[0] is ≤ every element in the top-k, so no matter how many future values arrive, this one can never rise into the top-k — discarding it loses nothing. Values > heap[0] replace the root ($O(logk)$) and keep the invariant. This "small values are dead on arrival" insight is the whole trick of streaming top-k. ———————————————— TIME COMPLEXITY : $O(logk)$ per add ; $O(k)$ construction. ==========

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `priority_queue<int, vector<int>, greater<int>>` is a min-heap by default with the comparator – exactly the structure needed.
- A fixed `vector` of size k with manual sift is faster but far more code.
- `heapq.heappushpop(heap, x)` fuses push-and-pop into one C call and is faster than a separate push then conditional pop.
- When the heap is already full, `heappushpop` returns the smallest; if x is larger than the root it is inserted and the old minimum is discarded in a single operation.

INTERVIEW TRAPS & SANITY CHECKS

Using a MAX-heap of size k, whose top is the largest of the retained set – that gives you the largest, not the k-th largest. Or returning `heap[0]` before the heap has reached size k.

Sanity check. After each add, the heap holds exactly the k largest values seen so far (or all values if fewer than k have arrived), so its minimum is the k-th largest. `heappushpop` maintains this in one call because it only evicts when the new value exceeds the current minimum.

INTUITION CHECK — 3 QUESTIONS

Q1. Why can you discard the smaller elements permanently?

Answer. Because at most k values ever matter for the answer, and only ever more values arrive. Once a value is not among the k largest, no future addition can promote it – adding new elements only pushes the k-th largest upward. So discarded values are provably irrelevant.

Q2. What if the heap is not yet full?

Answer. Then every value so far is a candidate, and the answer is simply the heap's minimum. `heappushpop` handles this automatically: while `len(heap) < k` it behaves as a plain push.

Q3. How does this differ from problem 215?

Answer. 215 has the whole array available, so sorting ($O(n \log n)$) or `nth_element`/quickselect ($O(n)$) is possible. Here the input is a stream, so only the retained top k may be stored, which forces the $O(n \log k)$ heap.

1046 Last Stone Weight Easy

ESSENCE & BOUNDS

Return the smallest possible weight of the remaining stone, or 0 if none remain. 1 <= stones.length <= 30 with weights 1..100, so at most 30 pops of a pair and an answer of at most 100. Weights are small enough that a counting bucket works too.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

- Max-heap simulation (best) Negate every weight (heapq = min-heap). While ≥ 2 stones: pop two, smash, push back the difference as a negative if non-zero.

Python (intuition kernel)

```python
heap = [-w for w in stones]
heapq.heapify(heap)
while len(heap) > 1:
    a = -heapq.heappop(heap) # heaviest
    b = -heapq.heappop(heap) # second heaviest
    if a != b:
        heapq.heappush(heap, -(a - b))
return -heap[0] if heap else 0

```

C++ (intuition kernel)

```cpp
while (heap.size() > 1) { int a = heap.top();
heap.pop(); // heaviest
    int b = heap.top(); heap.pop(); // second heaviest
    if (a != b) heap.push(a - b); // survivor of the
smash

```

Time ? **Space ?**

one push (and at most one pop) per element; hence the stated bounds.

- Repeated sort (naive baseline) Sort descending each round, combine the two front elements. ———— DEEP DIVE - Negative-weight idiom in Python heapq exposes only min-heaps, but a max-heap is one negation away: store -w and every pop() returns -largest. Arithmetic on the negatives mirrors the positives: a - b == -((-a) - (-b))... careful: we pop -a and -b (largest weights a ≥ b), remainder a-b becomes -(a-b) = (-a) - (-b) ... simplest: store w, pop gives -a, -b; push -(abs diff) when diff > 0. We implement that directly. ———— TIME COMPLEXITY : $O(nlogn)$ — n pushes + n/2 smash rounds. MEMORY COMPLEXITY: $O(n)$. ==========

Python (intuition kernel)

```python
stones = stones[:]
while len(stones) > 1:
 stones.sort()
 a, b = stones.pop(), stones.pop()
 if a != b:
 stones.append(a - b)
return stones[0] if stones else 0

```

C++ (intuition kernel)

```cpp
while (stones.size() > 1) {
 std::sort(stones.begin(), stones.end(), std::greater<int>());
 int a = stones[0], b = stones[1];
 stones.erase(stones.begin(), stones.begin() + 2);
 if (a != b) stones.push_back(a - b);

```

Time O(n log n) — n pushes + n/2 smash rounds. **Space** O(n).

the sort dominates; the post-sort scan is linear; hence the stated bounds.

VISUAL STATE TRACE

```
stones = [2,7,4,1,8,1] -> answer 1
max-heap (negated in Python):
[8,7,4,2,1,1]
pop 8, pop 7 -> smash:  8-7 = 1 -> heap {4,2,1,1,1}
pop 4, pop 2 -> smash:  4-2 = 2 -> heap {2,1,1,1}
pop 2, pop 1 -> smash:  2-1 = 1 -> heap {1,1,1}
pop 1, pop 1 -> smash:  0 -> heap {1}
pop 1 alone -> heap empty
ANSWER 1
the SINGLE-STONE case matters:  at the end the heap may hold
one element with no partner, and that value is the answer.
invariant:  the heap always contains the current multiset of
surviving stone weights, so popping two maxima and reinserting
the difference is exactly one smash.
```

LANGUAGE MECHANICS: PYTHON VS C++

heapq exposes only min-heaps, but a max-heap is one negation away: store -w and every pop() returns -largest. Arithmetic on the negatives mirrors the positives: a - b == -((-a) - (-b))... careful: we pop -a and -b (largest weights a $\geq$ b), remainder a-b becomes -(a-b) = (-a) - (-b) ... simplest: store w, pop gives -a, -b; push -(abs diff) when diff $>$ 0. We implement that directly. ———————— TIME COMPLEXITY : $O(nlogn)$ — n pushes + n/2 smash rounds. ==========

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `priority_queue<int>` is a max-heap with no negation needed.
- With weights <= 100 a `vector<int> count(101)` descending sweep gives $O(W + n)$ with no heap at all.

- heapq is min-only: store negated weights and read `-heapq.heappop(h)`.
- The counting approach is elegant here: `for w in range(100, 0, -1)` and consume the count in pairs.

INTERVIEW TRAPS & SANITY CHECKS

Continuing to pop twice when only one stone remains. The `while len(h) > 1` guard handles this; the bug is a `while h:` loop that pops twice unconditionally.

Sanity check. The heap holds exactly the weights of the stones not yet smashed. Each iteration removes the two heaviest and inserts their difference, which is precisely the problem's operation, so the invariant is preserved and the final top is the minimum surviving weight.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is the answer the heap's top rather than the sum?

Answer. Because smashing two stones a >= b replaces them with a - b, and the total weight decreases by exactly 2b. Greedily smashing the two heaviest is optimal, and when the heap empties the answer is 0; when one stone remains its weight is the answer, not a sum.

Q2. What changes when weights exceed 100?

Answer. The counting bucket becomes infeasible (the array would be too large) and you must fall back on the heap, which is $O(n \log n)$ and works for any magnitude. The bucket is purely an optimisation enabled by the weight bound.

Q3. Is greedy optimal here?

Answer. Yes, and the exchange argument is short: if an optimal schedule does not smash the two heaviest together first, you can swap that pairing in without increasing the final weight, since the heavier stone can only ever reduce itself by pairing with something no heavier.

10. Backtracking

#	Problem	Level	One-line intent
0017	**Letter Combinations of a Phone Number**	Medium	Index i into the digit string; each level picks a letter.
0039	**Combination Sum**	Medium	Index-based recursion, reuse the current candidate to allow repeats.
0040	**Combination Sum II**	Medium	Same index, but start at i+1 and skip duplicate values.
0046	**Permutations**	Medium	One position at a time, choosing any unused value.
0051	**N Queens**	?	One column per row; three conflict sets prune instantly.
0078	**Subsets**	Medium	Every node of the tree is a subset, so emit as you go.
0079	**Word Search**	Medium	Mark in place, search the four neighbours, unmark.
0090	**Subsets II**	Medium	Sort, then skip equal values at the same recursion level.
0131	**Palindrome Partitioning**	Medium	Precompute the palindrome table, then DFS over cuts.
0395	**Longest Substring with At Least K Repeating Chars**	Medium	Split on a bad character, or slide a window by distinct count.

0017 Letter Combinations of a Phone Number Medium

ESSENCE & BOUNDS

Return all letter combinations that the digit string represents, mapping each digit to its 2-4 letters. 0 <= digits.length, digits[i] != '1' (no letters for 1), so an empty digits string yields one empty combination. 2^n output size, so the output dominates the runtime.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

- Backtracking (best) dfs(d): d == len(digits) → record; else loop the digit's letters, append, recurse(d+1), pop.

Python (intuition kernel)

```python
if not digits:
out, cur = [], []
 if d == len(digits):
 out.append("".join(cur))
 return
 for ch in KEYPAD[digits[d]]:
 cur.append(ch)
 dfs(d + 1)

```

C++ (intuition kernel)

```cpp
for (char ch :  map_[(*d_)[pos] - '2']) {
cur_.push_back(ch);
 dfs(pos + 1);
 cur_.pop_back(); // backtrack the letter

```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

- Iterative product Start [""]; per digit extend every prefix by every letter. ——————————————— DEEP DIVE - Why empty input must return [] [""] seed would yield [""] for digits="" — an invalid combination (digits produced no letters). Both idioms special-case it. Answer size = letters(digit); "999" → 4^3 = 64. ——————————————————————— —— TIME COMPLEXITY : $O(4^n)$ worst case (all 9s). MEMORY COMPLEXITY: $O(n)$ DFS depth. ==========

Python (intuition kernel)

```python
if not digits:
out = [""]
for d in digits:
 out = [p + ch for p in out for ch in KEYPAD[d]]
return out

```

C++ (intuition kernel)

```cpp
 const std::array<std::string, 8> map = {"abc", "def",
"ghi", "jkl",

```

Time O(4^n) worst case (all 9s). **Space** O(n) DFS depth.

Follows directly from the traversal count: $O(4^n)$ worst case (all 9s). time, $O(n)$ DFS depth. space.

VISUAL STATE TRACE

```
digits = "23"
2 -> a b c 3 -> d e f
i=0 path="
pick 'a' -> i=1 pick 'd' -> 'ad' <- leaf
pick 'e' -> 'ae'
pick 'f' -> 'af'
pick 'b' -> 'bd' 'be' 'bf'
pick 'c' -> 'cd' 'ce' 'cf'
invariant:  path is always a prefix of exactly one output, and
len(path) == i.  The tree has 3*3 = 9 leaves for "23".
key = digits[i] - '0'; letters = KEYS[key]
for ch in letters:  dfs(i+1, path + ch)
```

LANGUAGE MECHANICS: PYTHON VS C++

[""] seed would yield [""] for digits="" — an invalid combination (digits produced no letters). Both idioms special-case it. Answer size = letters(digit); "999" → 4^3 = 64. ——————————————————— TIME COMPLEXITY : $O(4^n)$ worst case (all 9s). ==========

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `static const string KEYS[10]` indexed by `digits[i] - '0'` beats a map.
- `string path` with `push_back`/`pop_back` avoids the $O(n)$ copy that `path + ch` forces at every node.
- Mutate a single list with append/pop instead of concatenating strings – avoids an $O(L)$ copy per node.
- A dict keyed by digit character is clearer than a list indexed by digit minus zero.

INTERVIEW TRAPS & SANITY CHECKS

Indexing the keys table with the digit's character value rather than the digit minus '0'. That reads out of bounds and returns the wrong key for every multi-digit input.

Sanity check. `dfs(i, path)` enumerates all completions of the suffix starting at digit index i. The recursion is exhaustive because every letter of digit i is tried, and it emits no duplicates because each output corresponds to exactly one choice sequence.

INTUITION CHECK — 3 QUESTIONS

Q1. What is the size of the output space?

Answer. The product of the letter counts, between 2^n and 4^n. For n = 8 that is up to 65536 combinations, so any solution is output-bound: the algorithm is Θ(output size) and cannot beat enumeration.

Q2. Why not a greedy per-position choice?

Answer. Every letter at every position is valid independently – there is no constraint linking positions. So the solution set is the cartesian product, which is exactly what recursion over independent choices generates.

Q3. How does string concatenation affect the complexity?

Answer. `path + ch` copies the whole prefix each time, making the traversal $\Theta(n \cdot \text{output})$ in the worst case. Mutating one buffer with push/pop brings it down to Θ(output), which is optimal.

0039 Combination Sum Medium

ESSENCE & BOUNDS

Return all combinations that sum to target, where each number may be used unlimited times. 1 <= candidates.length <= 30 and 1 <= target <= 30, so the output is small but recursion depth is bounded by target (each element adds at least 1). Zero targets would loop forever.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

- Backtracking with repeat (best) dfs(i, rem): rem == 0 → record. For j in i..n: skip c[j] > rem (prune), take, recurse(j) — same j = unlimited use — then pop.

Python (intuition kernel)

```python
out, cur = [], []
if rem == 0:
out.append(cur[:])
return
for j in range(i, len(candidates)):
if candidates[j] > rem:
continue
cur.append(candidates[j])

```

C++ (intuition kernel)

```cpp
for (size_t i = 0; i < candidates.size(); ++i) { int
c = candidates[i];
for (int t = c; t <= target; ++t) { for (auto& combo
: dp[t - c]) { auto next = combo;
next.push_back(c);
dp[t].push_back(next);
}
}

```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

- DP table of combinations dp[t] lists combos for sum t; extend dp[t-c] with c for t = c..T. ——— DEEP DIVE - The one-character difference from Subsets II recurse(j+1) forbids reuse; recurse(j) allows it. The j from the LOOP start also forbids order permutations (no (2,3) after (3,2)) because j only moves forward. Pruning c[j] > rem cuts most of the tree when the target is small relative to candidates. ——— TIME COMPLEXITY : O(n^(target/min)) worst case. MEMORY COMPLEXITY: $O(target/min)$. ==========

Python (intuition kernel)

```python
dp = [[] for _ in range(target + 1)]
dp[0] = [[]]
for c in candidates:
for t in range(c, target + 1):
for combo in dp[t - c]:
dp[t].append(combo + [c])
return dp[target]

```

C++ (intuition kernel)

```cpp
for (size_t i = 0; i < candidates.size(); ++i) { int
c = candidates[i];
for (int t = c; t <= target; ++t) { for (auto& combo
: dp[t - c]) { auto next = combo;
next.push_back(c);
dp[t].push_back(next);
}
}

```

Time O(n^(target/min)) worst case.	**Space** O(target/min).

Follows directly from the traversal count: O(n^(target/min)) worst case. time, $O(target/min)$. space.

VISUAL STATE TRACE

```
candidates = [2,3,6,7], target = 7
dfs(i=0, rem=7):
candidates[0]=2
dfs(0, 5):  candidates[0]=2 <- SAME index, reuse allowed
dfs(0, 3):
dfs(0, 1):  1 % 2 != 0, dead
candidates[1]=3 -> 3==3 -> HIT [2,2,3]
candidates[1]=3 -> 3 < 5
dfs(1, 2):
candidates[1]=3 > 2 -> dead
candidates[2]=6 > 5 candidates[3]=7 > 5 dead
candidates[1]=3
dfs(1, 4):  candidates[1]=3 ...  3+?  no
candidates[2]=6
dfs(2, 1):  6 > 1 dead
candidates[3]=7
dfs(3, 0):  7 == 7 -> HIT [7]
ANSWER [[2,2,3], [7]]
two index rules that make it work:
recurse with i -> element reusable (problem 39)
recurse with i + 1 -> element single-use (problem 40)
```

LANGUAGE MECHANICS: PYTHON VS C++

recurse(j+1) forbids reuse; recurse(j) allows it. The j from the LOOP start also forbids order permutations (no (2,3) after (3,2)) because j only moves forward. Pruning c[j] > rem cuts most of the tree when the target is small relative to candidates. ——————————————————————— TIME COMPLEXITY : O(n^(target/min)) worst case. ==========

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- Sort candidates and prune with `if (c[i] > rem) break` – a large practical speedup.
- Pass `vector<int>& cur` by reference and push/pop, avoiding a copy per leaf.

- Sorting the list first and breaking out of the loop early prunes an enormous amount of the tree.
- `cur + [c]` at the leaf copies once per solution, which is acceptable since it happens only at output points.

INTERVIEW TRAPS & SANITY CHECKS

Recursing with `i + 1` instead of `i`, which silently forbids reuse and returns only [[7]] for the example above. The distinction between problems 39 and 40 is exactly one character.

Sanity check. `dfs(i, rem)` enumerates all combinations summing to `rem` drawn from candidates[i:]. Recursing with the same `i` permits unbounded reuse of candidates[i], and recursing with `i+1` guarantees combinations are generated in non-decreasing candidate order, so no solution is produced twice.

INTUITION CHECK — 3 QUESTIONS

Q1. Why does starting at the same index prevent duplicates?

Answer. Because a combination like [2,3] can only be built by taking 2 then 3. If you always move forward or stay, you never build [3,2], and the monotonic construction is unique per multiset. Sorting makes this even more obvious by letting you break when a candidate exceeds the remainder.

Q2. What bounds the recursion depth?

Answer. The target, since each element is at least 1 – depth is at most `target`. With candidates containing 0 the recursion would not terminate, so a zero candidate requires an explicit guard.

Q3. How do you prune aggressively?

Answer. Sort ascending, then at each level break as soon as candidates[i] > rem. Any later candidate is larger, so the entire remaining subtree at that level is infeasible. This typically removes most of the tree.

0040 Combination Sum II Medium

ESSENCE & BOUNDS

Return all unique combinations that sum to target, using each input element at most once. 1 <= candidates.length <= 30, all values 1..30, distinct, target 1..40. Duplicates in the input must produce duplicate combinations, so input-level dedupe is mandatory.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

- Sorted DFS (best) for j in i..n: if j > i and c[j]==c[j-1]: continue; if c[j] > rem: break; take, recurse(j+1), pop.

Python (intuition kernel)

```python
candidates.sort()
out, cur = [], []
if rem == 0:
out.append(cur[:])
return
for j in range(i, len(candidates)):
if candidates[j] > rem:
break

```

C++ (intuition kernel)

```cpp
for (size_t j = i; j < a_->size(); ++j) {
if ((*a_)[j] > rem) break; // sorted $\{}to$ prune hard
if (j > i && (*a_)[j] == (*a_)[j - 1]) continue; // skip dup
cur_.push_back((*a_)[j]);
dfs(j + 1, rem - (*a_)[j]); // j+1:  use each once
cur_.pop_back();

```

Time ? **Space** ?

the sort dominates; the post-sort scan is linear; hence the stated bounds.

- Frequency-map branching Group values with counts; branch per distinct value 0..cnt copies. ———————————————— DEEP DIVE - break vs continue after sorting In Combination Sum I (unlimited use, unsorted) we `continue` because a big candidate doesn't rule out smaller ones later. Here sorting guarantees c[j] > rem ⇒ every later candidate is also too big, so `break` prunes harder with identical semantics. ———————————————— TIME COMPLEXITY : $O(2^n)$ worst case. MEMORY COMPLEXITY: $O(n)$. ==========

Python (intuition kernel)

```python
from collections import Counter
freq = list(Counter(candidates).items())
out, cur = [], []
if rem == 0:
out.append(cur[:])
return
if idx == len(freq):
return

```

C++ (intuition kernel)

```cpp
for (int take = i; take < static_cast<int>(a_->size()); ++take) {
if (take > i && (*a_)[take] == (*a_)[take - 1]) continue;
cur_.push_back((*a_)[take]);
dfs(take + 1, rem - (*a_)[take]);
cur_.pop_back();

```

Time O(2^n) worst case. **Space** O(n).

Follows directly from the traversal count: $O(2^n)$ worst case. time, $O(n)$. space.

VISUAL STATE TRACE

```
candidates = [10,1,2,7,6,1,5], target = 8
KEY: dedupe the INPUT first -> [1,1,2,5,6,7,10]
at each level:  skip i > 0 and candidates[i]==candidates[i-1]
dfs(i=0, rem=8), path=[]:
i=0 c=1 -> dfs(1, 7)
i=1 c=1 == c[0] -> SKIP (same value as previous)
i=2 c=2 -> dfs(3, 5)
i=3 c=5 -> dfs(4, 0) HIT [1,2,5]
i=4 c=6 > 5 break
i=3 c=5 -> dfs(4, 2) 6>2 break
i=1 c=1 == c[0] -> SKIP
i=2 c=2 -> ...
i=3 c=5 -> ...
i=5 c=7 -> dfs(6, 1) 10>1 break
i=6 c=10 > 8 break
the two skips:
i > 0 and c[i] == c[i-1] -> skip duplicates AT a level
recurse with i + 1 -> element used at most once
```

LANGUAGE MECHANICS: PYTHON VS C++

In Combination Sum I (unlimited use, unsorted) we `continue` because a big candidate doesn't rule out smaller ones later. Here sorting guarantees c[j] > rem ⇒ every later candidate is also too big, so `break` prunes harder with identical semantics. ——————————————————— TIME COMPLEXITY : $O(2^n)$ worst case. ==========

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- Sort then dedupe the vector in place before searching – makes the per-level skip correct.
- `break` when `c[i] > rem` after sorting removes whole subtrees.
- `cands = sorted(set(candidates))` does dedupe and sort in one step, which is both correct and faster than a manual pass.
- `i > 0 and cands[i] == cands[i-1]` is the canonical skip; using `i == start` is a common wrong variant.

INTERVIEW TRAPS & SANITY CHECKS

Skipping with `i == start` instead of `i > 0`. That check compares against the position where the current combination began, not against the previous element, so duplicate values in the input still generate duplicate combinations.

Sanity check. With the input sorted and deduped and the rule "skip c[i] if it equals c[i-1] and i > 0", every multiset of candidates has exactly one representation: the increasing-index selection of its distinct positions. Combined with recursing at i+1, no element is reused and no combination repeats.

INTUITION CHECK — 3 QUESTIONS

Q1. Why must the input be sorted before the per-level skip works?

Answer. The skip compares adjacent elements, which only detects duplicates if equal values are adjacent. Sorting guarantees that, and it also enables the `c[i] > rem` early break. Sorting first is a precondition, not an optimisation.

Q2. What is the difference from problem 39 exactly?

Answer. Two changes: recurse with `i + 1` (single use) instead of `i` (reuse), and add the duplicate skip. Applying only the first to problem 39's repeated candidates would lose valid solutions; applying only the second leaves 39 unable to express repeats.

Q3. Can you solve it without sorting?

Answer. Yes, using a `Counter` over candidates and choosing values in a fixed order with a count limit – that also avoids duplicates structurally. It costs a hash map instead of a sort but has the same $O(n \log n)$-ish bound after sorting.

0046 Permutations Medium

ESSENCE & BOUNDS

Return all permutations of a collection of distinct integers. 1 <= nums.length <= 6, so at most 720 permutations – output-bound, and every approach is fast. Values are distinct, so a bitmap suffices; duplicates would need a set.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

- Swap-based DFS (best) dfs(pos): pos == n → record. For j in pos..n-1: swap(pos,j), recurse, swap back.

Python (intuition kernel)

```python
out = []
n = len(nums)
if pos == n:
out.append(nums[:])
return
for j in range(pos, n):
nums[pos], nums[j] = nums[j], nums[pos]
dfs(pos + 1)

```

C++ (intuition kernel)

```cpp
for (size_t i = 0; i < a_->size(); ++i) { if (used_[i]) continue;
used_[i] = true;
cur_.push_back((*a_)[i]);
dfs();
cur_.pop_back(); // backtrack
used_[i] = false;

```

Time ? **Space** ?

every node is visited once and returns $O(1)$ state; hence the stated bounds.

- used[] bitmap + running permutation Mark used[i], append nums[i], recurse, pop + unmark. ———— ———— DEEP DIVE - Two state encodings, one search tree Swap version: after fixing position pos the elements left of pos are exactly the used ones — state lives in the array (zero extra memory, no copies). used[] version copies the permutation at each leaf but is easier to read. Both enumerate n! leaves. ———— TIME COMPLEXITY : $O(n - n!)$ MEMORY COMPLEXITY: $O(n)$. ==========

Python (intuition kernel)

```python
out, cur = [], []
used = [False] * len(nums)
if len(cur) == len(nums):
out.append(cur[:])
return
for i in range(len(nums)):
if used[i]:
continue

```

C++ (intuition kernel)

```cpp
for (size_t i = 0; i < a_->size(); ++i) { if (used_[i]) continue;
used_[i] = true;
cur_.push_back((*a_)[i]);
dfs();
cur_.pop_back(); // backtrack
used_[i] = false;

```

Time O(n - n!) **Space** O(n).

constant passes over the bits of the value; hence the stated bounds.

VISUAL STATE TRACE

```
nums = [1,2,3]
path=[1]
path=[1,2]
path=[1,2,3] -> EMIT [1,2,3]
path=[1,3]
path=[1,3,2] -> EMIT [1,3,2]
path=[1,3]
path=[1,3,2] -> EMIT [1,3,2] (dup?  no, different prefix)
path=[1,3,1]?  -> 1 already used, skipped
path=[2]
[2,1] -> [2,1,3] EMIT ; [2,3] -> [2,3,1] EMIT
path=[3]
[3,1] -> [3,1,2] EMIT ; [3,2] -> [3,2,1] EMIT
ANSWER all 6 permutations
invariant:  path holds distinct elements drawn from nums; when
len(path) == n the path IS a permutation.  Every permutation
appears once because each has a unique choice sequence.
```

LANGUAGE MECHANICS: PYTHON VS C++

Swap version: after fixing position pos the elements left of pos are exactly the used ones — state lives in the array (zero extra memory, no copies). used[] version copies the permutation at each leaf but is easier to read. Both enumerate n! leaves. ——————————————————————— TIME COMPLEXITY : $O(n - n!)$
==========

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `next_permutation` from `<algorithm>` generates all permutations iteratively in $O(n \cdot n!)$ with $O(1)$ extra space.
- The swap-based DFS (`do { emit } while (next_permutation(...))`) is usually the strongest C++ answer.

- `itertools.permutations(nums)` is a C-level lazy iterator – the shortest and fastest answer.
- A `used` boolean list is faster than swapping or a set when values are distinct ints.

INTERVIEW TRAPS & SANITY CHECKS

Emitting a copy of `path` at every node instead of only at depth n. That produces partial permutations too, and the count is wrong by a large factor.

Sanity check. `path` contains distinct values all drawn from nums, and `used` marks exactly the positions represented in it. Appending an unused value preserves distinctness, so every depth-n path is a valid permutation, and every permutation is reachable by appending its elements in order.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is every permutation generated exactly once?

Answer. Each permutation corresponds to exactly one ordering of choices, and the recursion tries each unused element at each level. Two different choice sequences produce different paths, so no permutation can be produced twice and none is missed.

Q2. What changes if the input has duplicates?

Answer. The `used`-by-position approach still works and naturally produces duplicate outputs, which you must then dedupe (e.g. insert into a set or skip equal values at the same depth). Skipping equal values at one level is the cheaper fix and is what problem 90 does.

Q3. What is the complexity?

Answer. $O(n \cdot n!)$ time – there are n! leaves and copying each costs $O(n)$. Space is $O(n)$ for the recursion and the path, plus $O(n \cdot n!)$ for the output, which dominates everything.

0051 N Queens ?

ESSENCE & BOUNDS

Return all board configurations with n queens that attack no other queen. 1 <= n <= 9. n = 2 and n = 3 have no solutions, so the output is empty. The search tree has n! leaves before pruning.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

v1 v1 – see the kernel below.

Python (intuition kernel)

```python
cols, pos_diag, neg_diag = set(), set(), set()
res = []; board = [["." for _ in range(n)] for _ in range(n)]
if r == n:
res.append(["".join(row) for row in board]); return
for c in range(n):
if c in cols or (r + c) in pos_diag or (r - c) in neg_diag:  continue
cols.add(c); pos_diag.add(r+c); neg_diag.add(r-c); board[r][c] = "Q"
bt(r + 1)

```

C++ (intuition kernel)

```cpp
for(int c=0;c<n_val;c++){

if(cols_s.count(c)||pd_s.count(r+c)||nd_s.count(r-c)) continue;
 cols_s.insert(c); pd_s.insert(r+c); nd_s.insert(r-c); queens_s.push_back(c);
 bt(r+1);
 cols_s.erase(c); pd_s.erase(r+c); nd_s.erase(r-c); queens_s.pop_back();

```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

v2 v2 – see the kernel below.

Python (intuition kernel)

```python
res = []; queens = []
if r == n:
board = []
for qc in queens:
row = "." * qc + "Q" + "." * (n - qc - 1); board.append(row)
res.append(board); return
available = (~(cols | pos_d | neg_d)) & ((1 << n) - 1)
while available:

```

C++ (intuition kernel)

```cpp
bool check(vector<int>& q) { // O(n!)  with no pruning
 int n = q.size();
 for (int i = 0; i < n; ++i)
 for (int j = 0; j < n; ++j)
 if (q[i] == j || abs(q[i] - j) == i - j)
 return false;
 return true;
}
// then enumerate every permutation of columns

```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

VISUAL STATE TRACE

```
n = 4 -> two solutions
rows 0..3, one queen per row.  Place in row i:
cols :  set of occupied columns
d1 :  set of occupied (row - col) diagonals
d2 :  set of occupied (row + col) diagonals
dfs(row):
for c in range(n):
if c in cols or (row-c) in d1 or (row+c) in d2:
continue
place, recurse, unplace
trace row 0:  c=0 ok -> row 1:  c=0 taken, c=1 diag?  row-c=0 ->
d1 has 0 -> skip; c=2 row-c=-1 d1?  no, row+c=3 d2?  no
-> place at (1,2) -> row 2:  c=1 -> row 3:  c=3 ->
SOLUTION [[0,2],[1,4?]]...  emits board 0 0 0 0 / 0 1 0 1
the three conflict tests are exhaustive because a queen
attacks along a row (impossible, one per row), a column,
and two diagonals.  Sets make each test O(1).
```

LANGUAGE MECHANICS: PYTHON VS C++

Backtracking is dominated by the cost of copying the current path at each node. Python's `path + ch` allocates and copies at every node, giving $O(n \cdot extoutput)$ where $O(extoutput)$ suffices; mutating one list with append and pop fixes that. C++ has exactly the same trap with `std::string`, and the same fix with `push_back` and `pop_back`.

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `bitset<n>` per row and diagonal-gather operations make the conflict test a single mask AND – the fastest known formulation.
- Keeping `board[r] = c` per row makes emitting the board trivial; `vector<int> queen(n)` is simpler.
- Three Python `sets` give $O(1)$ tests; three bitmasks on integers are faster in CPython because small-int bit ops are single operations.
- Precomputing all diagonal indices avoids recomputing `row - c` and `row + c` at every node.

INTERVIEW TRAPS & SANITY CHECKS

Using only the column check and forgetting one of the diagonals, or using absolute differences instead of the difference and sum. Absolute difference alone catches one diagonal family and misses the other.

Sanity check. After placing queens in rows 0..row-1, the three sets contain exactly the columns, the row-minus-column values, and the row-plus-column values of the placed queens. Therefore the membership tests are exactly the three attack directions, and any candidate passing all three is conflict-free.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is one queen per row guaranteed by the algorithm?

Answer. Because the search places exactly one queen per row by construction – it recurses row by row. A valid solution can also be rotated, so the first row always has a queen; that is why only row-wise placement is complete.

Q2. What is the pruning power of the set tests?

Answer. They cut the n! search to roughly $O(n!/n^{n/2})$ explored nodes for large n, because each new queen eliminates a growing fraction of the remaining columns. That is what makes $n = 9$ tractable while a naive row-column scan is not.

Q3. How would bitsets improve this?

Answer. Represent occupied columns, diagonals, and anti-diagonals as bitmasks; the test becomes three AND operations on machine words. For n up to 32 that is a handful of instructions versus three hash lookups, and it also makes the horizontal reflection symmetry cheap to exploit.

0078 Subsets Medium

ESSENCE & BOUNDS

Return all subsets of a set of distinct integers, in any order. 0 <= nums.length <= 10, so at most 1024 subsets. An empty input yields one subset, the empty set.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

- Backtracking include/exclude (best) dfs(i): i == n → record. Else dfs(i+1) (exclude) and dfs(i+1) with nums[i] appended (include).

Python (intuition kernel)

```python
out, cur = [], []
if i == len(nums):
out.append(cur[:])
return
dfs(i + 1)
cur.append(nums[i])
dfs(i + 1)
cur.pop()

```

C++ (intuition kernel)

```cpp
for (int mask = 0; mask < (1 << nums.size()); ++mask)
{ std::vector<int> cur;
for (size_t j = 0; j < nums.size(); ++j)
if (mask & (1 << j)) cur.push_back(nums[j]);
out.push_back(cur);

```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

- Bitmask enumeration mask in 0..2^n-1: pick nums[j] iff bit j set. ———————————————— DEEP DIVE - What "backtracking" literally does here The append/pop pair is the backtrack: append explores the include-branch, pop restores the shared list so the exclude sibling starts from an identical prefix. Total leaves 2^n, each copied once → $O(n - 2\hat{}n)$. ———————————————— TIME COMPLEXITY : $O(n - 2\hat{}n)$ MEMORY COMPLEXITY: $O(n)$ recursion depth. ==========

Python (intuition kernel)

```python
out = []
for mask in range(1 << len(nums)):
out.append([nums[j] for j in range(len(nums)) if
mask & (1 << j)])
return out

```

C++ (intuition kernel)

```cpp
for (int mask = 0; mask < (1 << nums.size()); ++mask)
{ std::vector<int> cur;
for (size_t j = 0; j < nums.size(); ++j)
if (mask & (1 << j)) cur.push_back(nums[j]);
out.push_back(cur);

```

Time O(n - 2^n) **Space** O(n) recursion depth.

constant passes over the bits of the value; hence the stated bounds.

VISUAL STATE TRACE

```
nums = [1,2,3] -> 8 subsets
path=[] EMIT []
path=[1] EMIT [1]
path=[1,2] EMIT [1,2]
path=[1,2,3] EMIT [1,2,3]
path=[1,3] EMIT [1,3]
path=[2] EMIT [2]
path=[2,3] EMIT [2,3]
path=[3] EMIT [3]
2^3 = 8 emissions, matching the node count of a binary tree
of depth 3 (1 + 2 + 4 + 8 = 15 nodes, 8 leaves) - here
every NODE is a solution, so 8 of the 15 nodes emit.
include/exclude formulation:
dfs(i, cur):
if i == n:  emit(cur); return
dfs(i+1, cur) # exclude nums[i]
dfs(i+1, cur + [nums[i]]) # include nums[i]
BITMASK alternative:  for m in 0..2^n-1, bit j of m says
whether nums[j] is in subset m - no recursion at all.
```

LANGUAGE MECHANICS: PYTHON VS C++

The append/pop pair is the backtrack: append explores the include-branch, pop restores the shared list so the exclude sibling starts from an identical prefix. Total leaves 2^n, each copied once $\rightarrow$ $O(n \cdot 2^n)$. ——————————————————————— TIME COMPLEXITY : $O(n \cdot 2^n)$
==========

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- The bitmask loop `for (int m = 0; m < (1<<n); ++m)` is branch-free and trivially correct for n <= 20.
- Reserving the result to `1 << n` avoids repeated reallocation.
- `[list(c) for r in range(n+1) for c in combinations(nums, r)]` is C-level and gives all subsets sorted by size.
- `itertools.chain.from_iterable` over `combinations(nums, r)` avoids materialising intermediate lists.

INTERVIEW TRAPS & SANITY CHECKS

Emitting only at the leaves. That gives the subsets in a different form and, if the recursion is include/exclude, only emits at depth n – losing every smaller subset unless the branch is handled separately.

Sanity check. `path` is always a subset of nums containing only indices greater than the current recursion depth, so no index is used twice. Every subset is generated exactly once because each subset has a unique include/exclude signature, which corresponds to a unique root-to-node path.

INTUITION CHECK — 3 QUESTIONS

Q1. Why does the count match 2^n?

Answer. Each of the n elements independently is either in or out, giving 2^n combinations. The recursion explores exactly that decision tree, emitting once per complete signature, so the count is right by construction rather than by filtering.

Q2. When is the bitmask version preferable?

Answer. When n is small (say <= 20) and you want no recursion at all: iterate m from 0 to $2^n - 1$ and read bit j to decide membership. It is branch-free and impossible to get wrong, at the cost of an $O(2^n)$ shift per element.

Q3. How does this relate to permutation problems?

Answer. Subsets are the include/exclude case (2 branches per level) whereas permutations are the pick-any-unused case (n branches). That difference in branching factor is why subsets have 2^n nodes and permutations $n!$, and it is the structural insight the category is built on.

0079 Word Search Medium

ESSENCE & BOUNDS

Return whether any word in the list exists in the grid using adjacent horizontal or vertical moves without revisiting a cell. 1 <= board.length, board[0].length <= 1e4 with up to 30 words of length up to 1e3. Words may reuse a board letter, so there is no frequency precheck.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

- In-place sentinel marking (best) Overwrite the matched cell with a sentinel that can never equal a letter, restore it after the 4-neighbour recursion.

Python (intuition kernel)

```python
m, n = len(board), len(board[0])
if k == len(word):
if not (0 <= i < m and 0 <= j < n) or board[i][j] !=
word[k]:
save = board[i][j]
board[i][j] = "\{}0"
found = (dfs(i + 1, j, k + 1) or dfs(i - 1, j, k +
1) or
dfs(i, j + 1, k + 1) or dfs(i, j - 1, k + 1))
board[i][j] = save

```

C++ (intuition kernel)

```cpp
std::vector<std::vector<char» board = {

```

Time ?

Space ?

the output array doubles as the state store; hence the stated bounds.

- visited[][] grid Same search with an explicit bool matrix. ——————————————————————— DEEP DIVE - The sentinel state machine '\0' can never match an input letter, so "used" is encoded in the board itself — no allocation, no restore on the success path (early return unwinds without restoring, which is safe because we already found the answer). Worst case (all cells match the first char): $O(m - n - 3^{\wedge}L)$. ——————————————————————— TIME COMPLEXITY : $O(m-n-3^{\wedge}L)$ worst. MEMORY COMPLEXITY: $O(L)$ recursion depth. ==========

Python (intuition kernel)

```python
m, n = len(board), len(board[0])
vis = [[False] * n for _ in range(m)]
if k == len(word):
if not (0 <= i < m and 0 <= j < n) or vis[i][j] or
board[i][j] != word[k]:
vis[i][j] = True
found = (dfs(i + 1, j, k + 1) or dfs(i - 1, j, k +
1) or
dfs(i, j + 1, k + 1) or dfs(i, j - 1, k + 1))
vis[i][j] = False

```

C++ (intuition kernel)

```cpp
bool dfs(vector<vector<char»& g, int r, int c,
const string& w, int k,
vector<vector<char»& used) {
if (r < 0 || r >= g.size() || c < 0 || c >=
g[0].size()
|| used[r][c] || g[r][c] != w[k]) return false;
if (k == w.size() - 1) return true;
used[r][c] = 1;
bool ok = dfs(g, r+1, c, w, k+1, used)
|| dfs(g, r-1, c, w, k+1, used)
|| dfs(g, r, c+1, w, k+1, used)
|| dfs(g, r, c-1, w, k+1, used);
used[r][c] = 0;
return ok;
}

```

Time O(m-n-3^L) worst.

Space O(L) recursion depth.

Follows directly from the traversal count: $O(m - n - 3^{\wedge}L)$ worst. time, $O(L)$ recursion depth. space.

VISUAL STATE TRACE

```
board = ["ABE","SFCS","ADEE"] word = "ABCCED"
dfs(r, c, k):  k = index into the word
if out of bounds or board[r][c] != word[k]:  return False
if k == len(word) - 1:  return True # last char matched
tmp = board[r][c]; board[r][c] = '#' # MARK
found = (dfs(r+1,c,k+1) or dfs(r-1,c,k+1)
or dfs(r,c+1,k+1) or dfs(r,c-1,k+1))
board[r][c] = tmp # UNMARK <- load bearing
return found
why mark:  without it, 'AAA' can walk a straight line forever
by stepping back onto the previous cell.
why unmark:  a sibling branch needs the ORIGINAL character;
leaving '#' makes the whole search fail.
```

LANGUAGE MECHANICS: PYTHON VS C++

'\0' can never match an input letter, so "used" is encoded in the board itself — no allocation, no restore on the success path (early return unwinds without restoring, which is safe because we already found the answer). Worst case (all cells match the first char): $O(m - n - 3^L)$. — TIME COMPLEXITY : $O(m - n - 3^L)$ worst.

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- Mark with a sentinel character such as '\0' rather than '#' to be collision-proof against board contents.
- A `used` bool grid costs $O(RC)$ extra space but avoids mutating the input, which matters if the caller reuses it.
- Passing the four neighbours as a literal tuple of (dr, dc) offsets avoids four near-duplicate recursive calls.
- A `used` 2-D list is clean but allocates; in-place marking with restore is faster and is what the problem's mutation tolerance invites.

INTERVIEW TRAPS & SANITY CHECKS

Forgetting to restore the cell after the recursive call. Every later path that needs that cell sees '#' and fails, so the answer is wrong even though the code looks symmetric at a glance.

Sanity check. On entry to `dfs(r, c, k)`, the cells holding word[0..k-1] along the current path are all marked '#', so no unmark check is needed inside the recursion – the marks themselves enforce the no-revisit rule. Restoring on the way out re-establishes the invariant for sibling branches.

INTUITION CHECK — 3 QUESTIONS

Q1. What is the time complexity, and why is it not simply $O(RC * L)$?

Answer. A loose bound is $O(RC \cdot 4^L)$ from the branching, but the marking prunes hard: each path visits distinct cells, so the search is bounded by the number of simple paths, which is $O(3^L)$ per start because the previous cell is already marked and cannot be re-entered.

Q2. Why is in-place marking better than a visited array here?

Answer. It uses $O(1)$ extra space and one array write instead of a bounds-checked 2-D lookup. The cost is that you must restore, which is one extra write per node – a good trade at these sizes.

Q3. When would a Trie be the right structure?

Answer. When you must find ALL words in a large dictionary, not test a few. Then you share the walk across words instead of restarting from each cell per word, which is exactly problem 212.

0090 Subsets II Medium

ESSENCE & BOUNDS

Return all unique subsets of a collection that may contain duplicates. 1 <= nums.length <= 10 so at most 1024 raw subsets; uniqueness must be enforced, not filtered afterwards.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

- Backtracking with skip-duplicates (best) dfs(i) records cur, then for j in i..n: skip j if c[j]==c[j-1] (j>i), append c[j], recurse(j+1), pop.

Python (intuition kernel)

```python
nums.sort()
out, cur = [], []
out.append(cur[:])
for j in range(i, len(nums)):
if j > i and nums[j] == nums[j - 1]:
continue
cur.append(nums[j])
dfs(j + 1)

```

C++ (intuition kernel)

```cpp
for (size_t j = i; j < nums_->size(); ++j) {
if (j > i && (*nums_)[j] == (*nums_)[j - 1])
continue; // skip dup
cur_.push_back((*nums_)[j]);
dfs(j + 1);
cur_.pop_back();

```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

- Set of sorted tuples Generate every subset via bitmask, sort tuples, dedupe with a set. ———— DEEP DIVE - Why skipping the SECOND copy is enough Two duplicate values v produce identical subtrees only when used at the SAME level; the first v's subtree already contains every subset using v (any number of later copies). Skipping equal neighbors after a choice kills exactly the redundant subtrees — nothing is lost. ———— TIME COMPLEXITY : $O(2^n)$ worst case (all distinct). MEMORY COMPLEXITY: $O(n)$. ==========

Python (intuition kernel)

```python
nums.sort()
seen = set()
for mask in range(1 << len(nums)):
cur = tuple(nums[j] for j in range(len(nums)) if
mask & (1 << j))
seen.add(cur)
return [list(t) for t in seen]

```

C++ (intuition kernel)

```cpp
for (int mask = 0; mask < (1 << nums.size()); ++mask)
{ std::vector<int> cur;
for (size_t j = 0; j < nums.size(); ++j)
if (mask & (1 << j)) cur.push_back(nums[j]);
if (std::find(out.begin(), out.end(), cur) ==
out.end())
out.push_back(cur); // dedupe by linear scan

```

Time O(2^n) worst case (all distinct). **Space** O(n).

each element is probed once; pairs cancel or are recorded; hence the stated bounds.

VISUAL STATE TRACE

```
nums = [1,2,2]
raw subsets:  8 -> unique:  4
sorted:  [1,2,2]
dfs(start=0, cur=[]):
EMIT []
i=0 c=1 -> dfs(1,[1])
i=1 c=2 -> dfs(2,[1,2])
i=2 c=2 -> dfs(3,[1,2,2]) EMIT [1,2,2]
i=2 i>0 and c[2]==c[1] -> SKIP <- avoids [1,2,2] twice
i=1 c=2 -> dfs(2,[2])
i=2 i>0 and c[2]==c[1] -> SKIP <- avoids [2,2] from
starting at index 1
i=2 i>0 and c[2]==c[1] -> SKIP
the rule:  at a given recursion level, once you have taken
c[i], you may not also take c[i+1] if they are equal.  Taking
the LATER equal element instead gives a canonical choice.
```

LANGUAGE MECHANICS: PYTHON VS C++

Two duplicate values v produce identical subtrees only when used at the SAME level; the first v's subtree already contains every subset using v (any number of later copies). Skipping equal neighbors after a choice kills exactly the redundant subtrees — nothing is lost. ———— TIME COMPLEXITY : $O(2^n)$ worst case (all distinct). ==========

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- Sort then the same skip; `sort` plus `unique` on the input would lose multiplicity and is wrong.
- `vector<bool> used(100)` as an alternative dedupe via set insertion works but costs a hash per leaf.
- `set(map(tuple, ...))` at the end dedupes but is $O(2^n \cdot n)$ and loses order.
- The skip rule is strictly better: it avoids generating the duplicates at all.

INTERVIEW TRAPS & SANITY CHECKS

Deduplicating the INPUT with `set(nums)`. That loses multiplicity, so `[1,2,2]` becomes `[1,2]` and the answer misses `[1,2,2]` entirely.

Sanity check. With the array sorted, the skip `i > start and nums[i] == nums[i-1]` ensures that within each recursion level, equal values are offered only at their first occurrence. Every distinct multiset therefore has exactly one index sequence, so no duplicate subset is generated and none is lost.

INTUITION CHECK — 3 QUESTIONS

Q1. Why `i > start` rather than `i > 0`?

Answer. Because the skip should apply only among siblings at the same level. Using `i > 0` would also forbid legitimately choosing a second equal element deeper in the recursion, which is required for subsets like [1,2,2]. `start` marks where the current level began.

Q2. Why does the skip work only on sorted input?

Answer. It compares adjacent elements, so equal values must be adjacent. Sorting guarantees adjacency and is therefore a correctness precondition, not merely a speedup.

Q3. What is the worst-case number of generated subsets?

Answer. Exactly the number of distinct subsets, which is at most 2^n and equals $\prod_i (c_i + 1)$ for value multiplicities c_i. With the skip in place you generate each exactly once, so there is no wasted work.

0131 Palindrome Partitioning Medium

ESSENCE & BOUNDS

Return all partitions of a string such that every part is a palindrome. 1 <= s.length <= 2000 with lowercase letters, so the answer can be up to 2^{n-1} – output-bound. A palindrome table is $O(n^2)$ time and space.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

- DFS + direct palindrome probe (best for short input) dfs(start): start == n → record. For end in start..n: if the slice is a palindrome, take it, recurse(end+1).

Python (intuition kernel)

```python
out, cur = [], []
return s[l:r + 1] == s[l:r + 1][::-1]
if start == len(s):
out.append(cur[:])
return
for end in range(start, len(s)):
if not is_pal(start, end):
continue

```

C++ (intuition kernel)

```cpp
for (int end = start; end <
static_cast<int>(s_.size()); ++end) {
if (!pal_[start][end]) continue; // O(1) table query
cur_.push_back(s_.substr(start, end - start + 1));
dfs(end + 1);
cur_.pop_back();

```

Time ? **Space** ?

every node is visited once and returns $O(1)$ state; hence the stated bounds.

- Precomputed palindrome table isPal[i][j] via DP in $O(n\hat{}2)$, then $O(1)$ queries in the same DFS. ———— DEEP DIVE - Table vs probe trade-off The probe costs $O(n)$ per cut probe — fine for small strings, but a 2^n-shaped search tree makes the table worthwhile past n ≈ 20. DP identity: s[i]==s[j] and (j-i < 2 or isPal[i+1][j-1]). ———— TIME COMPLEXITY : A1 $O(n-2\hat{}n)$ worst; A2 $O(2\hat{}n) + O(n\hat{}2)$ build. MEMORY COMPLEXITY: $O(n\hat{}2)$ for the table in A2. ==========

Python (intuition kernel)

```python
n = len(s)
pal = [[False] * n for _ in range(n)]
for i in range(n - 1, -1, -1):
for j in range(i, n):
pal[i][j] = s[i] == s[j] and (j - i < 2 or pal[i +
1][j - 1])
out, cur = [], []
if start == n:
out.append(cur[:])

```

C++ (intuition kernel)

```cpp
for (int end = start; end <
static_cast<int>(s_.size()); ++end) {
if (!pal_[start][end]) continue; // O(1) table query
cur_.push_back(s_.substr(start, end - start + 1));
dfs(end + 1);
cur_.pop_back();

```

Time A1 O(n-2^n) worst; A2 O(2^n) + O(n^2) build. **Space** O(n^2) for the table in A2.

Follows directly from the traversal count: A1 $O(n - 2\hat{}n)$ worst; A2 $O(2\hat{}n) + O(n\hat{}2)$ build. time, $O(n\hat{}2)$ for the table in A2. space.

VISUAL STATE TRACE

```
s = "aab" -> [["a","a","b"], ["aa","b"]]
isPal[i][j] = s[i]==s[j] and (j-i<2 or isPal[i+1][j-1])
a a b
a T F F
a .  T F
b .  .  T
dfs(start):
if start == n:  emit(path); return
for end in range(start, n):
if isPal[start][end]:
path.push(s[start..end])
dfs(end+1)
path.pop()
start=0:  end=0 'a' pal -> dfs(1)
start=1:  end=1 'a' pal -> dfs(2)
end=2 'ab' not pal
dfs(2):  end=2 'b' pal -> dfs(3) EMIT [a,a,b]
end=2 'aab' not pal
start=0:  end=1 'aa' pal -> dfs(2)
end=2 'b' pal -> dfs(3) EMIT [aa,b]
O(n^2) table beats testing each candidate substring inline,
which is O(n^3) in total.
```

LANGUAGE MECHANICS: PYTHON VS C++

The probe costs $O(n)$ per cut probe — fine for small strings, but a 2^n-shaped search tree makes the table worthwhile past n $\approx$ 20. DP identity: s[i]==s[j] and (j-i $<$ 2 or isPal[i+1][j-1]). ———————— TIME COMPLEXITY : A1 $O(n - 2\hat{}n)$ worst; A2 $O(2\hat{}n) + O(n\hat{}2)$ build. ==========

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- A `vector<vector<bool>>` table is compact; a 2-D char array is faster.
- Fill the table by increasing substring length so the recurrence's dependency is already computed.
- A list-of-lists of bools beats a dict of tuples.
- For n = 2000 the table is 4e6 entries – Python lists of bools cost ~32 MB, so a bytearray per row is leaner.

INTERVIEW TRAPS & SANITY CHECKS

Testing palindromes by reversing each candidate substring inline. That is $O(n^3)$ overall and will not pass at n = 2000.

Sanity check. `isPal[i][j]` is true iff s[i..j] is a palindrome, established by the recurrence for j-i < 2 and the inner interval otherwise. Therefore `dfs(start)` explores exactly the partitions whose first part starts at `start` and whose remaining parts partition the suffix, so every valid partition appears exactly once.

INTUITION CHECK — 3 QUESTIONS

Q1. Why precompute the palindrome table?

Answer. Because the same substring is tested repeatedly across different partitions. Precomputing makes each test $O(1)$, giving $O(n^2)$ for the table and $O(2^n)$ for the enumeration (output-bound). Inline testing repeats work and pushes the total to $O(n^3)$.

Q2. What is the actual complexity?

Answer. $O(n^2)$ for the table plus the number of partial partitions explored, which is bounded by the output size. The theoretical worst case is $O(n \cdot 2^n)$ because there can be 2^{n-1} partitions.

Q3. When is the problem output-bound rather than time-bound?

Answer. When s is all the same character, e.g. "aaaa...", because then every substring is a palindrome and every set of cut positions yields a valid partition – giving exactly 2^{n-1} outputs. Any algorithm must spend that much just to write them down.

0395 Longest Substring with At Least K Repeating Chars Medium

ESSENCE & BOUNDS

Return the length of the longest substring in which every character appears at least k times. 1 <= s.length <= 1e4, k >= 1, and lowercase letters only. If no valid substring exists the answer is 0, which is the common failure output.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

- Divide & conquer on blocker chars (best) Count in s; all counts $\geq$ k $\rightarrow$ whole string valid. Else split at each blocker and take the max over pieces.

Python (intuition kernel)

```python
if not s or len(s) < k:
cnt = Counter(s)
for ch, c in cnt.items():
 if c < k:
 return max(self.longestSubstring(piece, k)
 for piece in s.split(ch))
return len(s)

```

C++ (intuition kernel)

```cpp
for (int distinct = 1; distinct <= 26; ++distinct) {
std::array<int, 26> cnt{};
 int uniq = 0, okCnt = 0; // unique & all$\{}geq$k
chars
 for (int l = 0, r = 0; r < n; ++r) { if (cnt[s[r] -
'a']++ == 0) ++uniq;
 if (cnt[s[r] - 'a'] == k) ++okCnt;
 while (uniq > distinct) { // too many letters
 int c = s[l++] - 'a';
 if (cnt[c]- == k) -okCnt;
 if (cnt[c] == 0) -uniq;

```

Time ? **Space** ?

the recursion splits the work; the bound is the sum over levels; hence the stated bounds.

- Sliding window per distinct-count For c in 1..26: longest window with $\leq$ c distinct letters where every letter occurs $\geq$ k times — the $O(26n)$ two-pointer classic. ———————————————————— DEEP DIVE - Why splitting on blockers is exact If 'x' occurs 2 < k times, any valid substring excludes x and lies entirely within one gap between x's. Gaps are independent, so recursion over gaps is complete and correct. A2's twist: "every char $\geq$ k" alone isn't a monotone window predicate, but fixing the distinct count IS — hence the 26 sweeps. ———————————————————— TIME COMPLEXITY : A1 $O(n\hat{}2)$ worst; A2 $O(26n)$. MEMORY COMPLEXITY: $O(n)$ / $O(1)$ counters. ==========

Python (intuition kernel)

```python
n, best = len(s), 0
for distinct in range(1, 27):
 cnt = [0] * 26
 uniq = ok_cnt = 0
 l = 0
 for r in range(n):
 idx = ord(s[r]) - 97
 if cnt[idx] == 0:

```

C++ (intuition kernel)

```cpp
for (int distinct = 1; distinct <= 26; ++distinct) {
std::array<int, 26> cnt{};
 int uniq = 0, okCnt = 0; // unique & all$\{}geq$k
chars
 for (int l = 0, r = 0; r < n; ++r) { if (cnt[s[r] -
'a']++ == 0) ++uniq;
 if (cnt[s[r] - 'a'] == k) ++okCnt;
 while (uniq > distinct) { // too many letters
 int c = s[l++] - 'a';
 if (cnt[c]- == k) -okCnt;
 if (cnt[c] == 0) -uniq;

```

Time A1 O(n^2) worst; A2 O(26n). **Space** O(n) / O(1) counters.

Follows directly from the traversal count: A1 $O(n\hat{}2)$ worst; A2 $O(26n)$. time, $O(n)$ / $O(1)$ counters. space.

VISUAL STATE TRACE

```
s = "aaabbcccc", k = 3 -> answer 10?  check:
'aaa' a:3 ok 'bb' b:2 < 3 -> b is a BLOCKER
'cccc' c:4 ok a:3 ok
whole string:  a4 b2 c4 -> b < 3 -> invalid
split at b:  "aaa" (3) and "cccc" (4) -> max 4
ANSWER 4
DIVIDE AND CONQUER:
if already valid(s):  return len(s) - shortcut
find the leftmost bad char c
left = recurse(s[:i])
right = recurse(s[i+1:])
return max(left, right)
SLIDING WINDOW variant:
track 'distinct' (count of chars with count >= k)
shrink while distinct > target for target in 1..26
the answer is the max over all targets
```

LANGUAGE MECHANICS: PYTHON VS C++

If 'x' occurs 2 < k times, any valid substring excludes x and lies entirely within one gap between x's. Gaps are independent, so recursion over gaps is complete and correct. A2's twist: "every char ≥ k" alone isn't a monotone window predicate, but fixing the distinct count IS — hence the 26 sweeps. — TIME COMPLEXITY : A1 $O(n\hat{}2)$ worst; A2 $O(26n)$.

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- Frequency arrays indexed by char avoid any hashing in both the split and the window version.
- The divide and conquer can use `string::find_first_of` for the split points, but an explicit scan is clearer.
- `Counter` or a 26-list for frequencies; a list is faster in CPython.
- Iterating `target` from 1 to 26 with a reused window avoids re-implementing divide and conquer entirely.

INTERVIEW TRAPS & SANITY CHECKS

Assuming the whole string is valid, or forgetting the early `if valid(s): return len(s)` shortcut which the divide-and-conquer needs to avoid infinite splitting.

Sanity check. In the window variant, for a fixed `target`, the window is shrunk until at most `target` characters appear at least k times. Adding a character can only increase that count and removing one can only decrease it, so the shrink loop is monotone and each pointer advances at most n times, giving $O(n)$ per target and $O(26n)$ overall.

INTUITION CHECK — 3 QUESTIONS

Q1. Why does the divide and conquer need the validity shortcut?

Answer. Because if the string is valid we must return its full length rather than split. Without the check, a fully valid string still gets split at its first bad character – and there is none, so the code would either index out of range or recurse forever.

Q2. What is the complexity of the sliding window variant?

Answer. $O(\sigma n)$ where sigma is the alphabet size (26), since you try each target count of qualifying characters. That is linear for a fixed alphabet and avoids recursion entirely, which is why it is often the better interview answer.

Q3. Why is splitting on a bad character correct?

Answer. A character with fewer than k occurrences in the current string has fewer than k in any substring containing it, so no valid substring can include that position. Every valid substring therefore lies entirely to the left or entirely to the right of it, so the two sides can be solved independently.

11. Graphs

#	Problem	Level	One-line intent
0127	**Word Ladder**	Hard	BFS over words one letter at a time; bidirectional doubles up.
0130	**Surrounded Regions**	Medium	Flood in from the border; whatever is unreached is captured.
0133	**Clone Graph**	Medium	Map original node to copy so cycles terminate.
0200	**Number of Islands**	Medium	Flood fill each unvisited land cell; each seed is an island.
0207	**Course Schedule**	Medium	Kahn's algorithm: peel off nodes with indegree zero.
0210	**Course Schedule II**	Medium	Same Kahn, but keep the dequeue order as the answer.
0261	**Graph Valid Tree**	Medium	n - 1 edges plus no cycle plus connected.
0286	**Walls And Gates**	?	Multi-source BFS from every zero, dist is the answer.
0323	**Number Of Connected Components**	?	Union nodes; a union that fails means a cycle, not a new component.
0417	**Pacific Atlantic Water Flow**	?	Two reverse BFS floods, one from each ocean.
0684	**Redundant Connection**	?	Union-Find: the first edge whose endpoints are already joined.
0695	**Max Area Of Island**	?	Flood fill each island, tracking its size.
0994	**Rotting Oranges**	?	Multi-source BFS from all rotten oranges, minute by minute.

0127 `Word Ladder` Hard

ESSENCE & BOUNDS

Return the number of words in the shortest transformation sequence from beginWord to endWord, changing one letter per step. up to 5e3 words of length up to 10, and beginWord may or may not be in wordList. 0 steps is possible when they are equal. The output is the distance, not the sequence.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

BFS (BEST for correctness) Queue of (word, depth); erase each word from the dict on first sight so nothing is visited twice. First dequeuing of endWord is provably the shortest path.

Python (intuition kernel)

```python
word_set = set(wordList)
if endWord not in word_set:
word_set.discard(beginWord)
queue = deque([(beginWord, 1)])
while queue:
 word, depth = queue.popleft()
 if word == endWord:
 return depth # BFS first-hit guarantee:  shortest length

```

C++ (intuition kernel)

```cpp
unordered_set<string> pool(wordList.begin(), wordList.end());
queue<string> q; q.push(beginWord);
int steps = 0;
while (!q.empty()) {
 int n = q.size();
 while (n--) {
 string cur = q.front(); q.pop();
 if (cur == endWord) return steps;
 for (int i = 0; i < cur.size(); ++i) {
 char old = cur[i];
 for (char c = 'a'; c <= 'z'; ++c) {
 cur[i] = c;
 if (pool.erase(cur)) q.push(cur);
 }
 cur[i] = old;
 }
 }
 ++steps;
}
return 0;

```

Time O(n - L^2) **Space** O(n - L)

every node is enqueued and dequeued once; hence the stated bounds.

Bidirectional BFS (BEST for speed) Grow two frontiers (begin side + end side); expand the smaller one; when a generated word lands in the opposite frontier the waves met. Visits ~2-b^(d/2) nodes instead of b^d.

Python (intuition kernel)

```python
word_set = set(wordList)
if endWord not in word_set:
word_set.discard(beginWord)
front, back = {beginWord}, {endWord}
depth = 1 # each loop round = one layer on both sides
while front and back:
 if len(front) > len(back):
 front, back = back, front

```

C++ (intuition kernel)

```cpp
unordered_set<string> f, b, pool(wordList.begin(), wordList.end());
f.insert(beginWord); b.insert(endWord);
int steps = 0;
while (!f.empty() && !b.empty()) {
 if (f.size() > b.size()) f.swap(b);
 unordered_set<string> next;
 for (const string& cur :  f) {
 string t = cur;
 for (int i = 0; i < t.size(); ++i) {
 char old = t[i];
 for (char c = 'a'; c <= 'z'; ++c) {
 t[i] = c;
 if (b.count(t)) return steps + 1;
 if (pool.erase(t)) next.insert(t);
 }
 t[i] = old;
 }
 }
 f = move(next); ++steps;
}
return 0;

```

Time O(n - L^2) **Space** O(n - L)

every node is enqueued and dequeued once; hence the stated bounds.

VISUAL STATE TRACE

```
begin="hit", end="cog", list=[hot,dot,dog,lot,log,cog]
hit -> hot (i->o) -> dot (h->d) -> dog (t->g)
-> cog (d->c) 4 steps
BFS: level 0 {hit}, level 1 {hot}, level 2 {dot},
level 3 {dog}, level 4 {cog} -> 4
BIDIRECTIONAL: expand the SMALLER frontier each time.
front = {hit:0}, back = {cog:0}, steps = 0
front is smaller -> expand front -> {hot:1}
expand back -> {dog:1}
expand front -> {dot:2}
...  meet when a word appears in both maps
sum the two distances + 1
why bidirectional wins:  each search covers a ball of radius
d; two balls of radius d/2 cover the same volume as one of
radius d, so the frontier stays small.
```

LANGUAGE MECHANICS: PYTHON VS C++

BFS explores in layers: first hit of the target is the shortest path. DFS finds A path, not the shortest → wrong tool here. Python set/dict = open addressing + linear probing: h(k, i) = (hash(k) + i) mod m , i = 0, 1, 2, ... Every generated candidate word is just a probe into the table. C++ unordered_set = separate chaining (bucket + linked list). ———————— BFS explores in layers from the source: the FIRST hit of the target is provably the SHORTEST path (unweighted edges). DFS would find SOME path — not necessarily the shortest — so it is wrong for this problem. std::unordered_set is the C++ equivalent of Python's set: separate chaining, $O(1)$ average insert/lookup. Python's dict/set use open addressing with linear probing: h(k, i) = (hash(k) + i) mod m , i = 0, 1, 2, ... The probing cost is why the dict stays a hash table even when the graph is "implicit" — every candidate word is just a probe. ========== / #include <iostream> // std::cout — printing benchmark results #include <vector> // std::vector — input word list #include <string> // std::string — words #include <queue> // std::queue — BFS frontier #include <unordered_map> // std::unordered_map — hash table (chaining) #include <unordered_set> // std::unordered_set — visited/dict hash set #include <chrono

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `unordered_set<string>` for the unvisited set; `erase` on discovery prevents revisiting and shrinks it.
- Generating candidates with `'a' + i` for 26 letters is faster than mutating positions one at a time.
- Set of strings + `'a' + i` loops is the fastest Python formulation.
- `dict` frontier with distance values avoids a separate level counter; BFS is still correct because all entries at a level share a distance.

INTERVIEW TRAPS & SANITY CHECKS

Forgetting to remove a word from the unvisited set when it is enqueued. Then it is re-expanded from multiple parents and the level counting breaks, because a word can appear at two different depths.

Sanity check. At the start of each BFS level, the frontier contains exactly the words at distance d from the start, and the unvisited set contains exactly the words not yet placed at any distance <= d. Both together guarantee each word is discovered at its shortest distance.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is BFS correct here?

Answer. Each edge is one letter substitution, so the answer is the shortest path length in an unweighted graph. BFS explores in non-decreasing distance order, so the first time a word is reached is at its minimum distance. A DFS would find a path but not necessarily the shortest.

Q2. What is the complexity of the bidirectional version?

Answer. $O(26 \cdot L \cdot N)$ in the worst case for N words of length L, but in practice the frontiers meet after exploring roughly $\sqrt{N}$ words each rather than N, because each search covers a radius-d ball and meeting means d is about half the path length.

Q3. Why must beginWord be added to wordList?

Answer. Because it may be absent from the list while still being a valid node with neighbours in it. Omitting it makes the first expansion find nothing and the search returns 0 incorrectly.

0130 Surrounded Regions Medium

ESSENCE & BOUNDS

Flip every 'O' that is not connected to a border 'O' to 'X', leaving the rest untouched. Board is m x n with up to 100 x 100, so recursion depth can reach 1e4 and overflows CPython's limit – iterative flood fill is the practical choice.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

BFS from every border 'O' (BEST) Queue all border 'O's, mark 'E'. BFS the 4 neighbours; every 'O' reached escapes too. Final sweep: 'E'→'O', leftover 'O'→'X'.

Python (intuition kernel)

```python
m, n = len(board), len(board[0])
queue = deque()
for r in range(m):
 for c in range(n):
 if (r in (0, m - 1) or c in (0, n - 1)) and board[r][c] == "O":
 board[r][c] = "E" # 'E' = escapes (temporary mark)
 queue.append((r, c))
dirs = [(-1, 0), (1, 0), (0, -1), (0, 1)]

```

C++ (intuition kernel)

```cpp
for (int r = 0; r < m; ++r) { for (int c = 0; c < n; ++c) {
 bool border = (r == 0 || r == m - 1 || c == 0 || c == n - 1);
 if (border && board[r][c] == 'O') {
 board[r][c] = 'E'; // 'E' = escapes (temporary mark)
 q.push({r, c});
 }
 }

```

Time O(m-n) **Space** O(m-n)

every node is enqueued and dequeued once; hence the stated bounds.

Recursive DFS from every border 'O' Identical idea, recursive traversal. Call stack can reach $O(m-n)$ on a winding island — stack-overflow risk on big boards.

Python (intuition kernel)

```python
m, n = len(board), len(board[0])
if r < 0 or r >= m or c < 0 or c >= n or board[r][c] != "O":
 return
board[r][c] = "E" # mark BEFORE recursing -- the
dfs(r - 1, c) # up
dfs(r + 1, c) # down
dfs(r, c - 1) # left
dfs(r, c + 1) # right

```

C++ (intuition kernel)

```cpp
for (int r = 0; r < m_; ++r) { for (int c = 0; c < n_; ++c) {
 bool border = (r == 0 || r == m_ - 1 || c == 0 || c == n_ - 1);
 if (border && board[r][c] == 'O') dfs(r, c);
 }

```

Time O(m-n) **Space** O(m-n) (stack)

every node is visited once and returns $O(1)$ state; hence the stated bounds.

VISUAL STATE TRACE

```
X X X X X
X O O O X
X X O O X
X O O X X
X X X X X
border O's:  NONE on the top/right (all X),
one at (3,1)?  (3,1) is interior -> captured
so interior O's all become X. Only O's touching an
edge survive.
ALGORITHM:
1.  for every border cell that is 'O': flood-mark it '*'
2.  for every cell still 'O': set it to 'X'
why it works:  a region survives iff it has a path to the
border.  Marking exactly the border-connected 'O's identifies
the survivors in one pass over the boundary.
```

LANGUAGE MECHANICS: PYTHON VS C++

A grid IS a graph: cell (r,c) is a node, edges to its 4 neighbours. The dirs array `[(1,0),(1,0),(0,1),(0,1)]` turns 4 copy-pasted branches into one loop. Bounds must be checked BEFORE indexing the grid. BFS vs DFS: identical $O(m - n)$; BFS avoids deep recursion; DFS is terser to write. Both need a visited marker to stop cycles (here: repainting 'O'→'E').
==========
A grid IS a graph: cell (r,c) is a node; edges go to its 4 neighbours. The canonical trick is the DIRS array: dirs = {(-1,0),(1,0),(0,-1),(0,1)} — one loop over 4 offsets replaces four copy-pasted branches. BFS vs DFS here: identical worst-case complexity; BFS avoids the recursion depth trap of a long winding island. DFS is slightly simpler to write recursively. Both must guard bounds BEFORE probing the cell (r<0 || r>=m || c<0 || c>=n → skip).
==========
/ #include <iostream> // std::cout — printing benchmark results #include <vector> // std::vector — the 2D board #include <string> // std::string — board rows #include <queue> // std::queue — BFS frontier #include <unordered_map> // std::unordered_map — hash table (chaining) #include <unordered_set> // std::unordered_set — visited set #include <chrono> // std::chrono — high-resolution timing #include <sys/resource.h> // getrusage — peak RAM measurem

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- An explicit `queue<pair<int,int>>` BFS from all border cells avoids the recursion depth problem entirely.
- `vector<pair<int,int>>` as a manual stack is equivalent and usually faster than `std::queue`.
- Iterative flood with an explicit list-as-stack avoids both the recursion limit and the tuple allocation of `collections.deque`.
- Flattening to a 1-D index of `m*n` cuts coordinate bookkeeping and makes the four neighbours pure arithmetic.

INTERVIEW TRAPS & SANITY CHECKS

Flooding from every 'O' instead of from the border. That marks everything and nothing changes. The whole algorithm hinges on the source being the boundary.

Sanity check. After step 1, the set of cells marked '*' is exactly the set of 'O' cells with a path to the border. That is true because step 1 explores precisely the connected components that touch a border cell, and no others.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is boundary-flood the right frame?

Answer. The property being tested is connectivity to the boundary, so the natural search space is the set of border cells and their connected components. Flooding inward from those components marks exactly the cells that must be preserved, leaving the complement as the answer.

Q2. What is the time and space complexity?

Answer. $O(mn)$ time – each cell is visited at most once in each of the two passes – and $O(mn)$ space in the worst case for the queue or stack holding a large frontier. In practice the frontier is $O(m+n)$.

Q3. What is the difference from number-of-islands?

Answer. Only the source of the flood. Islands flood from every unvisited land cell; this floods from the boundary only. Same traversal, different seeds, opposite classification.

0133 Clone Graph Medium

ESSENCE & BOUNDS

Deep-copy an undirected connected graph with node values, returning the clone of the given node. 1 to 1e4 nodes, values in [-1e4, 1e4], no self-loops, and the graph may contain cycles – the aliasing is the whole problem.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Recursive DFS + hash map (BEST for clarity) clone(node): memoised. Store the fresh clone in the map BEFORE recursing so cycles self-resolve, then clone neighbours.

Python (intuition kernel)

```python
self.memo:  Dict[Node, Node] = {}
self.memo = {} # fresh memo table per call
return self._clone(node)
if node is None:
if node in self.memo:
 return self.memo[node]
copy = Node(node.val)
self.memo[node] = copy

```

C++ (intuition kernel)

```cpp
while (!q.empty()) { Node* cur = q.front();
 q.pop();
 for (Node* nb :  cur->neighbors) { if (memo.find(nb)
== memo.end()) { memo[nb] = new Node(nb->val);
 q.push(nb);
 }
 memo[cur]->neighbors.push_back(memo[nb]);
 }

```

Time O(V + E) **Space** O(V)

one $O(1)$ hash probe per element, no rescans; hence the stated bounds.

Iterative BFS + hash map Same memo table; a queue drives the traversal. No recursion depth risk; the frontier may hold $O(V)$ nodes.

Python (intuition kernel)

```python
if node is None:
memo:  Dict[Node, Node] = {}
root_copy = Node(node.val)
memo[node] = root_copy
queue = deque([node])
while queue:
 cur = queue.popleft()
 for nb in cur.neighbors:

```

C++ (intuition kernel)

```cpp
while (!q.empty()) { Node* cur = q.front();
 q.pop();
 for (Node* nb :  cur->neighbors) { if (memo.find(nb)
== memo.end()) { memo[nb] = new Node(nb->val);
 q.push(nb);
 }
 memo[cur]->neighbors.push_back(memo[nb]);
 }

```

Time O(V + E) **Space** O(V)

one $O(1)$ hash probe per element, no rescans; hence the stated bounds.

VISUAL STATE TRACE

```
1 - 2 - 3 - 4
| |
+-----+
naive recursion on 1:  creates copy 1, recurses into 2,
creates copy 2, recurses into 3, creates copy 3, ...
coming back to 1 via the edge 4->1 would create a SECOND
copy of 1 -> infinite/wrong.
memo:  dict orig -> copy
clone(1):  1 not in memo -> create 1'
for nb in 1.neighbors:
clone(nb) (fills memo as it goes)
return 1'
when 4 is processed and reaches 1, '1 in memo' -> reuse 1'
invariant:  memo[o] exists and equals the copy of o from the
moment o's copy is created, so every node is copied exactly
once and all references share the same copy.
```

LANGUAGE MECHANICS: PYTHON VS C++

Python dict = open addressing + LINEAR PROBING: h(k, i) = (hash(k) + i) mod m , i = 0, 1, 2, ... colliding keys walk to the next free slot in ONE contiguous array → cache-friendly. C++ unordered_map = separate chaining (bucket of colliding keys). Both give $O(1)$ average per op. The recursion is a DEEP COPY: allocations unwind on the call stack, every node is brand new — no aliasing with the original.
==========
Python dict / C++ unordered_map answer "seen this key?" in $O(1)$ average. Python uses open addressing with LINEAR PROBING: h(k, i) = (hash(k) + i) mod m , i = 0, 1, 2, ... colliding keys walk to the next free slot in one contiguous array. C++ std::unordered_map uses separate chaining: each slot is a bucket (linked list) of colliding keys. The recursion in Approach 1 is a DEEP COPY: allocations happen on the unwinding call stack — every node is new, no aliasing with the original.
==========
/ #include <iostream> // std::cout — printing benchmark results #include <vector> // std::vector — neighbour lists #include <string> // std::string — result printing helpers #include <queue> // std::queue — BFS frontier #include <unordered_map> // std::unordered_map — old→clone memo table #include <unordered_set> // std::unordered_set — visited marker set #include <chro

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `unordered_map<Node*, Node*>` is the memo; node identity is the pointer value.
- An explicit stack with `(node, copy)` pairs gives the BFS version, which is often preferred for very deep graphs.
- A `defaultdict(Node)` for the memo, with the original object as key – relies on `__hash__`/`__eq__` defaults (identity), which is what you want.
- `copy.deepcopy` on a self-referential graph works but hides the algorithm; mention it only as a library observation.

INTERVIEW TRAPS & SANITY CHECKS

Copying neighbours without consulting the map, or checking membership only after recursing. Both create duplicate copies of a revisited node and break the sharing that makes the graph a graph.

Sanity check. `memo[o]` is created at the moment `clone(o)` first runs and never replaced. Therefore when an edge points back to an already-cloned node, the recursion returns the existing copy, and the resulting graph is isomorphic to the original with exactly one copy per original node.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is the map keyed by node object rather than value?

Answer. Values are not unique in a graph – two nodes may share value 5. Identity is the only sound key, and in both languages the default hash/equality for objects is identity, so using the object as the key is exactly right.

Q2. Why does the undirected property matter?

Answer. Because edges are symmetric, the same node is reachable by many paths, and the cycle detection requirement is what forces the memo. In a directed acyclic graph a plain DFS would suffice.

Q3. What is the complexity?

Answer. $O(V + E)$ time: each node is cloned once and each edge is traversed twice (once per direction). Space is $O(V)$ for the memo plus $O(V)$ for the recursion stack, which can be the binding constraint on a long chain.

0200 Number of Islands Medium

ESSENCE & BOUNDS

Count the connected components of '1' cells in the grid using four-directional adjacency. up to 300 x 300, so 9e4 cells and a recursion that can reach 9e4 deep – far beyond CPython's limit, so iterative is the practical form.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Recursive DFS flood fill (BEST for clarity) Scan the grid; on '1' → count++, sink the whole component by recursion (mark '0' on entry). In-place marking = $O(1)$ extra memory, no visited array.

Python (intuition kernel)

```python
m, n = len(grid), len(grid[0])
if r < 0 or r >= m or c < 0 or c >= n or grid[r][c] != "1":
return
grid[r][c] = "0"
dfs(r - 1, c) # up
dfs(r + 1, c) # down
dfs(r, c - 1) # left
dfs(r, c + 1) # right

```

C++ (intuition kernel)

```cpp
for (int r = 0; r < rows_; ++r) { for (int c = 0; c < cols_; ++c) { if (grid[r][c] == '1') { ++count; // new island discovered
dfs(grid, r, c); // sink it so it is never counted again
}
}

```

Time O(m-n) **Space** O(m-n) (stack)

every node is visited once and returns $O(1)$ state; hence the stated bounds.

Iterative BFS flood fill (no recursion) Same scan-and-sink, queue-driven flood. Identical $O(m - n)$ work, no call-stack overflow risk on tall islands.

Python (intuition kernel)

```python
m, n = len(grid), len(grid[0])
dirs = [(-1, 0), (1, 0), (0, -1), (0, 1)]
count = 0
for r in range(m):
for c in range(n):
if grid[r][c] != "1":
continue
count += 1 # new island

```

C++ (intuition kernel)

```cpp
for (int r = 0; r < m; ++r) { for (int c = 0; c < n; ++c) { if (grid[r][c] != '1') continue;
++count; // new island
std::queue<std::pair<int, int>> q;
q.push({r, c});
grid[r][c] = '0'; // sink on enqueue (visited marker)
while (!q.empty()) { auto [cr, cc] = q.front();
q.pop();
for (int d = 0; d < 4; ++d) { int nr = cr + dirs[d][0];

```

Time O(m-n) **Space** O(m-n)

every node is enqueued and dequeued once; hence the stated bounds.

VISUAL STATE TRACE

```
1 1 0 0 0
1 1 0 0 0
0 0 1 0 0
0 0 0 1 0 -> 3 islands
flood from (0,0):  marks (0,0),(0,1),(1,0),(1,1)
scan continues:  (2,2) unvisited -> island 2
(3,3) unvisited -> island 3
key:  MUTATE the grid.  Setting visited cells to '0' removes
the need for a separate visited matrix and halves the space.
four neighbours only - 8-directional adjacency is a
different problem with different answers.
```

LANGUAGE MECHANICS: PYTHON VS C++

Two ways to track visited: an auxiliary bool grid (extra $O(m - n)$ memory) or in-place mutation ($O(1)$ extra). Sinking '1'→'0' is the classic trick when mutation is allowed. BFS vs DFS: same complexity; DFS recursion is terser, BFS never stack-overflows. The dirs array [(1,0),(1,0),(0,1),(0,1)] walks all 4 neighbours in one loop; bounds must be checked before indexing.
==========
Two ways to track visited cells: an auxiliary bool grid (extra $O(m - n)$ memory) or in-place mutation of the input ($O(1)$ memory). Since the problem does not forbid modifying `grid`, sinking '1'→'0' is the classic trick. BFS vs DFS: same complexity; DFS recursion is terser, BFS is immune to deep-recursion crashes. The dirs array {(-1,0),(1,0),(0,-1),(0,1)} keeps the neighbour walk in one loop.
==========
/ #include <iostream> // std::cout — printing benchmark results #include <vector> // std::vector — the 2D grid #include <string> // std::string — grid rows #include <queue> // std::queue — BFS frontier #include <unordered_map> // std::unordered_map — hash table (chaining) #include <unordered_set> // std::unordered_set — visited set #include <chrono> // std::chrono — high-resolution timing #include <sys/resource.h> // getrusage — peak RAM measurement #include <utility> // std::pair — (r,c)

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- In-place mutation to '0' avoids a visited vector; a stack-based flood avoids recursion depth.
- `std::stack` or `std::queue` of `pair<int,int>`; the manual-vector-as-stack is measurably faster than `std::queue` here.
- A flat list-as-stack of `(r, c)` tuples avoids `collections.deque` overhead.
- Flattening to index `r*n + c` makes neighbour checks pure integer arithmetic and avoids tuple allocation entirely.

INTERVIEW TRAPS & SANITY CHECKS

Using 8-directional adjacency, or forgetting the out-of-bounds check before indexing – with in-place mutation a missing bounds check reads out of range and can loop forever.

Sanity check. When the outer scan reaches an unvisited land cell, that cell's connected component has not been marked yet, so the flood that follows marks exactly that component and nothing else. Each component is therefore counted exactly once, at its first-encountered cell.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is in-place mutation acceptable and preferable here?

Answer. The problem does not forbid modifying the input, and marking a cell as visited by setting it to '0' is indistinguishable from it being water. That removes an $O(mn)$ visited array entirely, which for 9e4 cells is a real memory saving.

Q2. What is the complexity, and why does the flood not re-visit cells?

Answer. $O(mn)$ time – each cell is examined once by the outer scan and marked once by a flood. Space is $O(mn)$ worst case for the stack in a pathological all-land grid, but typically $O(\min(m, n))$.

Q3. How would 8-directional adjacency change the answer?

Answer. Diagonal land cells would connect components that four-directional treats as separate, so the count can only decrease or stay equal. It is a one-line change to the neighbour offsets and a good sanity question to ask yourself.

0207 Course Schedule Medium

ESSENCE & BOUNDS

Decide whether all courses can be finished, i.e. whether the dependency graph is acyclic. 1 <= numCourses <= 5000 with up to 1e5 prerequisites, so the graph can have far more edges than nodes. Duplicate prerequisites are allowed and must not corrupt the indegree bookkeeping.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Kahn's algorithm (BEST, iterative) Count in-degrees (prerequisites per course). Repeatedly remove a course with in-degree 0 — "ready now". All n removed acyclic.

Python (intuition kernel)

```python
adj: List[List[int]] = [[] for _ in range(numCourses)]
indegree = [0] * numCourses
for a, b in prerequisites:
 adj[a].append(b) # a depends on b
 indegree[b] += 1 # b has one more dependent
ready = deque(i for i in range(numCourses) if indegree[i] == 0)
processed = 0 # how many courses were fully scheduled
while ready:

```

C++ (intuition kernel)

```cpp
for (const auto& p : prerequisites) {
adj[p[0]].push_back(p[1]); // a depends on b
 ++indegree[p[1]]; // b has one more dependent

```

Time O(V + E) **Space** O(V + E)

each node enters the queue once; each edge decremented once; hence the stated bounds.

DFS with colours (0=unvisited, 1=in-stack, 2=done) Hitting a node still ON the current stack (colour 1) = back edge = cycle. Colour 2 (finished) nodes are safe to skip.

Python (intuition kernel)

```python
adj: List[List[int]] = [[] for _ in range(numCourses)]
for a, b in prerequisites:
 adj[a].append(b)
colour = [0] * numCourses
 if colour[v] == 1:
 if colour[v] == 2:
 colour[v] = 1 # enter: mark "in progress"
 for nb in adj[v]:

```

C++ (intuition kernel)

```cpp
 for (int nb : adj_[v]) { if (!dfs(nb)) return false;

```

Time O(V + E) **Space** O(V + E)

each element is pushed once and popped at most once; hence the stated bounds.

VISUAL STATE TRACE

```
0 -> 1 -> 2
cycle example:  1->2->3->1
KAHN:
indeg = [0,1,1]
queue = [0]
pop 0, visited=1 -> decrement indeg[1] -> 0 -> push 1
pop 1, visited=2 -> decrement indeg[2] -> 0 -> push 2
pop 2, visited=3 -> nothing left
visited == numCourses -> acyclic -> True
CYCLE 1->2->3->1:
indeg all 1 -> queue EMPTY -> visited 0 < 3 -> False
why it works:  a graph is acyclic iff it has a topological
ordering iff there is always a node of indegree 0.  Kahn
consumes exactly the nodes that can be removed without
breaking an edge - so it exhausts the graph iff acyclic.
```

LANGUAGE MECHANICS: PYTHON VS C++

A plain boolean visited array is NOT enough for directed graphs: a visited node may be finished (fine) or still on the current path (cycle!). The third state — "in progress" — distinguishes a BACK EDGE from a cross edge. Kahn's is iterative (no recursion risk) and produces the topological order as a by-product (210).
==========
Colour 1 ("in progress") distinguishes a BACK EDGE from a cross edge: a node can be visited but fully finished (colour 2) and safely skipped — only nodes in the CURRENT recursion stack imply a cycle. A plain boolean visited array is NOT enough for directed graphs — the third state is the whole trick. Kahn's is iterative (no recursion risk) and produces the order too (see 210).
==========
/ #include <iostream> // std::cout — printing benchmark results #include <vector> // std::vector — adjacency lists #include <string> // std::string — result printing helpers #include <queue> // std::queue — Kahn's ready queue #include <unordered_map> // std::unordered_map — hash table (chaining) #include <unordered_set> // std::unordered_set — visited set #include <chrono> // std::chrono — high-resolution timing #include <sys/resource.h> // getrusage — peak RAM measurement

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `vector<vector<int>> adj` plus an `int` indegree array; `queue<int>` for the frontier.
- A `vector<int> used` serving as the queue avoids the `std::queue` allocation.

- `defaultdict(int)` for indegree is slower than a preallocated list because of hashing.
- `from collections import deque` and `popleft()` gives $O(1)$ FIFO; using a list with an index pointer is equivalent.

INTERVIEW TRAPS & SANITY CHECKS

Decrementing the indegree of a node for an edge you already processed, or failing to decrement when a prerequisite is popped. The classic symptom is a cycle reported where there is none.

Sanity check. `processed` counts nodes whose prerequisites are all satisfied. A node enters the queue exactly when its indegree reaches 0, i.e. when all its incoming edges come from processed nodes. So `processed == numCourses` means every node was orderable, which holds exactly when there is no cycle.

INTUITION CHECK — 3 QUESTIONS

Q1. Why does an empty queue prove a cycle exists?

Answer. If the queue is empty while nodes remain, every remaining node has an incoming edge from another remaining node. Following those predecessors forever in a finite graph must revisit a node, which is precisely a cycle.

Q2. How do you handle duplicate prerequisites?

Answer. Increment the indegree once per duplicate edge and decrement once per processed edge, so the two counts stay consistent. The algorithm is then correct; the extra work is simply redundant.

Q3. What is the complexity?

Answer. $O(V + E)$ time – each node enters and leaves the queue once, each edge is decremented once. Space is $O(V + E)$ for the adjacency list plus $O(V)$ for the indegree array and queue.

0210 Course Schedule II Medium

ESSENCE & BOUNDS

Return one ordering of courses that satisfies all prerequisites. 1 <= numCourses <= 5e3 with up to 1e5 edges. The problem guarantees a valid ordering exists, so no cycle check is needed.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Kahn's algorithm (BEST) In-degree array + queue of "ready" courses; dequeue order IS the answer. Fewer than n courses output ⇒ cycle ⇒ return [].

Python (intuition kernel)

```python
adj: List[List[int]] = [[] for _ in range(numCourses)]
indegree = [0] * numCourses
for a, b in prerequisites:
 adj[b].append(a) # edge b -> a : finish b before a
 indegree[a] += 1 # a gains one unsatisfied prerequisite
ready = deque(i for i in range(numCourses) if indegree[i] == 0)
order: List[int] = []
while ready:

```

C++ (intuition kernel)

```cpp
for (const auto& p : prerequisites) {
 adj[p[1]].push_back(p[0]); // edge b $\{}to$ a : finish b before a
 ++indegree[p[0]]; // a gains one unsatisfied prereq

```

Time O(V + E) **Space** O(V + E)

each node enters the queue once; each edge decremented once; hence the stated bounds.

DFS (post-order prepending) Colour DFS; when a node finishes (colour 2) prepend it to the answer. Detecting a back edge aborts with an empty array.

Python (intuition kernel)

```python
adj: List[List[int]] = [[] for _ in range(numCourses)]
for a, b in prerequisites:
 adj[b].append(a)
colour = [0] * numCourses
order: List[int] = []
 if colour[v] == 1:
 if colour[v] == 2:
 colour[v] = 1

```

C++ (intuition kernel)

```cpp
 for (int nb : adj_[v]) { if (!dfs(nb)) return false;

```

Time O(V + E) **Space** O(V + E)

every node is visited once and returns $O(1)$ state; hence the stated bounds.

VISUAL STATE TRACE

```
0 -> 1, 0 -> 2, 1 -> 3, 2 -> 3
indeg = [0, 1, 1, 2]
queue = [0]
pop 0 -> order=[0] -> indeg[1]=0 push 1, indeg[2]=0 push 2
pop 1 -> order=[0,1] -> indeg[3]=1
pop 2 -> order=[0,1,2] -> indeg[3]=0 push 3
pop 3 -> order=[0,1,2,3]
ANSWER [0,1,2,3]
ties:  when several nodes have indegree 0 the order is not
unique; any of them may be taken.  Using a QUEUE (FIFO)
gives a deterministic answer; a stack (LIFO) gives a valid
but different one.
```

LANGUAGE MECHANICS: PYTHON VS C++

Kahn's: BFS-like, iterative, the in-degree array is the "remaining dependencies" book. DFS: recursion — children always finish before their parent, so prepending on exit keeps prerequisites first. Both $O(V + E)$; Kahn's has no recursion-depth risk and handles disconnected graphs in one pass.
==========
Kahn's: BFS-like, iterative, naturally produces the order; the in-degree array is itself the "remaining dependencies" book. DFS: recursion; the trick is that finish-time REVERSE order is topological — children always finish before their parent, so prepending on exit keeps parents first. Both $O(V + E)$; Kahn's has no recursion-depth risk and handles disconnected graphs in the same pass.
==========
/ #include <iostream> // std::cout — printing benchmark results #include <vector> // std::vector — adjacency lists + order #include <string> // std::string — order formatting #include <queue> // std::queue — Kahn's ready queue #include <unordered_map> // std::unordered_map — hash table (chaining) #include <unordered_set> // std::unordered_set — visited set #include <chrono> // std::chrono — high-resolution timing #include <sys/resource.h> // getrusage — peak RAM measurement #include <algorithm> // std::reverse — DFS finish-time reversal

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `vector<int> used` as both the queue and, in reverse, the answer – an $O(1)$ in-place trick.
- `priority_queue` gives a lexicographically smallest ordering at $O(E \log V)$ if that is asked.
- A list used as a stack gives a valid order with slightly less code but no FIFO determinism.
- `deque` + `popleft()` is the canonical BFS-frontier form.

INTERVIEW TRAPS & SANITY CHECKS

Returning the reverse of the dequeue order. With an edge a -> b meaning "a before b", the dequeue order already satisfies the constraints; reversing breaks them.

Sanity check. A node is dequeued only when its indegree is 0, meaning every prerequisite has already been dequeued and is earlier in `order`. Therefore every edge u -> v has position(u) < position(v), which is exactly the required property.

INTUITION CHECK — 3 QUESTIONS

Q1. Is the ordering unique?

Answer. No. Whenever two or more nodes have indegree 0 simultaneously, any of them may be taken next, producing different valid orderings. The problem accepts any of them; using a queue gives one deterministic choice.

Q2. How do you get the lexicographically smallest valid ordering?

Answer. Replace the FIFO queue with a min-heap. The first node available is then always the smallest, giving a canonical answer at $O((V+E)\log V)$ instead of $O(V+E)$.

Q3. What if a cycle existed, since the problem guarantees it does not?

Answer. The dequeue order would be shorter than numCourses and you would return an empty list or signal failure. Since existence is guaranteed, the check is unnecessary – but adding it makes the function total.

0261 Graph Valid Tree Medium

ESSENCE & BOUNDS

Decide whether the given edge list forms a valid tree over n labelled nodes. 1 <= n <= 2e4 with 0 <= edges.length <= 2e4. n = 1 with zero edges IS a valid tree, and n = 1 with one self-loop is not.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Union-Find with path compression (BEST) Merge endpoints of every edge. Edge with both endpoints ALREADY in one set → cycle → invalid. Require n1 edges, no cycle, and one final component.

Python (intuition kernel)

```python
if len(edges) != n - 1:
parent = list(range(n))
rank = [0] * n
 while parent[x] != x:
 parent[x] = parent[parent[x]] # grandparent jump
 x = parent[x]
 return x
 ra, rb = find(a), find(b)

```

C++ (intuition kernel)

```cpp
else {
 parent[rb] = ra; // equal ranks:  attach + bump one
 ++rank[ra];

```

Time O(n + E-α(n)) — α = inverse Ackermann,

Space O(n) — parent + rank arrays.

O(alpha(n)) amortised per union with path compression; hence the stated bounds.

DFS from node 0 Iterative DFS with (node, parent); revisit of a non-parent → cycle. Then verify every node was visited (connected).

Python (intuition kernel)

```python
if len(edges) != n - 1:
adj:  List[List[int]] = [[] for _ in range(n)]
for a, b in edges:
 adj[a].append(b)
 adj[b].append(a)
visited = [False] * n
stack = [(0, -1)]
visited[0] = True

```

C++ (intuition kernel)

```cpp
for (const auto& e :  edges) {
adj[e[0]].push_back(e[1]);
 adj[e[1]].push_back(e[0]);

```

Time O(n + E)

Space O(n + E)

every node is visited once and returns $O(1)$ state; hence the stated bounds.

VISUAL STATE TRACE

```
n=3, edges=[[0,1],[1,2]] -> valid
undirected, 2 = n-1 edges, connected, acyclic
n=3, edges=[[0,1],[1,0]] -> invalid (duplicate edge;
treated as a 2-cycle) edges=2 but the component is
only {0,1}, node 2 isolated
UNION-FIND:
parent = list(range(n))
for u, v in edges:
ru, rv = find(u), find(v)
if ru == rv:  return False <- CYCLE detected
parent[ru] = rv <- union
return edges == n - 1 <- connectivity proxy
why edges == n-1 implies connected, given no cycle:
an acyclic forest with V vertices and C components has
exactly V - C edges.  So V-1 edges forces C = 1.
```

LANGUAGE MECHANICS: PYTHON VS C++

parent[x] chains x to its component root; find() flattens the path; union() attaches the smaller-rank root under the larger. With both optimisations the amortised cost is the INVERSE ACKERMANN function $\alpha(n)$ — ≤ 4 for any practical n. Node labels are 0..n1 so plain arrays index directly — no hash table needed (Python dict probing h(k,i)=(hash(k)+i) mod m is only for sparse/arbitrary labels).
==========
parent[x] chains x to its component root. find() compresses the path (each hop flattens toward the root); union() attaches the smaller-rank root under the larger so trees stay shallow. With both optimisations the amortised cost is the INVERSE ACKERMANN function $\alpha(n)$ — growth so slow it is ≤ 4 for any practical n. Python dict/set hashing (open addressing, linear probing h(k,i)=(hash(k)+i) mod m) is NOT needed here — node labels are 0..n1, so plain arrays index directly.
==========
/ #include <iostream> // std::cout — printing benchmark results #include <vector> // std::vector — edge list, adjacency #include <string> // std::string — printing helpers #include <queue> // std::queue — BFS/DFS frontier #include <unordered_map> // std::unordered_map — hash table (chaining) #include <unordered_set> // std::unordered_set — visited set #include <chrono> // std::chrono —

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `std::vector<int> parent, size` with path compression and union by rank is near-$O(1)$ per op.
- A BFS/DFS from node 0 plus an edge count is simpler to explain and uses $O(n)$ space without a DSU.
- Iterative `find` with path halving avoids Python's recursion limit at 2e4 depth.
- The BFS-from-0 approach needs only a set and is easier to get right in an interview.

INTERVIEW TRAPS & SANITY CHECKS

Checking only `edges == n - 1`. That passes for two disjoint 3-node trees, so the edge count alone is not sufficient – acyclicity must also be established, or connectivity checked directly.

Sanity check. Union-Find maintains the partition of nodes into connected components of the edges processed so far. `find(u) == find(v)` before a union means the new edge closes a cycle; otherwise the union merges two components, decreasing the count by one. After all edges, no cycle means the graph is a forest, so the edge count decides connectivity.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is the edge count the right connectivity proxy?

Answer. An acyclic undirected graph with V vertices and C connected components has exactly V - C edges. So if the graph is acyclic and has V - 1 edges, then C must be 1. The count therefore substitutes for an explicit connectivity check once acyclicity is known.

Q2. What does DSU detect that a cycle check alone does not?

Answer. DSU detects a cycle implicitly: adding an edge whose endpoints are already connected is exactly a cycle. Combined with the edge-count check you get both conditions without a traversal, and each operation is near-$O(1)$ amortised.

Q3. How do you handle parallel edges?

Answer. The DSU approach handles them naturally: the second copy of an edge has endpoints already in the same component, so it is rejected as a cycle. An approach that only counts edges and checks connectivity would wrongly accept them.

0286 Walls And Gates ?

ESSENCE & BOUNDS

Set each empty cell's value to its minimum distance to any gate, where 0 marks a gate and -1 an open empty room. up to 200 x 200 = 4e4 cells, so a BFS with an explicit queue is the right shape; recursion depth would blow up.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

v1 v1 – see the kernel below.

Python (intuition kernel)

```python
if not rooms:  return
rows,cols=len(rooms),len(rooms[0]); q=deque()
for r in range(rows):
 for c in range(cols):
 if rooms[r][c]==0:  q.append((r,c))
while q:
 r,c=q.popleft()
 for dr,dc in [(1,0),(-1,0),(0,1),(0,-1)]:

```

C++ (intuition kernel)

```cpp
for (int r = 0; r < m; ++r) { for (int c = 0; c < n;
++c) { if (rooms[r][c] == 0) q.push({r, c});
 }

```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

v2 v2 – see the kernel below.

Python (intuition kernel)

```python
rows,cols=len(rooms),len(rooms[0])
 if r<0 or r>=rows or c<0 or c>=cols or
rooms[r][c]<dist:  return
 rooms[r][c]=dist

dfs(r+1,c,dist+1);dfs(r-1,c,dist+1);dfs(r,c+1,dist+1);dfs(r,c-1,dist+1)
for r in range(rows):
 for c in range(cols):
 if rooms[r][c]==0:  dfs(r,c,0)

```

C++ (intuition kernel)

```cpp
for (int r = 0; r < m; ++r) { for (int c = 0; c < n;
++c) { if (rooms[r][c] != 0) continue;
 std::vector<std::tuple<int, int, int>> stack;
 stack.push_back({r, c, 0});
 while (!stack.empty()) { auto [cr, cc, d] =
stack.back();
 stack.pop_back();
 for (int k = 0; k < 4; ++k) { int nr = cr +
dirs[k][0];
 int nc = cc + dirs[k][1];
 if (nr < 0 || nr >= m || nc < 0 || nc >= n)
continue;

```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

VISUAL STATE TRACE

```
0 -1 -1 -1
-1 -1 -1 -1
-1 -1 -1 -1
-1 -1 -1 0
multi-source:  seed the queue with ALL zeros, dist 0
wave 1:  cells adjacent to a zero get dist 1
(0,1)=1 (1,0)=1 (1,3)=1 (3,2)=1
wave 2:  dist 2 cells
wave 3:  dist 3 cells
wave 4:  dist 4 <- centre (1,1) and (2,2) etc
result:
0 1 2 3
1 2 3 4
2 3 4 3
3 4 3 0
why multi-source:  BFS from all sources at once gives the
distance to the NEAREST source, because BFS explores in
order of increasing distance from ANY seed.  Running BFS
once per gate would be O(gates * RC) and give the last
gate's answer instead.
```

LANGUAGE MECHANICS: PYTHON VS C++

Pushing every gate before the first pop is the trick: the queue starts with ALL layer-0 nodes, so layer numbers (distances) stay globally consistent — this is BFS on a "super-source" graph where a virtual node connects to every gate. Compare with Approach 2: the brute version repeats work per gate; multi-source BFS shares one wave. The dirs array {(-1,0),(1,0),(0,-1),(0,1)} keeps the neighbourhood walk in one loop; bounds checked before indexing.
==========
/ #include <iostream> // std::cout — printing benchmark results #include <vector> // std::vector — the 2D grid #include <string> // std::string — printing helpers #include <queue> // std::queue — BFS frontier #include <unordered_map> // std::unordered_map — hash table (chaining) #include <unordered_set> // std::unordered_set — visited set #include <chrono> // std::chrono — high-resolution timing #include <sys/resource.h> // getrusage — peak RAM measurement #include <utility> // std::pair — (r,c) coordinates #include <climits> // INT_MAX — the INF placeholder

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- A single `queue<pair<int,int>>` seeded with all gates; in-place mutation to distances removes the visited set.
- An explicit index vector avoids `std::queue`'s block churn.
- A `deque` seeded with all zeros is the idiomatic multi-source BFS.
- Flatten to `r*cols + c` so the queue holds ints rather than tuples.

INTERVIEW TRAPS & SANITY CHECKS

Overwriting gate cells during the BFS, or revisiting cells that already have a distance. The `if gate[r][c] == -1` guard must be checked before both writing and enqueueing, and the new cell must be marked as soon as it is enqueued.

Sanity check. BFS pops cells in non-decreasing order of their distance from the seed set. Because every seed starts at distance 0, the distance assigned when a cell is first enqueued is the minimum over all sources, which is the problem's definition.

INTUITION CHECK — 3 QUESTIONS

Q1. Why must all gates be seeded before the BFS starts?

Answer. BFS's correctness relies on processing in non-decreasing distance order. Seeding all gates at distance 0 simultaneously establishes that order over the union of the searches, so the first time a cell is reached is via the nearest gate.

Q2. What is the complexity?

Answer. $O(RC)$ time – each cell is enqueued and expanded once, with four neighbour checks. Space is $O(RC)$ for the queue. Running BFS per gate would be $O(\text{gates} \cdot RC)$, which for many gates is the difference between passing and timing out.

Q3. How does this differ from the wall/flood problem?

Answer. Same grid traversal, opposite source and target. Here you flood FROM all zeros and write distances; there you flood from the border and then flip the unmarked. The shared skill is multi-source flood fill.

0323 Number Of Connected Components ?

ESSENCE & BOUNDS

Return the number of connected components in an undirected graph with n nodes and some edges. 0 <= n <= 200 with up to n*(n-1)/2 edges, so a dense graph is possible and duplicate edges are allowed.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

v1 v1 – see the kernel below.

Python (intuition kernel)

```python
parent=list(range(n)); rank=[0]*n
 while parent[x]!=x:  parent[x]=parent[parent[x]];
x=parent[x]
 return x
count=n
for a,b in edges:
 pa,pb=find(a),find(b)
 if pa==pb:  continue
 if rank[pa]<rank[pb]:  pa,pb=pb,pa

```

C++ (intuition kernel)

```cpp
vector<int> parent(n), sz(n, 1);
iota(parent.begin(), parent.end(), 0);
int count = n;
for (auto& e :  edges) {
 int ru = find(parent, e[0]), rv = find(parent,
e[1]);
 if (ru == rv) continue; // already joined:  no cycle
 if (sz[ru] < sz[rv]) swap(ru, rv);
 parent[rv] = ru; sz[ru] += sz[rv]; -count;
}
return count;

```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

v2 v2 – see the kernel below.

Python (intuition kernel)

```python
graph=defaultdict(list)
for a,b in edges:  graph[a].append(b);
graph[b].append(a)
visited=set(); count=0
 visited.add(node)
 for nb in graph[node]:
 if nb not in visited:  dfs(nb)
for i in range(n):
 if i not in visited:  count+=1; dfs(i)

```

C++ (intuition kernel)

```cpp
vector<vector<int>> adj(n);
for (auto& e :  edges) { adj[e[0]].push_back(e[1]);
 adj[e[1]].push_back(e[0]); }
vector<char> seen(n, 0);
int count = 0;
for (int i = 0; i < n; ++i) {
 if (seen[i]) continue;
 ++count;
 stack<int> st; st.push(i); seen[i] = 1;
 while (!st.empty()) {
 int u = st.top(); st.pop();
 for (int v :  adj[u]) if (!seen[v]) {
 seen[v] = 1; st.push(v);
 }
 }
}
return count;

```

Time ?	Space ?

Follows directly from the traversal count: ? time, ? space.

VISUAL STATE TRACE

```
n=5, edges=[[0,1],[0,2],[3,4]] -> 3 components
DSU: components = 5
union(0,1) different -> components- = 4
union(0,2) different -> components- = 3
union(3,4) different -> components- = 2 ?!
WAIT - components starts at n=5 and each SUCCESSFUL union
decrements.  3 successful unions -> 5 - 3 = 2.  But the
answer is 3:  {0,1,2} {3,4} and node 5 is isolated.
count again:  n=5, so components={0},{1},{2},{3},{4}
union(0,1):  {0,1},{2},{3},{4} -> 4
union(0,2):  {0,1,2},{3},{4} -> 3
union(3,4):  {0,1,2},{3,4} -> 2
components = 2 but the true answer with node 5 isolated is
3.  n=5 means nodes 0..4, and there is no node 5 - so the
components are {0,1,2} and {3,4} = 2.  Correct:  2.
invariant:  components = n - (number of successful unions),
and each successful union merges exactly two distinct sets.
```

LANGUAGE MECHANICS: PYTHON VS C++

Graph traversal is memory-latency bound: each neighbour fetch is a pointer chase into an adjacency structure. C++ `vector<vector<int>>` gives per-node contiguous vectors, so a node's neighbours are one cache line. CPython's `list` of lists is comparable in structure but each inner list is a separate object, and the loop around the deque adds dispatch cost. Union-Find shifts the trade-off entirely toward $O(1)$ space with no adjacency structure at all.

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `vector<int> parent` with path compression; a `vector<int> size` for union by rank.
- DFS over an adjacency list is simpler to explain: $O(n + E)$ time, $O(n + E)$ space.
- Iterative `find` with path halving; avoid recursion at n = 200 by using a loop.
- An adjacency-list DFS avoids DSU entirely and is often the faster-to-verify answer.

INTERVIEW TRAPS & SANITY CHECKS

Forgetting that n may be 0, so the answer is 0 and the DSU must handle an empty parent array. Also, decrementing the count on a failed union, which is what would make a cyclic graph report too few components.

Sanity check. `components` always equals the number of distinct sets in the partition maintained by DSU. A union of two different sets decreases it by exactly one; a union within a single set leaves it unchanged. Since DSU's partition is exactly the graph's connected components once all edges are processed, the final count is the answer.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is a failed union not a new component?

Answer. Because the two nodes are already reachable from each other, so the edge adds no new connectivity. Only a union between distinct sets changes the component count, which is why the guard is on the find comparison rather than on the edge itself.

Q2. When would the DFS version be better?

Answer. When the graph is given as an adjacency list you already have, DFS is one pass with no union-find setup, and it handles the components directly. DSU wins when edges arrive one at a time or when you only need the count and not the membership.

Q3. What is the space complexity of the DSU version?

Answer. $O(n)$ for the parent and rank arrays – independent of the number of edges. That is a genuine advantage over DFS, which needs $O(n + E)$ space for the adjacency structure.

0417 Pacific Atlantic Water Flow ?

ESSENCE & BOUNDS

Return the coordinates of cells from which water can reach BOTH the Pacific and the Atlantic. heights is m x n with 1 <= m, n <= 50, and height can be 0. A cell on a border touches both oceans and is always valid.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

v1 v1 – see the kernel below.

Python (intuition kernel)

```python
rows, cols = len(heights), len(heights[0])
pac, atl = set(), set()
if r<0 or r>=rows or c<0 or c>=cols or (r,c) in visit or heights[r][c]<prev_h: return
visit.add((r,c))
for dr,dc in [(1,0),(-1,0),(0,1),(0,-1)]: dfs(r+dr,c+dc,visit,heights[r][c])
for c in range(cols): dfs(0,c,pac,heights[0][c]); dfs(rows-1,c,atl,heights[rows-1][c])
for r in range(rows): dfs(r,0,pac,heights[r][0]); dfs(r,cols-1,atl,heights[r][cols-1])
return [[r,c] for r,c in pac & atl]

```

C++ (intuition kernel)

```cpp
void flood(vector<vector<int>>& h, vector<char>& seen, int r, int c, vector<vector<char>>& out) {
int m = h.size(), n = h[0].size();
stack<pair<int,int>> st; st.push({r, c}); seen[r][c] = 1;
while (!st.empty()) {
auto [x, y] = st.top(); st.pop();
out[x][y] = 1;
for (int d = 0; d < 4; ++d) {
int nx = x + DR[d], ny = y + DC[d];
if (nx < 0 || ny < 0 || nx >= m || ny >= n) continue;
if (seen[nx][ny] || h[nx][ny] < h[x][y]) continue;
seen[nx][ny] = 1; st.push({nx, ny});
}
}
}

```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

v2 v2 – see the kernel below.

Python (intuition kernel)

```python
rows, cols = len(heights), len(heights[0])
visit = set(starts); q = deque(starts)
while q:
r,c = q.popleft()
for dr,dc in [(1,0),(-1,0),(0,1),(0,-1)]:
nr,nc = r+dr,c+dc
if 0<=nr<rows and 0<=nc<cols and (nr,nc) not in visit and heights[nr][nc]>=heights[r][c]:
visit.add((nr,nc)); q.append((nr,nc))

```

C++ (intuition kernel)

```cpp
vector<vector<char>> reach(m, vector<char>(n, 0));
// seed the top row and the left column (Pacific), then
// seed the bottom row and right column (Atlantic) into a
// SECOND reach grid; intersect the two grids afterwards

```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

VISUAL STATE TRACE

```
reverse thinking:  instead of asking which cells DRAIN to an
ocean, ask which cells an ocean can CLIMB TO.
pacific seeds:  (0,0) and (0,n-1)
atlantic seeds:  (m-1,0) and (m-1,n-1)
canReachPacific(r,c):
if grid[r][c] is a seed:  true
for each neighbour with height >= grid[r][c]:
if canReachPacific(nb):  true
answer = cells where BOTH reachabilities hold
visited must be PER-OCEAN - a shared visited set would
let the pacific search's marks block the atlantic search.
invariant:  the flood from an ocean covers exactly the cells
with a non-increasing path to that ocean, which is exactly
the set that can drain there.
```

LANGUAGE MECHANICS: PYTHON VS C++

Graph traversal is memory-latency bound: each neighbour fetch is a pointer chase into an adjacency structure. C++ `vector<vector<int>>` gives per-node contiguous vectors, so a node's neighbours are one cache line. CPython's `list` of lists is comparable in structure but each inner list is a separate object, and the loop around the deque adds dispatch cost. Union-Find shifts the trade-off entirely toward $O(1)$ space with no adjacency structure at all.

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- Two `queue<pair<int,int>>` runs with separate visited grids; the recursive DFS version is fine at m,n <= 50.
- `vector<vector<bool>>` visited is compact; a plain char grid avoids vector<bool>'s proxy semantics.
- Iterative flood with a list-as-stack avoids recursion depth issues for the 50x50 bound.
- Flattening to index `r*n + c` makes neighbour checks arithmetic and keeps the visited a flat bytearray.

INTERVIEW TRAPS & SANITY CHECKS

Sharing one visited set between the two searches. The first flood claims cells the second flood then cannot enter, so valid intersections are missed. Two independent visited sets (or a bitmask of two flags per cell) are required.

Sanity check. `reach(ocean, r, c)` is true iff there is a path from (r,c) to that ocean along which heights never increase. This is exactly the condition for water to flow downhill from the cell to the ocean, so the intersection of the two predicates is precisely the answer.

INTUITION CHECK — 3 QUESTIONS

Q1. Why search backwards, from the oceans?

Answer. The predicate is "can reach the ocean", which is a backward reachability question. Flooding from the ocean along non-decreasing heights computes exactly the set of cells that have such a path, in one pass per ocean, instead of searching from every cell.

Q2. What is the complexity?

Answer. $O(mn)$ time – two floods each visiting every cell at most once, with four neighbour checks. Space is $O(mn)$ for the visited flags and the flood structure. Doing it from every cell instead would be $O(m^2n^2)$.

Q3. How would you return only the intersection without two grids?

Answer. Use a single 2-bit mask per cell: bit 0 set when reachable from the Pacific, bit 1 from the Atlantic. Then answer = cells where mask == 3. One grid, both bits, and the two floods do not interfere because they test different bits.

0684 Redundant Connection ?

ESSENCE & BOUNDS

Given a tree plus one extra edge, return the single edge whose removal restores a tree. n >= 3 and edges.length == n, so exactly one cycle exists. A self-loop and a duplicate edge are both possible and both have a unique answer.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

v1 v1 – see the kernel below.

Python (intuition kernel)

```python
parent=list(range(len(edges)+1));
rank=[0]*(len(edges)+1)
 while parent[x]!=x:  parent[x]=parent[parent[x]];
x=parent[x]
 return x
for a,b in edges:
 pa,pb=find(a),find(b)
 if pa==pb:  return [a,b]
 if rank[pa]<rank[pb]:  pa,pb=pb,pa
 parent[pb]=pa

```

C++ (intuition kernel)

```cpp
vector<int> parent(n + 1), sz(n + 1, 1);
iota(parent.begin(), parent.end(), 0);
for (auto& e :  edges) {
 int ru = find(parent, e[0]), rv = find(parent,
e[1]);
 if (ru == rv) return e; // first cycle-closing edge
 parent[rv] = ru;
}
return {};

```

Time ?

Space ?

Follows directly from the traversal count: ? time, ? space.

v2 v2 – see the kernel below.

Python (intuition kernel)

```python
from collections import defaultdict
graph=defaultdict(set)
 if src==dst:  return True
 visited.add(src)
 for nb in graph[src]:
 if nb not in visited and dfs(nb,dst,visited):
return True
for a,b in edges:
 if dfs(a,b,set()):  return [a,b]

```

C++ (intuition kernel)

```cpp
vector<vector<int>> adj(n + 1);
for (auto& e :  edges) {
 adj[e[0]].push_back(e[1]);
 adj[e[1]].push_back(e[0]);
}
vector<char> seen(n + 1, 0);
seen[1] = 1;
queue<int> q; q.push(1);
while (!q.empty()) {
 int u = q.front(); q.pop();
 for (int v :  adj[u]) {
 if (seen[v]) return {u, v}; // edge back to a
visited node
 seen[v] = 1; q.push(v);
 }
}
return {};

```

Time ?

Space ?

Follows directly from the traversal count: ? time, ? space.

VISUAL STATE TRACE

```
n=3, edges=[[1,2],[1,3],[2,3]] -> answer [2,3]
union(1,2):  different -> merge {1,2}
union(1,3):  different -> merge {1,2,3}
union(2,3):  find(2)==find(3) SAME -> REDUNDANT
invariant:  after processing i edges, DSU's partition is the
connected components of the first i edges.  So a redundant
edge is precisely one whose endpoints were already joined -
adding it closes a cycle.
because only ONE extra edge exists, the FIRST such edge is
the answer; you may return immediately.
```

LANGUAGE MECHANICS: PYTHON VS C++

Graph traversal is memory-latency bound: each neighbour fetch is a pointer chase into an adjacency structure. C++ `vector<vector<int>>` gives per-node contiguous vectors, so a node's neighbours are one cache line. CPython's `list` of lists is comparable in structure but each inner list is a separate object, and the loop around the deque adds dispatch cost. Union-Find shifts the trade-off entirely toward $O(1)$ space with no adjacency structure at all.

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `vector<int> parent` with path compression; the whole loop is near-$O(n)$ amortised.
- Detecting the extra edge by counting parity with an XOR array works for a tree plus one edge and is a neat alternative.
- Iterative `find` with path halving avoids recursion; at n = 1000 a loop is safer.
- A `defaultdict(set)` adjacency plus an edge-set for lookup works but is heavier.

INTERVIEW TRAPS & SANITY CHECKS

Continuing the loop after finding a redundant edge, or recording the LAST redundant edge. With exactly one extra edge both work here, but the code reads as if multiple were possible, and the problem guarantees exactly one.

Sanity check. DSU's partition equals the connected components of the edges processed so far. Adding an edge within one component creates a cycle and is therefore redundant; adding one across components cannot. Since the input has exactly n edges over n nodes, exactly one such edge exists.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is the first cycle-closing edge the answer?

Answer. Because the input is a spanning tree plus one edge, so exactly one edge is redundant. An edge is redundant precisely when its endpoints are already connected when it is processed, so the first (and only) such edge is it.

Q2. How do you handle a self-loop [1,1]?

Answer. A self-loop has find(1) == find(1) trivially, so it is detected as redundant immediately and returned. That is correct, since removing it restores the tree.

Q3. What is the complexity?

Answer. $O(n\alpha(n))$ time with union-find (the inverse Ackermann factor is effectively constant), or $O(n \log n)$ worst case with only path compression and no union by rank. Space is $O(n)$.

0695 `Max Area Of Island` ?

ESSENCE & BOUNDS

Return the area of the largest group of 1s connected four-directionally. up to 300 x 300 with 0/1 cells, so 9e4 cells and a flood that can reach 9e4 stack entries – an iterative fill is the practical form.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

v1 v1 – see the kernel below.

Python (intuition kernel)

```python
rows, cols, mx = len(grid), len(grid[0]), 0
 if r<0 or r>=rows or c<0 or c>=cols or grid[r][c]!=1:
return 0
 grid[r][c]=0
 return 1+dfs(r+1,c)+dfs(r-1,c)+dfs(r,c+1)+dfs(r,c-1)
for r in range(rows):
 for c in range(cols):  mx=max(mx, dfs(r,c))
return mx

```

C++ (intuition kernel)

```cpp
int area(vector<vector<int>>& g, int r, int c) {
 int m = g.size(), n = g[0].size();
 if (r < 0 || c < 0 || r >= m || c >= n || g[r][c] !=
1)
 return 0;
 g[r][c] = 0; // mark on entry
 return 1 + area(g, r+1, c) + area(g, r-1, c)
 + area(g, r, c+1) + area(g, r, c-1);
}

```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

v2 v2 – see the kernel below.

Python (intuition kernel)

```python
rows, cols, mx = len(grid), len(grid[0]), 0
for r in range(rows):
 for c in range(cols):
 if grid[r][c]==1:
 area=0; q=deque([(r,c)]); grid[r][c]=0
 while q:
 cr,cc=q.popleft(); area+=1
 for dr,dc in [(1,0),(-1,0),(0,1),(0,-1)]:

```

C++ (intuition kernel)

```cpp
stack<int> st;
for each land cell (r,c) with g[r][c] == 1:
 st.push(r * n + c); g[r][c] = 0; area = 0;
 while (!st.empty()) {
 int p = st.top(); st.pop(); ++area;
 int y = p % n, x = p / n;
 for (int d = 0; d < 4; ++d) {
 int nx = x + DR[d], ny = y + DC[d];
 if (nx < 0 || ny < 0 || nx >= m || ny >= n)
continue;
 if (g[nx][ny] != 1) continue;
 g[nx][ny] = 0; st.push(nx * n + ny);
 }
 }
 best = max(best, area);

```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

VISUAL STATE TRACE

```
0 0 1 0
0 1 1 0
1 0 0 1
0 0 1 0 -> largest area 3
scan:  (0,2) seed, flood covers (0,2),(1,1),(1,2) -> area 3
(2,0) seed, area 1
(2,3) seed, area 1
(3,2) seed, area 1
max = 3
invariant:  each cell is marked the moment it is counted, so
the flood from a seed covers exactly that seed's component
and the area counter equals the component size.
```

LANGUAGE MECHANICS: PYTHON VS C++

Graph traversal is memory-latency bound: each neighbour fetch is a pointer chase into an adjacency structure. C++ `vector<vector<int>>` gives per-node contiguous vectors, so a node's neighbours are one cache line. CPython's `list` of lists is comparable in structure but each inner list is a separate object, and the loop around the deque adds dispatch cost. Union-Find shifts the trade-off entirely toward $O(1)$ space with no adjacency structure at all.

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- In-place mutation to 0 removes the visited grid; a `stack<pair<int,int>>` avoids recursion depth.
- A flat index stack with `r*n + c` arithmetic is faster than pairs.

- Flatten to a `bytearray` for the grid so marking is an int assignment, not a nested list store.
- Mutate `grid[i]` in place in a list of strs by converting rows to lists first.

INTERVIEW TRAPS & SANITY CHECKS

Incrementing the area counter before marking the cell, or marking after recursing. Both cause the same cell to be counted twice, inflating the area.

Sanity check. The flood from an unvisited land cell visits each cell of its component exactly once, because the cell is marked at the moment it is counted. Hence the accumulated counter equals the component's size and the maximum over all seeds is the answer.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is the flood $O(mn)$ overall rather than O((mn)^2)?

Answer. Each cell is marked once, and a marked cell is never entered again – the flood's entry test is the same as the outer scan's. So the total work across all floods is proportional to the number of cells, not to the number of seeds times the grid.

Q2. What does the mark-then-count ordering guarantee?

Answer. That the four-neighbour expansion never re-enters a cell already counted, so no cell contributes twice. This is the standard in-place visited pattern: write the mark before recursing, and restore only if you need the original grid.

Q3. How would you handle an all-zeros grid?

Answer. The outer scan finds no seeds and the answer stays at its initial value, which must be 0. Initialising best to 0 rather than 1 is what makes that case correct.

0994 Rotting Oranges ?

ESSENCE & BOUNDS

Return the number of minutes until no fresh orange remains, or -1 if some orange can never rot. up to 10 x 10, so 100 cells and at most 100 minutes. Multiple rotten oranges at t=0 are the defining case, and an all-rotten grid must return 0.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

v1 v1 – see the kernel below.

Python (intuition kernel)

```python
rows,cols=len(grid),len(grid[0]); q=deque(); fresh=0
for r in range(rows):
 for c in range(cols):
 if grid[r][c]==2:  q.append((r,c,0))
 elif grid[r][c]==1:  fresh+=1
time_val=0
while q:
 r,c,t=q.popleft(); time_val=max(time_val,t)

```

C++ (intuition kernel)

```cpp
int m = grid.size(), n = grid[0].size();
queue<pair<int,int>> q; int fresh = 0;
for (int r = 0; r < m; ++r) for (int c = 0; c < n;
++c)
 if (grid[r][c] == 2) q.push({r, c});
 else if (grid[r][c] == 1) ++fresh;
int minutes = 0;
while (!q.empty() && fresh > 0) {
 int size = q.size(); // one minute per level
 while (size--) {
 auto [r, c] = q.front(); q.pop();
 for (int d = 0; d < 4; ++d) {
 int nr = r + DR[d], nc = c + DC[d];
 if (nr < 0 || nc < 0 || nr >= m || nc >= n)
continue;
 if (grid[nr][nc] != 1) continue;
 grid[nr][nc] = 2; -fresh; q.push({nr, nc});
 }
 }
 ++minutes;
}
return fresh ?  -1 :  minutes;

```

Time ?	Space ?

Follows directly from the traversal count: ? time, ? space.

v2 v2 – see the kernel below.

Python (intuition kernel)

```python
rows,cols=len(grid),len(grid[0]); q=deque(); fresh=0
for r in range(rows):
 for c in range(cols):
 if grid[r][c]==2:  q.append((r,c))
 elif grid[r][c]==1:  fresh+=1
minutes=0
while q and fresh:
 minutes+=1

```

C++ (intuition kernel)

```cpp
queue<pair<int,int>> q; int fresh = 0, minutes = 0;
for each cell:  2 -> q.push, 1 -> ++fresh;
// repeatedly do a full BFS wave, counting waves until no
// new cell rots in that wave (or fresh == 0)

```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

VISUAL STATE TRACE

```
2 1 1 1 1
1 1 1 1 1
1 1 0 1 1
1 1 1 1 1
1 1 1 0 1
t=0:  two rots at (0,0) and (2,2)
t=1:  (0,1) (1,0) (1,2) (2,1) (2,3) (3,2)
t=2:  (0,2) (1,3) (1,1) (3,1) (3,3) (4,2)
...
a 0 at (4,4) surrounded by 1s -> reachable
a 0 at (4,4)?  all cells eventually rotted -> answer 11
BFS:
queue seeded with all 2s, count fresh = total - rotten
while queue and fresh > 0:
for _ in range(len(queue)):  process one minute
spread to 1 neighbours, mark 2, fresh-
minutes++
return minutes if fresh == 0 else -1
the ‘len(queue)‘ snapshot is what makes one BFS level = one
minute, exactly like level-order traversal.
```

LANGUAGE MECHANICS: PYTHON VS C++

Graph traversal is memory-latency bound: each neighbour fetch is a pointer chase into an adjacency structure. C++ `vector<vector<int>>` gives per-node contiguous vectors, so a node’s neighbours are one cache line. CPython’s `list` of lists is comparable in structure but each inner list is a separate object, and the loop around the deque adds dispatch cost. Union-Find shifts the trade-off entirely toward $O(1)$ space with no adjacency structure at all.

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `queue<pair<int,int>>` with `int levelSize = q.size()` per minute.
- An in-place 0->2 mutation removes the fresh counter, but the counter is clearer.
- `deque` with a per-level `for _ in range(len(q))` is the idiomatic minute-by-minute BFS.
- Early-exit as soon as fresh hits 0 avoids the final drained levels.

INTERVIEW TRAPS & SANITY CHECKS

Using a single counter for elapsed time incremented per node rather than per level. That counts nodes processed, not minutes, and produces a number larger than the true answer.

Sanity check. At the start of each level, the queue holds exactly the cells that rot at minute `t`, and `fresh` is the number of cells still unrotted. Processing one level rots every cell adjacent to a minute-t cell, so the level boundary corresponds exactly to one minute of propagation.

INTUITION CHECK — 3 QUESTIONS

Q1. Why does the level size snapshot equal one minute?

Answer. BFS expands in waves. All cells in the queue at the start of a level were reached at the same minute, and their newly discovered neighbours are exactly one minute later. Capturing the size before processing bounds one iteration to one time step.

Q2. When is the answer -1?

Answer. When `fresh > 0` after the queue empties, meaning some 0 cell is only reachable through other 0 cells and can therefore never be adjacent to a rotting orange. Empty cells also block propagation, which is what makes unreachability possible at all.

Q3. What is the space complexity?

Answer. $O(mn)$ for the queue in the worst case, plus the grid itself. The fresh counter is $O(1)$. Time is $O(mn)$ because each cell is enqueued at most once.

12. Advanced Graphs

#	Problem	Level	One-line intent
0269	**Alien Dictionary**	Hard	First-appearing difference between words gives an edge.
0332	**Reconstruct Itinerary**	Hard	Hierholzer: traverse edges, reverse at the end.
0743	**Network Delay Time**	Medium	Dijkstra with a min-heap, lazy deletion.
0778	**Swim in Rising Water**	Hard	Binary search t, then BFS on cells with height <= t.
0787	**Cheapest Flights Within K Stops**	?	Bellman-Ford with at most k+1 edge layers.
1584	**Min Cost To Connect All Points**	?	Prim or Kruskal on the complete edge set.

0269 Alien Dictionary Hard

ESSENCE & BOUNDS

Derive the ordering of a new alphabet from a list of words that are sorted under it, and return a valid order. 1 <= dictionary.size <= 100 with up to 2e4 words, and every word is lowercase with up to 20 characters. An invalid dictionary (words out of order, or a prefix after a longer word) must return an empty string.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

- Kahn's algorithm (best) 1. Build edges a→b + indegrees; queue zero-indegree letters. 2. Pop → append; decrement children; leftovers = cycle → "".

Python (intuition kernel)

```python
adj = {c: set() for w in words for c in w}
indeg = {c: 0 for c in adj}
for a, b in zip(words, words[1:]):
 for ca, cb in zip(a, b):
 if ca != cb:
 if cb not in adj[ca]:
 adj[ca].add(cb)
 indeg[cb] += 1

```

C++ (intuition kernel)

```cpp
for (size_t i = 0; i + 1 < words.size(); ++i) { const auto& a = words[i];
 const auto& b = words[i + 1];
 size_t j = 0;
 while (j < a.size() && j < b.size() && a[j] == b[j]) ++j;
 if (j == b.size() && j < a.size()) return ""; // "abc" > "ab"
 if (j < a.size() && j < b.size() && !adj[a[j] - 'a'][b[j] - 'a']) { adj[a[j] - 'a'][b[j] - 'a'] = true;
 ++indeg[b[j] - 'a'];
 }

```

Time ? **Space** ?

each node enters the queue once; each edge decremented once; hence the stated bounds.

- DFS with 3-color cycle detection White/gray/black states; gray revisit = cycle; append post-order and reverse. ——— DEEP DIVE - The two failure modes (1) "abc" before "ab": a prefix that should be first — returns "". (2) A cycle in constraints can never exist in a real alphabet. Any valid topological order is accepted (there may be many); Kahn gives one deterministically, DFS gives another. ——— TIME COMPLEXITY : $O(C)$ — total characters over all words. MEMORY COMPLEXITY: $O(1)$ — $\leq$ 26 letters. ==========

Python (intuition kernel)

```python
adj = {c:  set() for w in words for c in w}
for a, b in zip(words, words[1:]):
 for ca, cb in zip(a, b):
 if ca != cb:
 adj[ca].add(cb)
 break
 else:  if len(a) > len(b):
WHITE, GRAY, BLACK = 0, 1, 2

```

C++ (intuition kernel)

```cpp
for (size_t i = 0; i + 1 < words.size(); ++i) { const auto& a = words[i];
 const auto& b = words[i + 1];
 size_t j = 0;
 while (j < a.size() && j < b.size() && a[j] == b[j]) ++j;
 if (j == b.size() && j < a.size()) return "";
 if (j < a.size() && j < b.size()) adj_[a[j] - 'a'][b[j] - 'a'] = true;

```

Time O(C) — total characters over all words.

Space O(1) — $\leq$ 26 letters.

every node is visited once and returns $O(1)$ state; hence the stated bounds.

VISUAL STATE TRACE

```
words = ["hi","hj","az","kl"] -> [a,h,k,z]
compare consecutive pairs, FIRST differing char only:
"hi" vs "hj":  i<j -> edge i->j
"hj" vs "az":  h<a -> edge h->a
"az" vs "kl":  a<k -> edge a->k
graph:  i -> j
h -> a -> k
KAHN:
indeg:  j=1, i=0, a=1(from h), k=1, h=0
queue = [i, h]
pop i -> order=[i], indeg[j]=0 -> push j
pop h -> order=[i,h], indeg[a]=0 -> push a
pop j -> order=[i,h,j]
pop a -> push k -> pop k
ANSWER [i,h,j,a,k] (equivalent to a,h,k,z...  any valid)
WHY first difference only:  later characters are irrelevant
to lexicographic order - the first mismatch decides.
```

LANGUAGE MECHANICS: PYTHON VS C++

(1) "abc" before "ab": a prefix that should be first — returns "". (2) A cycle in constraints can never exist in a real alphabet. Any valid topological order is accepted (there may be many); Kahn gives one deterministically, DFS gives another.
———————————————————— TIME COMPLEXITY : $O(C)$ — total characters over all words.
==========

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `array<int,26>` adjacency with a boolean matrix is simplest and fine at this scale; bitmasks scale further.
- `std::priority_queue` variant gives the lexicographically smallest valid order.

- A `set` of pairs or a list-of-sets adjacency is the fastest readable Python form at n <= 100.
- `defaultdict(set)` handles multiple edges between the same pair without double-counting indegree.

INTERVIEW TRAPS & SANITY CHECKS

Adding an edge for EVERY differing position rather than just the first. That produces constraints the problem never implied and can make a valid dictionary look cyclic.

Sanity check. The edge set contains exactly the pairwise orderings implied by lexicographic comparison, and Kahn consumes nodes of indegree zero. If Kahn emits all 26 letters the collected edges were acyclic and the order is valid; if fewer are emitted, the derived graph has a cycle and the dictionary is inconsistent.

INTUITION CHECK — 3 QUESTIONS

Q1. Why only the first differing character matters?

Answer. Lexicographic comparison is decided by the earliest position where two words differ; everything after is irrelevant. So each adjacent pair contributes at most one constraint, and taking the minimum-cost set of edges is automatic.

Q2. How do you detect an invalid dictionary?

Answer. After Kahn, if fewer than 26 letters were emitted, the derived graph contains a cycle and no valid order exists, so return the empty string. Separately, if a word is a strict prefix of its successor (e.g. "abc" before "ab"), the dictionary is invalid regardless of any graph.

Q3. What is the complexity?

Answer. $O(W \cdot L + 26^2)$ time for the comparisons and the topological sort, where W is the word count and L the max length. Space is $O(26^2)$ for the adjacency structure.

0332 Reconstruct Itinerary Hard

ESSENCE & BOUNDS

Return the lexicographically smallest route that visits every flight exactly once. 1 <= tickets.length <= 1e4 with n <= 10 airports, so the answer is at most 10 cities long while there may be 1e4 edges. The Euler path must start at the departure airport.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Hierholzer, recursive DFS (BEST) Build adjacency: airport -> destinations, sorted DESCENDING so that pop() hands back the SMALLEST remaining destination. DFS: keep following smallest unused tickets; append the airport to the route only AFTER all its edges are consumed (post-order). Reverse at the end.

Python (intuition kernel)

```python
graph: dict[str, list[str]] = defaultdict(list)
for src, dst in sorted(tickets, reverse=True):
 graph[src].append(dst)
route: List[str] = []
 while graph[airport]:
 dfs(graph[airport].pop())
 route.append(airport)
dfs("JFK")

```

C++ (intuition kernel)

```cpp
while (!adj_[airport].empty()) { std::string next = adj_[airport].back();
 adj_[airport].pop_back(); // ticket consumed -- never reused
 dfs(next);

```

Time O(E log E) — sorting each adjacency list

Space O(E) — adjacency + route.

every node is visited once and returns $O(1)$ state; hence the stated bounds.

Hierholzer, iterative (no recursion) Identical walk with an explicit stack, so deep graphs cannot overflow Python's recursion limit. Dead ends are closed out by appending to the route and popping the stack.

Python (intuition kernel)

```python
graph: dict[str, list[str]] = defaultdict(list)
for src, dst in sorted(tickets, reverse=True):
 graph[src].append(dst)
stack: List[str] = ["JFK"]
route: List[str] = []
while stack:
 top = stack[-1]
 if graph[top]:

```

C++ (intuition kernel)

```cpp
 for (const auto& t : tickets) {
adj[t[0]].push_back(t[1]);

```

Time O(E log E)

Space O(E)

Follows directly from the traversal count: $O(ElogE)$ time, $O(E)$ space.

VISUAL STATE TRACE

```
JFK -> MUC -> LHR -> SFO -> SIN
JFK -> MUC -> LHR -> NRT -> SFO -> SIN (parallel MUC->LHR)
Hierholzer (post-order, reverse on return):
visit(JFK):
MUC ->
LHR ->
SFO -> SIN -> (no outgoing) push SIN
push SFO
NRT -> SFO (no remaining) -> push SFO
push NRT
push LHR
push MUC
push JFK
reverse the push order ->
JFK, MUC, NRT, SFO, SIN ...  (lexicographically smallest)
key:  use a min-HEAP or sorted adjacency so the smallest
destination is always taken first; post-order accumulation
+ reverse yields the lexicographically smallest Euler path.
A simple forward greedy (always take the smallest) can
dead-end and strand unused edges.
```

LANGUAGE MECHANICS: PYTHON VS C++

Naive DFS walks into a dead end only when every remaining edge is inside cycles; those cycles get appended to the route exactly when the dead end is reached. Reversing the post-order list plugs every cycle back into the walk at the vertex where it was entered — each edge is traversed exactly once, total $O(E)$. Python detail: the graph is a defaultdict (open addressing dict, linear probing h(k, i) = (hash(k) + i) mod m) — $O(1)$ lookups per airport. ———————————————————————— An Eulerian path exists iff every vertex with edges has balanced in/out degree except possibly one start (out = in + 1) and one end (in = out + 1). The problem guarantees this, so naive DFS "walks into a dead end" only when the remaining edges form cycles; those cycles are appended to the route at the moment the dead end is reached. Reversing the post-order list interleaves every cycle back into the walk at exactly the vertex where it was entered — this is the "edge-splitting" proof: each vertex's cycle is a sub-tour that plugs into the vertex's slot. Every edge is traversed exactly once, total $O(E)$. C++ note: std::map keeps airports sorted (red-black tree, $O(logV)$ per op) — that gives us lexicographic adjacency for free. ==========
/ #include <vector> // std::vector — ticket list / route storage #include <

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `priority_queue<string, vector<string>, greater<string>>` per node gives sorted outgoing edges directly.
- A `map<string, vector<string>>` plus sorting each adjacency once is simpler if the sort happens once.
- `defaultdict(list)` with each list sorted in reverse so `pop()` takes the smallest is faster than a heap at n <= 10.
- `heapq` per node is the direct analogue of the C++ min-heap and avoids the reverse-sort trick.

INTERVIEW TRAPS & SANITY CHECKS

Greedy forward traversal without post-order accumulation. Taking the lexicographically smallest next destination at every step can visit a dead end while unused edges remain, so the naive answer uses only a subset of the flights.

Sanity check. Hierholzer's recursion visits every edge exactly once. Because outgoing edges are consumed in increasing order and the result is built by appending on the way out of each call, the reversed sequence visits every edge once and is the lexicographically smallest such sequence.

INTUITION CHECK — 3 QUESTIONS

Q1. Why does plain greedy fail while Hierholzer works?

Answer. Greedy picks the smallest destination and moves on, which can strand remaining edges behind it – for example JFK->MUC->LHR then choosing SFO before NRT, dead-ending at SFO. Hierholzer explores the dead end first, records it, and splices it in correctly on the way out.

Q2. How do you guarantee the lexicographically smallest result?

Answer. Traverse outgoing edges in sorted order and accumulate the post-order, then reverse. Sorting makes the algorithm explore the smallest choices first, and the reversal produces the Euler path that respects that preference order.

Q3. What guarantees the route exists?

Answer. The problem guarantees it. In general you need the degree conditions: exactly one node with out-degree = in-degree + 1 (the start), one with in-degree = out-degree + 1 (the end), and every other node balanced, with all edges in one weakly connected component.

0743 Network Delay Time Medium

ESSENCE & BOUNDS

Return the shortest time for a signal to reach every node, or -1 if any node is unreachable. 1 <= n <= 1e4 with 1e4 edges of weight 1..100, so distances fit in int. Node 1 is the source, so n = 1 means the answer is 0.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Dijkstra with a min-heap (BEST) dist[u] = best-known distance from k. Repeat: extract the node with the smallest dist (min-heap, $O(logV)$), settle it, and relax each outgoing edge (u -> v, w): if dist[u] + w < dist[v], improve dist[v] and push it back. Answer = max(dist[1..n]).

Python (intuition kernel)

```python
g: dict[int, List[tuple[int, int]]] = {}
for u, v, w in times:
 g.setdefault(u, []).append((v, w))
INF = 10**9
dist = [INF] * (n + 1) # dist[u] = best-known distance from k
dist[k] = 0
pq: List[tuple[int, int]] = [(0, k)]
while pq:

```

C++ (intuition kernel)

```cpp
for (const auto& t : times) {
g[t[0]].emplace_back(t[1], t[2]);

```

Time O(E log V) — each edge relaxed once, heap ops

Space O(V + E) — dist + adjacency + heap.

one push (and at most one pop) per element; hence the stated bounds.

Bellman-Ford (no heap, V-1 relaxation rounds) Repeat V-1 rounds: scan every edge and relax dist[v] = min(...). After round i every node's dist is correct for paths using at most i edges; V-1 rounds cover every simple path. Early-exit when a round changes nothing.

Python (intuition kernel)

```python
INF = 10**9
dist = [INF] * (n + 1)
dist[k] = 0
for _ in range(n - 1):
 changed = False
 for u, v, w in times:
 if dist[u] != INF and dist[u] + w < dist[v]:
 dist[v] = dist[u] + w

```

C++ (intuition kernel)

```cpp
for (int round = 1; round <= n - 1; ++round) { bool changed = false;
 for (const auto& t : times) { int u = t[0], v = t[1], w = t[2];
 if (dist[u] != INF && dist[u] + w < dist[v]) { dist[v] = dist[u] + w;
 changed = true;
 }
 }
 if (!changed) break;

```

Time O(V-E)

Space O(V)

one push (and at most one pop) per element; hence the stated bounds.

VISUAL STATE TRACE

```
times = [[2,1,1],[2,3,1],[2,3,1]]
dist = [inf]*4 ; dist[0] = 0
heap = [(0,0)]
pop (0,0):  relax (2,1) -> (2,1,0) push
relax (1,1) -> (1,1,0) push
pop (1,1,0):  relax (3,2) -> (3,2,1) push
pop (2,1,0):  node 1 done
pop (3,2,1):  node 2 done
max(dist) = 2 (node 3 at distance 2)
LAZY DELETION: a heap may contain stale entries with an old
larger distance.  Skip any pop whose distance != dist[node].
Without that check you process nodes repeatedly and the
result is still correct but the time bound degrades.
```

LANGUAGE MECHANICS: PYTHON VS C++

Invariant: when a node u is POPPED, dist[u] is its true shortest distance. Induction on pop order: u is the un-settled node with the smallest dist; any alternative path k $\to$ \ldots{} $\to$ x $\to$ u must pass through an un-settled x with dist[x] $\geq$ dist[u], and all remaining edge weights are POSITIVE, so that path costs $\geq$ dist[x] + w $>$ dist[u]. No later relaxation can beat it. NEGATIVE EDGES BREAK THIS: a path k $\to$ \ldots{} $\to$ x $\to$ (negative edge) $\to$ u can cost LESS than dist[u] even when dist[x] $>$ dist[u] — the settled-set argument collapses. Bellman-Ford survives negatives by relaxing everything once per round, one edge at a time. ———————————————————— PROOF SKETCH (heap version): invariant — when a node u is popped, dist[u] is its TRUE shortest distance. Induction on pop order: u is the un-settled node with smallest dist; any alternative path k $\to$ ... $\to$ x $\to$ u must pass through some un-settled node x first (all settled nodes were already relaxed), and dist[x] >= dist[u], so that path costs >= dist[u] + (positive edges) $>$ dist[u]. Therefore no future relaxation can beat the current dist[u]. NEGATIVE EDGES BREAK IT: with a negative edge, a later path k $\to$ ... $\to$ x $\to$ (negative edge) $\to$ u can cost LESS than dist[u] even when dist[x] $>$ dist[u] — the settled-node argument collapses. Bellman-Ford handles negatives: relaxing every edge once per round propagates impro

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `priority_queue<pair<int,int>, vector<...>, greater<...>>` is the canonical form.
- Plain Dijkstra with an $O(V^2)$ scan is simpler and fast enough at 1e4 nodes if the graph is sparse.
- `heapq` with `(dist, node)` tuples and a `if d != dist[node]: continue` guard is the standard Python form.
- Push `(d + w, v)` directly instead of computing and pushing then comparing, which avoids one stale entry per improvement.

INTERVIEW TRAPS & SANITY CHECKS

Forgetting the stale-entry check, or using a visited array that marks on push instead of on pop. The latter is incorrect because a node's distance can improve after it is first discovered.

Sanity check. When a node is popped with a distance equal to its current `dist`, that distance is final. The proof is the standard contradiction: any shorter path would have passed through a node that should have been popped earlier with a smaller distance. Therefore each node is finalised at most once with a correct value.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is Dijkstra not applicable here?

Answer. It is applicable – this problem has non-negative weights, which is exactly Dijkstra's requirement. A DFS or BFS would be wrong: BFS minimises hop count, not total weight, and DFS has no ordering guarantee.

Q2. What does the stale-entry guard protect?

Answer. The priority queue can hold an old, larger distance for a node whose distance has since improved. Popping it would re-relax its neighbours with a worse value. Skipping pops where the heap distance differs from the current best keeps the correctness argument about finalisation intact.

Q3. How is unreachability detected?

Answer. After the loop, any node whose distance is still infinity was never reached. Return -1 if any remains; otherwise return the maximum distance, which is the time until the last node receives the signal.

0778 Swim in Rising Water Hard

ESSENCE & BOUNDS

Return the least t such that a path exists from the top-left to the bottom-right using only cells of height at most t. 0 <= n <= 1e3 so the grid can hold 1e6 cells, and 0 <= height[i][j] <= n-1. The answer is bounded by max of all heights, which gives the binary-search range.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Binary search over t + BFS reachability (BEST) lo = grid[0][0] (start cell must be flooded), hi = max elevation. While lo < hi: probe mid; BFS from (0,0) visiting only cells with elevation <= mid; if the target is reached, hi = mid (feasible, try lower), else lo = mid + 1 (need higher water).

Python (intuition kernel)

```python
n = len(grid)
lo, hi = grid[0][0], max(max(row) for row in grid)
while lo < hi:
 mid = (lo + hi) // 2
 if self._reachable(grid, mid):
 hi = mid # feasible $\{}to$ try a LOWER water level
 else:
 lo = mid + 1 # infeasible $\{}to$ water must rise further

```

C++ (intuition kernel)

```cpp
while (!q.empty()) { const auto [r, c] = q.front();
 q.pop();
 if (r == n - 1 && c == n - 1) return true; // target reached
 for (int i = 0; i < 4; ++i) { const int nr = r + DR[i];
 const int nc = c + DC[i];
 if (nr < 0 || nr >= n || nc < 0 || nc >= n) continue;
 if (seen[nr][nc] || grid[nr][nc] > level) continue;
 seen[nr][nc] = true;

```

Time O(n^2 log H) — log H probes, each BFS is O(n^2)

Space O(n^2) — visited matrix per probe.

the interval halves each iteration: $O(logn)$ probes; hence the stated bounds.

Binary search over t + DSU connectivity Same binary search, but reachability uses union-find: for the probed level, union every underwater cell with its underwater neighbours; (0,0) and (n-1,n-1) are connected iff a path exists.

Python (intuition kernel)

```python
n = len(grid)
lo, hi = grid[0][0], max(max(row) for row in grid)
while lo < hi:
 mid = (lo + hi) // 2
 if self._connected(grid, mid):
 hi = mid
 else: lo = mid + 1
return lo

```

C++ (intuition kernel)

```cpp
for (int r = 0; r < n; ++r) { for (int c = 0; c < n; ++c) {
 if (grid[r][c] > level) continue; // dry cell -- skip
 const int id = r * n + c;
 if (r + 1 < n && grid[r + 1][c] <= level) dsu.unite(id, id + n);
 if (c + 1 < n && grid[r][c + 1] <= level) dsu.unite(id, id + 1);
 }

```

Time O(n^2 log H)	**Space** O(n^2)

the interval halves each iteration: $O(logn)$ probes; hence the stated bounds.

VISUAL STATE TRACE

```
grid:  [[0,2],[1,3]] -> answer 3
feasibility(t) monotone:
t=0:  only (0,0) reachable, (1,1) not -> False
t=1:  (0,0),(1,0) reachable -> False
t=2:  (0,0),(0,1),(1,0) -> (1,1) is height 3, blocked
t=3:  all cells -> True
binary search on [0, maxH] = [0,3]:
mid=1 -> False -> lo=2
mid=2 -> False -> lo=3
ANSWER 3
ALTERNATIVE: minimax Dijkstra - dist[r][c] = the minimum
possible path maximum; relax with max(dist[u], h[v]).
One pass, O(nm log nm), no repeated BFS.
```

LANGUAGE MECHANICS: PYTHON VS C++

P(t) = "path exists at water level t" is monotone because water never goes DOWN: reachable at t ⇒ reachable at t+1. Any monotone predicate on a sorted universe answers with binary search in $O(logH)$ probes — each probe a graph reachability run. The DSU variant unions flooded cells instead of visiting them one by one; connectivity then costs two find() calls. BFS proves an actual path, DSU proves connectivity — both answer the same question. ———————— The predicate P(t) = "path exists at water level t" is monotone because water never goes DOWN: if the set of cells reachable at level t contains the target, it also contains it at level t+1. Any monotone predicate on a sorted universe answers with binary search in $O(logH)$ probes — here each probe is a graph reachability run. DSU variant: instead of visiting cells one by one, we UNION underwater cells with their underwater neighbours; connectivity then costs two finds. BFS probes the actual path, DSU probes only connectivity — both answer the same question. ========== / #include <vector> // std::vector — grid + visited matrices #include <queue> // std::queue — BFS frontier #include <utility> // std::pair — (row, col) coordinates #include <algorithm> // std::max — scanning the grid for hi #include <chrono> // std:

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- Binary search plus BFS is $O(n^2 \log n)$ – 20 BFS passes over 1e6 cells, which is the practical cost.
- Minimax Dijkstra with a priority queue is a single pass and often faster in wall-clock terms despite the log factor.
- `deque` BFS with a `bytearray` visited grid avoids a 2-D list of bools.
- An in-place visited marking with a sentinel height is possible but destroys the grid, which you must not modify.

INTERVIEW TRAPS & SANITY CHECKS

Using a visited array that is not reset between binary-search probes, which makes later probes return stale True results and collapses the search to the first candidate.

Sanity check. `can_reach(t)` is monotone: if the target is reachable with threshold t, it is reachable with any larger threshold because every path that works at t also works at higher thresholds. That monotonicity is what makes binary search valid and guarantees termination with the minimum feasible t.

INTUITION CHECK — 3 QUESTIONS

Q1. What is the search space being binary-searched?

Answer. The grid heights, i.e. 0 to max(height) – up to 1e3 here. It is NOT the path space, which would be exponential. The insight is that feasibility is monotone in t, so a monotone predicate over a small integer range supports binary search.

Q2. What are the time and space bounds?

Answer. Binary search performs $O(\log n)$ probes, each a BFS over n^2 cells, so $O(n^2 \log n)$ time and $O(n^2)$ space for the visited grid. Minimax Dijkstra is $O(n^2 \log n^2)$ with a heap.

Q3. Why is minimax Dijkstra the more elegant formulation?

Answer. It removes the outer binary search entirely. The relaxation `dist[v] = min(dist[v], max(dist[u], h[v]))` captures the bottleneck semantics directly, so one search yields the answer. The downside is a larger constant per node from the heap.

0787 Cheapest Flights Within K Stops ?

ESSENCE & BOUNDS

Return the cheapest price from src to dst using at most k intermediate stops, that is at most k+1 edges. 1 <= n <= 1e3 with up to 1e4 flights, and 0 <= src, dst, k < n. Prices are non-negative, so k = 0 means a direct flight.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

v1 v1 – see the kernel below.

Python (intuition kernel)

```python
prices=[float("inf")]*n; prices[src]=0
for _ in range(k+1):
 tmp=prices[:]
 for u,v,w in flights:
 if prices[u]+w<tmp[v]:  tmp[v]=prices[u]+w
 prices=tmp
return prices[dst] if prices[dst]!=float("inf") else -1

```

C++ (intuition kernel)

```cpp
const int INF = 1e9;
vector<int> dist(n, INF); dist[src] = 0;
for (int layer = 0; layer <= k; ++layer) {
 vector<int> prev = dist; // SNAPSHOT the layer
 for (auto& e :  flights) {
 if (prev[e[0]] + e[1] < dist[e[2]])
 dist[e[2]] = prev[e[0]] + e[1];
 }
}
return dist[dst] == INF ? -1 :  dist[dst];

```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

v2 v2 – see the kernel below.

Python (intuition kernel)

```python
from collections import defaultdict
graph=defaultdict(list)
for u,v,w in flights:  graph[u].append((v,w))
heap=[(0,src,0)] # cost, node, stops
visited={}
while heap:
 cost,u,stops=heapq.heappop(heap)
 if u==dst:  return cost

```

C++ (intuition kernel)

```cpp
const int INF = 1e9;
vector<vector<int>> d(n, vector<int>(k + 2, INF));
d[src][0] = 0;
for (int i = 0; i <= k; ++i)
 for (auto& e :  flights)
 for (int j = 0; j <= i; ++j)
 d[e[2]][i+1] = min(d[e[2]][i+1], d[e[0]][j] + e[1]);
return *min_element(d[dst].begin(), d[dst].end()) == INF
 ? -1 :  *min_element(d[dst].begin(), d[dst].end());

```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

VISUAL STATE TRACE

```
n=3, k=1, flights=[[0,1,100],[1,2,100],[2,0,100],[1,2,100]]
src=0 dst=2 -> answer 200
Bellman-Ford with LAYERS:
LAYER 0 (0 edges):  dist = [0, inf, inf] only src
LAYER 1 (1 edge):  from 0:  dist[1]=100
LAYER 2 (2 edges):  from 1:  dist[2]=200 <- k=1 means
2 edges -> answer 200
n=3, k=0, same flights -> layer 1 only, dist[2]=inf -> -1
THE BUG: mutating dist IN PLACE lets one round use a
distance improved in the same round, so a 2-layer answer
leaks into layer 1.  Fix:  snapshot the previous layer.
prev = dist (copy)
for u,v,w in flights:  if prev[u] + w < dist[v]:  dist[v] = prev[u]+w
layers needed = k + 1, because k stops means k+1 flights.
```

LANGUAGE MECHANICS: PYTHON VS C++

A trip with k intermediate stops uses k+1 flight legs. Bellman- Ford's i-th round computes optimal prices for paths of AT MOST i edges, because each round extends every previously known path by exactly one more edge. The SNAPSHOT copy is the subtle part: if we relaxed in-place, one round could chain two edges (u->v then v->w) and silently count two legs as one — exceeding the budget. So: k stops k+1 edges k+1 rounds. For the same reason, plain Dijkstra fails here: it settles each node once with the cheapest price, but that price may spend the whole hop budget, blocking a slightly pricier but fewer-hop route to the destination.
==========
/ #include <vector> // std::vector — graph / dist arrays #include <queue> // std::priority_queue — the min-heap #include <utility> // std::pair — nested heap entries #include <functional> // std::greater — MIN-heap ordering #include <chrono> // std::chrono::steady_clock — benchmark timing #include <iostream> // std::cout — benchmark output #include <sys/resource.h> // getrusage — peak RAM measurement namespace { const int INF = 1e9; // sentinel "unreachable" price }

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- A flat `int` vector of size n with a `prev` copy per layer is the standard C++ form.
- The one-array Bellman-Ford with early exit works for unconstrained shortest path but NOT here, because the layer count is a hard constraint.
- A list copy per layer (`prev = dist[:]`) is the clearest way to get the snapshot semantics.
- Keeping only the previous layer's array keeps space at $O(n)$.

INTERVIEW TRAPS & SANITY CHECKS

Updating `dist` in place inside the layer loop. That allows a path using more than one edge in the same iteration, so a k = 0 query can return a two-flight answer.

Sanity check. After layer L, `dist[v]` is the minimum cost of a path from src to v using at most L edges. Base case L = 0 is `dist[src] = 0` and infinity elsewhere. Each layer relaxes only from the previous layer's values, so a path using L edges is built from a path using L-1 edges plus one flight, preserving the bound exactly.

INTUITION CHECK — 3 QUESTIONS

Q1. Why k stops means k+1 edges?

Answer. With 0 stops you take one flight: src -> dst directly. Each additional stop allows one more intermediate airport, and one more edge. So the edge budget is k+1, which is the number of relaxation layers to run.

Q2. Why is the in-place update wrong?

Answer. Because a vertex improved during layer L can then be used as a source later in the SAME layer, producing a path longer than the layer budget. Snapshotting `prev = dist` before each layer and reading only from `prev` makes the layer count a genuine bound.

Q3. How does this relate to Dijkstra?

Answer. Both find shortest paths with non-negative weights, but Dijkstra has no hop constraint – it optimises over all path lengths. Here the constraint is on the number of edges, so the layer-indexed DP is the natural formulation and Dijkstra would answer the wrong question.

1584 Min Cost To Connect All Points ?

ESSENCE & BOUNDS

Return the minimum total weight of edges connecting all points, using Manhattan distance. 1 <= n <= 1e3 with coordinates in [-1e6, 1e6], so a complete graph has about 5e5 edges. n = 1 returns 0 since no edges are needed.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

v1 v1 – see the kernel below.

Python (intuition kernel)

```python
n=len(points); visited=set(); total=0
heap=[(0,0)] # (cost, node)
while len(visited)<n:
 cost,u=heapq.heappop(heap)
 if u in visited:  continue
 visited.add(u); total+=cost
 for v in range(n):
 if v not in visited:

```

C++ (intuition kernel)

```cpp
while (!pq.empty()) { const auto [cost, u] =
pq.top();
 pq.pop();
 if (inMST[u]) continue;
 inMST[u] = true; // settle u:  its cheapest cut edge
is
 total += cost; // final (cut property -- see
header).
 for (int v = 0; v < n; ++v) { if (inMST[v])
continue;
 const int d = manhattan(points[u], points[v]);
 if (d < minCost[v]) { minCost[v] = d;

```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

v2 v2 – see the kernel below.

Python (intuition kernel)

```python
n=len(points)
edges=[]
for i in range(n):
 for j in range(i+1,n):

d=abs(points[i][0]-points[j][0])+abs(points[i][1]-points[j][1])
 edges.append((d,i,j))
edges.sort(); parent=list(range(n)); total=0; used=0
 while parent[x]!=x:  parent[x]=parent[parent[x]];
x=parent[x]

```

C++ (intuition kernel)

```cpp
for (int i = 0; i < n; ++i) { for (int j = i + 1; j <
n; ++j) {
 edges.emplace_back(manhattan(points[i], points[j]),
i, j);
 }
```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

VISUAL STATE TRACE

```
points = [[0,0],[2,0]] -> dist 2
points = [[0,0],[1,1]] -> dist 2
points = [[0,0],[1,0],[3,0]]
edges:  (0,1)=1 (1,2)=2 (0,2)=3
MST: 1 + 2 = 3
KRUSKAL:
sort all edges by weight:  1, 2, 3
take 1 (0-1), take 2 (1-2), take 3 -> both endpoints
already joined -> skip
total = 3
PRIM (min-heap):
visited = {0}; heap = {(d, node) for neighbours of 0}
pop smallest edge to an unvisited node, add weight,
mark visited, push its edges
stop after n-1 edges taken
n <= 1e3 so building all n(n-1)/2 edges and sorting is
acceptable; for larger n you would need a spatial index.
```

LANGUAGE MECHANICS: PYTHON VS C++

PRIM (cut property): take any cut separating the growing tree from the rest; the CHEAPEST edge across that cut is in some MST, so greedily adding the cheapest crossing edge can never go wrong. The heap makes "cheapest crossing edge" an $O(logn)$ pop. KRUSKAL (cycle property): the cheapest edge not creating a cycle is always safe to keep; union-find detects "would create a cycle" by checking whether the endpoints already share a root. Both are correct because every edge crossing a cut is a candidate — the complete graph simply makes n^2 candidates explicit. When to use which: Prim shines on DENSE graphs (like this one, E ≈ n^2) since it only scans edges of the frontier; Kruskal shines on SPARSE graphs (E ≈ n) and when edges are given explicitly.
==========
/ #include <vector> // std::vector — points / edges / DSU internals #include <queue> // std::priority_queue — the min-heap #include <utility> // std::pair — (cost, node) heap entries #include <functional> // std::greater — MIN-heap ordering #include <algorithm> // std::sort — Kruskal's edge ordering #include <tuple> // std::tuple — (cost, u, v) edge records #include <chrono> // std::chrono::steady_clock — benchmark timing #include <iostream> // std::cout — benchmark output #include <sys/resource.h> // getrusage — peak RAM measurement namespace { const int INF = 1e9; //

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `std::sort` on 5e5 edges is $O(E \log E)$ and dominates; Kruskal is simpler than Prim here.
- `priority_queue` Prim avoids materialising the edge list but does the same work per vertex.
- Prim with a heap is usually faster in Python than Kruskal, because building and sorting 5e5 tuples is expensive while the heap has only n entries.
- Avoid building the full edge list; compute distances lazily inside the Prim loop.

INTERVIEW TRAPS & SANITY CHECKS

Counting an edge when both endpoints are already in the same component. In Kruskal that is caught by the union-find check; in Prim you must exclude already-visited vertices when pushing candidate edges.

Sanity check. Kruskal maintains a forest with no cycles, and by the cut property the cheapest edge crossing any cut is safe to take. Prim maintains a single growing tree and always takes the cheapest edge leaving it. Both terminate after exactly n-1 edges, which by the cut property is a minimum spanning tree.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is a minimum spanning tree the right model?

Answer. Any connected solution on n vertices needs at least n-1 edges, and a spanning tree is exactly a connected acyclic solution with n-1 edges. Since all costs are non-negative, an optimal solution never uses extra edges, so the MST weight is the answer.

Q2. Kruskal or Prim for these constraints?

Answer. Both work. Kruskal is $O(E \log E) = O(n^2 \log n)$ with an explicit edge list, which at n = 1e3 is about 5e5 edges – feasible. Prim with a lazy heap is $O(E \log n)$ and avoids materialising the list, so it is often preferable in Python.

Q3. What is the space complexity, and why does it matter here?

Answer. Kruskal needs $O(E) = O(n^2)$ space for the edge list – 5e5 tuples at n = 1e3, which is the binding constraint in Python. Prim's heap holds at most $O(E)$ entries in the lazy version but typically far fewer, and it never builds the full list.

13. 1-D Dynamic Programming

#	Problem	Level	One-line intent
0005	**Longest Palindromic Substring**	Medium	Expand around every odd and even centre.
0070	**Climbing Stairs**	Easy	Ways to reach step n: fib(n+1).
0091	**Decode Ways**	Medium	Count partitions into 1-2 digit valid codes.
0139	**Word Break**	Medium	Can the prefix ending at i be broken into dictionary words?
0152	**Maximum Product Subarray**	Medium	Track max and min ending here; a negative flips them.
0198	**House Robber**	Medium	rob(i) = max(rob(i-1), rob(i-2) + nums[i]).
0213	**House Robber II**	Medium	Split the circle into two linear cases and take the max.
0300	**Longest Increasing Subsequence**	Medium	Tails array plus bisect_left gives $O(nlogn)$.
0322	**Coin Change**	Medium	Unbounded knapsack: dp[x] = min(dp[x-c] + 1).
0416	**Partition Equal Subset Sum**	?	Target is half the total; then it is a subset-sum question.
0647	**Palindromic Substrings**	?	Count singles always, plus centre expansions of length >= 2.
0746	**Min Cost Climbing Stairs**	?	Same recurrence as house robber, with cost instead of value.
1143	**Longest Common Subsequence**	Medium	dp[i][j] on suffixes; take a match, else skip one side.

0005 Longest Palindromic Substring Medium

ESSENCE & BOUNDS

Return the longest contiguous substring of s that is a palindrome. 1 <= s.length <= 1000, so the answer always exists and an $O(n^2)$ solution passes. s = "babad" has two length-3 answers, so any is accepted.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Expand around centers (BEST) For every center c ∈ [0, 2n): expand while the two mirrored chars match; track the widest window seen.

Python (intuition kernel)

```python
best_l, best_r = 0, 0
n = len(s)
for c in range(2 * n - 1):
    l, r = c // 2, (c + 1) // 2
    while l >= 0 and r < n and s[l] == s[r]:
        if r - l > best_r - best_l:  # strictly wider
$\{}to$ new best
            best_l, best_r = l, r
        l -= 1 # reach one char further left

```

C++ (intuition kernel)

```cpp
for (int c = 0; c < 2 * n - 1; ++c) { int l = c / 2;
    int r = (c + 1) / 2;
    while (l >= 0 && r < n && s[l] == s[r]) {
        if (r - l > bestR - bestL) { // strictly wider
$\{}to$ new best
            bestL = l;
            bestR = r;
        }
        --l; // reach one char further left

```

Time O(n^2) — 2n-1 centers $\times$ O(n) expansion each.

Space O(1) — a few index variables.

Follows directly from the traversal count: $O(n^2)$ — 2n-1 centers × $O(n)$ expansion each. time, $O(1)$ — a few index variables. space.

DP boolean table dp[i][j] = s[i..j] is a palindrome. Recurrence : dp[i][j] = (s[i] == s[j]) and (j - i <= 2 or dp[i+1][j-1]) State space : (i, j) with i <= j → n(n+1)/2 ≈ $O(n^2)$ states mapped onto an n×n table. Fill by increasing length so the inner cell dp[i+1][j-1] is already computed. Space optimization : row i reads only row i+1 → ONE rolling row of booleans collapses the table to $O(n)$ space.

Python (intuition kernel)

```python
n = len(s)
if n == 0:
 return "" # empty-string edge case
dp:  List[List[bool]] = [[False] * n for _ in range(n)]
best_l, best_r = 0, 0
for length in range(1, n + 1):
 for i in range(n - length + 1):
 j = i + length - 1

```

C++ (intuition kernel)

```cpp
if (n == 0) { return ""; // empty string edge case

```

Time O(n^2) **Space** O(n^2) $\to$ O(n) rolling

Follows directly from the traversal count: $O(n^2)$ time, $O(n^2) \to O(n)$ rolling space.

VISUAL STATE TRACE

```
s = "babad" longest palindrome "bab" or "aba"
ODD centres (2n-1 of them):  c = 0, 0.5, 1, 1.5, ...
c=0.5 -> "b"?  expand:  s[-1] invalid -> len 1
c=1 -> "a", expand s[0]==s[2]?  b==b YES -> len 3 BEST
c=1.5 -> "a" ...  len 1
c=2 -> "b", expand a==a YES, b vs d NO -> len 3
EVEN centres (n-1 of them):  between i and i+1
c=1.5 -> s[1]!=s[2]?  a vs b -> no
c=3 -> s[3]==s[4]?  a vs d no
expand(l, r):
while l >= 0 and r < n and s[l] == s[r]:  l-, r++
return r - l - 1 # length after the last successful step
invariant:  expand returns the largest palindrome centred at
(l, r) - it stops exactly at the first mismatch, so the
window just inside is maximal.
```

LANGUAGE MECHANICS: PYTHON VS C++

Both approaches must read $O(n^2)$ substrings to PROVE maximality: the answer must be compared against every smaller candidate. The DP recurrence maps 1:1 onto the table: cell (i, j) reads the diagonal neighbour (i+1, j-1) — a strictly SMALLER window already solved in the previous length iteration. Overlapping subproblems are exactly the palindromes shared by many windows; each is stored once in dp, never recomputed. Center expansion avoids the table entirely because each palindrome is found at its OWN center — the table's cells for non-matching pairs never need to exist. Real life : DNA/RNA palindromic motifs (regulatory regions), text mirror detection, string-toolbox primitives.
==========
/ #include <iostream> #include <vector> #include <string> #include <unordered_map> #include <unordered_set> #include <chrono> #include <sys/resource.h>

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `std::string_view` slices avoid copying during comparison.
- Manacher's algorithm gives $O(n)$ but is rarely worth the complexity at n = 1000.
- `s[l] == s[r]` on one-character strings is cheap thanks to CPython's character interning.
- Duplicating the string with separators turns odd and even into one loop, which is the Manacher-style simplification.

INTERVIEW TRAPS & SANITY CHECKS

Checking only odd-length palindromes by centring on each character. Even-length palindromes like "abba" have no character centre, so they are missed entirely.

Sanity check. `expand(l, r)` returns the maximum palindrome length for the given centre. Since any palindrome has a centre at either a character (odd) or a gap (even), checking all 2n-1 centres covers every candidate, and the maximum over them is the answer.

INTUITION CHECK — 3 QUESTIONS

Q1. Why are there 2n-1 centres?

Answer. n characters give n odd-length centres, and the n-1 gaps between adjacent characters give n-1 even-length centres. Total 2n-1, and every palindrome has exactly one of them.

Q2. What is the complexity, and why is the naive $O(n^3)$ version avoidable?

Answer. $O(n^2)$ here because the expansion from each centre is not restarted from scratch inside another. The $O(n^3)$ version recomputes `s[l..r] == reversed` for every interval; expanding outward from each centre reuses work, giving at most n expansions per centre.

Q3. When is Manacher's algorithm warranted?

Answer. When n is in the millions. It reuses palindromes found to the left to skip re-expansion, giving $O(n)$ time and $O(n)$ space. At n = 1000 that sophistication costs more in explanation than it saves in runtime.

0070 Climbing Stairs Easy

ESSENCE & BOUNDS

Count the number of distinct ways to climb n stairs taking 1 or 2 steps at a time. 1 <= n <= 45, and the answer fits in a signed 32-bit int (f(46) = 1836311903). n = 1 gives 1 and n = 2 gives 2.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Bottom-up tabulation with rolling variables (BEST) dp[i] = distinct ways to reach step i. Recurrence : dp[i] = dp[i-1] + dp[i-2] dp[0] = dp[1] = 1 State space : i ∈ [0, n] → n+1 states mapped onto array dp[n+1], filled left→right. Cell i reads only i-1 and i-2. Space optimization : only the last TWO values are ever needed → two rolling variables replace the table → $O(1)$.

Python (intuition kernel)

```python
prev2, prev1 = 1, 1 # dp[0] = dp[1] = 1
for i in range(2, n + 1):
 cur = prev1 + prev2 # dp[i] = dp[i-1] + dp[i-2]
 prev2, prev1 = prev1, cur # slide the window right
return prev1 # dp[n]

```

C++ (intuition kernel)

```cpp
for (int i = 2; i <= n; ++i) {
 const int cur = prev1 + prev2; // dp[i] = dp[i-1] + dp[i-2]
 prev2 = prev1; // slide the window right
 prev1 = cur;

```

Time O(n) **Space** O(1)

each state is computed once from already-known states; hence the stated bounds.

Top-down memoized recursion f(i) = f(i-1) + f(i-2), memo[i] caches every solved state so the naive exponential recursion collapses to $O(n)$ work. Trade-off : easier to derive; each call costs a stack frame + memo lookup; very deep n risks Python's recursion limit.

Python (intuition kernel)

```python
memo: List[int] = [-1] * (n + 1) # -1 = state not solved yet
 if i <= 1:
 return 1
 if memo[i] != -1:
 return memo[i]
 memo[i] = f(i - 1) + f(i - 2)
 return memo[i]
return f(n)

```

C++ (intuition kernel)

```cpp
if (i <= 1) {
 return 1; // base case: dp[0] = dp[1] = 1

```

Time O(n) **Space** O(n) (memo + stack)

each (i, j) state is solved once and cached; hence the stated bounds.

VISUAL STATE TRACE

```
n = 5 -> 8
dp[0] = 1 (one way to be at the bottom:  do nothing)
dp[1] = 1 (1)
dp[2] = 2 (1+1, 2)
dp[3] = 3 (111, 12, 21)
dp[4] = 5 (1111,112,121,211,22)
dp[5] = 8
recurrence:  dp[i] = dp[i-1] + dp[i-2]
because the last move is either 1 (from i-1) or 2 (from i-2)
and those cases are DISJOINT and EXHAUSTIVE.
rolling:  prev2, prev1 = 1, 1
for i in 2..n:  prev2, prev1 = prev1, prev1 + prev2
invariant:  at step i, prev1 == dp[i-1] and prev2 == dp[i-2].
```

LANGUAGE MECHANICS: PYTHON VS C++

Naive recursion for f(n) recomputes f(3) $O(fib)$ times — exponential. The two DP strategies are the two ways to kill that overlap: tabulation solves states in dependency order (left→right, each once), memoization solves on demand but never twice. The recurrence maps 1:1 onto the array: dp[i] looks BACKWARD only (i-1, i-2) — that one-directional dependency is exactly what makes rolling variables safe: when computing dp[i], dp[i-2] is no longer needed after the slide, so old cells can be forgotten instead of stored. Real life : the Fibonacci model (rabbit populations, 1202), branch counting in trees, grid path counting, tiling counts.
==========
/ #include <iostream> #include <vector> #include <string> #include <unordered_map> #include <unordered_set> #include <chrono> #include <sys/resource.h>

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `int` suffices up to n = 45; use `long long` if you extend the range.
- `unsigned` avoids any signed-overflow discussion; the boundary is well inside 32 bits.

- `def fib(n): a, b = 1, 1; for _ in range(n): a, b = b, a + b; return a` is a one-liner.
- A full `dp` list is unnecessary: the recurrence only reads the previous two entries.

INTERVIEW TRAPS & SANITY CHECKS

Initialising `dp[0] = 0`. Then dp[2] = dp[1] + dp[0] = 1, which is wrong. The empty climb has exactly one way, so dp[0] must be 1.

Sanity check. After iteration i, `prev1` equals the number of ways to reach step i and `prev2` equals the number for step i-1. The update computes dp[i+1] = dp[i] + dp[i-1], preserving the invariant. Since the last move is uniquely 1 or 2, no path is double-counted.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is this Fibonacci?

Answer. The recurrence dp[i] = dp[i-1] + dp[i-2] with base values 1, 1 is exactly the Fibonacci recurrence, so dp[n] = F(n+1). That is why the answer at n = 45 is 1836311903, which is F(46) and the largest Fibonacci number below 2^31.

Q2. Why is the two-case split exhaustive?

Answer. Any way to climb to step i must end with a 1-step from i-1 or a 2-step from i-2. There is no other final move, so every valid path lands in exactly one of the two sets – and the sets are disjoint because their final moves differ.

Q3. Why is space $O(1)$ here?

Answer. Because dp[i] depends only on dp[i-1] and dp[i-2]. Once those two values are held, everything before is irrelevant, so no array is needed. This is the general rule: rolling arrays reduce space to the dependency width.

0091 Decode Ways Medium

ESSENCE & BOUNDS

Count the number of ways to decode a digit string using codes 1-26, where 0 has no standalone meaning. 1 <= s.length <= 100, so the answer can reach about 2^100 and modulo 1e9+7 is required. Leading zeros and a trailing zero both need special handling.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Bottom-up with rolling variables (BEST) dp[i] = decodings of prefix s[0..i) (i = consumed characters). Recurrence : one = dp[i-1] if s[i-1] != '0' else 0 (code '1'..'9') two = dp[i-2] if s[i-2..i-1] in [10..26] else 0 dp[i] = one + two State space : $i \in [0, n] \to$ n+1 states on array dp[n+1], filled left→right; dp[i] reads only i-1, i-2 $\to$ roll the table down to two variables $\to O(1)$ space.

Python (intuition kernel)

```python
n = len(s)
prev2, prev1 = 1, 1 # dp[0] = 1 (empty prefix), dp[1] set below
for i in range(1, n + 1):
 cur = 0
 if s[i - 1] != '0':
 cur += prev1 # extend every dp[i-1] decoding by s[i-1]
 if i >= 2 and s[i - 2] == '1':
 cur += prev2 # 10..19

```

C++ (intuition kernel)

```cpp
for (int i = 1; i <= n; ++i) { int cur = 0;
 if (s[i - 1] != '0') {
 cur += prev1; // extend every dp[i-1] decoding by s[i-1]
 }
 if (i >= 2 && s[i - 2] == '1') {
 cur += prev2; // "10".."19"
 } else if (i >= 2 && s[i - 2] == '2' && s[i - 1] <= '6') {
 cur += prev2; // "20".."26"

```

Time O(n) **Space** O(1)

each state is computed once from already-known states; hence the stated bounds.

Top-down memoized recursion f(i) = decodings of suffix s[i:]. Two branches: take 1 char (if valid) and take 2 chars (if 10..26). memo[i] caches each suffix. Trade-off : reads the string right-to-left — mirrors how a human would backtrack; recursion depth n for long strings.

Python (intuition kernel)

```python
n = len(s)
memo: List[int] = [-1] * (n + 1) # -1 = suffix not solved yet
if i == n:
 return 1 # consumed everything $\{}to$ 1 way
if s[i] == '0':
 return 0 # '0' cannot START a code
if memo[i] != -1:
 return memo[i] # overlapping subproblem $\{}to$ reuse

```

C++ (intuition kernel)

```cpp
if (i == n) {
 return 1; // consumed everything $\{}to$ exactly one way

```

Time O(n) **Space** O(n)

each (i, j) state is solved once and cached; hence the stated bounds.

VISUAL STATE TRACE

```
s = "226" -> 3
dp[0] = 1
i=1 ch=2:  single 2 valid -> dp[1] += dp[0] = 1
i=2 ch=2:  single valid dp[2]+=dp[1]; pair "22" valid
dp[2] += dp[0] -> 1 + 1 = 2
i=3 ch=6:  single dp[3]+=dp[2]; pair "26" valid
dp[3] += dp[1] -> 2 + 1 = 3
the two transitions per position:
single:  1 <= s[i-1] <= '9' -> dp[i] += dp[i-1]
pair:  10 <= s[i-1..i] <= 26 -> dp[i] += dp[i-2]
ZEROS: "06" is invalid as a single (0 alone) and
"06" is not in 10..26, so it contributes nothing.
"10" is valid as a pair only.  A '0' can never be a single.
invariant:  dp[i] counts decodings of the first i characters,
partitioned by whether the last group is length 1 or 2.
```

LANGUAGE MECHANICS: PYTHON VS C++

dp[i] = dp[i-1] + dp[i-2] is only valid when each term's ending code is legal: '0' kills the 1-digit branch, '27'..'99' kills the 2-digit branch. This is Fibonacci with a filter matrix — the same recurrence pattern as 0070 (unfiltered) and 0198 (max instead of sum). State-space math: n+1 states, each $O(1)$ to combine $\rightarrow O(n)$. The backward-only reads (i-1, i-2) again justify rolling vars: cell i is independent of cells < i-2, so those can be discarded. Real life : T9/text messaging digit-to-word mapping, ISBN and barcode check decoding, prefix-code word segmentation.
==========
/ #include <iostream> #include <vector> #include <string> #include <unordered_map> #include <unordered_set> #include <chrono> #include <sys/resource.h>

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `long long` with `% 1e9+7` at each step; taking the modulus before adding can overflow int.
- A rolling two-variable form keeps space at $O(1)$.
- Comparing with character bounds (`'1' <= s[i-1] <= '9'`) beats `int()` conversion in a hot loop.
- Rolling `prev2, prev1` avoids a length-n list; the list version is fine at n = 100.

INTERVIEW TRAPS & SANITY CHECKS

Treating '0' as a decodable single character. It is not, so `"100"` has exactly one decoding (1-0-0 as 10,0) and `"10"` has one (10), while a naive single-char check would give wrong counts.

Sanity check. `dp[i]` equals the number of decodings of s[0..i-1]. Every such decoding ends either with a 1-digit group or a 2-digit group, and those are disjoint, so summing `dp[i-1]` (when the single digit is valid) and `dp[i-2]` (when the pair is in 10..26) gives exactly dp[i].

INTUITION CHECK — 3 QUESTIONS

Q1. Why must '0' be excluded from the single-digit transition?

Answer. No letter maps to 0 alone. If '0' were allowed as a single digit, `"10"` would decode as both 1-0 and 10, which is wrong. Excluding it, and requiring 10..26 for the pair, is exactly the standard rule that makes decoding a valid prefix code.

Q2. Why is the answer taken modulo 1e9+7?

Answer. The count grows like the Fibonacci sequence in the best case, so for s of length 100 the value exceeds any fixed-width type by an enormous margin. The problem specifies the modulus, so it must be applied at each addition.

Q3. What is the space complexity, and can it be $O(1)$?

Answer. Yes – dp[i] reads only dp[i-1] and dp[i-2], so two rolling variables suffice, giving $O(1)$ space against $O(n)$ time. The full array is only useful if you need to reconstruct the decodings.

0139 Word Break Medium

ESSENCE & BOUNDS

Decide whether s can be segmented into one or more dictionary words. 1 <= s.length <= 300 and 1 <= wordDict.length <= 1000 with words of length up to 20, so the substring checks are bounded by max word length.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Bottom-up DP + hash set (BEST) dp[i] = s[0:i] is breakable. dp[0] = True (empty prefix). Recurrence : dp[i] = OR over j < i of (dp[j] and s[j:i] in wordSet) State space : i ∈ [0, n] → n+1 booleans on array dp[n+1], filled left→right; cell i reads every earlier cell j → the table is essential (no rolling-variable shortcut).

Python (intuition kernel)

```python
word_set: Set[str] = set(wordDict)
n = len(s)
dp: List[bool] = [False] * (n + 1)
dp[0] = True # empty prefix is trivially breakable
for i in range(1, n + 1):
 for j in range(i):
 if dp[j] and s[j:i] in word_set:
 dp[i] = True # one valid split is enough

```

C++ (intuition kernel)

```cpp
for (int i = 1; i <= n; ++i) { for (int j = 0; j < i; ++j) {
 if (dp[j] && wordSet.count(s.substr(j, i - j))) { dp[i] = true; // one valid split suffices
 break; // stop probing further split points
 }
 }

```

Time O(n^2-m) — n^2 (i, j) pairs $\times$ O(m) slice hash,

Space O(n) for dp + O(total word chars) for the set.

one $O(1)$ hash probe per element, no rescans; hence the stated bounds.

Top-down memoized DFS (suffix-first) f(i) = can s[i:] be broken? Try each dictionary word w: if s[i:] starts with w AND f(i+len(w)) → True. memo[i] caches suffixes. Trade-off : explores only word-aligned boundaries (usually fewer checks than all n^2 boundaries); recursion depth up to n.

Python (intuition kernel)

```python
word_set: Set[str] = set(wordDict)
memo: List[int] = [-1] * len(s) # -1 unknown, 0 false, 1 true
if i == len(s):
 return True # suffix consumed $\{}to$ breakable
if memo[i] != -1:
 return bool(memo[i]) # overlapping subproblem $\{}to$ reuse
for w in wordDict:
 if s.startswith(w, i) and f(i + len(w)):

```

C++ (intuition kernel)

```cpp
if (i == static_cast<int>(str.size())) {
 return true; // suffix fully consumed $\{}to$ breakable

```

Time O(n-W-m)

Space O(n) memo + set.

every node is visited once and returns $O(1)$ state; hence the stated bounds.

VISUAL STATE TRACE

```
s = "leetcode", dict = ["leet","code"] -> True
dp[0] = True (empty prefix is breakable)
i=1 "l":  no word ends here -> False
i=2 "le":  no
i=3 "lee":  no
i=4 "leet":  "leet" in dict, dp[0] -> dp[4] = True
i=5..8:  no word
i=9 "leetcode":  "code" in dict, dp[4] -> dp[9] = True
dp[i] = any( s[i-L:i] in words and dp[i-L] )
for each word length L <= i
the ordering matters:  dp[i-L] is already known when
processing i, since L >= 1.
invariant:  dp[i] is True iff s[0..i-1] can be segmented.
Every segmentation of that prefix ends with SOME dictionary
word of length L, leaving a breakable prefix of length i-L.
```

LANGUAGE MECHANICS: PYTHON VS C++

Option 1 (here): std::unordered_set<std::string> — separate chaining buckets, $O(1)$ average membership. Each probe hashes the slice $\rightarrow O(m)$ per probe, m = word length. Option 2 (trie): walk the slice through a trie one char at a time; no hashing of the slice — $O(m)$ worst case too, but shares the prefix walk across probe positions and avoids re-hashing. Option 3 (string hashing / rolling hash): precompute a polynomial rolling hash of s so any slice s[j:i) is $O(1)$ to hash, then probe the set of word hashes — turns dp into $O(n\hat{}2)$ total. Prefer it when words are long; a hash collision check (compare actual strings on hit) keeps it correct. State-space math: n+1 boolean states; each state scans $\leq$ n split points $\rightarrow O(n\hat{}2)$ combine work. Real life : predictive text input, dictionary-driven sentence segmentation, regex tokenizers.
==========
/ #include <iostream> #include <vector> #include <string> #include <unordered_map> #include <unordered_set> #include <chrono> #include <sys/resource.h>

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `unordered_set<string>` for the dictionary; the substring comparison then hashes a short string.
- A `maxLen` bound limits the inner loop to at most maxLen checks per position.

- A `set` of strings plus a slice per length; slicing is the expensive part, so iterating lengths outer and checking membership is efficient at these sizes.
- `any()` with a generator is concise; an explicit loop is marginally faster.

INTERVIEW TRAPS & SANITY CHECKS

Iterating forward and checking `s[:i+1]` in the dictionary, which only finds segmentations where the FIRST word is the whole prefix. The recurrence must extend at the END, using already-known dp for shorter prefixes.

Sanity check. `dp[i]` is true iff s[0..i-1] is breakable. Any breakable prefix ends with some dictionary word w of length L, and what precedes it is s[0..i-L-1], which is breakable iff dp[i-L]. Since L >= 1, dp[i-L] is already computed, so the recurrence is well-founded and complete.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is dp[0] true?

Answer. The empty string is segmentable into zero words, which is the base case that lets the first word of any non-empty breakable string be recognised. Setting dp[0] to false breaks everything.

Q2. What is the complexity?

Answer. $O(n \cdot L^2)$ where L is the max word length – n positions, up to L candidate lengths each, and each substring comparison is $O(L)$. With L <= 20 that is at most 120k character comparisons, comfortably fast.

Q3. How does this relate to the decode-ways recurrence?

Answer. Structurally identical: both compute a prefix-reachable table where dp[i] aggregates over the possible lengths of the last group. The difference is the validity test – decode-ways checks a numeric range, word-break checks dictionary membership.

0152 Maximum Product Subarray Medium

ESSENCE & BOUNDS

Return the largest product of a non-empty contiguous subarray. 1 <= nums.length <= 1e5 with -1e4 <= nums[i] <= 1e4, so products reach about 1e400 and must be taken mod 1e6+7 or handled carefully – the problem asks for the maximum mod 1e6+7 of the maximum product.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

- Kadane with running min (best) maxEnd = max(a, maxEnd*a, minEnd*a); minEnd = min(same); best = max(best, maxEnd).

Python (intuition kernel)

```python
best = max_end = min_end = nums[0]
for a in nums[1:]:
 if a < 0:
 max_end, min_end = min_end, max_end
 max_end = max(a, max_end * a)
 min_end = min(a, min_end * a)
 best = max(best, max_end)
return best

```

C++ (intuition kernel)

```cpp
for (size_t i = 1; i < nums.size(); ++i) { int a = nums[i];
 if (a < 0) std::swap(maxEnd, minEnd); // sign flip swaps roles
 maxEnd = std::max(a, maxEnd * a);
 minEnd = std::min(a, minEnd * a);
 best = std::max(best, maxEnd);

```

Time ? Space ?

one pass, two running scalars; hence the stated bounds.

- Prefix/suffix sweep Scan products left→right and right→left, reset at zeros, take the max — the winning subarray is captured by one of the sweeps. ——————————————————————— — DEEP DIVE - Why prefix-dropping fails [-2, 3, -4]: dropping the negative prefix gives max 3, but the full array scores 24. Keeping minEnd preserves the option "multiply the worst prefix by the next negative", which is the only case where dropping loses. Zeros auto-reset both extremes. ——————————————————————— TIME COMPLEXITY : $O(n)$ MEMORY COMPLEXITY: $O(1)$ ==========

Python (intuition kernel)

```python
best = nums[0]
cur = 1
for x in nums:
 cur *= x
 best = max(best, cur)
 if x == 0:
 cur = 1
cur = 1

```

C++ (intuition kernel)

```cpp
for (int x : nums) { // left-to-right products
 cur *= x;
 best = std::max(best, cur);
 if (x == 0) cur = 1;

```

Time O(n)	**Space** O(1)

each interval enters and leaves the frontier once; hence the stated bounds.

VISUAL STATE TRACE

```
nums = [2,3,-2,4] -> 6 (the subarray [2,3])
i=0 x=2 :  max=2 min=2 best=2
i=1 x=3 :  max=6 min=3 best=6
i=2 x=-2:  candidates {3, 2*-2=-4, 6*-2=-12}
max = max(3,-4,-12) = 3 <- start fresh at x
min = min(3,-4,-12) = -12
best stays 6
i=3 x=4 :  candidates {4, 3*4=12, -12*4=-48}
max = 12, min = -48, best = 12?  no best = 6?
recount:  at i=3 max=12, best = max(6, 12) = 12?
but the true answer is 6...  because MODULO. The problem
asks for max product MOD 1e6+7.  2*3=6, 3*-2*4=-24,
so the true max is 6.  The i=3 computation must use
dp[i-1]=3:  candidates {x=4, prevmax*4=12, prevmin*4=-48}
-> 12 is not right because [2,3,-2] is not a product 3.
KEY: prevmax at i=2 is 3 and prevmin is -12; so the
max at i=3 is max(4, 3*4, -12*4) = 12 -> but [3,4]
is NOT contiguous!  prevmax 3 came from a subarray that
ended at index 2, so 3*4 = 12 IS [.., 3, 4].  Which is 3*4
where 3 is from index 2?  At i=2 the max product ending at 2
is max(3, -4, -12) = 3, attained by [3] alone.  So [3,4]
= 12 is valid and IS a subarray.  But the expected answer is
6 for the un-modded variant - so the discrepancy is the
MODULUS: 12 is not the max MOD 1e6+7 either.
The actual sample answer for [2,3,-2,4] is 6; rechecking:
subarrays:  2, 2*3=6, 2*3*-2=-12, -2, -2*4=-8, 3, 3*-2=-6,
3*-2*4=-24, -2*4=-8, 4.  Maximum = 6.  So 12 must not be
reachable, meaning the trace above mis-tracked prevmax.
```

LANGUAGE MECHANICS: PYTHON VS C++

[-2, 3, -4]: dropping the negative prefix gives max 3, but the full array scores 24. Keeping minEnd preserves the option "multiply the worst prefix by the next negative", which is the only case where dropping loses. Zeros auto-reset both extremes. ——————————————————————— TIME COMPLEXITY : $O(n)$
==========

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `long long` for the running max/min; products overflow 32 bits at n = 1e5 with values 1e4.
- Apply the modulus at each multiplication to keep values bounded, since only the answer is needed modulo 1e6+7.
- Python's arbitrary-precision integers mean overflow is impossible; apply the modulus only if you want to match C++ speed.
- `max/min` over a three-element candidate list is the clearest expression of the recurrence.

INTERVIEW TRAPS & SANITY CHECKS

Tracking only the maximum. A negative number turns the smallest product into the largest, so dropping the running minimum loses the answer for cases like [-2,3,-4] where the answer comes from multiplying two negatives.

Sanity check. `maxEnd` is the maximum product of a subarray ending at i and `minEnd` the minimum. Both extend the previous step's extrema, and also consider restarting at i. Since multiplication by a negative reverses order, the maximum at step i is max(x, maxEnd*x, minEnd*x) and the minimum is the min of the same three.

INTUITION CHECK — 3 QUESTIONS

Q1. Why must you keep the minimum product?

Answer. Multiplying by a negative reverses the ordering, so the smallest previous product can become the largest after extension. Case `[-2,3,-4]`: at index 2 the min is -4*3 = -12, and multiplying by -4 gives 48 – the true answer – which no maximum-only variant can find.

Q2. Why include x itself as a candidate?

Answer. Because the optimal subarray may start at i rather than extend the previous one. With all-negative prefixes the previous products could all be smaller in magnitude than restarting, and `[-2]` at index 0 must be handled without a previous element.

Q3. What is the complexity?

Answer. $O(n)$ time and $O(1)$ space – three candidates per element and two running scalars. The modulo does not change either bound.

0198 House Robber Medium

ESSENCE & BOUNDS

Return the maximum sum of money obtainable without robbing adjacent houses. 1 <= nums.length <= 1e4 and 0 <= nums[i] <= 1e9, so the answer can reach about 1e13 and C++ needs `long long`. n = 1 returns nums[0].

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Bottom-up with rolling variables (BEST) dp[i] = max loot from houses 0..i. Recurrence : dp[i] = max(dp[i-1], nums[i] + dp[i-2]) dp[0] = nums[0], dp[1] = max(nums[0], nums[1]) State space : i ∈ [0, n) → n states on array dp[n]; cell i reads only i-1 and i-2 → collapse to two rolling variables (prev2, prev1) → $O(1)$ space.

Python (intuition kernel)

```python
if not nums:
 return 0 # empty street $\{}to$ nothing to take
if len(nums) == 1:
 return nums[0] # single house $\{}to$ take it
prev2, prev1 = nums[0], max(nums[0], nums[1])
for i in range(2, len(nums)):
 cur = max(prev1, nums[i] + prev2) # skip i vs rob i
 prev2, prev1 = prev1, cur # slide the window right

```

C++ (intuition kernel)

```cpp
if (nums.empty()) {
 return 0; // empty street $\{}to$ nothing to take

```

Time O(n) **Space** O(1)

each state is computed once from already-known states; hence the stated bounds.

Top-down memoized recursion f(i) = max loot from houses 0..i; same recurrence, solved on demand and cached in memo[i]. Trade-off : readability and natural derivation; costs recursion depth n and memo lookup per state.

Python (intuition kernel)

```python
n = len(nums)
if n == 0:
memo: List[int] = [-1] * n # -1 = house range not solved yet
 if i < 0:
 return 0 # no houses left $\{}to$ no loot
 if memo[i] != -1:
 return memo[i] # overlapping subproblem $\{}to$ reuse
 memo[i] = max(f(i - 1), nums[i] + f(i - 2))

```

C++ (intuition kernel)

```cpp
 if (i < 0) { return 0; // no houses left $\{}to$ no loot
 2

```

Time O(n) **Space** O(n)

each (i, j) state is solved once and cached; hence the stated bounds.

VISUAL STATE TRACE

```
nums = [2,7,9,3,1] -> 12 (2+9+1)
dp[0] = 2
dp[1] = max(2, 7) = 7
dp[2] = max(7, 2+9=11) = 11
dp[3] = max(11, 7+3=10) = 11
dp[4] = max(11, 11+1=12) = 12
the decision at house i:
SKIP it -> dp[i-1]
TAKE it -> dp[i-2] + nums[i] (forces skipping i-1)
invariant:  dp[i] is the best total from houses 0..i.
rolling:  prev2, prev1 = 0, 0
prev2, prev1 = prev1, max(prev1, prev2 + x)
```

LANGUAGE MECHANICS: PYTHON VS C++

Any valid selection over houses 0..i either excludes house i (then it IS a valid selection over 0..i-1) or includes it (then houses i-1 is excluded, leaving a selection over 0..i-2) — the two branches are exhaustive AND disjoint, so max() over them is exact. This is weighted interval scheduling on consecutive intervals — the same recurrence drives meeting-room allocation and task scheduling with 1-slot cool-down. State-space math: n states, $O(1)$ combine each $\rightarrow$ $O(n)$. The backward-only reads (i-1, i-2) again justify rolling variables: dp[i-3] and older are never consulted again. Real life : job scheduling with exclusion constraints, resource harvesting, and cellular network interference avoidance.
==========
/ #include <iostream> #include <vector> #include <string> #include <unordered_map> #include <unordered_set> #include <chrono> #include <sys/resource.h> #include <algorithm>

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `vector<long long> dp(n)` is readable; two rolling variables give the same result in $O(1)$ space.
- `unsigned long long` is unnecessary since all values are non-negative.

- A one-line rolling loop is idiomatic; a full list is only useful if you need to reconstruct the chosen houses.
- `max(prev1, prev2 + x)` is a single C-level call.

INTERVIEW TRAPS & SANITY CHECKS

Using `dp[i-2] + nums[i]` with `dp[i-2]` defaulting to 0 for i = 1 without also allowing the skip case, which returns nums[1] and ignores the possibility of robbing only house 0.

Sanity check. `dp[i]` is the maximum obtainable from the first i+1 houses. Any optimal solution either omits house i (leaving dp[i-1]) or includes it, which forces omitting house i-1 and gives dp[i-2] + nums[i]. The two cases are disjoint and exhaustive, so the recurrence is correct.

INTUITION CHECK — 3 QUESTIONS

Q1. Why does this reduce to house robber II with a split?

Answer. In the circle variant the first and last houses are adjacent, so a solution cannot contain both. Split the array at the last house and take the max of two linear robber problems: houses 0..n-2 and houses 1..n-1. Both exclude the wrap-around conflict, and every valid circular solution lies in one of them.

Q2. What is the space complexity and why is $O(1)$ achievable?

Answer. $O(1)$ because dp[i] reads only dp[i-1] and dp[i-2]. Two rolling scalars capture the entire dependency, which is the general technique for a linear-order recurrence with constant dependency width.

Q3. What does the greedy "take the largest" give?

Answer. 7 on [2,7,9,3,1] versus the optimal 12. Taking 9 blocks both 7 and 3, so the local maximum is not globally optimal. That counterexample is the standard justification for the DP.

0213 House Robber II Medium

ESSENCE & BOUNDS

Rob houses arranged in a circle, returning the maximum sum without robbing two adjacent houses. 1 <= nums.length <= 3e4 and 0 <= nums[i] <= 1e9, so sums reach 3e13 and C++ needs `long long`. n = 1 returns nums[0] and n = 2 returns the larger of the two.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Two linear passes, rolling variables (BEST) ans = max(robLine(nums[0..n-2]), robLine(nums[1..n-1])) robLine uses the 0198 recurrence with rolling variables. State space : two independent 1D runs, each $O(n)$ states $\rightarrow$ $O(1)$ rolling space per run.

Python (intuition kernel)

```python
n = len(nums)
if n == 1:
 return nums[0]

```

C++ (intuition kernel)

```cpp
for (int i = start; i < end; ++i) { const int cur = std::max(prev1, nums[i] + prev2);
 prev2 = prev1; // slide the window right
 prev1 = cur;

```

Time O(n) **Space** O(1)

each row is read once and discarded: two arrays at most; hence the stated bounds.

Two passes with FULL dp tables (contrast) Identical split, but each run materialises the full dp[] array so the recurrence→array mapping is visible cell by cell. Trade-off : same $O(n)$ time, $O(n)$ space — teaches why rolling variables exist: dp[i] only reads i-1, i-2, so the table is over-allocated.

Python (intuition kernel)

```python
n = len(nums)
if n == 1:
 return nums[0] # a single house: no circle to worry about

```

C++ (intuition kernel)

```cpp
if (n == 1) { return nums[0];

```

Time O(n) **Space** O(n)

Follows directly from the traversal count: $O(n)$ time, $O(n)$ space.

VISUAL STATE TRACE

```
nums = [2,3,2] -> 3 (houses 0 and 2 are adjacent)
the circle means house n-1 and house 0 are neighbours, so
a solution cannot contain both.  Two exhaustive cases:
case A: exclude the last house -> rob linear [2,3] = 3
case B: exclude the first house -> rob linear [3,2] = 3
ANSWER max(3, 3) = 3
case A: dp over [2,3] -> dp[1]=max(2,3)=3
case B: dp over [3,2] -> dp[1]=max(3,2)=3
the single house special case:
n == 1 -> return nums[0] (excluding 'first' or 'last'
would leave an empty slice)
invariant:  any optimal circular solution avoids house 0 or
avoids house n-1 (or both), so it appears in one of the two
linear sub-problems.
```

LANGUAGE MECHANICS: PYTHON VS C++

Any optimal plan is a set of houses with no adjacency. If it does not use house n-1 it is a valid plan on 0..n-2 → captured by case A. If it does not use house 0 it is valid on 1..n-1 → case B. Every plan misses at least ONE of the two neighbours (n-1 or 0) because the plan can contain neither... except when it contains neither, in which case it is contained in BOTH cases. So max(caseA, caseB) covers every feasible plan exactly. State-space math: $2 \times O(n)$ states, $O(1)$ combine → $O(n)$ total. Real life : ring topologies (token rings, round-robin scheduling on circular corridors), any 0198 variant with wraparound edges.
==========
/ #include <iostream> #include <vector> #include <string> #include <unordered_map> #include <unordered_set> #include <chrono> #include <sys/resource.h> #include <algorithm>

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- Pass (lo, hi) bounds to one shared helper rather than duplicating the loop for the two cases.
- `long long` throughout: the linear sub-problems already reach 3e13.
- `max(rob(nums[:-1]), rob(nums[1:]))` with `n == 1` guarded first is the whole solution.
- Slicing costs $O(n)$ extra memory but the arrays are at most 3e4, so it is irrelevant here.

INTERVIEW TRAPS & SANITY CHECKS

Forgetting the n = 1 case, or assuming the two linear problems are independent when they share every interior house. The overlap is fine because both are valid solutions to the circle; the max just picks the better one.

Sanity check. Any valid selection omits house 0 or omits house n-1, since they are adjacent. Therefore every valid selection is a valid linear selection within [0, n-2] or within [1, n-1], and conversely each linear selection is valid circularly. So the answer is exactly the max of the two linear robber results.

INTUITION CHECK — 3 QUESTIONS

Q1. Why are the two cases exhaustive?

Answer. Because selecting both house 0 and house n-1 is illegal in the circle. Every legal selection therefore excludes at least one of them, which places it inside one of the two linear ranges. The union covers all legal solutions and neither contains an illegal one.

Q2. What is the complexity?

Answer. $O(n)$ time – two linear passes of length n-1 – and $O(1)$ space per pass with rolling variables, plus $O(n)$ if you slice. The constant factor is two, which is negligible.

Q3. How does this generalise to a ring of k neighbours?

Answer. You would need 2^k cases (one for each subset of the boundary that must be excluded), which becomes intractable quickly. The two-case split works only because the conflict is between exactly two endpoints.

0300 `Longest Increasing Subsequence` Medium

ESSENCE & BOUNDS

Return the length of the longest strictly increasing subsequence. 1 <= nums.length <= 1000, so the $O(n\hat{}2)$ DP passes, but the problem asks for the $O(nlogn)$ approach. Duplicates are legal and must NOT extend the subsequence.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

O(n^2) DP (BEST when n $\leq$ 2000) dp[i] = length of LIS ending at index i. Recurrence : dp[i] = 1 + max(dp[j] for j < i and nums[j] < nums[i]) base dp[i] = 1 (the element alone) State space : i ∈ [0, n) → n states on array dp[n]; cell i scans every earlier cell j → full table required.

Python (intuition kernel)

```python
n = len(nums)
dp: List[int] = [1] * n # every element alone is length-1 LIS
best = 1
for i in range(1, n):
 for j in range(i):
 if nums[j] < nums[i]:
 dp[i] = max(dp[i], dp[j] + 1)
 best = max(best, dp[i])

```

C++ (intuition kernel)

```cpp
for (int i = 1; i < n; ++i) { for (int j = 0; j < i; ++j) { if (nums[j] < nums[i]) { dp[i] = std::max(dp[i], dp[j] + 1);
 }
 }
 best = std::max(best, dp[i]);

```

Time O(n^2) **Space** O(n)

Follows directly from the traversal count: $O(n^2)$ time, $O(n)$ space.

Patience sorting + binary search (BEST for big n) tails array stays SORTED; for each num, find the first slot where tails[slot] >= num (lower bound) and replace it.

Python (intuition kernel)

```python
tails: List[int] = [] # tails[k] = smallest tail of a k+1-length LIS
for num in nums:
 lo, hi = 0, len(tails)
 while lo < hi:
 mid = (lo + hi) // 2
 if tails[mid] < num:
 lo = mid + 1 # extend-past candidates stay
 else: hi = mid # shrink the insertion window

```

C++ (intuition kernel)

```cpp
for (const int num : nums) {
 const auto it = std::lower_bound(tails.begin(), tails.end(), num);
 if (it == tails.end()) {
 tails.push_back(num); // num is a record tail $\{}to$ extends LIS
 } else { }

```

Time O(n log n) **Space** O(n)

the sort dominates; the post-sort scan is linear; hence the stated bounds.

VISUAL STATE TRACE

```
nums = [10,9,2,5,3,7,101,18] -> length 4
tails[i] = smallest possible TAIL of an increasing
subsequence of length i+1 seen so far.
x=10: bisect_left(tails,10)=0 -> tails=[10]
x= 9: pos=0 -> tails=[9] (9 beats 10 as a tail)
x= 2: pos=0 -> tails=[2]
x= 5: pos=1 -> tails=[2,5]
x= 3: pos=1 -> tails=[2,3]
x= 7: pos=2 -> tails=[2,3,7]
x=101:pos=3 -> tails=[2,3,7,101]
x= 18:pos=3 -> tails=[2,3,7,18] 18 replaces 101 as tail
ANSWER len(tails) = 4 (the subsequence is 2,3,7,101)
why bisect_LEFT not bisect_right: strict increase means
a value equal to tails[pos] must NOT extend; it replaces it.
nums=[2,2,2] with bisect_right would give length 3 - wrong.
invariant: tails is strictly increasing and tails[i] is
the minimum tail for length i+1. Replacing at position p
keeps every shorter subsequence viable and improves the
longest candidate.
```

LANGUAGE MECHANICS: PYTHON VS C++

Invariant: tails[k] is the SMALLEST tail achievable for a length k+1 subsequence. Because each slot is replaced only by smaller values, tails stays strictly increasing — so binary search applies. lower_bound(tails, num) finds the

first slot that is not smaller: • found → replacing tails[slot] with the smaller num can only help future extensions (never hurts — slot's length class exists with an equal-or-better tail); • not found → num is larger than every tail → it EXTENDS the longest sequence, appending a new slot. Time math: n numbers × $O(logn)$ search = $O(nlogn)$ vs $O(n\hat{}2)$ comparisons of the DP table — the table's $O(n\hat{}2)$ work is replaced by the sorted-array trick. Real life : patience sorting is the optimal Solitaire strategy; LIS powers LCS, diff tools, and dependency-chain analysis. ==========
/ #include <iostream> #include <vector> #include <string> #include <unordered_map> #include <unordered_set> #include <chrono> #include <sys/resource.h> #include <algorithm>

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `std::lower_bound` is exactly `bisect_left`; using `upper_bound` breaks strictness.
- `vector<int> tails` with `reserve(n)` avoids reallocation in the hot path.
- `bisect_left(tails, x)` is a C-level binary search, so the whole method is $O(n \log n)$ with no Python loop inside.
- `max(tails, default=0)` handles n = 0 if the empty case is allowed.

INTERVIEW TRAPS & SANITY CHECKS

Using `bisect_right`. That makes the algorithm compute the longest NON-DECREASING subsequence, so [2,2,2] returns 3 instead of 1. Also, confusing the LIS length with the `tails` contents – `tails` is not itself a valid subsequence, only a record of best tails.

Sanity check. `tails[i]` is the smallest possible final value of any increasing subsequence of length i+1 among the processed prefix. When a new x arrives at position p = bisect_left(tails, x), every entry before p is < x so x can extend the length-p subsequence, and any entry at or after p is >= x so x cannot extend those. Replacing tails[p] with x therefore improves the best tail without losing any longer option.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is the length of `tails` the answer?

Answer. tails has one entry per achievable subsequence length, and tails[i] is the minimum tail for length i+1. Its presence proves a subsequence of that length exists. So len(tails) is the maximum achievable length, even though the array itself is not a subsequence.

Q2. Why not use the $O(n\hat{}2)$ DP?

Answer. It is correct and simpler, and passes at n = 1000. But it is $O(n^2)$ time where the tails method is $O(n \log n)$, which is the difference between passing and failing at n = 1e5. Interviewers ask for the faster one deliberately.

Q3. How do you reconstruct the actual subsequence?

Answer. Store predecessor indices: for each element record which tails position it replaced, then walk backwards from the final position. That costs an extra array and makes the method $O(n \log n)$ time with $O(n)$ space.

0322 `Coin Change` Medium

ESSENCE & BOUNDS

Return the fewest coins summing to amount, or -1 if impossible. 0 <= amount <= 1e4 with 1 <= coin.length <= 1e4, so amount 0 gives 0 coins. The answer fits in int. Duplicates in the coin list are harmless.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Bottom-up tabulation (BEST) dp[x] = min coins to make amount x. Recurrence : dp[x] = min(dp[x - c] + 1) over coins c ≤ x dp[0] = 0, dp[x] = +∞ until reached State space : x ∈ [0, amount] → amount+1 states on array dp[], filled left→right. Cell x reads every dp[x-c] → the whole prefix table must stay (no rolling shortcut).

Python (intuition kernel)

```python
dp: List[int] = [INF] * (amount + 1)
dp[0] = 0 # zero coins make zero amount
for x in range(1, amount + 1):
 for c in coins:
 if c <= x and dp[x - c] != INF:
 dp[x] = min(dp[x], dp[x - c] + 1)
return dp[amount] if dp[amount] != INF else -1

```

C++ (intuition kernel)

```cpp
for (int x = 1; x <= amount; ++x) { for (const int c : coins) { if (c <= x && dp[x - c] != INF) { dp[x] = std::min(dp[x], dp[x - c] + 1);
 }
 }

```

Time O(amount $\times$ n_coins)
Space : O(amount)

Space ?

each state is computed once from already-known states; hence the stated bounds.

Top-down memoized recursion f(x) = min coins for x; branch over each coin, cache f(x). Trade-off : explores only amounts actually reachable; recursion depth ≤ amount/cheapest-coin; memo is a full amount-sized array.

Python (intuition kernel)

```python
memo: List[int] = [-2] * (amount + 1) # -2 = unknown, -1 = impossible
memo[0] = 0
 if x < 0:
 return INF # overshot $\{}to$ not a valid path
 if memo[x] != -2:
 return memo[x] # overlapping subproblem $\{}to$ reuse
 best = INF
 for c in coins:

```

C++ (intuition kernel)

```cpp
if (x < 0) {
 return INF; // overshot $\{}to$ invalid path

```

Time O(amount $\times$ n_coins)
Space : O(amount)

Space ?

each (i, j) state is solved once and cached; hence the stated bounds.

VISUAL STATE TRACE

```
amount = 11, coins = [1,2,5] -> 3 (5+5+1)
dp[0] = 0
dp[1] = dp[0]+1 = 1
dp[2] = min(dp[1], dp[0]) + 1 = 1
dp[3] = min(dp[2],dp[1]) + 1 = 2
dp[4] = min(dp[3],dp[2]) + 1 = 2
dp[5] = min(dp[4],dp[3],dp[0]) + 1 = 1
...
dp[11]= min(dp[10],dp[9],dp[6]) + 1 = 3
two loop orders, DIFFERENT semantics:
coins outer, amount inner -> UNBOUNDED use (this problem)
amount outer, coins inner -> 0/1 use (each coin once)
invariant: dp[x] is the minimum coin count for exactly x.
Every solution for x ends with some coin c, leaving x-c,
so min over all coins of dp[x-c]+1 is exact.
```

LANGUAGE MECHANICS: PYTHON VS C++

Because every coin can be reused any number of times, the table is filled forward (left→right) with NO reverse iteration — the "unbounded" flag. Compare with 0416 Partition Equal Subset Sum, where each item is usable ONCE → the same table must be iterated BACKWARD to avoid reusing an item. State-space math: amount+1 states × n_coins combine ops each = $O(amount - n)$. The recurrence reads strictly SMALLER indices (x-c < x) → the dependency graph is a DAG → tabulation in index order is always safe. Real life : vending machine change dispensers, resource allocation, parcel-fill minimisation.
==========
/ #include <iostream> #include <vector> #include <string> #include <unordered_map> #include <unordered_set>

#include <chrono> #include <sys/resource.h> #include <algorithm> namespace { // Sentinel for "unreachable" amounts — must exceed any real answer. const int INF = 1e9; }

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `vector<int> dp(amount+1, amount+1)` with `amount+1` as the infinity sentinel avoids INT_MAX overflow on the +1.
- Sorting coins and breaking when c > x is a safe pruning.
- A `min()` generator over the coin list is concise; an explicit loop avoids the generator overhead per state.
- `[amount+1]*(amount+1)` initialisation is idiomatic and fast.

INTERVIEW TRAPS & SANITY CHECKS

Using `amount + 1` as infinity and then `dp[x] + 1` without guarding – if dp[x-c] is still the sentinel, the sum exceeds it and dp[x] can be assigned a value larger than the sentinel, which then gets compared incorrectly.

Sanity check. dp[x] equals the minimum number of coins summing to exactly x. Any optimal multiset for x contains some coin c as its last element, and removing it leaves a valid solution for x-c with dp[x-c] coins. Since coin c ranges over every coin <= x, minimising dp[x-c] + 1 over all such c is exactly the optimum. dp[0] = 0 seeds it.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is the coin loop outer?

Answer. With coins outer and amount increasing, the dp array already accounts for the current coin when computing larger amounts, so the same coin can be used repeatedly. That is unbounded knapsack. With amount outer, each state is computed from the previous states only, which corresponds to using each coin at most once.

Q2. How is the impossible case detected?

Answer. If no combination exists, every dp[x-c] remains the sentinel, so dp[x] stays above the sentinel and you return -1. Using `amount + 1` as the sentinel works because no real answer can exceed amount (one coin per unit) when coin 1 exists, and without coin 1 the sentinel is never beaten.

Q3. How does problem 518 differ?

Answer. 518 asks for the NUMBER of combinations rather than the minimum count. Same unbounded-knapsack skeleton, but the transition sums instead of minimising: ways[x] += ways[x - coin].

0416 `Partition Equal Subset Sum` ?

ESSENCE & BOUNDS

Decide whether the multiset can be split into two parts with equal sum. 1 <= nums.length <= 200 with 1 <= nums[i] <= 100, so the total is at most 2e4 and a 1-D DP of that size is trivial. An odd total makes it impossible immediately.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

v1 v1 – see the kernel below.

Python (intuition kernel)

```python
total=sum(nums)
if total%2:  return False
target=total//2; dp={0}
for n in nums:
 dp={x+n for x in dp}|dp
return target in dp

```

C++ (intuition kernel)

```cpp
int S = accumulate(nums.begin(), nums.end(), 0);
if (S & 1) return false;
int T = S / 2;
vector<char> dp(T + 1, 0); dp[0] = 1;
for (int x :  nums)
 for (int s = T; s >= x; --s) // DESCENDING: 0/1 use
 dp[s] |= dp[s - x];
return dp[T];

```

Time ?	Space ?

Follows directly from the traversal count: ? time, ? space.

v2 v2 – see the kernel below.

Python (intuition kernel)

```python
total=sum(nums)
if total%2:  return False
target=total//2; dp=[False]*(target+1); dp[0]=True
for n in nums:
 for j in range(target,n-1,-1):
  dp[j]=dp[j] or dp[j-n]
return dp[target]

```

C++ (intuition kernel)

```cpp
vector<char> dp(T + 1, 0); dp[0] = 1;
for each x in nums:
 for (int s = T; s >= x; -s) dp[s] |= dp[s - x];
// bit-packing variant:
// long long bits = 1;
// for (int x :  nums) bits |= bits « x;
// return (bits » T) & 1;

```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

VISUAL STATE TRACE

```
nums = [1,5,11,5] -> True (1+5+5 = 11)
total = 22, target = 11
1-D subset-sum over nums with capacity target:
cap=1 :  dp[1] via 1 -> {0,1}
cap=5 :  via 5 -> dp[1] too -> {0,1,5,6}
cap=11:  via 11 -> dp[0] -> {0,1,5,6,11,12,16,17}
cap=11:  via 5 (cap now 6) -> {0,1,5,6}
reachable totals up to 11:  0,1,5,6
11 IS reachable -> True
DESCENDING loop is essential:
for c in nums:
for s in range(target, c-1, -1):  dp[s] |= dp[s-c]
ascending would reuse nums[c] multiple times.
invariant:  after processing k elements, dp[s] is true iff
some subset of those k sums to s.  Descending order ensures
each element is used at most once.
```

LANGUAGE MECHANICS: PYTHON VS C++

DP tables are contiguous arrays, so memory behaviour is the same in both languages. The divergence is the inner loop: C++ reads two integers, adds, and compares – a few instructions that the compiler vectorises or at least pipelines; CPython performs list indexing, int addition, and a comparison, allocating a new int for each intermediate sum. At 1e5 to 1e6 cells that difference is one to two orders of magnitude.

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `vector<char> dp(target+1)` is 1 byte per state instead of 4 for an int.
- `dp[s] |= dp[s-c]` on a char array is a simple bitwise OR.

- A list of bools or a bytearray; a `bytearray` is the most compact.
- The bitmask trick packs all 2e4 capacities into a single big integer, making the whole DP one machine-word-per-64-states operation.

INTERVIEW TRAPS & SANITY CHECKS

Iterating the capacity loop ascending. That lets a single element be used multiple times and turns a 0/1 subset-sum into an unbounded one, giving wrong answers such as True for [1,3] with target 2.

Sanity check. `dp[s]` is true iff a subset of the elements processed so far sums to exactly s. The descending capacity loop guarantees that when dp[s] is written, dp[s-c] still reflects only previously processed elements, so each element is used at most once and the invariant holds.

INTUITION CHECK — 3 QUESTIONS

Q1. Why does an odd total immediately mean False?

Answer. Two equal subsets would each sum to total/2, which must be an integer. An odd total has no integer half, so no such split can exist. That single check eliminates half the inputs with no DP work.

Q2. What if the DP uses a boolean vector instead?

Answer. Use `vector<bool>` or a Python `bytearray` for 1 bit/byte per capacity, or the integer bitmask trick for $O(total/word)$ memory. A boolean is essential – an integer min-DP would be solving the coin-change problem instead.

Q3. How does this relate to coin change?

Answer. Identical skeleton (unbounded vs 0/1 aside), different question. Coin change asks for the minimum number of coins; this asks for a subset whose count is unconstrained, so a boolean reachable-state vector is the right state.

0647 Palindromic Substrings ?

ESSENCE & BOUNDS

Count the number of contiguous palindromic substrings, counting occurrences rather than distinct values. 0 <= s.length <= 1000, so the count can reach about 5e5 and fits in int. s = "aaa" has 6 palindromic substrings: three length-1 and three length-2.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

v1 v1 – see the kernel below.

Python (intuition kernel)

```python
count=0
 nonlocal count
 while l>=0 and r<len(s) and s[l]==s[r]:  count+=1; l-=1; r+=1
for i in range(len(s)):  expand(i,i); expand(i,i+1)
return count

```

C++ (intuition kernel)

```cpp
int count = n;
for (int i = 0; i < n; ++i) {
 for (int l = i, r = i + 1; l >= 0 && r < n &&
 s[l] == s[r]; -l, ++r)
 ++count;
 for (int l = i, r = i; l >= 0 && r < n &&
 s[l] == s[r]; -l, ++r)
 if (l != r) ++count;
}
return count;

```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

v2 v2 – see the kernel below.

Python (intuition kernel)

```python
n=len(s); dp=[[False]*n for _ in range(n)]; count=0
for i in range(n-1,-1,-1):
 for j in range(i,n):
 if s[i]==s[j] and (j-i<=2 or dp[i+1][j-1]):
 dp[i][j]=True; count+=1
return count

```

C++ (intuition kernel)

```cpp
int count = 0;
for (int i = 0; i < n; ++i) for (int j = i; j < n; ++j) {
 bool ok = true;
 for (int l = i, r = j; l < r; ++l, --r)
 if (s[l] != s[r]) { ok = false; break; }
 if (ok) ++count;
}
return count;

```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

VISUAL STATE TRACE

```
s = "aba" -> 4 (3 singles + "aba")
n single-character palindromes:  3
for each centre i, expand while s[l]==s[r] and l<r:
i=0:  expand(-1,1) no
i=1:  expand(0,2) a==a -> count 1; expand(-1,3) stop
i=2:  expand(1,3) no
TOTAL 3 + 1 = 4
"aaa":  singles 3; expansions:  (0,1) (1,2) (0,2) = 3
TOTAL 6 - matches aaa, aa, aa, a, a, a
invariant:  each palindrome has exactly one centre and one
radius, so counting expansions counts each palindrome once.
Overcounting happens only if you count both (l,r) and (r,l),
which the l<r condition prevents.
```

LANGUAGE MECHANICS: PYTHON VS C++

DP tables are contiguous arrays, so memory behaviour is the same in both languages. The divergence is the inner loop: C++ reads two integers, adds, and compares – a few instructions that the compiler vectorises or at least pipelines; CPython performs list indexing, int addition, and a comparison, allocating a new int for each intermediate sum. At 1e5 to 1e6 cells that difference is one to two orders of magnitude.

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- A `long long` result is prudent if n were larger; at n = 1000, int is fine.
- Manacher's algorithm is $O(n)$ but unnecessary at this size.
- Expanding in Python is $O(n^2)$ worst case but n <= 1000 makes it trivial.
- `s[l] == s[r]` on interned characters is cheap.

INTERVIEW TRAPS & SANITY CHECKS

Counting only palindromes of length 1, or forgetting to also count the even-length expansions (the gaps between characters). Both undercount; "aaa" needs both families.

Sanity check. Every palindromic substring has a unique centre – either a character (odd length) or a gap (even length) – and a unique radius. Iterating all 2n-1 centres and expanding outward to the first mismatch visits each palindrome exactly once.

INTUITION CHECK — 3 QUESTIONS

Q1. How do you get 6 for "aaa"?

Answer. Three length-1 palindromes (a at 0, 1, 2), two length-2 (aa at 0-1, 1-2), and one length-3 (aaa). The expansion from centre 1 hits radius 1 (aa) and radius 2 (aaa); from centres 0.5 and 1.5 it hits radius 1 each.

Q2. How does this differ from the longest-palindromic-substring problem?

Answer. Same traversal, different output: here you count every successful expansion instead of tracking the maximum radius. That makes it a counting problem, which is why the answer is a number rather than a string.

Q3. What is the complexity?

Answer. $O(n^2)$ time because a string like "aaaa..." expands to radius $O(n)$ at each of $O(n)$ centres. $O(1)$ extra space beyond the string. At n = 1000 that is at most 5e5 character comparisons.

0746 Min Cost Climbing Stairs ?

ESSENCE & BOUNDS

Return the minimum total cost to reach the top of a staircase, where you may climb 1 or 2 steps and each step i has a cost. 2 <= n <= 1000 with 0 <= cost[i] <= 1000, so the answer fits in int. cost is indexed 0..n-1 and step 1 costs cost[0].

MULTI-SOLUTION INTUITION (PYTHON VS C++)

v1 v1 – see the kernel below.

Python (intuition kernel)

```python
n=len(cost)
for i in range(2,n):
cost[i]+=min(cost[i-1],cost[i-2])
return min(cost[-1],cost[-2])

```

C++ (intuition kernel)

```cpp
int prev2 = 0, prev1 = cost[0];
for (int i = 2; i <= n; ++i) {
 int cur = cost[i - 1] + min(prev1, prev2);
 prev2 = prev1; prev1 = cur;
}
return prev1;

```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

v2 v2 – see the kernel below.

Python (intuition kernel)

```python
n=len(cost); dp=[0]*(n+1)
for i in range(2,n+1):
dp[i]=min(dp[i-1]+cost[i-1],dp[i-2]+cost[i-2])
return dp[n]

```

C++ (intuition kernel)

```cpp
vector<int> dp(n + 1, 0);
dp[0] = 0; dp[1] = cost[0];
for (int i = 2; i <= n; ++i)
 dp[i] = cost[i - 1] + min(dp[i - 1], dp[i - 2]);
return dp[n];

```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

VISUAL STATE TRACE

```
cost = [1,100,1,1,1,100,1,1,100,1] n=10 -> 6
the recurrence, with 1-BASED step indexing:
dp[0] = 0 (before the first step)
dp[1] = cost[0]
dp[i] = min(dp[i-1] + cost[i-1],
dp[i-2] + cost[i-1])
trace with 0-based cost and 1-based steps:
i=1:  dp[1] = 0 + cost[0] = 1
i=2:  min(dp[1]+cost[1], dp[0]+cost[1])
= min(1+100, 0+100) = 100
i=3:  min(dp[2]+cost[2], dp[1]+cost[2])
= min(100+1, 1+1) = 2
i=4:  min(2+1, 100+1) = 3
i=5:  min(3+1, 2+1) = 4
i=6:  min(4+100, 3+100) = 103
i=7:  min(103+1, 4+1) = 5
i=8:  min(5+1, 103+1) = 6
i=9:  min(6+100, 5+100) = 105
i=10:min(105+1, 6+100) = 106 ?
the off-by-one:  cost[i-1] is paid for ARRIVING at step i.
The correct trace gives dp[10] = 6.
```

LANGUAGE MECHANICS: PYTHON VS C++

DP tables are contiguous arrays, so memory behaviour is the same in both languages. The divergence is the inner loop: C++ reads two integers, adds, and compares – a few instructions that the compiler vectorises or at least pipelines; CPython performs list indexing, int addition, and a comparison, allocating a new int for each intermediate sum. At 1e5 to 1e6 cells that difference is one to two orders of magnitude.

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- Two rolling integers give $O(1)$ space; a full `vector` is only needed for reconstruction.
- `int` suffices since the answer is bounded by n * max(cost).

- Rolling `prev2, prev1` is the whole solution.
- Careful indexing: `prev1 + cost[i-1]` is the transition to step i, and the loop must run to i = n inclusive even though cost has length n.

INTERVIEW TRAPS & SANITY CHECKS

The off-by-one between the step number and the cost index. Climbing to step i means paying `cost[i-1]`, and the loop must iterate i from 2 to n over an array of length n – a loop that stops at n-1 never reaches the top.

Sanity check. `dp[i]` is the minimum cost to be standing on step i. The final move into step i is from i-1 (paying cost[i-1]) or from i-2 (paying cost[i-1] as well, since you skip a step). The two are disjoint, so the minimum is exact.

INTUITION CHECK — 3 QUESTIONS

Q1. How is this the same shape as house robber?

Answer. Both are "choose i or skip i-1". In house robber, choosing i forces skipping i-1 because they are adjacent; here, taking a 2-step jump from i-2 skips step i-1's cost. The recurrences are structurally identical – only the operation changes from addition to minimisation.

Q2. What is the base case?

Answer. dp[0] = 0 (standing before any step, free) and dp[1] = cost[0] (you must pay for the first step you climb). Getting dp[1] wrong is the usual indexing bug.

Q3. What are the complexities?

Answer. $O(n)$ time and $O(1)$ space with two rolling variables, against $O(n)$ space for a full dp array. The time bound is optimal since every cost must be read.

1143 `Longest Common Subsequence` Medium

ESSENCE & BOUNDS

Return the length of the longest common subsequence of two strings. 1 <= m, n <= 1000, so an $O(mn)$ table has 1e6 cells – fine. At m = n = 1000 the $O(mn)$ DP is required; the 1-D rolling form saves the space.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

- Full DP table Standard 2D recurrence, easy to read and verify.

Python (intuition kernel)

```python
m, n = len(text1), len(text2)
dp = [[0] * (n + 1) for _ in range(m + 1)]
for i in range(1, m + 1):
 for j in range(1, n + 1):
 if text1[i - 1] == text2[j - 1]:
 dp[i][j] = dp[i - 1][j - 1] + 1
 else:  dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
return dp[m][n]

```

C++ (intuition kernel)

```cpp
for (int i = 1; i <= m; ++i) { for (int j = 1; j <= n; ++j)
 cur[j] = (text1[i - 1] == text2[j - 1])
 ?  prev[j - 1] + 1
 :  std::max(prev[j], cur[j - 1]);
std::swap(prev, cur);

```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

- Rolling two rows (memory-optimized) prev/cur rows; identical recurrence, $O(n)$ memory. ———— DEEP DIVE - Longest-path interpretation State (i,j) on a grid; a diagonal step (match) earns +1, right/ down steps are free — dp is a longest path. Optimal substructure holds because a prefix of an optimal LCS is optimal for the corresponding prefix pair. 1000×1000 strings = 1M table cells (~8 MB) vs 8 KB for the rolling variant. ———— TIME COMPLEXITY : $O(m - n)$ MEMORY COMPLEXITY: $O(m - n)$ or $O(n)$ rolling.

Python (intuition kernel)

```python
m, n = len(text1), len(text2)
prev = [0] * (n + 1)
for i in range(1, m + 1):
 cur = [0] * (n + 1)
 for j in range(1, n + 1):
 if text1[i - 1] == text2[j - 1]:
 cur[j] = prev[j - 1] + 1
 else:  cur[j] = max(prev[j], cur[j - 1])

```

C++ (intuition kernel)

```cpp
for (int i = 1; i <= m; ++i) { for (int j = 1; j <= n; ++j)
 cur[j] = (text1[i - 1] == text2[j - 1])
 ?  prev[j - 1] + 1
 :  std::max(prev[j], cur[j - 1]);
std::swap(prev, cur);

```

Time O(m-n) **Space** O(m-n) or O(n) rolling.

each state is solved once; the rest are cached lookups; hence the stated bounds.

VISUAL STATE TRACE

```
a = "abcde", b = "ace" -> 3 ("ace")
dp[i][j] = LCS of a[:i], b[:j]
if a[i-1] == b[j-1]:  dp[i][j] = dp[i-1][j-1] + 1
else:  dp[i][j] = max(dp[i-1][j], dp[i][j-1])
base:  dp[0][j] = dp[i][0] = 0
matching chars:  taking BOTH is always optimal - if they
are equal, an optimal LCS can always be arranged to end
with that matched pair, so no choice is needed.
mismatching chars:  you must drop at least one of them,
hence the max.
rolling rows:  only dp[i-1][*] and dp[i][j-1] are read, so
keep one array and a diagonal scalar.
```

LANGUAGE MECHANICS: PYTHON VS C++

State (i,j) on a grid; a diagonal step (match) earns +1, right/ down steps are free — dp is a longest path. Optimal substructure holds because a prefix of an optimal LCS is optimal for the corresponding prefix pair. 1000×1000 strings = 1M table cells (~8 MB) vs 8 KB for the rolling variant. ———— TIME COMPLEXITY : $O(m - n)$

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `vector<int> dp(n+1)` with a saved diagonal gives the 1-D form at $O(n)$ space.
- `string_view` avoids copying the inputs if they arrive by value.

- A single list reused across rows plus a `diag` variable is the standard Python space optimisation.
- `array('H')` halves memory again since values are bounded by min(m, n) <= 1000.

INTERVIEW TRAPS & SANITY CHECKS

Using the LONGEST COMMON SUBSTRING recurrence when characters match, forgetting that equal characters mean take both. The substring problem instead resets to 0 on a mismatch; the subsequence problem takes a max.

Sanity check. dp[i][j] equals the LCS length of the prefixes a[:i] and b[:j]. Equal trailing characters can be appended to any LCS of the shorter prefixes, giving dp[i-1][j-1] + 1, and no longer LCS exists because any common subsequence must use those trailing characters as a matched pair. Unequal trailing characters force at least one of them to be excluded, giving the max of the two one-sided recursions.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is dp[i-1][j-1] + 1 correct on a match?

Answer. Any common subsequence of a[:i] and b[:j] either uses the final matched pair as its last element or not. If not, it is a common subsequence of the shorter prefixes and has length at most dp[i-1][j-1]. If yes, its length is at most dp[i-1][j-1] + 1, and that bound is achievable.

Q2. Why is the space reduction to O(min(m,n))?

Answer. The recurrence reads only the previous row and the current row's left neighbour, so one array plus a saved diagonal suffices. Swapping the strings so the shorter one is the array dimension minimises that further.

Q3. What is the complexity?

Answer. $O(mn)$ time and $O(\min(m, n))$ space with rolling rows, against $O(mn)$ space for the full table. At 1000 x 1000 the full table is 4 MB in C++ or about 30 MB in Python with per-cell int objects.

14. 2-D Dynamic Programming

#	Problem	Level	One-line intent
0010	**Regular Expression Matching**	Hard	dp[i][j] on prefixes; '.' and '*' are the two branches.
0062	**Unique Paths**	?	dp[r][c] = dp[r-1][c] + dp[r][c-1], one path each.
0063	**Unique Paths II**	Medium	Obstacles get 0; the recurrence is unchanged.
0064	**Minimum Path Sum**	Medium	dp[r][c] = grid[r][c] + min(dp[r-1][c], dp[r][c-1]).
0072	**Edit Distance**	?	Insert, delete, or replace; take the cheapest of three.
0097	**Interleaving String**	?	3-D state collapsed to 2-D: interleaving of prefixes.
0115	**Distinct Subsequences**	?	dp[i][j] = dp[i-1][j] + dp[i-1][j-1] when chars match.
0309	**Best Time To Buy And Sell Stock With Cooldown**	?	Three-state DP: holding, sold, resting.
0312	**Burst Balloons**	?	Pick the LAST balloon to burst in each interval.
0329	**Longest Increasing Path In A Matrix**	?	Memoised DFS with four increasing directions.
0494	**Target Sum**	?	Turn assignment into subset sum: (S + k) / 2.
0518	**Coin Change Ii**	?	Coins outer, amount inner – count combinations.

0010 Regular Expression Matching Hard

ESSENCE & BOUNDS

Decide whether the whole string matches the whole pattern, where '.' matches any single character and '*' applies to the preceding element. 1 <= s.length <= 20 and 1 <= p.length <= 20, so an $O(mn)$ table is 400 cells – recursion depth is also bounded, so memoised recursion is perfectly acceptable.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Bottom-up 2D DP (BEST) dp[i][j] = does s[0..i-1] match p[0..j-1]? Base: dp[0][0] = True. p[j-1] != '*': dp[i][j] = dp[i-1][j-1] AND (s[i-1] == p[j-1] OR p[j-1]=='.') p[j-1] == '*': dp[i][j] = dp[i][j-2] (zero copies) OR (dp[i-1][j] AND (s[i-1] == p[j-2] OR p[j-2]=='.'))

Python (intuition kernel)

```python
n, m = len(s), len(p)
dp = [[False] * (m + 1) for _ in range(n + 1)]
dp[0][0] = True # empty string matches empty pattern
for j in range(2, m + 1):
 if p[j - 1] == "*":
 dp[0][j] = dp[0][j - 2]
for i in range(1, n + 1):
 for j in range(1, m + 1):

```

C++ (intuition kernel)

```cpp
{ const int n = static_cast<int>(s.size());
 const int m = static_cast<int>(p.size());
 std::vector<std::vector<bool>> dp(n + 1, std::vector<bool>(m + 1, false));
 dp[0][0] = true; // empty string matches empty pattern
 for (int j = 2; j <= m; ++j)
 { if (p[j - 1] == '*')
 {
 dp[0][j] = dp[0][j - 2]; // drop "x*" ${}to$ match nothing

```

Time O(|s|·|p|) **Space** O(|s|·|p|)

each state is computed once from already-known states; hence the stated bounds.

Top-down memoized DFS (same recurrence, recursion-first) solve(i, j) computes the same value lazily, recursing only on the states it needs and memoizing them.

Python (intuition kernel)

```python
memo: dict[tuple[int, int], bool] = {}
if (i, j) in memo:
return memo[(i, j)] # memo hit $\{}to$ O(1)
if j == len(p):
ans = i == len(s)
else:
first_match = i < len(s) and (s[i] == p[j] or p[j]
== ".")
if j + 1 < len(p) and p[j + 1] == "*":

```

C++ (intuition kernel)

```cpp
{ if (memo_[i][j] != -1)
{
return memo_[i][j] == 1; // already answered $\{}to$
reuse (memo hit)
}
bool ans = false;
if (j == static_cast<int>(p.size()))
{
ans = (i == static_cast<int>(s.size())); // pattern
spent: string must be too

```

Time O(|s|·|p|) **Space** O(|s|·|p|)

every node is visited once and returns $O(1)$ state; hence the stated bounds.

VISUAL STATE TRACE

```
s="aab", p="c*a*b" -> match
dp[i][j] = does s[:i] match p[:j]
if p[j-1] == '*':
dp[i][j] = dp[i][j-2] # zero occurrence
if i>0 and (p[j-2]=='.' or s[i-1]==p[j-2]):
dp[i][j] |= dp[i-1][j] # one more occurrence
else:
dp[i][j] = i>0 and j>0 and
(p[j-1]=='.' or s[i-1]==p[j-1])
and dp[i-1][j-1]
base: dp[0][0] = True
dp[0][j] = dp[0][j-2] if p[j-1]=='*'
dp[i][0] = False for i > 0
'*' always refers to the element BEFORE it, so p[:j-2]
removes the pair (element, star). That is why the zero-
occurrence branch steps back TWO, not one - the classic
off-by-one.
```

LANGUAGE MECHANICS: PYTHON VS C++

p = "c*" against s = "cccc": the '*' must decide: zero copies → drop "c*" (dp[i][j-2]); one more copy → consume one matching char and KEEP "c*" (dp[i-1][j]). The OR lets ONE '*' absorb any run length, including zero. Table is (|s|+1)×(|p|+1) so the empty-string prefixes are representable. ==========
p = "c*" against s = "ccca": at dp[4][2] (s="cccc"? no — s[0..3] = "cccc", p = "c*") the '*' must decide: • zero copies → dp[4][0] (drop "c*" entirely, keep all of s); • one more copy → s[3]=='c' == p[0] → dp[3][2] (consume one 'c', pattern stays "c*" so it may repeat). The OR is what lets one '*' absorb ANY run length — including zero. Table shape: (|s|+1) × (|p|+1) so index i/j means "first i / first j characters consumed" and the empty-string prefixes are representable. ==========
/ #include <iostream> #include <vector> #include <string> #include <chrono> #include <sys/resource.h>

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `vector<vector<char>» dp(m+1, vector<char>(n+1))` avoids vector<bool>'s proxies.
- A 1-D rolling row with a saved diagonal works here too, saving the full table.

- Memoised recursion with `functools.lru_cache` is shorter and equally fast at these bounds.
- A `list` of `bytearray` rows is compact; a list of lists of bools costs one object per cell.

INTERVIEW TRAPS & SANITY CHECKS

Stepping back one position instead of two on the '*' branch. `dp[i][j-1]` would still contain the '*' and give an infinite recursion; the correct step is `dp[i][j-2]` to remove the element-star pair.

Sanity check. dp[i][j] is true iff s[:i] matches p[:j]. A pattern ending in '*' matches in exactly two ways – the preceding element occurs zero times (leaving p[:j-2]) or one more time (extending a match of s[:i-1] against p[:j]). Those are disjoint and exhaustive, so the disjunction is exact.

INTUITION CHECK — 3 QUESTIONS

Q1. Why does '*' step back two?

Answer. Because '*' is a modifier on the element before it, not a standalone matcher. Removing the pattern's contribution means removing both the element and the star, which is a step of two.

Q2. When is the top-down version better?

Answer. When the state space is sparse – for example if many dp[i][j] combinations are never reachable. Memoisation then avoids computing them, whereas the bottom-up table fills every cell whether or not it matters.

Q3. How would you extend the pattern language?

Answer. Adding a construct means adding a branch to the recurrence. Each construct adds a fixed number of look-back steps (1 for a plain matcher, 2 for a modifier), so the table indices must be derived from the construct's arity rather than hard-coded.

0062 `Unique Paths` ?

ESSENCE & BOUNDS

Count the paths from the top-left to the bottom-right of a grid moving only right or down. 1 <= m, n <= 100 and no blocked cells, so the answer is C(m+n-2, m-1), which for 100 x 100 exceeds 64 bits – either take the problem's modulo or use `long long` carefully with a modulo.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

v1 v1 – see the kernel below.

Python (intuition kernel)

```python
dp=[[1]*n for _ in range(m)]
for r in range(1,m):
 for c in range(1,n):  dp[r][c]=dp[r-1][c]+dp[r][c-1]
return dp[m-1][n-1]

```

C++ (intuition kernel)

```cpp
{
 std::vector<std::vector<int>> dp(m,
std::vector<int>(n, 0));
 for (int c = 0; c < n; ++c)
 { dp[0][c] = 1;
 }
 for (int r = 0; r < m; ++r)
 { dp[r][0] = 1;
 }

```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

v2 v2 – see the kernel below.

Python (intuition kernel)

```python
dp=[1]*n
for _ in range(1,m):
 for c in range(1,n):  dp[c]+=dp[c-1]
return dp[n-1]

```

C++ (intuition kernel)

```cpp
{
 std::vector<int> dp(n, 1); // row 0:  every cell
reachable 1 way
 for (int r = 1; r < m; ++r)
 { for (int c = 1; c < n; ++c)
 { dp[c] = dp[c] + dp[c - 1];
 }
 }
 return dp[n - 1];

```

Time ?	Space ?

Follows directly from the traversal count: ? time, ? space.

v3 v3 – see the kernel below.

Python (intuition kernel)

```python
from math import comb
return comb(m+n-2, m-1)

```

C++ (intuition kernel)

```cpp
// closed-form:  C(m+n-2, m-1) computed in O(min(m,n))
long long r = 1;
int k = min(m - 1, n - 1), tot = m + n - 2;
for (int i = 1; i <= k; ++i) r = r * (tot - k + i) / i;
return (int)r;

```

Time ?	Space ?

Follows directly from the traversal count: ? time, ? space.

VISUAL STATE TRACE

```
m=3 n=3 -> 6

1 1 1

1 2 3

1 3 6

each cell = sum of the cell above and the cell to the left,

because the last move into it must be one of those two.

base:  row 0 and col 0 are all 1 (exactly one way along an edge)

the path count is a binomial coefficient:

C(m+n-2, m-1) = C(4,2) = 6

the 1-D rolling version:

dp = [1]*n

for r in 1..m-1:

for c in 1..n-1:

dp[c] += dp[c-1] # dp[c] was above, dp[c-1] is left

return dp[n-1]
```

LANGUAGE MECHANICS: PYTHON VS C++

When row r is computed, dp[c] still holds the (r-1) value UNTIL we overwrite it. Computing left→right, dp[c-1] was already overwritten with row r's value → we get dp[r][c-1] for free, while dp[c] (still old) supplies dp[r-1][c]. One array, both dependencies, no copies. Answer location after all rows: dp[n-1]. ==========
/ #include <iostream> #include <vector> #include <string> #include <chrono> #include <sys/resource.h>

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- In-place 2-D DP reuses the grid as the table, giving $O(1)$ extra space.
- `vector<long long>` with an explicit modulo if the judge expects one; the true value at 100x100 does not fit.
- The 1-D rolling version is 5 lines and the fastest Python formulation here.
- Modulo the value at each step to keep Python's arbitrary-precision multiplications from growing.

INTERVIEW TRAPS & SANITY CHECKS

Off-by-one on the base cases. Row 0 and column 0 must be 1, not 0 – starting at (0,0) there is exactly one way to be there, and that single path propagates along the edges.

Sanity check. dp[r][c] equals the number of paths to (r, c). Every such path's last step is either down (from (r-1,c)) or right (from (r,c-1)), and those two sets are disjoint, so summing them is exact. The boundary cells have only one predecessor, giving 1.

INTUITION CHECK — 3 QUESTIONS

Q1. How is the answer related to a binomial coefficient?

Answer. A path is a sequence of m-1 downs and n-1 rights, so choosing which m-1 of the m+n-2 moves are downs determines the path uniquely. That is exactly C(m+n-2, m-1), which is a closed form if you want to avoid the DP entirely.

Q2. Why do the boundary cells equal 1?

Answer. A cell on the top row can only be reached by moving right from its left neighbour, so there is exactly one path. Same for the left column. Setting them to 1 rather than 0 makes the interior recurrence produce the right counts.

Q3. What does the blocked-cell variant change?

Answer. Blocked cells get 0 and the recurrence stays identical, because 0 correctly means "no path reaches here". That is the whole difference between problem 62 and 63 – one conditional in the fill loop.

0063 Unique Paths II Medium

ESSENCE & BOUNDS

Count the obstacle-constrained right/down paths across a grid. 1 <= m, n <= 100, and if the start or end is blocked the answer is 0. m = 1 or n = 1 is legal and often has no path.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

- In-place DP (best) Reuse the grid as the table; seed (0,0), sweep row-major.

Python (intuition kernel)

```python
n = len(grid[0])
dp = [0] * n
dp[0] = 0 if grid[0][0] == 1 else 1
for row in grid:
 if row[0] == 1:
 dp[0] = 0
 for j in range(1, n):
 if row[j] == 1:

```

C++ (intuition kernel)

```cpp
for (int i = 1; i < m; ++i) { // rest of the grid
 g[i][0] = g[i][0] == 1 ? 0 : g[i - 1][0];
 for (int j = 1; j < n; ++j)
 g[i][j] = g[i][j] == 1 ? 0 : g[i - 1][j] + g[i][j - 1];

```

Time ? **Space** ?

the output array doubles as the state store; hence the stated bounds.

- 1-D rolling row dp[j] += dp[j-1] per row; obstacle cells set dp[j] = 0. ———————————————— DEEP DIVE - Mutation safety Cells read (0/1 obstacle flags) and written (path counts) can never be confused because counts ≥ 0, and each cell is computed only after its feeders — top-down + left-to-right order. Rolling row: dp[j] alone is the whole column's history. ———————————————— TIME COMPLEXITY : $O(m - n)$ MEMORY COMPLEXITY: $O(1)$ in-place / $O(n)$ rolling.
==========

Python (intuition kernel)

```python
m, n = len(grid), len(grid[0])
if grid[0][0] == 1:
grid[0][0] = 1
for j in range(1, n):
 grid[0][j] = 0 if grid[0][j] == 1 else grid[0][j - 1]
for i in range(1, m):
 grid[i][0] = 0 if grid[i][0] == 1 else grid[i - 1][0]
 for j in range(1, n):

```

C++ (intuition kernel)

```cpp
for (auto& row : g) { if (row[0] == 1) dp[0] = 0;
 for (int j = 1; j < n; ++j) { if (row[j] == 1) dp[j] = 0;
 else dp[j] += dp[j - 1];
 }

```

Time O(m-n) **Space** O(1) in-place / O(n) rolling.

each row is read once and discarded: two arrays at most; hence the stated bounds.

VISUAL STATE TRACE

```
grid:  [[0,0,0],[0,1,0],[0,0,0]] -> 2
1 1 1
1 X 1 X = obstacle, gets 0
1 1 2
rule:  if obstacle[r][c] -> dp[r][c] = 0
else dp[r][c] = dp[r-1][c] + dp[r][c-1]
setting an obstacle to 0 is exactly right:  "no path can
END at an obstacle".  The propagation then correctly kills
everything that would have to route through it.
1-D rolling:
dp = [1]*n
for r in range(m):
for c in range(n):
if grid[r][c] == 1:  dp[c] = 0
elif c > 0:  dp[c] += dp[c-1]
```

LANGUAGE MECHANICS: PYTHON VS C++

Cells read (0/1 obstacle flags) and written (path counts) can never be confused because counts ≥ 0, and each cell is computed only after its feeders — top-down + left-to-right order. Rolling row: dp[j] alone is the whole column's history. ——————————————————————— TIME COMPLEXITY : $O(m - n)$
==========

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `vector<vector<int>> dp(m, vector<int>(n,1))` with the obstacle check is clearest.
- Reusing `grid` as the DP table is safe only if the caller does not need the original, which the problem does not forbid but which is impolite.
- The 1-D rolling form handles obstacles correctly because `dp[c] = 0` and then subsequent cells add from it, propagating the block.
- Careful: the first row must also be processed, so the loop starts at r = 0, not 1.

INTERVIEW TRAPS & SANITY CHECKS

Starting the fill loop at row 1 and column 1, which leaves row 0 and column 0 unprocessed – so obstacles on the boundary are ignored and the top-left cell is not checked.

Sanity check. dp[r][c] is the number of obstacle-respecting paths to (r,c). An obstacle admits none, so it stores 0. Otherwise every path arrives from above or from the left, disjointly, so the sum is exact. The invariant propagates the blocking effect correctly to all downstream cells.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is setting an obstacle to 0 sufficient?

Answer. Because the recurrence only ever adds predecessor values. A zero at an obstacle means no path can arrive there, so any cell depending on it loses that contribution automatically – the blockage propagates without special handling.

Q2. What happens if the start cell is an obstacle?

Answer. dp[0][0] is set to 0, and then every subsequent cell adds from a chain of zeros, so the whole table becomes 0 and the answer is 0. That is exactly the required behaviour, with no explicit guard.

Q3. Why does the rolling version need the obstacle check before the column check?

Answer. Because an obstacle must be written as 0 rather than accumulated. If the `dp[c] += dp[c-1]` ran first and the obstacle overwrote it afterwards, the previous row's contribution for that cell would leak in incorrectly.

0064 Minimum Path Sum Medium

ESSENCE & BOUNDS

Return the minimum sum of numbers along a right/down path from the top-left to the bottom-right. 1 <= m, n <= 200 with 0 <= grid[i][j] <= 200, so the answer can reach 4e4 and fits in int. Non-negative values mean a single-cell grid returns that cell.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

- In-place DP (best) Seed row 0 and column 0 with prefix sums, then the recurrence.

Python (intuition kernel)

```python
m, n = len(grid), len(grid[0])
dp = [0] * n
for i in range(m):
 for j in range(n):
 if i == 0 and j == 0:
 dp[j] = grid[i][j]
 elif i == 0:  dp[j] = grid[i][j] + dp[j - 1]
 elif j == 0:  dp[j] = grid[i][j] + dp[j]

```

C++ (intuition kernel)

```cpp
const Case cases[] = { {{{1, 3, 1}, {1, 5, 1}, {4, 2, 1}}, 7},
 {{{1, 2, 3}, {4, 5, 6}}, 12},
 {{{5}}, 5}

```

Time ? **Space** ?

the output array doubles as the state store; hence the stated bounds.

- 1-D rolling row dp[j] = grid[i][j] + min(above, left) using a single array. ———————————————— DEEP DIVE - DP ≡ Dijkstra on a DAG With only right/down edges the grid is a DAG, so shortest paths follow topological order — no heap needed. Optimal substructure: the prefix of an optimal path through (i,j) is itself optimal, else swapping in a cheaper feeder improves the whole path. ———————————————— TIME COMPLEXITY : $O(m-n)$ MEMORY COMPLEXITY: $O(1)$ in-place / $O(n)$ rolling.
==========

Python (intuition kernel)

```python
m, n = len(grid), len(grid[0])
for j in range(1, n):
 grid[0][j] += grid[0][j - 1]
for i in range(1, m):
 grid[i][0] += grid[i - 1][0]
 for j in range(1, n):
 grid[i][j] += min(grid[i - 1][j], grid[i][j - 1])
return grid[m - 1][n - 1]

```

C++ (intuition kernel)

```cpp
vector<int> dp(n, INT_MAX); dp[0] = 0;
for (int r = 0; r < m; ++r) {
 for (int c = 0; c < n; ++c) {
 int top = (r == 0) ?  INT_MAX : dp[c];
 int left = (c == 0) ?  INT_MAX : dp[c - 1];
 dp[c] = grid[r][c] + min(top, left);
 }
}
return dp[n - 1];

```

Time O(m-n) **Space** O(1) in-place / O(n) rolling.

each row is read once and discarded: two arrays at most; hence the stated bounds.

VISUAL STATE TRACE

```
grid:  [[1,3,1],[1,5,1],[4,2,1]] -> 7
1 4 5
2 7 6
6 8 7 (path 1->3->1->1->1 = 7)
dp[r][c] = grid[r][c] + min(above, left)
boundary:  row 0 = cumulative sum from the left
col 0 = cumulative sum from above
1-D rolling:  the first row must be handled specially because
there is no 'above' predecessor:
dp = [0]*n; dp[0] = grid[0][0]
for c in 1..n-1:  dp[c] = dp[c-1] + grid[0][c]
for r in 1..m-1:
dp[0] += grid[r][0]
for c in 1..n-1:  dp[c] = grid[r][c] + min(dp[c], dp[c-1])
```

LANGUAGE MECHANICS: PYTHON VS C++

With only right/down edges the grid is a DAG, so shortest paths follow topological order — no heap needed. Optimal substructure: the prefix of an optimal path through (i,j) is itself optimal, else swapping in a cheaper feeder improves the whole path. ——————————————————————— TIME COMPLEXITY : $O(m-n)$
==========

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `vector<vector<int>> dp = grid` with in-place accumulation reuses the grid, $O(1)$ extra space.
- `INF = 1e9` as the initial value works because grid values are non-negative, so the sentinel is always replaced.
- The 1-D rolling form is the fastest Python version; a full table is 4e4 cells which is fine but slower.
- `min()` per cell is a Python call; a conditional expression is measurably faster at this size.

INTERVIEW TRAPS & SANITY CHECKS

Initialising the whole first row with `grid[0][c]` alone instead of the cumulative sum, which forgets that reaching (0,c) requires traversing every cell before it.

Sanity check. dp[r][c] is the minimum path sum ending at (r,c). Every such path's final step comes from above or from the left, disjointly, so the cheaper predecessor plus the current cell is the optimum. The boundary cells have a single predecessor, giving a running sum.

INTUITION CHECK — 3 QUESTIONS

Q1. What changes from the unique-paths version?

Answer. Only the operator: + of counts becomes `min` of costs. The topology – which cells feed which – is identical, so the same boundary handling and the same rolling-array trick apply unchanged.

Q2. Why is the first row special in the rolling version?

Answer. The rolling array relies on dp[c] still holding the previous row's value while dp[c-1] holds the current row's. That only works from row 1 onward; row 0 has no previous row, so it must be filled by a plain left-to-right cumulative sum.

Q3. What if grid values could be negative?

Answer. The algorithm still works – there is no assumption on sign – but the sentinel initialisation must then be a large POSITIVE value, since a negative sentinel would be chosen by min and poison the boundary.

0072 Edit Distance ?

ESSENCE & BOUNDS

Return the minimum number of single-character edits (insert, delete, replace) turning word1 into word2. 0 <= m, n <= 500 with lowercase letters, so an $O(mn)$ table has 2.5e5 cells – a full table is fine but the rolling row is far leaner.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

v1 v1 – see the kernel below.

Python (intuition kernel)

```python
m,n=len(w1),len(w2)
dp=[[0]*(n+1) for _ in range(m+1)]
for i in range(m+1):  dp[i][0]=i
for j in range(n+1):  dp[0][j]=j
for i in range(1,m+1):
 for j in range(1,n+1):
 if w1[i-1]==w2[j-1]:  dp[i][j]=dp[i-1][j-1]
 else:
dp[i][j]=1+min(dp[i-1][j],dp[i][j-1],dp[i-1][j-1])

```

C++ (intuition kernel)

```cpp
{ const int m = static_cast<int>(word1.size());
 const int n = static_cast<int>(word2.size());
 std::vector<std::vector<int>> dp(m + 1,
std::vector<int>(n + 1, 0));
 for (int i = 0; i <= m; ++i)
 { dp[i][0] = i; // delete everything
 }
 for (int j = 0; j <= n; ++j)
 { dp[0][j] = j; // insert everything

```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

v2 v2 – see the kernel below.

Python (intuition kernel)

```python
m,n=len(w1),len(w2)
dp=list(range(n+1))
for i in range(1,m+1):
 prev=dp[0]; dp[0]=i
 for j in range(1,n+1):
 temp=dp[j]
 if w1[i-1]==w2[j-1]:  dp[j]=prev
 else:  dp[j]=1+min(dp[j],dp[j-1],prev)

```

C++ (intuition kernel)

```cpp
{ const int m = static_cast<int>(word1.size());
 const int n = static_cast<int>(word2.size());
 std::vector<int> dp(n + 1, 0);
 for (int j = 0; j <= n; ++j)
 { dp[j] = j; // row 0 = insert j characters
 }
 for (int i = 1; i <= m; ++i)
 {

```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

VISUAL STATE TRACE

```
word1="horse", word2="ros" -> 3
dp[i][j] = edit distance between word1[:i] and word2[:j]
if word1[i-1] == word2[j-1]:  dp[i][j] = dp[i-1][j-1]
else:  dp[i][j] = 1 + min(
dp[i-1][j-1], # replace
dp[i-1][j], # delete from word1
dp[i][j-1]) # insert into word1
base:  dp[i][0] = i (delete all i)
dp[0][j] = j (insert all j)
the three branches are EXHAUSTIVE: to align the two
prefixes you either match, substitute, skip one from the
left, or skip one from the right.  Levenshtein adds transpose
as a fourth option and gains sub-word accuracy.
```

LANGUAGE MECHANICS: PYTHON VS C++

Any edit script can be reordered so its LAST operation is one of the three above — so the min over them is exact, never approximate. The "match" case cannot be beaten by deleting+reinserting the pair (that would cost 2), so we take dp[i-1][j-1] directly. ==========
/ #include <iostream> #include <vector> #include <string> #include <chrono> #include <sys/resource.h>

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- A `vector<int> prev(n+1), cur(n+1)` pair of rows is the standard $O(n)$ space form.
- `std::vector<int> dp((m+1)*(n+1))` in one allocation beats a 2-D vector of vectors for cache locality.
- A single list plus a saved diagonal variable gives $O(n)$ space.
- Since the values are bounded by max(m,n), `array('H')` halves the memory again.

INTERVIEW TRAPS & SANITY CHECKS

Forgetting the base rows: dp[i][0] = i and dp[0][j] = j. Without them the first column and row are garbage and every subsequent cell inherits the error.

Sanity check. dp[i][j] is the edit distance between the first i and j characters. Matching trailing characters cost nothing extra, so the optimum is dp[i-1][j-1]. Otherwise the last operation must be a replace, delete, or insert, and each leaves a smaller subproblem of the same kind. The three cases are disjoint, so the minimum is exact.

INTUITION CHECK — 3 QUESTIONS

Q1. Why does a match cost nothing?

Answer. If the last characters are equal there is always an optimal alignment that pairs them, removing one character from both subproblems and adding no cost. Any alignment that leaves them unmatched cannot be cheaper, since the rest still needs at least dp[i-1][j-1] edits.

Q2. What does Levenshtein add?

Answer. Transposition of two adjacent characters as a single edit. That is a fourth branch, dp[i-2][j-2] + 1, which makes the distance sensitive to word transpositions that plain Levenshtein penalises twice.

Q3. What are the complexities?

Answer. $O(mn)$ time and $O(\min(m, n))$ space with two rolling rows. The time is optimal for the general case: recovering the alignment requires examining essentially all (i, j) pairs.

0097 `Interleaving String` ?

ESSENCE & BOUNDS

Decide whether s3 is an interleaving of s1 and s2, preserving the relative order within each. 0 <= s1, s2 <= 100 and lengths must sum to s3, which is worth checking first as an $O(1)$ rejection. The table is 1e4 cells.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

v1 v1 – see the kernel below.

Python (intuition kernel)

```python
m,n=len(s1),len(s2)
if m+n!=len(s3):  return False
dp=[[False]*(n+1) for _ in range(m+1)]
dp[0][0]=True
for i in range(1,m+1):  dp[i][0]=dp[i-1][0] and s1[i-1]==s3[i-1]
for j in range(1,n+1):  dp[0][j]=dp[0][j-1] and s2[j-1]==s3[j-1]
for i in range(1,m+1):
 for j in range(1,n+1):

```

C++ (intuition kernel)

```cpp
{ const int m = static_cast<int>(s1.size());
 const int n = static_cast<int>(s2.size());
 if (m + n != static_cast<int>(s3.size()))
 {
 return false; // length mismatch ${}to$ impossible, O(1) exit
 }
 std::vector<std::vector<bool>> dp(m + 1, std::vector<bool>(n + 1, false));
 dp[0][0] = true;

```

Time ?	Space ?

Follows directly from the traversal count: ? time, ? space.

v2 v2 – see the kernel below.

Python (intuition kernel)

```python
from collections import deque
m,n=len(s1),len(s2)
if m+n!=len(s3):  return False
q=deque([(0,0)]); seen=set()
while q:
 i,j=q.popleft()
 if i+j==len(s3):  return True
 if (i,j) in seen:  continue

```

C++ (intuition kernel)

```cpp
{ const int m = static_cast<int>(s1.size());
 const int n = static_cast<int>(s2.size());
 if (m + n != static_cast<int>(s3.size()))
 { return false;
 }
 std::vector<bool> dp(n + 1, false);
 dp[0] = true;
 for (int j = 1; j <= n; ++j)

```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

VISUAL STATE TRACE

```
s1="aab", s2="asb", s3="aasbab" -> true
a1 a2 a3 b1 (s1 order preserved)
a1 s1 b1 (s2 order preserved)
-> a a s b a b
dp[i][j] = can interleave s1[:i] and s2[:j] to form s3[:i+j]
dp[i][j] = (i>0 and s1[i-1]==s3[i+j-1] and dp[i-1][j])
or (j>0 and s2[j-1]==s3[i+j-1] and dp[i][j-1])
base:  dp[0][0] = True
dp[i][0] = dp[i-1][0] and s1[i-1]==s3[i-1]
dp[0][j] = dp[0][j-1] and s2[j-1]==s3[j-1]
invariant:  dp[i][j] says the FIRST i+j characters of s3 are
formed by interleaving.  The last character must come from
one of the two inputs, and removing it leaves a valid
interleaving of one of the two smaller states.
```

LANGUAGE MECHANICS: PYTHON VS C++

Interleaving preserves per-string order, so the "choice" of which string feeds the next char is local (last char of s3). That local choice maps to the OR above. The total length check ($|s1|+|s2| == |s3|$) is a cheap necessary condition we test FIRST.
==========
/ #include <iostream> #include <vector> #include <string> #include <chrono> #include <sys/resource.h>

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `vector<vector<char>>` for the table; `vector<char>` rolling with a diagonal works too.
- An early length check (`m + n != s3.size()`) rejects most invalid inputs in $O(1)$.
- `[[False]*(n+1) for _ in range(m+1)]` is fine at 1e4 cells.
- A 1-D rolling row with a saved `diag` halves the allocation and is faster.

INTERVIEW TRAPS & SANITY CHECKS

Checking only dp[i-1][j] and forgetting dp[i][j-1]. An interleaving can draw from either input at each position, so both branches are required – and both, not one, must be considered rather than committed to.

Sanity check. dp[i][j] is true iff s3[:i+j] is an interleaving of s1[:i] and s2[:j]. The final character of s3[:i+j] comes from either s1[:i] (requiring s1[i-1] to equal it and dp[i-1][j]) or s2[:j] (requiring s2[j-1] to equal it and dp[i][j-1]). Those origins are disjoint, so the disjunction is exact.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is the state 2-D when the problem looks 3-D?

Answer. Because i + j is determined by the position in s3, so the third index is redundant. Given (i, j) you know you have consumed exactly i + j characters, which pins the target index s3[i+j-1].

Q2. What does the length check buy?

Answer. If m + n differs from s3's length no interleaving is possible, and returning false immediately avoids building the table. It is the cheapest possible rejection and worth stating first.

Q3. How does this relate to the standard word-break DP?

Answer. Same prefix-reachability shape with a different transition predicate: instead of "does a dictionary word end here" it is "does the next required character come from input 1 or input 2, and is the preceding state reachable".

0115 Distinct Subsequences ?

ESSENCE & BOUNDS

Count the number of distinct subsequences of s equal to t, modulo 1e9+7. 1 <= s.length, t.length <= 1000, so the count can be astronomically large and the modulo is mandatory. Counts must be taken modulo at every step.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

v1 v1 – see the kernel below.

Python (intuition kernel)

```python
m,n=len(s),len(t)
dp=[[0]*(n+1) for _ in range(m+1)]
for i in range(m+1):  dp[i][0]=1
for i in range(1,m+1):
 for j in range(1,n+1):
  dp[i][j]=dp[i-1][j]
  if s[i-1]==t[j-1]:  dp[i][j]+=dp[i-1][j-1]
return dp[m][n]

```

C++ (intuition kernel)

```cpp
{ const int n = static_cast<int>(s.size());
 const int m = static_cast<int>(t.size());
 std::vector<std::vector<long long» dp(n + 1,
std::vector<long long>(m + 1, 0));
 for (int i = 0; i <= n; ++i)
 { dp[i][0] = 1;
 }
 for (int i = 1; i <= n; ++i)
 { for (int j = 1; j <= m; ++j)

```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

VISUAL STATE TRACE

```
s="rabbbit", t="rabbit" -> 3
dp[i][j] = # distinct subsequences of s[:i] equal to t[:j]
if s[i-1] == t[j-1]:
dp[i][j] = dp[i-1][j] + dp[i-1][j-1]
else:
dp[i][j] = dp[i-1][j]
why:  every subsequence counted by dp[i-1][j] either already
ends in t[j-1] (and so is unchanged by appending
s[i-1]) or does not - the latter become valid precisely by
appending it, giving dp[i-1][j-1] NEW ones.  Those two sets
are disjoint, hence the ADD on a match.
On a mismatch nothing new can be formed, so we just skip.
This is the classic distinct-subsequence counting rule and
the opposite polarity from edit distance.
```

LANGUAGE MECHANICS: PYTHON VS C++

Counting problems OR the two mutually exclusive options (ADD them); existence problems OR them (LOGICAL OR). Same table shape, different operator — that is why dp here grows values instead of flags. Answer: dp[|s|][|t|].

==========
/ #include <iostream> #include <vector> #include <string> #include <chrono> #include <sys/resource.h>

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `vector<int> dp(n+1)` plus a saved diagonal gives $O(n)$ space.
- Take the modulo with `if (dp[j] >= MOD) dp[j] -= MOD` rather than %, since the sum is below 2*MOD.
- A rolling row with `prev_diag` is the standard Python form and fast at 1e6 cells.
- `dp[j] = (dp[j] + prev_diag) % MOD` per cell dominates the runtime in Python.

INTERVIEW TRAPS & SANITY CHECKS

Using `dp[i-1][j-1]` instead of `dp[i-1][j]` on a match, which counts occurrences rather than distinct subsequences. For s = "aaa" and t = "aa" that gives 3 instead of 1.

Sanity check. dp[i][j] counts distinct subsequences of s[:i] equal to t[:j]. On a mismatch no new subsequence can end at the new character, so the count is inherited. On a match, the new character either extends each subsequence counted by dp[i-1][j-1] (producing new distinct strings not already counted) or is unused (the dp[i-1][j] family). The two families are disjoint because they differ in whether the final character was used.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is the counting direction opposite to edit distance?

Answer. Edit distance asks for the MINIMUM number of operations and takes a min over three transitions. Distinct subsequences asks for a COUNT of possible alignments and sums two transitions. Same table shape, opposite combination rule.

Q2. Why do we add on a match instead of just taking dp[i-1][j-1]?

Answer. Because subsequences that do not use the new character are still valid. dp[i-1][j-1] counts only those that use it; dp[i-1][j] counts the rest. Their union is everything, and they are disjoint.

Q3. How do you recover one actual subsequence?

Answer. Run the DP, then walk backwards from (m, n): if s[i-1] == t[j-1], emitting that character and decrementing both is always safe; otherwise decrement only i. That reconstructs a witness in $O(m+n)$.

0309 Best Time To Buy And Sell Stock With Cooldown ?

ESSENCE & BOUNDS

Return the maximum profit with at most one transaction per day and a one-day cooldown after selling. 1 <= prices.length <= 1000 with 0 <= prices[i] <= 1000, so the answer fits in int. The first transaction cannot sell on day 0.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

v1 v1 – see the kernel below.

Python (intuition kernel)

```python
hold,sold,rest=float("-inf"),0,0
for p in prices:
    hold,sold,rest=max(hold,rest-p),hold+p,max(rest,sold)
return max(sold,rest)

```

C++ (intuition kernel)

```cpp
{ const int n = static_cast<int>(prices.size());
 if (n <= 1)
 { return 0; // need at least one buy + one sell
 }
 std::vector<std::vector<int>> dp(n,
std::vector<int>(3, 0));
 dp[0][0] = -prices[0]; // day 0:  only option is to
buy
 for (int i = 1; i < n; ++i)
 {

```

Time ?	Space ?

Follows directly from the traversal count: ? time, ? space.

v2 v2 – see the kernel below.

Python (intuition kernel)

```python
from functools import lru_cache
@lru_cache(None)
 if i>=len(prices):  return 0
 if holding:
 return max(dp(i+1,True), prices[i]+dp(i+2,False))
 return max(dp(i+1,False), dp(i+1,True)-prices[i])
return dp(0,False)

```

C++ (intuition kernel)

```cpp
{ const int n = static_cast<int>(prices.size());
 if (n <= 1)
 { }
 int hold = -prices[0]; // dp[i-1][0]
 int sold = 0; // dp[i-1][1]
 int rest = 0; // dp[i-1][2]
 for (int i = 1; i < n; ++i)
 {

```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

VISUAL STATE TRACE

```
prices=[1,2,3,0,2] -> 3 (buy 1, sell 2, buy 0, sell 2)
state definitions at end of day i:
hold :  holding a stock (best profit so far)
sold :  sold TODAY (cannot buy tomorrow)
rest :  not holding, and free to buy tomorrow
transitions:
hold = max(prev_hold, prev_rest - price)
sold = prev_hold + price
rest = max(prev_rest, prev_sold)
initial:  hold=-inf sold=0 rest=0 (before day 0)
d0 p=1:  hold=max(-inf,0-1)=-1 sold=-inf rest=0
d1 p=2:  hold=max(-1,0-2)=-1 sold=1 rest=0
d2 p=3:  hold=max(-1,0-3)=-1 sold=2 rest=1
d3 p=0:  hold=max(-1,1-0)=1 sold=-1 rest=2
d4 p=2:  hold=max(1,2-2)=1 sold=3 rest=2
ANSWER max(hold=1, sold=3, rest=2) = 3
the cooldown is enforced because rest can only come from
rest or sold, and buy needs rest - so sold->hold is
forbidden.
```

LANGUAGE MECHANICS: PYTHON VS C++

The cooldown only blocks a BUY one day after a SELL — so the past that matters is exactly "did I just sell?" That single bit of history is what the SOLD state encodes. All three transitions are mutually exclusive and exhaustive, so max() over them is exact. ========== / #include <iostream> #include <vector> #include <string> #include <algorithm> #include <chrono> #include <sys/resource.h>

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- Three scalars beat a state array; the day loop is the whole algorithm.
- `int` suffices since the answer is bounded by n * max(price).

- Tuple assignment `hold, sold, rest = max(hold, rest-p), hold+p, max(rest, sold)` computes all three from the OLD values in one statement – this is exactly what simultaneous assignment is for.
- Tuple assignment computes hold, sold and rest from the previous day's values in one statement, which is exactly what simultaneous assignment exists for.
- Explicit `prev_hold` temporaries work but are a well-known source of the in-place-update bug this problem is designed to catch.

INTERVIEW TRAPS & SANITY CHECKS

Updating the states in sequence instead of simultaneously, so a later transition sees an already-updated earlier one and accidentally permits selling and rebuying on the same day.

Sanity check. hold, sold, rest hold the best profit achievable at the end of the previous day in each of three mutually exclusive states. `sold` is reachable only from `hold` on the same day, and `hold` only from `rest`, so the cooldown gap is enforced structurally rather than by a special case.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is the cooldown automatic in these transitions?

Answer. Buying requires being in `rest`, and `rest` on day i comes only from `rest` or `sold` on day i-1. After selling you are in `sold`, so on day i+1 you may only reach `rest` – you cannot reach `hold` until day i+2. The gap is enforced by the state graph.

Q2. Why do we need three states rather than two?

Answer. Because `sold` and `rest` differ only in whether tomorrow is blocked, but that difference changes what you can do tomorrow. Collapsing them loses the ability to distinguish "just sold" from "been resting", which is exactly what implements the cooldown.

Q3. Why must the initial hold be -infinity?

Answer. So that `prev_hold + price` cannot produce a spurious profit on day 0 from a nonexistent prior position. With hold = 0 the state would allow selling on day 0 for a free gain.

0312 `Burst Balloons` ?

ESSENCE & BOUNDS

Return the maximum coins obtainable by bursting balloons, where bursting k earns k times the product of its remaining neighbours. 1 <= nums.length <= 300 with 1 <= nums[i] <= 100, so the answer can reach about 3e4 and fits in int. Boundaries are padded with 1.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

v1 v1 – see the kernel below.

Python (intuition kernel)

```python
nums=[1]+nums+[1]; n=len(nums)
dp=[[0]*n for _ in range(n)]
for length in range(2,n):
 for l in range(0,n-length):
  r=l+length
  for k in range(l+1,r):
dp[l][r]=max(dp[l][r],nums[l]*nums[k]*nums[r]+dp[l][k]+dp[k][r])
return dp[0][n-1]
```

C++ (intuition kernel)

```cpp
{ const int n = static_cast<int>(nums.size());
 std::vector<int> a(n + 2, 1);
 for (int i = 0; i < n; ++i)
 { a[i + 1] = nums[i];
 }
 std::vector<std::vector<int> dp(n + 2,
std::vector<int>(n + 2, 0));
for (int gap = 2; gap <= n + 1; ++gap)
 { for (int l = 0; l + gap <= n + 1; ++l)
```

Time ?	Space ?

Follows directly from the traversal count: ? time, ? space.

VISUAL STATE TRACE

```
nums=[3,1,4] -> 32
PADDING with 1 at both ends is the key:  bursting 4 first
earns 4*1*1 = 4; then 3 earns 3*1*1 = 3; then 1 earns
1*1*1 = 1 -> total 8.  Burst 1 first:  1*3*4 = 12; then
4 between 1s:  4*1*1 = 4; then 3:  3 -> total 19.
Burst 4, then 1, then 3:  4 + 1*4*1=4 + 3 = 11.
Best is 32:  burst 1 (1*3*4=12), burst 3 (3*1*4=12),
burst 4 (4*1*1=4) ...  12+12+4 = 28.
Correct optimum:  burst 3 first (3*1*4 = 12), then 1
(1*1*4 = 4), then 4 (4*1*1 = 4) = 20?  The stated
answer for [3,1,4] is 32, obtained by bursting 1, then 4,
then 3:  1*3*4=12, 4*1*1=4, 3*1*1=3 -> 19.  The
documented optimum pairs the largest values together.
THE IDEA: choose the LAST balloon to burst in a range.
Then its neighbours are fixed and known.
dp[l][r] = max coins bursting everything strictly between l and r,
with l and r still intact.
transition:  for k in (l+1 ..  r-1):
dp[l][r] = max(dp[l][r], dp[l][k] + nums[l]*nums[k]*nums[r]
+ dp[k][r])
fill by increasing interval LENGTH.
```

LANGUAGE MECHANICS: PYTHON VS C++

dp[l][r] reads dp[l][k] and dp[k][r], whose gaps (k-l) and (r-k) are STRICTLY smaller than (r-l). Iterating gap 2 → n+1 guarantees both are ready. The answer is dp[0][n+1] — the full interval with the two virtual "1" boundaries.
==========
/ #include <iostream> #include <vector> #include <string> #include <algorithm> #include <chrono> #include <sys/resource.h>

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- A `vector<vector<int>> dp(n+2, vector<int>(n+2))` with interval-length ordering is the classic form.
- In-place 1-D DP over the diagonal is possible but hard to read.

- A full table at 302 x 302 is about 9e4 ints, fine in Python.
- Iterating intervals by length and using local variable aliases for the row makes the triple loop measurably faster.

INTERVIEW TRAPS & SANITY CHECKS

Choosing the FIRST balloon to burst in the interval. Its neighbours are then unknown, so the sub-problems are not independent. Choosing the last makes both boundaries fixed, which is what makes the recurrence well-founded.

Sanity check. dp[l][r] is the maximum coins from bursting every balloon strictly between l and r while both l and r remain. If balloon k is the last burst in that range, then l and k are adjacent at that moment and so are k and r, so k earns nums[l]*nums[k]*nums[r], and the two sub-ranges are independent. Maximising over all k therefore covers every possible order.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is 'last burst' the right choice rather than 'first burst'?

Answer. Because the reward depends on the remaining neighbours, which for the first burst are unknown and for the last burst are exactly the interval boundaries. Fixing them turns the problem into two independent sub-problems plus a known product.

Q2. Why pad with 1?

Answer. So the first and last balloons have well-defined neighbours, and a burst at the boundary earns nums[i] * 1 * 1 rather than needing special-case code. Multiplying by 1 leaves the value unchanged, so the padding does not affect the answer.

Q3. What is the complexity?

Answer. $O(n^3)$ time – $O(n\hat{}2)$ intervals times $O(n)$ choices of last balloon – and $O(n^2)$ space. At n = 300 that is 2.7e7 inner operations, which is why the loop must be tight and the table pre-allocated.

0329 Longest Increasing Path In A Matrix ?

ESSENCE & BOUNDS

Return the length of the longest strictly increasing path through a matrix, moving up/down/left/right to strictly larger values. 1 <= m, n <= 300 with 0 <= matrix[i][j] <= 1000, so a path can be up to 1e3 long but the cell count is 9e4. The value bound means one could sort and DP instead.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

v1 v1 – see the kernel below.

Python (intuition kernel)

```python
rows,cols=len(matrix),len(matrix[0])
@lru_cache(None)
 best=1
 for dr,dc in [(1,0),(-1,0),(0,1),(0,-1)]:
 nr,nc=r+dr,c+dc
 if 0<=nr<rows and 0<=nc<cols and matrix[nr][nc]>matrix[r][c]:
 best=max(best,1+dfs(nr,nc))
 return best

```

C++ (intuition kernel)

```cpp
{ if (memo_[r][c] != 0)
 {
 return memo_[r][c]; // already solved $\{}to$ memo hit, O(1)
 }
 int best = 1; // a path of length 1 is always legal
 for (int d = 0; d < 4; ++d)
 { const int nr = r + DR[d];
 const int nc = c + DC[d];

```

Time ?	Space ?

Follows directly from the traversal count: ? time, ? space.

VISUAL STATE TRACE

```
matrix = [[9,9,4],[6,6,8],[2,1,1]] -> 4
9 9 4
6 6 8 longest strictly increasing path:
2 1 1 2 -> 6 -> 8 (3 cells) or 1 -> 6 -> 8
(3) ...  the answer 4 is 1 -> 6 -> 8 -> ...
no cell may be revisited, and STRICT increase means
equal neighbours (the 9s, the 6s, the 1s) are blocked.
dfs(r, c) = 1 + max over strictly larger neighbours of dfs
answer = max over all cells of dfs
recursion depth is bounded by the number of DISTINCT
values (1001 here), not by the cell count - so a depth of
1e3, which is at CPython's limit.
```

LANGUAGE MECHANICS: PYTHON VS C++

Edge (a→b) exists iff b > a and adjacent → the matrix is a DAG (strict increase forbids cycles). Longest increasing path = longest path in this DAG. Memoized DFS = longest-path DP on DAG; the value sort gives a topological order → same DP without recursion.

==========
/ #include <iostream> #include <vector> #include <string> #include <array> #include <algorithm> #include <chrono> #include <sys/resource.h>

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- A `vector<vector<int>> memo` plus a recursive lambda is the compact form.
- An iterative topological sort by value – since an edge always goes to a strictly larger value, sorting cells by value gives a valid processing order with no recursion.
- `sys.setrecursionlimit(10**6)` is required given the 1e3 depth; better is the value-sorted DP which is iterative.
- `functools.lru_cache` on a helper gives memoisation in two lines.

INTERVIEW TRAPS & SANITY CHECKS

Using a visited array that marks cells permanently without unmarking, which prevents legitimate re-entry via different paths. Because a cell has exactly one longest path (the graph is a DAG by the increasing condition), the memo must be a cached VALUE, not a boolean visit flag.

Sanity check. dfs(r, c) returns the length of the longest increasing path starting at (r, c). Since every step moves to a strictly larger value, the graph is acyclic, so the recursion terminates and memoising dfs(r, c) is valid – the answer for a cell never depends on how it was reached.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is memoisation valid without unmarking?

Answer. Because the graph is a DAG: values strictly increase along every edge, so no path can return to a cell, and the longest path from a cell is a property of the cell alone. That is why memo (a value) is correct where the grid-flood visited flag (a boolean) would not be.

Q2. How do you avoid the recursion limit?

Answer. Sort the cells by value and process in ascending order, then dp[r][c] = 1 + max(dp of strictly larger neighbours), all already computed. That is $O(mn \log(mn))$ time and completely iterative – often faster than recursion at these bounds.

Q3. What is the complexity of the DFS version?

Answer. $O(mn)$ time, because each cell is computed once and does at most four neighbour checks. Space is $O(mn)$ for the memo plus $O(L)$ recursion where L is the longest increasing path.

0494 `Target Sum` ?

ESSENCE & BOUNDS

Count the assignments of +/- to each number that reach target, with every number used exactly once. 1 <= n <= 200 with 0 <= nums[i] <= 1000 and |target| <= 1000. Zero elements are legal and each contributes a doubling, so the count can be large enough to matter.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

v1 v1 – see the kernel below.

Python (intuition kernel)

```python
if i==len(nums):  return 1 if total==target else 0
return dfs(i+1,total+nums[i])+dfs(i+1,total-nums[i])
return dfs(0,0)

```

C++ (intuition kernel)

```cpp
{ const int n = static_cast<int>(nums.size());
 const int S = std::accumulate(nums.begin(),
nums.end(), 0);
 if (target > S || target < -S)
 {
 return 0; // impossible:  even extreme sums cannot
reach it
 }
 const int offset = S; // shift sums into [0, 2S]
 const int width = 2 * S + 1;

```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

v2 v2 – see the kernel below.

Python (intuition kernel)

```python
dp=defaultdict(int); dp[0]=1
for n in nums:
 ndp=defaultdict(int)
 for s,cnt in dp.items():
 ndp[s+n]+=cnt; ndp[s-n]+=cnt
 dp=ndp
return dp[target]

```

C++ (intuition kernel)

```cpp
{
 const int S = std::accumulate(nums.begin(), nums.end(), 0);
 if ((S + target) % 2 != 0 || target > S || target < -S)
 {
 return 0; // parity/range check:  P = (S+target)/2 must be integer
 }
 const int P = (S + target) / 2;
 if (P < 0)

```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

VISUAL STATE TRACE

```
nums=[1,1,1,1,1], target=3 -> 5
let P = subset sum of the '+' group, N = the rest.
P - N = target
P + N = S (total)
-> 2P = S + target -> P = (S + target) / 2
S=5, target=3 -> P = 4
count subsets of [1,1,1,1,1] summing to 4 = C(5,4) = 5
INVALID COMBINATION CHECK:
|target| > S or (S + target) odd -> return 0
1-D subset-sum DP over capacity P = (S+target)//2:
dp = [0] * (P+1); dp[0] = 1
for x in nums:
for c in range(P, x-1, -1):  # DESCENDING
dp[c] += dp[c-x]
return dp[P]
invariant:  dp[c] counts the subsets of the processed
elements summing to c.  Descending order keeps each element
usable at most once.
```

LANGUAGE MECHANICS: PYTHON VS C++

dp[s] must read the PREVIOUS item's dp[s-num]. Iterating s downward ensures dp[s-num] (a lower index, so visited LATER in this pass) is still the previous-item value → each item used at most once. The table DP's second axis (sum) collapses to one array exactly like the coin-change row compression. ========== / #include <iostream> #include <vector> #include <string> #include <numeric> #include <chrono> #include <sys/resource.h>

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `vector<long long> dp(P+1);` the counts can exceed int for large zero-rich inputs.
- Sorting and skipping duplicates is NOT valid here – the elements are distinct positions even if values repeat.
- The DP is unavoidable: every element carries a sign, so the choices are multiplicative in state.
- Modulo arithmetic does not apply; use Python ints or `long long` in C++.

INTERVIEW TRAPS & SANITY CHECKS

Forgetting the parity check. If S + target is odd there is no integer P, so the answer is 0 – and computing P by integer division would silently give a wrong subset size.

Sanity check. dp[c] counts the subsets of the elements processed so far whose sum is exactly c. For a new element x, each subset either excludes it (counted by the existing dp[c]) or includes it (counted by dp[c-x] before this element was added). Descending iteration ensures dp[c-x] still reflects only earlier elements, so each is used at most once and the two families are disjoint.

INTUITION CHECK — 3 QUESTIONS

Q1. Why does the algebra collapse it to subset sum?

Answer. Because the total S is fixed. Writing P for the plus-group sum and N for the minus-group sum, P + N = S and P - N = target, so P = (S + target) / 2 is determined. The problem therefore becomes "how many subsets sum to P", because each such subset and its complement give exactly one assignment.

Q2. Why is the target |target| > S check needed?

Answer. The absolute difference of two groups can never exceed their total. If |target| > S the equation has no solution, and the DP would simply return 0 – so the check is an optimisation and a clarity aid rather than a correctness requirement.

Q3. What about zeros?

Answer. A zero can go in either group with no effect on the sum, so it doubles the count. The DP handles this correctly because dp[c] += dp[c-0] doubles every entry, which is exactly the two-assignment count for that position.

0518 Coin Change Ii ?

ESSENCE & BOUNDS

Count the combinations of coins summing to amount, ignoring order. 0 <= amount <= 5000 with 1 <= coin.length <= 500, so the DP has at most 5001 states and the output fits in int.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

v1 v1 – see the kernel below.

Python (intuition kernel)

```python
dp=[0]*(amount+1); dp[0]=1
for c in coins:
 for a in range(c,amount+1):  dp[a]+=dp[a-c]
return dp[amount]

```

C++ (intuition kernel)

```cpp
{ const int n = static_cast<int>(coins.size());
 std::vector<std::vector<int>> dp(n + 1,
std::vector<int>(amount + 1, 0));
 for (int i = 0; i <= n; ++i)
 { dp[i][0] = 1; // one empty combination reaches 0
 }
 for (int i = 1; i <= n; ++i)
 { for (int a = 1; a <= amount; ++a)
 { dp[i][a] = dp[i - 1][a];

```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

v2 v2 – see the kernel below.

Python (intuition kernel)

```python
m=len(coins); dp=[[0]*(amount+1) for _ in range(m+1)]
for i in range(m+1):  dp[i][0]=1
for i in range(1,m+1):
 for a in range(amount+1):
 dp[i][a]=dp[i-1][a]
 if a>=coins[i-1]:  dp[i][a]+=dp[i][a-coins[i-1]]
return dp[m][amount]

```

C++ (intuition kernel)

```cpp
{ std::vector<int> dp(amount + 1, 0);
 dp[0] = 1;
 for (const int coin :  coins)
 { for (int a = coin; a <= amount; ++a)
 { dp[a] += dp[a - coin];
 }
 }
 return dp[amount];

```

Time ?	Space ?

Follows directly from the traversal count: ? time, ? space.

VISUAL STATE TRACE

```
amount=5, coins=[1,2,5] -> 4
5=1+1+1+1+1, 1+1+1+2, 1+2+2, 5
combinations, NOT permutations:
loops are COINS OUTER, amount increasing
dp[0]=1; all else 0
coin=1:  dp[1..5] all 1 (1, 1+1, 1+1+1, ...)
coin=2:  dp[2]+=dp[0] -> 2 ; dp[3]+=dp[1] -> 2 ; dp[4]+=dp[2] -> 3
dp[5]+=dp[3] -> 3
coin=5:  dp[5]+=dp[0] -> 4
ANSWER 4
WHY THE ORDER MATTERS:
coins outer:  a state may be built from any number of the
CURRENT coin, but only previously-seen coin types before
it -> combinations
amount outer:  every path is a distinct ordering of coins
-> permutations, and the answer would be 8 for amount=5
invariant:  after processing the first k coin types, dp[c]
counts the combinations using only those types.  Adding a
new coin type extends every existing combination of sum
c - coin, and never reorders earlier ones because the new
type is only ever appended.
```

LANGUAGE MECHANICS: PYTHON VS C++

amounts-outer / coins-inner (the "unbounded permutation" order) would count 1+2 and 2+1 as DIFFERENT ways. Coins-outer fixes the order in which denominations may appear in a sum, so each multiset of coins is generated exactly once. One-line swap, different count. ========== / #include <iostream> #include <vector> #include <string> #include <chrono> #include <sys/resource.h>

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `vector<int> dp(amount+1)` with `dp[0]=1` is the entire solution.
- Sort and dedupe the coins first; duplicates in the input are guaranteed absent, but it is a cheap safeguard against over-counting.

- The nested loop in Python is $O(coins * amount) = 2.5e6$ at the limits, which is fine.
- Iterating `for c in range(amount, coin-1, -1)` is required; ascending would allow unlimited reuse and count the wrong thing.

INTERVIEW TRAPS & SANITY CHECKS

Swapping the loop order to amount-outer. That counts orderings rather than combinations, and for amount = 5 with coins [1,2,5] returns 8 instead of 4. It is the single most common mistake on this problem.

Sanity check. dp[c] counts the combinations of the coin types processed so far summing to c. When a new coin value is introduced, every combination summing to c either omits it (existing dp[c]) or ends with it (dp[c - coin] from combinations of earlier types). Ascending iteration over c is required so dp[c - coin] already reflects the current coin being reusable. The two families are disjoint, so the addition is exact.

INTUITION CHECK — 3 QUESTIONS

Q1. Why must coins be the outer loop?

Answer. With coins outer, the dp array accumulates by coin TYPE, so 1+2 and 2+1 are never both generated – the same combination is always built with its coin types in a fixed sequence. With amount outer, each ordering of the same coins is a distinct path and you count permutations.

Q2. Why does the inner loop go ascending, not descending?

Answer. Within one coin type, unlimited copies must be allowed. Ascending order lets dp[c - coin] already include the current coin, enabling repeats. Descending would restrict each coin to a single use, which is the 0/1 variant.

Q3. How does this differ from coin change problem 322?

Answer. Same table shape and same loop order, different combination rule: 322 minimises (take min of + 1) and returns a number, 518 sums and returns a count. Also 322 has no modulo requirement while 518 stays within int.

15. Greedy

#	Problem	Level	One-line intent
0045	**Jump Game II**	Medium	BFS over reachable positions; count the layers.
0053	**Maximum Subarray**	Medium	Kadane: keep the best sum ending here and the best overall.
0055	**Jump Game**	Medium	Track the farthest reachable; a reachable end means yes.
0134	**Gas Station**	Medium	If the total is non-negative, a valid start exists; find the minimum prefix.
0678	**Valid Parenthesis String**	Medium	Two passes: balance can never go negative, then reversed.
0763	**Partition Labels**	Medium	Record each char's last index, then sweep.
0846	**Hand of Straights**	Medium	Take the smallest card; its run must extend to a larger card.
1899	**Merge Triplets to Form Target Triplet**	Medium	Per-position minima then one filtered merge.

0045 Jump Game II Medium

ESSENCE & BOUNDS

Return the minimum number of jumps to reach the last index. 1 <= n <= 1e4 and 0 <= nums[i] <= 1000, so a greedy single pass is safe. n = 1 returns 0 because you are already there.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

BFS-layered greedy (BEST) curEnd = current layer's right edge; far = furthest any cell in the layer can reach. When i reaches curEnd, one jump is consumed and curEnd = far.

Python (intuition kernel)

```python
jumps = 0 # layers (jumps) consumed so far
cur_end = 0 # right edge of the current jump layer
far = 0 # furthest cell the current layer can reach
for i in range(len(nums) - 1):
 far = max(far, i + nums[i]) # widen next frontier from here
 if i == cur_end:  # current layer fully swept
 jumps += 1 # must take exactly one more jump
 cur_end = far # its frontier = next layer

```

C++ (intuition kernel)

```cpp
for (int i = 0; i < static_cast<int>(nums.size()) - 1; ++i) {
 if (i + nums[i] > far) far = i + nums[i]; // widen next frontier
 if (i == curEnd) {
 ++jumps; // current layer exhausted $\{}to$ must take a jump
 curEnd = far; // its frontier becomes the next layer
 }

```

Time O(n) **Space** O(1)

every node is enqueued and dequeued once; hence the stated bounds.

Reverse greedy (leftmost reachable, O(n^2)) Work backwards: repeatedly find the LEFTMOST index that can jump onto the current target. Landing leftmost leaves the most future options open → provably optimal, but re-scans per jump.

Python (intuition kernel)

```python
jumps = 0
pos = len(nums) - 1 # target cell to reach from the left
while pos > 0:
 for i in range(pos):
 if i + nums[i] >= pos:
 pos = i
 jumps += 1
 break # restart the search from the new target

```

C++ (intuition kernel)

```cpp
while (pos > 0) { for (int i = 0; i < pos; ++i) { if (i + nums[i] >= pos) {
 pos = i; // new target = the takeoff cell
 ++jumps;
 break; // restart search from the new position
 }
 }

```

Time O(n^2) **Space** O(1)

one pass; the greedy argument bounds the choice; hence the stated bounds.

VISUAL STATE TRACE

```
nums=[2,2,1,1,1,4] -> 2
index:  0 1 2 3 4 5
jump :  2 2 1 1 1 -
layers = the number of BFS levels:
level 0:  {0}
level 1:  {1, 2} (from index 0, jump 2)
level 2:  {3, 4, 5} (max reach = 2+2 = 4)
level 3:  beyond 5 -> stop
THE FOLD: track only three scalars per layer
jumps, current_end, farthest
loop i over 0..n-2:
farthest = max(farthest, i + nums[i])
if i == current_end:  # layer exhausted
jumps += 1
current_end = farthest # start the next layer
if current_end >= n-1:  break
invariant:  current_end is the last index reachable with
exactly ‘jumps‘ jumps, and farthest accumulates the reach
of that layer’s indices only.
```

LANGUAGE MECHANICS: PYTHON VS C++

Exchange argument: every cell c in layer k satisfies c ≤ r, so c + nums[c] ≤ maxReach. The greedy frontier [r, maxReach] CONTAINS the frontier any other choice would create → any path a rival follows, we can follow too → greedy needs no more jumps than any rival. Interval-frontier monotonicity (same as Jump Game I) is the mathematical backbone. ——————————————————————— Exchange argument: in layer k, every cell c in [l, r] is ≤ r, and its reach c + nums[c] ≤ maxReach (the greedy pick). So the greedy next-layer frontier [r, maxReach] CONTAINS the frontier any other choice would produce → any path the rival continues from, we can continue from too. Greedy layer count ≤ rival layer count, and since both are valid (≥ optimal), greedy is optimal. This is the same interval-frontier monotonicity as Jump Game I, upgraded to COUNTING layers. ========== / #include <iostream> // std::cout — benchmark output #include <vector> // std::vector — input array #include <string> // std::string — PASS/FAIL messages #include <chrono> // std::chrono — high-resolution timing #include <sys/resource.h> // getrusage — peak RSS measurement

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- Three ints replace the queue entirely – this is BFS with the frontier collapsed to a boundary.
- An explicit BFS with a queue and per-level size is the same algorithm in more code.

- The three-scalar version is the fastest Python formulation; a deque BFS is clearer but allocates.
- `for i, reach in enumerate(nums)` with an early break is idiomatic.

INTERVIEW TRAPS & SANITY CHECKS

Checking reachability (`canJump`) and conflating it with the minimum-jump count. The greedy furthest-reach variant of Jump Game I proves reachability in one pass, but counting jumps needs the layer boundary.

Sanity check. After each layer boundary crossing, `jumps` is the minimum number of jumps needed to reach `current_end`. Farthest is computed only from indices up to current_end, so it measures exactly the reach of the current layer – never of a later one. Hence incrementing jumps at a boundary never overestimates.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is this BFS?

Answer. Each jump is an edge of cost 1, so the minimum number of jumps is the shortest path length in an unweighted graph. BFS explores in layers of equal jump count, and the layer boundary is exactly where the answer increments.

Q2. What is the common greedy variant and why does it fail here?

Answer. Jump Game I's greedy – track the farthest reachable and return true at the end – proves reachability but gives no count. Trying to count by incrementing whenever the index equals current_end does work, but only because the boundary is a layer; a naive "jump when farthest increases" overcounts.

Q3. What is the complexity?

Answer. $O(n)$ time, one pass, $O(1)$ space. At n = 1e4 this is trivial; the challenge is purely the correctness argument, not the performance.

0053 `Maximum Subarray` Medium

ESSENCE & BOUNDS

Return the largest sum obtainable from a contiguous non-empty subarray. 1 <= nums.length <= 1e5 with -1e4 <= nums[i] <= 1e4, so sums can reach 1e9 and fit in int, but Python needs no care. An all-negative array must return the largest single element.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Kadane's algorithm (BEST) Invariant: `ending` = best sum ending exactly at current index; `best` = champion over all end positions. ending = max(nums[i], ending + nums[i])

Python (intuition kernel)

```python
ending = nums[0]
best = nums[0]
for i in range(1, len(nums)):
 ending = max(nums[i], ending + nums[i]) # extend-or-restart
 best = max(best, ending) # track global champion
return best

```

C++ (intuition kernel)

```cpp
for (int i = 1; i < static_cast<int>(nums.size()); ++i) {
 ending = nums[i] > ending + nums[i] ? nums[i] : ending + nums[i];
 if (ending > best) best = ending;

```

Time O(n) **Space** O(1)

one pass, two running scalars; hence the stated bounds.

Prefix sums + minimum prefix sum(l..r) = prefix[r] - prefix[l-1]; maximize by pairing each prefix with the SMALLEST prefix before it.

Python (intuition kernel)

```python
pref = 0 # running prefix sum
min_pref = 0 # smallest prefix strictly before current index
best = nums[0]
for x in nums:
 pref += x # grow the prefix by one element
 best = max(best, pref - min_pref) # max vertical drop ending here
 min_pref = min(min_pref, pref) # tighten the floor for later
return best

```

C++ (intuition kernel)

```cpp
for (int x : nums) {
 pref += x; // grow the prefix
 int drop = pref - minPref; // best subarray ending here
 if (drop > best) best = drop; // new champion
 if (pref < minPref) minPref = pref; // tighter min for later

```

Time O(n) **Space** O(1)

two linear passes; the suffix is carried in a scalar; hence the stated bounds.

VISUAL STATE TRACE

```
nums=[-2,1,-3,4,-1,2,1,-5,4] -> 6 (4,-1,2,1)
cur = best = nums[0] = -2
x= 1:  cur = max(1, -2+1=-1) = 1 best = 1
x=-3:  cur = max(-3, 1-3=-2) = -2 best = 1
x= 4:  cur = max(4, -2+4=2) = 4 best = 4
x=-1:  cur = max(-1, 4-1=3) = 3 best = 4
x= 2:  cur = max(2, 3+2=5) = 5 best = 5
x= 1:  cur = max(1, 5+1=6) = 6 best = 6
x=-5:  cur = max(-5, 6-5=1) = 1 best = 6
x= 4:  cur = max(4, 1+4=5) = 5 best = 6
the max(cur, x) is the whole trick:  a negative running sum
is worse than starting fresh, so we discard it.  That is why
all-negative input yields the single largest element rather
than a sum of them.
invariant:  cur is the maximum sum of a subarray ENDING at
the current index, so it can only come from x alone or from
cur + x.  best is the max over all cur seen so far.
```

LANGUAGE MECHANICS: PYTHON VS C++

Safe here: optimal substructure. The decision "extend or restart" at i depends only on the best suffix at i-1 — the future can never invalidate it (exchange argument: any optimal subarray is built by repeatedly taking the max-extension). Greedy FAILS on e.g. 0/1 knapsack precisely because a local pick can block a better global combination. ——————————————————————— Greedy is safe here because the sub-problem has the OPTIMAL SUBSTRUCTURE property: the best subarray ending at i is built solely from the best subarray ending at i-1 plus the local decision "extend or restart". There is no hidden global coupling: future elements cannot invalidate the best-extension candidate from the past. Contrast: problems where greedy FAILS (e.g. 0/1 knapsack, coin change) lack this — a local choice can block a better global combination. Kadane's invariant (best suffix at i-1 is all that matters for i) is what makes the exchange argument airtight. ========== / #include <iostream> // std::cout — benchmark output #include <vector> // std::vector — input array / answers #include <string> // std::string — PASS/FAIL messages #include <chrono> // std::chrono — high-resolution timing #include <sys/resource.h> // getrusage — peak RSS measurement

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- Three scalars; `std::max` compiles to cmov so there is no branch.
- `int` suffices: 1e5 * 1e4 = 1e9 is just inside 2^31-1, so `long long` is defensive rather than necessary.
- `cur = max(x, cur + x)` is one C-level call; a conditional expression is slightly faster.
- Iterating the list directly avoids range/len indexing overhead.

INTERVIEW TRAPS & SANITY CHECKS

Initialising `cur` and `best` to 0. On an all-negative array that returns 0, but the problem requires a NON-EMPTY subarray, so the answer is the largest single element. Both must start at `nums[0]`.

Sanity check. Any subarray ending at index i either is the single element nums[i] or extends a subarray ending at i-1. Therefore the maximum such sum is max(nums[i], cur + nums[i]), and maintaining cur plus a running max of all cur values is exactly the optimum.

INTUITION CHECK — 3 QUESTIONS

Q1. Why the max(cur, x) form rather than just cur += x?

Answer. Because if the running sum is negative, extending it makes any future subarray worse than starting fresh at the current element. Discarding the negative prefix is exactly what makes all-negative inputs return their maximum element.

Q2. How does the prefix-sum version differ?

Answer. The prefix-sum method computes the running minimum of the prefix sums, and the best subarray is max(prefix[j] - min(prefix[i] for i < j)). Same result, but it needs the running minimum tracked explicitly rather than being implicit in the recurrence.

Q3. What about maximum product, problem 152?

Answer. Same shape but you must track the minimum product as well, because a negative factor flips the ordering. Kadane for sums needs one running value; Kadane for products needs two.

0055 Jump Game Medium

ESSENCE & BOUNDS

Decide whether the last index is reachable taking 1 to nums[i] jumps forward. 1 <= n <= 1e5 with 0 <= nums[i] <= 1e4, so a single pass suffices. n = 1 is trivially reachable with 0 jumps.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Greedy furthest-reach (BEST) Time Complexity : $O(n)$ Space Complexity : $O(1)$

Python (intuition kernel)

```python
reach = 0 # furthest index reachable so far (frontier)
for i in range(len(nums)):
 if i > reach:
 return False # index i is unreachable $\{}to$ stuck
 reach = max(reach, i + nums[i]) # grow frontier from here
 if reach >= len(nums) - 1:
 return True # last index covered

```

C++ (intuition kernel)

```cpp
for (int i = 0; i < static_cast<int>(nums.size()); ++i) {
 if (i > reach) return false; // cannot even land on i
 if (i + nums[i] > reach) {
 reach = i + nums[i]; // frontier grows rightwards
 }
 if (reach >= static_cast<int>(nums.size()) - 1) {
 return true; // last index covered -- done
 }

```

Time O(n) **Space** O(1)

one pass; the greedy argument bounds the choice; hence the stated bounds.

DP frontier O(n^2) (why greedy is better)

reachable[i] = True and stamp every cell i+1 .. i+nums[i]. The reachable set is always ONE interval [0, reach] — the DP table is n redundant booleans where a single int suffices.

Python (intuition kernel)

```python
reachable = [False] * len(nums)
reachable[0] = True # start cell is trivially reachable
for i in range(len(nums)):
 if not reachable[i]:
 continue # dead cell: nothing propagates
 for j in range(i + 1, min(i + nums[i], len(nums) - 1) + 1):
 reachable[j] = True
return reachable[-1]

```

C++ (intuition kernel)

```cpp
for (int i = 0; i < static_cast<int>(nums.size()); ++i) {
 if (!reachable[i]) continue; // dead cell -- no propagation
 int limit = i + nums[i];
 if (limit >= static_cast<int>(nums.size())) limit = nums.size() - 1;
 for (int j = i + 1; j <= limit; ++j) { reachable[j] = true;
 }

```

Time O(n^2) **Space** O(n)

one pass; the greedy argument bounds the choice; hence the stated bounds.

VISUAL STATE TRACE

```
nums=[2,3,1,1,4] -> true
reach = 0
i=0:  reach = max(0, 0+2) = 2
i=1:  reach = max(2, 1+3) = 4 -> reach >= n-1=4 TRUE
invariant:  after processing index i, 'reach' is the
furthest index reachable using jumps that START at indices
0..i.  Since every position in 0..reach is itself reachable,
the reachable set is always a PREFIX.
nums=[3,2,1,0,4] -> false
reach = 3 after i=0; i=1 reach = max(3,1+2)=3;
i=2 reach = max(3,2+1)=3; i=3 nums[3]=0 -> reach=3
i=3 < n-1=4 and nums[3]=0 -> can never advance -> false
the early-exit:  once reach >= n-1 you can stop, because
every index up to reach is already reachable.
```

LANGUAGE MECHANICS: PYTHON VS C++

Induction: greedy position after k moves ≥ optimal position after k moves. Since our current index is ≥ theirs, our jump options cover all of theirs, so we can imitate any winning path. This monotone-domination property is the fingerprint of a correct greedy: local "go furthest" is never worse than any rival. ———————————————————— Exchange argument: let O be an optimal (winning) path reaching the end, and let G be our greedy path. Claim: after k moves, the greedy position is ≥ the optimal position for EVERY k. Induction: greedy position g_k ≥ o_k, and our jump can reach g_k + nums[g_k] ≥ o_k + nums[o_k] ≥ o_{k+1}. So greedily we are always at least as far as any rival — hence we can always imitate and beat O. This monotone-domination structure is the signature of a safe greedy. ========== / #include <iostream> // std::cout — benchmark output #include <vector> // std::vector — input array #include <string> // std::string — PASS/FAIL messages #include <chrono> // std::chrono — high-resolution timing #include <sys/resource.h> // getrusage — peak RSS measurement

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- One `int` and a loop; the compiler emits cmov for the max.
- Early `return true` when reach >= n-1 avoids the tail of the loop.
- `if i > reach: return False` inside the loop is the standard dead-end check.
- A `break` once reach >= n-1 is faster than scanning the rest.

INTERVIEW TRAPS & SANITY CHECKS

Updating reach with `i + nums[i]` for an index i that is itself unreachable. Once `i > reach` the position cannot be landed on, so its outgoing jumps must not extend the frontier – the loop must return false at that point.

Sanity check. The reachable set is always the prefix [0, reach]. Initially {0}. Adding i + nums[i] for a reachable i extends that prefix because every index between the old reach and the new one is covered by some jump from within the reachable region. So the prefix form is preserved and the answer is reach >= n-1.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is the reachable set always a prefix?

Answer. Because jumps only move forward and you can always choose to jump less than the maximum. So from any reachable index you can land on every intermediate index too, which makes the reachable set an initial segment rather than an arbitrary subset.

Q2. What if nums[i] could be 0 in the middle?

Answer. A zero means you must land exactly on i, contributing nothing. That can create a dead end only if i is the current frontier, which the `i > reach` check catches. It cannot create a gap in the prefix.

Q3. How does this become the minimum-jump problem?

Answer. Track the layer boundaries instead of just the reach. Increment a jump counter each time the current index equals the current layer's end, which counts the BFS depth of the reachable region – that is problem 45.

0134 Gas Station Medium

ESSENCE & BOUNDS

Return the starting gas-station index from which the circuit can be completed, or -1 if none exists. 1 <= n <= 1e4 with 0 <= gas[i], cost[i] <= 1e4, so totals reach 1e8 and fit in int. n = 1 always works.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Total sum + single-pass candidate (BEST) Time Complexity : $O(n)$ Space Complexity : $O(1)$

Python (intuition kernel)

```python
total = 0 # net -- is the WHOLE circle feasible?
tank = 0 # running fuel from the current candidate start
start = 0 # current best candidate
for i in range(len(gas)):
    net = gas[i] - cost[i]
    total += net # global balance
    tank += net # local balance since 'start'
    if tank < 0:

```

C++ (intuition kernel)

```cpp
for (int i = 0; i < static_cast<int>(gas.size()); ++i) { int net = gas[i] - cost[i];
    total += net; // accumulate the global balance
    tank += net; // accumulate the local balance
    if (tank < 0) { start = i + 1;
    tank = 0; // new candidate begins with an empty tank
    }

```

Time O(n) **Space** O(1)

Follows directly from the traversal count: $O(n)$ time, $O(1)$ space.

Brute force O(n^2) Try every start and simulate n stops; the quadratic retry wastes the prefix-dead insight.

Python (intuition kernel)

```python
n = len(gas)
for start in range(n):
 tank = 0
 ok = True
 for step in range(n):
 i = (start + step) % n # wrap around the circle
 tank += gas[i] - cost[i]
 if tank < 0:  # died mid-trip $\{}to$ try next start

```

C++ (intuition kernel)

```cpp
for (int start = 0; start < n; ++start) { int tank =
0;
 bool ok = true;
 for (int step = 0; step < n; ++step) {
 int i = (start + step) % n; // wrap around modulo n
 tank += gas[i] - cost[i];
 if (tank < 0) { ok = false; break; } // died
mid-trip
 }
 if (ok) return start; // this start survived all n
stops

```

Time O(n^2) **Space** O(1)

nested enumeration dominates: $O(n\hat{}k)$ pairs, no pruning; hence the stated bounds.

VISUAL STATE TRACE

```
gas=[1,2,3,4,5], cost=[3,4,5,1,2] -> start 3
diff = [-2,-2,-2,3,3], total = 0
start=0, tank=0
i=0:  tank += -2 = -2 < 0 -> tank=0, start=1
i=1:  tank += -2 = -2 < 0 -> tank=0, start=2
i=2:  tank += -2 = -2 < 0 -> tank=0, start=3
i=3:  tank += 3 = 3 >= 0
i=4:  tank += 3 = 6 >= 0
total = 0 >= 0 so start 3 is valid.
WHY THE SKIP IS CORRECT: if the tank goes negative between
s and i, then no index in (s, i] can be a valid start -
the prefix sum from s reaches its minimum at i, so starting
anywhere inside would be strictly worse.  Those indices are
eliminated in one step.
necessity:  total < 0 is impossible (the circuit consumes
more fuel than exists).  total >= 0 is sufficient by the
skip argument.
```

LANGUAGE MECHANICS: PYTHON VS C++

Let P < 0 be the net-sum of [start, j]. For any s in that window the tank at j when starting at s is (P prefix[start..s)) ≤ P < 0 — every inner candidate dies at or before j. The remaining tail [j+1..] is identical for all candidates, so the whole window is dead → resetting to j+1 loses nothing. The total ≥ 0 guarantee ensures a live window always survives to the end. ——————————————————————————— Let P = prefix sum of net over [start, j], and P < 0 at j. For ANY s in [start, j), the tank at j when starting at s equals (prefix sum of [s, j)) = P - (prefix of [start, s)). Since the remaining walk [j+1 ...] adds the same tail to every candidate, the candidate whose carried-in value is LARGEST does best — that is s = start itself. If even start dies (P < 0), every s in the window dies too. Greedy reset = throwing away a provably-dead window; the total ≥ 0 check guarantees a live window remains. ========== / #include <iostream> // std::cout — benchmark output #include <vector> // std::vector — gas / cost arrays #include <string> // std::string — PASS/FAIL messages #include <chrono> // std::chrono — high-resolution timing #include <sys/resource.h> // getrusage — peak RSS measurement

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- A single pass with two ints; `long long` is defensive since totals reach 1e8 but int suffices.
- Early `return -1` when the running total goes negative avoids the rest of the loop.

- `if tank < 0:  start = i + 1; tank = 0` is the whole reset logic.
- A running `total` check plus the start scan can be merged into one pass, which it already is.

INTERVIEW TRAPS & SANITY CHECKS

Forgetting the global total check. A negative total means no start works regardless of the candidate the loop settles on, so returning that candidate is wrong.

Sanity check. `start` is always an index from which the circuit has been successfully completed from `start` to the current index i (the tank never went negative in that segment). When the tank drops below zero at i, all indices in [start, i] are proven invalid, so `start = i + 1` preserves the invariant. At the end, total >= 0 guarantees the wrap-around segment from start back around also succeeds.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is total >= 0 sufficient?

Answer. The skip argument shows the loop eliminates only genuinely invalid starts. If total >= 0, then after removing the valid segment [start, n-1], the wrap-around segment [0, start-1] must also be non-negative in cumulative sum – otherwise total would be negative. So no further elimination is needed.

Q2. Why can all indices in (start, i] be eliminated at once?

Answer. The cumulative sum from start to i is negative, so i is where the running minimum lies. Starting at any k strictly inside makes the tank at i equal to that negative sum plus a positive-or-zero contribution, which is still negative. So none of them can work.

Q3. What if the circuit were not circular?

Answer. The problem becomes trivial: you only need the prefix sums to be non-negative, so the answer is the first index where the running sum never dips below zero again. The circular wrap-around is exactly what makes gas station non-obvious.

0678 `Valid Parenthesis String` Medium

ESSENCE & BOUNDS

Decide whether a string of '(' and ')' is valid by inserting at most one pair anywhere. 0 <= s.length <= 5e3, and an empty string is valid. The count of opens minus closings must be in {-1, 0, 1}.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Two-pass counter sweep (BEST, intuitive) Left→right with '*' as '(' : ')' must never be unmatched. Right→left with '*' as ')' : '(' must never be unmatched.

Python (intuition kernel)

```python
lo = 0 # minimum possible open-count (stars as ')'/empty)
hi = 0 # maximum possible open-count (stars as '(')
for c in s:
 if c == '(':
 lo, hi = lo + 1, hi + 1 # both bounds grow
 elif c == ')':
 lo, hi = lo - 1, hi - 1 # both bounds shrink
 else:

```

C++ (intuition kernel)

```cpp
for (char c :  s) {
 if (c == '(' || c == '*') ++balance; // star can open
 else --balance; // ')' consumes one open
 if (balance < 0) return false; // unmatched ')' -- dead

```

Time O(n) **Space** O(1)

one $O(1)$ counter update per element; hence the stated bounds.

Min/max open-count interval (single pass) Time Complexity : $O(n)$ Space Complexity : $O(1)$ The two passes of Approach 1 are exactly the two extremes of this one interval walked separately. ———————————————————— DEEP DIVE — why 2 numbers replace 3^m assignments Naive: enumerate 3^m star assignments. Greedy insight: balances form a contiguous interval because +1/1/0 moves of a random walk fill every integer between min and max — no holes. Tracking [lo, hi] is an EXACT compression of the state set. Exchange argument: if any assignment works, the extreme assignments (stars-as-open / stars-as-close) survive every prefix, so the interval never dies. ———————————————————— REAL LIFE • Regex / shell glob linting: validating patterns with wildcards before compiling them. • Compiler syntax analysis: bracket matching with optional tokens in grammar productions. • Text editors: live-validating unbalanced snippets while typing. ==========

Python (intuition kernel)

```python
balance = 0
for c in s:
 balance += 1 if c in "(*" else -1 # star can open
 if balance < 0:
 return False # unmatched ')' $\{}to$ dead
balance = 0
for c in reversed(s):
 balance += 1 if c in ")*" else -1 # star can close

```

C++ (intuition kernel)

```cpp
for (char c :  s) {
 if (c == '(') { ++lo; ++hi; } // both bounds grow
 else if (c == ')') { -lo; -hi; } // both bounds shrink
 else { -lo; ++hi; } // '*' widens the interval
 if (hi < 0) return false; // even the max died
 if (lo < 0) lo = 0; // clamp:  surplus closed

```

Time O(n) **Space** O(1)

Follows directly from the traversal count: $O(n)$ time, $O(1)$ space.

VISUAL STATE TRACE

```
s = "))((" -> valid (insert one pair:  ( ))(( )) no)
insert at index 2:  "))" + "(" + "((" -> ()(() ...
simpler:  "))<" becomes ")>)((" -> insert '(' at 0 and ')' at 4:
"(" + "))((" + ")" = "())(()" hmm
correct:  needed opens = 2, needed closes = 2 -> one pair
insufficient!  but the answer is TRUE.
correct count for "))((":  opens=2, closes=2.  Inserting ONE
pair gives 3 of one and 2 of the other -> counts are
(2,3) or (3,2), never equal.  So one insert canNOT fix the
counts...  yet the answer is true.  Rechecking:  insert '(' at
position 0 -> "(())((" ; then ')' still needed.
the actual valid form:  "())()" needs 1 pair inserted into
"))((?  ( ) ) ( ( -> insert '(' before first ')' and ')'
after last '(' = "(())(()" -> that has 4 opens 2 closes.
Resolution:  for "))((" the answer IS true - insert one
'(' at index 0 giving "(()((" is not it either.
Correct:  needed = |opens - closes|.  Here |2-2| = 0, so
zero pairs are needed by count; the only issue is ordering.
First pass (forward):  balance never negative?
) -> balance -1 NEGATIVE -> needs one '(' inserted at 0
) -> balance -2 still negative
( -> -1
( -> 0
Second pass (reverse) counts ')' as open:
( -> -1 ...  also negative -> needs one ')' inserted
max needed = max(1, 1) = 1 <= 1 -> TRUE. The single pair
is '(' at the front and ')' at the back - ONE pair supplies
BOTH because they are the two characters of a pair.
```

LANGUAGE MECHANICS: PYTHON VS C++

Naive: enumerate 3^m star assignments. Greedy insight: balances form a contiguous interval because +1/1/0 moves of a random walk fill every integer between min and max — no holes. Tracking [lo, hi] is an EXACT compression of the state set. Exchange argument: if any assignment works, the extreme assignments (stars-as-open / stars-as-close) survive every prefix, so the interval never dies. ———————————————————————— Naive: 3^m assignments (m = #stars). Greedy insight: balances are a contiguous interval because the +1/1/0 moves of a random walk fill every integer between min and max — no "holes" can appear. Hence tracking 2 numbers replaces tracking 3^m states: the compressed representation is exact. This is the same exchange argument that makes the two-pass greedy exact: if SOME assignment works, the extreme assignments (all-as-open / all-as-close greedies) survive every prefix. ========== / #include <iostream> // std::cout — benchmark output #include <vector> // std::vector — test batch container #include <string> // std::string — the input sequence #include <chrono> // std::chrono — high-resolution timing #include <sys/resource.h> // getrusage — peak RSS measurement

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- Two `int` counters, two passes – no allocation.
- `std::count` for the totals plus one pass is an alternative formulation.
- `s.count('(')` and a single pass suffice for some variants; the two-pass version is unambiguous.
- A one-liner: `needed = max(-min_prefix, max_suffix_deficit)` over the two directions.

INTERVIEW TRAPS & SANITY CHECKS

Checking only that the counts are balanced, or only one direction. "))((" has equal counts but is invalid, and ")(" needs a '(' at the front even though the counts differ by zero... more precisely "((" needs a ')' at the end while the forward scan looks fine.

Sanity check. The forward scan's `low` is the most negative prefix balance; inserting exactly `-low` opens at the front fixes every prefix. The backward scan's `low` similarly measures the opens that need closes inserted. Both must be at most 1, and one inserted pair contributes one open and one close, so the requirement is max(forward deficit, backward deficit) <= 1.

INTUITION CHECK — 3 QUESTIONS

Q1. Why are two passes needed?

Answer. A string can be broken in two independent ways: too many closers early (fixed by adding an open at the front) and too many opens late (fixed by adding a close at the end). One direction of scanning catches only one of them, and a single inserted pair supplies one of each character.

Q2. How do you compute the required inserts in one expression?

Answer. Forward: let L be the minimum prefix balance of #open minus #close. You need -L opens at the front if $L < 0$. Backward: let R be the minimum balance of #close minus #open over reversed prefixes. You need -R closes at the end if $R < 0$. The answer is true iff max(-L, -R, 0) <= 1.

Q3. What is the complexity?

Answer. $O(n)$ time and $O(1)$ space – two counters over two passes, no allocation in either language. That is a rare problem where the optimal solution uses no extra memory at all.

0763 `Partition Labels` Medium

ESSENCE & BOUNDS

Return the number of contiguous substrings such that each contains no repeated characters. 0 <= s.length <= 1e4 over lowercase letters, so 26 is the alphabet size and the last-index array is tiny. An empty string has 0 partitions.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Last-index map + sweep (BEST) Time Complexity : $O(n)$ Space Complexity : $O(1)$ (26 letters → constant-size last table.) ————————————————————————

Python (intuition kernel)

```python
last = [-1] * 26
for i, ch in enumerate(s):
 last[ord(ch) - 97] = i # overwrite $\{}to$ ends at final spot
parts:  List[int] = []
seg_start = 0 # left edge of the current part
seg_end = 0 # furthest extent the current part must reach
for i, ch in enumerate(s):
 seg_end = max(seg_end, last[ord(ch) - 97]) # must cover char's end

```

C++ (intuition kernel)

```cpp
for (int i = 0; i < static_cast<int>(s.size()); ++i) { last[s[i] - 'a'] = i;

```

Time O(n) **Space** O(1)

each interval enters and leaves the frontier once; hence the stated bounds.

Merge per-character intervals Every character spans [first[c], last[c]]; sort the ≤26 intervals and merge overlapping ones (like "Merge Intervals"); emit merged lengths. Time $O(n + klogk)$, k ≤ 26 → $O(n)$ effective. ———————————————————————— DEEP DIVE — why the earliest safe cut is provably maximal Exchange argument: let O be any valid partition with first cut o. If o < e (= our first cut) then some char in [0, o] still occurs after o (else e ≤ o) → O's first part leaks → invalid. Every valid first cut is ≥ e, so cutting at e loses nothing; induction on the remainder makes greedy optimal for all cuts. ———————————————————————— REAL LIFE • Playlist splitting: group songs so an artist appears in one block (per-artist last index). • Log sharding: partition a log stream by key groups. • Database shard planning: split hot keys so each key lives in one shard. ==========

Python (intuition kernel)

```python
first = [-1] * 26 # earliest occurrence per character
last = [-1] * 26 # final occurrence per character
for i, ch in enumerate(s):
 c = ord(ch) - 97
 if first[c] == -1:
 first[c] = i
 last[c] = i
intervals = sorted((first[c], last[c])

```

C++ (intuition kernel)

```cpp
for (int i = 0; i < static_cast<int>(s.size()); ++i) { int c = s[i] - 'a';
 if (first[c] == -1) first[c] = i; // first sighting
 last[c] = i; // keep overwriting $\{}to$ final

```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

VISUAL STATE TRACE

```
s = "ababcbacadefegdehijhklij" -> 3
last = {a:7, b:5, c:3, d:9, e:11, f:8, g:13, h:12,
i:16, j:15, k:14, l:...}
sweep from left:
start=0, end=0
i=0 'a' -> end = max(0, 7) = 7
i=1 'b' -> end = max(7, 5) = 7
i=2 'a' -> 7
i=3 'b' -> 7
i=4 'c' -> max(7,3)=7
i=5 'a' -> 7
i=6 'c' -> 7
i=7 'd' -> max(7,9) = 9 end exceeded i=7 -> CLOSE
partition 1 = s[0..7], length 8
restart at 8:  'e' -> max(8,11) = 11
i=8 'f' -> 11, i=9 'g' -> 13
i=10 'e' -> 13, i=11 'd' -> 13, i=12 'f' -> 13,
i=13 'g' -> 13 -> CLOSE at 13, length 6
restart at 14:  'h' i k l j i h -> close at 19
total partitions = 3
```

LANGUAGE MECHANICS: PYTHON VS C++

Exchange argument: let O be any valid partition with first cut o. If o < e (= our first cut) then some char in $[0, o]$ still occurs after o (else e $\leq$ o) $\rightarrow$ O's first part leaks $\rightarrow$ invalid. Every valid first cut is $\geq$ e, so cutting at e loses nothing; induction on the remainder makes greedy optimal for all cuts. ——————————————————————————— Greedy exchange: let G cut at index e = segEnd (earliest safe), let O be any optimal partition with first cut at o. Claim o $\geq$ e: if o < e then some character in $[0, o]$ has a later occurrence after o (since e is the earliest point where ALL characters in $[0, e]$ have appeared) $\rightarrow$ O's first part leaks $\rightarrow$ O invalid. Hence every valid first cut is $\geq$ e, so cutting at e never loses parts. By induction on the remainder, greedy emits the maximal count. ========== / #include <iostream> // std::cout — benchmark output #include <vector> // std::vector — result lengths #include <string> // std::string — the input #include <chrono> // std::chrono — high-resolution timing #include <sys/resource.h> // getrusage — peak RSS measurement

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `array<int,26> last` of 104 bytes, always in L1.
- `vector<int> last(26, -1)` plus a pass over the string; two linear passes, no hashing.
- A list of 26 ints indexed by `ord(c) - 97` avoids dict hashing entirely.
- `s.count(c, i)` is an alternative but is $O(n^2)$.

INTERVIEW TRAPS & SANITY CHECKS

Closing the partition as soon as `i == end` using a stale end, or resetting `end` without restarting at i+1. The partition boundary must be exactly the max last-index seen in the current segment, and the new segment starts at i+1.

Sanity check. `end` is the maximum last-occurrence index among all characters seen so far in the current segment. While i <= end, some character in the segment reappears after i, so the segment cannot be closed yet. When i reaches end, every character in the segment occurs only within it, making it a valid partition and the maximum possible one.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is each produced partition maximal?

Answer. Because the segment cannot be cut anywhere inside it – some character straddles each internal boundary. So there is exactly one valid partition of each length starting at 0, and the answer counts them, not all possible partitions.

Q2. What is the complexity?

Answer. $O(n)$ time – two passes, one to build last-occurrence indices and one to sweep – and $O(1)$ space for the fixed 26-entry table. The alphabet size is what makes it constant rather than $O(n)$ space.

Q3. How would this change with a large alphabet?

Answer. The table becomes $O(sigma)$ space instead of $O(1)$. A hash map would still give $O(n)$ time, but the array is faster and the choice between them is exactly the constant-factor-versus-space trade-off.

0846 Hand of Straights Medium

ESSENCE & BOUNDS

Decide whether the hand can be partitioned into groups of k consecutive cards. 1 <= hand.length <= 1e4 with 1 <= card[i] <= 1e4, and hand.length must be divisible by k or the answer is immediately false. Duplicate values are common and must be handled.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

- Sorted counter + greedy runs (best) Count cards; repeatedly take the smallest value v and consume v..v+groupSize-1; any gap fails.

Python (intuition kernel)

```python
if len(hand) % groupSize:
cnt = Counter(hand)
for first in sorted(cnt):
 if cnt[first] == 0:
 continue
 need = cnt[first]
 for v in range(first, first + groupSize):
 if cnt[v] < need:

```

C++ (intuition kernel)

```cpp
while (!cnt.empty()) {
 int first = cnt.begin()->first; // smallest
remaining
 for (int k = 0; k < groupSize; ++k) { int v = first
+ k;
 auto it = cnt.find(v);
 if (it == cnt.end()) return false; // run can't
close
 if (--it->second == 0) cnt.erase(it);
 }

```

Time ?	Space ?

one $O(1)$ counter update per element; hence the stated bounds.

- Sort + pointer scan Sort, walk runs skipping already-used cards. ———————————————————— — DEEP DIVE - Forced decisions = provably optimal Smallest remaining x can't be mid-run (x-1 consumed or absent), so it must open a run, and that run is fully determined. No branch to choose, no suboptimal alternative — greedy is optimal by construction. Counter dict keys are sorted on demand, making "smallest remaining" $O(logn)$ amortized. ———————————————————— TIME COMPLEXITY : $O(nlogn)$ MEMORY COMPLEXITY: $O(n)$ ==========

Python (intuition kernel)

```python
if len(hand) % groupSize:
hand.sort()
used = [False] * len(hand)
for i in range(len(hand)):
 if used[i]:
 continue
 prev, placed = hand[i], 1
 used[i] = True

```

C++ (intuition kernel)

```cpp
for (size_t i = 0; i < hand.size(); ++i) { if
(used[i]) continue;
 int prev = hand[i], placed = 1;
 used[i] = true;
 for (size_t j = i + 1; j < hand.size() && placed <
groupSize; ++j)
 if (!used[j] && hand[j] == prev + 1) { // next
consecutive
 used[j] = true;
 prev = hand[j];
 ++placed;

```

Time O(n log n) **Space** O(n)

the sort dominates; the post-sort scan is linear; hence the stated bounds.

VISUAL STATE TRACE

```
hand=[1,2,3,3,4,4,5], k=3 -> true
sorted:  1 2 3 3 4 4 5
counts:  1:1 2:1 3:2 4:2 5:1
x=1:  need a run 1,2,3 -> take 1 each
counts:  1:0 2:0 3:1 4:2 5:1
x=3 (smallest remaining):  run 3,4,5 -> take 1 each
counts:  3:0 4:1 5:0
x=4:  run 4,5,6 -> count[6] == 0 -> FALSE
so [1,2,3,3,4,4,5] k=3 is FALSE.
true example:  hand=[1,2,3,3,4,4,5,6] k=3
x=1:  1,2,3
x=3:  3,4,5
x=4:  4,5,6
-> TRUE
invariant:  x is the SMALLEST remaining card, so any group
containing it MUST start at x.  That forces the run
x..x+k-1, and consuming those is never wrong - there is
no alternative grouping for x.
```

LANGUAGE MECHANICS: PYTHON VS C++

Smallest remaining x can't be mid-run (x-1 consumed or absent), so it must open a run, and that run is fully determined. No branch to choose, no suboptimal alternative — greedy is optimal by construction. Counter dict keys are sorted on demand, making "smallest remaining" $O(logn)$ amortized. ——————————————————————— TIME COMPLEXITY : $O(nlogn)$ ==========

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- A 1e4-entry count array is 40 KB, so a counting sort plus index walk avoids hashing.
- `std::map<int,int>` with iterator advance handles the run extraction in one pass.
- `Counter(hand)` plus iterating `sorted(counts)` gives the same forced-choice logic.
- Skip zero-count keys quickly by breaking once the key exceeds max(card) - k + 1.

INTERVIEW TRAPS & SANITY CHECKS

Taking any run rather than the one starting at the smallest card. That can consume a card needed by a later forced group and wrongly return false.

Sanity check. When x is the smallest card with a positive count, every valid partition must place x at the start of some run, since no smaller card remains. Therefore the run x, x+1, ..., x+k-1 is mandatory, and decrementing one of each is the only possible choice. If any is missing the partition is impossible.

INTUITION CHECK — 3 QUESTIONS

Q1. Why must the run start at x?

Answer. Because x is the smallest remaining value. A run containing x could not start before x, since that card is gone, and runs are consecutive – so x is necessarily the first element of its run.

Q2. What is the complexity?

Answer. $O(n + \sigma \log \sigma)$ time to count and sort the distinct values, then $O(\sigma \cdot k)$ for the run consumption, bounded by n overall. Space is $O(\sigma)$ for the counter.

Q3. Why is the divisibility check useful?

Answer. It is an $O(1)$ rejection that catches a large class of impossible inputs before any work. Every group has exactly k cards, so hand.length mod k must be 0 for any partition to exist.

1899 Merge Triplets to Form Target Triplet Medium

ESSENCE & BOUNDS

Return the maximum possible value of a merged triplet that equals the target, where each triplet is merged element-wise and values must stay below target. 1 <= triplets.length <= 1000 with 1 <= values <= 1000, and a guaranteed solution exists. Zero-valued triplets are legal.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

- Filter then merge Keep triplets with all coordinates $\leq$ target; coordinate-wise max; compare to target.

Python (intuition kernel)

```python
ok = [t for t in triplets
 if all(t[i] <= target[i] for i in range(3))]
acc = [0, 0, 0]
for t in ok:
 for i in range(3):
  acc[i] = max(acc[i], t[i])
return acc == target

```

C++ (intuition kernel)

```cpp
if (t[0] <= target[0] && t[1] <= target[1] && t[2] <= target[2]) { a = std::max(a, t[0]);
 b = std::max(b, t[1]);
 c = std::max(c, t[2]);

```

Time ?	Space ?

Follows directly from the traversal count: ? time, ? space.

- Running best (in-place) Track (a,b,c) best-so-far over admissible triplets; identical math, $O(1)$ state. —— DEEP DIVE - Why "merge everything admissible" is optimal Since max(a,b) $\geq$ a, merging never destroys a coordinate. If ANY subset reaches the target, the full admissible set reaches it too (it dominates every subset coordinate-wise). And inadmissible triplets can't help — a coordinate beyond target can never come back down. Nothing to choose $\Rightarrow$ greedy trivially optimal. — TIME COMPLEXITY : $O(n)$ MEMORY COMPLEXITY: $O(1)$

Python (intuition kernel)

```python
a = b = c = 0
for t in triplets:
 if t[0] <= target[0] and t[1] <= target[1] and t[2] <= target[2]:
  a = max(a, t[0])
  b = max(b, t[1])
  c = max(c, t[2])
 return [a, b, c] == target

```

C++ (intuition kernel)

```cpp
if (t[0] <= target[0] && t[1] <= target[1] && t[2] <= target[2]) { a = std::max(a, t[0]);
 b = std::max(b, t[1]);
 c = std::max(c, t[2]);

```

Time O(n) **Space** O(1)

the output array doubles as the state store; hence the stated bounds.

VISUAL STATE TRACE

```
triplets=[(2,2,3),(1,2,3),(1,1,4)], target=5 -> 5
filtered to values strictly BELOW 5 per position:
pos0:  min(2,1,1) = 1
pos1:  min(2,2,1) = 1
pos2:  min(3,3,4) = 4 -> min(3,3,3) = 3
so keep only triplets with a[0]==1, b[0]==1, c[0]==3
-> (1,2,3) -> 1+2+3 = 6 != 5?  the third is (1,1,4)?
corrected filtering:  keep triplets that EQUAL the target in
some position, otherwise discard:
(2,2,3):  c==3 < 5, a,b not 5
(1,2,3):  c==3 < 5
(1,1,4):  none equal 5, all < 5
all three survive the <5 test; the minima are 1,1,3
requiring a==1, b==1, c==3 -> (1,2,3) has b=2
-> with (1,1,4):  a=1,b=1,c=4 -> sum 6
target 5 requires (1,1,3) which does not exist.
The actual filter keeps triplets that are BELOW target in
every position; then the three per-position minima are
taken; then we merge and if the sum equals target, that is
the answer.  Where the sum exceeds target, some position
must be lowered - and the per-position minimums already
give the lowest reachable triple.
```

LANGUAGE MECHANICS: PYTHON VS C++

Since max(a,b) $\geq$ a, merging never destroys a coordinate. If ANY subset reaches the target, the full admissible set reaches it too (it dominates every subset coordinate-wise). And inadmissible triplets can't help — a coordinate beyond target can never come back down. Nothing to choose $\Rightarrow$ greedy trivially optimal. ——————————————————————— TIME COMPLEXITY : $O(n)$
==========

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `array<int,3> best = {INT_MAX,INT_MAX,INT_MAX}` and one pass, $O(n)$ time $O(1)$ space.
- Two passes: filter first (also $O(1)$ space), then merge. The filter makes the merge step trivially correct.

- A list of three sentinels and comparisons beats `min()` calls in a hot loop.
- A generator with `min(...)` per position is concise; `zip(*triplets)` plus `min` is the shortest form.

INTERVIEW TRAPS & SANITY CHECKS

Taking the per-position minimum over ALL triplets instead of only those below target. A triplet with a value above target can never participate, so including it lowers a minimum to an unusable value and the merge fails.

Sanity check. Any triplet participating in a solution must have all three values strictly below target, since a merged value equals one of the contributing triplets' values in its position. Among the filtered triplets, the per-position minima form a lower bound that is simultaneously achievable – so if any merge works, the minima merge works too.

INTUITION CHECK — 3 QUESTIONS

Q1. Why are per-position minima sufficient?

Answer. If some triplet in the filtered set has $a[0] > \min(a)$ then replacing it with the minimiser is never worse, since the constraint is only $a[0] + b + c == \text{target}$ and lowering one coordinate cannot overshoot. So the minima triple is achievable and is the lowest, making it the best candidate.

Q2. What if the minima sum exceeds the target?

Answer. Then no merge can equal the target, because the minima triple is a lower bound on every achievable sum. Since the problem guarantees a solution exists, that branch is unreachable for valid input.

Q3. Why the strict < comparison on target?

Answer. A merged value equals one contributing value, and the target requires the SUM to equal target, so any single value equal to target would force the other two to be zero. Using < filters the general case; using <= is only safe when zeros are allowed and handled.

16. Intervals

#	Problem	Level	One-line intent
0056	**Merge Intervals**	Medium	Sort by start; merge when start <= current end.
0057	**Insert Interval**	Medium	Three phases: before, overlapping block, after.
0252	**Meeting Rooms**	Easy	Sort by start; any start < previous end is a conflict.
0253	**Meeting Rooms II**	Medium	Sweep starts and ends separately with two pointers.
0435	**Non-overlapping Intervals**	Medium	Sort by END; keep an interval only if it starts after the last kept one.
1851	**Minimum Interval to Include Each Query**	Hard	Offline: sort queries, sweep, min-heap keyed by length.

0056 `Merge Intervals` Medium

ESSENCE & BOUNDS

Merge all overlapping intervals into a minimal set of disjoint intervals. 1 <= intervals.length <= 1e4 with up to 1e9 endpoints. An interval may be [x, x] and touching intervals such as [1,4] and [4,5] DO merge.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Sort by start + sweep into a result list (BEST) Sort ascending. Walk once. If the current interval starts after the last block ends -> new block. Else extend the block's end.

Python (intuition kernel)

```python
intervals.sort()
out: List[Interval] = []
for it in intervals:
 if not out or out[-1][1] < it[0]:
 out.append(it)
 else: out[-1][1] = max(out[-1][1], it[1])
return out

```

C++ (intuition kernel)

```cpp
for (const std::vector<int>& it : intervals) { if (out.empty() || out.back()[1] < it[0]) {
out.push_back(it);
 } else {
 out.back()[1] = std::max(out.back()[1], it[1]);
 }

```

Time O(n log n) **Space** O(n)

the sort dominates; the post-sort scan is linear; hence the stated bounds.

Sort by start + in-place compaction Same rule, but merged blocks are written back into the FRONT of the input list (slot k) and the tail is trimmed — the input array itself becomes the answer.

Python (intuition kernel)

```python
intervals.sort()
k = 0 # number of merged blocks written to the front
for i in range(1, len(intervals)):
 if intervals[k][1] >= intervals[i][0]:
 intervals[k][1] = max(intervals[k][1], intervals[i][1])
 else: k += 1
 intervals[k] = intervals[i]
del intervals[k + 1:] # trim the absorbed tail

```

C++ (intuition kernel)

```cpp
for (int i = 1; i < static_cast<int>(intervals.size()); ++i) { if (intervals[k][1] >= intervals[i][0]) {
 intervals[k][1] = std::max(intervals[k][1], intervals[i][1]);
 } else { ++k;
 intervals[k] = intervals[i];
 }

```

Time O(n log n) **Space** O(1) extra

the sort dominates; the post-sort scan is linear; hence the stated bounds.

VISUAL STATE TRACE

```
input:  [[1,3],[2,6],[8,10],[15,18]]
-> [[1,6],[8,10],[15,18]]
sorted by start:  (1,3) (2,6) (8,10) (15,18)
out = []
for s, e in sorted:
if out and s <= out[-1][1]:  # overlap OR touch
out[-1][1] = max(out[-1][1], e) # extend to the max end
else:
out.append([s, e])
trace:
(1,3) -> out=[[1,3]]
(2,6) -> 2 <= 3 -> out=[[1,6]]
(8,10) -> 8 > 6 -> out=[[1,6],[8,10]]
(15,18)-> 15 > 10 -> out=[[1,6],[8,10],[15,18]]
WHY <= AND NOT <:
[1,4] and [4,5] share the point 4, so they form one
contiguous interval [1,5].  Using < would emit two.
why sort by START and not by end:  with start-sorted input,
once an interval does not overlap the current group, no
later one can either - starts only increase.
```

LANGUAGE MECHANICS: PYTHON VS C++

Two closed intervals [a1,a2], [b1,b2] overlap when a1 <= b2 && b1 <= a2. Sorting by start turns "does it merge?" into a one-sided question: it.start <= lastEnd ? merge : new block. No hash map, no heap, no tree: a plain sort + sweep is optimal because the merge relation is transitive along the sorted order. (Contrast: for INSERT-INTERVAL style problems the same sorted sweep pattern works — one interval at a time instead of a batch.)
==========
/ #include <vector> // std::vector — intervals storage #include <algorithm> // std::sort, std::max — sort + merge bookkeeping #include <utility> // std::pair — used by the coverage validator #include <chrono> // std::chrono — high-resolution benchmark timing #include <iostream> // std::cout — printing PASS/FAIL + timings #include <sys/resource.h> // getrusage — peak RAM measurement

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `sort(v.begin(), v.end())` on `vector<pair<int,int>>` uses lexicographic order, which sorts by start first – exactly what is needed.
- In-place compaction merges into `v[out++]` for $O(1)$ extra space beyond the sort.
- `sorted(intervals)` on tuples sorts by start then end, which is fine.
- Modifying the output in place (`out[-1][1] = ...`) avoids allocating new pairs per merge.

INTERVIEW TRAPS & SANITY CHECKS

Using < instead of <= for the overlap test, which splits [1,4] and [4,5] into two intervals when they should merge into [1,5].

Sanity check. `out[-1]` always holds the merged interval covering the last accepted group, and its end is the maximum end among that group's intervals. If the next start exceeds that end, no later interval can overlap it either, since later starts are no smaller – so appending is safe and final.

INTUITION CHECK — 3 QUESTIONS

Q1. Why sort by start rather than by end?

Answer. Sorting by start gives a monotone frontier: once an interval's start passes the current group's maximum end, every remaining start is even larger, so the group is closed permanently. Sorting by end gives no such guarantee and the merge test becomes non-local.

Q2. Why do touching intervals merge?

Answer. Because [1,4] and [4,5] have no gap – their union is the single interval [1,5], and the problem asks for the minimal set of disjoint intervals. Splitting them would not be minimal.

Q3. What is the complexity?

Answer. $O(n \log n)$ time for the sort and $O(n)$ for the sweep, with $O(n)$ space for the result (or $O(1)$ extra with in-place compaction). The sort dominates, which is why sorting is unavoidable here.

0057 Insert Interval Medium

ESSENCE & BOUNDS

Insert one interval into a set of non-overlapping sorted intervals and return the merged result. 0 <= intervals.length <= 1e4 and the input is guaranteed sorted and disjoint, so a single three-phase pass suffices. An empty input is legal.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Three-phase linear sweep (BEST) Phase A: copy intervals with end < new.start. Phase B: absorb every interval with start <= new.end, extending new_interval. Phase C: copy the rest.

Python (intuition kernel)

```python
out: List[Interval] = []
i, n = 0, len(intervals)
while i < n and intervals[i][1] < new_interval[0]:
 out.append(intervals[i])
 i += 1
while i < n and intervals[i][0] <= new_interval[1]:
 new_interval[0] = min(new_interval[0],
intervals[i][0])
 new_interval[1] = max(new_interval[1],
intervals[i][1])

```

C++ (intuition kernel)

```cpp
vector<Interval> out;
int i = 0, n = intervals.size();
while (i < n && intervals[i].end < ni.start)
 out.push_back(intervals[i++]);
ni.start = min(ni.start, intervals[i].start);
ni.end = max(ni.end, intervals[i].end);
while (i < n && intervals[i].start <= ni.end) {
 ni.start = min(ni.start, intervals[i].start);
 ni.end = max(ni.end, intervals[i].end);
 ++i;
}
out.push_back(ni);
while (i < n) out.push_back(intervals[i++]);
return out;

```

Time O(n) **Space** O(n)

each interval enters and leaves the frontier once; hence the stated bounds.

Binary-search the merge window, then splice left = first index whose end >= new.start (bisect_left over ends) right = last index whose start <= new.end (bisect_right over starts) The whole merge zone is [left, right]; splice around it.

Python (intuition kernel)

```python
ends = [iv[1] for iv in intervals] # O(n) scan for the searches
starts = [iv[0] for iv in intervals]
left = bisect.bisect_left(ends, new_interval[0])
right = bisect.bisect_right(starts, new_interval[1]) - 1
out: List[Interval] = intervals[:left] # untouched BEFORE window
if left <= right:
 new_interval[0] = min(new_interval[0], intervals[left][0])
 new_interval[1] = max(new_interval[1], intervals[right][1])

```

C++ (intuition kernel)

```cpp
auto lo = lower_bound(intervals.begin(), intervals.end(), ni,
 [](const Interval& a, const Interval& b) { return a.start < b.start; });
// splice [lo, hi) into one merged interval, keep the rest

```

Time O(log n + n) **Space** O(n)

Follows directly from the traversal count: $O(logn + n)$ time, $O(n)$ space.

VISUAL STATE TRACE

```
intervals=[[1,3],[6,9]], new=[2,5]
-> [[1,5],[6,9]]
phase 1 (strictly before):  s < new.start -> copy through
(1,3):  1 < 2 -> copy
(6,9):  6 < 2?  no
phase 2 (overlapping):  s <= cur.end -> absorb
(6,9):  6 <= cur.end(5)?  no
phase 3 (after):  copy the rest
code:
i=0
while i < n and intervals[i].end < new.start:
out.push(intervals[i]); i++ # phase 1
new.start = min(new.start, intervals[i].start)
new.end = max(new.end, intervals[i].end)
while i < n and intervals[i].start <= new.end:
new.start = min(new.start, intervals[i].start)
new.end = max(new.end, intervals[i].end)
i++ # phase 2 (grow on both)
out.push(new)
while i < n:  out.push(intervals[i++]) # phase 3
the GROW on both ends is essential:  [2,5] absorbing (6,9)
must extend new.end to 9, which may then absorb more.
```

LANGUAGE MECHANICS: PYTHON VS C++

Two variants of the same question: • "first index where end >= x" -> std::lower_bound over ends • "last index where start <= x" -> upper_bound over starts - 1 When the interval list lives in a balanced tree / sorted array, these become $O(logn)$ window queries — the pattern behind range trees and interval trees. For ONE insert, linear sweep is simpler and equally fast; the binary-search version shines when intervals are huge (n = 10^7) or when many inserts stream in (each one $O(logn)$ to locate + $O(k)$ to splice). ========== / #include <vector> // std::vector — interval storage #include <algorithm> // std::sort, std::min, std::max #include <utility> // std::pair — coverage validator #include <chrono> // std::chrono — benchmark timing #include <iostream> // std::cout — printing results #include <sys/resource.h> // getrusage — peak RAM measurement

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `std::vector<Interval> out; out.reserve(n+1)` avoids repeated reallocation.
- Insert into the vector then merge in place is an alternative when avoiding a second container matters.
- A list comprehension for phases 1 and 3 with an index for phase 2 is readable; the index-based version avoids scanning the prefix twice.
- `bisect` can locate the insert point, but the linear scan is already $O(n)$ and simpler.

INTERVIEW TRAPS & SANITY CHECKS

Failing to grow `new.end` after absorbing an interval, which is required when a later interval starts inside the newly extended range. Also, assuming only one interval overlaps rather than a whole block.

Sanity check. After phase 1, all copied intervals end strictly before the insertion point. During phase 2, `new` is the union of the insertion interval and every absorbed interval, so its start and end are the running min and max. Phase 2 stops exactly when the next start exceeds that union, at which point the union can never grow again.

INTUITION CHECK — 3 QUESTIONS

Q1. Why three phases rather than one pass with a flag?

Answer. Because the roles differ: before the insertion point you copy verbatim, inside the overlapping block you absorb and grow, and after it you copy verbatim again. A single flag variable cannot express "grow the range" while still needing to copy unchanged intervals on both sides.

Q2. Can you do it with binary search?

Answer. Yes – `bisect` on the interval starts locates the first interval whose end >= new.start in $O(\log n)$, after which the absorb loop is still $O(k)$. That gives $O(\log n + k)$ time, but the total is still bounded by $O(n)$ because of the copy.

Q3. What makes the absorb loop able to grow?

Answer. Each absorbed interval may extend `new.end` beyond the value that was tested, so the loop condition must be re-evaluated against the updated end. That is why the min/max updates happen inside the loop and not before it.

0252 Meeting Rooms Easy

ESSENCE & BOUNDS

Return true if two meetings share no overlap. 0 <= meetings.length <= 1e6, and meetings are half-open [start, end), so a meeting ending exactly when another starts is NOT a conflict. An empty list is trivially true.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Sort by start + adjacent overlap check (BEST) Sort. Walk once: if next.start < prev.end the previous meeting is still running -> False. Strict '<' encodes half-open semantics.

Python (intuition kernel)

```python
intervals.sort()
for i in range(1, len(intervals)):
 if intervals[i][0] < intervals[i - 1][1]:

```

C++ (intuition kernel)

```cpp
for (int i = 1; i <
static_cast<int>(intervals.size()); ++i) { if
(intervals[i][0] < intervals[i - 1][1]) { return false;
 }

```

Time O(n log n) **Space** O(1)

the sort dominates; the post-sort scan is linear; hence the stated bounds.

Chronological event sweep (sweep-line) Flatten meetings into events (time, +1 start / -1 end). Sort by time with ENDS before STARTS at equal times. active += delta; active > 1 at any moment -> False.

Python (intuition kernel)

```python
events: List[Tuple[int, int]] = []
for s, e in intervals:
 events.append((s, 1))
 events.append((e, 0))
events.sort()
active = 0
for _, kind in events:
 active += 1 if kind == 1 else -1

```

C++ (intuition kernel)

```cpp
for (const std::vector<int>& it : intervals) {
 events.push_back({it[0], 1}); // start : +1 active, tie kind 1
 events.push_back({it[1], 0}); // end : -1 active, tie kind 0

```

Time O(n log n) **Space** O(n)

each interval enters and leaves the frontier once; hence the stated bounds.

VISUAL STATE TRACE

```
meetings sorted by start:
[0,10] [10,20] [20,30] -> all adjacent, no overlap
[0,10] [5,6] -> 5 < 10 -> OVERLAP -> false
prev_end = -inf
for s, e in sorted:
if s < prev_end: return False # STRICT <, not <=
prev_end = e
return True
WHY ONLY THE PREVIOUS END MATTERS:
the intervals are sorted by start, so starts are
non-decreasing. If s >= prev_end then this meeting starts
after the previous one ended; and every earlier meeting
ended at or before prev_end, so none can conflict. Hence a
single running maximum of past ends suffices.
```

LANGUAGE MECHANICS: PYTHON VS C++

The sweep line turns "do intervals overlap anywhere?" into "does the counter ever exceed 1?". Every interval adds +1 at its start and -1 at its end; between events the answer is constant. Tie-breaking encodes the interval semantics: closed [a,b] : +1 before -1 (they overlap at the shared point) half-open [a,b): -1 before +1 (ending at b frees the room at b) This same event trick powers Meeting Rooms II (count rooms), skyline problems, and calendar clash detectors. The C++ pair sort (time, then kind) gives you the tie-break for free. ========== / #include <vector> // std::vector — intervals + events #include <utility> // std::pair — (time, delta) events #include <algorithm> // std::sort #include <chrono> // std::chrono — benchmark timing #include <iostream> // std::cout — printing results #include <sys/resource.h> // getrusage — peak RAM measurement

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `sort` on `vector<pair<int,int>>` compares by start first automatically.
- `std::sort` on 1e6 pairs is about 0.2 s; the sweep afterwards is negligible.

- `sorted(meetings)` on 1e6 tuples works but allocates; sorting an index array is slower in CPython.
- Iterating the sorted list directly avoids index arithmetic.

INTERVIEW TRAPS & SANITY CHECKS

Using <= instead of <, which declares [0,10] and [10,20] a conflict. LeetCode meetings are half-open, so touching is fine.

Sanity check. After processing k meetings, `prev_end` is the maximum end among them. The next meeting conflicts with some earlier one exactly when its start is less than that maximum – because a conflict requires a previous interval whose end exceeds the new start. So one running value suffices instead of comparing against all previous intervals.

INTUITION CHECK — 3 QUESTIONS

Q1. Why does only the previous end matter?

Answer. The list is sorted by start, so starts are non-decreasing. If the new start is at least the previous end, it is at least every earlier end too (since prev_end is the running max). If it is smaller, it conflicts with the meeting that set that max.

Q2. What would change if intervals were closed?

Answer. The comparison becomes <= instead of <, because [0,10] and [10,20] would then share the point 10. That one character is the entire difference between the two conventions and a classic source of wrong answers.

Q3. How does this differ from meeting rooms II?

Answer. Here you only need to know WHETHER a conflict exists, which the single running maximum answers in $O(n)$ after sorting. There you need the maximum OVERLAP, which requires either a sweep line over events or a min-heap of active end times.

0253 Meeting Rooms II Medium

ESSENCE & BOUNDS

Return the maximum number of meetings happening simultaneously. 0 <= meetings.length <= 1e5 with up to 1e9 endpoints. An empty list gives 0. Half-open intervals again, so a meeting ending at time t does not overlap one starting at t.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Sort by start + min-heap of end times (BEST) Sort by start. For each meeting: if the earliest-ending active meeting is over (heap top <= start), pop it (room reused). Push this meeting's end. Answer = max heap size.

Python (intuition kernel)

```python
intervals.sort() # by start
ends: List[int] = []
rooms = 0
for s, e in intervals:
 if ends and ends[0] <= s:
 heapq.heappop(ends) # that room is now free
 heapq.heappush(ends, e) # this meeting occupies a room until e
 rooms = max(rooms, len(ends))

```

C++ (intuition kernel)

```cpp
for (const std::vector<int>& it : intervals) { if (!ends.empty() && ends.top() <= it[0]) { ends.pop(); // that room is now free
 }
 ends.push(it[1]); // this meeting occupies a room until it[1]
 rooms = std::max(rooms, static_cast<int>(ends.size()));

```

Time O(n log n) **Space** O(n)

the sort dominates; the post-sort scan is linear; hence the stated bounds.

Two pointers over sorted starts & ends Sort starts and ends SEPARATELY. If the next start precedes the earliest free time (starts[s] < ends[e]) a new room is needed; otherwise the room at ends[e] frees -> advance e.

Python (intuition kernel)

```python
starts = sorted(s for s, _ in intervals)
ends = sorted(e for _, e in intervals)
rooms = 0
active = 0
e = 0
for s in range(len(starts)):
 while e < len(ends) and ends[e] <= starts[s]:
 active -= 1

```

C++ (intuition kernel)

```cpp
for (const std::vector<int>& it : intervals) { starts.push_back(it[0]);
 ends.push_back(it[1]);

```

Time O(n log n) **Space** O(n)

the sort dominates; the post-sort scan is linear; hence the stated bounds.

VISUAL STATE TRACE

```
meetings=[[1,3],[2,6],[4,8]] -> 3
starts = [1,2,4]
ends = [3,6,8]
two-pointer sweep:
s=0 e=0 count=0 best=0
starts[0]=1 < ends[0]=3 -> count=1 best=1 s=1
starts[1]=2 < ends[0]=3 -> count=2 best=2 s=2
starts[2]=4 > ends[0]=3 -> count=1 e=1
starts[2]=4 < ends[1]=6 -> count=2 s=3
s exhausted, drain remaining ends -> done
best = 2, not 3.  Correct answer for [[1,3],[2,6]] is 2.
For 3 simultaneous you need e.g.  [[1,3],[2,6],[2,8]]:
starts[1]=2 < ends[0]=3 -> count 2
starts[2]=2 < ends[0]=3 -> count 3 BEST
invariant:  'count' is the number of meetings started minus
ended so far, i.e.  exactly the number in progress.  The
sweep advances whichever side has the smaller next
timestamp, so no meeting is counted twice.
```

LANGUAGE MECHANICS: PYTHON VS C++

A naive sweep must know "when is the next meeting over?" — a min-heap makes that its root in $O(1)$. Crucially the heap only contains meetings whose start we HAVE seen: expired ones (root end <= current start) are popped before pushing, so heap.size() == number of concurrently open meetings — the exact quantity we maximise. Python: heapq (min-heap). C++: std::priority_queue with std::greater<int> to invert max→min. Lazy deletion is NOT needed here (we pop while root is expired, not once) — but the same pattern reappears in 1851 where expiry is checked per query.
==========
/ #include <vector> // std::vector — intervals + sorted copies #include <queue> // std::priority_queue — min-heap of end times #include <algorithm> // std::sort, std::greater #include <functional> // std::greater<int> — min-heap comparator #include <chrono> // std::chrono — benchmark timing #include <iostream> // std::cout — printing results #include <sys/resource.h> // getrusage — peak RAM measurement

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- Two `vector<int>` sorts plus one merge pass is $O(n \log n)$ with $O(n)$ space.
- `std::priority_queue` of end times with a lazy expiry is the alternative at $O(n \log n)$.
- `sorted([s for s,_ in meetings])` plus `sorted([e for _,e in meetings])` and a manual two-pointer loop avoids heap overhead.
- A heapq of end times with `while heap and heap[0] <= s:  heappop` is very readable and competitive.

INTERVIEW TRAPS & SANITY CHECKS

Advancing the end pointer only while `ends[e] <= starts[s]`, which double-counts when several ends share a timestamp. The correct step compares timestamps and advances one side per iteration.

Sanity check. At every step, `count` equals the number of intervals whose start has been swept but whose end has not, which is exactly the number simultaneously in progress. Advancing the smaller next timestamp keeps that true, so the maximum recorded count is the true maximum overlap.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is the sweep correct when several events share a timestamp?

Answer. Because each iteration advances exactly one pointer by exactly one interval, and the comparison always picks the earlier event. Ties can be broken either way for half-open intervals, since a meeting ending at t and another starting at t never count together – the end is processed first.

Q2. Why does the heap version need a separate expiry loop?

Answer. A heap of end times can hold many stale entries whose ends are in the past. Before counting a new start you must pop every end <= s, not just the minimum, so the size reflects live meetings. That inner while loop is what keeps the heap meaningful.

Q3. What is the complexity?

Answer. $O(n \log n)$ time for the sorts, with an $O(n)$ sweep afterwards. Space is $O(n)$ for the two sorted arrays or the heap. A sweep-line over 2n events is the same bound with a different constant.

0435 Non-overlapping Intervals Medium

ESSENCE & BOUNDS

Return the maximum number of non-overlapping intervals that can be selected. 1 <= intervals.length <= 1e5 with up to 1e9 endpoints, and the answer is the count KEPT, not the count removed. Intervals touching at an endpoint do not overlap.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Greedy: sort by END, keep earliest-ending (BEST) Sort by end. Track the last KEPT end; intervals starting before it overlap the kept interval -> remove. Otherwise keep + advance.

Python (intuition kernel)

```python
if not intervals:
intervals.sort(key=lambda iv:  iv[1])
kept = 1 # the first interval is always kept
last_end = intervals[0][1] # end of the last KEPT interval
for i in range(1, len(intervals)):
 if intervals[i][0] < last_end:
 continue
 kept += 1

```

C++ (intuition kernel)

```cpp
 [](const std::vector<int>& a, const std::vector<int>& b) { return a[1] < b[1];

```

Time O(n log n) **Space** O(1)

the sort dominates; the post-sort scan is linear; hence the stated bounds.

Sort by start + DP dp[i] = best kept-count of a subset ending at i: dp[i] = 1 + max(dp[j]) for j < i with end[j] <= start[i] Answer = n - max(dp). Provably correct, no greedy leap — $O(n^2)$.

Python (intuition kernel)

```python
n = len(intervals)
if n == 0:
intervals.sort() # by start
dp = [1] * n
best = 1
for i in range(1, n):
 for j in range(i):
 if intervals[j][1] <= intervals[i][0]:

```

C++ (intuition kernel)

```cpp
for (int i = 1; i < n; ++i) { for (int j = 0; j < i; ++j) { if (intervals[j][1] <= intervals[i][0]) { dp[i] = std::max(dp[i], dp[j] + 1);
 }
 }
 best = std::max(best, dp[i]);

```

Time O(n^2) **Space** O(n)

the sort dominates; the post-sort scan is linear; hence the stated bounds.

VISUAL STATE TRACE

```
intervals=[[1,2],[2,3],[3,4],[1,3]] -> 2
sorted by END: (1,2) (2,3) (1,3) (3,4)
last_end = -inf, kept = 0
(1,2):  1 >= -inf -> keep, last_end = 2, kept=1
(2,3):  2 >= 2 -> keep, last_end = 3, kept=2
(1,3):  1 >= 3?  NO -> skip
(3,4):  3 >= 3 -> keep, last_end = 4, kept=3
-> 3, so the answer for this input is 3
WHY END-SORT IS THE GREEDY WIN:
exchange argument - let E be the earliest-ending
interval.  Any optimal solution's first interval O ends no
earlier than E, so replacing O with E leaves every later
interval still compatible.  Hence some optimal solution
starts with E, and the problem reduces to the same problem
on the intervals starting after E's end.
invariant:  after each accepted interval, last_end is the
end of the last KEPT one, and the kept set is optimal for
the prefix considered so far.
```

LANGUAGE MECHANICS: PYTHON VS C++

Greedy needs a proof, and interval scheduling has the cleanest one: if an optimal solution keeps some interval x first and x does not end first overall, swap x for the earliest-ending interval g (end[g] <= end[x]). Since g ends no later, every interval that was compatible with x is compatible with g — the swapped set is still optimal. Repeating the swap yields the greedy solution. The DP in Approach 2 needs no such argument but pays $O(n^2)$; the binary-search-over-dp-ends variant recovers $O(nlogn)$ with the same DP table (upper_bound on ends).
==========
/ #include <vector> // std::vector — intervals + dp table #include <algorithm> // std::sort, std::max #include <chrono> // std::chrono — benchmark timing #include <iostream> // std::cout — printing results #include <sys/resource.h> // getrusage — peak RAM measurement

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `sort` with a custom comparator on `.second`; a `vector<pair<int,int>>` of (end, start) makes the default lexicographic sort work.
- Reversing the pairs so end is first is the trick that avoids writing a comparator.
- `sorted(intervals, key=lambda iv: iv[1])` sorts by end explicitly.
- `intervals.sort(key=lambda iv: iv[1])` mutates in place, avoiding a copy of 1e5 tuples.

INTERVIEW TRAPS & SANITY CHECKS

Sorting by START instead of end. That greedy is not optimal: on [[1,100],[2,3],[3,4]] it takes [1,100] and keeps nothing else, giving 1 instead of the optimal 2.

Sanity check. Let E be the earliest-ending remaining interval. For any optimal solution, replacing its first interval with E cannot reduce the count, because E ends no later than anything it replaces, so all subsequent intervals remain compatible. Therefore taking E is safe, and the invariant holds inductively after restricting to intervals starting at or after E's end.

INTUITION CHECK — 3 QUESTIONS

Q1. Why does earliest-end beat earliest-start?

Answer. The exchange argument. Taking the earliest-ending interval leaves the most room for whatever comes after; taking the earliest-starting one may occupy a long span and block several short intervals that would otherwise fit.

Q2. What is the answer when the problem asks for removals?

Answer. It is the complement: intervals.length minus the maximum number kept. LeetCode 435 asks for removals, and the common bug is returning the kept count directly.

Q3. What if intervals touching at an endpoint overlap?

Answer. The comparison becomes > rather than >=. With half-open intervals touching is fine, which is the standard convention and why >= is correct here.

1851 Minimum Interval to Include Each Query Hard

ESSENCE & BOUNDS

For each query k, return the smallest interval covering k. 1 <= intervals.length, queries.length <= 1e4, and each query k equals intervals.length. The queries need not be sorted, which is why the offline reordering matters.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Offline: sort queries + sweep + min-heap (BEST) Sort (query, index) pairs. Pointer p walks intervals by start; push (length, end) for every start <= q. Pop while top's end < q (lazy deletion). Top is the shortest interval covering q.

Python (intuition kernel)

```python
intervals.sort() # by start -- arrival order for the sweep
pairs = sorted((q, i) for i, q in enumerate(queries))
heap:  List[tuple] = []
ans = [-1] * len(queries)
p = 0 # sweep pointer into intervals
for q, idx in pairs:
 while p < len(intervals) and intervals[p][0] <= q:
 s, e = intervals[p]

```

C++ (intuition kernel)

```cpp
sort(queries.begin(), queries.end());
sort(intervals.begin(), intervals.end());
priority_queue<pair<int,int>, vector<pair<int,int>>,
 greater<pair<int,int>> pq; // (length, right)
vector<int> ans(queries.size());
int i = 0;
for (auto [k, idx] :  queries) {
 while (i < intervals.size() && intervals[i].first <= k)
 pq.push({intervals[i].second - intervals[i].first,
 intervals[i].second});
 while (!pq.empty() && pq.top().second < k) pq.pop();
 ans[idx] = pq.empty() ?  -1 :  pq.top().first;
}
return ans;

```

Time O((n + q) log n) **Space** O(n + q)

the sort dominates; the post-sort scan is linear; hence the stated bounds.

Brute force, length-sorted (contrast) Sort intervals by length ascending; the first interval containing a query is the shortest one for it.

Python (intuition kernel)

```python
intervals.sort(key=lambda iv:  iv[1] - iv[0])
ans = [-1] * len(queries)
for i, q in enumerate(queries):
 for s, e in intervals:
 if s <= q <= e:
 ans[i] = e - s + 1 # shortest interval covering q
 break
return ans

```

C++ (intuition kernel)

```cpp
for (int k :  queries) {
 int best = INT_MAX;
 for (auto& iv :  intervals) // every query pays O(n)
 if (iv.first <= k && k <= iv.second)
 best = min(best, iv.second - iv.first);
 ans.push_back(best);
}

```

Time O(n * q) worst case	**Space** O(n)

nested enumeration dominates: $O(n \hat{} k)$ pairs, no pruning; hence the stated bounds.

VISUAL STATE TRACE

```
intervals=[[3,4],[2,3],[1,4]] queries=[2,3,4] -> [3,2,1]
sort queries:  (2,0) (3,1) (4,2)
sort intervals by start:  (1,4) (2,3) (3,4)
heap of (length, right), i pointer
k=2:  add all with start <= 2 -> (1,4) len 3
-> top = 3 answer[0] = 3
k=3:  add start <= 3 -> (2,3) len 1, (3,4) len 1
heap {(3,4),(1,3),(1,4)}
pop while heap[0].right < k=3 -> (1,3) popped
heap {(3,4),(1,4)}
top length = 1 answer[1] = 1
k=4:  pop while right < 4 -> (3,4) has right 4, keep
top = (3,4) len 1 answer[2] = 1
INVARIANT: the heap holds exactly the intervals whose
start <= k; after popping stale tops, every remaining entry
covers k.  So heap[0] is the minimum length among them.
```

LANGUAGE MECHANICS: PYTHON VS C++

A heap cannot remove arbitrary elements cheaply. Instead of deleting, we mark stale: entries whose END passed the current query are still inside the heap, but a peek at the top finds them (they can only be stale at the ROOT — all others are irrelevant while a smaller/equal item sits above). Popping while root is stale is amortized $O(1)$ per pop because every pushed entry is popped at most once. This "keep dead entries, evict on peek" pattern is the standard lazy-deletion idiom (also used by Dijkstra with duplicate distance entries).
==========
/ #include <vector> // std::vector — intervals + queries #include <queue> // std::priority_queue — min-heap of (len, end) #include <utility> // std::pair — heap entries + query pairs #include <algorithm> // std::sort, std::greater #include <functional> // std::greater<std::pair<int,int» #include <chrono> // std::chrono — benchmark timing #include <iostream> // std::cout — printing results #include <sys/resource.h> // getrusage — peak RAM measurement

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `priority_queue<pair<int,int>, vector<...>, greater<...»` keyed on (length, right) gives the minimum top directly.
- Sorting the queries pairs once and restoring results by the saved original index avoids any lookup.

- `heapq` with `(length, right)` tuples gives the same minimum-top semantics.
- Sorting `[(k, i) for i, k in enumerate(queries)]` keeps the original index attached, so results are restored in order.

INTERVIEW TRAPS & SANITY CHECKS

Popping expired intervals only from the top. An interval with right < k is unusable, but since it may not be the minimum length it can sit inside the heap; that is harmless as long as the top is cleaned, because a shorter expired entry would have been popped and re-checked.

Sanity check. After adding every interval with start <= k, the heap contains all candidates. After the while loop, no interval with right < k can remain at the top, so the top entry has start <= k <= right, meaning it covers k. Since the heap is a min-heap on length, the top is the shortest covering interval.

INTUITION CHECK — 3 QUESTIONS

Q1. Why sort the queries?

Answer. Because as k increases the set of covering intervals only changes by additions – intervals that stop covering k never start covering a larger k. So a single left-to-right sweep suffices for queries in increasing order. Sorting is what makes one pass possible instead of one pass per query.

Q2. Why is the heap keyed on length rather than start?

Answer. Because the answer is the minimum length. Keying on start would give the earliest-covering interval, which is not what is asked. The length is computed when the interval is inserted and never changes.

Q3. What is the complexity?

Answer. $O(n \log n + m \log m + n \log n + m)$ – sorting both lists plus at most one heap push and pop per interval. Space is $O(n + m)$.

17. Math & Geometry

#	Problem	Level	One-line intent
0043	**Multiply Strings**	Medium	Digit-array accumulation from the low end; no big integers.
0048	**Rotate Image**	Medium	Transpose, then reverse each row.
0050	**Pow(x, n)**	Medium	Exponentiation by squaring: halve n, square x.
0054	**Spiral Matrix**	Medium	Four boundaries that shrink after each pass.
0066	**Plus One**	Easy	Scan from the right, carry until a non-9 is found.
0073	**Set Matrix Zeroes**	?	Use the first row and column as the marker storage.
0201	**Bitwise AND of Numbers Range**	Medium	The AND of a range is the common prefix; right-shift to find it.
0202	**Happy Number**	?	Floyd cycle detection over the digit-square-sum chain.
2013	**Detect Squares**	?	Pick the bottom-left corner, then count pairs along its edges.

0043 `Multiply Strings` Medium

ESSENCE & BOUNDS

Return the product of two non-negative integer strings, without using built-in big-integer conversion. 1 <= num1, num2 <= 10 digits, so the product is at most 20 digits and needs a string. Leading zeros are not present but zeros inside are legal.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Digit-by-digit product array (BEST) res[i+j] accumulates a_i·b_j; carry flows to res[i+j+1] (the next power of ten). Exactly m+n slots (max digit count of the product).

Python (intuition kernel)

```python
if num1 == "0" or num2 == "0":
 return "0"
m, n = len(num1), len(num2)
res:  list[int] = [0] * (m + n)
for i in range(m - 1, -1, -1):  # least significant first
 a = int(num1[i]) # i-th digit
 for j in range(n - 1, -1, -1):
 b = int(num2[j]) # j-th digit

```

C++ (intuition kernel)

```cpp
for (int i = m - 1; i >= 0; -i) {
 const int a = num1[i] - '0'; // i-th digit (least significant first)
 for (int j = n - 1; j >= 0; -j) {
 const int b = num2[j] - '0'; // j-th digit
 res[i + j + 1] += a * b; // accumulate the pair product
 }

```

Time O(m-n) **Space** O(m+n)

Follows directly from the traversal count: $O(m-n)$ time, $O(m+n)$ space.

Row-by-row long multiplication (textbook form) Multiply num1 by each digit of num2, producing one partial row per digit ("123×6=738", "123×5=6150"), then add every row into a running sum. Same $O(m-n)$ math, two passes instead of one.

Python (intuition kernel)

```python
if num1 == "0" or num2 == "0":
 return "0"
m, n = len(num1), len(num2)
total:  list[int] = [0] * (m + n)
for j in range(n - 1, -1, -1):
 b = int(num2[j])
 carry = 0
 for i in range(m - 1, -1, -1):

```

C++ (intuition kernel)

```cpp
if (num1 == "0" || num2 == "0") { return "0";

```

Time O(m-n) **Space** O(m+n)

Follows directly from the traversal count: $O(m-n)$ time, $O(m+n)$ space.

VISUAL STATE TRACE

```
num1="123", num2="456" -> "56088"
m[3] n[3] (digit arrays, least significant first)
result[n+m] = zeros
for i in 0..2:  outer = num1
for j in 0..2:  inner = num2
mul = m[i]*n[j]
total = mul + result[i+j]
result[i+j] = total % 10
result[i+j+1] += total / 10 # CARRY to the next slot
each slot can accumulate more than 9 because we carry the
quotient forward rather than normalising immediately.
i=0 j=0:  3*6=18 -> r[0]=8, r[1]+=1
i=0 j=1:  3*5=15 -> r[1]=5, r[2]+=1
i=0 j=2:  3*4=12 -> r[2]=2, r[3]+=1
i=1 j=0:  2*6=12 -> r[1]=7, r[2]+=1
...
final digits (ls first) 8 8 0 8 6 5 -> "56088"
invariant:  after processing pairs, every position holds the
correctly weighted partial sum of the single-digit products,
with carries propagated to the next higher slot.
```

LANGUAGE MECHANICS: PYTHON VS C++

num1 = a_i-10^i and num2 = b_j-10^j, so by distributivity: num1 × num2 = _i _j (a_i-b_j) - 10^(i+j) The product of digits a_i-b_j ∈ [0,81] lands in positional slot (i+j); overflow above 9 is a CARRY that belongs to slot (i+j+1) — exactly the next power of ten. This is why res[i+j] accumulation needs no intermediate rows: carries are just deferred, and one final left-to-right pass normalizes them. The i+j trick turns the "shifted row + add" textbook picture into a single $O(m-n)$ sweep. ==========
/ #include <iostream> // std::cout — printing benchmark results #include <string> // std::string — digit strings #include <vector> // std::vector — product accumulator array #include <unordered_map> // (unused here; kept for uniform include set) #include <unordered_set> // (unused here; kept for uniform include set) #include <chrono> // std::chrono — steady_clock timing #include <sys/resource.h> // getrusage — peak RAM measurement

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `vector<int> res(n+m)` over exact-size digit arrays; converting back needs the leading-zero trim.
- `reverse()` on the digit vectors is $O(n)$ and removes the index arithmetic.

- Python's arbitrary-precision `int(num1) * int(num2)` works but violates the problem's intent – worth naming as a shortcut and rejecting.
- Reversing via `num1[::-1]` and indexing from the end avoids an explicit reverse call.

INTERVIEW TRAPS & SANITY CHECKS

Forgetting to accumulate the carry with += rather than assignment, or dropping the final carry slot. With 20 digits the product needs n+m slots exactly, so an off-by-one truncates the leading digit.

Sanity check. result[k] accumulates contributions from every digit pair (i, j) with i + j == k, weighted by 10^k. Each such contribution is m[i] * n[j] * 10^(i+j), so the sum over all pairs is exactly the product. Normalising each slot with % 10 and pushing the quotient to k+1 preserves the total.

INTUITION CHECK — 3 QUESTIONS

Q1. Why does the result need exactly n+m slots?

Answer. The product of an m-digit and an n-digit number has at most m+n digits. Any fewer would truncate, and one more would leave a guaranteed-zero leading slot. That bound is also why no slot can overflow outward.

Q2. Why is `result[i+j+1] += total / 10` safe?

Answer. Because total = mul + result[i+j] <= 81 + 9 + 9 = 99 in the worst case with fully normalised slots, so the quotient is at most 9 and fits in the next slot without further propagation. That bound relies on normalising every slot as you go, which the code does.

Q3. How does this differ from the textbook row-by-row method?

Answer. The textbook version adds whole rows, each requiring an $O(m)$ addition pass, giving $O(mn + mn) = O(mn)$ with much more code. The digit-array version writes directly into shared positions, so it is one pass of $O(mn)$ with no intermediate rows.

0048 Rotate Image Medium

ESSENCE & BOUNDS

Rotate an n x n square matrix 90 degrees clockwise in place. n x n is guaranteed square, which is what makes the two-step algorithm valid. Up to 20 x 20. Elements must not use an extra matrix.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Transpose + reverse (BEST) Step 1: swap (r, c) with (c, r) for r < c only (each pair once). Step 2: reverse every row.

Python (intuition kernel)

```python
n = len(matrix)
for r in range(n):
 for c in range(r + 1, n):
 matrix[r][c], matrix[c][r] = matrix[c][r], matrix[r][c]
for r in range(n):
 lo, hi = 0, n - 1
 while lo < hi:
 matrix[r][lo], matrix[r][hi] = matrix[r][hi], matrix[r][lo]

```

C++ (intuition kernel)

```cpp
for (int r = 0; r < n; ++r) { for (int c = r + 1; c < n; ++c) { const int tmp = matrix[r][c];
 matrix[r][c] = matrix[c][r];
 matrix[c][r] = tmp;
 }

```

Time O(n^2) **Space** O(1)

Follows directly from the traversal count: $O(n^2)$ time, $O(1)$ space.

In-place 4-way ring swaps (single pass) Peel the matrix ring by ring. On each ring, each top-edge cell (i, j) has a 4-cell orbit: (r,c) → (c,n-1-r) → (n-1-r,n-1-c) → (n-1-c,r) → back Rotate those 4 partners with one temp variable.

Python (intuition kernel)

```python
n = len(matrix)
for i in range(n // 2):
 last = n - 1 - i
 for j in range(i, last):
 offset = j - i # distance along the edge
 top = matrix[i][j]
 matrix[i][j] = matrix[last - offset][i]
 matrix[last - offset][i] = matrix[last][last - offset]

```

C++ (intuition kernel)

```cpp
for (int i = 0; i < n / 2; ++i) { const int last = n - 1 - i;
 for (int j = i; j < last; ++j) {
 const int offset = j - i; // how far along the edge
 const int top = matrix[i][j];
 matrix[i][j] = matrix[last - offset][i];
 matrix[last - offset][i] = matrix[last][last - offset];
 matrix[last][last - offset] = matrix[j][last];
 matrix[j][last] = top;

```

Time O(n^2) **Space** O(1)

the output array doubles as the state store; hence the stated bounds.

VISUAL STATE TRACE

```
[[1,2,3],[4,5,6],[7,8,9]] -> [[7,4,1],[8,5,2],[9,6,3]]
transpose (mirror on the main diagonal):
1 4 7
2 5 8
3 6 9
reverse each row (mirror on the anti-diagonal):
7 4 1
8 5 2
9 6 3
clockwise = transpose then reverse rows
counterclockwise = reverse rows then transpose
WHY: a 90-degree clockwise rotation maps (r,c) -> (c, n-1-r).
Transposing maps (r,c) -> (c,r); reversing the row maps
(c,r) -> (c, n-1-r).  The composition is the rotation.
transpose in place:
for r in 0..n-1:
for c in r+1..n-1:
swap(m[r][c], m[c][r])
```

LANGUAGE MECHANICS: PYTHON VS C++

Rotation is the linear map R = [[0,-1],[1,0]] in matrix form. Transpose T swaps indices; row-reversal F flips the second index. The identity F-T = R (right-multiply check: (F-T)(r,c) = F(c,r) = (c, n-1-r)) is why "transpose then reverse rows" is EXACTLY a 90° turn — no angles, no trigonometry, just index arithmetic. ========== / #include <iostream> // std::cout — printing benchmark results #include <string> // std::string — formatting rows for output #include <vector> // std::vector — the matrix #include <unordered_map> // (unused here; kept for uniform include set) #include <unordered_set> // (unused here; kept for uniform include set) #include <chrono> // std::chrono — steady_clock timing #include <sys/resource.h> // getrusage — peak RAM measurement

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `std::swap` plus two nested loops with the upper-triangular bound; the whole matrix stays in cache.
- `std::reverse` on each row of a `vector<int>` is a single library call per row.

- `[list(row) for row in zip(*matrix)]` transposes in one line, then reverse each row.
- `matrix.reverse()` on the outer list would rotate 180, not 90 – the common slip.

INTERVIEW TRAPS & SANITY CHECKS

Reversing the outer list instead of each row, which produces a 180-degree rotation. Also, looping the transpose over the full square rather than the strict upper triangle, which double-swaps and undoes itself.

Sanity check. After the transpose, entry (r, c) holds what was at (c, r); the strict-upper-triangle loop guarantees each pair is swapped exactly once. After the per-row reversal, entry (c, r') holds what the transpose put at (c, n-1-r'), which is the clockwise rotation.

INTUITION CHECK — 3 QUESTIONS

Q1. Why must the matrix be square?

Answer. The two-step method composes two reflections of the same square, so it is only a rotation when the dimensions match. For a non-square matrix you would need padding, or the explicit four-way ring rotation.

Q2. What is the alternative single-pass algorithm?

Answer. Four-way ring swaps: for each layer, swap the four elements at offsets 0, 1, 2, 3 of each ring position. That is one pass with $O(1)$ extra space and no intermediate transpose state, at the cost of harder-to-read index arithmetic.

Q3. What is the complexity?

Answer. $O(n^2)$ time and $O(1)$ extra space. The transpose visits n(n-1)/2 pairs and the reversal touches every element once.

0050 `Pow(x, n)` Medium

ESSENCE & BOUNDS

Return x raised to n for integer n, handling negative exponents. -2^31 <= n <= 2^31 - 1, so the exponent needs 64-bit arithmetic in C++ to shift safely. x can be 0, and 0 to the 0th power is 1.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Iterative exponentiation by squaring (BEST) Sweep the BITS of n. Maintain base = x^(2^k) (doubling k each step via base *= base). Whenever bit k of n is set, multiply base into the running result.

Python (intuition kernel)

```python
exp = n
if exp < 0:
 x = 1.0 / x
 exp = -exp
result = 1.0
base = x # base = x^(2^k) as the loop advances k
while exp > 0:
 if exp % 2 == 1:

```

C++ (intuition kernel)

```cpp
if (exp < 0) { x = 1.0 / x;
 exp = -exp;

```

Time O(log n) **Space** O(1)

Follows directly from the traversal count: $O(logn)$ time, $O(1)$ space.

Recursive halving Recurrence: n even → p = pow(x, n//2), return p·p; n odd → return x · p · p.

Python (intuition kernel)

```python
if n == 0:
 return 1.0 # x^0 = 1 for every x
if n < 0:
 return 1.0 / self.myPow(x, -n)
p = self.myPow(x, n // 2) # halve: p = x^(n//2)
(floor for odd n)
return p * p if n % 2 == 0 else p * p * x

```

C++ (intuition kernel)

```cpp
if (n == 0) { return 1.0; // x^0 = 1 for any x

```

Time O(log n) **Space** O(log n)

Follows directly from the traversal count: $O(logn)$ time, $O(logn)$ space.

VISUAL STATE TRACE

```
x=2, n=10 -> 1024
binary:  10 = 1010 = 8 + 2
result=1, base=2, exp=10
exp even -> exp=5, base=4 (square)
exp odd -> result*=4=4, exp=4, base=16
exp even -> exp=2, base=256
exp even -> exp=1, base=65536
exp odd -> result *= 65536 -> 262144?  no
trace carefully:
r=1 b=2 e=10 e even -> e=5, b=4
r=1 b=4 e=5 e odd -> r=4, e=4, b=16
r=4 b=16 e=4 e even -> e=2, b=256
r=4 b=256 e=2 e even -> e=1, b=65536
r=4 b=65536 e=1 e odd -> r=262144, e=0 -> stop
result 262144 != 1024 -> the doubling is wrong
CORRECT algorithm:
r=1 b=2 e=10
e even -> e=5, b=b*b=4
e odd -> r=r*b=4, e=4, b=b*b=16
e even -> e=2, b=256
e even -> e=1, b=65536
e odd -> r=r*b=262144, e=0
262144 = 2^18, not 2^10.  The binary of 10 is 1010;
the squares used were 2^2,2^4,2^8,2^16 and r took 2^2
and 2^16 -> so the update order must be:
if e odd:  r *= b; e -= 1
b *= b; e //= 2
r=1 b=2 e=10
e even:  b=4, e=5
e odd:  r=4, e=4; b=16, e=2
e even:  b=256, e=1
e odd:  r=4*256=1024, e=0 CORRECT
```

LANGUAGE MECHANICS: PYTHON VS C++

The recurrence is: pow(x, n) = pow(x, n/2)^2 (n even) pow(x, n) = x - pow(x, n/2)^2 (n odd, integer div) because n = 2k ⇒ x^n = (x^k)^2, and n = 2k+1 ⇒ x^n = x-(x^k)^2. Subtlety: exponent n = -2^31 cannot be negated in a signed 32-bit int (overflow!). Safe pattern: if n < 0, return 1.0 / pow(x, -n) using a LONG (64-bit) negated exponent. Doubles need no overflow worries — multiplication and division are IEEE-754 exact enough.
==========
/ #include <iostream> // std::cout — printing benchmark results #include <string> // std::string — formatting result lines #include <vector> // std::vector — (unused; uniform include set) #include <unordered_map> // (unused here; kept for uniform include set) #include <unordered_set> // (unused here; kept for uniform include set) #include <chrono> // std::chrono — steady_clock timing #include <sys/resource.h> // getrusage — peak RAM measurement

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `long long n` is mandatory: shifting a 32-bit n right is fine but the multiplication needs 64-bit intermediates.
- `unsigned long long` for the accumulator avoids signed-overflow UB on intermediate squares even though the final answer fits.

- Python's `**` handles this in one operation; writing the loop is about demonstrating the algorithm.
- `pow(x, n)` with three arguments does modular exponentiation, useful if a modulus is added.

INTERVIEW TRAPS & SANITY CHECKS

Squaring the base before consuming the odd bit, or decrementing e after halving it. The two lines must be ordered: multiply into the result and subtract one while odd, THEN square and halve.

Sanity check. The loop maintains result * base^e == x^n_original. Each iteration either squares (preserving base^e as (base^2)^(e/2) when e is even) or extracts one factor of base into result and decrements e (preserving the product). When e reaches 0, result is the answer.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is the algorithm logarithmic?

Answer. Every iteration halves e, so there are at most log2(|n|) iterations – at most 31 for a 32-bit exponent. Naive repeated multiplication is $O(n)$ and would be astronomically slow for n = 2^31.

Q2. How are negative exponents handled?

Answer. They are not, structurally: the halving loop terminates only for positive e. You must first convert x^-k into (1/x)^k, which requires either a float or integer division, and x = 0 with a negative exponent is undefined.

Q3. Why does 0^0 equal 1 here?

Answer. Because result is initialised to 1 and the loop returns result when e becomes 0, before any base multiplication. That is the standard convention and matches the judge's expectation.

0054 Spiral Matrix Medium

ESSENCE & BOUNDS

Return all elements of a matrix in spiral order. m, n >= 1 with up to 1e6 elements, so a single row or column must terminate correctly. The 1 x n case is the classic off-by-one source.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Boundary shrinking with 4 pointers (BEST) Track top, bottom, left, right. Sweep each of the four edges of the current layer in turn, shrinking its bound after the sweep. Two guards (top <= bottom / left <= right) keep single-row and single-column layers from being double-visited.

Python (intuition kernel)

```python
out: List[int] = []
top, bottom = 0, len(matrix) - 1
left, right = 0, len(matrix[0]) - 1
while top <= bottom and left <= right:
 for c in range(left, right + 1):
 out.append(matrix[top][c])
 top += 1 # top edge consumed $\{}to$ shrink
 for r in range(top, bottom + 1):

```

C++ (intuition kernel)

```cpp
while (top <= bottom && left <= right) { for (int c = left; c <= right; ++c) { out.push_back(matrix[top][c]);
 }
 ++top; // top edge consumed $\{}to$ shrink from above
 for (int r = top; r <= bottom; ++r) { out.push_back(matrix[r][right]);
 }
 --right; // right edge consumed $\{}to$ shrink from the right
 if (top <= bottom) { for (int c = right; c >= left; --c) { out.push_back(matrix[bottom][c]);
 }

```

Time O(m-n) **Space** O(1) extra

Follows directly from the traversal count: $O(m - n)$ time, $O(1)$ extra space.

Layer-by-layer recursive peeling Same spiral as a recursion: peel the outer layer, recurse on the inner (m-2)×(n-2) rectangle. Same $O(m - n)$ work; the recursion stack costs O(min(m, n)/2) memory instead of $O(1)$.

Python (intuition kernel)

```python
out: List[int] = []
self._peel(matrix, 0, len(matrix) - 1, 0, len(matrix[0]) - 1, out)
return out
l: int, r: int, out: List[int]) -> None:

```

C++ (intuition kernel)

```cpp
if (t > b || l > r) {
 return; // layer collapsed $\{}to$ whole spiral is emitted

```

Time O(m-n) **Space** O(min(m, n)/2)

Follows directly from the traversal count: $O(m-n)$ time, O(min(m, n)/2) space.

VISUAL STATE TRACE

```
[[1,2,3],[4,5,6],[7,8,9]] -> 1 2 3 6 9 8 7 4 5
top=0 bottom=2 left=0 right=2
while top <= bottom and left <= right:
top row:  1 2 3 (c from left to right)
right col:  6 9 (r from top+1 to bottom)
top++
bottom row:  8 7 (only if top <= bottom!)
right-
left col:  4 (only if left <= right!)
bottom-
THE GUARDS ARE ESSENTIAL:
after the top row, bottom- must be conditional on
top <= bottom, else a 1-row matrix emits a duplicate row.
after the right col, right- must be conditional on
left <= right, else a 1-column matrix emits duplicates.
trace:
top row 1 2 3 ; right col 6 9 ; top=1
bottom row 8 7 (bottom=2 >= top=1 ok) ; right=1 ; bottom=1
left col 4 ; bottom=0 -> loop ends
```

LANGUAGE MECHANICS: PYTHON VS C++

The spiral partitions the grid into disjoint segments: every cell belongs to exactly one of the four sweeps of exactly one layer. Invariant: after processing layer k, all cells with k ≤ min(r, c, m-1-r, n-1-c) are visited. The layer index of any cell is min of its 4 edge distances — exactly what the guards track. Total work = over layers of (perimeter) = m-n. ========== / #include <iostream> // std::cout — printing benchmark results #include <string> // std::string — formatting result lines #include <vector> // std::vector — matrix + spiral output #include <unordered_map> // (unused here; kept for uniform include set) #include <unordered_set> // (unused here; kept for uniform include set) #include <chrono> // std::chrono — steady_clock timing #include <sys/resource.h> // getrusage — peak RAM measurement

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- Four `int` boundaries and a lambda `push` that appends and advances; one lambda removes four duplicated lines.
- `vector<int> out; out.reserve(m*n)` avoids repeated reallocation.

- A list of four boundaries with `while top <= bottom and left <= right` is the cleanest form.
- `out.extend(row[left:right+1])` for the top row is C-speed, though the explicit loop is more uniform.

INTERVIEW TRAPS & SANITY CHECKS

Emitting the bottom row or left column without re-checking the corresponding boundary condition. On a single-row matrix that duplicates every element; on a single-column matrix it duplicates the column.

Sanity check. The four boundaries always delimit the unvisited rectangle. Each pass emits exactly the outer ring and then shrinks the appropriate boundaries by one, so every element is emitted exactly once and the loop terminates when the rectangle is empty.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is there no extra space?

Answer. The traversal order is determined entirely by the four boundaries, so no visited array or queue is needed. A BFS-based spiral would need $O(mn)$ extra space; the boundary walk uses $O(1)$ beyond the output.

Q2. What is the output space complexity?

Answer. $O(mn)$ for the result, which is unavoidable since every element is returned. Auxiliary space is $O(1)$, making this an output-optimal $O(mn)$ algorithm.

Q3. How do you handle a 1 x n matrix?

Answer. The top row emits all n elements, then the loop must not emit a bottom row. The `if top <= bottom` guard before the bottom-row pass is what enforces that; without it you emit the row twice.

0066 Plus One Easy

ESSENCE & BOUNDS

Increment a number represented as an array of digits by one. 1 <= digits.length <= 100, so the array can represent a number far beyond any fixed integer type. All elements are 0..9 and the leading digit is never zero, except for the array [0].

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Backwards scan with carry (BEST) carry = 1 at the last digit; for each digit right → left: add carry; 10 → set 0 and continue; otherwise return immediately. All-9 input → prepend a leading 1.

Python (intuition kernel)

```python
out = digits[:]  # work on a copy
for i in range(len(out) - 1, -1, -1):
 if out[i] < 9:
 out[i] += 1 # carry absorbed right here
 return out # done -- left digits unchanged
 out[i] = 0 # 9+1 = 10 $\{}to$ 0, carry on
return [1] + out # [9,9,9] $\{}to$ [1,0,0,0]

```

C++ (intuition kernel)

```cpp
for (int i = static_cast<int>(out.size()) - 1; i >= 0;
-i) { if (out[i] < 9) {
 out[i] += 1; // carry absorbed right here
 return out; // done -- digits left unchanged
 }
 out[i] = 0; // 9+1 = 10 $\{}to$ 0, carry propagates

```

Time O(n) **Space** O(1) extra

Follows directly from the traversal count: $O(n)$ time, $O(1)$ extra space.

Integer round-trip (contrast) int(digits) + 1 → back to digits. Correct only while the value fits a machine int; silently wrong beyond ~19 digits.

Python (intuition kernel)

```python
value = 0
for d in digits:
 value = value * 10 + d
value += 1
out:  List[int] = []
while value > 0:
 out.append(value % 10)
 value //= 10

```

C++ (intuition kernel)

```cpp
for (const int d :  digits) { value = value * 10 +
d;

```

Time O(n) **Space** O(n)

Follows directly from the traversal count: $O(n)$ time, $O(n)$ space.

VISUAL STATE TRACE

```
[1,2,5] -> [1,2,6]
[4,3,2,2,9] -> [4,3,2,3,0]
[9,9] -> [1,0,0] (new leading digit)
[0] -> [1]
i = n-1
while i >= 0 and digits[i] == 9:
digits[i] = 0 # 9 -> 0, carry out
i -= 1
if i >= 0:
digits[i] += 1
else:
digits.insert(begin, 1) # every digit was 9
WHY THE 9-RUN MATTERS: only trailing 9s produce a carry.
The first non-9 from the right absorbs the carry and stops
it.  If no non-9 exists, the answer gains a digit.
trace [4,3,2,2,9]:
i=4 digits[4]=9 -> 0, i=3
i=3 digits[3]=2 != 9 -> stop
digits[3] += 1 -> [4,3,2,3,0] CORRECT
```

LANGUAGE MECHANICS: PYTHON VS C++

The carry propagates exactly until the first non-9 digit from the right. Average case: a uniformly random digit is 9 with probability 1/10, so the expected walk length is 1 + 1/10 + 1/100 + \ldots{} = 10/9 $\rightarrow$ $O(1)$ average! Only the worst case (all 9s) costs the full $O(n)$ walk. Amortized over many increments, this is the same analysis as a binary counter's carry chain. ========== / #include <iostream> // std::cout — printing benchmark results #include <string> // std::string — formatting result lines #include <vector> // std::vector — the digit array #include <unordered_map> // (unused here; kept for uniform include set) #include <unordered_set> // (unused here; kept for uniform include set) #include <chrono> // std::chrono — steady_clock timing #include <sys/resource.h> // getrusage — peak RAM measurement

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `digits.insert(digits.begin(), 1)` is $O(n)$ and fine; a two-pointer fill into a preallocated vector is $O(1)$ amortised instead.
- The int-to-string conversion is deliberately forbidden – that is the point of the problem.
- `[int(d) for d in str(int(s) + 1)]` violates the intent; it also fails on a 100-digit string in fixed-width languages but works in Python, which is exactly why the loop is the right answer here.
- `digits.insert(0, 1)` is a single $O(n)$ operation and reads clearly.

INTERVIEW TRAPS & SANITY CHECKS

Checking only the last digit. [9,9] and [1,9,9] both need the carry to propagate and a new leading digit, and treating them as single-digit cases returns wrong answers.

Sanity check. After the while loop, all digits at indices greater than i have been set to 0 and i is the highest index whose digit is not 9 (or -1 if none exists). Those trailing zeros are exactly the carry-out positions, and incrementing digits[i] – or prepending 1 when i == -1 – performs the addition with no further propagation.

INTUITION CHECK — 3 QUESTIONS

Q1. Why does only trailing 9s carry?

Answer. Because adding 1 to a digit d yields d+1 unless d == 9, which yields 10 and writes 0 with a carry. So the carry propagates leftward precisely through a maximal run of 9s and stops at the first smaller digit.

Q2. What if every digit is 9?

Answer. Then i reaches -1 and the answer needs one more digit than the input. That is the [9,9] to [1,0,0] case, and prepending 1 is the only correct handling.

Q3. What is the complexity?

Answer. $O(n)$ time – each digit is touched at most twice – and $O(1)$ extra space apart from the possible insert, which is $O(n)$ and happens at most once.

0073 Set Matrix Zeroes ?

ESSENCE & BOUNDS

Set every cell sharing a row or column with an original zero to zero, in place. m, n up to 200 and the output matrix is huge – copying it is explicitly disallowed. The first row and first column may themselves be zeroed, which is the whole subtlety.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

v1 v1 – see the kernel below.

Python (intuition kernel)

```python
ROWS, COLS = len(matrix), len(matrix[0])
rowZero = False
for r in range(ROWS):
 for c in range(COLS):
 if matrix[r][c] == 0:
 matrix[0][c] = 0
 if r > 0:  matrix[r][0] = 0
 else:  rowZero = True

```

C++ (intuition kernel)

```cpp
for (int r = 0; r < m; ++r) { for (int c = 0; c < n;
++c) { if (matrix[r][c] == 0) { if (c == 0) {
 zero_first_col = true; // (0,0) is taken by row 0
 } else { matrix[0][c] = 0;
 }
 matrix[r][0] = 0;
 }
 }

```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

v2 v2 – see the kernel below.

Python (intuition kernel)

```python
r_zero = set(); c_zero = set()
for r in range(len(matrix)):
 for c in range(len(matrix[0])):
 if matrix[r][c] == 0:
 r_zero.add(r); c_zero.add(c)
for r in range(len(matrix)):
 for c in range(len(matrix[0])):
 if r in r_zero or c in c_zero:

```

C++ (intuition kernel)

```cpp
for (int r = 0; r < m; ++r) { for (int c = 0; c < n;
++c) { if (matrix[r][c] == 0) { zero_row[r] = true;
 zero_col[c] = true;
 }
 }

```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

VISUAL STATE TRACE

```
[[1,1,1],[1,0,1],[1,1,1]]
the 0 at (1,1) means row 1 and column 1 become 0:
1 0 1
0 0 0
1 0 1
THREE-STEP ALGORITHM:
1.  mark rows/cols that contain a zero, storing the flags
IN THE FIRST ROW and FIRST COLUMN
2.  for each cell (r,c) with r>0 and c>0:
if row_flag[r] or col_flag[c]:  m[r][c] = 0
3.  zero out the first row and first column from their flags
the first row/column are used as the marker array because
the problem forbids extra space.  If m[0][0] itself is a
zero, every cell must be zero - step 3 handles that from
the flags, not from the overwritten value.
why the corner cell needs a separate flag:
if only m[0][0] were zero we'd wipe the marker storage
before reading it.  Keeping the flags in row 0 / col 0 and
processing them LAST avoids the ordering problem.
```

LANGUAGE MECHANICS: PYTHON VS C++

The failure mode of a naive in-place attempt: you zero a row, and the zeros you just WROTE get re-read as "original zeros", cascading the zeroing everywhere. The marker trick sidesteps it because Pass 1 only READS original data and only WRITES to the marker row/column; Pass 2 only READS markers. The two passes never interleave — that separation is the whole algorithm. ========== / #include <iostream> // std::cout — printing benchmark results #include <string> // std::string — formatting result lines #include <vector> // std::vector — the matrix #include <unordered_map> // (unused here; kept for uniform include set) #include <unordered_set> // (unused here; kept for uniform include set) #include <chrono> // std::chrono — steady_clock timing #include <sys/resource.h> // getrusage — peak RAM measurement

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `vector<bool> rowFlag(m), colFlag(n)` is the $O(m+n)$ extra space version the problem forbids but interviewers often accept as the first attempt.
- `std::vector<char>` avoids vector<bool>'s proxy overhead and is clearer than a bitset.
- `rows = set(), cols = set()` is the readable $O(m+n)$ version.
- The three-step in-place version is what the problem asks for; the marker row/column trick is the point.

INTERVIEW TRAPS & SANITY CHECKS

Using m[0][0] as the sole marker for row 0 and column 0 simultaneously. If the original m[0][0] was a zero, the marker loses its meaning and the answer is wrong; separate flags in row 0 and column 0, applied last, are required.

Sanity check. After step 1, `rowFlag[r]` and `colFlag[c]` record whether row r and column c contained an original zero, and the interior of the matrix is still unmodified. Step 2 therefore zeroes exactly the cells sharing a row or column with a zero, using only original information. Step 3 applies the flags to the marker row and column last, so their values are read before being destroyed.

INTUITION CHECK — 3 QUESTIONS

Q1. Why must the first row/column be processed last?

Answer. Because they hold the flags. If you zeroed them earlier you would lose the record of whether rows 0 or columns 0 contained zeros. Applying them last means the information has already been consumed by the interior pass.

Q2. What is the alternative with extra space?

Answer. Two boolean arrays of size m and n. That is $O(m+n)$ space, trivially correct, and worth writing first – the in-place version is the interesting follow-up that the constraint forces.

Q3. What is the complexity?

Answer. $O(mn)$ time for the three passes and $O(1)$ extra space for the in-place version. Each cell is examined a constant number of times.

0201 Bitwise AND of Numbers Range Medium

ESSENCE & BOUNDS

Return m & (m+1) & ... & n for 0 <= m <= n <= 2^31-1. 0 <= m <= n <= 2^31 - 1, so 31 bits suffice. m == n returns m. The answer is always a power-of-two-aligned block.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

- Clear the lowest set bit (best) while left < right: right &= right - 1. Converges to the common prefix.

Python (intuition kernel)

```python
while left < right:
 right &= right - 1
return right

```

C++ (intuition kernel)

```cpp
while (left != right) { // find common prefix
 left >>= 1;
 right >>= 1;
 ++shift;

```

Time ? **Space** ?

each element is probed once; pairs cancel or are recorded; hence the stated bounds.

- Shift both until equal, shift back Shift left/right rightwards until they match; the prefix shifted back left is the answer. ——————————————————————— DEEP DIVE - Why the prefix survives Where left and right first differ, the range spans the carry boundary (e.g. 5=101 to 7=111: the middle bit flips) — so that bit and everything below is 0 in the AND. Only bits where both endpoints (hence every number) agree stay 1. right&(right-1) removes the lowest set bit; iterating until right $\leq$ left lands exactly on the shared prefix. ——————————————————————— TIME COMPLEXITY : $O(1)$ — at most 31 iterations. MEMORY COMPLEXITY: $O(1)$ ==========

Python (intuition kernel)

```python
shift = 0
while left != right:
 left >>= 1
 right >>= 1
 shift += 1
return left << shift

```

C++ (intuition kernel)

```cpp
while (left != right) { // find common prefix
 left >>= 1;
 right >>= 1;
 ++shift;

```

Time O(1) — at most 31 iterations. **Space** O(1)

one shift-and-mask per bit position; hence the stated bounds.

VISUAL STATE TRACE

```
m=5, n=7 -> 4
5 = 101
6 = 110
7 = 111
AND:
101
110
111
--
100 = 4
COMMON PREFIX: the top bit (bit 2) is shared; everything
below differs because the range crosses a power-of-two
boundary at 8.
shift-right approach:
sh = 0
while m != n:  m >>= 1; n >>= 1; sh += 1
return m << sh
m=5 n=7 -> 2,3 sh=1 -> 1,1 sh=2 -> stop
return 1 << 2 = 4 CORRECT
shift-until-equal alternative:
while m < n:  m >>= 1; n >>= 1
return m
invariant:  shifting both endpoints right by the same amount
preserves the common prefix, and they become equal exactly
when the remaining bits are all that they share.  Shifting
back left restores the zeroes below the prefix, which is the
required AND.
```

LANGUAGE MECHANICS: PYTHON VS C++

Where left and right first differ, the range spans the carry boundary (e.g. 5=101 to 7=111: the middle bit flips) — so that bit and everything below is 0 in the AND. Only bits where both endpoints (hence every number) agree stay 1. right&(right-1) removes the lowest set bit; iterating until right $\leq$ left lands exactly on the shared prefix. ——————————————————————— TIME COMPLEXITY : $O(1)$ — at most 31 iterations.
==========

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `unsigned` avoids sign extension on » for the 31-bit bound, though int is also correct since both operands stay non-negative.
- The shift-counter form generalises to any range type; the `while m < n` form is a C++/Python one-liner but relies on m <= n.
- `m ^ n` gives the bits that differ; their highest set bit marks the shift amount, so `shift = m ^ n .bit_length()` is an equivalent single-expression form.
- Python's » is a logical shift on positive ints, so no masking is needed anywhere in this problem.

INTERVIEW TRAPS & SANITY CHECKS

\begin{itemize} \item U \item s \item i \item n \item g \item \item ‘ \item w \item h \item l \item e \item m \item ! \item = \item : \item > \item 1 \item ; \item t \item o \item u \item a \item f \item c \item r \item d \item . \item T \item p \item x \item , \item 5 \item 7 \item v \item 4 \end{itemize}

Sanity check. A t e v r y s p b o h n d i a f g m u , l . T c q x w N D

INTUITION CHECK — 3 QUESTIONS

Q1. Why does m == n after shifting imply the answer?

Answer. When both endpoints collapse to the same value v, every number in the range agrees with v on all remaining high bits and the lower bits of the range contain a transition, so the AND of those low bits is 0. Hence the answer is v followed by sh zeros.

Q2. What is the simpler formulation?

Answer. `while m < n: m >>= 1; n >>= 1; return m`. Because m <= n always, and m < n guarantees a differing low bit, the loop shifts out exactly the non-common part, so no shift counter is needed.

Q3. What is the complexity?

Answer. $O(\log n)$ time – at most 31 shifts – and $O(1)$ space. In C++ use `unsigned` to avoid sign-extension issues on the right shift; in Python the integers are arbitrary-precision and never negative here.

0202 Happy Number ?

ESSENCE & BOUNDS

Return true if repeatedly summing the squares of the digits reaches 1. Any integer, typically within int. Numbers above 243 always fall into a cycle, so the chain is bounded and the loop must terminate.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

v1 v1 – see the kernel below.

Python (intuition kernel)

```python
visit = set()
while n not in visit:
 visit.add(n)
 n = self.sumOfSquares(n)
 if n == 1:  return True
output = 0
while n:
 digit = n % 10

```

C++ (intuition kernel)

```cpp
while (n > 0) { const int d = n % 10; // peel the last digit
 sum += d * d; // square it into the sum
 n /= 10; // drop the digit

```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

v2 v2 – see the kernel below.

Python (intuition kernel)

```python
slow, fast = n, self.sumOfSquares(n)
while fast != 1 and slow != fast:
 slow = self.sumOfSquares(slow)
 fast = self.sumOfSquares(self.sumOfSquares(fast))
return fast == 1
output = 0
while n:
 digit = n % 10

```

C++ (intuition kernel)

```cpp
while (n > 0) { const int d = n % 10;
 sum += d * d;
 n /= 10;

```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

VISUAL STATE TRACE

```
n=19 -> true
19 -> 1 + 81 = 82
82 -> 64 + 4 = 68
68 -> 36 + 64 = 100
100 -> 1 + 0 + 0 = 1 HAPPY
n=2 -> false
2 -> 4 -> 16 -> 37 -> 58 -> 89 -> 145 -> 42 -> 20
-> 4 (cycle detected)
Floyd on the function f(n) = sum of digit squares:
slow = f(n), fast = f(f(n))
while fast != 1 and slow != fast:
slow = f(slow); fast = f(fast))
return fast == 1
WHY IT TERMINATES: any n >= 1000 maps to at most 4*81=324,
and 999 -> 243, so the chain is bounded by 243 and must
eventually repeat.  Floyd detects the repeat in O(1) space.
ALTERNATIVE: a set of visited values, O(1) space in
practice since the chain is bounded by 243, and clearer.
```

LANGUAGE MECHANICS: PYTHON VS C++

For a k-digit number the successor is $\leq$ k-81. For k $\geq$ 4 that is strictly smaller than the number itself (10^(k-1) > 81k for k $\geq$ 4), so every chain drops into $\leq$ 3-digit territory fast and the state space is bounded $\rightarrow$ cycles are GUARANTEED to be found, never an infinite walk. This is the same argument that proves termination of the whole process. ========== / #include <iostream> // std::cout — printing benchmark results #include <string> // std::string — formatting result lines #include <vector> // std::vector — (unused; uniform include set) #include <unordered_map> // (unused here; kept for uniform include set) #include <unordered_set> // std::unordered_set — seen-state set (A1) #include <chrono> // std::chrono — steady_clock timing #include <sys/resource.h> // getrusage — peak RAM measurement

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `int sumOfSquares(int n)` with a `while (n) { s += (n%10)*(n%10); n /= 10; }` loop.
- An `unordered_set<int>` visited set is simpler but allocates; Floyd avoids that entirely.

- `sum(int(c)**2 for c in str(n))` is concise and readable at this scale.
- `divmod(n, 10)` gives digit and quotient in one operation, avoiding two separate divisions.

INTERVIEW TRAPS & SANITY CHECKS

Using a visited set without a termination bound, or forgetting that the cycle may not include 1 – so the loop condition must check both `fast == 1` and `slow == fast`, otherwise a non-1 cycle spins forever.

Sanity check. The chain n -> f(n) -> f(f(n)) -> ... is eventually periodic because it is bounded. Floyd's two pointers advance at speeds 1 and 2, so they must meet inside the cycle; meeting at a value of 1 means the chain reaches 1, and meeting elsewhere means it is trapped.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is the chain bounded?

Answer. A number with d digits maps to at most 81d. For d = 4 that is 324, and 324 has three digits so it maps to at most 243. From then on the chain stays inside 1..243, a finite set, so it must repeat.

Q2. What is the general technique here?

Answer. Cycle detection on a deterministic function of a bounded domain – exactly Floyd's tortoise and hare, identical in structure to linked-list cycle detection and to the duplicate-number problem. The invariant is that the two pointers meet inside the cycle.

Q3. Why does meeting at 1 mean happy?

Answer. If the chain reaches 1 then it stays at 1, since f(1) = 1. So 1 is a fixed point, and the fast pointer reaches it within at most one full cycle of steps. Meeting elsewhere means the chain entered a cycle that does not contain 1.

2013 Detect Squares ?

ESSENCE & BOUNDS

Return the total number of axis-aligned squares in which at least one other point shares the bottom row and at least one shares the left column. 0 <= points.length <= 1000 with 0 <= x[i], y[i] <= 1000, so coordinates fit in 10 bits and a 1001 x 1001 counting grid is about 4 MB. Duplicate points are legal and are the reason the answer multiplies two counts.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Approach 1 Approach 1 – see the kernel below.

Python (intuition kernel)

```python
rows, cols = Counter(), Counter()
for x, y in points:
 rows[y] += 1
 cols[x] += 1
res = 0
for x, y in points:
 right = rows[y] - 1 # minus (x, y) itself
 up = cols[x] - 1
 if right > 0 and up > 0:  res += right * up
return res

```

C++ (intuition kernel)

```cpp
unordered_map<int, int> rowCnt, colCnt;
for (auto& p :  points) { ++rowCnt[p.second];
++colCnt[p.first]; }
int res = 0;
for (auto& p :  points) {
 int right = rowCnt[p.second] - 1;
 int up = colCnt[p.first] - 1;
 if (right > 0 && up > 0) res += right * up;
}
return res;

```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

Approach 2 Approach 2 – see the kernel below.

Python (intuition kernel)

```python
cnt = Counter(points)
row, col = Counter(), Counter()
for (x, y) in points:
 row[y] += cnt[(x, y)] # weight duplicates
 col[x] += cnt[(x, y)]
res = 0
for (x, y) in points:
 right = row[y] - cnt[(x, y)]
 up = col[x] - cnt[(x, y)]
 if right > 0 and up > 0:  res += right * up * cnt[(x,
y)]
return res

```

C++ (intuition kernel)

```cpp
// weight every point by how many times it occurs,
// then the same row/col product scales by that weight

```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

VISUAL STATE TRACE

```
points = [(0,0),(1,0),(0,1),(1,1),(0,2)] -> 1
for each (x, y) treat it as the BOTTOM-LEFT corner:
(0,0):  right = # points at (x'>0, y) = 2 -> (1,0),(1,1)
up = # points at (x, y'>0) = 2 -> (0,1),(0,2)
both sides present -> +1 * 1 = 1 square
(1,0):  right = 0 -> skip
(0,1):  right = 1 -> (1,1); up = 1 -> (0,2)
-> +1 square (0,1)-(1,1)-(1,2)?  (1,2) absent
but the opposite orientation also counts:
the point (0,1) can be the TOP-LEFT of the
square whose bottom-left is (0,0).  So every
square is found twice unless you only count
the (x, y) = bottom-left orientation and use
right * up.
RIGHT * UP IS THE TRAP: a square needs a point strictly to
the RIGHT (x' > x) and one strictly ABOVE (y' > y), and
the two must pair up into a distinct square.  Multiplying
the two counts is valid because each pair determines a
unique corner (x', y').
```

LANGUAGE MECHANICS: PYTHON VS C++

These problems are arithmetic on small integers, so both languages are fast. The C++ concern is overflow – intermediate products in multiply-strings, reverse-integer, and pow can exceed 32 bits – while Python's arbitrary precision removes that class of bug entirely. Matrix rotations differ more meaningfully: both work in place on the same memory, but C++ gets in-register swaps where Python writes through list indirection.

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `unordered_map<int, unordered_map<int,int>>` counts per point in $O(1)$ expected but allocates two nodes per lookup; a flat 1001 x 1001 int grid indexed `x * 1001 + y` is a single 4 MB block with perfect locality.
- Sorting by (x, y) lets the right-count be read off from a running group size, avoiding the nested map entirely.
- `collections.Counter` over the points gives per-point counts, but the row and column tallies need two separate counters keyed by y and by x.
- A 2-D list of 1001 x 1001 ints is about 8 MB of pointers plus small-int caching; a flat list of 1e6 entries indexed `x * 1001 + y` is one allocation.

INTERVIEW TRAPS & SANITY CHECKS

Using >= instead of > for the neighbour tests, which makes a point on the same row or column count as a side and inflates the answer. Also iterating both the $(\text{right}, \text{up})$ and $(\text{left}, \text{down})$ products, which double-counts every square.

Sanity check. For each point (x, y), right(x, y) counts points on row y strictly right of x and up(x, y) counts points on column x strictly above y. Every square has exactly one bottom-left corner, and its two far sides are exactly one member of each set, so right * up counts each square exactly once from that corner.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is `right * up` correct rather than a combination count?

Answer. Because the problem defines the square by one point on the bottom row and one on the left column of the chosen corner. Each ordered pair (right point, up point) determines one unique opposite corner, so the number of squares is exactly the product of the two counts.

Q2. Why does a single corner orientation avoid double counting?

Answer. Every axis-aligned square has a unique bottom-left corner and a unique top-right corner. Anchoring on the bottom-left with strict right/up neighbours means each square is visited only at its bottom-left point, so no counting rule needs a divide-by-two.

Q3. What is the complexity?

Answer. $O(n)$ time for n points, with $O(1)$ expected lookup per point. Space is $O(\sigma)$ for the coordinate maps or $O(10^6)$ for a flat counting grid when the coordinate range is bounded.

18. Bit Manipulation

#	Problem	Level	One-line intent
0007	**Reverse Integer**	Medium	Pre-multiply guard against INT_MIN; or use long long.
0136	**Single Number**	Easy	XOR cancels pairs; the leftover is the answer.
0190	**Reverse Bits**	Easy	Loop 32 times: shift the input in, shift the output out.
0191	**Number of 1 Bits**	Easy	n &= n - 1 clears the lowest set bit; count the iterations.
0268	**Missing Number**	Easy	XOR the range 0..n with the array: pairs cancel.
0338	**Counting Bits**	?	ones[i] = ones[i » 1] + (i & 1).
0371	**Sum Of Two Integers**	?	AND for the sum, XOR for the carry, recurse until carry is 0.

0007 `Reverse Integer` Medium

ESSENCE & BOUNDS

Return the integer with its 32-bit signed digits reversed, clamped to the 32-bit signed range. -2^31 <= x <= 2^31 - 1, and the reversed value may fall outside that range – e.g. 1534236469 reverses to 9646324351. Python has no such limit, so a naive reverse works there.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Pop & push with pre-multiply guards (BEST) Check `result > INT_MAX // 10` before growing (any growth overflows) or `result == INT_MAX // 10 and digit > 7` (only the largest digit overflows at the exact boundary). Mirror on the negative side with -214748364 and guard digit -8 — two's complement gives the negative range one extra unit, so the two guard digits DIFFER (7 vs -8).

Python (intuition kernel)

```python
neg = x < 0
n = -x if neg else x # work on magnitude:  // truncates
result = 0
while n != 0:
 digit = n % 10
 n //= 10
 if result > INT32_MAX // 10 or (result == INT32_MAX // 10 and digit > 7):
 result = result * 10 + digit

```

C++ (intuition kernel)

```cpp
while (n != 0) {
 int d = static_cast<int>(n % 10); // last digit (sign preserved)
 rev = rev * 10 + d; // append to the growing answer
 n /= 10; // drop the digit we consumed

```

Time O(log10 |x|) **Space** O(1)

Follows directly from the traversal count: $O(log10|x|)$ time, $O(1)$ space.

Unbounded accumulation, single final range check Exploit Python's big ints: accumulate WITHOUT guards (nothing can overflow), then clamp once at the end: if the reversal escapes [-2^31, 2^31 - 1] → 0. Same loop, fewer branches, and it makes the "mask to 32 bits" idea explicit: the hardware does this clamping silently, so the algorithm must do it deliberately.

Python (intuition kernel)

```python
sign = -1 if x < 0 else 1
value = abs(x)
result = 0
while value != 0:
 result = result * 10 + (value % 10)
 value //= 10
result *= sign
if result < INT32_MIN or result > INT32_MAX:

```

C++ (intuition kernel)

```cpp
long long r = 0;
while (x != 0) { r = r * 10 + x % 10; x /= 10; }
if (r > INT_MAX) return INT_MAX;
if (r < INT_MIN) return INT_MIN;
return (int)r; // 64-bit accumulator makes this safe

```

Time O(log10 |x|) **Space** O(1)

Follows directly from the traversal count: $O(log10|x|)$ time, $O(1)$ space.

VISUAL STATE TRACE

```
x = 123 -> 321
x = -123 -> -321
x = 120 -> 21 (leading zero dropped)
x = 1534236469 -> OUT OF RANGE -> 9646324351 > 2^31-1
-> must return 2147483647 (INT_MAX)
x = -2147483648 (INT_MIN) -> reverses to -8463847412
-> must return -2147483648
THE OVERFLOW TRAP: before appending a digit, check whether
result would exceed INT_MAX/10, or equal it with a digit > 7.
Without the guard, 1534236469 overflows and C++ signed
overflow is undefined behaviour - it may silently wrap.
guard:
if (result > INT_MAX / 10) return INT_MAX;
if (result == INT_MAX / 10 && d > INT_MAX % 10)
return INT_MAX;
PYTHON needs no guard:  arbitrary precision means the
reversed value is computed exactly and then compared.
```

LANGUAGE MECHANICS: PYTHON VS C++

Modulo 10 is a single x86 'idiv' remainder; integer multiply by 10 and add is one IMUL + one ADD — no heap allocation, no parsing overhead, no UTF-8 decode. The 32-bit bound is INT32_MAX = 2147483647 and INT32_MIN = -2147483648; building the result in a 64-bit long guarantees the comparison is always well defined (no UB from signed overflow during the build itself). Also note: 64-bit build handles INT32_MIN safely — INT32_MIN / -1 overflows in 32-bit, but never in 64-bit. ——————————————————————— TIME COMPLEXITY : $O(log10|x|)$ = $O(1)$ — at most 10 digits.

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- The guard approach avoids any 64-bit arithmetic and stays within int.
- `long long` accumulation is simpler: the reversed value of a 32-bit int always fits in 64 bits, so build it there and clamp at the end.
- `int(str(abs(x))[::-1]) * (1 if x >= 0 else -1)` is one line and exact, since Python ints are unbounded.
- `divmod(x, 10)` returns (quotient, remainder) with floored division, which handles negatives correctly in one call.

INTERVIEW TRAPS & SANITY CHECKS

Using `abs(INT_MIN)` in C++, which overflows because 2147483648 is not representable as int. `int(-INT_MIN)` is undefined behaviour – the classic manifestation is returning -2147483648 forever.

Sanity check. After k iterations, `result` holds the last k digits of x reversed, and `x` holds the remaining leading digits. Each step removes the last digit of x and appends it to result, so the combined value is preserved until the final clamp.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is the pre-multiply guard needed rather than clamping afterwards?

Answer. Because by the time the overflow has happened the value is already wrong – signed overflow is undefined behaviour, so the compiler may have produced anything. Detecting it before the multiplication keeps every intermediate value representable.

Q2. How does Python avoid this entirely?

Answer. Arbitrary-precision integers mean no intermediate value is ever out of range. The reversed value is computed exactly and then compared against the 32-bit limits, which is the clean approach – the C++ guard exists purely to work around a language restriction.

Q3. What is the complexity?

Answer. $O(\log_{10}|x|)$ time – one iteration per digit, at most 10 – and $O(1)$ space. No string conversion is needed in either language.

0136 Single Number Easy

ESSENCE & BOUNDS

Return the one element that appears exactly once while every other element appears twice. 1 <= nums.length <= 3e4 with values up to 3e4, so int suffices and the length is always odd. Exactly one such element is guaranteed to exist.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

XOR everything (BEST) acc = 0; for num in nums: acc ^= num. Pairs vanish (x ^ x = 0, x ^ 0 = x); order is irrelevant (commutative + associative).

Python (intuition kernel)

```python
acc = 0
for num in nums:
 acc ^= num
return acc

```

C++ (intuition kernel)

```cpp
for (int num :  nums) { acc ^= num;

```

Time O(n) **Space** O(1)

one accumulator pass, $O(1)$ space; hence the stated bounds.

Hash set toggle (O(n) space baseline) Add on first sight, remove on second. The only key left when the array ends is the singleton. Correct but fails the $O(1)$-space requirement — shown as the baseline XOR defeats.

Python (intuition kernel)

```python
seen:  set[int] = set()
for num in nums:
 if num in seen:
 seen.remove(num) # second occurrence $\{}to$ pair complete
 else:
 seen.add(num) # first occurrence $\{}to$ remember it
return next(iter(seen))

```

C++ (intuition kernel)

```cpp
for (int num :  nums) { auto it = seen.find(num);
 if (it != seen.end()) {
 seen.erase(it); // second occurrence $\{}to$ pair complete
 } else {
 seen.insert(num); // first occurrence $\{}to$ remember it
 }

```

Time O(n) **Space** O(n)

one $O(1)$ hash probe per element, no rescans; hence the stated bounds.

VISUAL STATE TRACE

```
nums = [4,1,2,1,2]
x ^ x = 0 and x ^ 0 = x, and XOR is associative
and commutative, so pair order is irrelevant:
4 ^ 1 ^ 2 ^ 1 ^ 2
= 4 ^ (1 ^ 1) ^ (2 ^ 2)
= 4 ^ 0 ^ 0
= 4
every paired value annihilates, leaving the single one.
Requires EVEN counts of the paired values - the reason
this fails for problem 137 where three copies appear.
invariant:  at any point, `r` equals the XOR of all elements
processed so far.  Since XOR is self-inverse, after the whole
pass only elements with odd multiplicity survive, and the
guarantee says exactly one does.
```

LANGUAGE MECHANICS: PYTHON VS C++

XOR truth table: 0^0=0, 0^1=1, 1^0=1, 1^1=0. Two identities: x ^ 0 = x (xor with zero is identity) x ^ x = 0 (xor with itself cancels) And xor is commutative (a^b = b^a) and associative ((a^b)^c = a^(b^c)), so the array order is irrelevant. Group every pair: (v1 ^ v1) ^ (v2 ^ v2) ^ ... ^ s = 0 ^ 0 ^ ... ^ s = s This is EXACTLY why disk arrays use XOR for parity: it can be undone by itself. ==========
/ #include <iostream> // std::cout — printing benchmark results #include <vector> // std::vector — the input array #include <cstdint> // std::int32_t — fixed-width element type #include <climits> // INT_MIN — edge case for the test table #include <chrono> // std::chrono::steady_clock — timing for benchmark #include <sys/resource.h>// getrusage — peak RAM measurement for benchmark #include <unordered_set> // std::unordered_set — approach 2's toggle set

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- A single accumulator `int r = 0; for (int x : nums) r ^= x;` – the tightest possible loop.
- `unsigned` avoids any concern about sign extension in the XOR, though int is correct here.

- `functools.reduce(operator.xor, nums, 0)` is a C-level fold.
- Bitwise NOT in Python yields -1-style results because Python integers are infinite-precision two's complement, so be careful with masking.

INTERVIEW TRAPS & SANITY CHECKS

Using XOR when three copies of a value may appear (problem 137). Then an odd count leaves the value itself rather than zero, and the single XOR is wrong – you need per-bit counts in that case.

Sanity check. `r` is always the XOR of the prefix processed so far. Because XOR is associative and commutative, the elements can be reordered freely, so every equal pair cancels regardless of adjacency. After the full pass the result is the XOR of all elements with odd multiplicity, which by the guarantee is the single one.

INTUITION CHECK — 3 QUESTIONS

Q1. Why does XOR cancel pairs but not triples?

Answer. Because a ^ a = 0, so two copies vanish. Three copies leave a ^ a ^ a = 0 ^ a = a. So XOR isolates elements with ODD multiplicity – exactly the guarantee for this problem and precisely the failure case for problem 137.

Q2. Why does the element order not matter?

Answer. XOR is both associative and commutative, so any parenthesisation and any permutation gives the same value. That is what allows pair cancellation even when the duplicates are far apart in the array.

Q3. What is the complexity?

Answer. $O(n)$ time and $O(1)$ space – one accumulator, one pass. That is optimal, since every element must be read.

0190 Reverse Bits Easy

ESSENCE & BOUNDS

Reverse the order of the bits of a 32-bit unsigned integer. The input is always a 32-bit unsigned integer, so exactly 32 iterations are required regardless of the value. A zero input must give zero, not be skipped.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Shift-and-OR assembly, 32 iterations (clear) result = 0; 32 times: result = (result « 1) | (n & 1); n »= 1. The input's bits are consumed low → high and emitted high → low.

Python (intuition kernel)

```python
n &= MASK32
n = ((n >> 16) | (n << 16)) & MASK32
n = (((n & 0xFF00FF00) >> 8) | ((n & 0x00FF00FF) << 8)) & MASK32
n = (((n & 0xF0F0F0F0) >> 4) | ((n & 0x0F0F0F0F) << 4)) & MASK32
n = (((n & 0xCCCCCCCC) >> 2) | ((n & 0x33333333) << 2)) & MASK32
n = (((n & 0xAAAAAAAA) >> 1) | ((n & 0x55555555) << 1)) & MASK32
return n

```

C++ (intuition kernel)

```cpp
for (int i = 0; i < 32; ++i) { result = (result << 1) | (n & 1);
 n >>= 1;

```

Time O(1) — exactly 32 fixed iterations. **Space** O(1)

one shift-and-mask per bit position; hence the stated bounds.

5 masked block swaps (divide & conquer) Swap halves, then 8-bit quarters, 4-bit blocks, 2-bit blocks, adjacent bits — five ops, no loop. Masks isolate every other block of the current size: 0xFF00FF00, 0xF0F0F0F0, 0xCCCCCCCC, 0xAAAAAAAA (and their bitwise complements) for sizes 8, 4, 2, 1.

Python (intuition kernel)

```python
result = 0
for _ in range(32):
 result = (result << 1) | (n & 1)
 n = (n >> 1) & MASK32
return result & MASK32

```

C++ (intuition kernel)

```cpp
n = ((n & 0xaaaa) >> 1) | ((n & 0x5555) << 1);
n = ((n & 0xcccc) >> 2) | ((n & 0x3333) << 2);
n = ((n & 0xf0f0) >> 4) | ((n & 0x0f0f) << 4);
n = ((n & 0xff00) >> 8) | ((n & 0x00ff) << 8);
n = (n >> 16) | (n << 16);
return n; // branch-free, five stages

```

Time O(1) — FIVE operations. **Space** O(1)

the recursion splits the work; the bound is the sum over levels; hence the stated bounds.

VISUAL STATE TRACE

```
input = 43261596 (0b00000010100101010000011110)
32 iterations, always:
result = (result « 1) | (n & 1)
n »= 1
after 32 steps result = 0b01111000111001001111000 = 964176192
WHY THE FIXED LOOP: a shorter loop (while n) would stop at
the highest set bit, but the answer must be a FULL 32-bit
reversal - leading zeros of the input become trailing zeros
of the output.  So the loop must run exactly 32 times.
ALTERNATIVE: swap 5-bit blocks, then 3-bit, then the
remaining bits - a divide-and-conquer reversal in 3 steps:
n = ((n & 0xaaaa) » 1) | ((n & 0x5555) « 1)
n = ((n & 0xcccc) » 2) | ((n & 0x3333) « 2)
n = ((n & 0xf0f0) » 4) | ((n & 0x0f0f) « 4)
n = ((n & 0xff00) » 8) | ((n & 0x00ff) « 8)
n = (n » 16) | ((n & 0xffff) « 16)
input 0b00000010100101010000011110
reverse 0b01111000111001001111000
```

LANGUAGE MECHANICS: PYTHON VS C++

Start from the requirement: output bit b must equal input bit 31b. Split at the midpoint: input bits 0..15 must land in output bits 16..31, i.e. SWAP THE HALVES. Repeat inside each half (8-bit quarters), then inside each quarter (4-bit blocks), etc. The masks isolate every other block of size 2^k: swap halves: (x » 16) | (x « 16) swap 8s: ((x & 0xFF00FF00) » 8) | ((x & 0x00FF00FF) « 8) swap 4s: ((x & 0xF0F0F0F0) » 4) | ((x & 0x0F0F0F0F) « 4) swap 2s: ((x & 0xCCCCCCCC) » 2) | ((x & 0x33333333) « 2) swap 1s: ((x & 0xAAAAAAAA) » 1) | ((x & 0x55555555) « 1) Each stage preserves the work of the previous one because the masks only touch bits that already live in mirrored positions.
==========
/ #include <iostream> // std::cout — printing benchmark results #include <vector> // std::vector — the test-case table #include <cstdint> // std::uint32_t — the 32-bit operand type #include <climits> // UINT32_MAX — edge case for the test table #include <chrono> // std::chrono::steady_clock — timing for benchmark #include <sys/resource.h>// getrusage — peak RAM measurement for benchmark

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `uint32_t` with a fixed `for (int i = 0; i < 32; ++i)` is the canonical form.
- The masked-block version is branch-free and runs in constant time with a handful of instructions, and is a classic bit-twiddling showcase.
- `int(f'{n:032b}'[::-1], 2)` is a one-liner but allocates a 32-character string and is much slower.
- Python's » on a positive int is a logical shift, so no sign handling is needed.

INTERVIEW TRAPS & SANITY CHECKS

Using `while n` instead of a 32-iteration loop. That stops early for inputs with leading zero bits, and the answer loses the trailing zeros the reversal should produce.

Sanity check. After i iterations, `result` holds the reversal of the i lowest bits of the original input, and n holds the remaining higher bits shifted down. Extending by one iteration appends the next input bit as the new most significant bit of result, which is exactly the reversal order.

INTUITION CHECK — 3 QUESTIONS

Q1. Why exactly 32 iterations?

Answer. Because the reversal is defined over 32 bits. Input bits that are zero still occupy positions, and they become trailing zeros in the output. A loop that stops when n reaches zero would silently drop them.

Q2. What happens to 0?

Answer. All 32 iterations run with n = 0, each doing `result = result « 1 | 0`, so result stays 0. The answer is correctly 0 rather than undefined.

Q3. Why might the masked-block version be faster?

Answer. It is branch-free, performs a fixed 5 operations instead of 32 iterations, and each mask-and-shift pair is a couple of machine instructions. It also expresses the reversal as three stages of larger block swaps, which is why it generalises to wider integers.

0191 Number of 1 Bits Easy

ESSENCE & BOUNDS

Return the population count of a non-negative 32-bit integer. 0 <= n <= 2^31 - 1, and n = 0 gives 0. Kernighan's algorithm is loop-invariant, so it runs once per set bit.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

Kernighan's algorithm (BEST) count = 0; while n: n &= n - 1; count += 1. One round per SET bit. Derivation of x & (x - 1): x = ...1 0 0 0 (lowest 1-bit at position k) x - 1 = ...0 1 1 1 (borrow: bit k → 0, below → all 1s) x & (x-1) = ...1-1 — 0 — 0-1 = x minus its lowest 1-bit

Python (intuition kernel)

```python
count = 0
n &= MASK32
while n != 0:
 n &= n - 1 # clears the lowest set bit (see deep dive)
 count += 1
return count

```

C++ (intuition kernel)

```cpp
while (n != 0) {
 n &= n - 1; // clears the lowest set bit (see deep dive)
 ++count;

```

Time O(popcount) $\leq$ O(32) = O(1) **Space** O(1)

Follows directly from the traversal count: $O(popcount) \leq O(32) = O(1)$ time, $O(1)$ space.

Scan all 32 bit positions count = 0; for i in range(32): count += n & 1; n »= 1. Checks every position including zeros — simple, 32 fixed rounds.

Python (intuition kernel)

```python
count = 0
for _ in range(32):
 count += n & 1
 n = (n » 1) & MASK32
return count

```

C++ (intuition kernel)

```cpp
for (int i = 0; i < 32; ++i) { count += n & 1;
 n »= 1;

```

Time O(32) = O(1) **Space** O(1)

constant passes over the bits of the value; hence the stated bounds.

VISUAL STATE TRACE

```
n = 13 -> 3
13 = 1101
n & (n - 1) clears the LOWEST set bit:
13 = 1101
12 = 1100
--
4 = 0100 one set bit gone -> count = 1
step 1:  n=13 (1101) -> n=12 (1100) count=1
step 2:  n=12 (1100) -> n= 8 (1000) count=2
step 3:  n= 8 (1000) -> n= 0 (0000) count=3
n == 0 -> return 3
iterations == popcount, NOT 32 - the loop-invariant
is the whole algorithm
```

LANGUAGE MECHANICS: PYTHON VS C++

Write x in binary: ...1 0 0 0, where the lowest 1-bit sits at position k with only 0-bits below it. Subtracting 1 borrows through those zeros, flipping the bit at k to 0 and every bit below it to 1: x = ...1 0 0 0 (bits above k unchanged) x - 1 = ...0 1 1 1 (bit k cleared, below becomes all-ones) Now AND them: positions above k match (both have the original bits), position k is 1 & 0 = 0, positions below are 0 & 1 = 0. Result: x with its lowest 1-bit removed — exactly one bit cleared, everything else untouched. ==========
/ #include <iostream> // std::cout — printing benchmark results #include <vector> // std::vector — the test-case table #include <cstdint> // std::uint32_t — the 32-bit operand type #include <climits> // UINT32_MAX — edge case for the test table #include <chrono> // std::chrono::steady_clock — timing for benchmark #include <sys/resource.h>// getrusage — peak RAM measurement for benchmark

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- `__builtin_popcount` compiles to a single POPCNT instruction on modern CPUs, which beats any loop – worth naming before writing the manual version.
- `std::bitset<32>` exposes a `count()` member with the same hardware advantage and no UB concerns on the shift.
- `bin(n).count('1')` is a one-liner but allocates a string of up to 32 characters and is much slower than the loop.
- Python 3.10+ exposes `int.bit_count()`, which is implemented in C and matches Kernighan's speed for small integers.

INTERVIEW TRAPS & SANITY CHECKS

\begin{itemize} \item M \item u \item t \item a \item i \item n \item g \item \item s \item d \item e \item h \item l \item o \item p \item c \item , \item ‘ \item w \item (\item \& \item = \item - \item 1 \item) \item m \item b \item r \item f \item y \item v \item . \item T \item x \end{itemize}

Sanity check. E a c h i t e r o n m v s x l y w b d u , f k q g p .

INTUITION CHECK — 3 QUESTIONS

Q1. Why does n & (n-1) clear the lowest set bit?

Answer. Subtracting 1 flips the lowest 1 to 0 and all trailing 0s to 1. ANDing keeps only the bits that are 1 in both, so the lowest 1 is removed and every higher bit is untouched.

Q2. How many iterations does it take?

Answer. Exactly the number of set bits, popcount(n). That is a genuine improvement over scanning all 32 positions, which is the alternative at worst case.

Q3. How does this relate to the C++ builtin?

Answer. `__builtin_popcount` compiles to a single POPCNT instruction on modern CPUs using a population-count microcode table, which beats any loop. Kernighan's algorithm is the version you write when the builtin is unavailable or when you want the derivation.

0268 Missing Number Easy

ESSENCE & BOUNDS

Return the one missing integer from the range 0..n inclusive. 1 <= nums.length <= 1e4 with 0 <= nums[i] <= 1e4, and exactly one value from 0..n is absent. n is nums.length by construction.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

XOR of all indices and values (BEST) acc = n; for i: acc ^= i ^ nums[i]. Present pairs cancel; acc ends as the unmatched index = the missing number. No overflow.

Python (intuition kernel)

```python
acc = len(nums)
for i, num in enumerate(nums):
 acc ^= i ^ num
return acc

```

C++ (intuition kernel)

```cpp
for (int i = 0; i < static_cast<int>(nums.size());
++i) { acc ^= i ^ nums[i];

```

Time O(n) **Space** O(1)

one accumulator pass, $O(1)$ space; hence the stated bounds.

Sum formula expected = n(n+1)/2; return expected sum(nums).

Python (intuition kernel)

```python
n = len(nums)
expected = n * (n + 1) // 2
return expected - sum(nums)

```

C++ (intuition kernel)

```cpp
for (int v :  nums) { actual += v;

```

Time O(n) **Space** O(1)

Follows directly from the traversal count: $O(n)$ time, $O(1)$ space.

VISUAL STATE TRACE

```
nums=[3,0,1] (n=3) -> missing 2
XOR 0^1^2^3 = 0
XOR nums = 3^0^1 = 2
answer = 0 ^ 2 = 2
why it works:  every value in 0..n appears exactly once
in the full range XOR; the present values cancel against it,
leaving only the missing one.
alternative (sum formula):
expected = n(n+1)/2 ; answer = expected - sum(nums)
XOR is preferred because the sum formula overflows int
for large n in C++ (n = 1e4 gives 5e7, safe; n = 1e5
would overflow - and it also breaks silently under
floating-point if you use /2.0).
invariant:  at any point 'r' is the XOR of the range XOR
accumulated so far and the array elements processed so far.
Present values appear twice and cancel; the missing one
appears once and survives.
```

LANGUAGE MECHANICS: PYTHON VS C++

We KNOW the full range {0..n} has n+1 distinct values. Its XOR depends only on n. Now xor in every ARRAY element: acc = n acc ^= 0 ^ nums[0] ^ 1 ^ nums[1] ^ ... ^ (n-1) ^ nums[n-1] Rearrange (commutative): every value v that IS present meets its own index v and cancels (v ^ v = 0). The one index m whose value is absent has nothing to cancel with — it is the sole survivor. Identical machinery to 0136 Single Number — same proof, new hat.
==========
/ #include <iostream> // std::cout — printing benchmark results #include <vector> // std::vector — the input array #include <cstdint> // std::int64_t — overflow-free sum lane #include <climits> // INT_MAX — scale check for the test table #include <chrono> // std::chrono::steady_clock — timing for benchmark #include <sys/resource.h>// getrusage — peak RAM measurement for benchmark

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- A single `int r = n; for (i=0..n-1) r ^= nums[i] ^ i;` folds the range XOR into one loop with no separate pass.
- `long long` for the sum formula if you choose that route, since `n*(n+1)/2` overflows int past n = 65535.
- `reduce(xor, nums, n)` plus a range fold is concise but allocates the range.
- The sum formula `n*(n+1)//2 - sum(nums)` is a one-liner in Python and exact.

INTERVIEW TRAPS & SANITY CHECKS

Using the sum formula with float division or with int overflow in C++. Both give wrong answers on large inputs, whereas the XOR approach has no arithmetic range concern at all.

Sanity check. `r` equals the XOR of 0..n XOR the elements of nums. Every value present in nums occurs in both and cancels; the missing value occurs only in the range, so it is what survives. The result is therefore exactly the missing number.

INTUITION CHECK — 3 QUESTIONS

Q1. Why is XOR better than the sum formula?

Answer. XOR has no overflow or precision hazard, and the same single loop handles the range and the array. The sum formula needs a wider type and careful division, both of which are failure modes in C++.

Q2. How does the index trick work?

Answer. Initialising r to n and then XORing with both nums[i] and i in the same loop folds the range 0..n-1 in for free, so only one pass is needed instead of one over the range and one over the array.

Q3. What is the complexity?

Answer. $O(n)$ time and $O(1)$ space. Both approaches are linear; XOR is preferred purely for robustness rather than speed.

0338 Counting Bits ?

ESSENCE & BOUNDS

Return an array where the element at index i is the number of 1 bits in i, for every i from 0 to n. 0 <= n <= 1e5, so the array has up to 1e5 entries and each count fits in a byte. i = 0 gives 0.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

v1 v1 – see the kernel below.

Python (intuition kernel)

```python
dp = [0] * (n + 1)
offset = 1
for i in range(1, n + 1):
 if offset * 2 == i:  offset = i
 dp[i] = 1 + dp[i - offset]
return dp

```

C++ (intuition kernel)

```cpp
for (int i = 1; i <= n; ++i) { dp[i] = dp[i >> 1] + (i & 1);

```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

v2 v2 – see the kernel below.

Python (intuition kernel)

```python
dp = [0] * (n + 1)
for i in range(1, n + 1):
 dp[i] = dp[i & (i - 1)] + 1
return dp

```

C++ (intuition kernel)

```cpp
for (int i = 1; i <= n; ++i) { dp[i] = dp[i & (i - 1)] + 1;

```

Time ? **Space** ?

Follows directly from the traversal count: ? time, ? space.

VISUAL STATE TRACE

```
n = 5 -> [0,1,1,2,1,2]
ones[0] = 0
i=1:  1»1 = 0, 1&1 = 1 -> 0 + 1 = 1
i=2:  2»1 = 1, 2&1 = 0 -> 1 + 0 = 1
i=3:  3»1 = 1, 3&1 = 1 -> 1 + 1 = 2
i=4:  4»1 = 2, 4&1 = 0 -> 1 + 0 = 1
i=5:  5»1 = 2, 5&1 = 1 -> 1 + 1 = 2
THE INSIGHT: shifting right by one removes the lowest bit
and leaves the rest unchanged, so
popcount(i) = popcount(i » 1) + (i & 1)
The RHS term is already computed since i » 1 < i.
BIT TRICK: ones[i] = ones[i & (i-1)] + 1 is the same idea
expressed with the clear-lowest-bit identity from problem 191.
invariant:  when computing ones[i], ones[i » 1] is already
final because i » 1 < i, and (i & 1) is exactly the bit
that i » 1 discarded.  So the sum is the popcount of i.
```

LANGUAGE MECHANICS: PYTHON VS C++

Splitting a number at the bit boundary: i = (i » 1) « 1 | (i & 1). The set bits of i = the set bits of (i » 1) — those are i's bits shifted down one position, count preserved — PLUS the lowest bit (i & 1), which the shift dropped. Since i » 1 is strictly smaller than i for i > 0, induction on i gives every value in $O(1)$. For i = 13 = 11012: dp[13] = dp[6] + 1 = dp[3] + 0 + 1 = 3. ========== / #include <iostream> // std::cout — printing benchmark results #include <vector> // std::vector — the output dp array #include <cstdint> // std::int32_t — element type of the dp table #include <climits> // INT_MAX — scale check for the test table #include <chrono> // std::chrono::steady_clock — timing for benchmark #include <sys/resource.h>// getrusage — peak RAM measurement for benchmark

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- A `vector<uint8_t>` of n+1 bytes; the values never exceed 32 so a full int array wastes 4x the cache.
- For the same reason POPCNT-style primitives give $O(n)$ with no table, but the DP table is the point of this problem.
- `[bin(i).count('1') for i in range(n+1)]` is a one-liner but allocates n strings and is orders of magnitude slower.
- `int.bit_count()` in Python 3.10+ is the C-level equivalent and still $O(n)$ overall with no extra table.

INTERVIEW TRAPS & SANITY CHECKS

\begin{itemize} \item I \item t \item e \item r \item a \item i \item n \item g \item \item u \item p \item w \item d \item h \item l \item ' \item o \item s \item [\item > \item 1 \item] \item f \item m \item y \item z \item , \item < \item v \item 0 \item - \item q \item c \item . \end{itemize}

Sanity check. W h e n c o m p u t i g s [ ] , > 1 a l r d y f b < (&) . T

INTUITION CHECK — 3 QUESTIONS

Q1. Why does the DP beat calling popcount n times?

Answer. Each popcount is $O(\log n)$ at worst without a builtin, giving $O(n \log n)$ overall. The recurrence computes every answer from a strictly smaller index, so the total is $O(n)$ with one shift, one AND, and one addition per element.

Q2. How does this relate to problem 191?

Answer. Same identity, different output. Here the identity is applied across indices to build a table; there it is applied within a single number to count its bits. Both come from the fact that clearing the lowest bit reduces popcount by exactly one.

Q3. What is the complexity?

Answer. $O(n)$ time and $O(n)$ space for the result, which is required since every value from 0 to n must be returned. No extra space beyond the output.

0371 Sum Of Two Integers ?

ESSENCE & BOUNDS

Return the sum of two integers using only bitwise operators. -1e5 <= a, b <= 1e5, so the sum fits in int. Negative numbers work because Python's integers are infinite two's complement; C++ needs explicit sign handling or the non-negative assumption.

MULTI-SOLUTION INTUITION (PYTHON VS C++)

v1 v1 – see the kernel below.

Python (intuition kernel)

```python
mask = 0xFFFFFFFF
while b != 0:
 tmp = (a & b) << 1
 a = (a ^ b) & mask
 b = tmp & mask
if a > 0x7FFFFFFF: return ~(a ^ mask)
return a

```

C++ (intuition kernel)

```cpp
while (b != 0) {
 int carry = (a & b) << 1; // carry from the ORIGINAL a
 a = a ^ b; // sum without carry
 b = carry;
}
return a;

```

Time ?	Space ?

Follows directly from the traversal count: ? time, ? space.

VISUAL STATE TRACE

```
a = 1, b = 2
a ^ b = 3 (no common bits -> sum is 3)
(a & b) « 1 = 0 -> no carry -> answer 3
a = 1, b = 3
a ^ b = 2 (01 ^ 11 = 10)
(a & b) « 1 = 4 (01 & 11 = 01, shifted)
now add 2 and 4:  2^4 = 6, (2&4)«1 = 0 -> answer 6
WHY IT WORKS: addition decomposes into a sum-without-carry
part (XOR, since a 1-bit appears in at most one addend) and
a carry part (AND, since both addends must have 1 to carry).
The carry must then be added to the XOR result, recursively.
termination:  (a & b) « 1 has strictly fewer set bits than
a | b, so the recursion depth is at most the bit width.
negative numbers:  in two's complement, -1 is all 1s, so
-1 ^ x == ~x and (a & b) « 1 can be negative.  The loop
'while b:' with 'b = (a & b) « 1' still terminates because
the shift eventually overflows to 0 in a fixed width.
```

LANGUAGE MECHANICS: PYTHON VS C++

Bit operations map almost directly onto single machine instructions in C++, so this category shows the largest per-operation gap. Python's integers are arbitrary-precision objects, so `&`, `|`, `^` and `»` allocate a new object per operation unless CPython's small-int cache applies – and Python's `~x` yields a negative number because integers are infinite two's complement. For pure bit work C++ is the natural home.

ALTERNATIVE DATA STRUCTURES & OPTIMISATIONS

- In C++ with signed ints, `a ^ b` and `(a & b) « 1` behave as two's complement only if the width is fixed; shifting into the sign bit is implementation-defined before C++20.
- `uint32_t` avoids all sign questions and the problem's inputs are non-negative there.

- `while b: a, b = a ^ b, (a & b) « 1` uses simultaneous assignment so the carry is computed from the OLD a – which is exactly why the tuple form is correct and two separate statements would not be.
- `while b: a, b = a ^ b, (a & b) « 1` uses simultaneous assignment so the carry is computed from the ORIGINAL a, which is exactly why the tuple form is correct.
- `divmod` plays no role here – the whole computation is bitwise, which is what keeps it working for negative operands.

INTERVIEW TRAPS & SANITY CHECKS

Computing the carry after already updating a. The carry must be derived from the original a and b, so in Python the simultaneous assignment `a, b = a ^ b, (a & b) « 1` is essential – doing `a ^= b` then `b = (a & b) « 1` uses the already-modified a and gives wrong answers.

Sanity check. At the top of each iteration, the original sum equals a + b. The update replaces that with `(a ^ b) + ((a & b) « 1)`, which is an exact identity for two's-complement addition. When b becomes 0 the identity forces a to equal the original sum.

INTUITION CHECK — 3 QUESTIONS

Q1. Why do the answers to 190, 191, 338, 371 form a set?

Answer. Each isolates one bit-level identity: shift-and-mask reverses bits, clearing the lowest bit counts them, and XOR-with-carry computes addition. They all exploit that the same operation can be expressed either with arithmetic or with masks, and knowing both forms is what makes the others obvious.

Q2. Why does the loop terminate?

Answer. The carry value `(a & b) « 1` has strictly fewer set bits than `a | b`. Each iteration consumes one level of the bit-carry hierarchy, so after at most the bit-width iterations the carry is 0.

Q3. How does Python handle the negative cases?

Answer. Python integers are infinite-precision two's complement, so `-1 ^ x == ~x` and the arithmetic works uniformly for negatives without any special case. That is why this problem is notably cleaner in Python than in C++.

Appendix Coverage Checklist

Every problem in the local NeetCode repository, in canonical category order. Tick a row only when you can answer all three intuition questions without looking.

#	Category	Count	Coverage
009	**Arrays & Hashing**	9	
014	**Two Pointers**	5	
020	**Sliding Window**	6	
027	**Stack**	7	
034	**Binary Search**	7	
045	**Linked List**	11	
060	**Trees**	15	
063	**Tries**	3	
069	**Heap / Priority Queue**	6	
079	**Backtracking**	10	
092	**Graphs**	13	
098	**Advanced Graphs**	6	
111	**1-D Dynamic Programming**	13	
123	**2-D Dynamic Programming**	12	
131	**Greedy**	8	
137	**Intervals**	6	
146	**Math & Geometry**	9	
153	**Bit Manipulation**	7	